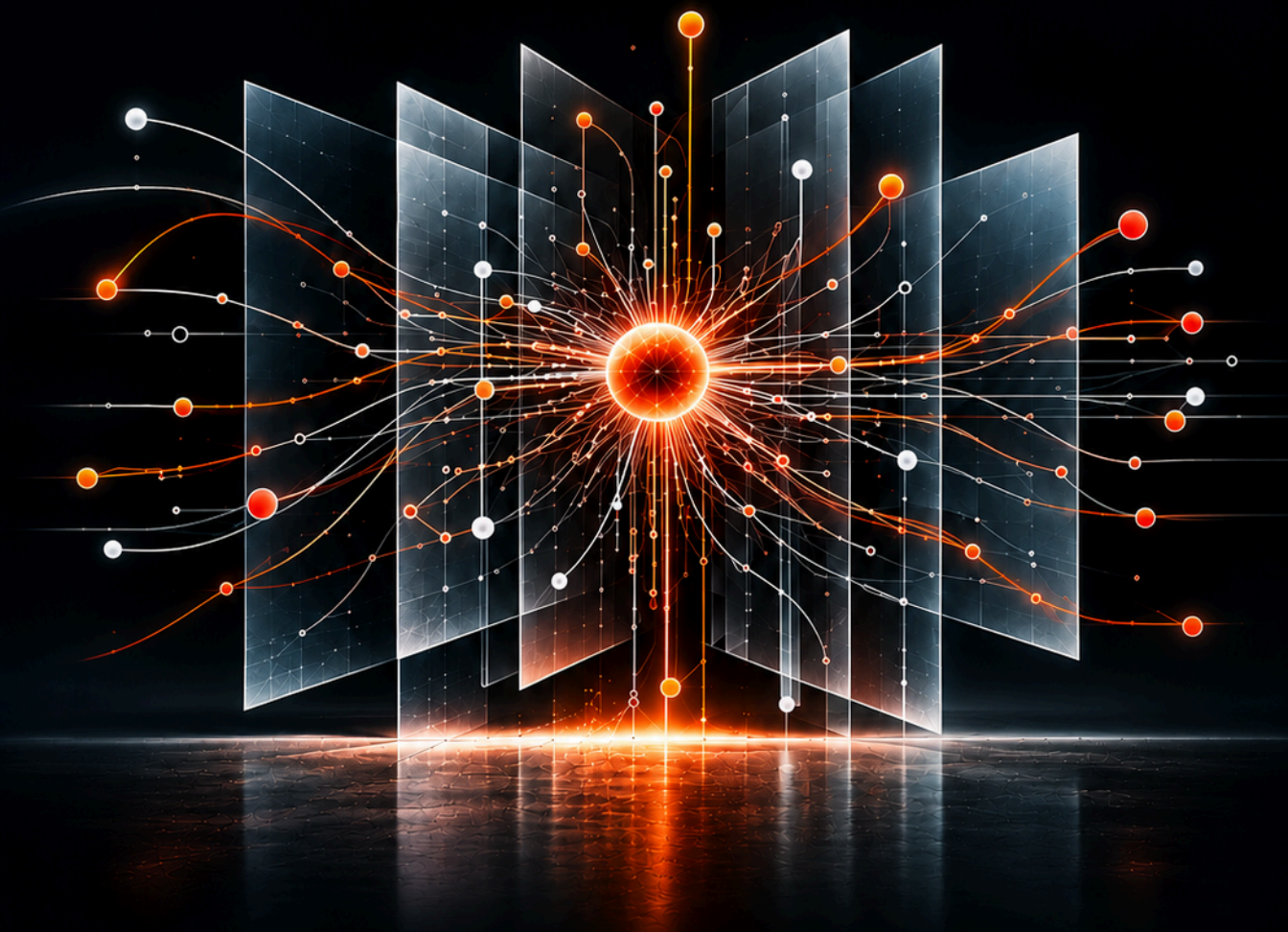
THE SYNAPSE

A PREDICATION-CENTRIC ARCHITECTURE FOR KNOWLEDGE GRAPHS, ONTOLOGY, AND ACCOUNTABLE AI



JAMES LEE STÄKELUM

FOREWARD BY SOWMYA KANDREGULA

The Symbol Grounding Framework Book Series

THE

SYNAPSE

A Predication-Centric Architecture for Knowledge Graphs, Ontology, and Accountable AI

James Lee Stäkelum

THE SYMBOL GROUNDING COMPANY

SYNAPSE
A Predication-Centric Architecture for Knowledge Graphs, Ontology, and Accountable AI

New edition
Copyright © 2026 James Lee Stäkelum

All rights reserved.

Published by
THE SYMBOL GROUNDING COMPANY
MMXXVI

SymbolGroundingCompany.ai

Contents

Foreword v

Preface vii

Acknowledgments xii

Figures and Tables xiii

A Note on the Text xv

The Wound **1**

1 · Why a Knowledge Graph 2

2 · Is SQL Already a Graph? 8

3 · What a Triple Drops 13

4 · The Promotion Chain 18

5 · The Unification Thesis 23

The Atom **29**

6 · Predication Is the Atom 30

7 · The Synapse 36

8 · Roles and Dimensions 42

9 · What a Predication Does 49

10 · Stance and Testimony 55

11 · Discourse Is a Linked Chain 60

12 · Synapse Groups 65

13 · How Groups Compose 71

14 · When Not to Assert 77

The Home **83**

15 · The Universal Meaning Substrate 84

16 · The Prose Compiler 90
17 · The Assertion Adapter 95
18 · Two Intakes, One Card 101
19 · Five Clocks, Correction and Withdrawal 107

The Machine **112**

20 · Compiler Hesitation 113
21 · LLMs Propose; gates Dispose 119
22 · Rehydration as Proof 124
23 · Answering from the Ledger 132

The Address and the Meaning **146**

25 · The Canonical Identifier 147
26 · The Microgloss 155
27 · Public and Private Dictionaries 162
28 · Grounding 169
29 · Entity Resolution and Ghosts 178
30 · One Meaning, Many Tongues 187
30A · The Decompression Table 194
30B · Decomposing Verbs and Aligning Meanings 206
30C · The Limits of Decomposition 214

The Ontology **220**

31 · The Nine Pains 221
32 · What an Ontology Is For 230
33 · The Recovery Method 240
34 · The Standing Ladder 250
35 · Governance and Drift 259
36 · Profiles and Alignment 268
37 · Semantic Debt 277
38 · The Finite Middle 285

The Warrant 293

39 · The Commitment Grid . . . 294

40 · Claim Capability . . . 301

41 · Finding, Decision, Action . . . 309

42 · The Conformance Ladder . . . 317

43 · Standing for Every Force . . . 325

44 · A Language for Permission . . . 333

The Shape 341

45 · Meaning Is Relational . . . 342

46 · The Conservation Law . . . 349

47 · The Chain of Degradation . . . 356

48 · Representation Prices Questions . . . 363

49 · The Grounding Problem . . . 370

The Negative Space 378

50 · A Semantics Is Its Prohibitions . . . 379

51 · Constraints Compose by Intersection . . . 386

52 · Alignment Is Jurisprudence . . . 394

53 · Every Store Has an Epistemology . . . 401

The Landing 408

54 · How a Synapse Lands . . . 409

55 · Landing in a Property Graph . . . 417

56 · Landing in RDF . . . 425

57 · Two Graphs, One Ledger . . . 434

57A · The Semantic Layer's Contract . . . 441

58 · The Minimal Viable System . . . 449

59 · Pattern Interaction . . . 456

60 · Plate: The Belief . . . 463

61 · Plate: The Notice . . . 470

The Model and the Wire 477

62 · What the Model May Do . . . 478

63 · Retrieval, Nearness and Citation . . . 485

64 · The Wire . . . 492

What This Makes Possible 499

65 · The Questions That Become Cheap . . . 500

66 · What This Does Not Unlock . . . 506

Close 512

67 · What This Book Refuses . . . 513

68 · The Compass Again . . . 519

APPENDIX 525

Appendix A · The Thirty Patterns . . . 526

Appendix B · Closed Sets and Grammar . . . 531

Appendix C · The Upper Ontology Landscape . . . 537

Appendix D · Glossary . . . 541

Appendix E · Schema Contract . . . 544

Appendix F · Golden Pairs and Acceptance Tests . . . 552

Appendix G · SPL 0.1 . . . 558

Appendix H · Governance Profile and Omega . . . 564

Appendix I · The Wire Dictionary . . . 569

Appendix J · Specimens . . . 573

Appendix K · Predictions . . . 577

Calhoun: one group, six units, one claim . . . 580

Ten Questions, Three Methods . . . 584

Index . . . 589

Foreword

By Sowmya Kandregula

There are people who think outside the box. And then there is James Lee Stäkelum, who will first ask who designed the box, why everyone agreed that it was a box, what evidence supports the existence of the box, whether its edges have sufficient provenance, and then, somewhere around midnight, propose an entirely different architecture for boxes.

That, in many ways, is the James I know.

I have had the privilege of knowing James not simply through his ideas, but as a person. What has always struck me is the combination of qualities he brings to difficult problems: genuine sincerity, almost infectious enthusiasm, enormous curiosity, and a refusal to accept "that is just how it is done" as a satisfactory answer.

Give James a problem, and he rarely begins by asking how to make the existing solution a little better.

He is more likely to ask whether we have been solving the right problem in the first place.

That instinct is at the heart of The Synapse.

For years, we have become increasingly good at building machines that can store information, retrieve information, connect information, and now generate remarkably fluent answers from information. But beneath all of that progress sits a deceptively simple question: What does the machine actually mean when it says that it knows something?

A line connecting two things in a knowledge graph can look wonderfully authoritative. John works for Acme. Simple.

But James does what James always does. Which John? Which Acme? Who says he works there? When was that true? According to what source? Under what definition? What evidence supports it? And what happens when another source disagrees?

Suddenly, a simple line isn't so simple anymore.

And that is where this book becomes interesting.

The Synapse asks us to reconsider the relationship between information and meaning. It asks whether facts divorced from context, provenance, authority, evidence, and conditions are really knowledge at all or merely convenient representations of something much richer.

This is an ambitious subject. Perhaps an audacious one.

But anyone who knows James will understand why he chose it.

He has always been drawn toward problems that are bigger than one application, one database, one company, or even one generation of technology. His mind naturally gravitates toward the underlying architecture: the common structure, the universal problem, the thing underneath the thing.

And yet what I appreciate most about James is not simply the scale of his thinking.

It is the sincerity behind it.

Technology is full of big claims. Artificial intelligence, in particular, has given us no shortage of predictions, promises, acronyms, revolutions, and occasionally very expensive PowerPoint presentations.

James brings something different.

He brings conviction, curiosity, and a willingness to follow an idea much farther than most people would have the patience or perhaps the courage to follow it. He doesn't merely enjoy imagining what machines might someday do. He cares deeply about whether we can understand why they do it, what their answers mean, where those answers came from, and whether those systems can ultimately be held accountable for the knowledge they represent.

That distinction matters.

As AI becomes more capable, the important question may no longer be simply:

Can a machine give us an answer?

The more important questions may become:

Can it show us what that answer means?

Can it show us why it believes it?

Can we trace it back to its source?

And when the stakes are high, can we trust the structure underneath it?

Those are not small questions.

This is not a small book.

And James has never been particularly interested in small questions anyway.

You may agree with every architectural choice in the pages ahead. You may disagree with some of them. In fact, knowing James, I suspect he would be perfectly happy if this book caused a few spirited arguments.

But I believe that is exactly how important ideas should be encountered: not passively, but thoughtfully; not as conclusions handed down to us, but as invitations to examine assumptions we have stopped noticing.

So as you begin The Synapse, I encourage you to read it in the same spirit in which I have come to know its author: with curiosity, openness, and a willingness to ask one more question after everyone else has decided the problem is solved.

Because every once in a while, progress begins with someone looking at something the rest of us have accepted as obvious and quietly asking: "But is that really what it means?"

James has been asking that question for a long time.

Now he invites the rest of us to ask it with him.

Preface

Five systems hold the same fact. None of them can prove it.

The ontology describes it. The graph connects it. The semantic layer defines it. The glossary names it. The model repeats it. Ask who asserted the fact, on what basis, and what would make it false, and the five answer differently, and none of them answers for the record.

That gap is not a missing column. It is the difference between a record that stores values and a record that stores claims, and it appears wherever those systems were built separately and then asked one question together.

This book is the map for that: one record at the centre, five operations around it, and an order of work that says which step comes first, what each step depends on, and what the whole of it refuses to promise.

You have probably been asked to fix that with a list of words: build the ontology, connect the knowledge graph, add a semantic layer, put governance around it, ground the terms, and make the AI use it. Each of those words is doing different work in different mouths, and the differences are not pedantry. They are where the projects stall.

Take *ontology*. To the data team it means a logical schema: entities, keys, cardinality. To the enterprise architect it means the upper framework the vendor supplied, a hundred and fifty classes and none of them yours. To the taxonomy group it means a tree of preferred terms. To the governance office it means the policy that says who may define a term and who may retire one. Four artifacts, four owners, four lifecycles, one word. A meeting that does not separate them agrees on a document and disagrees on a system.

Here is the gap in one record. A report says the seal may have eroded. The record keeps the hedge, the holder and the span. A year later an auditor asks whether the erosion was confirmed, and the record answers in one step, because the difference between reported and confirmed was never collapsed.

The rest of the stack behaves the same way. A knowledge graph promises relations; what it usually stores is connectivity, without the record of who asserted the connection. A semantic layer promises one definition; what it usually adds is a second description that must be kept in step with the first. Governance promises authority; what it usually produces is a committee. A dictionary promises grounding; what it usually gives is a definition that never reaches the row. Prose extraction promises meaning from documents; what it usually gives is entities with no acts.

Each artifact is defensible alone. Together they do not compose, because each was built to a different unit: the schema to the table, the graph to the node, the semantic

layer to the measure, the dictionary to the term, the extractor to the entity. A system assembled from five units holds five records of the same thing and no way to say which record is in force. The cost is not the storage. It is the answer that cannot be defended two years later, when the person who knew has left.

The objective is not to choose a winner among them. It is to find a unit that each of them can carry: who asserted it, about what, in which roles, on what basis, when, and what would defeat it. That unit is small enough to store in a row and complete enough to survive projection. Once it exists, the artifacts stop competing and start operating. The ontology says which roles and senses are permitted. The graph stores the assertions as a network. The semantic layer becomes a projection that declares what it drops. The dictionary binds a term to a sense. The extractor recovers assertions from prose. The governance language says who may change what. The wire carries assertions between systems.

The bet is one sentence long, and three consequences follow from it.

First, meaning is predication-centric. The smallest thing that can be wrong on its own is the predication, not the entity, the row, the document or the sentence. A predication carries a predicate, its participants, the dimensions that make it a claim, and the span that lets a reviewer check it.

Second, a claim record can be kept as a ledger. Corrections are supersessions rather than overwrites, the clocks stay separate, and every graph, report and model is a projection that must declare what it drops. The ledger is the thing defended. The diagram is a view.

Third, a dictionary can be more than a list of labels. A sense can be a compressed package that decomposes into parts, arrangements, functions and participants and, where the decomposition terminates, into semantic primes. Chapters 30A and 30B treat a word as a compression handle, describe the package and its acyclic decomposition, and introduce SPL, a constrained language for writing the packages. The proposed floor is the set of NSM primes; the book treats that as a hypothesis, not a foundation, and marks where a branch stops and why termination is not correctness.

The third argument is the most provisional, and the book says so. The decomposition is useful only if it changes a decision on a costly error type; otherwise it is an elegant elaboration of known ideas. The evaluation protocol is stated in the appendices, and the book does not claim its result in advance.

The flavour we chose, and why

Knowledge graphs come in several flavours, and the choice between them is a decision about what the system must be able to answer, not a matter of taste.

The **entity-centric** graph puts things at the nodes and relations on the edges. It is the default, it is fast, and it cannot say who asserted a relation or how confident anyone was. The **event-centric** graph makes actions into nodes, which handles many participants

and time. The **fact-centric** graph makes each assertion a node, which buys provenance. The **concept-centric** graph decouples terms from instances, which survives schema drift. The **document-centric** graph hangs entities off the passages that mention them, which is how retrieval stays traceable. The **role-centric** graph separates identity from the roles an entity plays. **Subgraphs** partition the whole by owner or tenancy. **Situation frames** bind entities, conditions and outcomes for policy evaluation.

None of those is a mistake, and the architecture uses all eight as projections: each is something a consumer can be given, and each is a view of something else that must be kept.

What is new here is not the topology. Reification is old, RDF-star made it cheaper, event sourcing made it familiar, and provenance vocabularies have carried attribution for years. A claim that ignored that lineage would deserve the scepticism it got.

What is new is the set of obligations placed on the claim, and the discipline of what happens downstream. The claim carries a holder, a span, a stance and a defeat condition. Corrections are supersessions rather than overwrites. Every projection declares what it dropped. A policy may require a confirmed claim and refuse a reported one.

Those obligations cost something, and the cost belongs in the same paragraph as the benefit. A claim ledger is slower than an edge store, heavier to author, and awkward for the tools built around nodes and edges. It needs an owner for the lexicon and a ruling for each conflict. It pays off on the day someone asks who said so, and that day is rarely the day the project starts.

So the choice is a reasoned one: use the claim as the unit where an answer must survive a challenge, and project the other flavours from it where speed, familiarity or traversal matter more. An entity-centric graph is not wrong. It is a view that has not been asked to justify itself yet.

The order of the work

The roadmap has a sequence, and the sequence is the argument. A claim needs a home before it can be compiled. It needs a compiler before it can be addressed. It needs an address before it can be governed. It needs governance before it can be licensed. It needs a license before it can be projected. It needs a projection before it can land. It needs to land before a model or an agent may act on it.

Each part of the book is one step in that order, and each step states what the previous step left unresolved.

Part I shows why a container holds rows and not meaning. Part II builds the unit: a verb at the hub, fifteen closed roles, eight dimensions, a span beneath. Part III gives the unit a home and two intakes, prose and tables. Part IV compiles sentences into records and proves the compile by rehydrating them. Part V gives the records names: identifiers, microglosses, public and private dictionaries, and the decomposition of a sense. Part VI governs the distinctions. Part VII states who may act on a claim. Part VIII asks what

meaning is once the record is relational. Part IX states the negative space, because a semantics is its prohibitions. Part X lands the record in real stores. Part XI says what a model may do and what the wire carries. Part XII says what became cheap and what did not.

Three of those steps are usually skipped, and they are the three that decide whether a system survives a regulator, an auditor or a court: the negative space, the warrant, and the declared loss. A record that cannot refuse, cannot license and cannot say what it dropped answers fluently and cannot be defended.

The decomposition chapters sit at the end of Part V, after the address and before the ontology. The placement is deliberate: a sense needs an address before it can be decomposed, and the ontology needs decompositions before it can govern them.

The book draws on four bodies of work seldom put in one room. Thematic roles from linguistics, in the line that runs from Fillmore and Jackendoff to the Davidsonian account of events, give the role set its shape. The Natural Semantic Metalanguage gives grounding a floor of primes rather than an infinite regress of definitions. The knowledge-graph pattern literature gives the recurring shapes a record takes when it is stored. The governance languages of permissions and obligations give the record its authority. None of those traditions was built to hold a claim with a holder and a stance, and each contributes something the record needs.

What the book does not claim

It does not claim a new theory of meaning. The ingredients are old: reductive paraphrase, thematic roles, structured meaning representations, sense links, ontology matching, necessary and sufficient conditions. The contribution, if there is one, is the closed loop: constrained packages, schema validation, force-aware comparison, reusable molecules, and separated lexical, definitional, ontological and assertion layers.

It does not claim that a graph replaces a database, or that a semantic layer removes the need for one. It does not claim that an ontology is complete, or that a compiler is cheap. It does not claim that the architecture has been measured at the scale your organization runs at; several of its rules are specified and unrun, and the chapters say which. It does not claim that a model can be trusted because a record exists. It claims that a record can be built which makes a model's answer checkable, and that the check is the part a business can defend.

It does not claim to be a step toward general intelligence. A meaning layer addresses conventional lexical meaning; it does not learn from experience, perceive, plan or revise its own concepts.

Who it is for

The architect who has to make five systems agree. The ontology engineer who has been asked for a graph and handed a spreadsheet. The data lead who owns a glossary nobody reads. The AI platform team that has been told to ground a model and has no idea what grounding would mean in their stores. The program manager who needs to know which step comes first and what it costs to skip one.

The book assumes a reader who knows what a table is and what a relation is, and who has sat in the meeting where one term was defined three ways. It does not assume a background in description logic and does not require one.

How to read it

The argument runs from Part I to Part V. A reader who wants the whole of it reads in order; each part closes on the question the next part answers.

The executive reads Part I, the close of Part IV, and Part XII: what the architecture buys, what it costs, and what it refuses to claim.

The builder reads the part that matches the work in front of them and then the appendix that specifies it. Appendix A carries the thirty patterns, Appendix E the schema contract, Appendix F the acceptance tests, Appendix G the language, and Appendix H the governance profile.

The wager

The wager is that meaning has a unit, that the unit is the predication, and that a record built from predications can be found, combined, governed, projected and defended without a meeting to settle what it said. If that is right, the stack stops being five projects and becomes one record with five operations. If it is wrong, the chapters that would show it wrong say so, and the record is still more defensible than a graph of nodes with no memory of who asserted them.

Five systems, one fact. The record can prove it.

Acknowledgments

A book about attribution owes its own. The argument here was assembled from other people's work, and the assembly is the author's responsibility rather than theirs.

The philosophical debt is to the event-based account of action that made the predication the unit of meaning, and to the linguists who showed that a verb's sense determines the participants it permits rather than the reverse. The book's fifteen roles are a working set built on that insight, and the set's errors are the author's.

The standards debt is to the upper-ontology community. BFO and CCO gave the book a vocabulary for arguing about categories, and Appendix C records where the book's grammar does not fit them. A refusal to align completely is a debt rather than a dismissal.

The engineering debt is to the practitioners who read drafts and asked the questions the book had not answered: what happens when two stores disagree, who may revoke a permission, and what a citation is for. Those questions shaped the parts on the ledger, the wire and the license.

The book's readers are its test. The preface states a standard the book applies to itself, and the standard is only meaningful if a reader can check it. Readers who find a claim they cannot check have found the book's next edition.

No institution funded this work, and no product is recommended in it. The tools named in the text are named because a claim has to land somewhere, and the choice of where is the reader's.

The dedication stands apart from these pages. It is owed to a person rather than to a literature, and the book's argument about free will and forgiveness is not an argument the record can settle.

Errors that remain are the author's, and a correction sent to the publisher will be recorded in the next version rather than defended in this one.

Figures and Tables

The book carries fourteen visuals, and each one is there because a relation or a comparison could not be held in prose. Eight are tables, set where the comparison is made; six are diagrams, drawn where the shape is not a table's shape. Nothing is illustrated twice, and no chapter carries a picture of its own sentence.

Ch.	Visual	What it carries
1	table	a graph, a row and join, and a claim record across six attributes
7	diagram	the synapse: predicate at the hub, participants on spokes, dimensions on the rim, span beneath
12	diagram	a group's five role-bound members, a nested group inheriting scope, and one synapse in two groups
13	table	the joins cross-classified by property, direction, removal, defeat and challenge, with the lifecycle non-join
22	diagram	the round trip: A to S to A-prime to S-prime, with the comparison edge and the refused diagonal
30B	diagram	the promise in order: speech before action, recognition after the speech, reliance on the action's failure, and why a prediction lacks the fourth
31	table	the nine pains by family against two programs, with family sums
42	table	the evidence each rung of the conformance ladder requires
52	table	five docket cases across rule, ruling, reason, remedy and standing
54	diagram	the three ordered tests that decide whether a claim lands as a node or stays an edge, and the residue that may be an edge
58	table	six collapses against tier, pattern, dependency and the maintenance scope
61	diagram	two chains meeting at the finding: the notice's four claims, the evidence's three records, and MISFILED citing a span from each

Continued on next page

(Continued)

Ch.	Visual	What it carries
64	table	the envelope states, who may write each, and what each becomes
68	table	the twelve parts and the Close against the question each inherits and the bearing each serves

A table is the preferred visual here: it is searchable, editable and typeset without a drawing step. A diagram appears only where position carries meaning that a row of cells cannot—a wheel, a nesting, a loop.

A Note on the Text

The companies in the worked examples are fictitious. Calhoun Industrial Group and its six units were invented for this book, and any resemblance to a real firm or project is unintended.

Three conventions govern the pages that follow, and a reader who knows them can check a claim rather than trust a tone.

Claims are written in one notation throughout: a verb, its roles in parentheses, its dimensions in brackets and its record fields in braces.

```
FAIL(theme=pump; time=4 March)[force=assertion; polarity=positive]{span; stance;
↪  version}
```

The notation is a reading aid rather than a serialization. It appears where a claim's shape matters and disappears where prose is clearer, and the two forms describe the same object.

Citations name a path and a heading rather than a line number, because a line number changes when a file is edited and a heading does not. A citation that cannot be followed is a citation that has not been made.

Figures are specified per chapter and drawn in TikZ; the list at the front records what each chapter's figures serve. A figure that cannot be drawn from its chapter's prose is a figure that has not been specified.

Quotations keep their original spelling, including where the book's own spelling differs. The book is otherwise set in American spelling, and the difference is deliberate rather than an oversight.

The appendices are reference material rather than argument. They carry the closed sets, the schema contract, the language, the governance profile, the wire dictionary, the specimens and the predictions, and each is written to be used rather than read through.

Directions are given as bearings rather than as a summary: claim, warrant, address and license. The last part of the book explains why the four are ordered as they are, and a reader in a hurry can begin there.

PART I
The Wound

why the container does not hold meaning.

1 · Why a Knowledge Graph

A knowledge graph represents relations as objects that can be stored and traversed. A relational database also represents relations, as foreign keys, so the difference is not whether relations are stored. The difference is whether a path is represented or reconstructed at query time.

Calhoun Industrial Group owns six business units and eight systems. Boreworks files an inspection note saying the seal on pump PB-100-7 may have eroded; HighwayWorks holds the pay item that will replace it; HaulWorks holds the load ticket for the spoil; the group office holds the monthly report that mentions none of them. A path question across those systems is a graph problem. The question that matters is not which records are connected, but who said the pump was failing, on what basis, and whether anyone confirmed it. The graph answers the first. The argument that follows is about the second.

A path is a sequence of links from a starting point to an end point. A path question asks which end points are reachable from a start through links of different types, to an unknown depth. A join returns rows that satisfy a condition. A traversal returns a closure: the set of nodes reachable through the stored links, together with the routes that reached them.

Here's an example: a recall asks which orders are reachable from a batch through a bill of materials. A fraud review asks which accounts are reachable from a device. A supplier review asks which products are reachable from a subcontractor. Each question starts at one node and follows links until the set is closed.

Relations are stored in both models

A relational schema stores a relation as a foreign key. The key constrains the value: the referenced row must exist. It does not record that the relation is a path the business follows, or the conditions under which the path holds.

A graph stores the relation as a link. The link is typed, carries its own properties, and can be traversed without a join. The difference is where the reconstruction happens: in the query, every time, or in the system, once.

A graph's link also carries direction and type. Direction matters for reachability: an approval points from the order to the approver, not the reverse, and a traversal that ignores direction returns a different set. Type matters for filtering: a supplier link and a payment link are not interchangeable, even when they connect the same nodes.

Here's an example: a `customer_id` column may reference a customer table in the same schema, a table in another service, or no table at all. A `batch_id` may reference manufacturing, and a `substitute_id` may exist only in a change note. In four production schemas examined for this project, 209 of 996 columns are identifier columns with no declared foreign key.[1] The relation is carried by convention, and each query that follows it reconstructs the convention.

Why the join is expensive

A path question can be written in SQL. The query requires a recursive common table expression, a union of link types, a depth bound, and a filter for links that are not in the schema. An expert can write it. The cost is readability and scope: a reviewer cannot easily verify the query, and a change to the depth bound changes the answer.

The depth bound is one source of cost. A bound of three reaches suppliers; a bound of four reaches suppliers of suppliers. The query text is the same; the scope is not, and the parameter is rarely reviewed with the care given to code.

The bound is a policy decision expressed as a query parameter. A team that sets it to three has decided that suppliers of suppliers are out of scope, and that decision is rarely written down or reviewed.

The second source is identity. Each hop may use a different key: the customer in commerce, the batch in manufacturing, the substitute in procurement, the shipment in logistics. The query reconciles those keys before it can walk, and the reconciliation is repeated for every question.

Identity is also difficult within one system. In one production dictionary, 22,129 duplicate identifier groups covered 45,742 rows. 97 percent were spelling variants of one entity, and 3 percent were different entities sharing a name.[2] A join that treats the key as the entity will be wrong in both directions.

A graph does not remove identity resolution. It moves the work to load time: the links are written once, with whatever resolution was performed, and the traversal follows them. The identity decisions remain; they are made earlier.

What traversal adds

A traversal returns the closure and the routes. The routes are the useful part. They show which link connected each member to the start, and they make the set reviewable. A list of results is a scope; a list with the path that reached each result is an account of how the scope was determined.

[1] 209 of 996 columns across four production schemas; 39 percent of the identifier columns. Measurement and parser: `measurements/M1_pilot.csv`, `book/M1_PILOT.md`.

[2] 22,129 duplicate canonical-ID groups across 45,742 rows: 21,647 (97.0%) orthographic variants, 482 (3.0%) different entities. Source: `known_gotchas.md` §2.1.

A search engine returns documents that mention the starting point. A graph returns the nodes reachable from it. The difference is closure: the set is defined by paths, not by similarity, and it is complete with respect to the links that were loaded.

Closure is also a completeness claim. The traversal returns every node reachable through the loaded links, so a missing result means a missing link, not a missing row. That property makes the graph useful for recall and dangerous for audit: the set is complete only with respect to what was loaded.

Here's an example: after a recall, follow-up questions arrive in the same review—which other batches used the substitute, which customers received the affected products through a distributor, which assemblies are one hop from the failed batch. Each question is another traversal from the same start, and each returns a set with routes.

The consequences are concrete. A recall scope that omits one product is a regulatory finding and a second recall. A supplier review that misses a subcontractor is an unplanned disruption. A fraud review that cannot reach a beneficiary leaves the network intact. The value of the graph is measured in the questions it makes answerable within the deadline.

What traversal does not add

A traversal returns the exposed orders. It does not return the claim that a link was authorized.

A substitution link exists because a loader wrote it. The graph does not record who approved the substitution, on what evidence, under which engineering change order, or when the approval took effect. It can store a timestamp and a user name as link properties, but a property is an annotation on the link, not a claim with a holder and a defeat condition.

Here's an example: an audit table records that a link was written by a service account at a time. It does not record the decision. The two records have timestamps and actors, and they answer different questions. Reconstructing the authorization becomes a search through email and a change-management system that used a different identifier for the same supplier.

A claim record carries the authorization in one row: holder, content, span, authority, valid time, defeat. The record need not be large. Six fields per claim are sufficient, provided each field is populated from a source that can be inspected. The difficulty is not storage; it is capturing the fields when the decision is made.

The graph is the correct representation for reachability and the incorrect representation for the claim that a link is true. An architecture needs both. Most architectures build the first and assume it is the second.

When a graph is not justified

If the question is two hops—which customer owns an order, which grant covers a person—a join is shorter, faster and easier to explain. If the corpus is small and the questions are fixed, a relational schema is a better investment. If no record exists and none is planned, a graph makes the existing ambiguity traversable, which is worse than leaving it in tables where the keys are visible.

A graph is justified when the questions are deep, the links are heterogeneous, and the route is part of the answer. Under those conditions, traversal is not a convenience; it is the readable form of the question.

There is a cost. A projection must be maintained. Link properties are copies of record fields, and copies drift. The graph is fast because the walk is precomputed; it is expensive because the walk must be rebuilt when the record changes, and the rebuild is another system with its own failures.

A graph also requires ownership of its vocabulary. If a team adds a `SUPPLIES` link that means three different things, the traversal returns routes that cannot be interpreted. The vocabulary requires the same governance as any shared model, and more, because the paths are the product.

The decision

The graph represents reachability. The record represents the claims that make a link authorized.

That division is the design decision. The graph is a projection of the record, shaped for traversal. The record is a set of claims, each with a holder, a content, a span, an authority, a valid time and a defeat condition.

The three artifacts differ on six attributes, and the differences are the argument.

Attribute	Graph (path stored)	Relational row and join (path reconstructed)	Claim record
what a question returns	the closure and the routes that reached it	rows that satisfy a condition	holder, content, span, authority, valid time, defeat
where identity is resolved	at load: the links are written once	in the query, every time	each field populated from a source that can be inspected
direction and type	typed, directed, carries its own properties	the key constrains the value	the claim's roles carry the relation

Continued on next page

(Continued)

Attribute	Graph (path stored)	Relational row and join (path reconstructed)	Claim record
authorization	a property on the link, not a claim	the audit table records that a link was written, not the decision	who authorized it, on what evidence, as of when, what would retract it
completeness	complete only with respect to what was loaded	the rows that are present	the claim that would retract it
what it cannot answer	who authorized the link	the direction and the path	the reachable set

The row that decides the architecture is the fourth. A graph can store a name and a timestamp on the link, and neither is a claim: the graph records that the link exists, and the record records that it was allowed to.

An architecture that builds only the graph produces faster answers with weaker defenses, while one that builds only the record produces defensible answers with slower walks. Most organizations choose the first because the second is harder to demonstrate within a quarter.

Here's an example: for a substitution claim, the record has six fields: the holder who authorized it, the content that names the substitute, the change order that carries the authorization, the evidence that the substitute met the specification, the date the authorization took effect, and the claim that would retract it. The graph carries the link; the record carries the six fields that make the link authorized.

The projection rule is the same for any view: declare what the view drops. The graph drops the holder, the span and the defeat condition; it keeps the path, the link type and the properties that were loaded. A projection that declares its loss can be trusted for the questions it was designed for.

The projection also has a version. A graph built from last quarter's record and a graph built from this quarter's are different artifacts, even when the schema is the same, and the difference is invisible unless the build is dated.

The test

Take a table with a status column. Ask who set the last value, on what evidence, and under what authority. The table cannot answer, because the record problem is not a table problem.

Take a path question the business asks—which customers are exposed through a supplier, which accounts are connected through a device, which services fail if a permit is withdrawn. Determine whether the system stores the path or reconstructs it on every query. If it reconstructs, a graph has a case.

Write two lists. The first is the paths the business needs to walk, with the cost of not answering them. The second is what the graph cannot say about those paths: who authorized the link, on what evidence, as of when, and what would retract it. The first list justifies the graph; the second defines the record.

Neither list is stable. Paths change when a supplier is added; warrants change when an authority is revoked. Both must be versioned, or the answer will be correct when it is produced and wrong when it is reviewed.

A system that can produce the route but not the authorization has increased the speed of an answer it cannot defend.

2 · Is SQL Already a Graph?

A relational join preserves keys and cardinality. It does not preserve the claim that a row encodes. The distinction matters when a row is used to answer who asserted something, on what evidence, under what authority, and as of when.

A claim is an assertion with a holder, a time, a basis and a defeat condition. A row is a set of values keyed by an identifier. A row can encode a claim without storing one.

What a join preserves

A join connects rows by a key. It preserves the key, the cardinality of the relationship, and referential integrity where the schema declares it.

Referential integrity is weaker than it appears. A foreign key guarantees that a referenced row exists when the constraint is checked; it does not guarantee that the referenced row is the entity the business means, or that the relationship holds under the conditions the business cares about. The key is a constraint on values, not a statement about the world.

Here's an example: an `orders` table with `customer_id` joined to `customers` answers which customer owns an order. The join is correct and fast.

A join also preserves the shape of a condition. Filters, aggregates and windows compose with it. For a question such as how many orders shipped this week, the relational query is the right instrument.

An outer join adds a third property: it preserves the row when the relation is missing and fills the missing side with null. The null is a hole with no type. Nothing in the result says whether the customer was deleted, never existed, or was merged into another record, and the consumer decides what the null means.

The join does not preserve the act that produced the row. It preserves the row's values and their relationships.

The distinction is not between SQL and graphs. A graph can store a row-like node without its claim, and a relational table can store a claim record if it has the fields. The question is what the structure represents, not which product is used.

What a row encodes

A status column records the outcome of an act. The value `approved` records the result of an approval; the column name records the category; the table records the subject. The actor, the evidence, the authority and the time are not on the row.

Here's an example: an order row with `status = approved` and `updated_at = 2026-04-11 09:14`. The row says the order is approved and that the row was last written at 09:14. It does not say who approved it, on what basis, or whether the approval was valid at that time.

The row encodes a predication without storing one. It represents the state that resulted from an act and omits the act's structure.

This is not a schema defect. A status column is an efficient representation for the question what state is this order in, which is the question most consumers ask.

Six questions

When the approval is challenged, six questions matter, and the joined row answers none.

Question	Row
Who approved it?	Not represented; the write path is recorded, not the decision.
In what words was it approved?	Not represented; the status value is a summary.
With what modality?	Not represented; approval and *should be approved* produce the same value.
Valid as of when?	updated_at is a write clock, not a valid time.
Under what standing?	Not represented; a manager, an auditor and a service account can all write the value.
With what force?	Not represented; an assertion, a directive and a commitment can all produce the same status.

The table is not an argument against SQL. It lists what a claim carries that a value does not.

When the row is sufficient

The row is correct for a large class of questions. If the question is whether an order is approved, which customer owns it, or how many orders shipped this week, the row answers directly and a claim record adds cost without adding an answer.

A question about state is answered by the state. A question about an assertion is answered by the assertion, and a row is not an assertion even when an assertion produced it.

A team that builds a claim record for every state question will maintain a second system that no consumer reads. The record earns its cost when the consequence of a wrong answer is high enough that the warrant must be inspectable.

Why the loss is invisible

The loss is invisible because the query succeeds. The support agent receives the customer name, the dashboard receives the count, and the compliance report receives the status distribution. Each consumer receives a correct answer to the question it asked, and the correctness makes the absence of the claim invisible until a question is asked that the consumer's interface does not support.

Here's an example: an audit table records that the row changed from pending to approved at 09:14, by `svc_approvals`, from `order-api`. The record identifies the write, the service account and the time. It does not identify the decision, the evidence or the authority.

The two records look similar because both contain a timestamp and an actor. They answer different questions.

`updated_at` records the last write, which may be a correction, a migration, a backfill or a no-op update. It is not the time the approval became true, and a query that treats it as a valid time will be wrong in a way that appears precise.

The aggregate is the next place the loss hides. A report that counts approved orders does not need the approver, so the report is correct and the claim is absent. The count becomes the fact that circulates, and the claim is not missed until someone asks who approved the orders in the count.

Adding columns

The first repair is to add the missing fields. The repair is partial.

It works for fields that are properties of the order: an approver identifier, a timestamp, a basis string. It fails for fields that are claims in their own right. A basis is a document with a span; an authority is a grant with a window; a modality is a dimension of the act. Stored as columns, they become values without structure.

Here's an example: six columns added to `orders` produce a wider row. The row contains an approver, a date and a basis string. It still cannot be challenged at the level of the decision, because the columns are values, not claims, and the relationships among them are not represented.

The maintenance cost is concrete. Each column is a schema change, a migration, a consumer update and a backfill.

The columns also change every consumer. A report that selects every column now carries fields it does not understand; a service that inserts a row must supply values it cannot derive. The schema grows, and the semantics do not.

Event sourcing is the closer alternative. An event log records changes as events and allows the current state to be reconstructed. It still records changes rather than claims: an event carries a payload, not a defeat condition, and replay reconstructs the row, not the warrant. The log also grows, and replay becomes expensive.

The claim record

The record that closes the gap has six fields per claim: holder, content, evidence span, authority, valid time, defeat. The fields are the minimum needed to answer the six questions.

Here's an example: for the approval of order 44021, the record identifies A. Rao as the holder, the approval as the content, the review note as the span, grant 12 as the authority, 11 April 2026 as the valid time, and a withdrawal claim as the defeat condition.

The record does not replace the row. It sits beside the row, and the row becomes a projection of the record's state. A consumer that needs the state reads the row; an auditor who needs the warrant reads the record.

Projection target

SQL has two roles. It is a source of state, where the row is the record, and a target for claims, where the row is a projection. The same database can play both roles for different questions; the question determines which role applies.

The projection may not become the only copy. When the warehouse is the only copy, the losses are permanent: the approval exists as a status value, the evidence as a free-text field, the authority in a table that may have been archived.

A projection should declare what it drops. The status view drops the holder, the span and the defeat condition; it keeps the state, the row identity and the timestamp.

A projection should also carry a date. A status row built from last quarter's record and one built from this quarter's are different artifacts, even when the schema is the same, and the difference is invisible unless the build is dated.

The projection contract

A projection contract states four things: the source record, the fields the projection keeps, the fields it drops, and the questions it is intended to answer.

Here's an example: a status view for a support dashboard keeps the order identifier, the status and the last write time. It drops the holder, the span and the defeat condition; it is intended for questions about current state, not authorization.

A contract does not make the projection correct. It makes the projection's limits inspectable.

The test

Take a table with a status column. Ask the six questions: who, in what words, with what modality, valid as of when, under what standing, with what force. If the row answers fewer than six, the table is a projection of a claim it does not contain.

Take the audit table. Ask whether it records the decision or the write. If it records the write, it is a history of the row, not a record of the claim.

Decide where the claim will be stored. The decision does not require a graph. It requires a record with a holder, a span, an authority, a valid time and a defeat condition, and a projection that puts the state back into the row.

Record the decision as well as the columns. A decision that is not recorded when it is made becomes a reconstruction later.

Write two lists. The first is the questions the row answers correctly, with the consumer for each. The second is the questions the row cannot answer, with the decision that depends on them. The first list is the warehouse's job; the second is the record's.

A join can be correct and the claim can be absent. The two facts are compatible, and the second matters when the row is challenged.

3 · What a Triple Drops

A triple records a relation among three terms: subject, predicate, object. It does not record the assertion that the relation was made: who holds it, in what words, with what force, valid as of when, under what standing, and what would defeat it.

The subject is what the statement is about, the predicate is the relation, and the object is what the relation reaches; the triple `(Cole, left, warehouse)` records that Cole left the warehouse. It does not record who said so, in what words, with what force, as of when, under what standing, or what would retract it.

A claim is an assertion with a holder, a time, a basis and a defeat condition. A triple is a relation with three terms. A triple can express the content of a claim and cannot express the claim, and the difference is the loss that follows.

Three terms

The triple is a compact notation, and its compactness is the reason it is used. Three terms fit in a sentence, a line of Turtle, or a row of a triple store. The notation composes: triples can be joined by shared terms, queried by pattern, and exchanged between systems that agree on the identifiers.

Here's an example: the sentence *Carter testified that she saw Cole leave the warehouse at 10:15* yields the triple `(Cole, left, warehouse)`. The triple is accurate. It is also silent about the testimony.

The omission is grammatical. The notation has three terms and does not represent the assertion, so the properties that make the relation a claim are absent.

A triple is also not a sentence. A sentence has a verb, a tense, a mood and a speaker; the triple records the verb's sense and omits the rest. The notation selects what it represents, and the selection is judged by the question at hand, which is why the same triple can be adequate for a lookup and inadequate for an audit.

The three-term form also constrains the content. A triple represents a binary relation; a ternary relation, such as one person giving an object to another, becomes a reified event with roles. The notation can represent the event, and it does so by turning the event into a resource, which is the same move as reification. The notation is not neutral about arity.

A triple's terms are identifiers. Two triples that use different identifiers for the same person are two disconnected statements; two triples that use the same identifier for two

people assert a false identity. The notation does not resolve either case. Identity is a separate decision, with its own evidence and its own failure modes.

The relation is not the assertion

A saying has a holder, an occasion and a basis. Carter held the observation; the detective recorded the testimony; the report carried it forward. Each of those is a relation of its own, with an author and a time.

The triple can represent the relations if they are reified. It cannot represent them as the meaning of the original statement, because the original statement's meaning includes the fact that it was said by someone, to someone, on some basis.

Here's an example: `(Carter, testified, statement1)` and `(statement1, asserts, (Cole, left, warehouse))` can be written as triples. The graph now has the holder and the content. It still has no defeat condition: nothing in the notation says what evidence would withdraw the testimony, or which later statement would supersede it.

The triple notation can carry any relation. The question is whether the relation it carries is the claim or a trace of the claim.

The distinction is between a mention and a use. The term `Cole` in the triple names a person; the assertion is about Carter's observation of that person. A store that keeps only the term keeps the mention and loses the use. The loss is invisible until the question is about the observer rather than the observed.

The loss increases when the testimony is nested. Carter's statement is reported by the detective, and the report is the document the store ingests. The triple `(Cole, left, warehouse)` may come from the report, from Carter's statement, or from a later summary, and the store cannot say which. The nesting is the meaning, not an ornament.

Six things the triple cannot carry

The losses are easier to see as a list. For the Carter statement, the triple answers the content question and none of the others.

Property	Triple	Why
Holder	Not represented	The asserter is not a term in the relation unless reified.
Wording	Not represented	The sentence's modifiers, hedges and order are lost.
Force	Not represented	Assertion, testimony, rumor and hypothesis produce the same triple.
Valid time	Optional predicate	A time can be attached; whether it is valid time or record time is a convention.

Continued on next page

(Continued)

Property	Triple	Why
Standing	Not represented	A witness, an official and an anonymous source are indistinguishable.
Defeat	Not represented	Nothing in the triple says what would retract it.

The first three losses are grammatical: the notation does not represent them. The last three are semantic: they can be approximated by more triples, and the approximation is a record built beside the triple rather than a property of it.

Valid time and record time are the most easily conflated. A triple can carry a date, and the date may be the time of the event, the time of the assertion, or the time the triple was written. Without a convention, the same date means three things; with a convention, the other two meanings are lost.

Standing is the least represented and the most consequential. A witness, an official finding, a rumor and a machine output can produce the same triple. The consumer cannot tell whether the statement is evidence, a conclusion or a repetition, and the distinction is what an audit asks for.

Reification moves the problem

Reification is the standard response. A statement becomes a resource with `subject`, `predicate` and `object` properties, and the statement resource can carry metadata.

Here's an example: `stmt1` has `subject Cole`, `predicate left`, `object warehouse`, `assertedBy Carter`, `recordedAt 10:15`. The reified statement now carries the holder and the time.

The reification does not carry the warrant. `assertedBy` is another triple, and the question *on what basis* moves to that triple. The same is true of `recordedAt`: it records when the statement was written, not when the event occurred or when the belief was held. The defeat condition is not represented at all, because it is a relation between a claim and a future state of evidence.

Named graphs and RDF-star reduce the cost of annotation. A named graph can label a set of triples with a context; RDF-star can attach a property to a triple. Both keep the annotation beside the relation, and neither makes the annotation a claim with a defeat condition. The regress is cheaper, not closed.

Reification also changes the unit of query. A query for `(Cole, left, warehouse)` becomes a query for a statement node and its properties, and every consumer must know which convention the store uses. The cost is not only storage; it is the loss of a single shape for the relation.

Reification also multiplies the record. Every statement becomes a node, every property of the statement becomes a triple, and the properties of those triples become further

nodes if they are to be attributed. The notation stays consistent and the warrant is not represented.

What triples are good for

The triple is the right notation for the relation layer. It is compact, composable, standardized and queryable. It is the basis of RDF, of most graph interchange, and of a large body of tooling. A book that dismissed it would be dismissing the most widely implemented model of relations.

The triple also supports inference. If `(Cole, left, warehouse)` and `(warehouse, partOf, site7)`, a rule can derive `(Cole, left, site7)`. The derivation is useful and it is not a claim about the world until someone asserts the rule and the premises. The notation supports the mechanism; the warrant remains outside.

The triple's greatest strength is exchange. Two systems that share identifiers can exchange statements without a schema negotiation. That strength is real and it is why the notation persists. It is also why the notation is dangerous: exchange succeeds even when the two systems mean different things by the same term.

The notation is also stable. RDF has survived changes in tooling, vendors and query languages because its unit is small and its semantics are minimal. A record format that changes every five years cannot make that claim. The stability is a reason to project into triples, not a reason to store only triples, because a projection can be rebuilt when the notation changes while a record cannot be recovered from a projection that dropped it.

The triple is also the right projection for a claim whose content is a simple relation. If the question is *which warehouse did Cole leave*, the triple answers it. If the question is *who says so*, the triple is the wrong instrument and the record is the right one.

The distinction is the same one that governs the graph in the first chapter: the notation is a projection, and a projection is judged by the questions it was built to answer.

The record a triple projects

A claim record for the Carter statement has seven fields: the holder, the content, the span, the force, the valid time, the standing and the defeat condition. The span and the content are separate, because the words and the assertion they carry are different objects.

Here's an example: the record identifies Carter as the holder, the sentence as the span, the testimony as the force, 10:15 as the valid time, witness as the standing, and a later recantation as the defeat condition. The triple `(Cole, left, warehouse)` is one projection of that record. It is the projection that answers the content question.

The record does not require a graph. It requires a table with those fields and a projection job that writes the triple. The graph is a consumer of the record, not its replacement.

The record projects into more than one triple. The content projects to `(Cole, left, warehouse)`; the holder projects to `(stmt1, assertedBy, Carter)`; the span projects to `(stmt1, span, doc12:14-16)`. Each projection answers one question, and none answers the defeat condition. A store that keeps the projections and not the record has the pieces of the claim without the claim, which is why the same content can be reconstructed five different ways in five different systems and none of them can say which reconstruction is authoritative.

What the loss costs

The cost appears when a statement is challenged. A regulator asks who observed the event, and the store returns a term; a court asks what words were used, and the store returns a normalized relation. An insurer asks whether the source was official, and the store returns a node with no standing. Each question sends the team back to the document, and the document is the record the store did not keep.

The cost is not paid at ingestion. It is paid at audit, in the time between the question and the document, and it is paid again each time the same question is asked.

The test

Take one triple from a store. Ask six questions of it: who says, in what words, with what force, valid as of when, under what standing, and what would retract it.

If the store answers fewer than six, it is a projection of a claim it does not contain. Reify the triple and ask again. The count will rise for some questions and stay at zero for the defeat condition, because defeat is not a property of a statement node.

Count the questions the store answers before and after reification. The count measures the projection, not the claim: a store can answer five of six and still fail the sixth, and the sixth is the one an auditor asks.

Then decide where the claim will be stored. If the answer is *in the graph, as properties*, the record is being rebuilt inside the projection, and the next projection will rebuild it differently. If the answer is *in a record beside the graph*, the triple can stay a projection.

A triple can be accurate and still be unattributable. The record is what makes it answerable.

4 · The Promotion Chain

A document becomes a decision through a sequence of representations. Each step is locally reasonable. The sequence loses information at every step.

A promotion chain is the ordered set of representations between a source and a decision; each hop changes the form of the information and the use to which it may be put. A hop is not a copy; it is a transformation with a purpose, and the purpose determines what the hop keeps.

Here's an example: a sentence in a review note becomes a passage in a search index, a clause in a parser, a predication in a record, an edge in a graph, a feature in a model, a line in a report, a number on a dashboard, and a decision to deny a refund. The decision can be defended only if the losses at those eight hops are known.

The eight hops

The chain is easiest to see as a table. Each row is a hop, the representation it produces, and the class of information it drops.

Hop	Representation	What it drops
1	document -> passage	layout, adjacent context, document structure
2	passage -> clause	sentence boundaries, anaphora, discourse links
3	clause -> predication	wording, force, holder, modality
4	predication -> edge	span, valid time, defeat condition
5	edge -> feature	individual claim identity, provenance
6	feature -> report	computation, raw values, exceptions
7	report -> dashboard	caveats, uncertainty, update time
8	dashboard -> decision	alternatives, authority, review

The table is not a claim that every chain has eight hops. A short chain may have three. The eight are the common set in an operational system, and the classes of loss are the same at any length.

The table is a diagnostic, not a law. A chain that skips a hop—a report built directly from a document—has fewer losses and less reach. A chain that adds a hop, such as

a model feature between the edge and the report, has one more class of loss and one more owner. The diagnostic asks which of the eight classes appear in a given chain, not whether the chain has exactly eight steps.

The classes fall into three families. The first hop loses form: layout, structure and context. The middle hops lose warrant: holder, force, time, standing and defeat. The last hops lose computation: exceptions, caveats and uncertainty. The families matter because they are repaired by different means: form by a better parser, warrant by a claim record, computation by a declared projection.

Each hop is reasonable

No hop in the table is a mistake. Each solves a local problem.

A passage is the unit a search engine can index. A clause is the unit a parser can analyze. A predication is the unit a record can store. An edge is the unit a graph can traverse. A feature is the unit a model can consume. A report is the unit a manager can read. A dashboard is the unit an operator can monitor. A decision is the unit an organization can act on.

The chain is built by specialists, and each specialist is accountable for the output of one hop. The passage builder is accountable for retrieval. The parser is accountable for the clause. The graph team is accountable for the edge. No one is accountable for the document-to-decision path, because the path is not anyone's unit.

Consider the first hop. A document becomes a passage because retrieval cannot search a whole document efficiently; the passage keeps the words and drops the layout, the heading hierarchy and the adjacent context. The hop is correct for retrieval and wrong for anyone who needs to know whether a clause was a heading, a footnote or a quotation. The loss is not an accident; it is the price of the passage.

That division of accountability is why the chain survives. It is also why the cumulative loss is invisible.

The loss is cumulative

Each hop validates its output against its input. The report is checked against the feature; the dashboard against the report; the decision against the dashboard. The validation is local, and it is usually correct.

None of the validations checks the decision against the document. The dashboard can be an accurate summary of the report, the report an accurate summary of the feature, and the feature an inaccurate summary of the predication. The error is not visible at any hop, because every hop is consistent with its immediate predecessor.

Here's an example: a report counts approved orders. The count is correct relative to the feature table. The feature table was built from edges. The edges were built from predications. One predication dropped the negation in *not approved*, and the edge records

approval. Every hop after the predication is correct; the count is wrong; and the error is eight hops away from the step that caused it.

The distance is the problem. A reviewer at the dashboard cannot see the predication, and a reviewer at the predication cannot see the dashboard's consequence.

A second example makes the distance concrete. A date is stored as a string in the document, parsed to a timestamp in the predication, truncated to a day in the feature, formatted for a region in the report, and rounded in the dashboard. Each hop is defensible; the decision rests on a date that no longer means what the document meant. The error is not in any hop's arithmetic; it is in the chain's accumulation.

Why the chain is hard to audit

An audit asks a simple question: what did this decision rely on? The answer requires the path, and the path is distributed across systems, owners and time.

Each system can produce its own logs. The search index can say which passage was retrieved; the parser can say which clause was analyzed. The graph can say which edge was traversed; the model can say which feature was used; the report can say which query produced the number. The logs do not compose into a single answer, because they use different identifiers, different clocks and different notions of a unit.

The identifier mismatch is the usual reason. The search index identifies a passage by offset; the parser identifies a clause by sentence number; the graph identifies an edge by a generated key; the report identifies a row by a metric name. A reviewer who wants to trace one decision must translate among four identifier schemes, and the translation is not recorded anywhere.

The clocks differ too. The document has a publication date; the parser has a run time; the graph has a load time; the report has a refresh time; the decision has an approval time. A reviewer who asks *what did we know on 3 April* must decide which clock the question refers to, and the chain records all five without saying which is which.

The chain also changes after the decision. A parser is upgraded, a feature is redefined, a report is rebuilt. The decision was made under one chain and is reviewed under another, and the difference is invisible unless each hop's output is versioned.

A chain that cannot reconstruct its own path cannot be audited, even when every hop is individually correct.

Carrying the losses

The repair is not to shorten the chain. Some chains must be long, because each hop serves a real consumer. The repair is to make each hop declare its loss.

A loss ledger records, for each hop, what was dropped, why it was dropped, and which question can no longer be answered from the output. The ledger is short: one row per hop, with the class of loss and the consumer who accepted it.

Here's an example: the predication hop records that it dropped the holder and the defeat condition, because the graph consumer needed the relation and not the warrant. The ledger says that the question *who approved this* is no longer answerable from the edge, and that the answer requires the claim record.

The ledger does not restore the loss. It makes the loss inspectable, which is what an audit needs.

A ledger row has four fields: the hop, the class of loss, the question no longer answerable, and the owner who accepted it. The row is short enough to review and specific enough to test.

The ledger has an owner. The owner is not the team that built the hop; it is the team accountable for the decision the chain supports. That team accepts the loss or asks for a shorter chain. A ledger that records the loss without recording the owner is a description of the problem rather than a control on it, and it will be filed with the other descriptions.

The ledger is not a data dictionary. A dictionary describes what a field means; the ledger describes what a hop dropped and which question is no longer answerable. The two documents are often confused, and the confusion produces dictionaries that describe the surviving fields and never mention the losses.

When the chain is short

A chain of two or three hops can be safe without a ledger, because the distance between the source and the decision is small enough for a reviewer to hold in mind. A direct query from a table to a report is traceable by reading the query.

The risk rises with distance. At eight hops, no reviewer holds the chain in mind; the ledger is the only representation of the whole. The decision to skip the ledger is a decision to rely on the reviewer's memory, and the memory is not auditable.

The test for a ledger is consequence, not length. A three-hop chain that supports a refund decision needs the ledger; a seven-hop chain that supports a dashboard nobody acts on may not. The question is what happens when the decision is challenged, not how many hops the chain has.

The test

Take one decision. Walk back one hop at a time. At each hop, name what the hop dropped and which question is no longer answerable from its output.

If every hop can name its loss, the chain is traceable. If a hop cannot, the chain has an unrecorded loss, and the decision is resting on a step nobody can inspect.

The walk is easier if the chain's outputs are versioned. A versioned output lets the reviewer compare the decision's chain with the current one, and the comparison shows whether the loss was introduced before or after the decision. Without versions, the walk ends at the first rebuild.

Then write the loss ledger for the chain. The ledger is not a summary of the system; it is the list of questions the system can no longer answer.

A decision that cannot name its losses is not traceable, even when every step is correct.

5 · The Unification Thesis

An ontology, a semantic layer and a knowledge graph are three representations of one record. Each answers a different class of question. Each fails the same six questions.

An ontology states which kinds of thing exist and which relations are permitted. A semantic layer states what a metric means and how it is computed. A knowledge graph states which things are connected and how they can be reached; the three are usually built by different teams, stored in different systems and governed by different owners.

The record underneath them is the same object: a claim, with a holder, a content, a span, an authority, a valid time and a defeat condition. The three representations are projections of that record, and the projections lose different information.

Three containers

The three representations are easiest to compare as containers of a question. Each container answers the questions its builders designed it for.

Container	Answers	Cannot answer
Ontology	what kind of thing this is; which relations are permitted	who asserted it; when it was true; what would defeat it
Semantic layer	what the metric means; how it is computed	who supplied the values; on what evidence; under what authority
Knowledge graph	what is connected to what; how to reach it	who authorized the link; in what words; with what force

The table is not a criticism of any container. Each is a specialized representation, and specialization is why each is useful. The table shows that the three specializations share a blind spot.

The containers are also owned differently. The ontology is owned by a data-governance group, the semantic layer by analytics engineering, and the graph by a platform team. Each owner is accountable for the questions its container answers, and none is accountable for the six questions the container cannot answer.

The three representations are often described as competing approaches, and the competition is real at the level of budget and ownership, but it is not real at the level of the question: each container answers a different question, and none answers the six.

The budget competition is real, and it is the reason the three containers are rarely compared on the question that matters, which is whether any of them can answer the six questions an auditor asks.

The same six questions

Ask the six questions of any of the three, and the answer is the same: the container does not hold the claim.

Here's an example: order 44021 is approved. The ontology can say the order is an instance of `ApprovedOrder`. The semantic layer can compute the count of approved orders. The graph can show the order connected to the approver. None of the three can say who approved it, in what words, with what force, valid as of when, under what standing, or what would retract it.

The failure is not in the answer each container gives. Each answer is correct for its question. The failure is that the question the auditor asks is not any of the three questions.

A second example shows the same failure in a metric. Three reports define `active customer` differently: one counts orders in 90 days, one counts logins in 30 days, one counts accounts not churned. The semantic layer can define all three metrics and compute them correctly. It cannot say which definition is in force for a given decision, because the decision's claim is not in the layer.

The cost appears at audit. A reviewer asks which definition applied to the orders in last quarter's report. The semantic layer can show the current definition and the current computation; it cannot show the definition in force when the report was produced, because the report's projection did not record the claim.

One cause

The three containers fail for one reason: none of them stores the claim. Each stores a projection of the claim, shaped for its own question, and each projection drops the properties the others also drop.

The ontology drops the holder and the time because a class assertion does not need them. The semantic layer drops the holder and the evidence because a metric does not need them. The graph drops the force and the defeat condition because a traversal does not need them. The three drops are different in form and identical in kind: the claim record is absent.

The three drops look different because the containers are different. The ontology drops the holder because a class assertion is about kinds, not people. The semantic layer drops the evidence because a metric is about values, not sources. The graph drops the

defeat condition because an edge is about connection, not withdrawal. The common cause is that each container represents a projection and none represents the assertion.

The difference is not in the technology each container uses but in what each container was asked to represent, and the common omission follows from the question, not from the implementation.

That is why merging the three does not help. A merged model still has no field for the holder, the span or the defeat condition. It has three sets of projections in one repository, and the same six questions still fail.

One ledger, three projections

The alternative is one record and three projections. The record stores the claim. The projections are built from it for the ontology, the semantic layer and the graph, and each projection declares what it drops.

The record has the fields the six questions require: holder, content, span, authority, valid time, defeat. The content may project into a class assertion, a metric definition or an edge; the holder, span, authority and defeat remain in the record.

Here's an example: the approval claim for order 44021 projects into an ontology assertion (`44021` is an `ApprovedOrder`), a metric (`approved_orders` includes 44021), and an edge (`44021` approved by `rao`). Each projection answers its question. The record answers the six.

The projections need identifiers. The ontology projection uses the class IRI, the metric projection uses the metric name, and the graph projection uses the edge key. The record needs one identifier that all three can resolve, and the identifier must be stable when the projections are rebuilt. Without it, the same claim becomes three records with three keys.

A projection is not a copy. A copy preserves the record; a projection selects the fields a consumer needs and drops the rest. The distinction matters because a copy can be used to rebuild the record, and a projection cannot. The three containers hold projections, and the record is the only copy.

The record is also not a data warehouse. A warehouse stores projections for consumers; the record stores the claims those projections depend on. The two can share a database, but they cannot share a grain: the warehouse's grain is the consumer's question, and the record's grain is the claim.

That difference in grain is why a warehouse cannot be the record even when it stores every field, because the warehouse's rows are shaped by the questions its consumers ask and the record's rows are shaped by the claims its sources make.

The projections lose differently, and the difference is the point. The ontology projection loses the wording and the time. The metric projection loses the individual claim and its evidence. The graph projection loses the force and the defeat condition. A consumer who needs one of those properties reads the record, not the projection.

The merge fallacy

The common alternative is to merge the three models into one. The merge is attractive because it reduces the number of systems and promises a single vocabulary. It does not produce a claim record.

A merged model can have a class for every entity, a metric for every measure and an edge for every relation. It still cannot represent the holder, the span, the authority, the valid time or the defeat condition, because those are not classes, metrics or edges. They are properties of an assertion, and the merged model is still a model of things, not of assertions.

The merge also concentrates the loss. When three projections are separate, a consumer can compare them and notice a disagreement. When they are merged, the disagreement is resolved by the merge, and the resolution is not recorded.

The merge also creates a governance problem. A merged model has one owner, and the owner must adjudicate every disagreement among the three original owners. The adjudication is a decision, and it needs a record of its own; otherwise the merge becomes a fourth projection with the same missing claim.

A merged model that records its adjudications has begun to build the record; a merged model that does not has simply moved the missing claim from three systems into one.

The move is not neutral: it hides the missing claim behind a single schema, and the hiding is discovered only when a decision is challenged and the schema cannot answer.

The decision

The decision is not which container to buy. It is where the claim record lives, and how the three projections are built from it.

The record does not need a new platform. It can live in the operational database, beside the tables it describes, with one table for claims and one projection job per container. The projection jobs are small: read the claim, write the class assertion, the metric row or the edge, and record the projection's loss.

The projection contracts are the work. Each contract states the source record, the fields the projection keeps, the fields it drops and the questions it is intended to answer. Three contracts make the three representations comparable, and the comparison is what an audit needs.

The contracts also make the record testable. A projection job that cannot name its loss has not been specified; a record that cannot answer the six questions has not been built. The test is not whether the three containers agree, but whether the record can explain their disagreement.

The record is small: six fields per claim. The projections are the systems the organization already has. The work is the projection contract: for each projection, state the

source record, the fields it keeps, the fields it drops and the questions it is intended to answer.

The contract is what makes the three representations comparable. Without it, each team believes its projection is the record, and the disagreement between them is discovered only when a decision is challenged.

The six benefits

The benefits are easier to defend as a table, because each one answers a failure of the entity-centric store.

Benefit	Predication-centric record	Entity-centric failure
reachability	links are stored and traversed	the path is reconstructed from keys at query time
correction	one claim row changes	the same fact is copied into several rows and views
attribution	holder, span, force, time and defeat are fields	the row has a value and no holder
projection	one ledger, many declared views	each view is a copy and the copies drift
federation	a shared roster and identifier resolve	every exchange negotiates a schema
governance	supersession and closed sets	updates overwrite and the vocabulary stays open

The table is not a scorecard. The benefits trade against cost: a claim record costs more rows than a status column, and a projection costs a rebuild when the record changes. The trade is worth it when the consequence of a wrong answer is high enough that the warrant must be inspectable.

Here's an example: a recall decision needs reachability, attribution and correction at once. The graph supplies the route, the claim record supplies the authorization, and the supersession chain supplies the correction. An entity-centric store can supply the route and must reconstruct the other two.

The test

Take one claim that all three containers represent. Ask the six questions of the ontology, the semantic layer and the graph. If all three fail, the organization has three projections and no record.

A failure case makes the test concrete. The ontology says the order is approved, the metric counts it as approved, and the graph shows no approver because the edge was never loaded. The three containers disagree, and none can say which is right. The record can: it holds the claim, the span and the absence of the edge, and the absence is a gap with a reason.

The record's answer is not that the three containers are wrong; it is that each is a projection with a declared loss, and the disagreement is exactly the loss the projections were supposed to make visible.

Then ask each container what it drops. If the container cannot answer, it has no projection contract, and its loss is unrecorded.

Then build the record for that one claim. The record does not require a new platform; it requires a table with the six fields and a projection job for each container. The test is whether the same claim can be answered from the record and projected into all three without changing its meaning.

A claim that survives three projections is a claim with a record. A claim that changes meaning in each projection is not one claim; it is three projections with no record.

PART II
The Atom

what the unit is.

6 · Predication Is the Atom

A representation has an atom when it has a smallest unit that can be wrong on its own. The atom is not the smallest unit of language, and it is not the sentence. It is the predication.

Atomicity here means attributability, not size. A unit is atomic if it can be evaluated, corrected or refuted without first deciding what the rest of the text means. A word is smaller and cannot be wrong on its own. A sentence is larger and can be wrong for the wrong reason: one clause is false and the sentence is false, or one clause is false and the sentence is still true because another clause carries the point.

The grain test is simple. Take a unit and ask three questions: can it be true or false by itself, can it be corrected without rewriting a larger unit, and does it name the act that makes it a claim. The unit that passes all three is the atom.

The grains

The test is easiest to apply as a table.

Unit	Can it be wrong on its own?	Can it be corrected alone?	What it lacks
word	No	No	truth, force, participants
phrase	No	No	a complete act
clause	Sometimes	Sometimes	one clause can carry two claims
sentence	Yes, but for too much	No	several claims share one truth value
document	Yes, but for everything	No	no isolable unit
predication	Yes	Yes	nothing required for attribution

A word cannot be false. *Valve* is neither true nor false; it is a term. A phrase such as *the closed valve* is a description, not an assertion. Negation alone does not make a phrase a claim; the phrase must also name an act and a holder. A sentence can be false, but its

falsity is not attributable to one place: a sentence with two clauses has two claims and one truth value.

The document is the far end of the same scale. A document can be false, and its falsity is attributable to nothing in particular: a report with one wrong number and twelve right ones is false, and the reader cannot tell which number to correct without reading the whole document. The document is a container of claims, not a claim.

The clause is also unstable across languages. A language that packs manner into the verb and a language that packs it into a satellite produce different clause boundaries for the same claim, and a record built on clauses inherits the difference. A record built on predications does not.

The clause is the closest competitor, and it fails for a specific reason. A clause is a packaging boundary, not a semantic one. The sentence *Maria opened the window and sat down* contains two predications, one for each verb, and the coordination packs them into one clause. If the correction is *Maria opened the window but did not sit down*, the clause must be split before either claim can be corrected. The clause was never the unit; it was the unit of punctuation.

The two failures

The grain test has two failure directions, and both are common.

Cut smaller and the pieces assert nothing. *Bob* cannot be false; *the red book* cannot be false; the terms are not claims, and no arrangement of them is a claim until a predicate binds them. A record of terms can be complete and say nothing.

Enlarge and claims hold each other hostage. *Bob gave Ava a red book* cannot be wrong about the color without also being wrong about the giving, because the sentence has one truth value; the error is real and its location is not. The sentence fails as a unit, and the unit is too large to correct.

The atom is the unit whose failure can be attributed to it alone. That is the whole of the criterion, and it is why the predication wins over both the word and the sentence.

What the predication carries

A predication carries the content of one act: a predicate, its participants, and the dimensions that make the act a claim.

The content is the predicate and its roles. *Close* has a theme; *approve* has an agent and a theme; *give* has an agent, a recipient and a theme. The roles are the participants the predicate requires, and they are closed by the role set, not by the verb's whim.

The predicate is a sense, not a surface verb. *Close*, *closed* and *closing* are three surfaces of one predicate, and the record stores the sense once. The surface belongs to the span; the sense belongs to the claim.

The dimensions are the properties of the act: voice, aspect, mood, tense, number, modality, polarity, force. They determine what the claim commits its holder to, and they

are not decoration. *The valve was closed* and *the valve was not closed* share a predicate and a theme and differ in polarity; the difference is the claim.

The roles and the dimensions are often conflated. A role answers who or what participated; a dimension answers how the act was made. *The valve was closed by the technician* adds an agent role; *the valve was closed* already has a voice and a tense, and those are dimensions. A record that stores the role in the dimension, or the dimension in the role, cannot answer either question.

The span is the source location. A predication without a span is a claim without a place to check, and the record treats it as incomplete rather than as true.

Here's a second example: *A. Rao approved order 44021 under grant 12 on 11 April* contains one predication with an agent, a theme, an authority and a time. The phrase *under grant 12* is a relation to an authority, and the record stores that relation in the claim's authority field. The predication is one; the sentence's clauses are packaging.

Here's an example: *The valve was not closed before the pump started* contains two predications. The first is `CLOSE(theme=valve, polarity=negative, tense=past, voice=passive, force=assertion)`. The second is `START(theme=pump, tense=past, force=assertion)`. The word *before* is a relation between the two, not a third predication. The sentence is one packaging; the record stores two claims and one relation.

A row is a predication

The same test applies to a table row, and the result is less obvious.

A row such as `orders(customer_id, product_id, ordered_at)` encodes a predication whose verb was fixed when the table was designed. The table name supplies the predicate: the row asserts an order. The columns supply the roles: customer is the agent, product is the theme, date is the time. The schema supplies the force, usually an assertion by the source system.

Here's an example: order 44021 with customer 847, product P-12 and `ordered_at` 2026-04-10 encodes `ORDER(agent=customer847, theme=productP12, time=2026-04-10)`. The row does not contain the word *order*; the table name does. A reader who treats the row as a fact rather than as an assertion has already lost the distinction between the source and the claim.

The row passes the grain test. It does not answer the six questions, because atomicity and completeness are different properties. The row can be corrected alone; it cannot be defended alone.

The schema's verb is easy to miss because it is not a word on the row. It is the table's purpose, written once in the design and never repeated in the data.

Why the sentence fails

The sentence is the unit prose is written in, and it is the wrong unit of representation for three reasons.

First, a sentence can contain several claims and one truth value. *The valve was closed and the pump started* is false if either clause is false, and true if both are true; the sentence does not say which clause failed. A correction must rewrite the sentence, not the claim.

Second, a sentence carries packaging that the claim does not need: word order, rhetorical emphasis, punctuation, conjunction. The packaging matters to a reader and not to a record. A record that stores the sentence stores the packaging with the claim and cannot separate them later.

Third, a sentence can be true while a clause is false, and the reverse. *Although the valve was closed, the pump did not start* is true when both clauses are true; it is also true when the valve was closed and the pump did start, because *although* concedes rather than asserts. The sentence's truth value is not the sum of its clauses.

Punctuation is packaging as well. A semicolon, a dash or a conjunction can change the force of a clause without changing its content, and a record that stores the punctuation stores a decision about emphasis that the claim did not make. The claim has no punctuation.

Fourth, a sentence can carry a stance that the claim does not. *Carter testified that she saw Cole leave* is a sentence about testimony; the inner claim, *Cole left*, is not asserted by the speaker. A record that stores the sentence stores the testimony; a record that stores only the inner predication loses the testimony and keeps the content. The atom must be the claim the speaker made, not the clause inside it.

The philosophical case

The grain test is an engineering test, and it rests on a distinction philosophy made first. Frege distinguished an object from a concept: an object is named, a concept is predicated, and the combination is a thought that can be true or false. A list of names is not a thought; the act of predication is what makes the list a claim.

Davidson gave the modern form. An action sentence quantifies over an event: *Jones buttered the toast* says there is an event of buttering, with Jones as agent and the toast as patient. The event is what adverbs modify and what can be wrong. A record that stores the participants and the verb has the terms and not the event.

The predication is the atom because it is the smallest unit that carries the event or state variable, the roles and the truth value. The hub is that variable; the spokes are its participants; the rim is how the variable is qualified.

The objection is that events are metaphysics, and a record needs only sentences. The record does not need events as furniture; it needs a bearer for the roles, the time and the polarity. The predication is that bearer, and the engineering test and the philosophical account agree on its boundary.

Frege's distinction also explains why the sentence is the wrong unit. A sentence can contain several predications and one truth value, and the truth value belongs to the combination, not to the parts. The record's atom is the part that can be true or false on its own, which is the predication and not the sentence.

The linguistic evidence points the same way. A verb's sense determines the participants it permits; the participants do not determine the verb. *Give* requires a recipient; *close* does not. The frame is a property of the predicate, which is why the hub is the predicate and the spokes are its roles.

The philosophical case is not decoration. It explains why a store that begins with entities has to invent a relation to recover the act, and why a store that begins with predications has the act and can derive the entities from its roles.

The caveat is that the record is not a Davidsonian semantics. It does not quantify over events in the logical sense, and it does not claim that every verb has an event argument. It uses the event or state as the bearer of the claim's roles, which is the part of the account the engineering needs.

The correction test

The atom is the unit at which a correction is confined. If a correction changes one claim and leaves the others intact, the unit is the claim. If a correction forces a rewrite of the surrounding text, the unit is larger than the claim.

Here's an example: the record says the valve was closed. The correction says it was not closed. If the record stores a sentence, the correction rewrites the sentence and may change the punctuation, the conjunction or the emphasis. If the record stores a predication, the correction changes the polarity on one row and leaves the pump claim intact.

The correction test is the operational form of the grain test. It asks not what the unit is called but what a correction costs.

The cost is measurable. A correction at the sentence level requires a reviewer to read the sentence, decide which clause is wrong, rewrite it and re-check the surrounding text. A correction at the predication level requires a reviewer to change one field and re-run the reconciliation. The second is cheaper, and the difference is the reason the atom matters operationally.

The atom and the encoding

The atom of representation is the predication. The synapse is its encoding: the hub, the spokes, the rim and the span. The distinction is between the claim and its storage shape, and the next chapter describes the shape.

A predication can be stored in a row, a document or a graph edge. The storage shape is a design choice; the atom is not.

The atom is independent of the storage technology. A predication can be a row in a relational table, a node in a graph, a document in a store or a record in a ledger. The storage choice affects performance and projection; it does not affect what counts as one claim.

This is the compiler's first constraint. A compiler that emits sentences has no unit to correct; a compiler that emits predications has one. The rest of the machine—the span, the roles, the dimensions, the gates—is built on that unit, and a machine that begins with sentences cannot recover it later.

The choice also fixes what a correction can name. A sentence-level store can name a rewrite; a predication-level store can name a claim. A record that cannot isolate one predication cannot isolate one correction, and a record that cannot isolate one correction cannot be audited at the claim level.

A record that stores sentences corrects sentences; a record that stores predications corrects claims. The second can audit the correction; the first can audit only the rewrite.

7 · The Synapse

A synapse is the storage shape of one predication: a predicate at the hub, participants on spokes, dimensions on the rim, and a span beneath.

The shape exists to keep one claim in one place. The hub holds the predicate occurrence and its sense. The spokes hold the participants and the roles they fill. The rim holds the dimensions that make the act a claim rather than a relation. The span holds the source location that lets a reviewer check it.

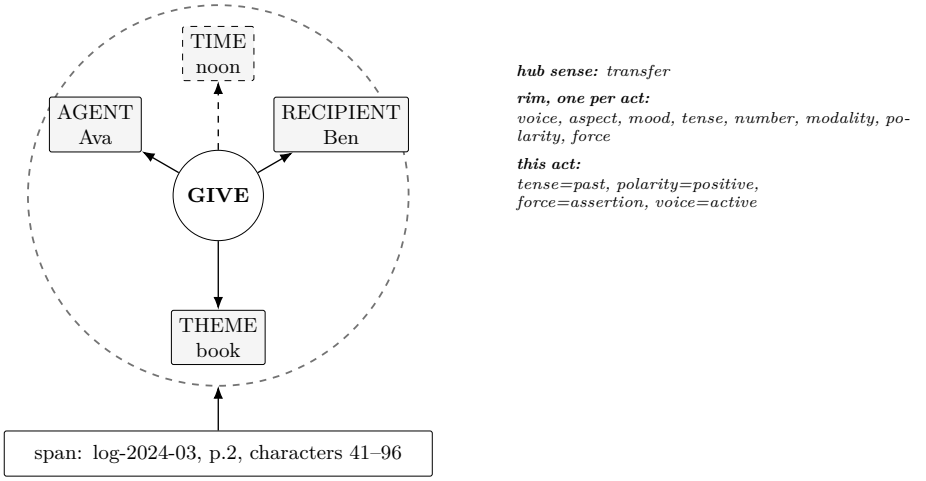


Figure 1: The synapse: a predicate at the hub, participants on spokes, eight dimensions on the rim, the span beneath.

The figure is the shape; the rest of the page is what each part is for. Read the rim as the reason two predicates cannot share one hub: they would have to agree on all eight dimensions at once.

A sentence diagram and a synapse look similar and answer different questions. The diagram describes the grammar of a sentence. The synapse records the claim a reader can correct, project or challenge.

What the synapse is not

It is not a sentence diagram. A diagram preserves word order and grammatical function; a synapse preserves roles and dimensions and drops word order. The two can be drawn to look alike, and the likeness is a trap: a diagram read as a record will be corrected at the level of the drawing.

It is not a frame. A frame is a type: the slots a verb permits. A synapse is an occurrence: one act, at one time, with one span. A frame can be reused; a synapse cannot.

It is not an edge. An edge is binary and carries two endpoints; a synapse carries a predicate, several roles and eight dimensions. An edge can represent a relation; it cannot represent the act that made the relation a claim.

The shape costs more rows than a sentence table. A sentence is one row; a synapse is one claim row plus one row per spoke. The cost buys correction at the claim level and projection into graphs, tables and documents without re-parsing. A team that needs neither can store sentences; a team that needs both cannot.

The choice between a sentence table and a synapse table is therefore a choice about what a correction can name, not a choice about storage cost, because the storage cost is small in both cases and the correction cost is not.

The synapse is not the claim record. It carries the predicate, the roles and the dimensions; the claim record adds the holder, the authority, the valid time and the defeat condition. A synapse without a claim row is a relation; a claim row without a synapse has no content.

The hub

The hub is the predicate, not the grammatical subject. A subject-centered store puts the frame on the subject and copies it onto every spoke; a predicate-centered store puts the frame once, at the hub, and lets the spokes refer to it.

Three propositions are usually said as one. The first is that every English clause has a verb, which is school grammar and useful for finding clauses. The second is that the hub is that verb token, which is false: *Hot stove!*, *the destruction of the city* and *On the table.* are claims with no tensed verb on the page. The third is that the hub is the predicate of the event or state, which is what the record needs. Separating the three is the move that makes verbless claims storable.

Here's an example: *The technician closed the valve* and *The valve was closed by the technician* are two sentences with one predication. The hub is `CLOSE` in both, with the same sense. The first sentence makes the technician the grammatical subject; the second makes the valve the grammatical subject. A store keyed on the subject would create two records; a store keyed on the predicate creates one.

A subject-centered store also duplicates the frame across spokes. If the frame is on the subject, every spoke that refers to the subject must carry a copy, and the copies can disagree. The predicate-centered store has one frame and no copies.

The hub requires a sense, not a surface verb. *Close* as in shut, *close* as in near, and *close* as in finish are three senses and three hubs. The surface string is not the hub; the sense is. A hub without a sense is an ungrounded relation, and the record refuses it. A hub has a sense.

Here's an example: *The bank closed* (shut), *the bank is close* (near), *the meeting closed* (finished) are three senses of the same string, and each hub is distinct. A store that keys on the string would merge them.

The spokes

A spoke is a participant in a capacity. It carries the role, the relation, the quantity and the kind of target, not merely a filler.

The role is one of the closed set: agent, theme, recipient, instrument, location, time and the rest. The relation is how the filler connects to the predicate. The quantity is how many or how much. The target kind says whether the filler is an entity, a literal, a span or another claim.

Here's an example: *Ava gave Ben a book at noon* has three spokes. `AGENT=Ava`, `RECIPIENT=Ben`, `THEME=book`, and a fourth for `TIME=noon` if time is carried on a spoke rather than the rim. Each spoke names its role and its filler; a spoke that names only the filler is a column, not a role binding.

The role set is closed, and the closure is what makes a spoke checkable. A new verb may require a new participant, and the participant must fit an existing role or the parse is refused. The alternative—a role per verb—makes two claims incomparable. A spoke has a role.

The spoke's fields matter because corrections happen at the spoke. A correction that changes the recipient changes one spoke and leaves the predicate and the other participants intact. A correction that changes the predicate changes the hub and may invalidate every spoke.

The predicate-centered shape also matches the correction. A correction that changes the verb changes the frame; a correction that changes a participant changes a spoke; a correction that changes the time changes the rim. The subject-centered shape forces every correction through the subject, even when the subject is unchanged.

The rim

The rim carries the dimensions of the act: voice, aspect, mood, tense, number, modality, polarity and force.

The dimensions are not decoration. *The valve was closed* and *The valve was not closed* share a hub and a theme and differ in polarity; the difference is the claim.

Force is the dimension that determines what the claim does. An assertion commits its holder to truth; a directive commits to an action; a question commits to an answer.

The hub and spokes can be identical across the three, and the force is the difference. The rim has eight dimensions.

The dimension set is closed for the same reason the role set is closed. A new dimension would make two synapses incomparable unless every consumer learned it, and the cost of a dimension is paid by every projection. The rim is closed.

Here's an example: *The valve was not closed before the pump started* has two synapses. The first has `CLOSE` at the hub, `theme=valve`, `polarity=negative`, `voice=passive`, `tense=past`, `force=assertion`. The second has `START` at the hub, `theme=pump`, `tense=past`, `force=assertion`. The word *before* is a link between the synapses, not a dimension of either.

One hub, one claim

Two predicates in one hub merge two commitments. Order, stance, polarity, frequency and word order become unanswerable at once, because the hub has one set of dimensions and two acts need two.

Here's an example: *Ava gave Ben a book and took a pen* contains two predications. A store that puts both verbs in one hub must choose one polarity, one tense and one force for the pair. The choice is not in the sentence; it is in the store, and it is invisible to the reader.

The merge is tempting because the sentence is one clause and the store wants one row. The temptation is the reason the synapse has a hub per predicate rather than a hub per clause.

Order is the clearest loss. *Ava gave Ben a book* and *Ben gave Ava a book* have the same hub, the same roles and different fillers; a merged hub cannot say which reading is in force. Stance is the second: a report and a fact share the hub and differ in the chain that holds them.

The two givings

A flatter failure makes the same point. Five role rows with no claim key can assemble into two readings.

Here's an example: the rows `(give, agent, Ava)`, `(give, recipient, Ben)`, `(give, theme, book)`, `(give, recipient, Cleo)`, `(give, theme, pen)` can be read as Ava gave Ben the book and Cleo the pen, or Ava gave Ben the pen and Cleo the book. The rows are identical; the readings differ. Without a claim key, the store cannot choose.

With a claim key, the rows attach to two hubs: hub 1 has agent Ava, recipient Ben, theme book; hub 2 has agent Ava, recipient Cleo, theme pen. The readings are distinct because the keys are distinct.

The fixture is small and the failure is general: a store without a claim key cannot distinguish two claims that share all their rows. The key is not a convenience; it is what makes the rows a claim.

A store that omits the key can still answer the question *what did Ava give* by aggregating the rows, and it cannot answer *what did Ava give to Ben* without guessing which theme belonged to which recipient.

The synapse's claim key is what resolves the two readings. Each spoke belongs to one hub, and each hub belongs to one claim. The key is not metadata around the relation; it is the relation's identity.

The storage shape

The synapse is a storage shape, not an illustration. One claim row carries the predicate occurrence and the rim. One row per spoke carries the role and the filler. A typed join connects the spoke to its claim.

The shape is small. A claim row has the hub, the sense, the dimensions and the span. A spoke row has the claim key, the role, the filler and the target kind. The join is a foreign key, not a diagram.

Row	Fields
claim	claim_id, hub sense, dimensions, span
spoke	claim_id, role, filler, target kind

The key is a foreign key. A claim has a span.

Here's an example: the valve sentence stores as two claim rows and four spoke rows. The claim rows carry `CLOSE` and `START`; the spoke rows carry the valve, the pump and their roles. A correction to the polarity changes one claim row and leaves the spokes intact.

The test

Take one sentence with two verbs. Decompose it into synapses: one hub per predicate, one spoke per participant, one rim per claim.

Then rebuild the sentence from the synapses. If the rebuild preserves the roles, the polarity, the force and the order of the claims, the decomposition is adequate. If it merges two claims or loses a participant, the shape is wrong.

Then ask the five questions of each synapse: who, what, when, under what force, and what would retract it. A synapse that cannot answer is incomplete; a sentence that cannot be decomposed is packaging.

Three failures are common. The decomposition merges two hubs; the rebuild loses a participant; the rim drops the polarity. Each failure is visible in the rebuild, and each is a defect in the shape rather than in the sentence.

The rebuild is the only test that catches all three, because the rebuild forces the store to reproduce the sentence's roles and dimensions from the recorded claim rather than from the original text.

A sentence diagram is a drawing of a sentence. A synapse is the record of a claim, and a correction can find it.

8 · Roles and Dimensions

A verb may be open; the frame is closed. The role set has fifteen members. The dimension set has eight. Every parse uses those sets or is refused.

The openness of the verb is what makes the representation usable; a new verb can enter without a schema change, because the predicate is a sense in the lexicon, not a column. The closure of the frame is what makes the representation checkable. A role outside the fifteen is not a new role; it is an error in the parse.

The open verb

A predicate is a sense, and senses are added to the lexicon as the corpus requires. *Approve*, *close*, *give* and *testify* are four predicates among many; a new one enters by adding a sense and its permitted frame, not by altering the table.

The open verb is the reason the record can grow. A closed verb list would require a schema change for every new action, and a schema change is a migration, a review and a version. The lexicon is the place where growth is cheap.

The open verb is also the reason the frame must be closed. If each verb brought its own roles, two claims with different verbs would be incomparable, and the comparison is what the record exists to support.

A new verb also brings a new sense, and a new sense can be ambiguous. *Close* as shut and *close* as finish are two senses with different frames; the lexicon must separate them before the parser can assign roles. The open verb is cheap only when the sense is resolved.

Fifteen roles

The roles are the addresses a participant can occupy. Each role protects a distinction; the loss column names what disappears when the role is folded into another.

The number is not sacred. It is the smallest set the corpus's questions need; a sixteenth role would enter by the fold test if a question required it. The number is a design decision, and the decision is revisable by amendment.

Role	The job it protects	What is lost if it folds
AGENT	the intentional initiator or controller	the wind becomes a chooser
PATIENT	what undergoes change, damage, creation or destruction	a damage claim loses its victim
THEME	what is moved, located, possessed or perceived without change	located things and broken things collapse
EXPERIENCER	who perceives, believes, feels or knows	giving and seeing become one relation
RECIPIENT	who receives the transferred thing	the addressee disappears into the beneficiary
BENEFICIARY	for whose benefit the act happens	addressee and beneficiary become one claim
TIME	when the act occurs	a deadline becomes a date
LOCATION	where the act occurs	a path loses direction
SOURCE	the endpoint a path starts from	*from Paris* must be recovered from a preposition
DESTINATION	the endpoint a path ends at	*to Paris* must be recovered from a preposition
MANNER	how the act is done	a method becomes a tool
INSTRUMENT	with what the act is done	a tool becomes a method
CAUSE	the non-choosing trigger	weather becomes an actor
REASON	the stated aim	a justification becomes a cause
ATTRIBUTE	a property used as a participant	measures and classifications are not represented

Here's an example: *The technician closed the valve with a wrench* assigns `AGENT=technician`, `THEME=valve`, `INSTRUMENT=wrench`. A parse that assigns the wrench to `MANNER` loses the distinction between how the act was done and what it was done with.

The roles are not arguments in the logical sense. An argument is a position in a predicate; a role is an address a question can use. The difference matters when a claim is queried: a question asks for the `AGENT` or the `THEME`, not for argument two.

The assignment is tiered. The first tier is the verb's frame: *close* requires a theme and permits an agent and an instrument. The second is the sentence's syntax: the passive subject is the theme. The third is the context: the wrench is the instrument because the sentence says *with*. A parser that skips a tier guesses.

The tiers also give the parser a refusal path. If the frame does not permit a role and the syntax does not supply it, the parser records a gap rather than inventing a filler; the gap is the honest output.

The role set is not a case grammar. Cases describe the form of a noun phrase; roles describe what a question can address. *The valve was closed* and *The technician closed the valve* have different grammatical subjects and the same `THEME`; the role is assigned by the question, not by the surface position.

Eight dimensions

The dimensions describe the act rather than its participants. Each has a closed value set.

Dimension	Values
voice	active, passive, middle
aspect	perfective, imperfective, habitual, progressive
mood	indicative, subjunctive, imperative, conditional
tense	past, present, future, none
number	singular, plural, mass
modality	actual, possible, necessary, permitted, obligatory
polarity	positive, negative
force	assertion, definition, directive, commitment, question, hypothesis, attribution, evaluation, fiction

Here's an example: *The valve was not closed* has `voice=passive`, `aspect=perfective`, `mood=indicative`, `tense=past`, `number=singular`, `modality=actual`, `polarity=negative`, `force=assertion`. The eight values are the whole frame of the act; a parse that omits the polarity has lost the claim.

The value sets are closed for the same reason the roles are closed. A dimension with open values cannot be compared; a dimension with closed values can be validated and projected. The closure is what makes the frame a frame rather than a bag of tags.

The dimensions are not tags. A tag is optional metadata; a dimension is a value the claim must have. A parse without polarity is not a claim with missing metadata; it is a claim that cannot be evaluated.

Mood and force are often conflated. Mood is a property of the verb's form; force is what the claim does. *Close the valve* is imperative in mood and directive in force; *The valve is closed* is indicative in mood and assertive in force. The two dimensions can vary independently, and a store that keeps only one loses the other.

The operator layer

The dimensions qualify one act. Some meaning does not: *not*, *every*, *must* and *before* relate to more than one act or bind a variable across acts. These are operators, and they need a scope.

Negation is the clearest case. *The valve was not closed because the pump started* has two readings: the valve was not closed, and the pump start is why; or the valve was closed, but not because the pump started. The words are identical; the scope of *not* differs. A `polarity=negative` value on the valve claim can represent the first reading and cannot represent the second.

Quantification is the second. *Every student read a book* can mean one book for all students or a book for each. The quantifier binds the object; the record needs a variable and a scope, not a single role filler. The same is true of *most*, *each*, *only*, mass nouns, collectives and generics.

Modality is the third. *The valve must be closed* is an obligation, not a fact; the modal scopes over the claim. A conditional scopes over two claims, and a counterfactual scopes over a claim and a premise that is false.

Time is the fourth. *Before* orders two events; it is a relation between claims, not a value on either. A tense dimension records when one act is located; a temporal operator records how two acts are ordered.

The operator layer is a set of rows: type, operand, scope and warrant. The row is not a fourth join; it is a scope annotation on a claim or a group. The distinction matters because an operator can move without changing the roles, and a role change can leave the operator in place.

Why not just dimensions? A dimension has a value on one claim, and the value is part of the claim's frame. An operator has scope over claims, and the scope is part of the record's structure. Polarity records the sign; the operator records what the sign applies to. Collapsing the two loses the second reading.

Here's an example: *All valves were not closed* is ambiguous between *no valve was closed* and *not all valves were closed*. The quantifier and the negation interact; one row cannot hold both. The record stores two operator rows with different scopes, and the ambiguity is visible rather than resolved by a guess.

Operators also compose with force. A directive with a universal quantifier (*every operator must record the valve state*) scopes over all operators; a directive with an existential (*some operator must record it*) scopes over at least one. The force is the same; the scope changes who is bound.

The record does not resolve every ambiguity. Where the source does not determine the scope, the record stores a candidate with a gap, and the gap is a first-class row. The alternative is a silent choice, and a silent choice is the failure the operator layer exists to prevent.

The test is simple: parse a sentence with two readings. If the record has one representation and no scope field, the scope is missing.

Why closure works

Closure turns the parse into a membership test. A validator can check every role against fifteen values and every dimension against its value set, and a parse that fails the test is refused rather than stored.

The validator is small. It reads the role code and the dimension values and compares them to the sets; a value outside the set is a refusal, not a repair. The refusal is the point: a parse repaired by the validator is a parse the validator cannot audit.

The validator also produces a comparable record. A refused parse is a row with a reason, and the reason is available to the reviewer; a repaired parse is a row without a reason.

An open role set fails in a specific way: two parses of the same sentence can use different roles and both be accepted. The failure is not a wrong answer; it is an incomparable answer, and the incomparability is discovered only when a comparison is attempted.

Comparison is the second benefit. Two claims with the same roles and dimensions can be compared field by field; two claims with different frames cannot be compared without a translation. The closed set makes the comparison a table lookup and the translation explicit.

Federation is the third benefit. Two systems that share the sets can exchange a parse without a schema negotiation, because the addresses are the same. The exchange still needs shared senses and a shared policy, and the addresses are the precondition.

Where the roles came from

The roles have a lineage, and the lineage limits what the roster can claim.

Fillmore's case grammar (1968) proposed deep cases—agentive, instrumental, dative, factitive, locative, objective—as the relations between a verb and its arguments. Gruber (1965) and Jackendoff (1972, 1983, 1990) developed thematic relations as part of conceptual structure, with the verb's lexical entry determining which relations it permits. Davidson (1967) and Parsons (1990) made the event an argument of the sentence and treated the roles as separate relations to that event.

Dowty (1991) argued that agent and patient are not primitive roles but clusters of entailments—proto-agent and proto-patient—and that argument selection is a matter of degree. The history is not a list of fifteen; it is a series of attempts to explain why verbs select the participants they do, and no attempt produced a universal inventory.

The roster adopted here is an engineering set, not a linguistic universal. The lineage supplies the shape—roles attach to an event or state, and the verb's sense determines which roles are permitted—and it does not supply the number. The number is a governance decision, defended in the next section, and the roster is versioned so that a later edition can say which fifteen it means. The roster is versioned.

The distinction matters for the reader who knows the literature. A linguist will ask which theory the fifteen come from; the honest answer is none. They are an address set

chosen for comparability, and the theory explains why an address set is the right kind of object, not why this one has fifteen members.

Two consequences follow. First, a role assignment is not a claim about a universal grammar; it is a claim about this record's frame, and it can be wrong in the ordinary way. Second, a comparison between two systems is a comparison of rosters, not of languages; the systems can share a roster and disagree about a sentence, and the disagreement is the useful signal.

Fillmore's cases were not identical to the fifteen. The factitive case, the objective case and the dative case cut the space differently, and the differences are the reason the roster cannot be read off the history. The book uses the history to justify the shape and the closure, and it does not use it to justify the membership.

The reader who wants the full argument can read Fillmore, Jackendoff and Dowty; the book's obligation is to state what it borrowed and what it did not.

What closure costs

The closure is not free. A distinction that does not fit the fifteen becomes a sense rather than a role, and a sense carries more machinery: a definition, a microgloss, a namespace and a governance process.

Here's an example: a team wants a `CONSENT` role for a consent registry. The role does not fit the fifteen, and adding a sixteenth would make every existing parse incomparable. The repair is to model consent as a claim of its own, with its own predicate and its own record, rather than as a role in another claim.

The closure also has to be amended when a genuine gap appears. The amendment requires a fixture, a ruling and a version; the cost is the reason the set changes rarely.

The amendment is a governance act, not a schema change. It requires a fixture that the new role would fix, a ruling that the existing roles cannot fix it, and a version that every consumer can adopt. The cost is the reason the set is stable.

A change to the set is a migration for every consumer. The parser, the validator, the projections and the queries all assume the set; a new role is a new case in every one of them.

A set that changes without a migration forces every consumer to version its own copy, and the copies drift until two systems that claim to share a frame no longer do.

The fold test

A request for a sixteenth role is tested by folding. Take the proposed role and fold it into the nearest existing role; if the fold loses a distinction that a question depends on, the role is real; if the fold loses nothing, the role is a sense.

Here's an example: a proposed `CONSENT` role folds into `AGENT` for the consenting party and into `THEME` for the consented act. The fold loses the distinction between the

consent and the act, and the role is not added because the distinction belongs to a separate claim.

The test

Take one sentence and parse it into roles and dimensions. Check every role against the fifteen; check every dimension value against its set.

If the parser needs a role outside the set, the distinction belongs in a sense or in a separate claim, not in a new role. If the parser leaves a dimension unset, the claim is incomplete and the record should refuse it.

Three failures are common. A role outside the fifteen is accepted; a dimension is left unset; a parse assigns the same filler to two roles. Each failure is visible in the validator or in the comparison, and each is a defect in the frame rather than in the sentence. The validator refuses the parse.

A closed set does not make the comparison true. It makes the comparison possible, which is the property the record needs before it can argue about truth.

Then compare the parse with a second sentence that expresses the same claim. If the roles and dimensions match, the closure is doing its job. If they do not, the difference is either a real difference in the claim or a defect in the parser.

The frame is closed so that the vocabulary can stay open, and the closure is what makes a check possible without a new schema for every verb.

9 · What a Predication Does

A predication does something as well as says something. The doing is its force.

The content of *The valve is closed* and *Close the valve* is the same relation: a valve, a closed state. The first asserts; the second directs. A store that keeps only the content cannot tell whether the record is a fact to be checked or an instruction to be obeyed, and the difference determines who is accountable and what would count as a failure.

Force is a dimension of the act, not a tone of the sentence. Tone is how the speaker sounds; force is what the claim commits its holder to; a whispered assertion and a shouted assertion have the same force; a polite directive and a rude directive have the same force.

Nine forces

The force set is closed at nine. Each force carries a different commitment and a different defeat condition.

The set is closed for the same reason the role set is closed: comparability. A tenth force would be a new value in every gate, and the gates would need a migration before they could read it. The nine are the values the corpus's questions require, and a force outside them is a gap rather than a new dimension.

Force	What it commits	What defeats it
assertion	the holder to truth	contrary evidence
definition	the holder to a usage	a better definition or a counterexample
directive	the holder to an action	refusal, impossibility or withdrawn authority
commitment	the holder to a future act	failure to perform
question	the holder to an answer	an answer, or a false presupposition
hypothesis	the holder to a test	a failed test
attribution	the holder to a source	a retracted or unreliable source

Continued on next page

(Continued)

Force	What it commits	What defeats it
evaluation	the holder to a standard	a changed standard or contrary evidence
fiction	the holder to an imagined frame	exit from the frame or authorial retraction

Here's an example: *The valve must be closed before the pump starts* is a directive with a modal obligation. Its content is the valve's state; its force is directive; its defeat is not contrary evidence but a refusal or an impossibility; a store that records the content and omits the force will treat a refusal as a factual error.

Force changes warrant, not content

The same content can carry several forces, and the roles and dimensions other than force remain the same.

Here's an example: `CLOSE(theme=valve)` appears in four predications. *The valve is closed* asserts. *Close the valve* directs. *Is the valve closed?* asks. *Suppose the valve is closed* hypothesizes. The theme is the valve in all four; the force is different; the warrant is different.

The content equality is what makes the four comparable. A reviewer can see that the same relation is asserted in one place, directed in another and questioned in a third, and the comparison is the beginning of an audit.

A commitment makes the same point. *I will close the valve* has the content `CLOSE(theme=valve)` and commits the holder to a future act. The theme is the same; the force is commitment; the defeat is failure to perform. A store that records the content and the assertion treats a broken promise as a false statement, and the two are different failures.

The warrant difference is what makes the four non-interchangeable. An assertion is defeated by evidence; a directive is defeated by refusal; a question is answered or exposed as presupposing something false. A store that collapses the four into one force loses the defeat condition, and the loss is invisible until a challenge arrives.

Every force has a defeat condition

The defeat condition is the property that makes force operational. A force without a defeat condition cannot be audited, because nothing can show that the claim has failed.

The defeat table is the reference. An assertion is defeated by contrary evidence, a definition by a better definition or a counterexample, and a directive by refusal, impossibility or withdrawn authority. A commitment is defeated by failure to perform, a question by an answer or a false presupposition, and a hypothesis by a failed test. An

attribution is defeated by a retracted or unreliable source, an evaluation by a changed standard or contrary evidence, and a fiction by exit from the frame.

A force whose defeat condition is missing cannot be audited. The record can store the force, and nothing can show that it failed; the audit can only report that the label was present. The defeat condition is what makes the force a commitment.

Here's an example: an attribution says *Carter testified that she saw Cole leave*. The attribution is defeated if Carter retracts, if the record of her testimony is shown to be unreliable, or if the testimony was about a different Cole. The content, *Cole left*, may still be true; the attribution is what fails, and the record must be able to record the failure without deleting the content.

The defeat condition is not a truth condition. An assertion can be true and defeated by a source that is unreliable; a directive can be obeyed and defeated by an authority that was withdrawn. The record keeps the two separate because the audit asks different questions: was the claim true, and was the claim entitled to be made.

The distinction between defeating the content and defeating the force is the reason the two are stored separately. A store that records only the content cannot say which failed.

Why force is stored

Force can be inferred by a model, and inference is not storage. A model can propose a force from the sentence's mood, the surrounding text or the source's habits; the proposal is a guess, and the guess is wrong often enough to matter.

The force column is the commitment. A parse writes the force it assigned, the reviewer can see it, and a gate can refuse a directive recorded as an assertion. An inferred force has no such column; the inference is repeated at every read, and two readers can infer differently.

The audit question is concrete. A reviewer asks whether the operator was told to close the valve or whether the record says the valve was closed. The first is a directive and its failure is non-compliance; the second is an assertion and its failure is contrary evidence. The stored force is what lets the reviewer ask the right question.

Here's an example: a policy sentence *Operators must record the valve state* is a directive. A model that reads the modal *must* as a description records an assertion about operators, and the downstream gate treats a compliance failure as a factual error. The error is not in the model's language ability; it is in the absence of a stored force.

Force also determines which gates apply. A directive can be checked for compliance; an assertion can be checked for evidence; a question can be checked for answerability. A record without force cannot choose the gate, and a gate applied to the wrong force produces a false pass or a false fail.

The gate trace makes the loss visible. A false pass on a directive leaves the compliance failure unrecorded; a false fail on an assertion sends the reviewer after evidence that the claim never required.

Force is not mood

Mood is a property of the verb's form. Force is a property of the act. The two correlate and are not identical.

Close the valve is imperative in mood and directive in force. *The valve is closed* is indicative in mood and assertive in force. *If the valve were closed* is subjunctive in mood and hypothetical in force. *The valve is closed, isn't it?* is indicative in mood and a question in force.

Here's an example: *You will close the valve* is indicative in mood and can be a directive, a prediction or a commitment depending on the context. The mood does not decide the force; the act does.

The distinction matters because a parser that reads mood as force will misclassify the sentence, and the misclassification will be invisible in the parse. The force dimension is assigned from the act, not from the form.

Performatives make the distinction visible. *I promise to close the valve* is indicative in mood and commits the speaker by the act of saying it. *I apologize* is indicative and neither describes nor predicts; it performs. A parser that reads mood as force will classify both as assertions, and the commitment or the apology will be lost.

What force is not

Force is not tone, stance or sentiment. Tone is how the sentence sounds; stance is the speaker's attitude toward the content; sentiment is the emotional charge. A sentence can be polite and directive, neutral and assertive, angry and questioning.

Here's an example: *Could you close the valve?* is polite in tone and directive in force; the politeness does not make it a question about ability. A parser that maps politeness to questioning will misclassify the act.

Force and accountability

Force determines who is accountable. An assertion makes the holder accountable for truth; a directive makes the issuer accountable for the instruction and the receiver accountable for compliance; a question makes the asker accountable for the presupposition; a commitment makes the holder accountable for performance.

Here's an example: *The valve must be closed* in a procedure is a directive to the operator; in a design review it is an assertion about the required state; in a question it is a request for confirmation. The accountable party changes with the force, and the record must name it.

States and events

A predication can be eventive or stative, and the difference changes the hub.

An eventive predication has an event that occurs, with a time and usually a change: *The technician closed the valve*. A stative predication has a state that holds, with a valid time and no change: *The valve is closed*. The state is not the event's aftermath; it is a different predication with a different hub.

The copula is the common surface. *The valve is closed* stores as `BE(theme=valve, attribute=closed)`, not as `CLOSE(theme=valve)`. The first says the valve has the property; the second says someone closed it, and the agent slot is empty. Storing the state as an event invents an agent the sentence never gave.

Four uses of *be* resolve to four hubs: property (`BE`), existence (`EXIST`), auxiliary (`BREAK` in *was broken by the wind*), and appearance (`LOOK` in *looks hot*). The drop test separates the copula from the appearance: *Hot stove!* preserves *is hot*, and *looks hot* does not.

States matter for the record because their defeat conditions differ. A state is defeated by a change or by evidence that it never held; an event is defeated by evidence that it did not occur. A store that collapses the two cannot say which challenge applies.

Where the verb hides

English buries the predicate in a noun or a fragment, and the record has to recover it without inventing one.

Fragments are claims. *Hot stove!* stores as `BE(theme=stove, attribute=hot)` with the span preserved; *On the table.* stores as `BE(theme=keys, location=table)` when the left context supplies the theme. The recovered subject is a named decision, not extra characters in the source.

Nominalization is licensed by morphology and argument structure. *The destruction of the city* is `DESTROY(patient=city)`, not `BE(theme=city, attribute=destruction)`; the noun is deverbal and its complement behaves like the verb's object. *The development team* is not a development of anything, and forcing a verb there invents an event.

Light verbs are the open case. *Make a decision* has two defensible compiles: `MAKE(theme=decision)` preserves the noun, and `DECIDE(agent=...)` dissolves it. The aggressive compile is licensed only when the noun is deverbal and the arguments map without invention; otherwise the conservative card is the honest one.

The rule is the same in all three cases: recover only when morphology and argument structure license it; otherwise store the copular or nominal card and declare the gap. A guessed verb cannot be unwound, because nothing in the card records that the source said *a decision was made*.

A store that recovers a verb only when the morphology and the arguments license it can still answer the query, and it can explain why the answer is a gap when they do not.

The test

Take one content and write it under three forces: an assertion, a directive and a question. Parse each into roles and dimensions.

The roles and the dimensions other than force should match. If they do not, the content is not the same and the test is comparing different claims.

Then write the defeat condition for each. If the three defeat conditions are the same, the force is not being stored; if they differ, the force is doing its work.

Then ask which gate applies. An assertion is checked against evidence; a directive against compliance; a question against answerability. A record that cannot answer the gate question has a force it cannot use. The gate trace records the choice.

Three failures are common. The force is inferred and not stored; the force is stored but the defeat condition is copied from another force; the force is stored and the gate ignores it. Each failure is visible in the defeat table or in the gate trace, and each produces a record that looks complete and cannot be challenged.

A predication that does not record its force records only half of what it does.

10 · Stance and Testimony

Testimony is a chain of holders, not a property of the claim. A statement is made by someone, on some basis, at some time; each of those is a link, and the chain is what makes the statement challengeable.

A claim record with a single holder field is a flattening. It says the claim has an owner and loses the sequence that produced it: the witness who observed, the officer who recorded, the analyst who extracted, the loader who wrote the edge. The owner field names one of them, and the choice is arbitrary.

The arbitrariness is the defect. A field that must choose one holder from five links is a field that cannot represent the chain, and the choice is usually made by the loader rather than by the record.

Five grades of support

A link's support is not all of one kind. The corpus uses five grades, and the grade determines what a challenge must show.

Grade	Support	What defeats it
1	direct observation	a contradiction by a better observer
2	instrument measurement	calibration failure or instrument fault
3	report by a participant	the participant's retraction
4	inference from documents	a missing or contrary document
5	repetition or rumor	the absence of an originating link

The grades are ordered by distance from the event, not by reliability. A grade-2 measurement can be stronger than a grade-1 observation, and a grade-5 repetition can be accurate. The grade says what kind of challenge applies, not how much credit the link deserves.

An organization that stores the four fields but not the grade can still answer who and when, and it cannot answer what kind of challenge would defeat the link, which is the question an audit asks first.

The Carter chain

The chain is easiest to see in a case with a witness and a report.

Here's an example: Carter saw Cole leave the warehouse at 10:15. Carter testified to Detective Ruiz on 12 April. Ruiz wrote a report on 13 April. An analyst extracted the sentence on 20 April. A loader wrote the edge on 21 April. The claim `(Cole, left, warehouse)` now sits in a graph, and the graph has one holder field.

The chain has five links, and each link has a holder, a source, a basis and a date. Carter is the holder of the observation; her testimony is the source of Ruiz's report; the report is the basis of the analyst's extraction; the extraction is the source of the loader's edge. The chain is the testimony.

The chain matters because attribution is not a property of the content. The same content can be held by a witness, a reporter and a rumor, and the three have different defeat conditions. A store that records only the content has one defeat condition for all three, and the condition is wrong for two of them.

A chain can be long. The Carter chain has five links; a regulatory finding can have twenty. The length is not the problem; the problem is a chain that is not stored, because a long chain reconstructed from memory is a chain with gaps.

A chain that is reconstructed after the fact also has a different first holder, because the person doing the reconstruction becomes the holder of the reconstructed link.

Four fields

A testimony link has four fields: the holder, the source, the basis and the date.

The holder is who asserts. The source is what the holder relied on: observation, a document, another person, an instrument. The basis is the support the source provides: a span, a measurement, a prior link. The date is when the assertion was made.

Here's an example: Carter is the holder, her observation is the source, the scene is the basis, and 10:15 on 11 April is the date. Ruiz is the holder of the report, Carter's testimony is the source, the interview note is the basis, and 13 April is the date.

The four fields are the minimum. A link without a holder is anonymous; a link without a source is unsupported; a link without a basis is uncheckable; a link without a date is undated. Each absence is a different defect, and the record refuses the link rather than filling the field with a default.

The basis is the most often omitted field. A source can be named without a basis—*Carter said so*—and the link is uncheckable because nothing says where in the testimony the claim appears. The basis is the span, the measurement or the prior link that makes the source inspectable.

What flattening loses

A flattened fact keeps the content and drops the chain. The loss is not cosmetic; it removes the paths by which the claim can be challenged or retracted.

Here's an example: the graph says Carter left the warehouse. A reviewer asks whether Carter was the observer or the reporter. The graph cannot answer. A reviewer asks whether the report was written before or after the interview. The graph cannot answer. A reviewer asks what would retract the claim. The graph has no field for it.

The flattening also removes the distinction between a witness and a repetition. A person who saw the event and a person who repeated a rumor produce the same triple, and the graph cannot tell them apart. The distinction is the difference between evidence and hearsay, and it is lost at the first hop.

The legal distinction is useful here. A court separates the witness, the report and the repetition because each has a different challenge procedure; the record separates them for the same reason. The procedure is not the point; the point is that the challenge cannot be chosen without the link.

Retraction follows the chain

A retraction is a link, not a deletion. When Carter withdraws her testimony, the withdrawal is a new link that defeats the Carter link and leaves the report and the extraction intact. The content may remain in the graph with `stance=retracted`, and the audit can see the sequence.

Here's an example: Carter retracts on 30 April. The graph still holds `(Cole, left, warehouse)`, and the chain says the first link is defeated and the downstream links are unsupported. A store that deletes the edge loses the history; a store that keeps the chain can show what changed.

A retraction that deletes the edge instead of adding a link also deletes the evidence that the claim was once held, and the audit cannot tell whether the claim was withdrawn or never made.

Chain and graph

The chain is not a second graph. It is a set of links attached to the claim, and the claim is the node the graph traverses. The chain answers who; the graph answers reachable. The two structures share the claim key and nothing else.

Here's an example: a query for the route from batch to customer uses the graph; a query for the approver of a link uses the chain. A system that stores only the graph can answer the first and not the second.

The cost of flattening

The cost is paid at review. A reviewer who needs the chain reconstructs it from email, interview notes and the audit log, and the reconstruction is a new chain with its own holders. The reconstruction is not the testimony; it is a second testimony about the first.

The cost is also paid at correction. A correction to a flattened fact changes the content and leaves the holder field unchanged, so the correction is attributed to whoever wrote the row rather than to the person who made the claim.

The storage cost is small. A link is four fields plus a claim key; a chain of five links is five rows. The cost is not the rows; it is the discipline of writing them when the testimony is taken, because a chain written later is a reconstruction.

Stance is a dimension

Stance is the holder's relation to the content: asserted, doubted, reported, denied, assumed. It is a dimension of the claim, stored with the predication, not a separate record that can drift.

Here's an example: Carter's claim has `stance=asserted` because she testified to what she saw. Ruiz's report has `stance=reported` because he recorded her testimony. The analyst's extraction has `stance=attributed` because the extraction attributes the claim to the report. The three stances are different, and the chain carries all three.

The stance is not the force. Force is what the claim does; stance is how the holder stands to it. A report can assert with `stance=reported`; an assertion can be `stance=doubted` when the holder is not sure. The two dimensions vary independently, and a store that keeps one loses the other.

Stance values are closed: asserted, reported, doubted, denied, assumed, retracted. The set is small because stance is a dimension, not a vocabulary. A stance outside the six is a new dimension and requires the same amendment as a new role.

Here's an example: the same content under `stance=doubted` and `stance=denied` has different defeat conditions; a doubt is defeated by evidence that resolves it, a denial by evidence that contradicts it.

Testimony is not a kind of force. A testimony can assert, report or doubt; the force is the act, the testimony is the chain. The two are stored separately because a testimony can change its force without changing its chain, and a chain can grow without changing its force.

The chain table

The chain is stored as a table of links, each pointing to the claim and to the previous link.

Field	Carter link	Ruiz link
claim	(Cole, left, warehouse)	(Cole, left, warehouse)
holder	Carter	Ruiz
source	observation	Carter's testimony
basis	scene at 10:15	interview note
date	11 April	13 April
stance	asserted	reported

The table is small and the chain is explicit. A query can follow the links from the graph back to the witness, and each link can be challenged on its own fields.

A query follows the chain with one join per link. The query is longer than a holder lookup and shorter than a reconstruction from email; the difference is that the query returns the links rather than a narrative.

The test

Take a claim in the graph. Ask who holds it, what the holder relied on, what supports the source, and when the assertion was made.

If the graph answers with one holder and no source, the chain has been flattened. If the record answers with a chain, follow it to the first link and check whether the first holder is a witness or a repetition.

Then ask what would retract the claim. If the answer requires the chain, the chain is doing its work. If the answer requires only the content, the chain was never stored.

Then ask for the grade of the first link. A grade-1 observation and a grade-5 repetition can produce the same content and require different challenges. If the record cannot name the grade, the chain is incomplete even when it has four fields.

Three failures are common. The chain is flattened to one holder; the stance is stored without the source; the grade is recorded as a number with no challenge procedure. Each failure is visible in the chain table, and each produces a claim that can be repeated but not defended.

A claim without a chain can be repeated. It cannot be attributed, and the difference is the reason the chain is stored.

11 · Discourse Is a Linked Chain

A document is a sequence of linked claims, not a bag of claims; the links carry the argument: what supports what, what contrasts with what, what is conceded and what follows.

A bag of claims preserves the sentences and loses the argument. The same six claims in a different order can support a conclusion, undermine it or leave it undecided, and a store that keeps only the claims cannot tell which.

A document's argument is not in its sentences alone. The sentences carry claims; the links carry the argument, and the argument is what a reader reconstructs when deciding whether to accept the conclusion.

Why a document is not a bag

The bag is the default because storage is easier. A table of claims is simple; a link table is a second table with a foreign key and a type. The simplicity is real, and it costs the argument: a bag cannot say why a claim is present, and the why is what a reader reconstructs.

Here's an example: a bag of the notice's five claims can be sorted by date, by source or by confidence, and none of the sortings produces the notice. The notice's meaning includes the order and the links, and the bag has neither.

The notice

A recall notice is a short document with a visible argument.

Here's an example: a notice alleges a violation. Its claims are that the valve was closed, the pump started, the interlock did not engage, an inspection is required, and the requirement applies except where the OEM procedure governs. Five claims, and the notice's force depends on how they are linked.

The first claim is the finding. The second is the cause. The third is the mechanism that connects them. The fourth is a directive that follows from the finding. The fifth is a concession that limits the directive. A reader who receives the five claims as a list has the facts and not the notice.

Six links

The link set is closed at six. Each type does a different job, and the job determines what a challenge to the link must show.

Link	What it does	What it preserves
elaboration	adds detail to a claim	the claim's scope
contrast	sets two claims against each other	the difference
cause	makes one claim the reason for another	the direction of explanation
sequence	orders claims in time or procedure	the order
attribution	assigns a claim to a source	the holder
concession	limits a claim without defeating it	the exception

Here's an example: *The interlock did not engage because the valve was closed* is a cause link from the valve claim to the interlock claim. *The inspection is required, except where the OEM procedure governs* is a concession link from the directive to the exception. The two links have different defeat conditions: a cause is defeated by a better explanation, a concession by a wider exception.

The six are not rhetorical labels. They are the relations a reader uses to reconstruct the argument, and a store that records them can answer why a claim is in the document rather than only that it is.

The set is closed for the same reason the role set is closed. A seventh link would be a new relation in every reader and every query, and the six are the relations the corpus's arguments require. A link outside the six is a claim about the argument that has not been modeled.

Direction is the second property. A cause link runs from the reason to the consequence; a sequence link runs from the earlier to the later; an attribution runs from the source to the claim. A store that loses direction can reverse the argument without changing the rows.

The six, one by one

Elaboration adds detail without changing the claim's truth. A contrast sets two claims in opposition and preserves the difference; removing the contrast makes the document look consistent when it was not. Attribution assigns a claim to a holder and preserves the source; removing it makes the claim look like the document's own.

Here's an example: *The valve was closed, but the interlock did not engage* is a contrast; *according to Ruiz, the valve was closed* is an attribution. The first preserves a tension, the second preserves a holder, and a bag keeps neither.

Link direction

Direction is a field, not a convention. A cause link from A to B and a cause link from B to A are different claims about the explanation, and a store that infers direction from the sentence order will reverse the argument when the sentence is passive.

Here's an example: *The pump started because the valve was closed* and *The valve was closed because the pump started* have the same terms and opposite causes. The direction field distinguishes them.

Islands and staircases

The links form shapes, and the shapes determine what a removal does.

An island is a set of claims with no link between them. The claims are independent; removing one leaves the others intact. A list of findings is an island.

An island is not a failure. A list of independent findings is often the correct shape, and forcing links onto it invents an argument the document did not make.

A staircase is a chain in which each claim depends on the one before it. Removing a step breaks the argument above it. A proof is a staircase.

A staircase is fragile by design. Each step is load-bearing, and a missing step is visible because the conclusion no longer follows.

A Russian doll is a nesting in which a claim contains another claim, and the inner claim's scope is the outer claim's scope. An attribution inside an attribution is a doll: *Ruiz reported that Carter said she saw Cole leave* has a testimony inside a report, and the inner claim cannot be evaluated without the outer holder.

The doll is the hardest shape to flatten. A testimony inside a report has two holders and two scopes; a store that keeps one holder loses the inner testimony, and a store that keeps both without scope cannot say which claim the inner holder was making.

Here's an example: the notice is a staircase from the finding to the directive, with an island of supporting measurements and a doll for the OEM attribution. The three shapes coexist in one document, and a store that records only the links cannot tell which shape a removal will break.

What flattening loses

A bag of claims loses the argument and the defeat path.

Here's an example: the same five claims in the order directive, concession, finding, mechanism, cause read as a procedure with an unexplained basis. The claims are identical; the argument is not. A reviewer who asks why the inspection is required cannot answer from the bag, because the why was carried by the links.

A second document makes the same point with the same claims and a different link set. Reverse the cause link and the notice says the closed valve was caused by the failed

interlock; the claims are unchanged, the finding is inverted, and a bag cannot show the inversion.

The bag also loses defeat. A cause link is defeated by a better explanation; a concession by a wider exception; a sequence by a different order. Without the link, the record has one defeat condition for all five claims, and the condition is wrong for four of them.

The link record

A link is stored as a row with the source claim, the target claim, the type, the scope and the order.

Field	Meaning
source	the claim the link starts from
target	the claim the link reaches
type	one of the six
scope	the document region the link governs
order	the position in the sequence

Here's an example: the cause link has source `valve closed`, target `interlock did not engage`, type `cause`, scope `§2`, order `2`. The concession link has source `inspection required`, target `OEM exception`, type `concession`, scope `§4`, order `4`.

The scope field is what makes the doll work. A link inside a nested claim is scoped to the inner claim, and the scope prevents the link from being read as a claim about the outer document.

The order field is not decoration. Two sequence links with the same source and target but different order produce different procedures, and the order is part of the document's meaning.

The link table is small: one row per relation, five fields plus a key. The cost is not storage; it is the discipline of recording the link when the argument is made.

The three shapes coexist in one document, and the shape is a property of the link set, not of the headings. A section can contain an island, a staircase and a doll at once, and a store that reads the headings will misclassify the shape.

The document as a graph

The document is a graph of claims and links, and the graph is a projection of the record. The claims are nodes; the links are edges; the scope is a property of the edge. The projection makes the argument traversable, and it loses the link's defeat condition.

Here's an example: a query for the support of the directive follows the cause link to the finding and the finding to the measurement. The query returns the path and not the defeat condition, and the reader who needs the defeat reads the link record.

What the six do not cover

The six do not cover every relation a document can express. A definition, an example and a summary are not links; they are claims with their own force. The six cover the relations that carry an argument, and the others are claims the argument uses.

Here's an example: a definition in the notice is a claim with `force=definition`, not a link. The argument uses the definition; the definition does not relate two claims.

The test

Take one document and list its claims. Then list the links between them.

Remove one link and ask whether the argument survives. If it survives, the link was decoration. If it does not, the link was load-bearing, and the record must store it.

Then reconstruct the document from the claims and the links. If the reconstruction preserves the conclusion, the defeat paths and the scope, the link set is adequate. If it produces a different argument from the same claims, the link set is incomplete.

Three failures are common. The links are stored as prose connectives; the direction is lost; the scope is global when it should be local. Each failure produces a document that reads correctly and reconstructs incorrectly.

A document is a chain of claims. The chain is part of what the document means, and a bag of the same claims is a different document.

12 · Synapse Groups

A group is a purpose plus a membership, not a container. The purpose says what the group is for; the membership says which synapses serve it and in what role.

A group's identity is the purpose its members serve, and its contents can change without changing the group. A group has a purpose. The purpose is a claim.

What a group is not

A group is not a folder, a table or a document section. A folder groups by label; a table groups by schema; a section groups by position. Each can hold members and none carries a purpose. The purpose is what makes the group checkable, and the three containers are what a group is often confused with.

Why groups exist

Groups exist because claims are used in bundles. A finding is rarely used alone; it is used with the instruction it supports and the authority that makes the instruction effective. The bundle is the unit of use, and the group is the bundle's record.

A group's boundary is its membership, not its subject. Two groups can share a subject and have different boundaries; the boundary is what the purpose includes and excludes.

The maintenance pack

A maintenance pack is a group with a clear purpose.

Here's an example: the pack exists to return an aircraft to service. Its members are the finding that the valve was closed, the instruction to inspect the interlock, the parts list for the inspection, the sign-off that authorizes the return, and the exception for the OEM procedure. Five members, five roles, one purpose.

The finding is the reason the pack exists. The instruction is what the pack directs. The parts list is what the instruction requires. The sign-off is what makes the pack effective. The exception is what limits it. Each member's role is named by the purpose, not by its position in the document.

Group role	What it contributes
finding	the reason the group exists
instruction	what the group directs
evidence	what supports the finding
authority	what makes the group effective
exception	what limits the group

A group can have more than one member in a role; the role is a capacity, not a single slot.

Here's a second example: a sign-off is a member with `group_role=authority` in the maintenance pack and a member with `group_role=evidence` in an audit group. The same synapse plays two roles, and the group role is what distinguishes them.

The figure reads the pack three ways at once: its five roles, the nested inspection group that inherits its scope, and the sign-off that belongs to two groups without belonging to either twice.

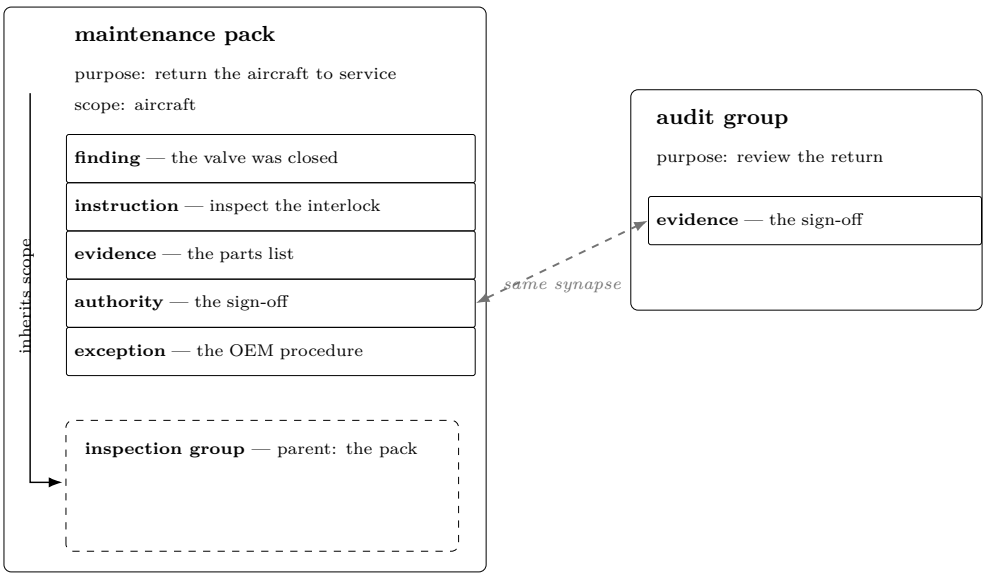


Figure 2: The pack's five members with their group roles, a nested inspection group inheriting the pack's scope, and one sign-off held in two groups in different roles.

Purpose and membership

A group's purpose is a claim about what the group is for. The purpose is not a label; it is a statement that can be wrong. A pack that says it returns the aircraft to service and does not authorize the return has the wrong purpose or the wrong membership.

The purpose is also not the title. A title is a name; a purpose is a claim with a truth value. A group whose title says *inspection* and whose purpose is *return to service* is a different group from one whose purpose is *schedule the inspection*, and the difference is in the purpose.

The purpose is governed like a claim. It has a holder, a date and a defeat condition; a group whose purpose is withdrawn is a group whose members are unsupported.

The membership is a set of role bindings. Each binding names a synapse and the role it plays in the purpose. The roles are group roles, not predicate roles: finding, instruction, evidence, authority, exception.

Here's an example: the sign-off is a member with `group_role=authority`; the finding is a member with `group_role=finding`. The same synapse can be a finding in one group and evidence in another, and the group role is what distinguishes the two.

Membership is by role, not proximity

Two synapses that appear next to each other are not a group. Proximity is a property of the document; membership is a property of the purpose.

Here's an example: a paragraph contains a finding and an unrelated note about the weather. The two are adjacent and not members of the same group. A store that groups by paragraph will put them together, and the group will have a member with no role in its purpose.

The role test is the membership test. If a member's role cannot be named, the member does not belong to the group, and the group's purpose is either incomplete or wrong.

The role test is also a boundary test. A member whose role is *context* is a member the purpose does not need; a member whose role is *authority* is a member the purpose cannot work without. The distinction is what keeps the group small. A group has a boundary.

Islands

An island is a group with no external link. The group has members and a purpose, and nothing outside it refers to it or is referred to by it.

An island is a defect, not a shape. A group that nothing can reach cannot be used, and a group that reaches nothing cannot support a decision. The defect is detected by asking which groups link to this one and which groups it links to.

Islands are detected by a query, not by a diagram. The query asks for groups with no inbound and no outbound link; the result is the island list, and each entry is a defect to repair or retire. The query returns a list.

Islands come in two kinds. A source island has no inbound link; a sink island has no outbound link. The first cannot be found; the second cannot be used.

Here's an example: a maintenance pack with no link to the aircraft it services is an island. The pack may be complete in itself, and the reader cannot tell which aircraft it applies to. The missing link is the defect.

The repair is a link, not a rename. A link to the aircraft, the site or the program gives the group a scope and a route; a rename gives it a better label and no route.

Nesting is scope

Groups nest, and the nesting is a scope, not a folder. A nested group inherits the scope of its parent; a folder does not.

Here's an example: an inspection group nested inside the maintenance pack inherits the pack's aircraft and date. A folder named *inspections* does not; the folder groups by label, and the label does not carry the aircraft or the date.

The difference matters when the inner group is queried. The nested group can answer which aircraft it applies to without repeating the aircraft in every member; the folder cannot, and the repetition is where the errors enter.

Scope inheritance is the reason a nested group can be small. The inner group carries only what is specific to it; the parent carries the aircraft, the date and the authority. A flat group that repeats the parent's fields in every member is larger and more error-prone.

A group's scope is not its subject. Two groups can share a subject and inherit different scopes; the scope is the parent's, and the subject is the group's own.

The group table

A group is stored as a row with a purpose, a parent, a scope and a membership.

Field	Meaning
group_id	the group's key
purpose	the claim about what the group is for
parent	the group that contains it, if any
scope	the inherited region or subject
member	a synapse and its group role

Here's an example: the maintenance pack has `purpose=return aircraft to service`, `parent=none`, `scope=aircraft`, and five member rows. The inspection group has `parent=maintenance pack` and inherits `scope=aircraft`.

The membership is a join table, not a list in the group row. One row per member, one role per row, and the group key on every row. The join is what makes the membership queryable; a list in a column is what makes it unqueryable.

Groups and documents

A document is a group with a purpose and a link set. The group's purpose is the document's argument; the membership is the claims; the links are the relations among them. The document is not a container of groups; it is a group whose members are claims and whose subgroups are sections.

Here's an example: the notice is a group whose purpose is to require an inspection, whose members are the finding, the instruction and the exception, and whose links are the cause and the concession.

A document that is stored as a group can answer which claims belong to its argument, and it cannot answer which groups its argument depends on, which is the question a reader asks when the document is challenged.

Group lifecycle

A group has a lifecycle: proposed, active, superseded, retired. The lifecycle is a claim about the group's status, with a holder and a date, not a column that changes silently. A retired group remains queryable; a deleted group does not. A group has a lifecycle.

The lifecycle is a claim, not a status column. A status column changes in place and keeps no holder; the lifecycle claim keeps the holder and the date. The date is part of it.

The cost of groups

Groups cost a table and a join. The cost is small compared with the cost of a document that cannot answer why a claim is present, and the join is the price of the answer.

A group that is stored as a folder can still be found by name, and it cannot answer which claims belong to its purpose, which is the question the audit asks. The join is a table.

Group queries

Three queries justify the group: what is in it, what role each member plays, and what it links to. A group that cannot answer the three is a container with a purpose-shaped name.

An organization that stores groups as folders can still retrieve documents by name, and it cannot answer which claims belong to a purpose, which is the question an audit asks of a maintenance pack.

The test

Take one group. Ask what it is for, which synapses are members, and what role each member plays.

If a member's role cannot be named, the member does not belong. If the purpose cannot be stated as a claim, the group is a container with a title.

Then ask which groups link to this one and which it links to. If the answer is none in both directions, the group is an island, and the island is a defect.

Then remove the group and ask whether any claim's meaning changes. If nothing changes, the group was decoration; if something changes, the group is part of the claim's meaning.

Three failures are common. The group is a folder with a purpose-shaped title; the members are grouped by proximity; the nesting is a folder rather than a scope. Each failure produces a group that looks organized and cannot be queried.

A group is a purpose with members, and the purpose is what makes the membership checkable.

13 · How Groups Compose

Groups compose by three joins: nest, contain and relate. A fourth is a design error. Lifecycle is a link, not a join.

The three joins are the grammar of composition. Each changes a different property: nest changes scope, contain changes membership, relate changes reference. A join that does none of the three is a line with a label, and the label is where the ambiguity enters.

Composition is claim-making

A layout arranges; a composition asserts. The difference is the warrant: a layout has no holder, and a join does. Two groups placed side by side are a layout; two groups joined by a relate are a claim that the relation holds, and the claim can be challenged.

Here's an example: a folder called *inspections* contains the inspection group because someone filed it there. The folder's arrangement asserts nothing; the containment join asserts that the inspection is a member of the maintenance pack with the role *instruction*; the first is storage; the second is a claim.

A layout can be rearranged without changing any claim, and a composition cannot be rearranged without changing which group is part of which, which is why the join needs a warrant and a date. A join row is small.

What composition is for

Composition answers three questions: what is in this group, what is it part of, and what does it refer to. The three joins are the three answers, and a composition that cannot answer one of them is incomplete.

The group's purpose determines which joins are permitted. A member whose role is not in the purpose cannot be contained; a child whose scope is not the parent's cannot be nested. The purpose is the filter on composition.

The three joins

Join	Property it changes	Direction	What removal breaks	Defeat condition	What the challenge targets
NEST	scope	the parent's scope flows down, never up	the inner group loses its inherited scope and no longer knows which aircraft it applies to	evidence that the parent's scope does not cover the child	the parent's scope
CONTAIN	membership	a member belongs to a group, not the reverse	the member leaves the group and no longer has a role in the purpose	evidence that the member's role does not belong to the purpose	the member's role
RELATE	reference	symmetric only when the link type says so; requires is not required_by reversed	the two groups remain independently meaningful, and the argument between them is gone	evidence that the relation does not hold	the link's meaning
lifecycle, not a join	none of the three	carried as a link with a warrant and a date	nothing structural; the record is older or newer	evidence that the state change was not warranted	the warrant

NEST is the join that lets an inner group inherit the parent's subject, date and authority without repeating them. CONTAIN is the join that binds a member to a group with a role in the group's purpose. RELATE is the join that connects two independently meaningful groups or claims without subordinating either.

Here's an example: the inspection group is nested in the maintenance pack, so it inherits the aircraft and the date. The valve finding is contained in the inspection group with `group_role=finding`. The inspection group relates to the parts list by a `requires` link. The three joins do three different jobs, and a store that uses one join for all three cannot say which job it did.

The direction of a join

NEST and CONTAIN have a direction. A group nested in a parent inherits the parent's scope; the parent does not inherit the child's. A member contained in a group has a role in the group's purpose; the group does not have a role in the member. RELATE is

symmetric only when the link type says so; a `requires` link is not a `required_by` link with the arrow reversed.

Here's an example: reversing the containment of the valve finding and the inspection group produces a group that is a member of a claim, which is not a composition the grammar permits. The direction is part of the type.

A join is a claim

A join is a claim about the relation, and a claim needs a warrant. The warrant names who asserted the relation, on what basis, and when.

A line on a diagram has no warrant. A join row does: `join_id`, `from`, `to`, `join_type`, `warrant`, `date` and `scope`. The row can be challenged on its warrant, and the challenge is different for each type: a nest is challenged on the parent's scope, a contain on the member's role, a relate on the link's meaning.

Here's an example: the containment of the valve finding in the inspection group has a warrant from the notice's structure and a date from the compile. A reviewer who says the finding belongs to a different group is challenging the warrant, not the row's existence.

Relate and the link vocabulary

RELATE carries the six discourse link types—elaboration, contrast, cause, sequence, attribution, concession—and any other relation the vocabulary admits. The join says only that a relation holds; the link type says which one.

Here's an example: the inspection group relates to the parts list by `requires`; the notice relates the finding to the directive by `cause`. Both are RELATE joins, and the link types are different.

Warrant and defeat

Each join has a different defeat condition. A nest is defeated by evidence that the parent's scope does not cover the child. A contain is defeated by evidence that the member's role does not belong to the purpose. A relate is defeated by evidence that the relation does not hold.

The defeat table is short, and it is what makes the join auditable. A join without a defeat condition is a line with a type.

Why not four

A fourth join is tempting because lifecycle looks like composition: a group is proposed, active, superseded or retired, and the stages seem to relate groups.

The lifecycle is a claim about a group's status over time, and it is carried as a link with a warrant and a date. It does not change scope, membership or reference, so it

is not a join. Making it a fourth join would mean a schema migration for every future relation that is temporal, and the migration would buy nothing the link row does not already carry.

The rule is the one the grammar needs: if a structure seems to need a fourth join, the model is wrong; if it needs another relation, add a link type, not a join. A new link type is a row. A new join is a schema migration, and a schema migration is a cost every consumer pays.

Each join changes something different

The three joins are distinguishable by what a removal does.

Remove a NEST and the inner group loses its inherited scope; the group still exists, and it no longer knows which aircraft it applies to.

Remove a CONTAIN and the member leaves the group; the claim still exists, and it no longer has a role in the purpose.

Remove a RELATE and the two groups remain independently meaningful; the argument between them is gone, and neither group changes.

The removal test is the type test. A join whose removal changes nothing is decoration; a join whose removal changes scope, membership or reference is load-bearing, and the change names its type.

Nesting is not containment

Nesting and containment are often confused because both put one thing inside another. Nesting changes scope; containment changes membership. An inner group can be nested without being a member, and a member can be contained without being nested.

Here's an example: the inspection group is nested in the maintenance pack and contains the valve finding. The group is not a member of the pack; it is a sub-scope. The finding is not nested in the group; it is a member.

The join table

A join is stored as a row, not as a foreign key on either end.

Field	Meaning
join_id	the join's key
from	the source group or claim
to	the target group or claim

Continued on next page

(Continued)

Field	Meaning
join_type	nest, contain or relate
warrant	who asserted the relation, on what basis
date	when the relation was asserted
scope	the region the relation governs

The join table is the composition ledger. A group's members are not a list in the group row; they are rows in the join table, and the same is true of its parent and its relations.

A join costs a row and a warrant. The row is cheap; the warrant is the work. A team that adds a join without a warrant has added a line, and the line will be challenged at the first audit.

The cost is lower than a fourth join, which costs a schema migration for every consumer.

The join key is not the pair of endpoints. Two groups can be joined by a contain and a relate at the same time, and the two joins are different claims. The key identifies the claim, not the connection.

Composition and documents

A document is a group composed by nest and contain. The sections nest; the claims contain; the links between sections are relate. A document that stores only the section order has the nest and not the contain, and it cannot answer which claim belongs to which argument.

The wrong type

A join with the wrong type is worse than a missing join. A contain recorded as a nest inherits scope it should not have; a relate recorded as a contain makes one group a member of another and loses its independence. The wrong type is invisible in a diagram and visible in a query.

The purpose filter

The purpose filters composition. A member with no role in the purpose is refused; a child with a scope outside the parent is refused. The refusal is the composition's quality control.

Here's a worked example: the notice composes from a finding, a directive, an exception and a parts list. The finding is contained with `group_role=finding`; the directive with `group_role=instruction`; the exception with `group_role=exception`; the parts list

relates by `requires`. The notice nests inside the program group. The composition has three contains, one relate and one nest, and each answers a different question. The row carries the warrant, the date and the type.

The test

Take one composition and name its joins. For each join, ask which property it changes: scope, membership or reference.

If a join changes none of the three, it is decoration or a lifecycle claim; it is not a join. If a composition needs a fourth property, the model is missing a link type, not a join.

Then remove the join and ask what breaks. If scope breaks, it is a nest; if membership breaks, it is a contain; if reference breaks, it is a relate. If nothing breaks, the join was a line.

Three failures are common. The join is stored as a foreign key on one end; the direction is inferred from the column name; the lifecycle is modeled as a fourth join. Each failure produces a composition that reads correctly and cannot answer which property it changed.

A composition that cannot say which property a join changed can still be rendered as a diagram, and it cannot be audited when the diagram is challenged by a reviewer who asks what the line means.

The three joins are the grammar, and every join is a claim that can be challenged, defeated and dated.

14 · When Not to Assert

A claim can fail without being false. Its presupposition can fail, or its implicature can be canceled, and in both cases the sentence is not a lie and not a truth.

The record has three options: store the claim, store the inference, or store the gap. Smoothing the first two into an assertion produces false certainty, and false certainty is worse than a declared gap because it cannot be challenged.

Presupposition

A presupposition is content the sentence takes for granted. It is not asserted, and it survives negation and questioning.

Here's an example: *The valve was closed again* presupposes that the valve was closed before. The assertion is the second closure; the presupposition is the first. *The valve was not closed again* still presupposes the first closure, and *Was the valve closed again?* still presupposes it. The presupposition is backgrounded, and the assertion is foregrounded.

The test is negation. If the content survives when the sentence is negated, it is a presupposition. If it disappears, it is part of the assertion.

The test extends to questioning and modality. *Was the valve closed again?* and *The valve must have been closed again* both keep the prior closure; the presupposition projects out of the question and the modal. The projection is the reason the background cannot be canceled by embedding the sentence in a larger one.

Triggers

Presuppositions come from triggers, and the trigger is part of the record.

Trigger	Example	Presupposition
again	closed again	it was closed before
stop	stopped approving	it was approving before
know	the auditor knows	the proposition is true
regret	regrets the delay	the delay occurred

Continued on next page

(Continued)

Trigger	Example	Presupposition
definite article	the valve	a unique valve is identifiable
factive verb	discovered the leak	the leak existed

Here's an example: *The auditor stopped approving orders* presupposes that the auditor was approving orders and asserts that the approving ended. A store that records only the ending loses the prior activity, and a query for who approved the orders before the stop cannot find it.

The trigger list is not closed. The table names the common triggers; a new trigger enters when a fixture shows that it projects under negation. The test is the same for every candidate.

When a presupposition fails

A presupposition fails when the backgrounded content is false or unidentifiable. The sentence is then neither true nor false; it is unusable.

Here's an example: the record says *the valve was closed again*, and the history says the valve was never closed before. The sentence is not false; the word *again* has nothing to attach to. A store that treats the sentence as an assertion of a second closure invents a first closure, and the invention is invisible because the sentence reads normally.

Definite descriptions fail in the same way. *The valve* presupposes a unique identifiable valve; if the document has two valves and no antecedent, the description is not false, it is ambiguous. The gap records the failed uniqueness, and the reviewer resolves it or refuses the claim.

The failure is a gap, not a correction. The record stores the trigger, the presupposed content, the span and the status; the reviewer sees that the background could not be established.

Implicature

An implicature is content inferred from the act of saying, not stated in the sentence. It is cancellable.

Here's an example: *Some valves were closed* implicates that not all were closed. The implicature is not part of the assertion; it can be canceled without contradiction: *Some valves were closed, and in fact all were*. The sentence remains true when the implicature is canceled, which is what distinguishes an implicature from an assertion.

The test is cancellation. If the content can be denied without contradicting the sentence, it is an implicature. If denying it contradicts the sentence, it is an assertion.

Scalar implicature

Scalar implicature is the common case. A weaker term implicates that the stronger term does not hold: *some* implicates *not all*; *warm* implicates *not hot*; *adequate* implicates *not good*.

Here's an example: a report says *the inspection was adequate*. The report implicates that the inspection was not good, and the implicature can be canceled by adding *and in fact it was excellent*. A store that records *adequate* and *not good* as one claim cannot represent the cancellation.

The cancellation is evidence. A speaker who says *some, and in fact all* has canceled the implicature; the record keeps both the term and the cancellation, and a later reader can see that the stronger claim was made explicitly.

The record keeps the term and the inference separately. The term is the claim; the inference is a note with a warrant and a cancellation condition.

Implicature depends on context. *Some* implicates *not all* when the speaker is assumed to be cooperative and to know the stronger term; in a context where the speaker is explicitly uncertain, the implicature weakens. The record stores the inference with its warrant, and the warrant names the assumption.

Grice's cooperative principle explains why implicature is inference rather than decoration: a speaker who says *some* and knows *all* is assumed to be giving the strongest relevant information. The record does not store the maxims; it stores the inference and the assumption that warrants it.

Why smoothing lies

Smoothing is the choice to store the presupposition or the implicature as an assertion. It produces a record that reads better and cannot be defended.

Here's an example: the sentence *the valve was closed again* is smoothed to two assertions: the valve was closed before, and the valve was closed now. The first assertion has no span; it was never said. A reviewer who asks for the evidence of the first closure finds the sentence, and the sentence only presupposes it.

The cost appears at review. A reviewer who challenges the first closure is told that the record asserts it and cannot produce the span; the challenge then turns on the record's own authority rather than on the source. The smoothing has moved the claim from the document to the database, and the database cannot defend it.

The smoothed record also loses the defeat path. A presupposition failure is defeated by evidence that the background does not hold; an assertion is defeated by contrary evidence. The two challenges are different, and smoothing chooses the wrong one.

Presupposition and assertion

The distinction is not academic. An assertion is foregrounded and challengeable by evidence; a presupposition is backgrounded and challengeable by the background. A record that stores both as assertions gives the reviewer one challenge where two are needed.

Projection

Presuppositions project. They survive negation, questioning, modality and conditionals; they do not survive every report of someone else's belief. *Carter believes the valve was closed again* may presuppose the prior closure or may only report Carter's belief, and the difference is a scope question the record marks.

The cost of resolving

The compiler does not resolve a presupposition by looking harder. The background may be in another document, and the search is a separate claim with its own warrant. The gap stays open until a claim fills it.

The gap table

A gap is stored as a row with a type, a trigger, a span and a status.

Field	Meaning
gap_type	presupposition, implicature, predicate, anaphora, scope
trigger	the word or construction that produced it
content	what is presumed or inferred
span	where the trigger occurs
status	open, resolved, refused

Here's an example: the *again* in the valve sentence produces a presupposition gap with `content=prior closure`, `span=§3`, `status=open`. The *some* in the report produces an implicature gap with `content=not all`, `status=open`.

The gap is not an error. It is the honest representation of what the sentence does not settle, and it is queryable: a reviewer can ask for every open presupposition in a document and see where the argument depends on a background it never established. The gap is a row.

The status vocabulary is small: open, resolved, refused. Open means the gap is visible and unresolved; resolved means a later claim filled it and the fill is linked; refused

means the compiler declined to guess and the claim was not stored. The three statuses distinguish a gap from an error. The gap is a row.

A gap table that is never queried is a confession without an audit, and the record needs both the confession and the query to be honest about what it does not know.

Gap queries

Three queries use the gap table: open gaps by document, gaps by trigger, and conclusions whose support includes an open gap. The third is the audit query, and it finds an argument resting on an unestablished background.

The record's honesty

The record can be honest without being complete. A gap says what the compiler could not settle; it does not pretend the sentence was never written. The distinction is the difference between a partial record and a false one.

What the gap buys

The gap buys a query. A reviewer can ask for every open presupposition in a document and see which conclusions depend on a background that was never established. The query is the audit of the document's assumptions, and it is only possible because the gaps are rows.

The cost of coverage

A compiler that maximizes coverage will smooth gaps, because a smoothed claim is countable and a gap is not. The metric rewards the wrong behavior unless the gap count is reported beside the coverage count.

The test

Take a sentence with *again*, *stop*, *some* or a definite article.

Negate it and ask whether the backgrounded content survives. If it survives, it is a presupposition; record the trigger and the content.

Cancel the inference and ask whether the sentence still holds. If it holds, it is an implicature; record the term and the cancellation.

Then ask whether the record asserts the background. If it does, the record has smoothed a gap into a claim. If it stores the gap, the record can be challenged at the right place.

Three failures are common. The presupposition is stored as an assertion; the implicature is stored as a term; the gap is stored without a trigger. Each failure produces a record that reads smoothly and cannot answer why the background is there.

A sentence that presupposes a prior closure can be true, false or unusable depending on a history the sentence does not contain, and the record's job is to keep that history visible rather than to choose one of the three on the reader's behalf.

A compiler that stores its gaps can be audited at the point where it did not know; a compiler that smooths them can be audited only after the error has become a fact.

PART III
The Home

where it lives, and how two intakes reach it.

15 · The Universal Meaning Substrate

The UMS is a ledger of claims, not a lake of data. A lake stores files for later interpretation. A ledger stores entries with keys, spans, order and corrections, and each entry is answerable on its own.

The distinction is the first design decision. A lake is judged by what it can hold; a ledger is judged by what it can answer. The UMS is built for the second, and the schema follows from the questions it must answer.

The empty quadrant

Two axes describe the existing systems.

The first axis is whether the predicate is open. A relational table fixes its verb in the schema; a triple store admits a new predicate as a new term. The second axis is whether the commitment is surfaced. A row stores a value; a claim record stores who asserted it, on what basis, under what authority, and what would defeat it.

	Commitment unsurfaced	**Commitment surfaced**
Predicate closed	the table row	an annotated ontology
Predicate open	the triple	**the UMS**

The UMS occupies the quadrant that the other three leave empty: the predicate is open, so a new verb enters without a schema change, and the commitment is surfaced, so the holder, force, time and defeat are stored with the claim.

Here's an example: `orders(customer, product, date)` is closed and unsurfaced; `(order44021, hasCustomer, customer847)` is open and unsurfaced; an ontology class with an annotation is closed and surfaced; a synapse with a claim row is open and surfaced. The fourth is the only one that can answer *who says so* without a join to a separate provenance table.

The quadrant matters because each of the other three can answer a question the UMS cannot, and the UMS can answer a question none of the other three can. A table is faster for state; a triple is cheaper for interchange; an annotated ontology is stronger for classification. The UMS is stronger for attribution, and attribution is the question an audit asks.

The ledger

A ledger is append-only. A correction is a new entry that supersedes an old one, and the old entry remains readable. The UMS applies the rule to claims: a claim is never updated in place, and a supersession row records what replaced what and why.

The ledger has twelve tables. Each table has a grain, a key and a purpose.

Table	Grain	Key	Purpose
document	one source version	document_id	what was read, and when
span	one character range	span_id	where a claim came from
sense	one meaning	sense_id	lemma, POS, microgloss, namespace
entity	one referent	entity_id	a thing a claim can be about
claim	one assertion	claim_id	holder, force, polarity, time, status
predication	one verb frame	predication_id	verb sense and dimensions
role_binding	one participant	binding_id	which role, filled by what
join_claim	one typed link	join_id	nest, contain or relate
group	one purpose	group_id	a purpose with a scope
group_ member	one membership	member_id	member and role in the purpose
authority	one grant	grant_id	who may do what, until when
supersession	one correction	supersession_id	what replaced what, and why

The table is the schema contract. The DDL is a projection of it, and the projection can change without changing the contract.

The twelve tables are not a suggestion. Each is the minimum for one of the six questions: document and span for the words, sense and entity for the terms, claim and predication for the assertion, role binding for the participants, join and group for the composition, authority for the standing, supersession for the correction.

The claim row

A claim row carries `claim_id`, holder, force, polarity, modality, valid time, status and span. The predication row carries the sense and dimensions. The binding rows carry the roles.

The split keeps the claim one row and the participants many. A claim with ten participants is ten bindings and one claim, not ten claims; a correction to one participant changes one binding and leaves the claim intact.

Why 3NF

The kernel is in third normal form because each fact has one home. A claim's holder is in the claim row, not repeated on every role binding; a predicate's sense is in the sense table, not copied into every predication. An authority's window is in the authority row, not restated on every claim it governs.

The normalization is not an aesthetic. It is what makes a correction confined. A correction to a holder changes one claim row and leaves the roles intact; a correction to a sense changes one sense row and every predication that uses it, which is the intended scope.

Here's an example: an approval claim and its three role bindings. The holder sits on the claim row; the roles sit on the bindings. Changing the holder changes one row. If the holder were copied onto each binding, the correction would be three updates and a chance to miss one.

The grain

The grain is the claim, not the document and not the row. A document is many claims; a row is one claim with a fixed verb. The grain is what makes the ledger countable: one claim, one row, one correction.

The claim key

The claim key is generated at write time and never reused. A reused key would merge two claims, and a merged claim cannot be superseded cleanly. The key is opaque; the sense, the holder and the span are the readable fields.

The correction

A correction is a new claim, not a changed value. The new claim carries the corrected content and a new span; the supersession names the old claim, the new claim and the reason. The old claim remains queryable, and a query for the state as of a date follows the chain to the head at that date.

Supersession

Supersession is the only update the ledger permits, and it is not an update. The old claim remains; a new claim carries the corrected content; a supersession row names both and the reason. A query for the current state follows the supersession chain to the head; a query for history stops at any point.

The span

A span is a character range in a source version, and it is the claim's evidence. A claim without a span is not refused because the schema is strict; it is refused because the claim cannot be checked. The span is also what makes rehydration possible: the compiler can re-slice the source and compare.

The sense

A sense is a meaning, not a word. The sense row carries lemma, part of speech, microgloss and namespace; the canonical ID is built from them. Two senses may share a lemma and must not share a microgloss in the same namespace, which is the rule that keeps a dictionary from collapsing two meanings into one.

The authority

An authority is a grant with a window: who may do what, until when. The claim's authority field points to the grant, and the grant's window is checked at read time. A claim whose grant has expired is not deleted; it is defeated, and the defeat is a row.

The invariants

The invariants are what DDL cannot express, and they are the part an implementer must test.

- Rows are append-only; a correction is a supersession, not an update.
- Every claim has a span; a claim without a span is refused.
- Every predication has a force and a sense; neither is inferred at read time.
- Every role is in the closed set; a role outside the set is refused.
- Every sense has one microgloss per namespace; siblings may not share one.
- A supersession names the claim it replaces and the claim that replaces it.

The invariants are the acceptance tests for the schema. A schema that accepts a claim with no span is not the UMS, whatever its tables are called.

What the UMS is not

It is not a data lake: a lake has files and no claim grain. It is not a graph store: a graph has edges and no holder. It is not an ontology: an ontology has classes and no defeat condition. It is not an API: an API is a projection of the ledger for one consumer.

The negative list matters because each of the four can be mistaken for the UMS when a team is buying infrastructure. The UMS is the ledger those systems project from, and the projection is declared.

Each of the four has a legitimate use. A lake is right for raw corpora; a graph is right for reachability; an ontology is right for classification; an API is right for one consumer. The UMS is right for the claim that all four project from, and it is not a replacement for any of them.

A lake is optimized for storage; a ledger is optimized for correction. The difference appears when a claim is challenged: a lake returns the file, and a ledger returns the claim, its span, its holder and its supersession chain.

The projection

The graph, the report and the API are projections. Each is built from the ledger, and each declares its loss. A projection that is not built from the ledger is a second record, and a second record drifts.

The six questions

The six questions are the ledger's acceptance criteria. A schema that answers five is a schema with a missing field; a schema that answers four is a schema with a missing table. The count is the measure.

The build sheet

The UMS build sheet has seven fields: purpose, interface, invariants, failure statuses, golden pairs, acceptance tests and non-goals. The invariants are above; Appendix E gives the rest. The sheet is what an implementer codes against, and the code is a projection.

The test

Take one claim. Ask the six questions: who holds it, in what words, with what force, valid as of when, under what standing, and what would defeat it.

If the ledger answers all six from one claim row and its span, the schema is doing its job. If an answer requires a join to a provenance table that is not part of the ledger, the commitment is not surfaced.

Then supersede the claim. The old row must remain readable, the new row must carry the correction, and the supersession row must name both. If the old row is overwritten, the ledger is a table with a history habit, not a ledger.

Then project the claim into a graph and a report. Each projection must declare what it drops. A projection that claims to keep everything is a copy, and a copy that is not the ledger will drift.

Three failures are common. The claim is stored as a row with a status column; the correction overwrites the old row; the projection is declared to keep everything. Each failure produces a ledger that looks complete and cannot answer the six questions after a correction.

An organization that stores claims as rows can answer the current state quickly, and it cannot answer who asserted the state, which is the question that arrives when the state is challenged.

A ledger is judged by the questions it can answer after the correction, not by the volume it can hold.

16 · The Prose Compiler

A prose compiler is a chain of contracts, not a chain of model calls; each stage answers one kind of question, emits an artifact, and hands that artifact to the next stage; the model proposes inside a stage; the program validates and commits.

The distinction is operational. A model call that returns prose cannot be checked at a boundary, because no artifact sits between the question and the answer. A model call that returns a constrained record can be validated against a schema, a closed role set and a span, and the validator's refusal is a named status rather than a silent repair.

Why stages

A single call can be simpler, and simplicity is a real cost. The staged compiler costs more code, more storage and more review; the cost is paid for three properties: a defect can be located, a stage can be replayed, and a refusal can be named. A single call has none of the three, because it has no boundary at which to stand.

One step, one transformation

The first rule is that a stage does one thing. A stage that both extracts and resolves cannot be debugged, because a defect in either half looks the same from outside.

The rule has a consequence: the stage order is semantic, not a workflow preference. A pronoun cannot be resolved before an alias map exists. A claim cannot be checked for a lost predicate before the spans are frozen. A role cannot be validated before the card exists to hold it.

The stages

The compiler has more stages than a diagram needs, and the count is less important than the division of questions.

Stage	The question	What it leaves behind
ingest	is the store the right shape, and what is markup?	a database that refuses a wrong-shape rebuild

Continued on next page

(Continued)

Stage	The question	What it leaves behind
span	where in the frozen bytes does everything sit?	spans, structure, a kept-or-dropped decision per span
reference	do these surfaces name one referent, and what does this pronoun point at?	one entity per referent; a three-state reference outcome
burst	where do the claims divide?	claim records with dimensions and nesting
compose	is there a group here, and by what right?	typed groups and joins, or nothing
synapses	which of the fifteen roles does this participant hold?	a hub, spokes, and the method that decided each
validate	does the result satisfy the guards?	a gate, before the writer runs
materialize	write it.	the only rows in the store
ground	which sense of this word?	a canonical identifier, provisional at minimum
rehydrate	what prose expresses every stored claim?	text produced from the ledger alone
round trip	did the meaning survive?	a critical-loss report

Two stages are irreversible. The span stage defines the coordinate space every later stage addresses, and the materialize stage is the only writer. Everything between them can be replayed against the artifact before it, which is how a contract change is tested without re-reading the source.

The span stage is irreversible because it defines the coordinate space. Every later stage addresses a span in those frozen bytes; if the bytes move, every address is wrong. The materialize stage is irreversible because it is the only writer: a replay can rebuild an artifact, and it cannot un-write a row.

Here's an example: *Bach moved to Leipzig. He became cantor there.* The span stage freezes two sentences; the reference stage resolves *He* to Bach and *there* to Leipzig. The burst stage produces `MOVE(Bach, Leipzig)` and `BECOME(Bach, cantor, Leipzig)`; the synapse stage assigns `AGENT=Bach, DESTINATION=Leipzig` and `THEME=Bach, ATTRIBUTE=cantor, LOCATION=Leipzig`. The order is the semantics: none of those decisions could be made in another order.

Where the model speaks

The dividing line is not difficulty. It is whether answering the question requires knowing a language.

Python reports structure as facts about position: this token is the subject of *broke*; this token carries the label GPE. Those are true statements about a parse that assert no meaning. The model names the role from the closed fifteen, chooses the sense, classifies causal versus sequential, and reads the verb dimensions. Those are language questions, and the model is the instrument for them.

What the model returns is a proposal: constrained records inside a fixed boundary, never SQL and never an identifier. The validator checks the records against the vocabularies, mints the identifiers and writes bound parameters in one transaction. A hard validation failure rolls back canonical materialization; a local unresolved item becomes a ghost or a candidate set instead of failing the document.

The validator's statuses are the compiler's vocabulary of refusal. `REFUSE_NO_SPAN`, `REFUSE_ROLE`, `QUARANTINE_UNRESOLVED` and `GAP_REFERENCE` are not error messages; they are outcomes that the next stage and the reviewer can read. The validator refuses.

What the model may not do

The model may propose a role, a sense, a force and a dimension value. It may not mint an identifier, write SQL, or choose a schema. Those are the program's commitments, and the boundary is what makes the proposal reviewable. The model proposes.

The candidate tier

The compiler keeps candidates and resolved claims in separate tiers. A candidate is a proposal that has not passed validation; a resolved claim is a row that has. The separation looks redundant after a clean run, when the candidate tables are empty, and it is load-bearing when a run fails: the candidates show what the model proposed, and the resolved rows show what the validator accepted.

The contract of a stage

A stage contract has four parts: the input artifact, the output artifact, the invariants the output must satisfy, and the statuses the stage may emit. The contract is what makes the stage replayable; without it, a stage is a function with a name.

Here's an example: the model proposes `CLOSE(theme=valve, polarity=negative, voice=passive, tense=past)`. The validator checks that `CLOSE` is a sense in the lexicon, that `valve` resolves to an entity, that `theme` is in the fifteen, and that the span exists. If any check fails, the proposal is refused with a status; it is not repaired by the validator, because a repaired proposal cannot be audited.

Batch by paragraph

Requests are batched by paragraph, section or speaker turn, never one per clause. The rule is economic and architectural at once. A per-clause schedule makes the model's judgment unreviewable at document scale, thousands of decisions with no batch to inspect, and it makes cost scale with the wrong quantity.

The batch is also the unit of review. A reviewer can read a paragraph, read the proposals for the paragraph and check the roles against the sentence. A reviewer cannot do that for ten thousand isolated calls.

The gates

The compiler's gates are the stages that do not call a model. They check the schema, the span, the closed role set, the force and the sense, and they refuse rather than repair.

The failure statuses are part of the contract: a claim with no span is refused; a role outside the fifteen is refused. A force that is missing is refused; a sense that is not in the lexicon becomes a candidate, not a guess. The status is the compiler's record of what it could not settle.

Here's an example: a sentence with two licensed antecedents for *he* produces a reference gap, not a resolution. The gap carries the candidate set and the span; a later claim can resolve it, and the resolution is a new row with its own warrant.

The objection

The objection is that one call can do all of this. A large model can read a paragraph and return a graph, and the pipeline is unnecessary.

The concession is that a single call can produce a plausible graph. The answer is that fidelity is one axis out of three. The others are attribution and replay. A collapsed compiler can produce a graph and cannot say which stage chose a role, which span supports it, or which rule would have caught the error. The staged compiler can replay one stage against its artifact, which is how a defect is located rather than guessed.

The single call also makes the model the validator. The model proposes and commits in one move, and the move has no refusal path. The staged compiler separates the two, and the separation is what gives the refusal a place to live.

Replay

Replay is the compiler's repair method. A stage is re-run against the artifact before it, not against the source; if the output changes, the contract at that stage changed, and the change is the defect. Replay is why the stage boundaries are worth their cost.

The cost

The staged compiler costs more than a single call. The cost is in code, storage and review time, and it is paid because the book's claim is not that a graph can be produced; the claim is that a graph can be defended.

A compiler that cannot replay a stage can still produce a graph, and it cannot tell whether the graph changed because the source changed or because the model did.

The build sheet

The compiler build sheet names the stage table, the proposal schema, the validator statuses and the golden pair. Appendix F carries the pairs and the acceptance tests.

The order

The order is the semantics. The span comes first. Every later stage addresses a span in the frozen bytes, and a moved byte breaks every address. The reference comes next. A pronoun needs an alias map before it can resolve to one entity. The burst follows. A claim cannot be checked before the spans exist. The write is last. It is the only irreversible commitment.

A stage that runs out of order still produces rows, and the rows cannot be replayed. The stage is the unit. The contract has four parts.

The test

Take one paragraph. Run the stages in order and keep every artifact.

Then replay one middle stage against its artifact. If the replay reproduces the next artifact, the contract at that stage is stable; if it does not, the stage has a hidden dependency.

Then rehydrate the claims from the ledger alone. If the text preserves the claims, the roles, the force and the gaps, the round trip is faithful. If it loses a claim, the loss report names the stage, and the stage is the repair site.

Then remove the model from one stage and replace it with a fixed proposal. If the validator accepts the proposal and the rest of the chain still runs, the boundary is real; if the chain needs the model's prose to continue, the boundary is not a contract.

The stages that do not call a model are the ones a reviewer can check without trusting the model.

17 · The Assertion Adapter

A row is an assertion by the source system, not a fact; the source chose the table, the columns and the values; the adapter's job is to recover that choice as a claim, with the source as its holder.

The distinction matters because a row looks like a fact and carries a claim; the table name supplies the predicate, the columns supply the roles and dimensions, and the primary key supplies the identity. The adapter reads the schema as a predication and the row as one occurrence of it.

Why the adapter is mechanical

The adapter is mechanical because the schema already made the semantic decisions. A table's purpose is its name; a foreign key is a role; an enum is a value set. The adapter does not infer intent; it reads the commitments the schema already encodes.

The mapping also runs in reverse. A claim with a predicate and roles can be projected into a table when the predicate is fixed and the roles are columns. The projection is lossy in the same way: the table drops the holder, the force and the defeat, and the adapter must declare the drop.

The mapping

The mapping is mechanical when the schema is read as a frame.

Schema element	Claim element
table purpose	predicate
primary key	claim identity
foreign key	role slot
column name	role or dimension
enum	closed value set
timestamp	valid time or record time
status value	state claim

The mapping is not a guess about intent. It is a reading of the schema's own commitments: a foreign key says the row depends on another row, which is a role relation. An enum says the value is one of a closed set, which is a dimension; a timestamp says when the row was written or when the state holds, and the schema usually does not say which.

The orders example

Here's an example: `orders(order_id, customer_id, product_id, ordered_at)`.

The table purpose is `ORDER`. The primary key `order_id` identifies the claim. `customer_id` is a foreign key and fills the `AGENT` role. `ordered_at` is the time. The row `(44021, 847, P-12, 2026-04-10)` reads as `ORDER(agent=customer847, theme=productP12, time=2026-04-10)`.

The reading is not the whole claim. The row does not say who at the source asserted the order, under what authority, or what would cancel it. The adapter adds the holder as the source system and the force as an assertion, and it records the row's provenance as a span-like reference to the source table and key.

The adapter also records the row's record time. The `ordered_at` column may be the valid time of the order; the load time is when the row entered the warehouse. The two times are different, and a table that stores only one cannot say which.

Foreign keys are role slots

The foreign key is the adapter's most important signal. It is not merely a pointer; it names a participant in the predicate.

Here's an example: `order_items(order_id, product_id, quantity)` has two foreign keys. `order_id` fills the `AGENT` slot of a `CONTAINS` predication, and `product_id` fills the `THEME` slot. The row is not a link between two entities; it is a claim that an order contains a product in a quantity.

The distinction changes the graph. A model that treats foreign keys as edges produces a graph of entities; a model that treats them as role slots produces claims with participants, and the claim can be queried for its force, time and defeat.

Status columns are states

A status column is a state claim, not an event. `status=approved` reads as `BE(theme=order44021, attribute=approved)`, not as `APPROVE(agent=?, theme=order44021)`. The approval event may exist in an audit table, and the status is its current state.

Here's an example: an order has `status=approved` and an audit row `approved_by=rao at 09:14`. The status is the state; the audit row is the event. The adapter stores both and links them, and it does not infer an approver from the status.

Nulls and grain

A null is not a value. It is one of three things, and the adapter must say which: the value is absent, unknown, or not applicable.

Here's an example: `customer_id` null on an order may mean the order was placed by a guest, the customer was deleted, or the column was added after the row. The three readings have different claims, and a store that maps null to `AGENT=null` has chosen one without saying so.

The grain is the primary key. One row is one claim, and a composite key is a claim with a composite identity. A table with no primary key has no grain, and the adapter refuses it rather than inventing one.

Composite keys and grain

A composite key is a claim with several participants in its identity. `order_items(order_id, line_no)` has a grain of one line, and the line number is part of the claim's identity. A table with a surrogate key has a grain the schema chose, and the adapter must read the key, not the row count.

Timestamps

Timestamps are the adapter's most common ambiguity. A column named `created_at` may be the record time; `effective_at` may be the valid time; `updated_at` may be either. The adapter reads the column name and the schema's comments, and it records the choice as a convention with a date.

Enums

An enum is a closed value set, and the adapter maps it to a dimension value. The mapping is not free: a status enum with `approved`, `pending` and `rejected` maps to a state attribute, and a type enum with `standard` and `expedited` maps to a classification. The adapter names which.

What the adapter cannot recover

The adapter cannot recover the holder, the authority or the defeat condition from the row. It can recover the source system as a holder and the schema as a frame; the rest requires a record beside the table.

It also cannot recover the joins. A foreign key is a role slot, not a discourse link; a table with two foreign keys is not a claim that the two entities are related outside the predicate. The adapter stores the claim and leaves the link question to the analyst.

The source holder

The source system is the holder, and the holder is a claim about the source. A warehouse table loaded from three systems has three holders, and the adapter records which system wrote the row. A row with no source is a row with no holder, and the adapter refuses it.

What the adapter is not

It is not an ETL tool: ETL moves data, and the adapter makes claims. It is not a schema matcher: a matcher aligns columns, and the adapter reads roles. It is not a validator: the validator checks the claim, and the adapter names the frame.

The adapter table

The adapter's output is a table of claims, one row per source row.

Field	Source
claim_id	generated
predicate	table purpose
holder	source system
force	assertion or state
valid_time	timestamp column, if declared
record_time	load time
span	source table, key and row hash
status	accepted, quarantined or refused

The span is not a character range; it is a source reference. The adapter's claim is checkable against the row, and the row is checkable against the source system's own history.

The loader

The loader is the adapter's write path. It reads a batch, maps each row, validates the claim against the schema and writes the claim and its bindings in one transaction. A row that fails validation is quarantined with a status; the batch continues.

The batch

The loader reads a batch and writes in one transaction. A batch that fails validation is quarantined; the rows that pass are written, and the rows that fail carry their status. The batch boundary is what makes the adapter's report readable. The batch is a unit.

Refusal statuses

The adapter's refusals are named: `REFUSE_NO_GRAIN`, `REFUSE_NO_PURPOSE`, `QUARANTINE_NULL`, `GAP_COLUMN`. The names matter because the loader's report is read by the schema owner, and the owner needs to know which decision was missing. The status is a row.

Schema comments

Schema comments are evidence, not authority. A comment that says *customer who placed the order* supports `AGENT`; a comment that says *billing account* supports a different role. The adapter records the comment and the role, so the reading can be challenged.

The purpose

The table's purpose is the predicate, and the purpose is a claim. A table named `orders` asserts ordering; a table named `order_status` asserts a state. The adapter reads the name as a purpose and records the reading.

The key

The key is the grain, and the grain is the claim's identity. A surrogate key is a claim the schema chose; a composite key is a claim with several participants. The adapter reads the key before it reads the columns.

Convergence

Convergence is the adapter's proof. A row and a sentence about the same fact must compile to the same claim; if they do not, the difference is a schema ambiguity or a compiler defect. The convergence test is the subject of the next chapter.

A convergence test that passes on one fact and fails on another does not prove the adapter. It proves only that the two lanes share a frame for the facts that were chosen, which is why the test needs a fixture set rather than a single row.

The build sheet

The adapter build sheet names the source table, the purpose, the grain, the role map, the time convention and the refusal statuses. Appendix E carries the table; appendix F carries the golden pair.

The test

Take one table. Name its purpose, its grain and its foreign keys.

Map each column to a role or a dimension. If a column maps to neither, the table is carrying a property the claim model does not have, and the adapter must declare the gap.

Then read one row as a claim and ask the six questions. The adapter answers the content question and the source question; it does not answer the authority or the defeat question. Those are the record's job.

Three failures are common. The table purpose is inferred from the row content; the foreign key is treated as an edge; the null is mapped to a value. Each failure produces a table that loads and a claim that cannot be defended.

An adapter that maps every column to a role will invent roles the table never had, and an adapter that maps none will leave the table as a file, so the mapping needs a rule for the columns that fit neither.

A row is an assertion by the source system. The adapter's honesty is in what it refuses to infer, and the refusal is recorded as a status.

18 · Two Intakes, One Card

Both intakes are assertions by a source. The row has the system that wrote it; the sentence has the speaker who said it; neither is the world, and the convergence test asks whether the two sources made the same claim.

Convergence is a property of the record shape, not of the tool or the storage engine. If the prose lane and the tabular lane produce the same claim record, the record is the convergence; if they produce different records, the difference is the finding.

The pair

Here's an example: the table row `(44021, 847, P-12, 2026-04-10)` in `orders`, and the sentence *Order 44021 was placed by customer 847 for product P-12 on 10 April 2026*.

The row lane reads `ORDER(agent=customer847, theme=productP12, time=2026-04-10)`, with the source system as holder and assertion as force. The prose lane reads the same predicate, the same roles and the same time, with the speaker as holder and assertion as force.

The pair converges when the five fields align: predicate, roles, time, stance and grain. The fields are the comparison, and each field has a failure signature.

Why convergence matters

An architecture with two intakes has two ways to be wrong. The prose lane can misread a sentence; the tabular lane can misread a schema. Convergence is the only test that compares the two readings, and it is the reason the book treats them as one machine rather than two products.

The pair in detail

The pair is chosen so that the same fact has a natural expression in both lanes. The row has a grain, a key and a time; the sentence has a predicate, roles and a tense. The comparison is only meaningful when the two are about the same fact at the same grain.

The five fields

Field	Row lane	Prose lane
predicate	table purpose	verb sense
roles	foreign keys and columns	role bindings
time	timestamp column	tense and temporal expression
stance	source assertion	speaker and force
grain	primary key	one claim per predication

The comparison is field by field, and a mismatch in one field is a different defect from a mismatch in another. The five fields are the minimum for a convergence test; a comparison that checks only the predicate will pass two claims that disagree about time.

The convergence test

The test takes one fact, compiles it through both lanes and compares the five fields.

Here's an example: the row and the sentence above produce the same predicate `ORDER`, the same `AGENT=customer847`, the same `THEME=productP12`, the same `time=2026-04-10`, the same `stance=asserted` and the same grain of one order. The test passes, and the two lanes produce one card.

The test is not a similarity score. A similarity score can pass two claims that share words and disagree about roles; the field comparison cannot, because each field is checked against the schema.

Failure signatures

A failure localizes. The signature names the field that disagreed, and the field names the repair.

Signature	What disagreed	Repair
missing verb	the row has a purpose, the sentence has no predicate	lexicon sense or a gap
coarser grain	the row aggregates, the sentence individuates	adapter grain or a second table

Continued on next page

(Continued)

Signature	What disagreed	Repair
missing time	the row has no timestamp, the sentence has a date	schema column or a declared gap
stance mismatch	the row is current state, the sentence is testimony	force and stance fields

Here's an example: a table of daily totals and a sentence about one order do not converge, and the signature is *coarser grain*. The repair is not to change the sentence; it is to add the line-level table or to declare that the aggregate is a different claim.

A stance mismatch is the most consequential. A row records the current state as an assertion by the system; a sentence may report what a person said. The two claims can share content and differ in holder and force, and a convergence test that ignores stance will call them the same.

Tool independence

The same pair must converge in SQLite, in a graph store and in RDF. If it converges in one and not another, the difference is the projection, not the claim.

Here's an example: the claim `ORDER(agent, theme, time)` projects into a SQL row, a graph edge with properties and an RDF triple with a time annotation. The three projections have different shapes and the same claim; the convergence test reads the claim, not the projection.

The tool independence is what makes convergence a property of the record. A test that passes only in one engine is a test of the engine.

What convergence does not prove

Convergence does not prove that the claim is true. Both sources can be wrong in the same way, and the record will converge on the same error.

Convergence also does not prove that the schema is complete. A pair can converge on the five fields and still omit the authority or the defeat condition; those are the record's job, not the convergence test's.

The test proves one thing: the two lanes read the same claim from the same fact. That is the property the architecture needs before it can compare sources.

Five fields, one at a time

Predicate: the row's table purpose and the sentence's verb sense must resolve to one predicate. `orders` and *placed* are the same predicate only if the lexicon says so; a table named `transactions` and a sentence with *paid* may be a different predicate.

Roles: the row's foreign keys and the sentence's role bindings must agree on role and filler. A row with `customer_id` and a sentence with *by customer 847* agree; a row with `billing_id` and a sentence with *for customer 847* do not.

Time: the row's timestamp and the sentence's temporal expression must agree on valid time. A row with `created_at` and a sentence with *on 10 April* may agree on record time and disagree on valid time.

Stance: the row is an assertion by the system; the sentence may be an assertion, a report or a question. The stance fields must agree, and a mismatch is not a content error.

Grain: the row's primary key and the sentence's predication must be one claim. A daily total and a single order are different grains, and the difference is the failure.

The failure cases

Missing verb is the lexicon failure: the table has a purpose and the sentence has no predicate the lexicon knows. The repair is a sense or a gap.

Coarser grain is the adapter failure: the row aggregates and the sentence individuates. The repair is a line-level table or a declaration that the aggregate is a different claim.

Missing time is the schema failure: the row has no timestamp and the sentence has a date. The repair is a column or a declared gap.

Stance mismatch is the force failure: the row is current state and the sentence is testimony. The repair is a stance field, not a content edit.

The gate

Convergence is a gate, not a score. A pair either passes the five-field comparison or it does not; a percentage hides which field failed, and the field is the repair site.

The fixture

The fixture is a set of pairs, not one pair. One pair can pass by luck; five pairs with different grains, times and stances test the frame. The fixture is part of Appendix F.

The report

The report names the pair, the five fields, the pass or fail per field, the signature and the repair. A report with a single pass or fail is not a report; it is a verdict without a location.

The key

The two lanes must mint the same claim key or a resolvable pair. If the keys differ, the records are two claims that happen to agree, and the next correction will split them.

The holders

The row's holder is the system; the sentence's holder is the speaker. Convergence requires the holders to be recorded separately even when the content agrees; a merged holder loses which source to challenge.

The time test

A time field can converge on the string and fail on the meaning. `2026-04-10` in a row and *10 April 2026* in a sentence agree on the date; a row's `created_at` and a sentence's *effective 10 April* do not agree on valid time even when the strings match.

Two ways to pass

A pair can pass for the wrong reason. The first is a shared word: both lanes say *order*, and the comparison stops at the predicate. The second is a shared grain: both lanes describe one order, and the comparison never asks whether the time is valid or record.

The first pass is a vocabulary match. The second is a grain match. Neither is a convergence, and both produce a report that says pass.

A convergence test needs a negative pair: a sentence and a row that share a word and disagree on a field. If the test passes the negative pair, the test is broken.

The negative pair

The negative pair is the test's control. It has the same predicate and a different role, time or stance; a convergence test that passes it is measuring words, not claims. The test needs a control.

A negative pair that shares the predicate and differs on the grain is the strongest control, because it catches the test that stops at the vocabulary and never reaches the claim.

The report's fields

The report names the pair, the predicate, the roles, the time, the stance, the grain, the verdict and the repair. A report that names only the pair and the verdict cannot be repaired, because the reader cannot tell which field disagreed. The report names the field. The signature is the field. The field is the repair.

A test that compares only the predicate can pass a row and a sentence that disagree about who did it, when it happened and whether it was asserted or reported, which is the whole of the claim.

The test

Take one fact that exists in both a table and a sentence. Compile it through both lanes.

Compare the five fields. If all five align, the two lanes produce one card; if one differs, classify the signature and repair the lane the signature names.

Then run the same pair in a second engine. If the claim converges in both engines, the record shape is doing the work; if it converges in one, the projection is doing the work.

Then ask what the test does not prove. A convergence test that is treated as a truth test has confused agreement with evidence.

Three failures are common. The pair compares predicates only; the row lane and prose lane use different grains; the stance is ignored because both are treated as facts. Each failure produces a convergence report that passes and a record that disagrees.

A convergence test that passes on the predicate and fails on the grain has proved only that the two lanes share a vocabulary, which is the least interesting part of the claim.

Two intakes, one card. The card is only as good as the fields it compares.

19 · Five Clocks, Correction and Withdrawal

`updated_at` says when the row changed. It does not say when the fact became effective, when the source asserted it, when a reviewer accepted it, or when a report began using it.

A dispute asks one of those questions, and a schema that keeps only the row clock answers a different one. The five clocks are not a refinement of `updated_at`; they are five different questions, and a claim record keeps them apart.

The five clocks

Clock	The question	Where it belongs
event time	when did the thing happen or hold?	the claim's validity interval
record time	when did the system learn it?	transaction time on the claim
valid time	when was the claim in force?	valid_from and valid_to
speech time	when was it said, and in what context?	the source anchor
assertion time	when did an authority put it forward?	the review event or transition

Here's an example: order 44021 was placed on 10 April, the warehouse learned it on 11 April, the approval was valid from 12 April, the approver said so in a review on 13 April, and the authority granted the approval on 14 April. Five clocks, five dates, one claim. A query that filters on `updated_at` returns one of the five and reports it as the other four.

The example is deliberately one claim. A claim with five clocks is not five claims; it is one claim with five dates, and each date answers a different question. A schema that stores one date and calls it `updated_at` has chosen the record time and hidden the other four.

Why five

One clock is a timestamp; five clocks are a model of time. A system that keeps one can say when it changed; a system that keeps five can say when the thing happened, when it learned, when it was valid, when it was said and when it was authorized.

The three disputes ask different questions. *When did it happen* selects event time. *What did we know* selects record time. *What was in force* selects valid time. The two remaining clocks answer who said it and who authorized it, and a record without them cannot separate a report from a decision.

The last two are the ones enterprise schemas drop. Speech time is the context of the source; assertion time is the moment an authority put the claim forward. Dropping them is a choice, and the choice should be named rather than hidden in a column default.

The five dates are separate questions, and the record answers each with its own field.

No overwrite

A correction is a new claim, not a changed value. The old claim remains readable, the new claim carries the corrected content, and a supersession row names both and the reason.

The supersession is what makes the five clocks useful. A reader can ask what the record said on a date, what was valid on a date, and what the source said on a date, and get three different answers from one chain.

Here's an example: the approval of order 44021 is corrected on 20 April. The old claim remains with its record time; the new claim carries the correction and its own record time; the supersession row names the old claim, the new claim and the reason. A query for the state on 15 April returns the old claim; a query for the current state follows the chain to the head.

The supersession chain is the clock's index. A query for the state on a date walks the chain to the head at that date; a query for what the source said walks the speech time; a query for what was valid walks the interval. The chain is what makes the dates queryable rather than decorative.

The bitemporal view

A bitemporal view has two axes: valid time and record time. The cell at (valid 12 April, record 20 April) is what the record believes about 12 April as of 20 April. A query for the state on 12 April as known on 15 April reads a different cell, and the difference is the correction.

Four acts

The lifecycle vocabulary separates four acts that one `DELETE` usually stands in for.

Act	What it may touch	What must survive	What it does not establish
supersede	standing of the prior claim	prior identity, interval and evidence	that the prior was false when made
retract	validity of the prior claim	the prior claim as a historical record	that the prior was false at all
redact	one named projection	canonical history and the source span	that the material is untrue
delete	the row itself	recoverability, authority and scope	that the record was worthless

The three destructive acts are retract, redact and delete; supersession is the correction that leaves the prior claim standing as history. The plan's three acts are these three, and the fourth is the ordinary correction.

The table is a permission table as much as a lifecycle table. A supersede may touch standing and may not touch the prior interval; a retract may touch validity and may not touch the prior identity. A redact may touch a projection and may not touch the span; a delete may touch the row and may not touch recoverability.

The correction record

A correction record has five fields: the prior claim, the new claim, the act, the authority and the reason. The act is one of the four; the authority is who may perform it; the reason is why. A correction without an authority is a change without a responsible party.

Retraction

Retraction is the sharpest case because the ordinary reading is wrong. A customer withdraws an allegation; the withdrawal ends the claim's validity and leaves the allegation standing as something that was alleged. The retraction does not establish that the original claim was false.

Here's an example: Carter testifies that she saw Cole leave, then recants. The recantation is a retraction: the testimony is no longer in force, and the record still shows that Carter testified. A store that deletes the testimony loses the fact that it was made and cannot answer what the witness said before the recantation.

The retraction test is a query: what did the witness say before the recantation? If the record can answer, the retraction preserved the testimony; if it cannot, the retraction deleted it and called the deletion a correction.

Redaction and deletion

Redaction touches one projection, not the canonical history. A public report may redact a name; the ledger keeps the name and the span, and the redaction is a claim about the projection.

Deletion is a lifecycle kind, not a privilege. It has an authority, a scope and a recoverability field, and it is reviewed by the same machinery as supersession. A deletion that leaves no recoverable row is a decision without a record, and the record is the point.

A deletion record has the same five fields as a correction: prior claim, authority, act, scope and recoverability. A redaction record adds the projection it touches. The two records are different because one destroys the row and one hides it; the schema keeps them apart.

The measurement case

A later measurement supersedes an earlier one, and the contradiction is dated rather than smoothed. A number that changes without a date turns a step change into a trend, and a trend is a claim nobody made.

Here's an example: the valve clearance is measured at 0.12 mm in March and 0.08 mm in April. The April measurement supersedes the March one, and the supersession carries both dates. A query for the March value returns 0.12; a query for the current value returns 0.08; the comparison is a dated change, not a contradiction.

The measurement case is the one where nobody is wrong. The March value was correct when measured; the April value is correct when measured; the supersession dates the change. A record that keeps only the latest value cannot tell a step change from a drift.

The read filter

Every read must state a standing. A query that does not filter on supersession returns retracted and superseded claims, and the count that includes them is wrong.

The filter is not new work. A ledger that already requires a query to say what it counts can require it to say which standing it counts. The cost is one predicate per query; the cost of forgetting is a number that includes a claim an authority withdrew.

The filter has three standings: current, as-of a date, and all. A query for current follows the chain to the head; a query for as-of stops at the date; a query for all returns the chain. The three standings are the read contract.

The audit query

The audit query asks what changed between two dates, and it is the query the five clocks exist to answer. It returns the claim, the old value, the new value, the act and the authority. A record that cannot answer it can still report the current value, and it cannot explain it.

The retraction query

The retraction query asks what was in force on a date. It reads the valid interval and the retraction act, and it returns the claim if the date is inside the interval and before the retraction. The query is the practical form of the distinction between validity and truth.

The test

Take one claim. Ask the five clocks and write the five dates.

Then correct it. The old claim must remain, the new claim must carry the correction, and the supersession must name both. If the old row is overwritten, the clocks are gone with it.

Then retract it. The claim must leave force, and the record must still show that it was made. If the retraction deletes the claim, the record has lost the difference between *never said* and *withdrawn*.

Then query the current state and the state on a date. If the two queries return different claims, the clocks are doing their work. If they return the same claim, the record has one clock and calls it five.

Three failures are common. The row is overwritten; the retraction deletes the claim; the query omits the standing. Each failure produces a record that answers the current state and cannot answer the history.

An audit that cannot separate the five clocks will date the correction to the row write, the fact to the source and the validity to the interval, and the three dates will disagree in the report.

A record that cannot answer what it said before the correction has no history; it has a current value and a story about how it got there.

PART IV
The Machine

how it is compiled, and how the compile is proved.

20 · Compiler Hesitation

A hole is legal; a guess is a defect. A prose compiler that cannot represent “I did not write a row because the source did not settle it” will confabulate, because the only alternative to writing a row is writing nothing, and writing nothing looks like success.

Hesitation is the set of refusals that keep the compiler honest; each refusal is a typed status, not an exception, and each has a fixture that fails when the brake is removed.

The binary and its cost

Most extraction systems make one decision per clause: write a row, or do not. The second branch has no representation, so it is indistinguishable from a failure, a skip and a clause that was never seen.

The compiler separates five moves: assert, gap, candidate, skip and refuse.

Move	What it writes	When it applies
assert	a claim row	the source settles the claim
gap	a legal gap with a trigger and span	the source leaves a reference or presupposition open
candidate	a candidate set with confidence	the source licenses more than one reading
skip	a skip receipt with the surface span	the clause will not compile
refuse	a validation status	the proposal violates the schema

The five moves are not five shades of caution. Each is a different record, and each can be queried.

Why five

The five moves correspond to five kinds of absence. Assert means no absence. Gap means a reference or presupposition is open. Candidate means the source licenses more than one reading. Skip means the clause could not be compiled. Refuse means the proposal violated the schema.

A system that collapses the five into one unresolved status cannot tell a reviewer what to do. The reviewer sees a hole and does not know whether to read the source, choose a candidate or fix the schema.

The assert

An assertion is the ordinary case, and it is the one that needs no brake. The compiler writes the claim, the span and the warrant, and the record moves on.

The thirteen brakes

The brakes are the situations that stop a write. The table is the list, not a set of preferences.

Brake	Situation	Emitted status	Forbidden move
H1	a group is proposed	group opens only if boundary, type, order and target are named	a group opened because the graph would look finished
H2	said that P	two cards and a NESTED_IN link, method=report	a quotation group of two peers
H3	two licensed antecedents	LEGAL_GAP plus candidate rows	a silent bind
H4	then, next, after, list position	PRECEDES or procedure ordinals	CAUSES or ENABLES
H5	a clause that will not compile	skip receipt with span; parent not complete	quiet success on the parent
H6	furniture the source marked	the rows the marks require, or a named gap	green where coverage stands in for sections
H7	group-as-nest-target not writable	specified-unbuilt label	flattening the child onto the parent
H8	a spoken turn with no headings	packaging group from layout, or a licensed discourse group	a fake heading
H9	sense, role, scope or identity doubt	resolution_candidate with confidence and evidence	a winner with no candidate siblings
H10	cite-strip, footnote drop, table drop	drop_policy_id, version, discarded spans	promoting may to is
H11	a later sentence retracts an earlier claim	RETRACTS, SUPERSEDES or REVISES link	rewriting the first card in place

Continued on next page

(Continued)

Brake	Situation	Emitted status	Forbidden move
H12	the test suite	harness asserts the gap after the run	a test that concatenates key and text
H13	a quoted clause in a determiner phrase	inner synapse plus a POINT join	a dictionary entity whose surface is the whole clause

The forbidden column is usually the tidier option. That is the point: the brake exists because the tidy move is the one that hides the uncertainty.

H3, the pronoun

Here's an example: *Bob met Bill. He left.* Two masculine antecedents are licensed, and the source does not choose. The compiler writes a `LEGAL_GAP` with the candidate rows, not a resolution.

A store that resolves *He* to Bill has made a choice the sentence did not make. The choice is invisible in the graph, and every later query about Bill inherits it. The gap is the honest record: it says the reference is open, the candidates are Bob and Bill, and the span is the second sentence.

The candidate set

A candidate set is the record of a reading the source licenses but does not choose. It carries the candidates, the evidence for each and a confidence. The set is not a resolution; it is a queue for review.

The skip receipt

A skip receipt is the record of a clause that did not compile. It carries the surface span, the reason and the parent. The parent is not complete; a skip is not a success. A store that marks the parent complete after a skip has hidden the hole.

The refusal

A refusal is a validation status for a proposal that violates the schema. It is different from a gap: a gap is a property of the source, and a refusal is a property of the proposal. The distinction matters because the repair is different.

Calibration

A refusal is cheap to emit and expensive to consume. It moves work to a reviewer, and if the refusal rate is high enough the reviewer becomes the compiler.

The brakes are not uniform. Nesting and marked sequence are high-reliability because the source printed the structure in dependency arcs and in words such as *first* and *then*. The weak cases are thematic sets with no coordination, unmarked resumption and unmarked self-correction; the last two fall to `LEGAL_GAP`, and the first routes to a paragraph-driven segmenter whose reliability is the open question.

The brakes concentrate where the text withheld its own structure. A compiler that gaps every pronoun is a straw man; the fixture is two licensed antecedents, not one.

The brakes fall into three classes. The first is structural: H1, H2, H4, H6 and H13 read a mark the source printed. The second is evidential: H3, H7, H8, H9 and H10 read a gap the source left. The third is operational: H5, H11 and H12 read the compiler's own state. The classes have different calibration profiles, and a single refusal rate hides the difference.

Three numbers matter. The trigger rate is how often a brake fires. The denial rate is how often a reviewer judges the trip wrong. The miss rate is how often the brake should have fired and did not. The first is cheap; the second and third require review, and they are the ones that tell a team whether the brake is calibrated.

The objection

The objection is that the brakes are set too tight. A compiler can refuse every group, gap every pronoun and keep every row legal while producing a bag of facts that cannot answer a question requiring composition.

The concession is real. Hesitation protects the record; it does not build it. The brakes are worth their cost only on a compiler that can also compose, and composition is a separate capability.

The audit's phrase for the failure is honesty without structure: a ledger of lonely cards that cannot answer a question requiring composition. The phrase is a useful warning because it names the limit of the brake set. Hesitation is a property of each write; composition is a property of the set.

The answer is not to remove the brakes. It is to measure them.

Measuring hesitation

Gap rate is the wrong number, because it is gameable in one direction: gap everything and legality is perfect.

The first measurement is mechanical. Remove a brake and its fixture must fail. A brake without a failing test is not a brake.

The second is the denial rate: the share of brake trips that a careful reader judges wrong. The corpus does not report it. The discipline that survives the gap is the one the file closes with: do not tidy a hesitation into a fact.

The denial rate

The denial rate is the brake's false-positive measure. A brake that fires on an unambiguous sentence is a brake that will be ignored, and a brake that is ignored is worse than no brake because it teaches the reviewer to skip the status.

The miss rate

The miss rate is the brake's false-negative measure. It requires a reviewer to find a case where the compiler asserted and should have gapped. The miss is invisible in the output, which is why it is the harder number.

The brake fixture

A brake has a fixture: a sentence, the expected status and the forbidden move. The fixture is run twice, once with the brake and once without. The first run must emit the status; the second must fail. A brake whose removal changes nothing is a comment.

The refusal log

The refusal log is the compiler's memory of what it did not write. It has one row per refusal: status, span, trigger, brake and date. The log is queryable, and the query is how a team learns which brake is doing the work. The log is a table.

A refusal log that is never read is a graveyard of good intentions, and a log that is read is the compiler's only evidence about its own calibration.

The status vocabulary

The status vocabulary is closed: assert, gap, candidate, skip, refuse. A compiler that adds a sixth status has changed the interface, and the interface is what the reviewer reads. The status is a row.

A brake that fires correctly and is never reviewed is indistinguishable from a brake that fires because the reviewer stopped reading, which is why the denial rate is the number that matters. The gap is legal.

Three failures are common. The compiler asserts a resolution for an ambiguous reference; the compiler gaps an unambiguous sentence; the brake has no fixture. Each failure produces a record that looks cautious and cannot be calibrated. The guess is a defect.

A refusal that is emitted, logged and never read is a ritual, and the ritual is the failure mode the brakes were supposed to prevent, because a ritual cannot be calibrated.

The test

Take one ambiguous sentence and one unambiguous sentence.

For the ambiguous sentence, the compiler must emit a gap or a candidate set, not a resolution. For the unambiguous sentence, it must assert, and a gap there is a false refusal.

Then remove one brake and run the fixture. If the fixture still passes, the brake was not load-bearing. If it fails, the brake is doing work, and the failure names the forbidden move.

Then read the refusal as a reviewer. If the refusal is wrong, the brake is mis-calibrated, and the calibration is the finding.

A compiler that can say *I did not write this* is more trustworthy than one that always writes something, and the difference is the gap.

21 · LLMs Propose; gates Dispose

A model may propose a reading of a text and may not commit one. The gate does not improve the proposal; it decides whether the proposal may exist.

Commit authority is what the architecture allocates, not intelligence; a model that reads a hard sentence well still may not write the row; a deterministic function that understands nothing still must. The reason is not modesty about models. A commit is a promise the system keeps after the model that made it has been replaced, and only something re-runnable can keep a promise.

The division

The model answers language questions: which sense, which role, which force, which attachment. The program answers structural questions: is the role in the closed set, is the span inside the frozen bytes, does the participant exist, is the stance reachable from the parent.

The division is not a division of difficulty. It is a division of kind. A language question needs a language model; a structural question needs arithmetic over the whole batch, and a sampler cannot perform arithmetic over a document it never holds.

The model's proposal

The proposal is a constrained record: predicate, roles, stance, span and candidates. It is not SQL, not an identifier and not a paragraph. The constraint is what lets the gate read it as data rather than as an effect.

Why a file, not an effect

A proposal that arrives as a file can be counted, re-read, hashed, diffed, cached and refused. A proposal that arrives as an effect on a connection cannot be any of those things. The boundary is not fastidiousness; it is what makes the proposal reviewable.

The write path

Only the write path commits. The model cannot open a connection, and the gate cannot write a row it did not validate. The separation is enforced by the process boundary, not by a convention in the prompt.

Why the model cannot check itself

A model produces a record by sampling forward, one token at a time, and it retains no view of the whole. The properties the ledger needs are properties of the whole, computed after generation.

Invariant	Quantifies over	Why a forward sample cannot establish it
closure	every role code in the batch against a fixed inventory	one off-list code in four hundred rows is invisible from inside the stream
partition	the clauses against the rows plus declared gaps	coverage is a subtraction over the document, and the model never holds the document
referential integrity	participant targets against rows that must already exist	order of existence is a global fact, not a local fact about the sentence
re-slicing	the cited range against the frozen source bytes	character arithmetic is the counting a sampler does unreliably
non-promotion	whether a row reachable only through a reported slot carries asserted stance	reachability is a graph property; the stance was written before the graph existed

A model can check its work, and the check is another sample from the same distribution, conditioned on the first. That is why fluent self-correction repairs style and rarely repairs reference.

Three tiers

The arrangement has three tiers. The syntax ledger makes no unstated semantic claim. The candidate tier holds rules, dictionary senses and model proposals. The canonical tier is reachable only through validation, and every accepted row points back to the evidence that licensed it.

The model enters at the candidate tier and never reaches the canonical tier directly. A better model proposes better candidates, and the gate's authority does not move.

The tiers are also a repair path. A failed proposal is not deleted; it is moved down a tier, where a reviewer can see what the model proposed and why the gate refused it. A proposal that is deleted takes its evidence with it.

A sentence and five gates

Here's an example: *The ministry said it may extend the deadline for firms that file late.*

A batched adjudication returns three records: a SAY record, an extension with possibility modality, and a filing. The records look plausible. The gates read different things.

Gate	What it reads	Result
vocabulary and folds	the fifteen role codes and the retired-code fold	PURPOSE is folded to REASON
stance reachability	whether a row whose only parent is a SAY record carries asserted stance	the extension is demoted to reported
span re-slice	the frozen source bytes at the cited offsets	passes for the SAY record and the filing
participant coverage	whether every participant has a mention in the span or a resolved antecedent	the firms fall outside the extension's span, so the row is held
identity	canonical ID format and namespace	the pronoun stays a mention, not an alias

The outcomes differ. The role is folded because the ledger cannot hold an off-grammar role. The stance is demoted because a structural fact about the parent contradicts the proposal. The participant row is held because the span does not cover it. None of the three is repaired by the model.

The outcomes

A gate has four outcomes: accept, fold, demote and hold. Accept commits the row. Fold maps an off-list value into the closed set and records the fold. Demote changes the force or stance on the strength of a structural fact. Hold sends the row to review with a status, and the status is queryable.

The fourth outcome is refusal: the proposal violates the schema and is not stored. Refusal is different from a gap. A gap is a property of the source; a refusal is a property of the proposal.

The outcomes are records, not log lines. An accept writes the row and the gate trace; a fold writes the original and the folded value; a demote writes the prior stance and the new one; a hold writes the status and the candidate. The trace is what makes the gate auditable after the fact.

The audit trail

Every accepted row points to the proposal, the gate trace and the span. The chain lets a reviewer ask why a claim exists and get an answer that is not the model's recollection.

The review

The gate decides whether a proposal may exist; it does not decide whether the reading is right. A reviewer reads the proposal, the span and the gate trace, and accepts or rejects the reading. The gate narrows the review; it does not replace it.

What the gate cannot see

The gate is not omniscient. A gate that shares the compiler's parse is blind to an error in that parse: if the parser attached the phrase to the wrong head, the gate checks the attachment and passes it.

The gate also cannot judge whether the model chose the right sense. It can check that the sense exists and that the span supports it; it cannot check that the sense is the one the author meant. That is a review question, and the record must preserve the candidate so a reviewer can answer it.

The last limit is the important one: a gate that cannot be shown to fail is not a gate. It is a green light with a schema attached.

The blind spot is structural, not accidental. A gate that reads the same parse as the compiler inherits the parser's errors; a gate that reads the source independently can catch them, and it costs a second parse. The choice is a design decision, and the design should say which it made.

A gate that shares the compiler's parse can confirm every structural property of a proposal and still pass the one error that matters, because the error is in the parse the gate and the compiler share.

The failure modes

The first failure is handing the model the pen: ask for JSON, receive JSON, load it. The model assigns identifiers, writes the record, resolves the pronoun and invents a role, and the bad value and the good value have the same type and the same shape.

The second failure is repairing instead of downgrading. A proposal that fails a gate is not re-prompted until the record looks finished; it is downgraded to a candidate, a ghost or a gap, and the downgrade is preserved. Re-prompting destroys the only evidence that an ambiguity was there.

The third failure is a gate that checks only structure. A well-formed wrong claim arrives with a span, a stance and a role, and the harness confirms the form while the reader is left to confirm the content.

A proposal that is refused and then re-prompted until it passes has not been validated; it has been trained on the gate, and the gate's future refusals are weaker for it.

The gate's own test

A gate has a fixture: a good proposal, a bad proposal and the expected outcome. The fixture is run with the gate and without it. Without the gate, the bad proposal enters; with it, the bad proposal is refused, folded, demoted or held. A gate whose removal changes nothing is decoration.

The model's limits

The model cannot assign an identifier, write SQL, or choose a schema. Those are the program's commitments. The boundary is what makes the proposal reviewable, and it is enforced by the interface, not by the prompt. The gate is a program.

The gate's limits

The gate can check that a role is in the set, that a span is in the bytes and that a stance is reachable. It cannot check that the author meant the sense; that is a review question. The review is a person.

None of the gates decides whether the reading is right; that decision belongs to the reviewer and the record.

The test

Take one sentence and one proposal. Run the five gates and record each outcome.

Then inject a bad proposal: an off-list role, a span outside the source, a participant with no mention, a reported row marked asserted. The gate must refuse, fold, hold or demote each one; a gate that accepts the bad proposal is not a gate.

Then remove one gate and run its fixture. If the fixture still passes, the gate was not load-bearing.

Then replay the batch. A gate that reads the frozen source and the closed sets should produce the same outcome on the same proposal; a gate that depends on the model's prose will not.

Three failures are common. The model is handed the pen; the failed proposal is repaired by re-prompting; the gate checks structure and not stance. Each failure produces a record that looks complete and cannot be audited. The trace is a row.

A model may be trusted to read a sentence and never to write the record. The gate does not make the reading better; it decides whether the reading may become a commitment.

22 · Rehydration as Proof

A gate can accept a record and still leave the important question unanswered. The record may satisfy every structural check and still say something the source did not. The test that asks whether the record kept the sentence is the round trip.

The loop

Dehydration packs a sentence into rows: a predicate hub, the role-bearing participants, the stance, the dimensions, and the links to whatever sits inside it; rehydration unpacks those rows into a sentence without looking at the original. The comparison is not between the source and the rebuild. It is between the first ledger and the ledger produced by recompiling the rebuild.

Take A as the source, S as its dehydration, A' as the rehydration of S, and S' as the dehydration of A'. The artifact under test is S against S'. Wording may move. Claims, roles, scope, stance, time and polarity may not.

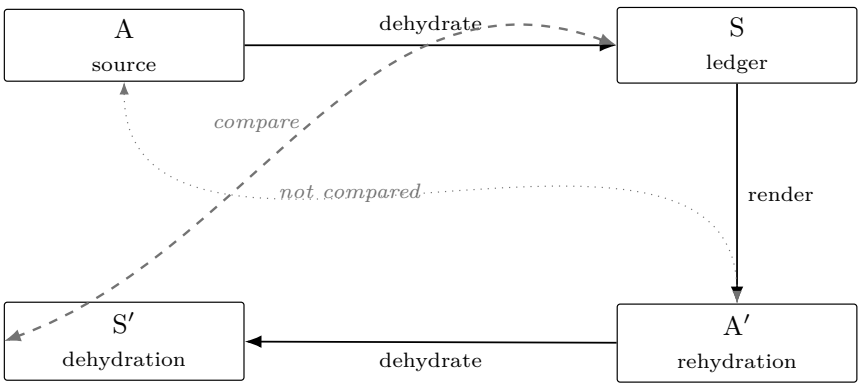


Figure 3: The round trip: A to S to A-prime to S-prime. The comparison is S against S-prime; A against A-prime is the comparison the test refuses.

The figure is the test's shape; the diagonal it leaves out is the one a demonstration would take, because a source compared with its rebuild varies only in wording and cannot fail.

Here is an example:

```
source: The auditor told the board that the vendor did not ship the parts in March.

S1 tell AGENT=auditor RECIPIENT=board tense=past stance\index{stance}=asserted
S2 ship AGENT=vendor PATIENT=parts TIME=march polarity=negative
 tense=past stance=reported
S1 --NESTED_IN--> S2
```

Four renders of that ledger show what the comparison is for.

Render	Claims	Stance	Time	Verdict
The auditor told the board that the vendor did not ship the parts in March.	2/2	pass	pass	pass
The vendor did not ship the parts in March.	1/2	promotion	pass	fail
The auditor informed the board that the vendor didn't send the parts in March.	2/2	pass	pass	pass
The auditor told the board that the vendor did not ship the parts until March.	2/2	pass	fail	fail

The third render is the point. Three words changed and nothing was lost, so a test that scored wording would mark it down while a structural comparison lets it through. The fourth render is the opposite case: one preposition moved the temporal claim, and the comparison catches it only because the temporal phrase carries its own preposition in the ledger; a loss the schema cannot represent is a loss the round trip cannot name.

What a pass means

The criterion is negative. A render is not scored for elegance or fluency; seven checks decide, and each fails independently: claim recall, entity recall, box fidelity for content, scope and condition links, absence of promotion, matching stance labels, matching order or procedure, and matching completion status.

The checks are independent because each can fail while the others pass; a render can recall both claims and still promote a reported claim to an asserted one; a render can keep the stance and move the time. Independence is what makes the checklist useful: it localizes the loss.

A single failed check fails the pass. A high average cannot outvote a dropped negation. The comparison is not a diff: a diff counts word changes, and the comparison counts claim changes.

Wording may change. A synonym swap passes when both words bind the same predicate and the same entity. A dropped box fails. Loss that cannot be named is the defect; paraphrase is not. The criterion is also what makes the loop usable on documents the compiler was not built around. It does not need a gold parse. It needs the second compile to return the same claims.

The renderer must not see the source

The obvious cheat is to let the renderer read the sentence it is supposed to rebuild. A model with the source repairs whatever the ledger dropped, and the repair is invisible because the output is fluent either way. The rule is blunt: in fidelity mode, the renderer receives the ledger and not the source.

A rule like that is worth nothing unless it is tested. The test is a perturbation. Hand the renderer a ledger whose claims were deliberately damaged, change a role binding or drop a participant, and confirm that the text changes. If the output stays faithful to the original despite the damage, the renderer is reading the source or the fixture, and every score it produced is void.

The fixed point

A round trip compares two components derived from one parse. That is its strength and its ceiling. Where the parse is right, the loop catches whatever the renderer or the ledger dropped. Where the parse is wrong in the same way on both sides, the error cancels and the loop passes.

Here is the canonical failure:

```
source: The valve was not closed before the pump started.
correct: NEG(close(valve)) BEFORE start(pump)
mutant: close(valve) BEFORE start(pump)
```

The correct ledger rehydrates to the source sentence, and the round trip passes. The mutant drops negation, rehydrates to “The valve was closed before the pump started,” and the round trip also passes. The comparison sees agreement because both sides use the same rule. The record is wrong and the test is green.

The same thing happens with deontic operators:

```
source: Employees do not have to work Saturday.
render: Employees must not work Saturday.
```

The words overlap almost completely and the obligation reverses. A lexical similarity score would call the pair close. An operator comparison calls it a contradiction. The round trip catches this case only if the ledger distinguishes permission from prohibition; if the compiler writes both as a bare `work` event, the renderer can reproduce the wrong stance and the loop stays green.

A fixed point is not a fidelity proof. It is agreement between two things that learned the same mistake. The useful question is not whether the loop can pass. It is which errors the loop can see.

Preservation and discrimination

A ledger that stores nothing round-trips perfectly. So does a ledger that conflates two meanings, because in both cases the writer and the reader agree with each other. The round trip tests preservation; it cannot test discrimination.

The second instrument asks a different question: do sentences that mean differently get different shapes? It runs minimal pairs and prints per-dimension occupancy. A pair that differs on an empty column is flagged as confounded, because the difference may have nothing to do with the dimension under test. A pair that the ledger represents with identical rows is a collision, and a collision is the defect the round trip cannot see.

The two tests are not substitutes. Preservation without discrimination stores everything and distinguishes nothing. Discrimination without preservation distinguishes shapes that no longer reconstruct the sentence. The stack needs both.

What spans catch

The span check compares the record to the source at exact offsets. The document is frozen; the bytes do not move. Each claim names the span it came from, and the check asks whether that span still contains what the claim says it contains. A dropped negation, a changed number, a moved date and a missing participant all leave a disagreement between the span and the row.

An offset is a pair of integers into the frozen bytes. The check compares the text inside the offset to the claim's literal, not the offset's length. A same-length substitution passes a length check and fails a text check. A span that starts one token late will still contain most of the phrase, which is why the comparison is exact rather than approximate.

The check is not infallible. A probe over a document collection once reported two matches where the true count was 514,071. The compiler had not lost half a million spans. The instrument had counted documents instead of spans and reported a unit error with complete confidence. The example is worth keeping because it shows the

failure mode at the level of the instrument: a check that points at the wrong bytes proves nothing about the record.

A smaller case shows the same discipline. The source says the pump started at 14:05; the record says 14:50; the span still contains 14:05. The span check fails the record, and the round trip may still pass because the renderer reproduces 14:50 from the wrong row. The two instruments disagree, and the disagreement is the finding.

Span comparison catches surface divergence. It does not catch a shared semantic reading. If the compiler reads "not closed before the pump started" as "closed after the pump started," and the renderer uses the same rule, the span may still contain the word *not*, the record may still carry a negation flag, and the resulting sentence may still be wrong. The span check sees a record that cites the right text. It cannot see that the record interpreted the text the same wrong way on both sides.

The second reader

The instrument that covers a shared parse is an independent reader. Independence is a property of the method, not of the vendor. A second model prompted with the same rules is a second instance of the same bias. A different parser, a different representation, a different language, or a human reading the span with the record beside it can be independent; a second copy of the first reader cannot.

The test for independence is a perturbation. Damage the ledger deliberately: change a role binding, drop a participant, reverse a polarity. Hand the damaged ledger to the rehydrator. If the output stays faithful to the original despite the damage, the rehydrator is reading the source or the fixture, and every score it produced is void. If the output changes in the way the damage predicts, the rehydrator is reading the ledger.

The second reader is expensive, and it is sampled rather than run on every record. The sample should be chosen to include the constructions the compiler is most likely to misread: nested reports, deontic modals, quantifier scope, negation under report, temporal prepositions. A disagreement between the first and second reader is a finding, not a tie to be averaged. It names a record that needs review.

The multilingual case makes independence harder. If the compiler and the reader share a translation convention, a mistranslation can survive both. The independent reader should work in the target language, not in the compiler's pivot, or the test repeats the convention it was meant to check.

An independent reader can also be wrong. Its disagreement is a finding, not a verdict. A third reader or a human with the span adjudicates, and the disagreement is recorded rather than averaged away.

The sample size is a judgment. A small sample catches a gross shared parse; a larger one catches rarer constructions. The sample should be reported with the result. An unreported sample is a claim without a denominator.

The objection

The strongest objection is not that the numbers are inflated. It is that the instrument is self-referential, and there is a case that shows it. A fresh document contained the sentence “Call the vendor if the seal leaks.” Extraction took the adverbial clause’s subject, *the seal*, as the matrix patient. Rehydration rendered “the seal Call the vendor.” Every field check passed, because extraction and rehydration were using the same erroneous structure. The error was found by reading fresh output, not by a gate.

Put in its strongest form: the round trip compares two components derived from one parse, so it measures agreement. Where the parse is wrong, agreement is guaranteed. Self-consistency can pass for the wrong reason, and no fidelity score repairs a model of language that is mistaken at the root.

That objection cannot be answered, only bounded. The defenses are procedural: fresh probes, adversarial review, mutation-tested checkers, and an independent extractor that does not share the first parse. The claim shrinks to what the evidence supports. Rehydration proves the ledger is faithful to itself and to its source under a shared reading. It does not prove the shared reading is right.

The surviving claim is still substantial. The round trip is the only instrument in the stack that asks the inverse question, and the inverse question is the one that finds omissions. A writer never notices what it failed to write.

The instrument stack

No single instrument covers the error classes. The stack is the proof.

Instrument	Catches	Misses
Round trip	inconsistent loss, role or stance changes, dropped nesting, wrong time or polarity	a parse shared by compiler and renderer
Span check	surface divergence, wrong offsets, missing words or numbers	a shared semantic misreading that still cites the right span
Independent reader	a shared parse, a wrong sense, a stance the first reader normalized	cost, sampling error, its own bias

The matrix has an uncovered cell: a shared semantic misreading that the independent reader also shares. The stack reduces that risk; it does not eliminate it. The honest claim is bounded. Rehydration proves that the ledger is faithful to itself and, with the span check, to its source under a shared reading. It does not prove that the shared reading is right.

The protocol

A document passes through five steps.

1. Freeze the source and record its hash. 2. Compile it into the ledger. 3. Rehydrate from the ledger with no access to the source. 4. Recompile the rehydration and compare the two ledgers on structure, not wording. 5. Run the span check on every claim and the independent reader on the sample.

A pass requires all three instruments to agree and the sample to be named. A green round trip alone is not a pass. A span check alone is not a pass, because it cannot see a shared reading. An independent reader alone is not a pass, because it is sampled and expensive and may share the bias it was meant to catch.

A document that fails one check is not partially kept. The loss is named, and the record returns to the compiler or the renderer with the instrument that found it. The record is also the audit trail: a later change to the ledger can be compared against the differences recorded here.

The pass is scoped to the document and the sample. It is not a certification of the compiler, and it does not transfer to a different genre or language without a new sample.

The sample should include a document the compiler was not built around; a suite of familiar documents measures the fixture, not the compiler. The instrument's own version belongs in the record: a check that changes between runs cannot be compared across runs.

The result is recorded per document: the claim count, the structural differences, the span disagreements, the reader sample, the unresolved cases and the versions of the instruments. The record is the evidence. A verdict without the instrument that produced it is a rumor with a timestamp.

The cost is real. The round trip is cheap enough to run on every document; the span check is cheap when the offsets are maintained; the independent reader is the expensive one and must be sampled. The stack is not a guarantee. It is the smallest set of instruments that can name the errors the others cannot see.

The budget should be stated before the test, not after. A document set that can afford only the round trip gets a weaker claim than one that can afford the span check and a sample of readers. The difference belongs in the record, because a reader who does not know which instruments ran will read the verdict as stronger than it is.

A pass is evidence, not a certificate. The record says which instruments ran and which did not, and that is the honest boundary.

What would refute this

Three results would break the claim.

First, a failed perturbation. Give the renderer a deliberately damaged ledger and get the original sentence back. That would mean the renderer is reading the source or the fixture, and every score it produced is void.

Second, hard-case parity. Assemble a set of nested reports, deontic modals, quantifier scope and negation under report, and show that faithful and collapsed renders are indistinguishable to the checks. The deontic pair is already catchable, which is evidence for the instrument; the untested constructions are where the claim is thin.

Third, homogeneity. If every case in the sample is uniform in the dimension under test, a green suite proves nothing about that dimension. A multi-paragraph span-tiling defect survived a fully green fixture suite because every fixture had zero blank runs.

What the proof is not

Rehydration is not a proof of source truth, compiler correctness or answer quality. It is a proof about the inverse: the ledger can produce the sentence again and cite the span it came from. A green loop over a wrong parse is a finding about the limit of the test. A failed span over a correct record is a finding about the offsets. The test is diagnostic, not legislative.

Rehydration is necessary and not sufficient. It answers whether the ledger can say the sentence again. It does not answer whether the sentence was understood. The difference is the difference between a record that is internally faithful and a record that is true to its source, and only the stack can narrow the gap.

A record that can be said again is a record that can be asked; the next chapter walks it.

23 · Answering from the Ledger

An answer can arrive as text alone, or as text with the claims that support it. A model produces the first. A ledger produces the second, and the claims are the license. The difference is not fluency; it is whether a reader can check the answer against the record.

The walk

The engine resolves identifiers, follows roles and links, walks groups, compares times, applies declared rules, and returns a result set beside the record of how it got there. It does not predict tokens and it has no weights to adjust. The model above it may translate a question into a plan and render the result into prose. Below that line, nothing probabilistic enters.

The engine's operations sit in three layers.

Layer	Representative operations	What it can do
Query	resolve an entity, follow a role, follow a link, walk a group, extract a time, check cardinality	exact retrieval; no similarity threshold
Derivation	chain, transitive closure, compare time, resolve identity, group, filter by stance	rule-based inference over what was retrieved
Reasoning	apply a trust lens, apply the governance profile, detect contradiction, rank by evidence, resolve to preferred	policy and epistemic ordering

Two operations carry more weight than their names suggest. A stance filter keeps only claims with a specified standing, and an evidence rank orders corroborated above asserted above reported above disputed above hypothetical. The rank is a list, not a probability. The filter is a membership test over a closed vocabulary. A lens applied at query time changes which claims enter an answer without rewriting any, and the lens is recorded in the plan, so the same question under a different lens is a different, replayable query.

The operation count is a taxonomy, not a build inventory. A list of twenty-three operations is an argument about what the engine must be able to do; it is not evidence that all twenty-three exist in one implementation.

The layers are deterministic by declaration: the same graph, the same inputs and the same plan produce the same output. The translator that turns a question into a plan is a model and sits outside that guarantee. The plan is data, and a hand-written plan can be run without the translator. That separation is what makes the engine replayable, and it is also the first thing to test.

What a walk is not

A walk is not a search. A search ranks candidates; a walk follows typed links and returns a claim or a refusal. A search can return adjacent text with no claim behind it. A walk cannot: if the link is missing, the walk stops.

A search returns text; a walk returns a claim. The claim is the unit of the walk.

A walk is also not a proof: it shows the claims it used, not that they are true. A license names where the engine looked and does not decide whether the claim is true.

The difference matters at the desk. A search result is a place to start reading; a licensed answer is a place to stop checking. A refusal is still a result.

The licensed answer

Here is an example:

```
question: Who approved order 44021?

walk:
 1 resolve order 44021 -> ORDER(44021)
 2 follow role: approved_by -> APPROVE(claim 8871, agent=A. Rao)
 3 follow link: authority -> GRANT(12, refunds < 5000)
 4 compare time: approval 2026-04-11 -> inside grant window

license: [8871, 44021, grant 12]
cost: 3 links, 2 joins
answer: A. Rao approved order 44021 on 2026-04-11, under grant 12. ```

The answer is a sentence. The license is the finding. A reader who disputes the answer
 ↪  has three places to stand: the approval claim, the order it names, or the grant
 ↪  that gave the approver authority. Each is a row with an
 ↪  identifier\index{identifier}, not a paragraph the engine wrote.

The license is printed with the answer, not described after it. If the answer appears
 ↪  alone, the reader has a sentence and no way to tell whether it came from the store
 ↪  or from a model's memory. The pair is the product.
```

The license table

The desk page is a table, not a paragraph.

Field	Content
Answer	the sentence the engine returns
Claims	the identifiers the walk used
Operations	the steps, in order
Links and joins	the traversal count
Cost	the count stated with the answer
Status	answered, partial or refused

A reviewer reads the table before the sentence. If the claims column is empty, the answer is a sentence without a finding. If the status is refused, the missing-link row is the useful part.

A question with a comparison

A second question shows the walk doing something a lookup cannot.

```
question: Did the vendor ship the parts before the audit?

S1 ship AGENT=vendor PATIENT=parts TIME=2026-03-04 tense=past
 ↪  stance\index{stance}=asserted
S2 audit THEME=shipment TIME=2026-03-11 tense=past stance=asserted

plan:
 1 resolve entity "the vendor" -> E_vendor
 2 follow role S1.AGENT = E_vendor -> {S1}
 3 follow role S1.PATIENT = parts -> {S1}
 4 resolve concept "the audit" -> {S2}
 5 extract time {S1} -> 2026-03-04
 6 extract time {S2} -> 2026-03-11
 7 compare time 03-04 BEFORE 03-11 -> true
 8 conclude -> yes, with S1 and S2 named
```

Every step records the operation, its input, its output, the claim it read and its duration. A reader who disputes the answer has a specific place to stand: the time on S1, the ordering rule in step seven, or the entity resolution in step one. None of those is a sentence the engine wrote.

The same plan without a time on S1 does not produce a hedged answer. The time extractor returns nothing, the engine emits a gap of type `TIME_NOT_FOUND` naming S1, and the conclusion takes its third slot: the temporal ordering cannot be determined. The

result still carries the two claims and their roles, because a partial result and a refusal are different outcomes.

The three outcomes

Every question has three outcomes: yes, no and unknown. The plan writes all three before execution, so the third slot is not an exception thrown after the fact. A system that can only answer yes or no will produce a no when it means “I did not find it,” and the difference between those two is the whole reason to build the machine.

The three outcomes are also a design constraint on the translator. A model that turns a question into a plan must be able to express the unknown branch, or the engine will be forced to choose between a false yes and a false no.

The refusal

Ask the same engine about order 44022:

```
question: Who approved order 44022?
walk: ORDER(44022) -> no approved_by link -> no APPROVE claim
result: REFUSE_NO_LICENCE
missing: an approval claim for 44022
resolves: add the claim, or ask a question the store can answer


The refusal\index{refusal} is an answer-shaped object with a status, the missing link
 ↪  and what would resolve it. It is not an error and not a failure of the engine. It
 ↪  is the engine saying that the walk did not reach a claim that entails the answer.

A system that can only answer yes or no will produce a no when it means "I did not find
 ↪  it." The difference between those two is the reason to build the machine. The third
 ↪  value is written at plan time, before the engine knows which branch it will take,
 ↪  so the refusal\index{refusal} is a designed outcome rather than an exception thrown
 ↪  after the fact.

A partial result and a refusal\index{refusal} are not the same thing. If the engine
 ↪  finds the order and the approval but cannot find the grant, it can return the two
 ↪  claims it did establish and a refusal that names the missing authority. The reader
 ↪  gets what survived and the exact gap, which is more useful than a single word.

A refusal\index{refusal} is a result.

## What a license is not

A citation points at a document. A license points at a claim and at the operation that
 ↪  used it. The difference matters when two answers disagree. Citations can be
 ↪  compared only by reading both documents again; two traces can be compared step by
 ↪  step until the divergence appears at a numbered operation.
```

```
The trace records, for every step, the operation, its input, its output, the claim it
 ↪  read and the time it took. A trace is not a proof that the claim is true. It is a
 ↪  record of what the engine did with the claim. That record is what makes the answer
 ↪  replayable: run the same plan against the same graph and the same inputs, and the
 ↪  same result follows.

The engine can also emit the plan rather than the answer alone. A reader who does not
 ↪  trust the implementation can run the plan against their own endpoint. The trace
 ↪  travels with the query, which is what makes the answer checkable by someone who did
 ↪  not build the system.

A different lens produces a different plan. Ask the same question with a stricter
 ↪  evidence rank and the walk may exclude a reported claim and return a
 ↪  refusal\index{refusal} where the earlier plan returned an answer. The two plans can
 ↪  be compared, and the divergence is a policy decision, not a bug.

## The query pack

The plan and the trace are portable. A query pack contains the plan, the claim
 ↪  identifiers it expects, the lens and the cost. A reader can run it against a copy
 ↪  of the graph, compare the result, and see whether the difference comes from the
 ↪  data or from the query.

This is why the engine emits a query pack rather than pretending to be a store. The
 ↪  pack is the unit of exchange. It lets a party who did not build the system
 ↪  reproduce a finding without accepting the system's word for it.

The plan is a document. A reader can print it, annotate it and run it elsewhere. That
 ↪  is what makes the answer portable.

## The cost of a walk

The cost of an answer is the links it touched and the joins it performed. The count is
 ↪  stated with the answer, because a query that touches ten thousand links can be
 ↪  correct and useless. A walk that names the claims it used has a bound; a walk that
 ↪  scans the store does not.

The count is not latency. A fast query that names no claims has no audit surface, and a
 ↪  slow query that names five claims may be the better instrument. The cost is the
 ↪  price of the license, not the price of the hardware.

The count is the audit surface.

The bound is also a design constraint. A question that requires an unbounded search is
 ↪  not a question the engine should answer by walking; it is a question for retrieval,
 ↪  or for a different store. The engine's value is that it can say what it touched.

A query for every order a customer ever placed touches the customer, every order, every
 ↪  approval and every grant. The count grows with the customer's history, and the
 ↪  answer is only as good as the oldest claim in the chain. A query for one order
 ↪  touches three links. Both are queries; only the second has a license a reader can
 ↪  inspect in a minute.
```

```
The engine should refuse a question whose cost cannot be bounded by the links it names.
 ↪  A refusal\index{refusal} that says "this would require scanning the store" is more
 ↪  honest than a scan that returns a result and hides its size.

## Inference and its scope

Three products hide under "query the graph, " and only one of them needs an ontology.

| Product | What it is | Example |
| ------------------- | ------------------- | ------------------- |
| \allowbreak{}—\allowbreak{}- | \allowbreak{}—\allowbreak{}- |
 ↪  \allowbreak{}—\allowbreak{}- |
| Slice | a subset of stored claims | everything the auditor said in this document |
| Lookup | one stored fact, if it was accepted | who acquired Acme |
| Inference | a conclusion never written as its own claim | a gas range is an
 ↪  appliance, via range, stove, appliance |

Slices and lookups are queries. An inference needs a taxonomy and often a part-whole
 ↪  relation, and those have to come from somewhere: the document, a project catalog,
 ↪  an external ontology loaded on purpose, or a curated file. A reasoner cannot invent
 ↪  the statements it follows. If part-whole reasoning is wanted, someone must have
 ↪  written the part-whole statements.

The scope rule is as important as the taxonomy. Reason over accepted content plus the
 ↪  ontology graph, never over claims that sit inside a report, a denial, a hypothesis
 ↪  or a question. Feed a reasoner "John said the factory is bankrupt" as a plain fact
 ↪  and it will infer a bankruptcy the article only reported. The stance\index{stance}
 ↪  field exists to prevent that, and an inference that ignores stance defeats the
 ↪  storage model it depends on.

## What the engine does not answer

The engine does not answer questions that require world knowledge outside the ledger,
 ↪  causal claims the store never recorded, predictions about the future, or judgments
 ↪  about whether a policy is just. It can walk the claims it has and return a
 ↪  refusal\index{refusal} when the walk does not reach one. That is not a failure of
 ↪  the engine; it is the boundary of the store.

The boundary is the reason the refusal\index{refusal} vocabulary matters. A refusal
 ↪  that says *no claim* is different from a refusal that says *the claim is reported,
 ↪  not accepted* and different again from *the claim is accepted but the time is
 ↪  missing*. The three send the reader to different places.

The boundary is the store.

## The objection

A refusal\index{refusal} is cheap to produce and expensive to live with. A checker that
 ↪  refuses most questions has a low error rate and no utility. The evaluation must
 ↪  report refusals with coverage, not only errors.
```

```
The strongest objection is not that the engine is slow or that the operations are hard
 ↪ to implement. It is that traversal buys nothing epistemic. A careful retrieval
 ↪ pipeline, or a careful model with the documents in context, can produce the same
 ↪ answer with a citation, and the citation is what a reader actually uses.

Put more sharply: the engine's knowledge is exactly the compiler's output. If
 ↪ extraction put the wrong time on the order, the engine will answer confidently and
 ↪ hand over a trace that documents its own error. Auditability then does real damage,
 ↪ because it converts an extraction mistake into an institutional finding with a
 ↪ provenance trail attached. The trace is a record of the engine's reasoning, not a
 ↪ warrant\index{warrant} that the reasoning rests on true rows.

The trace is not truth. It is a map of where the engine looked. That map is valuable
 ↪ when the store is right and when the store is wrong, but it cannot tell the
 ↪ difference on its own.

Part of this must be conceded, and the concession is specific. The engine does not
 ↪ prove its claims true. It localizes error; it does not prevent it. Determinism
 ↪ stops at the graph edge: the translator that turns a question into a plan is a
 ↪ model, the compiler that filled the graph is a model plus gates, and neither is
 ↪ covered by "same graph, same inputs, same outputs."

What remains is still worth building, but it needs to be stated at the right size. The
 ↪ advantage is refusal\index{refusal} and replay. A generated answer cannot reliably
 ↪ say "this is not in the store" and cannot be run again to see whether it comes out
 ↪ the same. A walk can do both, and when the store is wrong the walk says where. That
 ↪ is an epistemic advantage in the narrow sense that matters to an auditor: not a
 ↪ guarantee of truth, but a location for doubt.

## The test

The test is planted error, and it is cheap. Insert a false claim into the ledger, one
 ↪ that changes the answer to a question the engine can otherwise answer, and ask it.
 ↪ Then measure two things: whether the trace names the injected claim, and how long a
 ↪ competent reader takes to find it from the trace versus from the source documents.

If the trace names the right claim only when the question is easy, the license is
 ↪ decoration. If readers find the error no faster with the trace than without it, the
 ↪ engine's advantage disappears, and the honest description of it becomes a
 ↪ deterministic database with a query log.

The planted error is the cheapest test in the book. It needs one false claim, one
 ↪ question and one reader with a clock. If the trace names the claim, the license
 ↪ works. If the reader finds it faster with the trace than with the documents, the
 ↪ engine earns its cost.

A second test covers the determinism claim directly. Write a plan by hand, disable the
 ↪ translator, and run it. If the engine cannot execute a hand-written plan, the claim
 ↪ that it does not depend on a model is false.
```

```
A third test is comparative. On a question set with known answers, compare licensed
 ↪  answers against retrieval plus generation on two axes: answer accuracy and time to
 ↪  localize an error. If both are ties, the engine has an economic advantage at best,
 ↪  and the argument for it must be made on maintenance cost rather than epistemics.

## An auditor's engine

The engine answers only questions the structure can express. A question that needs
 ↪  world knowledge outside the ledger gets a refusal\index{refusal}, not an inference.
 ↪  That is a limit, and it is the refusal path working as designed. A system that
 ↪  answers beyond its model is guessing.

The desk page is the license table: answer, claims traversed, links used, joins
 ↪  performed, cost, and the refusal\index{refusal} forms. A reader who wants the
 ↪  answer without the table is asking for a sentence, not a finding.

This engine is not a faster model. It is an auditor's engine: it walks typed structure,
 ↪  returns the claims that license the answer, and refuses rather than generates when
 ↪  the walk fails. Its advantage is not that it knows more. It is that it can show
 ↪  where it stopped.

A licensed answer is not a free answer; the next chapter prices it.


```{=latex}
\markboth{The Machine}{}
\chapter[24 · The Economics of Compilation]{24 · The Economics of Compilation}
```

An answer has a bill, and the bill is not the document. A document can be embedded once and never read again; an answer is produced, defended, corrected and reused. Pricing compilation by the document measures the wrong thing, and it makes retrieval look unbeatable for a reason that has nothing to do with meaning.

The stack

Compilation costs more per document than embedding. That is not controversial. Embedding is one pass, no adjudication, no review queue and no schema. Compilation adds entity grounding, role assignment, ambiguity handling, validation and materialization, and then asks a person to defend part of it.

The bill has four lines.

Line	What it pays for	Share of the bill (designed)
Model calls	reading each sentence, proposing senses, roles, force and links	roughly half

Continued on next page
```

(Continued)

Line	What it pays for	Share of the bill (designed)
Gates	schema, span, type, dependency and stance checks	roughly one tenth
Review	a person reading the proposals that matter	roughly one third
Storage	claims, spans, versions, traces	the smallest line

The shares are designed, not measured. A reader's prices will differ. The shape is what matters: calls and review carry the bill, and storage is not the cost that decides whether the architecture is affordable. A team that budgets for storage has budgeted for the wrong line.

A storage-first budget will approve the wrong architecture because the line it watches is the one that grows most slowly, while the lines that decide the outcome are the ones it never sees.

A budget has owners and dates. The cost stack has lines and no owner until someone assigns one. The template is what turns the stack into a budget: each line gets a rate, each rate gets a person who can be asked about it, and each tranche gets a date. Without that, the stack is an estimate with a decimal point.

The difference between a cost estimate and a budget is not the arithmetic; it is the person who owns the line and the date by which the line must be defended.

Calls scale with claims, not documents. A document with eight claims needs eight readings and a document with eighty needs eighty, so the per-document figure hides a factor of ten. Review scales with the claims a person must defend, which is a smaller number and a more expensive hour. The gate is cheap per proposal and expensive to build once.

The first document

The first document is expensive for reasons that do not repeat. The model weights load, the schema loads, the gate loads, the review queue is empty and the first reviewer is learning the format. The second document in a resident process pays for calls and review, not for startup.

A cost estimate taken from a fresh process is an estimate of startup. Report both. A single number hides which one was measured, and the difference can be the difference between a program that looks impossible and one that looks merely expensive.

The same distinction applies to the review queue. The first review of a new genre is slow because the reviewer is building a rubric; the hundredth is faster because the rubric exists. The stack should be reported at the steady state the program expects, with the cold start named separately.

The unit

Two systems can answer the same question and return different objects. Retrieval returns text that is relevant; compilation returns a claim with a license. The price per document favors retrieval by a wide margin. The price per answered question narrows the gap. The price per audit reverses it.

Unit	Retrieval	Compilation
Per document	low; one pass	high; calls, gates, review
Per answered question	low, if the answer is in the text	moderate; the walk is cheap once the store exists
Per audit	high; a person rereads the source	low; the trace names the claims

The audit column is the one buyers forget. An answer that must be defended costs the defender's time, and the defender's time is spent finding the passage, checking the interpretation and reconstructing the reasoning. A license moves that work into the record. It does not eliminate it; it makes it inspectable.

A team that never has to defend an answer should not compile. That is the stop rule, and it is not a concession to the competition. It is the unit working.

A buyer who compares the two systems by the price of a document will conclude that compilation is impossible, and that conclusion will be correct for the buyer who never has to defend an answer.

A simple comparison makes the unit concrete. Suppose a compiled document costs two call-units and a fifth of a review-unit. A question over that document costs three links and no model call. A retrieval answer costs a search and a reader's minute, and the reader's minute is the expensive part. At one question per document, retrieval wins. At twenty questions over the same document, the compiled store amortizes the build, and the retrieval reader pays the minute twenty times. The unit, not the architecture, decides which number matters.

One thousand documents

Here is an example:

```
corpus: 1,000 documents, 8 claims each -> 8,000 claims
compile: 2 model calls per document + gates
review: 10% of claims -> 800 claims
compare: embedding: one pass per document, no license
unit: cost per answered question; cost per audit
stop: compile the 200 documents whose failure has a consequence
 embed the other 800
```

The compiled tranche is 200 documents and 1,600 claims. The reviewed subset is 160 claims. The other 800 documents are embedded and retrieved. The program does not pretend that the embedded documents are governed; it says which tranche is compiled and which is not.

The arithmetic is deliberately simple. A real program will have variable claim counts, a review rate that changes with genre, and a correction rate that is discovered rather than planned. The template is the point: it forces the four lines and the two units into one table before the first model call.

The stop rule

Compilation stops where the consequence does not justify it. The rule has four questions.

1. Would a wrong answer be quoted in a meeting, an audit or a filing? 2. Must someone defend the answer to a person who did not build the system? 3. Is the claim corrected often enough that a write is cheaper than a retrain? 4. Is the claim reused across questions, so one compiled row saves many reads?

A yes to the first two is enough to compile. A yes to the third or fourth strengthens the case. A no to all four sends the document to embedding, where it belongs.

The rule is a judgment, not a threshold. A team can set a price per review hour and a price per error, and the judgment becomes arithmetic, but the arithmetic is local. A bank and a research group will draw the line in different places. The stop rule prints the questions, not the line.

The 200-document tranche in the example is chosen by this rule. Orders, approvals and notices are claims whose failure has a consequence; background descriptions are not. The split is not a quality ranking. The embedded documents are still searchable and still useful. They are simply not governed.

Review is an escalation budget. Asking a person about every claim spends the reviewer's hour on claims nobody will challenge. The example reviews ten percent: the claims whose failure would be quoted. The rate is a design choice, and it should be argued, not inherited. A system that asks seventy-six thousand questions has already lost, whatever the latency per call; the fix is a better filter, not a faster queue.

The filter is where the review money is saved. Rank the claims by consequence and by uncertainty, review the top of the list, and leave the long tail to the gates. A gate can

reject a malformed claim without a person. It cannot decide whether the claim matters, which is why the ranking, not the review, is the design problem.

Where the money goes

The correction is where the bill concentrates. Correcting a fact in the ledger is a write: a new row, a review event and a timestamp. Correcting a fact inside weights is a retrain: data, compute, an evaluation, and the risk that fixing one fact degrades a thousand others.

The write is not free. It needs a reviewer, a reason and a version. But it is bounded, and it leaves a record. The retrain is unbounded in the sense that matters: no one can say which other answers moved.

This is the economic argument for compilation, and it is narrower than "better answers." Compilation buys correctability. A corpus that is read once and never corrected does not need it. A corpus whose errors are corrected by someone who must defend the correction does.

The correction budget is the strongest case for compilation and the easiest to overlook, because it arrives after the purchase. An order's approval date is wrong. In the ledger, the correction is one row, one review event and one timestamp. In weights, the correction is data, compute, an evaluation and the risk that fixing one date moves a thousand other answers. The write is bounded. The retrain is not.

The write leaves a record that a later reader can inspect, while the retrain leaves a change in behavior that no one can attribute to a particular fact.

Who pays

Three budgets are in play: ingestion, review and correction. The buyer approves the first, the reviewer pays the second in hours, and the auditor consumes the third in trust. They are not the same person, and a program that prices only the first has hidden two-thirds of the bill.

If no one owns the review budget, the cost is hidden in someone's week. If no one owns the correction budget, the cost is hidden in a retrain that happens once and is never attributed. The template names a line for each, which is the minimum a program can do before it commits.

A program that prices only ingestion and leaves review and correction to whoever happens to be available has not priced the system it is building, and the missing lines will arrive after the commitment.

The objection

The strongest objection is that at equal accuracy retrieval is cheaper by so much that the comparison is embarrassing. One embedding pass, no adjudication, no gates. If cost per correct answer is higher for the compiled system at equal accuracy, the architecture is a research program with a book attached.

Most of that should be conceded. At equal accuracy, retrieval wins on ingestion, always, and by more than a constant factor. The compiled system buys a kind of accuracy that answer-level scoring does not count: which claim licensed the answer, what its standing was, what would have to change for the answer to change. If the buyer does not value that, the buyer should not pay for it.

The objection is not answered by claiming compilation is cheap. It is answered by counting in the right place. The unit is the answered question that must survive an audit, and the audit is where the license earns its cost.

What would refute this

Take one corpus and fifty questions an operator actually asks. Measure total cost to a correct answer for both architectures, including review at a stated rate, and record accuracy on the same set. If the compiled cost per correct answer exceeds retrieval while accuracy is equal, the architecture is a research program, and the argument above is a preference dressed as economics.

A second result would force a narrower claim. If a resident process removes the startup penalty and the compiled cost per document is still an order of magnitude above retrieval, the correction argument must carry the case alone. A third: if the rule that protects semantic metrics blocks every efficiency change that would make compilation affordable, the architecture can be honest and unmaintainable at once.

If the compiled system is cheaper only because its reviewers are unpaid, the comparison has measured the accounting rather than the work, and the result is a bargain that no one who pays for review can repeat.

The concession

The figures here are designed, not measured. There is no benchmark in these pages, and a reader who wants one should run the template on their own corpus. The claim is about the shape of the stack and the choice of unit, not about a price list.

The template has four rows and two columns:

Line	Per tranche	Per answered question
Calls		

Continued on next page

(Continued)

Line	Per tranche	Per answered question
Gates		
Review		
Storage		

Fill the table before committing. If the per-answer column cannot be filled, the program does not yet know what it is buying.

The bill is per consequence, not per byte. A program that compiles what nobody will ask about has bought storage, not meaning. A program that embeds what it must defend has bought speed and a future argument.

The machine is priced; the next question is what its parts are called, and who may mint them.

PART V
The Address and the Meaning

what the parts are called, and what the names contain.

25 · The Canonical Identifier

A word is not a meaning, and a meaning is not an address. `bank` is six letters. In a public dictionary it carries twenty-five senses, and nothing in the string says which one a document meant; a model with a wider context window can choose a sense, and the choice lives inside the model that made it. A second system cannot inspect the choice, compare it, or dispute it. The alternative is to give each sense an address.

A string and a sense

Two sentences from one document show the problem.

```
The pilot banked sharply to the left.
The far bank was still in sight.
```

The first is a verb: an aircraft changing direction. The second is a noun: the land at the edge of a river. Both use the same six letters. A cache keyed on the lemma serves the wrong sense; a join keyed on the lemma collapses both into one node. Neither failure throws an error. The query returns rows, the join returns a graph, and the wrong answer appears at read time.

Lowercasing, trimming and stripping punctuation do not separate the two. They normalize the string, and the string was never the thing that had two senses. The sense had two senses.

An address separates them:

```
bank.n.financial_institution.public
bank.n.water_land_edge.public
bank.v.aeronautic_maneuver.public
```

The first address names the financial sense, the second the river sense, the third the verb. A system that receives one of those strings knows which sense was chosen without sharing a model, a vendor or a database with the sender.

The join is the point

The two sentences compile to two different predications.

Lemma	Part of speech	Microgloss	Address
bank	n	financial_institution	bank.n.financial_institution.public
bank	n	water_land_edge	bank.n.water_land_edge.public
bank	v	aeronautic_maneuver	bank.v.aeronautic_maneuver.public

The query that separates the river sense from the aviation sense is an equality, not a judgment:

```
WHERE address = 'bank.n.water_land_edge.public'
```

Before the microgloss sits inside the key, the same question is `WHERE lemma = 'bank'`, which returns twenty-five candidate senses per mention. For a document that mentions bank three times, that is seventy-five candidate rows and no identification. With the address, it is three rows, each naming the sense the compiler chose.

The verb and the noun differ in two fields at once, which is why the part of speech is not decoration. Drop it and the verb you `bank` merges with the noun you `bank`.

Four parts

The address has four parts.

Part	Answers	Example
Lemma	which word	bank
Part of speech	which grammatical role	n or v
Microgloss	how this sense differs from its siblings	financial_institution
Namespace	which dictionary owns the sense	public

The lemma scopes the word. The part of speech scopes the grammatical role; drop it and the verb you `bank` merges with the noun you `bank`. The microgloss scopes the sense inside the lemma. The namespace scopes the dictionary and the authority behind the definition.

A namespace is a governance claim, not a decoration. An unowned namespace drifts: two teams add senses under the same prefix, and the prefix stops meaning one dictionary.

The four parts are a contract only when the namespace has an owner who can refuse an entry.

The namespace is a claim

A namespace answers which dictionary owns the sense, and ownership is a claim that someone can be asked to defend. The same lemma can carry different senses in two dictionaries; the namespace is what keeps them from merging. If the namespace is spelled differently by two loaders, the portability it was meant to provide disappears at the first join.

The namespace also carries a version. A dictionary revision can change the definition behind an address without changing the address, and the revision is what tells a consumer whether the sense it depends on has moved. A namespace without a version is a name without a history.

Three clocks

If an address is a decision, it has a time. The system keeps five artifacts with five different clocks, and most identity failures are two of those clocks collapsed into one.

Artifact	Answers	Clock	Change rule
Row identifier	which row	immutable	never reused, never re-pointed
Canonical ID	which sense	versioned	an improvement increments the version
Microgloss	how this sense differs from its siblings	pinned at mint	revised only as a versioned change
Definition	what the sense means	revisioned with the entry	free text, never concatenated
Display string	how it reads to a person	none	derived at export, never stored

The separation is explicit. A microgloss revision does not change the row identifier; the address is a versioned label, not the sole immutable key. The microgloss is pinned at mint time, so two compiles of the same sense produce the same string. A definition lives outside the key because definitions contain periods, and a field must not contain the identifier's own delimiter.

Most identity failures are a collapse of two clocks. A display string stored as an address makes a rendering change into an identity change. A definition concatenated

into the key makes a revision into a new sense. A row identifier treated as a semantic address makes a database key into a meaning claim.

The microgloss inside the key

The microgloss must sit inside the address. If it sits beside an opaque identifier as a comment or a display field, a later process can re-point it without changing the key. The address stays the same, the gloss changes, and every consumer that trusted the address now points at a sense it never agreed to.

Inside the key, a gloss change is a new address. That is the point. The address is a decision with a version, and a version change is visible.

The microgloss is short because shortness is not its job. Its job is to separate the senses that share a lemma inside a namespace. `financial_institution` does not define `bank`; it separates this bank from `water_land_edge`, `memory_chip_row` and `aeronautic_maneuver`. A three-token fingerprint is enough for that job, and a full definition is too long and too punctuation-heavy for a dot-delimited field.

The delimiter rule is well-formedness, not taste. A definition can contain periods: `deposit money in a U.S. bank`, `the approx. value of a holding`, `a building in St. Louis`. Put a definition in a dot-delimited field and the parser cannot tell a separator from punctuation. The field must not contain the identifier's own delimiter.

Identity as a decision

A merge is a claim about sameness. It needs evidence, an owner, a date and a rollback. The receipt is the difference between identity and spelling.

Here is an example:

```
merge: bank.n.financial_institution.public
 ≡ bank.n.fin_bank.public
evidence: three systems, same definition, owner sign-off
date: 2026-04-11
rollback: restore both addresses; re-point consumers to the prior version
```

The receipt does not prove the merge correct. It makes the merge reviewable. A later reader can ask which systems agreed, which definition they used, who signed, and what would undo it.

A receipt has five fields: left address, right address, evidence, owner and rollback. A merge without a rollback is irreversible. A merge without an owner is a rumor. A merge receipt that names the evidence and the rollback turns an identity decision into a reviewable object, which is the difference between a naming convention and a governed address.

Three identities must stay separate.

Identity	Question	What it is not
Sense identity	which meaning	a string match
Referent identity	the same individual	a shared description
Representation identity	the same expression or artifact	a content hash

A content hash identifies bytes. Two expressions can differ syntactically and be logically equivalent; two identical hashes can carry the same text with different provenance. The hash is useful for integrity, not for sameness of meaning.

What the ID does not prove

An address says which sense was chosen. It does not say that the definition behind the sense is correct, that the source was read correctly, or that the claim is true. A legal address can still be wrong.

The definition can also change while the address stays the same. A revision behind an unchanged microgloss points at the same label for a different meaning, which is why an export needs pinned definition revisions and a declared package. An address answers which defined identity is meant; it does not answer what may be inferred from that identity.

The distinction matters at a boundary. Two systems that share an address share a decision about sameness. They do not share a model, a database or a warrant. The address is the precondition for comparison, not the comparison itself.

Duplicates are merges

A duplicate audit of one dictionary found 22,129 duplicate groups covering 45,742 rows. The instinct is to force uniqueness across all of them. Measurement said otherwise.

Group	Count	Share	Action
Orthographic variants	21,647	97.0%	accept the shared address
Different entities	482	3.0%	fix with a merge receipt

The variants are `naive` and `naïve`, `1 Cor` and `1 Cor.`. They should share an address. A blanket dedupe would have destroyed the correct grouping for 97% of the cases and called the destruction a cleanup.

The 3% are different things that happened to collide. They need a merge decision, not a delete. The distinction is the whole job: a duplicate detector that cannot tell a variant from a different entity is a spelling checker with a destructive default.

Uniqueness is not the defect test. The test is whether the colliding rows denote different entities. A collision that denotes the same entity is a variant; a collision that denotes different entities is a defect.

The cost of a wrong address

A wrong address does not throw. A dropped field produces a merge, the merge is silent, and the wrong answer surfaces at read time.

The measured instance is a character fold. A sanitizer converted a microgloss to ASCII and ignored every non-ASCII character, then returned `unknown` when nothing survived. Twenty-seven rows of Shavian lemmas came out as the same address; twenty-four rows of Braille lemmas shared another; fifteen rows of Hangul lemmas shared a third. The result was 22,129 duplicate addresses over 45,742 rows.

A duplicate audit then counted those rows as duplicates, which was a true statement about a row that had already stopped describing itself. A cache keyed on the lemma served the wrong sense. A join keyed on the lemma collapsed every sense onto one node.

The repair was cheap: 1,366 rows changed, harmful collisions driven to zero. What cannot be recalled is the query someone wrote against the merged address, shipped, and described to a customer as correct.

Uniqueness is local

The microgloss owes uniqueness to its siblings, not to the dictionary. It must separate the senses that share a lemma inside a namespace; it owes nothing to the rest of the list. A collision procedure can append the shortest non-overlapping token from each entry's gloss, and a four-character suffix if that still collides.

The measurement that governs the field is counterintuitive. The mean microgloss is three tokens, and almost all are exactly three. They are short because shortness is not the constraint; separation is. A rare token is a good discriminator and usually a poor description, which is why person entries fill with dates: `william_december_11` is discriminative because a date is rare, not because a date is a sense.

A namespace that changes spelling between two loaders makes the address portable in one vocabulary and opaque in the next, which is the failure the field was introduced to prevent.

The duplicate detector

A detector that proposes a merge must classify before it acts. It returns one of three answers: variant, different entity, or unknown. A variant shares the address; a different entity needs a merge receipt or a split; an unknown goes to review.

The detector is useful only if it can say "different" with the same confidence that it says "variant," because the destructive action is reserved for the second answer. A blanket

`DISTINCT` is not a detector; it is a decision to keep one row and discard the evidence for the other.

The test is a labeled pair set. Report precision and recall for variant and for different, and report the unknown rate separately. A detector that is accurate only when it is confident is a detector that has hidden its errors in the unknown bucket.

What the address costs

An address costs a microgloss decision per sense, a namespace owner, and a validator that refuses a missing part. The cost is paid once per sense. The benefit is paid every time a consumer joins on the address instead of guessing from the lemma.

A microgloss that is stable but not discriminative turns the address into a label, and a label that does not separate senses is the normalization problem in a longer string. The cost of the address is paid once per sense, while the cost of a wrong join is paid every time a consumer trusts the merged row, and the second cost arrives without an error message.

A validator refuses an address with a missing part, a delimiter inside a field, or a namespace with no owner. The refusal is the product: an invalid address never enters the store, and the consumer never has to discover the defect at join time. The validator is the cheap part of the address and the part most often skipped.

The cost is also uneven across a dictionary: a word with twenty-five senses needs twenty-five microgloss decisions, while a word with one sense needs one, and the address budget should be planned per sense rather than per lemma. The budget is per sense. A lemma with one sense is cheap.

A dictionary with a hundred thousand senses and an average of three senses per lemma carries three hundred thousand microgloss decisions, and the budget should be sized to that number rather than to the number of headwords.

The objection

The strongest objection is that the four fields are unnecessary. A UUID plus a gloss field is equally portable, since both are strings, and more stable, since a UUID never needs repair and never drifts when a normalization changes. Press harder and the structured key looks worse than neutral: it churned 45,742 rows of audit and 1,366 rows of repair, and its readable form invites a parser to treat syntax as semantics.

The stability charge should be conceded. The four fields are not the stability device; a separate immutable row identifier is. The address is a label, and the label is allowed to change when the sense changes.

The portability charge mistakes the axis. Both forms are equally portable as bytes. The axis that matters is predictability without a lookup: can two parties who have never met say something useful about an address before either queries a registry? With four fields, a disagreement is locatable in the field that differs: a verb against a noun, one

dictionary against another. With a UUID and a gloss, a disagreement is invisible until someone compares gloss strings under a normalization that lives outside the schema.

The merge hazard should also be conceded, then bounded. A readable key tempts a string match into an identity claim. The project names the temptation and forbids it: broader, narrower, related and equivalent relations must not all become identity substitutions. A UUID would suppress the temptation rather than answer it. What prevents the merge is governance, and governance needs something readable to govern.

The scope should be conceded too. Inside one store with one owner, a UUID plus a pinned gloss is adequate. The four fields earn their cost at a boundary where the other party has no database to call.

The concession

The addresses are long and ugly. `william_december_11` and `issues_forth_point_which` are not elegant. The ugliness is the precision: a short, readable token that separates the sense is doing its job, and a beautiful token that does not separate is decoration.

The namespace is the weakest part in practice. Teams disagree about how to spell it, and a namespace that changes spelling breaks the portability it was meant to provide. The address is only as stable as the namespace owner's discipline.

What would refute this

If an opaque identifier plus a pinned gloss, a published normalization, a validator and a namespace field matched the four-field address at cross-system comparison and retrieval-key prediction with no shared registry, the structure would be ceremony. The list is worth reading closely: the pinned gloss is the microgloss under another name, the namespace is the fourth field, and the normalization and validator are the grammar's two jobs.

The measurement that would settle it has not been made. What matters is consumption, not legality. If join traffic ran entirely through opaque identifiers and lookups, with the readable fields read only by people, the address would be a comment with a validator attached. The project counts collisions, legality and repair; it does not count how often an address field is the thing a query actually uses.

That gap is the weakest joint in the argument, and it is a gap rather than a refuted claim. A team that adopts the four fields should instrument the joins: count the queries that filter on the address and the queries that filter on the opaque key. If the address is never used, the team has built a label with a validator, and the label is not doing the work the design claims.

An identifier is an address for a sense; the next question is what makes the address precise.

26 · The Microgloss

The gloss is the part of the address that does the separating. Without it, the identifier is a lemma with punctuation. With it, the address names a sense, and a second system can tell which sense without sharing the dictionary that minted it.

Inside the address

A canonical identifier has four parts: lemma, part of speech, microgloss and namespace. The microgloss sits between the grammatical role and the dictionary that owns the sense.

```
approval.n.formal_acceptance.public
approval.n.favorable_regard.public
```

The first address is the act of approving. The second is a feeling of approval. Both share the lemma and the part of speech; the gloss is the only field that separates them.

The gloss is not a definition. A definition explains a meaning to a person; a gloss separates a sense from its nearest neighbor inside a key. A definition can contain periods, commas, articles and subordinate clauses. A key cannot, because the key's delimiter has to remain unambiguous. A definition in an address is a paragraph in a key.

The gloss is not a caption

A caption describes a thing from the outside. It can be moved, shortened or translated without changing what it names. A gloss is part of the address, and moving it changes the address. That is the difference between a comment and a key.

A caption that is wrong is a bad description. A gloss that is wrong is a bad identity. The second is more expensive, because every consumer that joined on the address inherited the error.

Distinguish, not describe

The test is the neighbor.

Sense	Gloss	Fits the neighbor?	Verdict
Formal acceptance by an authorized agent	formal_acceptance	no	works
A favorable opinion	favorable_regard	no	works
The act of approving	act_of_approving	yes, both	fails

`act_of_approving` is a description that fits every sense of *approval*. It restates the lemma with a suffix and separates nothing. `formal_acceptance` names the difference: one sense is an authorization, the other is a feeling.

The test is comparative, not absolute. A gloss does not have to be a good definition. It has to fail to fit the neighbor. If it fits the neighbor, the gloss has not done its work, and the address still names two senses with one string.

Here is the failure in a sentence:

```
bad: approval.n.act_of_approving.public
good: approval.n.formal_acceptance.public
```

The bad address is legal. It has four parts, a valid namespace and a gloss. It is also useless, because the gloss would fit the other sense. Legality is not the test.

The neighbor set

The neighbor is not one sense. It is every sibling: the senses that share the lemma, the part of speech and the namespace. A gloss has to separate the sense from all of them, and a gloss that separates one neighbor while fitting another has still failed.

Approval has at least three siblings in the public dictionary: the act of approving, a favorable opinion, and the formal authorization that a grant conveys. A gloss like `formal_acceptance` separates the authorization from the opinion. A gloss like `approval` separates nothing.

The test is local, not global. The gloss owes nothing to a different lemma, a different part of speech or a different dictionary. It owes everything to the siblings it sits beside.

The universal gloss

A gloss that fits every sense disambiguates none. The failure has a recognizable shape: it repeats the lemma (`act_of_approving`), names a category so broad that every sense belongs (`a kind of thing`), or describes the word's grammar rather than its meaning (`a noun`).

The universal gloss is worse than an empty field, because it looks like work. A reader sees a gloss and assumes someone chose it. The choice is real, but it carries no distinction, and the collision passes every structural check.

The test is always the same: put the gloss beside the nearest neighbor and ask whether it fits both. If it does, rewrite it or accept that the two senses are one.

The bad gloss taxonomy

Four shapes account for most failures.

Shape	Example	What it does
Tautology	act_of_approving	repeats the lemma with a suffix
Category	a_kind_of_thing	fits every sense in the language
Grammar	a_noun	describes the word, not the meaning
Definition	the_act_of_formally_accepting	too long and too punctuation-heavy for a key

Each shape is legal. Each passes a structural validator. Each fails the neighbor test, because the gloss would fit at least one sibling.

The taxonomy is useful because it names the repair. A tautology needs a differentia; a category needs a narrower class; a grammar label needs a meaning; a definition needs a shorter phrase.

Evidence and revision

A gloss is a decision, and a decision can be wrong. The wrongness is not a scandal; it is the reason the gloss has a receipt.

A revision records the old gloss, the new gloss, the evidence for the change, the owner and the date. The old address remains readable for the rows that used it; the new address is a new version. A revision that overwrites the old gloss without a record has hidden the change, and a consumer cannot tell which sense it agreed to.

The gloss is pinned at mint. Two compiles of the same sense produce the same string. A later improvement is a versioned change, not a silent re-spelling. The pin is what makes the address stable enough to compare; the revision is what makes it correctable.

The receipt does not prove the new gloss better. It makes the change reviewable. A later reader can ask which evidence supported the rewrite, who signed it, and what would undo it.

The gloss and the definition

A definition sits beside the address, not inside it. The address says which sense; the definition says what the sense means. An export carries both, and the definition carries its own revision.

A definition that lives inside the key would force every revision to become a new address, and every consumer that only wanted the old sense to migrate for a change it did not request. The separation keeps the address stable while the explanation improves.

The gloss is not a shorter definition. It is a different instrument with a different job. The definition explains; the gloss separates. A reader who wants to understand the sense reads the definition; a system that wants to join on the sense reads the address.

The uniqueness test

Two senses may not share a gloss inside the same lemma, part of speech and namespace. The check is a uniqueness query over siblings.

```
group by: lemma, part of speech, namespace
count: distinct glosses
flag: any group with fewer glosses than senses
```

A collision is a row pair with the same lemma, the same part of speech, the same namespace and the same gloss. The fix is mechanical: append the shortest non-overlapping token from each entry's definition, and if that still collides, add a short suffix. The fix is not a delete. Two senses that collide are two senses; the address is what failed.

A collision

Here is an example. Two entries both carry `approval.n.formal_acceptance.public`: one for the act of an authorized agent, one for the formal authorization a grant conveys. The uniqueness query groups by lemma, part of speech and namespace, counts distinct glosses, and flags the group because it has two senses and one gloss.

The repair is a rewrite, not a deletion. The first entry keeps `formal_acceptance`; the second becomes `grant_authorization`. If both still collide, the procedure appends the shortest non-overlapping token from each definition, and then a short suffix.

Two entries that share an address and a gloss are not two spellings of one sense; they are two senses that the identifier failed to separate, and the failure is visible only because the uniqueness query looks at siblings.

The repair receipt records the old gloss, the new gloss, the evidence, the owner and the date. A consumer that joined on the old address can see why the address changed and which rows moved. The receipt is the difference between a revision that a consumer can audit and a silent re-spelling that changes the meaning behind an address without changing the address.

The collision rate is reported, not zeroed. A dictionary with a reported rate can be audited; a dictionary that claims zero collisions has either solved the problem or stopped counting.

The test is cheap

The uniqueness query is one group-by and one count. It runs on every publication and costs less than a review hour. A dictionary that does not run it is choosing not to know.

Length and grammar

A gloss is short because the key is short, not because meaning is short. Three to six words is the working range. The mean in one dictionary is three tokens, and almost all glosses are exactly three.

Shortness is not the constraint; separation is. A rare token is a good discriminator and usually a poor description. Person entries fill with dates: `william_december_11` is discriminative because a date is rare, not because a date is a sense. The gloss is allowed to be ugly when the ugliness separates.

A gloss does not have to be grammatical. It has to be distinguishing. A phrase, a date, a rare noun and a short suffix are all legal if they separate the sense from its siblings.

The objection

The strongest objection is that a definition is better. A definition explains the sense to a person, and an explanation is what a dictionary is for. The objection is right about explanation and wrong about the key. A definition is too long, too punctuation-heavy and too likely to contain the delimiter. The gloss is the key-sized test, and the definition lives beside the address, not inside it.

A second objection is that an embedding can tell the senses apart. It can propose a split. The gloss is what makes the split addressable and stable. A model that chooses a sense and keeps the choice inside its weights has not given a second system anything to join on.

A third objection is that this is a naming detail. It is the only part of the address that carries the distinction. Without it, the identifier names a spelling. With it, the identifier names a sense.

The concession is real: a gloss can be wrong, and it will be revised. The revision is recorded, not hidden. The address is a decision with a history, not a timeless label.

What the gloss cannot do

A gloss cannot prove the definition correct. It can only separate the sense from its siblings. A gloss cannot establish reference; two mentions with the same address may still point at different individuals. A gloss cannot translate; the target language may need a different distinction.

The gloss is a narrow instrument with one job. It is precise about that job and silent about the rest.

The cost of a bad gloss

A bad gloss is cheap to write and expensive to inherit because the address enters caches, joins and reports that no one revisits when the string changes. The repair is a migration of consumers, and the migration is the cost that the original gloss decision did not price.

A gloss that is merely ugly costs nothing. A gloss that is wrong costs the consumers who trusted it. The string is the cheap part. The migration is the bill.

The migration is not a single update; it is every consumer that cached the old address, every report that quoted it, and every join that still expects the old string to mean the old sense.

The review question

The review question is not whether the gloss is elegant. It is whether the gloss fits the neighbor. A reviewer who cannot name the neighbor cannot approve the gloss.

That question is cheap to ask and hard to fake. It requires the reviewer to open the sibling sense, read its definition, and decide whether the proposed gloss would fit both. The answer is a judgment, but the judgment has a named object.

The gloss is not a name

A name is a surface form. A gloss is inside the key. `approval` is the name; `formal_acceptance` is the gloss. Two names can point at one address, which is what a synonym is. Two addresses can share a name, which is what ambiguity is.

A synonym is a second name for one address, and an ambiguity is two addresses behind one name, which is why the gloss has to sit inside the key rather than beside it.

The distinction matters for translation. A target language may need a different gloss because it draws the boundary in a different place. The address is the source sense; the target gloss is a new decision, not a copy.

The gloss is not a label that a display layer can change. It is part of the address, and changing it is changing the identity.

The long view

A definition can explain the sense to a person, but a definition in the key would turn every revision into a new address, and every consumer would migrate for a change it did not request. A gloss that separates the sense from its nearest neighbor but collides with a third sibling has moved the failure rather than fixed it, and the uniqueness query is what catches the move.

The dictionary that reports its collision rate is making a claim about its own defects, and a claim that can be checked is worth more than a claim that the rate is zero. The address is a decision that a consumer joins on, and a decision that changes without a receipt is a decision that the consumer cannot audit.

What would refute this

If a gloss that fits its nearest neighbor still separated the sense in use, the neighbor test would be unnecessary, and the address would be a caption with a validator attached. If a dictionary with no gloss field matched the four-part address at cross-system comparison, the gloss would be decoration. If collisions were so rare that a uniqueness check never fired, the check would be ceremony.

None of those results is available yet. The measured dictionaries have collisions, the cross-system comparison depends on the address, and the neighbor test is the cheapest check the scheme has.

The gloss is a test inside the address; the next chapter asks who may mint one.

27 · Public and Private Dictionaries

A dictionary does two jobs that cannot share a table. It addresses a sense for someone who has never met you, which requires that the address mean the same thing tomorrow and to a stranger. It also gives you a handle on something nobody has named yet, which requires that you can create it this afternoon. The public dictionary holds the first kind. The private dictionary holds the second. The boundary between them is not where the rows are stored; it is who is permitted to write.

Two jobs, one identifier

A public identifier is a promise made at minting. From the moment it exists it is an address that has been published, and whether anything currently points at it is irrelevant to whether a publication happened. Rows are routine and reversible; an address commits every reader who will ever hold it, including readers in another organization running a tool nobody has designed.

The failure is quiet. Suppose a public identifier is minted on a Tuesday for a sense the compiler met that morning. Every row that used that identifier on Monday is now a row about something else, and not one of those rows changed. The stored references still resolve. The queries still return rows. The answers are simply about a different subject than the one the reader was given.

The private side has the opposite property. A note can be revised, split, merged or abandoned without anyone outside being wrong, because nobody outside was told anything. That asymmetry is the reason for two dictionaries rather than one table with a flag.

A public address commits readers who are not in the room, while a private entry is allowed to be wrong before it is published, and the two rules exist because those commitments have different costs.

The two failures

A public dictionary that grows at runtime makes the past unciteable: a row that cited an address yesterday may cite a different sense today. The private dictionary that never grows makes the present unnameable, because every new term is a gap the compiler cannot close and the message cannot carry. The split is the smallest structure that keeps both failures from arriving at once. It is a permission boundary, not a storage decision.

Two growth rules

The public dictionary is frozen per release. It grows when its maintainers publish a new release, and it does not grow while a compiler is running. The private dictionary grows at runtime, under the same identifier grammar and a narrower scope.

Dictionary	Growth	Audience	Cost of a wrong entry
Public	per release	anyone who holds the address	every future reader of the address
Private	at runtime	the organization that minted it	its holder, until the term escapes

The public side is read-only during compilation. The private side is writable, and it is writable precisely because a wrong private grounding costs nobody but its holder.

The freeze is not caution. A shared identifier that can change under a running compiler makes the past unciteable: a row that cited the address yesterday now cites a different sense, and no row changed. A system whose past cannot be cited cannot be audited. The release is the only mechanism that lets the public side grow without breaking the rows that already depend on it.

The release

Public growth is a release. A release is a dated artifact with a manifest, a diff and a rollback; consumers pin a version, and a consumer that does not pin is running whatever the last release happened to contain. The release is the only moment at which a new public address can enter the shared vocabulary, and the date is what lets a later reader reconstruct which vocabulary a claim used.

A runtime mint would make every open consumer wrong at once, and none of them would receive an error. The release cadence is a governance parameter, not a technical detail. A release that never happens makes the public dictionary a fossil consulted for names, while the private side carries every new sense.

The release also makes the public side comparable. Two systems on the same release can compare addresses without sharing a model. Two systems on different releases can compare the addresses that did not change, and can see which ones did. That is the most a shared vocabulary can offer across versions.

The owner

Every namespace has an owner. The public owner is the release manager; the private owner is the team that mints under the prefix. An unowned namespace drifts: two teams add senses under the same prefix, and the prefix stops meaning one dictionary.

The owner is permitted to mint, revise and withdraw. The public owner does so through a release; the private owner does so at runtime, under the evidence rule. A namespace without an owner is a name without a decision.

Withdrawal is the weak part. A private term whose owner disappears stays readable and permanently local; no withdrawal, tombstone or reassignment is described. The scope outlives the scope's owner. That is a gap, and it is better stated than hidden.

The lookup order

Lookup is not a search. It probes public fragments, intersects the candidates, orders them by address and hands a shortlist to the model. The model selects a sense; the dictionary supplies the candidates and the address. A miss is recorded as a miss, not filled by the nearest neighbor.

The first hit wins, and a narrower scope never overrides a broader one. An industry dictionary does not get to redefine a public term. The order is the closure rule, and the order is what makes the result reproducible across systems that hold the same release.

A lookup that returns a ranked list has not resolved anything. The ranking is a candidate order, not a decision. The decision is the address the model selected, and the address is what the message carries.

The barrier is the write path

Lookup runs in a fixed order, broadest and most authoritative first. The public dictionary is consulted read-only. A hit adopts the public namespace. A miss falls to prepared private dictionaries, then to the growing private ones, and only then does a provisional identifier become official.

The first hit wins, and a narrower scope never overrides a broader one. An industry dictionary does not get to redefine a public term. The order is the closure rule, and the order is what makes the result reproducible.

The permission sits on the model, not on the operator. The model may propose or select a sense. It may not mint a new public identifier. That rule is enforced as a test rather than a convention: a term with no public match stays provisional, and its namespace never becomes public unless the entry came from the public dictionary. A bad write has nowhere to go.

The private side is governed too. An unsuccessful lookup alone does not establish a correct new sense. Appending an entry requires a definition, evidence, and duplicate and conflict review, because absence of a match is no evidence at all. A private dictionary that accepts every miss is a list of guesses with a namespace.

The message segment

The segment is not the sender's dictionary. It carries the definitions the message needs, with owner, scope and version. The receiver reads only what the claim requires, and the claim is unreadable without it.

A segment that carries the whole dictionary is a publication. A segment that carries one definition is a message. The policy decides which is allowed, and quarantine is the default when the policy cannot.

The segment also carries the private namespace, so the receiver knows which local vocabulary the definition came from. A definition without a namespace is a definition without an owner.

A message with two kinds of term

Here is an example. A procurement message says supplier S is preferred at tier 2.

```
claim: supplier S is preferred at tier 2
public: supplier S -> supplier.legal_entity.public
 tier -> tier.ordinal.public
private: preferred_supplier_tier
 definition embedded in the message segment
receiver: resolves the public terms by reference
 reads the private term by value
```

The receiver has the public dictionary and not the sender's private one. If the sender sends a pointer to `preferred_supplier_tier`, the receiver resolves nothing. If the sender embeds the definition, the receiver can read the claim and audit where it came from.

That is the rule: public by reference, private by value. The pointer works because the public address is shared. The value works because the private definition is not.

The message is not a document with a dictionary attached. The dictionary segment is part of the message, and the claim is unreadable without it. A receiver that drops the segment has a sentence with a hole in it, not a claim with a missing detail.

The leak

Embedding a private definition shares it. The receiver can read the definition, and a reader who can read it can copy it. That is the cost of making the claim portable, and it is a policy decision rather than a technical one.

The alternative is a pointer that resolves to nothing, which preserves the secret and loses the claim. A sender who chooses the pointer has chosen not to be understood. A sender who chooses the value has chosen to share the definition under whatever policy governs the message.

The default when the policy cannot decide is quarantine. The message is held with the missing symbol named, not guessed. A human or a later release resolves it, and the rows that did resolve remain readable.

Closure, not a firewall

A firewall blocks traffic. A closure rule resolves symbols. The difference matters when a message crosses an organizational boundary.

Closure walks the message's symbols through the lookup order until each one resolves or fails. A public symbol resolves against the public dictionary. A private symbol resolves against the segment carried by the message, or against a private dictionary the receiver already holds. A symbol that resolves nowhere is not passed through silently; it is named and held.

The closure rule is transitive in one direction: a private term can point at a public term, and the public term resolves normally. A public term cannot point at a private one, because the receiver may not have the private dictionary. The asymmetry is what keeps the public side portable.

A closure rule that guesses is a firewall with a dictionary beside it. The guess fills the hole, the message passes, and the receiver never learns that the sender's term was undefined.

Quarantine

An unresolved symbol is quarantined with a typed outcome.

```
status: QUARANTINE_UNRESOLVED
symbol: preferred_supplier_tier
missing: definition segment
resolves: attach the segment, or add the term to a private dictionary
```

The message is not deleted. The resolved claims remain readable, and the unresolved symbol is a named gap. A later release or a human reviewer can attach the definition and release the message.

Quarantine is the correct outcome because the alternatives are worse. Guessing produces a claim the sender did not make. Dropping the message loses the claims that did resolve. Passing the symbol through produces a reader who believes it resolved.

The quarantine record is also a measurement. A program can count how often a private term arrives without a segment, which terms are missing most often, and which senders have not updated their dictionaries. A guess hides all three.

The quarantine record

Each quarantine records the symbol, the message, the missing segment, the date and the owner who can resolve it. The count is a measurement of dictionary drift, not a log of failures.

A program can count which terms arrive without a segment, which senders have not updated their dictionaries, and which releases would resolve the most quarantines. A guess hides all three.

The record also makes the cost visible. A quarantine is a claim that could not be read, and the count is a price the sender imposed on the receiver. A sender who sees the count has a reason to attach the segment next time.

A sender who embeds the definition shares it with every receiver who can read the message, which is the cost of portability and the reason the decision belongs to policy rather than to the encoder.

The test

A message with a private term is tested twice: once without the segment and once with it. Without the segment, the closure check returns `QUARANTINE_UNRESOLVED` and names the symbol. With the segment, the same message resolves and returns the claim.

The expected result is one quarantine before and zero after. A test that passes without the segment is not testing closure; it is testing a guess, and the guess is indistinguishable from a resolution until the receiver tries to audit it.

When public and private collide

A private term and a public term can look like the same thing. The rule is to keep both and link them, never to merge them.

Equivalence uses `SAME_ENTITY` and `SAME_SENSE`. Broader, narrower and related links are not identity bindings; they record a relation and leave the local address intact. A public identifier never overwrites a local one, because the local row is what made the assertion, and replacing it changes what every downstream claim refers to.

Substitution is allowed only when an accepted mapping is compatible with the local sense. The mapping is recorded as a link, so it can be accepted, rejected or revised without touching the row that made the claim.

This is the part that looks like enrichment and is actually a rewrite. A public ID substituted for a local one changes the subject of every claim that used the local address. The link preserves the local claim and records the candidate match beside it.

The escape

A private term joined in a view and consumed by another team is public in practice. No permission was checked, no release minted it, and the namespace still says private. The name and the audience disagree.

The namespace records the origin, not the audience. That is why the split makes an escape visible after the fact rather than impossible in advance. A team that wants to prevent the escape has to govern the views, not only the dictionary.

The objection

The strongest objection is that the split is bureaucracy. A private identifier used as a join key in a downstream view is public in every respect that matters, and no permission was ever checked. The split cannot prevent a bad merge. It cannot improve a wrong sense. What it buys, on this reading, is a namespace field and a test that stops a model writing one string instead of another.

Most of that is right, and the concession should be stated before the defense. The split is a recording mechanism, not a correctness mechanism. It prevents one failure, the silent rewriting of an address, and it prevents that one by construction. It does not prevent a wrong sense, a wrong merge or a bad join.

What survives is narrower and still worth the machinery. Without the namespace there is no way to ask who decided, because there is no field that records the difference between a shared address and a local note. A dictionary can lose that distinction without breaking anything, and the recovery, when anyone notices, is to record retrospectively who stood behind the identifier rather than to roll it back. Deleting the identifier breaks exactly the rows that depend on it. What the split provides is a merge made legible, which is smaller than a merge prevented and is the claim the evidence supports.

What would refute this

Take two hundred private mints from a real corpus and, for each, find every row that depends on what it means rather than every row that uses it as a string. If no downstream dependency exists in any case, and no query result or reproducibility check changes when the namespace field is removed, the split is bureaucracy and should be deleted in favor of one table with a source column.

A second result would refute the governance claim while leaving the citation claim standing. If promotion never happens and the release cadence is zero over the period observed, the private side is not a staging area for shared meaning but the only dictionary in use, and the public one is a fossil consulted for names. Immutability would still hold. The claim that the boundary governs the growth of shared meaning would not.

The boundary is a closure rule, not a firewall; the next chapter asks what keeps a symbol tethered.

28 · Grounding

A symbol can be addressed and still float. The address says which sense the compiler chose; grounding says what evidence stands behind the choice. The two obligations are usually fused, and the fusion is the source of most of the confusion about what a lexicon owes.

Two obligations

Assignment is mandatory. Every node carries an address, and the minimum address has a provisional namespace. A document must compile in local-only mode, using the document and its private dictionaries alone. That compile builds entities, terms, addresses, aliases and graph structure with no public lookup at all.

Grounding is optional. It is the search that would upgrade a provisional namespace to a public one, and it either finds a public sense or records a miss. A floating row is not an unfinished row. It is a row whose meaning is owned by the document set, and it is entitled to stay that way.

The two modes are different promises. A floating row promises nothing to a reader who is not present. A tethered row claims that a public identifier names this content, and a claim of that kind owes three things in return.

What a tether consists of

A tether is not the identifier alone. It has three parts, and each can go missing independently.

Part	What it is	If it is absent
Identifier	the public address	no public claim; the row is local only
Version	the dictionary release the address was resolved against	a drifting address: same string, different dictionary, later
Basis	the span and gloss match that licensed the choice	a bare pointer: nobody can check why this sense was chosen

Remove the version and a receiver hydrates the identifier against a dictionary the sender never used. Remove the basis and the receiver has a pointer with no argument

behind it. The three parts together are what make the address disputable by someone who does not share your dictionary.

A tether table makes the parts visible.

Symbol	Public address	Version	Basis	Status
customer.retail	customer.retail.public	release 12	span 41:7, gloss match	tethered
priority customer	none	none	none	floating

The table is the desk page. A tether without a basis is a name with a version number. A basis without a version is an argument that cannot be reproduced.

What a tether buys

A tether buys disputability. The receiver can hydrate the address against the named release and inspect the basis, and a reader who disagrees can point at the release or the span rather than at the sender's judgment. Without the version, the receiver hydrates against whatever dictionary it holds; without the basis, the receiver has a pointer with no argument. The tether does not prove the sense correct; it makes the choice inspectable, which is the most a shared vocabulary can offer, and it is enough for a dispute to have a location.

A receiver that hydrates against the wrong release will read the right address with the wrong definition, and the error will look like a disagreement about the world rather than a disagreement about the dictionary.

The three stages

An address passes through three stages. The structural pass assigns a provisional address from the lemma and the part of speech. The sense-bearing pass adds the microgloss, and the address now names a sense. The public lookup may upgrade the namespace to public, and only the third stage is a tether.

A row can stop after stage two without being incomplete. It has an address, a gloss and a local owner; what it lacks is a public commitment. The middle address is not a failed third stage. It is a complete local address that has not made a public claim.

The stages are visible in the string. `publish.publish.verb.provisional` is a structural address; `publish.make_public_available.verb.provisional` is sense-bearing; `publish.make_public_available.verb.public` is tethered. A public lookup that finds no sense leaves the sense-bearing address in place; it does not erase the gloss.

Floating and tethered

Here is an example. A report says the order went to a priority customer.

```
customer.retail: public ID + definition + policy -> tethered
priority customer: private, no ID, no evidence -> floating
re-tether: add a public ID and evidence, or retire the term
```

`priority customer` is not a bad term. It is a term the report owns. Every report that uses it may mean something slightly different, and the difference is invisible until someone tries to compare two reports. The tether check flags it as floating, not as false.

`customer.retail` is tethered to a public sense, a definition and a policy. The difference is not confidence. It is whether the claim can be checked.

A floating symbol is legal. An undeclared floating symbol is not. The status has to be in the record, because a reader who cannot see the status will treat the symbol as if it had a public sense.

The local-only compile

A Spanish document cannot match an English public dictionary. The honest output is a ledger of provisional and document-scoped addresses, and a design that treated those rows as unfinished would have no honest output to offer at all.

The local-only compile is a complete ledger under a different promise. It passes the same structural checks: roles, spans, types, dependencies. It differs in the namespace, not in the grammar. A reader who cannot see the namespace will mistake a local ledger for a failed public one.

The local compile is also the safer default, because it does not force a guess. A pipeline with a completeness gate and no abstention path has exactly one way to close a gap, and it is a guess.

The status vocabulary

Grounding status is a state, not a boolean. The states run from unattempted to candidate, accepted, rejected, provisional and verified. A row can move between them, and the move is recorded.

The status column is what lets a consumer ask whether the address was ever checked, not merely whether it has a public-looking string. A boolean hides the difference between a row that was never attempted and a row that was attempted and rejected.

A status that is not recorded is a comment. The record is the difference between a floating row and a row that pretends to be grounded.

The status vocabulary is the only place where a consumer can see whether an address was attempted, rejected or verified, and a boolean would collapse those states into a claim that no one checked.

The mode in the record

The mode belongs in the record, not in a comment. A row that says provisional is making a different claim from a row that says public, and a consumer that cannot see the field will treat both as the same object. The status is the field that makes the difference visible, and the difference is the whole point of having two modes.

The record of a tether

A tether is recorded, not remembered. The record names the address, the version, the basis, the owner and the date; a tether without a record is a claim that cannot be audited. The record is the receipt, and the receipt is what lets a later reader check the choice rather than trust the sender.

A tether record that omits the owner leaves the receiver unable to ask who committed to the sense, and the omission is invisible until the sender has left.

The tether check

The tether check reads three fields and returns tethered, floating or broken. Broken is a tether that names an address but fails version or basis; it is not floating, and treating it as floating hides the defect. The check is cheap because it reads three fields, and it is worth running on every export because a broken tether is invisible to a consumer that never opens the basis.

A tether check that reports only tethered or floating has hidden the broken case, which is the case that matters most because the row looks grounded and is not. The check is not optional. A consumer that never runs it is trusting the string.

A broken tether is a claim that the sender made and cannot support, and the receiver who admits it has accepted a commitment on the sender's behalf.

The report

A tether check over a report is the smallest useful test.

```
priority customer: no public ID, no evidence -> FLOATING
customer.retail: public ID + version + basis -> TETHERED
re-tether: add the public ID and evidence, or retire the term
expected: 1 floating before, 0 after
```

The check is not a spell check. It reads the address, the version and the basis. A row with an address but no basis is not tethered. A row with a basis but no version cannot be hydrated, and the receiver cannot reproduce the choice.

The check is also not a verdict on the term. `priority customer` may be the right term for the report. The check says the report owns it, not that the report is wrong.

A report that lists a floating term without its status has told the reader that the term is public, which is the one claim the report was supposed to avoid.

The evidence question

Evidence coverage proves that stored spans re-slice the source. It does not prove that a span licensed the sense attached to it, and a row can cite the right sentence while choosing the wrong sense. The coverage number stays at one, and the error is invisible to the instrument that was supposed to catch it. A system that measured tether quality would need a second reader for senses, in the way the extraction lane needs one for parses; the corpus describes the coverage instrument and not the second reader.

The regress stops

A definition points at other definitions. `customer` points at `party`, `party` points at `entity`, and the chain can continue until someone asks what stops it. The answer is a public commitment, not a private intuition.

A public commitment is an address, a version and a basis that a stranger can inspect. The chain stops there because the commitment is shared, dated and disputable. A private intuition stops the chain for the person who holds it and for nobody else.

The stop is not arbitrary. A prime is a commitment that the language community has agreed to treat as a stopping point; a public dictionary entry is a commitment that a release has published. Both are social objects with dates. Neither is a private feeling that the analyst is tired of decomposing.

The regress also stops at a declared gap. A branch that cannot reach a public commitment is marked as unresolved, and the mark is the honest end of the chain. A gap is better than a private axiom wearing a public address.

Promotion without recompilation

A floating row can be promoted without recompiling the document. The claim does not depend on the namespace; the namespace is a label on the address. Promotion changes that label, not the roles, the participants or the verb dimensions.

The projection makes the modesty of the operation visible.

Field	Floating	Tethered
Predicate	publish	publish

Continued on next page

(Continued)

Field	Floating	Tethered
Participants	the committee, the minutes	identical
Tense, polarity, stance	past, positive, asserted	identical
Address	`publish.make_public_available.verb.provisional`	`publish.make_public_available.verb.public`

One field moves. That is why the step is safe to run, safe to skip and safe to repeat.

It is safe for the row and unsafe for its readers. A row whose namespace changed is a different claim to anyone holding a cached copy, because the string they stored did not change while its meaning did. The version is in the tether for exactly this reason. A receiver should not silently accept a materially older release without an explicit local decision.

Promotion is also idempotent in the intended design. The private dictionary grows and the public one does not, so a second run over the same corpus finds its own first-run misses as hits. Without that asymmetry, running the funnel twice would keep producing new addresses and floating would become indistinguishable from drift.

The cost of grounding

Grounding costs a lookup, a review and a version pin. The lookup is cheap; the review is the expensive part, because someone has to decide whether the candidate sense is the one the span supports. The version pin is cheap per row and expensive at the boundary, because every receiver has to hold the release the sender used. Promotion is cheap for the row and expensive for its readers, because a cached copy does not change when the namespace changes.

The override gap

A domain pack may override the baseline for specific addresses, so a medical deployment can define a term differently from a manufacturing one without forking the lexicon. If an override redefines a published identifier, the public/private line is not where the design says it is, because a public sense changed at runtime.

Whether the override mints a separate identifier or redefines the existing one is not stated. The seam is undrawn, and it matters: a redefinition would make every tethered row in that domain point at a sense the release never published.

The honest position is to mark the seam rather than to assume the benign reading. An override that mints a new address is an extension; an override that redefines a public address is a governance event that the release process has to see.

A domain pack that redefines a published address turns a release artifact into a runtime variable, and every tethered row in that domain inherits a sense the release never published.

Losing a tether

Grounding can be lost. A public address can be deprecated, a release can move the sense behind an unchanged gloss, an owner can leave, or a private dictionary can be retired. The row remains readable; the tether does not.

The loss is not always visible. A tether whose version is missing still resolves against whatever release the receiver happens to hold. A tether whose basis is missing still points at an address. The row looks grounded and is no longer checkable.

Re-tethering is a maintenance act. It takes the old address, the new evidence and the reason for the change, and it records a receipt. The receipt is the difference between a repair and a silent re-pointing.

A tether that is removed without a record is worse than a floating row. The floating row declares that it has no public commitment. The removed tether declares a commitment that no longer exists.

The two failures

A false tether is worse than a floating row. The floating row declares that it has no public commitment; the false tether declares one and cannot support it. A false tether is an address with no basis or a version that cannot be hydrated, and it is more dangerous than a floating row because it looks grounded to every consumer that reads only the string.

A floating row declares that it has no public commitment, while a broken tether declares a commitment that no longer exists, and the second claim is the one that misleads.

A tether that names an address and a version but no basis can be hydrated and cannot be checked, which is the most dangerous state because it passes the first test and fails the second.

The objection

The strongest objection is that the mode is a comment. If nothing downstream branches on floating versus tethered, the distinction is metadata that no query reads. The system ships a status column; the status column does not change an answer.

Part of that is answerable. The mode has teeth at a boundary. A message that cannot be hydrated is semantically invalid for its receiver, who must refuse, quarantine or request the missing material rather than guess. At that boundary the mode decides admission, and the failure it prevents is the one where both sides report success while the meaning differs.

The rest is a concession, and it is large. What gets measured is assignment and upgrade, not tether correctness. Evidence coverage proves that stored spans re-slice the source; it does not prove that a span licensed the sense attached to it. A system that measured tether quality would need a second reader for senses, in the way the extraction lane needs one for parses.

The distinction is still worth keeping, because without the status there is no field that records the difference between a shared commitment and a local note. A dictionary can lose that difference without breaking anything, and the loss is invisible until someone asks who decided.

The admission rule

At a boundary, the mode is an admission rule. A message that carries a tethered address and no version is not hydrated; a message that carries a floating address and no evidence is not admitted. The receiver refuses, quarantines or requests the missing material rather than guessing.

The rule is what gives the distinction teeth. Inside one store, the mode can be a comment because the store owns both sides. Across a boundary, the mode decides whether the receiver can read the claim at all.

A receiver that admits an unhydrated message has chosen to treat a pointer as a definition. The choice may be convenient and it is not the same claim.

A receiver that admits a message it cannot hydrate has decided that the sender's pointer is a definition, and the decision is a local policy that the sender never sees.

What the boundary enforces

The boundary enforces the mode, not the truth of the sense. A message that cannot be hydrated is refused or quarantined; a message that hydrates but chooses the wrong sense still passes. The rule is narrow, and it is the narrowness that makes it enforceable.

A receiver who needs more than hydration has to add a second reader. The boundary can tell whether the claim is readable; it cannot tell whether the reading is right.

What would refute this

Three tests, in ascending cost.

Compile one fixture in both modes and assert that the core predications are identical while the namespaces differ. If the namespaces do not differ, the modes do not exist.

Promote a floating row and diff the full record. If any field other than the namespace moves, grounding is part of the claim and the modesty of the operation is false.

Instrument the consumers. If no query, export or admission check ever reads the mode, the distinction is a comment. If a boundary check refuses an unhydrated message, the distinction has teeth.

If the namespaces differ but every consumer ignores the field, the modes exist in the schema and not in the system, and the distinction is a comment that survived a test.

Grounding is a claim with a receipt; the next chapter is what happens when two claims disagree.

29 · Entity Resolution and Ghosts

A mention is an exact span in one document. An entity is a claim that a canonical identity names the same thing. An alias is a surface form that may point at an entity. Resolution is the work that moves from the first to the second, and it is not a similarity threshold. It is candidate generation followed by a decision with a receipt.

Two stages

The first stage generates candidates. It probes the mention against names, aliases and identifiers, intersects the results for selectivity, and returns a bounded set. The set may be ranked by a model, a string score or a blocking key; the ranking is a proposal, not a decision.

The second stage decides. It merges, splits, mints or ghosts, and every one of those outcomes carries a record. The two stages are separate because they fail differently. Candidate generation fails by omission, and the fix is a wider probe. Resolution fails by false binding, and the fix is a receipt.

A one-stage threshold collapses the two. It merges when the score is high and rejects when the score is low, and the threshold is a guess with a number attached. The number can be tuned, but tuning does not make the decision reviewable.

The threshold is a guess

Similarity is useful for candidates and dangerous for decisions. `Acme Corp` and `ACME Corporation` look the same, and so do `Acme` and `Acme Holdings`. A threshold high enough to reject the second pair will reject some true matches; a threshold low enough to accept the first will merge companies that are legally distinct.

The pairs differ in what the evidence supports, not in how close the strings are. The first pair shares a tax identifier and two contracts. The second pair shares a name fragment and nothing else. A score can rank the pairs; it cannot tell you which evidence is present.

A threshold is also invisible in the result. The merged row does not say which threshold produced it, which candidates were rejected, or what would undo the merge. The decision is real and the record is absent.

The merge receipt

A merge is a claim that two identities name the same thing. It needs four fields: evidence, owner, date and rollback.

Field	What it records
Evidence	the contracts, identifiers, definitions or events that support the merge
Owner	the person or role accountable for the decision
Date	when the decision was made and against which release
Rollback	how to restore the two identities and re-point consumers

The receipt does not prove the merge correct. It makes the merge reviewable. A later reader can ask which evidence was used, who signed, and what would undo it.

A merge without a rollback is irreversible. A merge without an owner is a rumor. A merge with no evidence is a threshold in a costume.

Acme

Here is an example. Two contracts name `Acme Corp` and `ACME Corporation`.

```
mention A: "Acme Corp" -> candidate acme.legal_entity
mention B: "ACME Corporation" -> same candidate
evidence: two contracts, same tax ID, same registered address
owner: vendor master data team
date: 2026-04-11
rollback: restore both aliases; re-point consumers to prior identities
outcome: MERGE
```

`Acme` and `Acme Holdings` are a different case. They share a name fragment and no identifier; the correct outcome is two entities with a related link, not a merge. The difference is not the score. It is the evidence.

A merge that changes what every downstream claim means cannot be a side effect of deduplication. It is a decision about identity, and it belongs to whoever owns the vendor master.

Ghosts

A ghost is an entity referenced but not identified. It has a mention, a span and a status, and it is a record rather than an error.

```
mention: "supplier X" in contract C-9
span: C-9, offset 214
status: GHOST
candidate: none above the evidence floor
resolves: mint a supplier identity, or link to an existing one
```

The ghost is what the compiler knows: a name was used, the document is the evidence, and no identity was safe. Dropping the mention would lose the reference; inventing an entity would manufacture one. The ghost keeps the reference and the gap.

A ghost is not a bad row. It is a row about a gap. The distinction matters at export: a store that drops ghosts looks complete and is not, because the unresolved references were deleted before anyone could count them.

Four outcomes

Resolution has four legal outcomes, and each has a receipt.

Outcome	When it applies	Record
Merge	two identities name the same thing	evidence, owner, date, rollback
Split	one identity names two things	evidence, owner, date, rollback
Mint	a mention needs a new identity	definition, evidence, namespace, owner
Ghost	no identity is safe	mention, span, candidates, status

Split is the inverse of merge and needs the same discipline. A split without a receipt leaves the consumers of the old identity pointing at a row that no longer means what they stored.

Mint is a resolution outcome, not a failure to resolve. A new supplier, a new project or a new term may have no candidate because it is new. Minting it is a decision with a definition and an owner, and the decision is recorded.

Ghost is the outcome when the evidence does not support any of the other three. It is not a temporary error state; it is the honest result of a mention that cannot be placed.

Candidate, ghost, gap

Three objects are often collapsed into one status column, and the collapse destroys the reason each was recorded.

Object	What it holds	What closes it
Candidate	a bounded set of live interpretations	a predication that only one member satisfies, or a review
Ghost	a mention with no safe reading	new evidence, or a decision that none will come
Gap	a missing position in the structure	nothing; the source licensed the hole

A name with two referents is not the same defect as a name with no referent. A pronoun that could be either of two people is a candidate; a supplier with no identifier is a ghost; a subjectless clause is a gap. The three need different repairs.

The gap is the one the schema makes structural. A clause with no subject carries an omission status, so an omitted actor is recorded and never invented. The ghost carries a mention and a span, so the unresolved reference is addressable. The candidate carries a bounded set, so the decision can be reviewed rather than guessed.

The ghost cause problem

A ghost can be honest or it can be a pipeline failure. The same row shape covers both, and the difference matters.

A source may leave a reference open: a contract says “supplier X” and never names the supplier. That ghost is the corpus declining to be clear. A parser may fail to read a name, or a handoff may drop a payload; that ghost is the pipeline failing, and it should be fixed rather than preserved.

Arithmetic catches the silent drop. Passed plus ghosted equals reached, and a clause that is neither materialized nor ghosted breaks the equation. Arithmetic does not classify the cause. A ghost table with a kind, a surface form and a span but no cause field cannot tell the source’s ambiguity from the compiler’s error. A ghost count that does not separate source ambiguity from parser failure is a number that improves when the parser is fixed and worsens when the corpus is honest, which is the opposite of a quality signal.

A ghost that carries a cause field can be counted as a finding about the source or a defect in the compiler, and the two counts move in opposite directions when the system improves.

The cause field is the repair. A ghost whose cause is `SOURCE_AMBIGUOUS` is a finding about the document; a ghost whose cause is `PARSER_FAILURE` is a defect in the compiler. The count is not a measure of ambiguity until the causes are separated.

A ghost also needs a life. If it is collected after thirty days, it cannot ship in an export or accumulate resolution history; if it ships forever, it becomes a permanent unciteable object. The two rules cannot both be enforced as written. The honest design picks one and records the choice.

The Wilson fixture

Three documents mention Wilson.

```
D1: Wilson loaded the truck at 06:40.
D2: Wilson approved the invoice on Friday.
D3: Wilson signed the manifest.
```

The compiler does not decide that all three are one man. It mints two document-local identities and resolves each mention to whichever identity its own document supports. D3 has nothing to decide with, so the mention becomes a candidate.

Row	Content	Status
Mention m_31	span “Wilson” in D3	CANDIDATE
Candidate set cs_9	warehouse Wilson, approvals Wilson	open
Discriminating predication	none in D3	not yet

What closes the candidate is not a better string match. The surfaces are identical, so similarity has nothing to say. What closes it is a predication in D3, or in a linked document, that only one entity satisfies: a shift roster, a signature block, a fourth document naming the loading-dock Wilson as the receiver.

If a person is what their predications say they did, then disambiguation is a comparison of event sets, not of letters. The upgrade path is a query over the ledger, not a threshold over a score.

Four men

A name can be shared by four men: two military officers and two recipients of the same medal. The corpus keeps them apart by the events that constitute them, not by the string. Merge two of them and every later query about either man returns the other man’s deeds.

The merged row is well formed. It has exact spans, a plausible label and a confidence number. Nothing in its shape announces the error, and no downstream check recovers it, because the evidence for the error was the absence of a discriminating predication. Absence leaves no trace once the rows are fused.

A false merge is worse than a missing one because it stops being visible. A missing binding is a question; a false binding is an answer that closes the question.

Coverage

A store that exports only resolved facts looks identical to a store in which everything was resolved. The unresolved rows were deleted at export, and the reader cannot distinguish a corpus with no unresolved references from one whose references were removed.

The completeness claim is then unearned. A ghost row that ships is evidence of a gap; a ghost row that is deleted is evidence of nothing. The export rule has to choose which one the store is making.

The same failure appears in a coverage number. A count of resolved rows divided by all rows rises when the unresolved rows are removed, and the rise looks like improvement.

A ghost that is deleted to make the export look complete is a gap that the next reader cannot see, and the deletion is the one repair that makes the record worse.

The ghost queue

A ghost is a queue with an owner. It holds the mention, the span, the candidate set and the resolution history, and it waits for new evidence or a decision that none will come.

The queue is measurable. A program can count ghosts by cause, by owner, by age and by document. A ghost count without causes is a coverage figure with an unknown contamination rate.

Two causes matter. A source may leave a reference open, which is a finding about the document. A parser may fail to read a name, which is a defect in the compiler. The same row shape covers both, and a cause field is what separates them.

A ghost also needs a life. If it is collected after thirty days, it cannot ship in an export or accumulate resolution history. If it ships forever, it becomes a permanent object that nobody can cite. The two rules cannot both be enforced as written; the design picks one and records the choice.

Mint and split

Mint is a resolution outcome, not a failure to resolve. A new supplier, project or term may have no candidate because it is new. Minting it is a decision with a definition, evidence, a namespace and an owner.

Split is the inverse of merge. One identity turns out to name two things, and the split divides the claims, aliases and links between them. It needs the same receipt as a merge: evidence, owner, date and rollback.

A split without a receipt leaves consumers of the old identity pointing at a row that no longer means what they stored. The row is still readable; the meaning has moved.

Candidate generation

The first stage probes the mention against names, aliases and identifiers. It intersects the results for selectivity and orders the survivors by address, not by relevance. The shortlist is reproducible and not ranked; calling its members best matches would be false.

The candidate set is bounded by a blocking key or a middle. An unbounded candidate set is a pairwise comparison of every mention against every entity, which is a different algorithm and a different cost.

A model may rank the candidates. The ranking is a proposal; the merge is a decision. The distinction is what keeps the threshold from becoming the resolver.

The merge guard

A merge without evidence returns `REFUSE_MERGE_NO_EVIDENCE`. The guard is cheap: it reads the receipt fields before it writes the merge.

The test is one fixture. Before the evidence is attached, the merge is refused and the refusal names the missing field. After the evidence, owner and rollback are attached, the merge succeeds and the receipt is stored. Expected: one refusal before, zero after.

A guard that never fires is a line in a file. The fixture is what makes the guard a test rather than a comment.

The candidate is not a merge

A candidate set is not a merge waiting to happen. It is a record that the evidence is insufficient and more than one reading is live. The set closes when a predication or a review rules out all but one member, and the closure is recorded.

A candidate that is auto-merged because it ranks first is a threshold decision wearing the word candidate. The rank is a proposal; the decision needs a receipt.

A candidate set that closes only when a predication rules out every other member is a record of reasoning, while a set that closes because one member ranks first is a threshold in disguise.

The ghost is not a gap

A ghost has a mention; a gap has a position. A ghost says a name was used and no identity was safe. A gap says a role or a subject is missing from the structure.

The repairs differ. A ghost needs evidence or a mint; a gap needs the omitted structure, or an explicit omission status. Collapsing them into one status loses the reason each was recorded.

What resolution is not

Resolution is not deduplication. Deduplication removes rows; resolution adds a decision with a receipt. A dedupe that keeps one row destroys the evidence for the other, while a merge keeps both identities in the record and records which one the claims now use.

Resolution is also not a global merge. One string does not name one thing across a corpus; the rule is local, and the local decision is what the receipt records.

The cost of a false merge

A false merge contaminates every query that touches either identity. The repair is a split, and the split has to reconstruct which claims belonged to which entity after the two were fused.

The reconstruction is possible only if the merge receipt recorded the evidence and the rollback, and without it the split is a guess about which row was which after the damage.

A split that is attempted without the original evidence has to infer which claims belonged to which entity from the claims themselves, and the inference is the same resemblance test that caused the bad merge.

The resolution record

Every resolution writes a record: the mention, the candidate set, the outcome and the receipt. The record is what makes the decision reviewable, and it is the difference between a resolution and a side effect.

A resolution without a record cannot be replayed, and a decision that cannot be replayed cannot be audited. The record is the resolution.

A resolution record that omits the candidate set leaves the reader unable to tell whether the decision was a merge or a choice among several, and the omission hides the alternative that was rejected.

The objection

The strongest objection is that this makes resolution manual. Candidates can be automatic; merges are few relative to mentions. The manual part is the decision, and the decision is the part that changes what every downstream claim means.

A second objection is that ghosts will pile up. They are a queue with an owner, and the queue is measurable. The alternative is silent invention, which piles up in a place nobody can see.

The concession is real: candidate generation may use embeddings, blocking keys or any other proposal method. The merge still needs a receipt. The embedding proposes; the record disposes.

What would refute this

The test is consumer-side. Instrument what reads unresolved rows. If no query filters on resolution status, if no export carries ghost rows, and if no review queue ever opens a candidate set, the machinery is bookkeeping and the claim is decoration.

A second test is a scoring test. Run one fixture twice, once allowed to bind on resemblance and once required to abstain, and compare the acceptance scores. If the run with false bindings scores at least as well as the run required to abstain, the preference for explicit unresolved status is unimplemented, and the design's central wager is dead.

A third test is the merge receipt. Remove the receipt from a merge and ask whether any consumer notices. If nothing notices, the receipt is documentation; if a consumer cannot replay the decision, the receipt is load-bearing.

Resolution is governance, not scoring; the next chapter asks what survives when the surface changes.

30 · One Meaning, Many Tongues

A translation is not a second record. It is a rendering of the same record in another surface. If the predication is the atom, two languages that word one claim differently still produce the same hub with the same roles, and the difference lands in the surface span and the lexicon wing.

The seam is drawn by cost

The parts of the substrate that are expensive to change later were built language-neutral: identifiers, offsets, roles, operators, composition and provenance. The parts that are cheap to swap stayed English: the parser model, the stemmer, the marker lists, the templates and the public dictionary.

That is a seam drawn by the cost of change rather than by a feature list, which is why it can be checked instead of admired. A schema that is neutral only where neutrality is convenient is not neutral; it is English with universal column names. The expensive parts are the ones that would be costly to redo after a corpus exists, and those are the parts the design protects.

The seam also explains the failures. A parser label is cheap to change and easy to forget. A public dictionary is expensive to rebuild and easy to leave monolingual. The two failures look different because they sit on different sides of the seam.

One synapse, two sentences

Here is an example.

```
synapse: APPROVE(agent=A. Rao, theme=order 44021, time=2026-04-11, polarity=+)
English: A. Rao approved order 44021 on 11 April 2026.
Spanish: A. Rao aprobó el pedido 44021 el 11 de abril de 2026.
```

The two sentences are different strings. They are not different claims. The agent, the theme, the time and the polarity are the same, and the surface is what changed.

A second store would translate the triples and keep both copies in sync. Two stores in sync are two stores that will drift, because the next correction lands in one and not the other. One record with two renderings has one truth and two surfaces.

The sync rule is the part that fails silently. Two stores need a conflict rule, a correction path and a reconciliation job, and the reconciliation job is where the drift hides. One record has no sync rule because there is nothing to sync.

What survives

Translation preserves the structure and changes the surface. The structure includes the roles, the force, the time and the polarity. A translation that changes the force is not a translation; it is a different claim.

Element	Survives?	Example
Predicate sense	yes	approve, not “accept” in a different sense
Roles	yes	agent, theme
Force	yes	assertion stays assertion
Time	yes	11 April 2026
Polarity	yes	positive stays positive
Surface	no	English words, Spanish words

The test is structural. Two renderings are compared on the elements that must survive, not on the words. A synonym in the target language passes; a changed tense fails.

Ledger equivalence

Acceptance for a second language is ledger equivalence. Two ledgers, compiled independently, are compared on roles, polarity, modality, attribution, time and composition. The surfaces are never compared, because a good translation rarely matches word for word.

The test separates two failures that look the same in the output. A claim that was captured and then lost, and a claim that was never captured, both produce a weak sentence; one calls for a better renderer, the other for a better compiler. Ledger equivalence distinguishes them by comparing what was stored rather than what was said.

A surface-overlap test cannot make that distinction. It rewards a translation that copies the source words and punishes a translation that is correct in the target language but different in form.

The parser's vocabulary changes first

The first failure in a new language is not semantic. It happens inside the parser's label set, before any meaning is assigned.

The same grammatical position has different names in different models: `nsubj` in English, `sb` in German, `nsubj` in Spanish, `nsubj` in Chinese; `dobj` in English, `oa` in German, `obj` in Spanish. A normalization table maps the model's labels onto the substrate's roles. If the table lacks a label, the participant is dropped and the compile still reports success.

A measured case makes the cost concrete. Every Spanish prepositional object was dropped from the ledger because the model's oblique label was not renamed to the substrate's label. The run looked successful. A query for a sign event with a location returned one row for the Spanish ledger and zero for the English, and nothing in the status column distinguished a sentence that had no location from one whose location was discarded.

The repair is one position-name normalization table keyed by model, so adding a language is one row rather than one rule per stage. The table is boring, and it is the difference between a language that compiles and a language that appears to compile.

The table is also a test. A language whose labels are not in the table will compile with missing participants and no error, and the omission will look like a property of the source. The normalization table is the first thing to check when a new language produces suspiciously simple structures.

The four models

The same positions have different names across parsers.

Position	English	German	Spanish	Chinese
Subject	nsubj	sb	nsubj	nsubj
Direct object	dobj	oa	obj	dobj
Indirect object	dative	da	iobj	none
Determiner	det	nk	det	none

The mismatch happens before meaning is assigned, inside a string comparison, and it fails quietly. A normalization table keyed by model is the repair. Without it, a language

compiles with missing participants and no error, and the omission looks like a property of the source.

The instrument breaks before the language

The second failure is in the check, not in the language. A stance gate searched the output for a handful of English reporting words. No Chinese rendering contains those words, so the gate fired on every reported Chinese claim and capped the graph score for every non-English document.

The finding that Chinese stance was broken was a fact about the check. The repair reads the ledger's own surfaces, which is neutral by construction and, unlike a word list, can prove that attribution was rendered.

A stance gate that searches for English reporting words will fire on every reported Chinese claim, and the finding that Chinese stance is broken is a fact about the check rather than about the language.

A per-language failure is always a claim about two things: the language and the instrument. The instrument is the cheaper one to test first, and it is the one most often left in English.

One ID, two lemmas

The sense is shared; the lemma is not. `approve` and `aprobar` are different surface forms for one sense, and the address carries the sense rather than the spelling.

A language-specific address is still possible. Spanish and English senses may hold different addresses and be linked by a `SAME_SENSE` relation, with the link recorded as grounded or candidate. That is stronger than a forced universal identifier, because the link can record a case where two languages disagree about a sense boundary.

Language	Lemma	Address	Link
English	approve	approve.formal_acceptance.public	source
Spanish	aprobar	aprobar.formal_acceptance.public	SAME_SENSE

The shared thing is the sense identity, not the string. A universal identifier that forces one spelling cannot record the disagreement; a link can.

The public dictionary is the binding constraint

The schema can be neutral and the public dictionary can be monolingual, and the second fact decides what a non-English document can do. A public dictionary with one language across all its rows cannot match a Spanish mention structurally, so the

non-English compile leans on private dictionaries that start empty and produces a ledger of provisional addresses.

That is the binding constraint, not the schema. A second language needs a lexicon wing, a set of surface forms and senses in that language, before the neutral substrate can show what it can do. The schema is necessary and not sufficient.

The missing wing also changes the ghost rate. An unresolved name arriving in one script is a ghost with a language attached, and the ghost is evidence about the dictionary rather than about the document.

A wing is not a word list. It contains surface forms, senses, links to the shared identity and an owner. Without the wing, a non-English document can only produce provisional addresses and ghosts, and the neutral schema has nothing to show.

Span or paraphrase

A translation without a source span is a paraphrase. The rendering has to carry the span it came from, so a reader can compare the target sentence to the source sentence and see what moved.

The cross-language round trip is the test. Render the synapse into both languages, recompile each rendering, and compare the structures. The expected result is identical structures and two spans. Wording equality is not the test; a good translation rarely matches word for word.

The span is what makes the rendering auditable. Without it, the target sentence is a fluent paraphrase with no address, and a paraphrase cannot be checked against the source.

The second pair

The current evidence is one pair, compiled in one direction. A second pair, German into English or Spanish into Chinese, is the cheapest refutation: if the ledgers agree on the six dimensions, the claim strengthens; if they disagree where the source expressed the distinction, predication-centricity is an English artifact. A second pair also tests the normalization table, because a language whose labels need a new rule per stage will cost more than one row.

The pair is the evidence. The test is the claim.

The objection

The evidence is thin, and the thinness is structural. One pair, compiled in one direction. A runtime that is entirely English in the measured build. A public dictionary holding one language across all its rows, so a non-English document cannot match it and leans on private dictionaries that start empty.

Under those conditions, language independence is a schema property plus one passing test, and the test was built by the team that built the pipeline. A skeptic may ask whether the two ledgers agree because the substrate is neutral or because the same code made the same choices twice.

The concession should be stated without hedging. The English-Spanish pass shows that the critical dimensions are not English categories smuggled into column names. It does not show that Spanish meaning survives in general, or that a third language will behave. No current multilingual capability may be claimed beyond the measured pair.

The concession is not that the test is weak; it is that the test has been run once, by the team that built the pipeline, on a pair whose target language the runtime already handled.

What survives is the shape of the test and the position of the seam. A claim that two compiles agree on roles, polarity, modality, attribution and time is falsifiable; a claim that the output reads well is not.

What the test does not measure

The test does not measure style, idiom or fluency. A translation can be structurally correct and read badly, or read beautifully and change the force. The ledger equivalence test catches the second and says nothing about the first.

That is a limit, not a defect. Structural fidelity and translation quality are different questions, and a book about the record should not pretend to answer the second. A quality review belongs beside the structural test, not inside it.

What would refute this

Compile a second pair independently, German into English or Spanish into Chinese, and compare at the claim level. If the ledgers systematically disagree on roles, polarity, modality, attribution or time where the source expressed them, predication-centricity is an English artifact and the general claim fails for that pair.

Two weaker results would also damage it. A normalization table that needs per-stage rules rather than one row per language would make the seam expensive. A cross-language match that requires forcing two languages onto one spelling would make the identity claim false.

A gap should be printed rather than smoothed. The corpus disagrees with itself about whether a second language has been demonstrated; one document says no, another reports the English-Spanish pass as measured. A reader cannot tell which is stale, and a claim with an undated edge will be read as stronger than it is.

The rendering test

The cross-language round trip is a test of the record, not of the translator. Render the synapse into both languages, recompile each rendering, and compare the six dimensions. If the structures match, the record survived the surface change; if not, the rendering or the compiler failed, and the span check says which. The span is the receipt.

The test is structural. The surface is not. The record is.

The record is the invariant and the surface is the variable, which is why the test compares structures rather than words.

A translation is a rendering, not a second record; the next Part asks who governs the distinctions.

30A · The Decompression Table

A word is a handle, not a definition. *Wagon* is a short string that lets a speaker invoke a package: a load-bearing platform, a container, axles and wheels, a hitch, a pulling agent, and a purpose. The package is what the word compresses.

The compression is useful and lossy. It is useful because a speaker can say *wagon* instead of describing the package; it is lossy because the package is not in the string, and a reader who has never seen a wagon cannot decompress it from the letters. The dictionary is the decompression table, and a dictionary that stores only synonyms is a table of handles with no bodies.

Two uses of a definition

A definition has two uses that are often conflated. The first is identification: it tells a reader whether a thing belongs to the kind. The second is explanation: it tells a reader why the parts and the purpose go together. A taxonomy serves the first and not the second; a decompression table serves both, and the second is what makes the entry useful to a machine.

A definition that is only a taxonomy can tell a reader that a wagon is a vehicle, and it cannot tell a machine why a wagon has wheels, which is the question a search or an alignment asks.

What is inside the package

The package is more than a classification. A wagon is a kind of vehicle, and the classification is one relation among several.

Relation	What it contributes	Wagon example
is-a	classification	a wagon is a kind of vehicle
instance-of	a particular in a class	this wagon is an instance of wagon
has-part	composition	a platform, axles, wheels
connected-to	arrangement	the platform is mounted on the axles
has-purpose	intended function	to move things

Continued on next page

(Continued)

Relation	What it contributes	Wagon example
participates-in	event roles	something pulls it; it carries a load
status	definitional force, modality and typicality	a platform is necessary; animal pulling is typical; machine pulling is possible

`instance-of` is not a smaller ingredient of meaning. It connects an individual to a class. Following `is-a` upward takes a reader to broader categories, not to simpler ingredients. A decomposition that uses only `is-a` produces a taxonomy, and a taxonomy is not a definition.

Five status dimensions

The strength of a component is not one field. Four questions are often compressed into “necessary, typical or possible”, and the compression causes alignment errors.

Dimension	The question	Wagon example
scope	does this concern a class, an instance or an event?	the definition concerns the class
definitional status	does it determine membership?	a platform is necessary; a color is background
modality	is it actual, possible, required or intended?	the pulling is possible, not actual
typicality	is it a default or a universal?	animal pulling is typical
evidence status	why accept it?	documented, inferred, disputed

The dimensions are independent. A statement can be about an instance, describe a possibility and come from an uncertain source at the same time. A single field cannot hold all three, and an alignment that treats “not documented” as “false” will manufacture contradictions.

Partial definitions

A definition can be partial, and partiality must be recorded. If A lists wheels and load carrying and says nothing about propulsion, A does not thereby permit onboard propulsion; A may simply be incomplete.

An entry therefore carries a completeness status and a list of known unresolved aspects. For a selected scope, a component can be marked complete, and the mark needs a justification rather than a default.

The alignment output uses three values, not two: supported, contradicted and unknown. A fourth state, supported by one source and contradicted by another, keeps both sources rather than forcing an early resolution.

The DAG, not the chain

A definition expands into other meanings, and the expansion is a graph.

```
wagon
 -> vehicle
 -> platform, wheel, hitch
 -> transport
 -> move
 -> place
 -> before, after
 -> purpose
 -> want, think, do
```

The arrows mean *is used in the explication of*. The graph is directed from the complex meaning to the simpler ones, and it is acyclic: a definition that returns to its own term has not explained it.

The graph is not a tree. *Move* is used by *transport*, by *pull* and by *rotate*; storing it once and referring to it is what makes the decomposition a DAG rather than a duplicated tree.

What NSM is

The natural semantic metalanguage is a research program, not a dictionary and not a formal logic. It explains a complex meaning by reductive paraphrase: the meaning is restated in simpler expressions, and the restatement is tested against ordinary use.

The vocabulary is a small set of proposed irreducible meanings, the semantic primes. Recent inventories list about sixty-five, in categories: substantives (I, YOU, SOMEONE, SOMETHING, PEOPLE); determiners (THIS, THE SAME, OTHER); quantifiers (ONE, TWO, SOME, ALL, MANY); evaluators (GOOD, BAD); descriptors (BIG, SMALL); mental predicates (THINK, KNOW, WANT, FEEL); speech (SAY). Actions and events (DO, HAPPEN, MOVE); existence and possession (THERE IS, HAVE); life and death (LIVE, DIE); time (WHEN, NOW, BEFORE, AFTER); space (WHERE, HERE, ABOVE, BELOW, INSIDE, NEAR, FAR); logic (NOT, IF, BECAUSE, CAN); similarity (LIKE).

The primes are meanings, not English words. Each language has its own exponents, and an English word that looks like a prime may be used in a different sense. A molecule is a complex meaning used as a unit inside a larger explication while remaining decomposable: *drink* can use *water*, and *water* can use *mouth*.

The grammar matters as much as the vocabulary. NSM permits particular frames, and a list of prime words in any order is not an explication. A prime-vocabulary check is weaker than a frame check.

NSM is not a claim that every word reduces to the list. It is a disciplined floor where a branch can stop and be compared with other branches; the primes are proposed, and the explications are reviewed.

Molecules

A molecule is an intermediate meaning reused inside larger definitions. *Transport* is a molecule; *wheel* is a molecule; *animal* is a molecule. The molecule is not a prime and not a full definition of the word; it is a package that has its own entry and can be expanded when a task needs it.

The molecule is what keeps the DAG small. Without molecules, every definition would expand to the floor, and the graph would be enormous; with molecules, a definition can stop at a reviewed package and expand only the branch a question requires.

A molecule that is wrong spreads the error to every definition that references it, which is why the molecule needs a version, a reviewer and a dependency list.

Two kinds of cycle are possible. A direct cycle is *wagon -> cart -> wagon*; an indirect cycle is a strongly connected component of several definitions. The graph rejects both, and an entry in a cycle is marked unresolved rather than declared reduced.

The acyclicity is required for definitional unfolding, not for every relation in the system. The ontology layer and the assertion layer may contain cycles; only the expansion edges are constrained, because only those are used to replace a term with its definition.

Unfolding is not inference

Unfolding replaces a molecule reference with its definition while preserving the variable bindings. Inference derives a conclusion from definitions and rules.

The acyclic graph helps unfolding terminate. It does not make inference correct, complete or cheap. A first implementation can offer reliable unfolding and structural comparison without claiming a general theorem prover; selected entailment checks can come later, on a defined fragment.

The expression language

A package is an expression, not a paragraph. The expression has typed variables, a predicate, role bindings, force annotations and molecule references. The same abstract syntax has three views: a human notation, an exchange format and a normalized store. The three are views of one meaning, not three meanings.

The language is constrained. A parser reads it; a schema validator checks the structure; a name resolver checks molecule versions; a scope checker binds the variables; a type

checker checks role fillers; a dependency checker rejects cycles. Passing all six is well-formedness, not truth. Meaning review is a seventh step, and it is not automatic.

The constraint is what makes comparison possible. A free-prose gloss cannot be compared component by component, and an unconstrained graph cannot say which edge is a definition and which is an assertion.

The name

The expression language is the **Semantic Package Language**, SPL. The name is descriptive: a package is a sense's structured expression, and the language is the constrained notation it is written in.

The synthesis is new, not the ingredients. Reductive paraphrase and molecules come from NSM; theta roles from linguistic theory and FrameNet/PropBank practice; structured meaning from AMR; sense links from WordNet; necessary and sufficient conditions from ontology. The distinctive loop is the combination: constrained packages, schema validation, force-aware comparison, reusable molecules, and separated lexical, definitional, ontological and assertion layers. Whether it is important is empirical, and the test decides.

The specification

SPL 0.1 is specified in Appendix G. It defines four profiles—Core, Content, NSM and Domain—and the checks that separate syntax, reference, scope, type, dependency and review. It defines exact, necessary and sufficient definitions, and only the exact form may be unfolded as a meaning-preserving substitution. It defines the conflict taxonomy and the alignment tests, with proof status separate from similarity.

The grammar belongs in the appendix because it is a contract, not an argument.

Theta roles

The participant vocabulary is the book's fifteen roles: agent, patient, theme, experiencer, recipient, beneficiary, time, location, source, destination, manner, instrument, cause, reason and attribute.

The roles are an engineering inventory, not a claim about universal grammar. They connect the packages to FrameNet and PropBank practice without requiring either, and they keep the participant structure intelligible to a reader who already works with semantic roles. Patient versus theme, and cause versus reason, need operational definitions; the roles chapter gives them.

A role is not a substitute for composition. *Alex thinks Bea did not leave* needs a propositional complement and a negation, not a location or beneficiary spoke. The role inventory organizes participants; it does not encode every relation between predications.

Safe expansion

Expanding a molecule resolves the version, checks the arguments, substitutes parameters, renames local variables to avoid capture, preserves scope and keeps a trace to the source package.

The renaming is not a detail. Two molecule calls that both use a local variable `e` must not become claims about the same event; variable capture turns two packages into one, and the error is invisible after expansion.

The floor

The proposed floor is the NSM primes: sixty-five meanings that the natural semantic metalanguage treats as irreducible, with a constrained grammar for combining them.

The proposal is a hypothesis, and it is treated as one. Three claims are often conflated and should be kept apart:

1. A lexicon benefits from explicit, compositional definitions. 2. Definitions can be organized as an acyclic dependency graph. 3. Every meaning can be represented with the sixty-five primes and their permitted combinations.

The first two are engineering claims that can be tested now. The third is a theoretical commitment that requires evidence, and the evidence is not the existence of a DAG.

Termination is not correctness. A definition can reach a prime and still be wrong; a graph can be acyclic and still lose the distinction it was meant to preserve. The test is whether the decomposition keeps the distinctions a question depends on.

The wagon

A first-pass explication separates the kind from the design.

```
wagon_1(x):
 x is a physical artifact
 exists w: wheel(w) and has_part(x, w)
 intended_function: transport(carrier=x, cargo=y, origin=a, destination=b)
 under conditions C:
 pulling_event(agent=p, object=x) can_result_in
 movement_of(x) and movement_of(y)
 conditions: cargo_supported_or_retained_by(y, x)
 status: partially formalized
```

The sketch is not a finished logic. It is a way to make the missing commitments visible: the same `y` is supported, moved and relocated. The pulling is an event under conditions; the function is intended, not actual; and the conditions are listed rather than hidden behind an operator.

`has_part(x, wheels)` is ambiguous between "has some wheels", "has a particular wheel" and "has the class Wheel as a part". The sketch writes the existential form and leaves the other two for the lexicon.

The design variant adds the parts and their arrangement: a platform, a container, two axles with wheels, and a hitch. The variant is not the meaning of every wagon; it is one construction that satisfies the kind.

The purpose branch is the one that reduces furthest. *Purpose* expands into a configuration of people wanting an outcome, thinking that an action with the object can produce it, and doing something because of that. *Transport* expands into things being in one place before, moving because of an action, and being somewhere else after. The variables must be shared: the load supported by the platform is the load that moves, and the place before is not the place after.

The wheel branch is the one that resists. *Wheel* expands into a component, a roughly circular form, rotation around an axis, and contact with a surface. Each of those terms is still complex, and the honest entry marks the branch `partially decomposed` rather than forcing it to a prime.

```
wheel:
 status: partially decomposed
 unresolved: shape, rotation, attachment
```

The unevenness is the result, not a failure. Some branches reach the floor; some reach a molecule; some stop at a declared gap. A decomposition that claims uniform depth is either shallow or dishonest.

The stopping rule

A branch stops for one of three reasons. It reaches a prime; it reaches a reviewed molecule; or it reaches a declared gap. The three are different, and the entry records which.

A branch that stops because the analyst cannot think of a simpler paraphrase is a gap, not a prime. The status is `unresolved`, and the entry says so.

A decompression table is not a dictionary

A dictionary lists handles and their synonyms. A decompression table lists handles and their packages. The first helps a reader find a word; the second helps a machine compare a meaning. The two can share a page and they are not the same artifact.

What it buys

The compression layer is useful when a system must work with meaning rather than spelling.

Search without the word. A query can describe the package—a thing that carries a load, has no motor, and can be pulled—and the lexicon can match the description even when the user does not know the term.

Translation with a loss report. A source term and a target phrase can be compared on the package. The report says which components survived and which were dropped, instead of returning a fluency score.

Claim checking. An AI that says a wagon propels itself can be checked against the selected sense. The check is not a proof of truth; it is a mismatch between the claim and the definition, and the mismatch is reportable.

Data integration. Two systems that call the same thing *towable carrier* and *unpowered cargo wagon* can compare decompositions rather than labels, and the comparison can say whether one is narrower, broader or contradictory.

Controlled expansion. A task expands only the branch it needs. A capacity question expands the load and the platform; a failure question expands the parts and the connections; a translation question expands the sense distinctions that matter in the target language.

RAG validation. A retrieval pipeline expands only the molecules the question needs, interprets the retrieved claims, and checks the generated answer. It flags force escalation, a dropped participant, or an unsupported necessity, and names the components matched, assumed and unknown.

Knowledge-graph governance. A molecule change lists the dependent classes, mappings and queries; high-value classes can receive packages without rewriting the graph.

The layer is a critic, not a generator. It sits beside the ontology, the graph and the retriever; it does not produce the answer. It checks the claims an answer makes against the packages and the evidence, and it returns a report rather than a verdict.

What it does not

A definition is not evidence about an instance. The lexicon can say a wagon is intended to carry loads; it cannot say this wagon can carry nine hundred pounds.

The distinction is the same one the book makes between a claim and a row. The definition is a claim about a kind; the instance is a claim about an object; the two are different records, and the second needs its own evidence.

Decomposition is also not grounding by itself. A definition can be inspectable and still floating; grounding is the tether to evidence, and the tether is a separate relation. The decompression table is not the tether.

Not a step toward AGI

A semantic layer is not a step toward general intelligence.

General intelligence requires learning from raw experience, perception and grounding in a world, planning and agency, autonomous concept revision, handling of novelty outside any pre-authored lexicon, and integration of all of those at scale. SPL addresses one narrow slice: making selected conventional lexical meanings explicit, comparable and checkable.

What it can contribute is narrower than a breakthrough. It can reduce silent collapses of modal force, such as *intend* becoming *promise* or *possible* becoming *certain*. It can make ontology and schema alignment explainable. It can expose underspecified requirements. It can serve as a critic or constraint layer that a language-model system consults.

Those are reliability and auditability improvements for tasks that already sit inside today's systems. They are not a foundational step toward general intelligence.

The history is sobering. Formal semantics, situation semantics and several knowledge-representation languages have been proposed as important steps, and none became the critical path. The bottleneck has been learning, grounding, scale and the long tail of real-world variation, and a hand-authored meaning layer does not remove any of them.

The claim the book can defend is smaller and testable: explicit, validated meaning packages make some costly errors visible and correctable. That is worth building. It is not the same as building a mind.

Necessary and sufficient

Alignment depends on which components are necessary.

A necessary condition must hold for membership. A sufficient condition is a route into the class. A typical condition is a default that generates an expectation without licensing a contradiction when it fails.

The comparator computes coverage over the necessary components first. A missing necessary condition blocks equivalence and may block subsumption. An extra necessary

condition supports a narrower relation. Typical conditions shape the residual note, not the veto.

Shared necessary conditions do not prove equivalence. Two classes can share every documented condition and differ in an undocumented one, which is why the completeness field is part of the record rather than a note beside it.

Alignment by decomposition

Two ontologies may use different terms for the same concept. Comparing their decompositions can propose a relation: equivalent, narrower, broader, overlapping or contradictory.

The comparison is directional. One question is how much of A's definition B supports; the other is how much of B's A supports. A symmetric score hides the difference, and the difference is what names the relation.

Contradictions are checked first. If A requires no onboard propulsion and B requires onboard propulsion, the contradiction blocks equivalence even when every other component matches. Unknown conditions stay unknown; they are not counted as support.

The full method belongs to the alignment chapter. The decomposition supplies the substrate: a definition that can be compared component by component, with the strength of each component declared.

The mapping record has fields: source and target versions, the proposed relation, the supporting components, the conflicting components, the unknowns, the assumptions, the method and the review status. The relation is one of equivalent, narrower, broader, overlapping, incompatible or undetermined. The record is the product; a score is a summary of it.

The metalanguage

The expression language needs a grammar, not just a vocabulary. A combination of primes and molecules can be ill-formed, and the system needs a type system or a constrained grammar to refuse it. The grammar also has to say how force, negation, quantification and variable scope are expressed, because those are where a decomposition that looks reduced becomes ambiguous.

The alignment report

A comparison produces a report, not a score. The report lists shared components, components specific to each side, contradictions, unknowns, the proposed relation and the evidence for each component. The score is a summary of the report, never the report.

A contradiction blocks equivalence even when every other component matches. An unknown is not a contradiction; it is a question. The two are kept apart because the repair is different.

Here's an example: A lists wheels and load carrying; B adds no onboard propulsion. The report says shared structure and function, B-specific no propulsion, A unknown on propulsion, proposed relation unresolved between equivalence and narrower. The unknown blocks the equivalence claim.

The first build

The first build is smaller than the architecture. Take fifty to a hundred and fifty senses in one domain, with near-neighbors and atypical cases. Store the plain gloss, the structured expression, the dependencies and the status flags. Implement acyclicity, variable binding and undeclared-complexity checks.

Compare against three baselines: plain glosses plus an LLM, embeddings, and an existing ontology matcher. The metrics are concrete: alignment precision with explanations, reduction in false commitments, ambiguity-detection quality, reviewer time and abstention quality.

A build that compares against only one baseline cannot tell whether the gain comes from structure, from the glossary, or from the model that was already in the pipeline.

A repeatable reduction in one costly error type may justify expansion, if the benefit exceeds authoring, review, maintenance and runtime cost. A single win does not.

A first build that ignores the cost of authoring and review can look successful on a benchmark and fail in production, because the expense of writing and maintaining the explications is paid after the demonstration.

The test that settles it

The design is not settled by more formalization. It is settled by a head-to-head test on a narrow domain.

The test compares six baselines: labels and embeddings; plain glosses plus an LLM; glosses with a required output shape; packages with a structural comparator; packages with controlled expansion and reasoning; and an existing ontology matcher. The tasks are alignment with explanations, commitment-preserving extraction, and unsupported claims in retrieval. The metrics are false commitments, alignment precision, ambiguity detection, reviewer time and maintenance cost.

The baseline is the strongest credible alternative a competent team would deploy, not a weak prompt.

The unusual combination is the claim: force-first vetoes, exact-versus-necessary substitution, directional residuals, a critic layer, and a separation of well-formedness from adequacy. If the test shows no repeatable reduction on a costly error type, the layer is an elegant elaboration of known ideas, and the book says so.

SPL 0.1 is frozen for the test. A new operator is added after a measured failure, not before it.

The test

Take a hundred to three hundred senses in one domain, including near neighbors, ambiguous words and broken instances.

For each sense, store the plain explication, the structured explication, the dependencies, the necessary/typical/possible status and the unresolved branches.

Then run five tests: description-to-concept retrieval, near-neighbor discrimination, claim checking, translation loss, and expansion audit.

The strongest result is not that every path reaches sixty-five primes. It is that making the package explicit lets a system do something a plain dictionary, an embedding or an unaided model cannot, and the something is measured on a benchmark rather than asserted.

Three failures are common. The decomposition is a taxonomy; the definitions terminate by declaring difficult words primitive; the test measures fluency rather than meaning. Each failure produces a lexicon that looks complete and cannot answer a contrast case.

A word is a handle. The book's job is to show the body behind it, and to mark where the body is still missing.

30B · Decomposing Verbs and Aligning Meanings

A noun decomposes into conditions on a thing. A verb decomposes into a situation: participants, a speech act or a change, the content the participants hold, and the conditions under which the situation counts as the verb's sense.

The difference matters because a verb's meaning is not a list of participants. *Promise*, *persuade* and *intend* share an agent and a content and differ in what the agent does with the content and what the addressee is entitled to expect. A decomposition that lists the participants and stops has the frame and not the act.

The verb is a situation

A verb package declares its participants, then its structure.

Here's an example: *Alex promised Bea to come tomorrow.* The participants are Alex, Bea and the action of coming. The structure is a saying, a future action, a wanted recognition, and a reliance condition. The content of the promise is not the word *promise*; it is the action the promise is about.

The package separates the lexical meaning from the sentence content. *Promise to come*, *promise to pay* and *promise to stop* share the promise structure and differ in the action slot. That separation is what lets one package serve many sentences.

The package envelope

A package has a sense identity, parameters, a body, a definition direction, molecule references, a completeness status, provenance and a version. The envelope is the record; the body is the expression.

The direction is not decoration. A necessary condition says what must hold; a sufficient condition says what is enough; an exact condition says what is both. Only the exact form can be unfolded as a definition, and the difference is where most alignment errors enter.

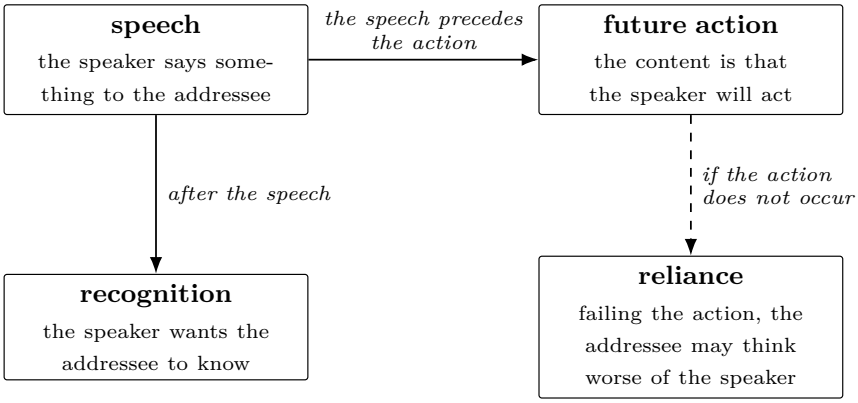
The two errors

The first error is a package that is a taxonomy: it says what kind of thing the sense is and nothing about its structure. The second is a package that is a paraphrase: it replaces one complex predicate with another and calls the replacement a decomposition. Both parse; neither decomposes.

The sequence

Order matters because the components are not a bag. The speech comes before the action; the recognition is about the addressee's knowledge after the speech; the reliance is conditional on the action's failure. A list of four features loses the order, and the order is part of the meaning.

The figure is that order: the fourth component hangs on the action's failure, not on the speaker's sincerity.



a prediction *shares the speech, the future action and the recognition, and stops: no reliance edge, so its failure leaves the predictor blameless. The fourth component is the whole difference.*

Figure 4: The promise's components in order: the speech before the action, recognition after the speech, and reliance conditional on the action's failure.

A promise that is insincere is still a promise, and a prediction that is accurate is still not a promise, because the difference is the reliance condition and not the speaker's belief.

The promise

A promise has four components, and the fourth is the one that distinguishes it from a prediction.

Component	Content
speech	the speaker says something to the addressee
future action	the content is that the speaker will do something after the speech
recognition	the speaker wants the addressee to know this
reliance	if the action does not occur, the addressee can think something bad of the speaker because of the speech

The first three are shared with a prediction or a statement of intention. The fourth is the normative edge. A prediction can fail and leave the predictor blameless; a promise that fails leaves the promiser exposed to blame, and the exposure is part of the meaning.

A promise does not require sincerity or success. The speaker may intend not to perform, and the action may not occur; the promise is still a promise. That is why the package marks the desire to perform as typical, not necessary.

Persuade

Persuade is harder because it involves a change in another person's thinking.

Here's an example: *Maya persuaded Leo to leave.* The package needs a saying, an intended influence on Leo's thought, and a resulting change: Leo came to think that leaving was something to do.

The three predicates—saying, intended influence, resulting change—are not primitives. The package keeps them as unresolved dependencies. That is the honest state. A decomposition that writes *Maya caused Leo to think* has replaced one complex predicate with another and called it progress.

The contrast set is what tests the analysis:

Verb	**What it adds or removes**
inform	the change in thought, without the intention to change action
request	the action is asked for, without the change in evaluation
threaten	the addressee's bad outcome is the lever
force	the change is not through thought
try to persuade	the intention, without the resulting change

A package for *persuade* must differ from each of these in at least one necessary component. If it does not, the decomposition has lost the distinction it was meant to keep.

The contrast table is also the test set. Each row names a verb whose package must differ from *persuade* in at least one necessary component; if the difference is only typical, the analysis is too weak.

Abstract concepts

Abstract meanings are configurations, not physical parts.

Here's an example: disappointment. Before the event, Bea wanted an outcome and thought it could happen. At the event, she knew it did not happen. Because of that, she felt something bad. The configuration uses want, think, know, happen, before and because; none of them is a physical component.

The package is a candidate, not a verdict. It must be tested against grief, regret, frustration and an ordinary unfulfilled wish. If the same configuration covers all five, it is too broad; the test is the contrast, not the elegance of the paraphrase.

Abstract concepts are where decomposition is most tempting and least reliable. The configuration is a hypothesis, and the contrast set is the evidence. A package that cannot separate disappointment from regret has not decomposed the concept; it has paraphrased it.

Competing paths

Two decompositions of the same sense can disagree without either being ill-formed. There are four cases:

Case	Response
compatible contributions	keep both; they describe different aspects
typical versus instance	keep both; the instance does not falsify the typical claim
competing definitions of one sense	keep as separate candidates with provenance; do not conjoin
logical contradiction after expansion	mark the premise set inconsistent; do not merge

A cycle is a fifth case and is not a definition at all. *Wagon -> cart -> wagon* has not explained either term; the entry is unresolved.

The statuses are distinct: ill-formed, internally inconsistent, competing but coherent, and compatible under different force. A system that collapses them into one *error* status cannot tell a reviewer what to do.

Compatible contributions are not a conflict. Two paths that add different aspects—one the carrying function, one the wheeled construction—are both true, and the package can keep both. The conflict begins when two paths assign incompatible force or scope to the same component.

Alignment

Alignment compares two packages, not two labels.

Necessary conditions are computed first. A missing necessary condition blocks equivalence and may block subsumption. An extra necessary condition supports a narrower relation. Typical conditions shape the residual, not the veto.

Here's an example: A is a wheeled load carrier; B is a wheeled load carrier without onboard propulsion. If both definitions are complete, B is narrower than A. If A's propulsion policy is undocumented, the relation is undetermined, not equivalent and not narrower. The unknown blocks the stronger conclusion.

The report names the shared components, the side-specific components, the contradictions, the unknowns and the proposed relation. A score is a summary of the report; it is not the relation.

The worksheet

The worksheet is five steps: fix the sense; name the participants; separate the lexical structure from the sentence content; draft the components; test the contrasts. The steps are ordinary; the discipline is refusing to skip the contrast.

The noun

A noun package is a condition on an entity: category, structure, function and the status of each. The wagon is the worked example; the wheel is the branch that resists.

The report

The comparator returns a relation and a residual. The residual names the components that were unsupported, the extra restrictions on one side and the unknowns. A report with a relation and no residual is a score with a label.

What alignment does not prove

Alignment does not prove that the classes are identical in every model; it proves a relation under the selected theory and the accepted definitions. A different theory or a revised definition can change the result. The report should name the theory and the versions.

The seed domain

The first domain is commitment and speech-act verbs: intend, expect, request, offer, accept, promise, approve, cancel, complete. The domain is chosen because the errors are expensive and the contrasts are clear.

The evaluation

The evaluation is not whether the package looks precise. It is whether it reduces false commitments or bad alignments on a seed domain, against a strong prompt and an ontology matcher, at a cost that justifies the authoring.

The package's honesty

A package is honest when it names what it has not resolved. The wheel is partially decomposed; *persuade* keeps saying, influence and change as unresolved; the promise marks the desire to perform as typical rather than necessary. The unresolved list is part of the meaning, not a defect to hide.

The eight cases

The test set is eight pairs, one feature changed at a time: role reversal, negation, extra restriction, missing restriction, different sense, equivalent definition, ambiguous scope and contradiction. The pair is the unit; the report must name the feature.

Two definitions can share every documented condition and still differ in an undocumented one, which is why the relation is undetermined rather than equivalent.

The output

The output is not a label. It is a relation, a residual, and a sentence the reviewer can read: what matched, what blocked, and what remains unknown. The residual is the report.

A verb package that cannot say which component a contrast case removes has not decomposed the verb, because the contrast is the only evidence that the component was doing work.

Three failures are common. The package lists participants and calls it a definition; the contrast set is missing; the alignment reports a score with no residual. Each failure produces a package that looks complete and cannot answer why a mapping is blocked.

The output is a sentence.

The distinction

A package that keeps the order, the participants and the force can answer why a promise differs from a prediction, and a score cannot. The comparison is useful only when it names the feature that blocks the stronger relation, because that feature is the repair site. The pair is the test. The relation is a claim. The order is the meaning.

An alignment that reports only a percentage leaves the reviewer with no repair, because the percentage does not say which component failed or which definition must change.

A contrast set that never removes a necessary component cannot show that the component is necessary, and a package tested only on positive examples cannot show that it decomposes anything.

An alignment report that names the relation, the residual and the exact versions is auditable, because a reviewer can challenge one component without re-running the whole comparison. A package that cannot name the feature that blocks equivalence has not compared two meanings; it has compared two strings and reported a number that cannot be repaired. The test is not whether the package looks precise; it is whether a reviewer can use the residual to repair the definition, which is the only operation that changes the next comparison. A contrast set that removes one component at a time is the only evidence that the package's necessary conditions are necessary rather than decorative.

A comparison that gives a relation without the component that blocked a stronger one leaves the reviewer with a verdict, and a verdict cannot repair the definition that produced it. The test set is only as good as its negative pairs, because a package that passes only the positive examples has shown that it can describe, not that it can distinguish.

An evaluation that measures only errors caught, and not the review time or the authoring cost, will recommend a layer that is accurate and unaffordable.

An evaluation that measures only the errors caught, and never the review time or the authoring cost, will recommend a layer that is accurate and unaffordable, which is not a recommendation. The test is whether the layer changes a decision on a costly error type, not whether it produces a representation that looks more precise than a gloss, because precision that does not change a decision is decoration. A package that is correct and never used has not earned its authoring cost, and a package that is used and never challenged has not been tested, so the evaluation must measure both. The decision is the test.

The test

Take one verb and write its participants, its structure and its contrast set.

Then remove one necessary component and ask whether the verb still applies. If it does, the component was typical, not necessary. If it does not, the component is part of the sense.

Then compare two packages with one feature changed. If the report cannot name the feature that blocks equivalence, the comparison is a score, not an analysis.

A verb is not a list of participants, and an alignment is not a similarity score. Both are claims about what must hold, and both are judged by the distinction they keep.

30C · The Limits of Decomposition

A decomposition is worth its cost only where it changes a decision. Everywhere else it is an expense with no customer: authoring time, review time, a molecule to maintain, and a definition that looks precise and is never used.

The previous two chapters argued that a sense can be represented as a package and that packages can be compared. The other side is what resists decomposition, what the decomposition costs, and what evidence would show that the cost is justified.

What resists

Not every branch reaches a prime, and not every branch should be forced.

Branch	Why it resists	Honest status
geometry and mechanics	shape, rotation and contact need a spatial model, not a paraphrase	partially decomposed
social and institutional concepts	authority, promise and ownership depend on conventions and enforcement	molecule plus context
causal relations	a cause is inferred from evidence and background assumptions	unresolved or domain-specific
perception and grounding	a definition is not an image, a measurement or an experience	outside the layer
individual identity	two records can share every description and still be two things	outside the layer
novel and technical terms	the sense is minted in a community the lexicon has not absorbed	gap, then mint

The wheel is the worked case. A first pass says a wheel is a component with a circular form that rotates around an axis and contacts a surface. Each of those terms is still complex, and the honest entry marks the branch partially decomposed rather than forcing it to a prime.

The table is not a list of failures. Each row is a boundary between what the layer can state and what another system must supply: a spatial model for the wheel, an institutional

record for the promise, evidence for the cause, a measurement for the perception. The boundary is the useful output; pretending the boundary is not there is the failure.

The institutional case

A promise depends on a convention that the addressee may rely. The convention is not in the words and not in the speaker's head; it is in the institution that enforces it. A package can state the reliance condition and cannot reduce the institution to primes.

An institution is not a prime and not a molecule. It is a system of enforcement that the package can cite and cannot reduce, which is why the promise branch stops at a social boundary rather than at a semantic one. The gap is the honest boundary, and the institution is the system that fills it.

The technical case

A new technical term has no reviewed molecule and no prime paraphrase. The first package is a candidate with a definition, examples and unresolved dependencies. It becomes a molecule only after review; until then, it is a mint in progress.

Three claims, not one

The material conflates three claims, and the book must keep them apart.

1. Structured, validated packages improve selected information-processing tasks. 2. Reusable, acyclic definition dependencies are maintainable at a useful scale. 3. Every relevant meaning can be represented with a universal prime inventory.

The first two are engineering claims that can be tested now. The third is a theoretical commitment that requires evidence, and the existence of a DAG is not evidence for it.

A project can succeed on the first two and remain agnostic on the third. It can also fail on the first two while the third is true, which is why the theoretical question cannot substitute for the test.

The cost

The cost is not the storage. It is the labor.

Cost	What it consumes
authoring	a reviewer writes and validates a package
review	a second reader checks the sense, the force and the contrasts
maintenance	a molecule changes and its dependents are re-reviewed

Continued on next page

(Continued)

Cost	What it consumes
propagation	a wrong molecule spreads to every package that references it
coverage	the layer is silent outside its coverage, and silence looks like absence
overgeneration	packages are written because the schema invites them, not because a decision needs them

The last two are the dangerous ones. A layer that covers ten percent of the vocabulary and is silent about the rest can be mistaken for a layer that has checked everything. A team that writes a thousand packages and changes no decision has produced a maintenance liability with a schema.

The cost is easiest to see in a molecule change. One reviewer edits a molecule; every dependent package must be re-read, every mapping that cites it must be re-checked, and every cached comparison must be invalidated. The cost is not the edit; it is the fan-out.

The coverage illusion

Coverage is a percentage of the vocabulary the layer has packages for, and it says nothing about the decisions the layer supports. A layer that covers ten percent and changes a decision is worth more than a layer that covers ninety percent and changes none.

The wrong question

The wrong question is whether the layer can decompose everything. The right question is whether it can decompose the senses that carry a costly decision, and whether the decomposition changes the decision. A universal lexicon is a research program; a seed domain is a product.

The right question is narrower: for this domain, does the package reduce a false commitment or a bad alignment enough to pay for its authoring? If yes, extend. If no, stop.

The stopping rule

A branch stops for one of three reasons, and the reason is recorded.

- It reaches a prime.
- It reaches a reviewed molecule.
- It reaches a declared gap.

A branch that stops because the analyst cannot think of a simpler paraphrase is a gap, not a prime. The status is `unresolved`, and the entry says so.

The depth budget is also a stop. A task that needs the carrying function does not need the metallurgy of the axle; expanding to the budget and declaring the residual is cheaper and more honest than expanding to the floor for every query.

The residue

The residual is not a failure. It is the record of where the layer stopped, and it is what makes the layer auditable. A package with no residual is either complete or hiding its gaps.

The stopping record

A stop is recorded with a type, a reason and a reviewer. The type is prime, molecule or gap; the reason is what the analyst could not reduce; the reviewer is who accepted the stop. The record is what makes the stop a decision rather than a surrender.

The cost test

The cost test compares the package's authoring and review hours with the errors it prevents. If the hours exceed the errors, the package is a candidate for removal; if the errors exceed the hours, the package has a case.

The domain

The seed domain is small on purpose. A narrow domain makes the contrast set visible and the authoring cost countable, and a broad domain makes both invisible.

The cost of silence

The layer is silent outside its coverage, and a reader who does not know the boundary will treat silence as absence, which is why the coverage map is part of the output. The map is the boundary.

The third claim

The third claim, that every meaning reduces to a universal prime inventory, is not needed for the layer to work. It is a research question, and the evaluation does not depend on it. The third claim is not the product.

The test

The evaluation is a head-to-head test on a narrow domain, not a demonstration of the formalism.

The seed domain is commitment and speech-act verbs: intend, expect, request, offer, accept, promise, approve, cancel, complete. The baselines are labels and embeddings; a strong gloss-plus-model prompt; a gloss with a required output shape; packages with a structural comparator; packages with controlled expansion and reasoning; and an existing ontology matcher.

The test is run on pairs, not on single senses. Each pair differs by one feature, and the report must name the feature that blocks the stronger relation. A test that scores the pair without naming the feature is a similarity score with a different name.

The two failures

The first failure is a package that claims completeness it does not have. The second is a package that declares so many gaps that it cannot be used. The first hides the boundary; the second mistakes the boundary for the work.

The metrics are false commitments, alignment precision, false equivalence, false alarms, appropriate abstention, reviewer time, authoring time, maintenance cost and runtime. The pass condition is a repeatable reduction in one costly error type, large enough to exceed the authoring and review cost. A single win on a toy set does not pass.

The result is not claimed in advance. If the treatment does not beat the strong prompt and the structural comparator, the layer is an elegant elaboration of known ideas, and the book says so.

Not a step toward AGI

A meaning layer is not a step toward general intelligence. General intelligence requires learning from raw experience, perception and grounding, planning and agency, autonomous concept revision, novelty handling and integration at scale. The layer addresses conventional lexical meaning and none of the rest.

What it can do is smaller: reduce a force collapse, make an alignment explainable, expose an underspecified requirement, or give a checking layer something to consult. Those are reliability improvements on tasks inside current systems, and they are worth having without being a theory of mind.

The test of a decomposition

Take one sense. Decompose it until it stops, and write down why it stopped.

If it stopped at a prime, record the prime. If it stopped at a molecule, record the molecule and its version. If it stopped at a gap, record the gap. If it stopped because the budget ran out, record the budget.

Then ask what decision the decomposition changes. If the answer is none, the package is a candidate for removal, not a permanent part of the lexicon.

A decomposition that changes no decision is a package looking for a customer. The customer may arrive later, and the package should be marked as speculative rather than deleted; the record says which.

Then ask what it cost. If the authoring and review cost exceeds the error it prevents, the package has not earned its place, however precise it looks.

A layer that covers ten percent of the vocabulary and changes a decision is worth more than a layer that covers ninety percent and changes none, because the decision is the product.

A package that cannot say why it stopped cannot be reviewed, and a review that cannot see the stop is a review of the prose rather than of the decomposition.

Three failures are common. The package hides a gap behind a precise operator; the package covers a domain and changes no decision; the molecule change is not propagated. Each failure produces a lexicon that looks complete and cannot be maintained.

The boundary is the output. The residue is the record. The decision is the product.

The value of a decomposition is the distinction it keeps, and the distinction is worth keeping only when someone acts on it.

PART VI
The Ontology

who governs the distinctions.

31 · The Nine Pains

A program can have addresses for five senses of *customer* and still have no answer to which sense governs; the failure is not a missing address. It is a missing decision about which commitment is in force, and it shows up in nine recognizable ways before anyone calls it an ontology problem.

Nine complaints, one condition

The nine pains are usually presented as nine unrelated failures: the wrong tool, the wrong upper ontology, the wrong vendor, bad timing. Each has a different purchase attached, and a program that treats them one at a time buys nine remedies and fixes none.

The diagnosis is narrower. Each pain is a failure to recover, decide or export a commitment that the artifacts already contain. The schemas, ETL jobs, glossaries, enums and mapping tables are evidence of what the organization already commits to; the missing piece is the record of which commitments are in force.

A pain is a symptom with a named broken transition behind it. It is not a rung on a maturity ladder, and it is not a mood; the transition is what the diagnostic names, and the first artifact is what would prove the score.

The nine

The nine pains, in the customer's words, with what each looks like and what it costs.

Pain	What it looks like	What it costs
Invisible ontology	new projects redo schema archaeology	weeks per integration, on every new source
Definition civil war	*customer* means five things	a merge nobody can audit
Greenfield trap	twelve to eighteen months in an editor, drifting from operations	a model of a company that has moved
Ontology theatre	generated classes with confidence and no origin	a vocabulary that cannot be falsified

Continued on next page

(Continued)

Pain	What it looks like	What it costs
Conformity theatre	spreadsheet alignment, claimed conformance	nothing demonstrable to a buyer or an auditor
Silent drift	the schema changed, the types froze	reports that stay subtly wrong
AI amplification	unprovenanced types fed to retrieval and agents	hallucination becomes structural
Mappings die in the ETL	the real knowledge sits in code	each project re-derives it
Lock-in anxiety	the tool wants to become the place knowledge lives	leaving means rebuilding

The count grew from seven to nine. The two additions are real: mappings dying in the ETL and lock-in anxiety break transitions the first seven did not name. The count is not the claim; the routing is.

The five customers

Five teams use the word *customer* and mean five different commitments. Sales means the party that placed an order. Compliance means the party that passed KYC. Finance means the billing account. Product means the workspace. Support means the end user who opens a ticket.

Each schema is internally consistent. The ETL joins them on `cust_nm`, and the join is the merge; a report that counts customers counts whichever sense the join happened to keep, and no one can say which, because the merge left no ruling.

The civil war is not a disagreement about words. It is five commitments on one string, and the first artifact is a conflict row with two holders and two dates. The row does not settle the meaning; it makes the disagreement reviewable.

The hundred-acquisition estate

A hundred-acquisition estate has three schemas, one mapping table and one ETL job joining `cust_nm`. The mapping table is the real ontology: it says which source column becomes which target column, under which transformation, for which reporting year.

The knowledge is in the code. A new integration re-derives it because no one recovered the mapping as a record with an owner and a date. That is the invisible ontology: the organization already commits to a structure, and the structure is not written where anyone can inspect it.

The first artifact is a candidate register with a span per row. The span points at the mapping line, the schema column or the glossary entry, and the register says what

the artifact already commits to. Weeks per integration become a review instead of an archaeology project.

The greenfield trap

A greenfield program spends twelve to eighteen months in an editor. The model is coherent, the editor is productive, and operations keep changing. By the time the model ships, it describes a company that has moved.

The trap is not the authoring. It is the authoring before recovery. The authored model is a set of candidates that still has to be checked against the artifacts, and that check is what recovery does.

A program that recovers first does not avoid the editor. It enters the editor with a register of what the organization already commits to, and the editor becomes a place to decide among candidates rather than a place to invent a world.

The theatre

Ontology theatre is a generated class tree with confidence labels and no origin. The classes look like a model, the confidence looks like evidence, and no one can falsify either.

Conformity theatre is the spreadsheet version. A team aligns two vocabularies in a worksheet, claims conformance, and has nothing demonstrable to a buyer or an auditor. The artifact is a row in a file that no one can replay.

The first artifact for the family is a provenance span and a non-empty loss report on export. The span says where the class came from; the loss report says what the export dropped. A generated class with no span is a candidate, not a commitment.

The ETL is the ontology

The mapping table is where the organization's commitments actually live. It encodes the join, the transformation, the exception and the reporting year, and it does so in a language that only the ETL team reads.

When the mapping dies in the ETL, each new project re-derives it. The re-derivation is not free, and it is not consistent: two projects derive two mappings from the same source and neither is recorded as the successor.

Recovering the mapping is the first artifact. The register records the source column, the target, the transformation, the owner and the date. The ETL still runs; the knowledge is no longer only in the code.

Lock-in anxiety

A tool that wants to become the place knowledge lives is a tool that makes leaving expensive. The anxiety is rational: if the ontology exists only inside the vendor's model, an export is a rebuild.

The only test is an export into a second tool, counted for loss. The test is specified here and not run. The artifact is the loss report: what survived, what was generalized, what was omitted and what was introduced.

A vendor that will not produce the loss report is asking the buyer to trust the exit. The diagnostic records the anxiety and names the test rather than pretending the anxiety is a mood.

The scoring rules

Each pain is scored 0—3: absent, suspected, visible, structural. A score above one owes an artifact. A score of one is a suspicion that may be recorded without a document. A score of zero needs no evidence.

The rule is not a measurement. It is the difference between a diagnostic and a survey. A score of three with no schema, ticket, missed date or report behind it is theatre by construction, whatever the number.

The score is a route, not a rank. The family sum names the first artifact, and the choice between two families is the workshop's output. The instrument deliberately does not choose for the room.

Three families

The nine sort into three families, and each family names one first artifact.

Family	Pains	The failed commitment	First artifact
Recovery	invisible ontology, mappings die in the ETL, greenfield trap	a commitment that exists but was never recovered	a candidate register with a span per row
Governance	definition civil war, silent drift, conformity theatre	a commitment that was recovered but never decided	a conflict row with two holders and two dates
Representation	ontology theatre, AI amplification, lock-in anxiety	a commitment that was decided but never exported with its provenance	a provenance span and a non-empty loss report

The families are not stages. A program can have a recovery failure and a representation failure at the same time, and the first artifact for each is different.

The family is what makes the diagnostic discriminate. Two programs can carry the same total score and need different repairs, because the high cells sit in different families.

Two programs, one total

Here is an example. Program A is a hundred-acquisition estate: three schemas, one mapping table, one ETL job joining `cust_nm`, and a 2019 glossary. Program B is a twelve-month greenfield authoring program: one model, an editor, no register.

Both are scored 0—3 on the nine pains, in the family order: recovery, governance, representation.

```
Program A: 3 3 0 | 3 2 2 | 0 1 2 = 16
Program B: 1 0 3 | 1 2 2 | 2 2 3 = 16
```

The scores are the author's illustration, not measurements. The point is the route, not the number.

Pain	Family	Program A	Program B
invisible ontology	recovery	3	1
mappings die in the ETL	recovery	3	0
greenfield trap	recovery	0	3
definition civil war	governance	3	1
silent drift	governance	2	2
conformity theatre	governance	2	2
ontology theatre	representation	0	2
AI amplification	representation	1	2
lock-in anxiety	representation	2	3
family sum	recovery, governance, representation	6, 7, 3	4, 5, 7
total		16	16

The table is the same score as the block above, printed so the route is visible: the high cells sit in different families, and the totals agree. A single index would average the route away, which is why the diagnostic does not print one.

Program A's high cells are in recovery and governance: invisible ontology, mappings in the ETL, definition civil war. Its first repair is a conflict row for Account, Workspace and Site, with two holders and two dates. Program B's high cells are in recovery and representation: greenfield trap, ontology theatre, AI amplification, lock-in anxiety. Its first repair is a provenance span and a loss report on export.

The totals are identical and the repairs are not. A sum of 16 would have sent both programs to the same place, and the place would have been wrong for at least one of them.

Program A, row by row

Program A's recovery rows are 3, 3, 0. The invisible ontology is structural: three schemas and a mapping table that nobody recovered. Mappings in the ETL is structural: the join on `cust_nm` is the only record. The greenfield trap is absent; the estate has no greenfield program.

Its governance rows are 3, 2, 2. Definition civil war is structural: five teams, five customer columns. Silent drift is visible: a schema change ticket with no type update. Conformity theatre is visible: an alignment spreadsheet with no replay.

Its representation rows are 0, 1, 2. Ontology theatre is absent. AI amplification is suspected: a retrieval pipeline over unprovenanced types. Lock-in anxiety is visible: an export ticket that has not been run.

Program B, row by row

Program B's recovery rows are 1, 0, 3. Invisible ontology is suspected: the editor holds the model. Mappings in the ETL is absent: no ETL owns the vocabulary. The greenfield trap is structural: twelve months in an editor, no register.

Its governance rows are 1, 2, 2. Definition civil war is suspected: one team, one model. Silent drift is visible: the editor and operations disagree. Conformity theatre is visible: a conformance deck with no artifact.

Its representation rows are 2, 2, 3. Ontology theatre is visible: generated classes with confidence and no origin. AI amplification is visible: the classes feed a retrieval pipeline. Lock-in anxiety is structural: the tool is the only place the model lives.

The workshop

The workshop needs one hour, the nine pains and permission to say the program is failing. Score each pain, name the artifact for every score above one, identify the two pains that cost the most and are cheapest to fix, write the one-paragraph case for stopping the current approach, and re-score in three months.

The permission is the awkward prerequisite. A room that cannot say failing will score everything low and call the result alignment. A room that scores everything high has found everything, which is the same as finding nothing.

A diagnostic that scores everything high has found everything, which is the same as finding nothing, and the room should treat the result as a failure of the instrument rather than a success of the program.

The output is a diagnosis in the team's own language, evidence for every score and a defensible next step, even if that step is stop.

The re-score

The re-score is the progress metric. The same program, the same rows, the same raters and the same artifacts, scored again after three months, produces a delta that is not a slide.

A program that improves from 16 to 10 on the same rows has evidence. A program that improves only its average has a story.

The delta is also a test of the instrument. If the rows move without the artifacts changing, the scores were a mood.

The stop case

The stop case is one paragraph, written by the room, naming the approach that is failing and the artifact that would show progress. It is not a guarantee that stopping succeeds; it is a record that the room can read the evidence.

A stop case without an artifact is a budget request in the other direction.

The two costs

Every pain has a cost and a repair cost. The diagnostic asks for both because the highest-cost pain is not always the cheapest to fix, and the first repair should be the one the program can finish.

Program A's civil war costs a merge nobody can audit; the conflict row is cheap to produce. Program B's lock-in costs an exit; the provenance span for the greenfield classes is cheaper and more urgent.

A program that chooses the highest-cost pain without checking the repair cost will start a project it cannot finish, and the diagnostic asks for both numbers for that reason.

The artifact rule

A score above one owes an artifact, and the artifact is named in the row. The rule is what turns the diagnostic from a survey into a test: a score with no artifact is theatre, whatever the number.

Program A has six scores above one and six artifacts to name. After the naming, the guard returns zero.

The artifact rule is cheap to state and hard to satisfy, because a score of three with no schema, ticket or missed date behind it is a claim about the room rather than about the estate.

The two costs are not a tie-breaker; they are the reason the diagnostic refuses to rank the pains by severity alone, and the reason the workshop names a first repair rather than a first complaint.

The average refused

The maturity index is the anti-walkthrough. Both profiles average 1.78 and are reported as fifty-nine percent mature. The totals are identical and the repairs are not, and the average cannot say which row moved.

The average is the one number the instrument must refuse. It lets a program look better without getting better, because a high score in a cheap family can offset a low score in the family that is failing.

The diagnostic prints the nine rows, the family sums and the first artifact. It does not print a single index.

The evidence guard

A score above one owes an artifact. The rule is the difference between a diagnostic and a survey.

```
SELECT pain, score FROM diagnostic
WHERE score > 1 AND artifact IS NULL;
```

Program A has six scores above one before step three: 3, 3, 3, 2, 2, 2. The guard returns six rows. After the artifacts are named, it returns zero.

The guard tests the diagnostic's honesty, not the estate. A score of two with no schema, ticket, missed date or report behind it is theatre by construction.

The threshold is not validated. It is a decision rule: above suspicion, a score owes a document. A score of zero needs no evidence, and a score of one is a suspicion that may be recorded without a document.

What the instrument cannot do

The instrument is designed, not measured. The scoring has not been validated across organizations, the reliability study is specified and unrun, and the scores are not comparable between companies. The printed standing is `designed`, and the diagnostic prints the missing reliability rather than the score.

The instrument also cannot detect lock-in from inside the estate. The only test is an export into a second tool, counted for loss. That test is specified here and not run.

The concession should be stated in its strongest form. Nine pains in the customer's words is a sales list, not an instrument: unvalidated, scoreable to fit any conclusion, and blind to a program failing for reasons no artifact can show. The reliability half is granted. The usefulness half is answered by the mapping: a pain routes to a broken transition,

the transition routes to a family, and the family routes to a first artifact. A reader can run that mapping on Monday.

The diagnostic does not fix management. Management decides whether anyone acts; recovery decides whether there is anything to act on. Three of the nine—invisible ontology, mappings in the ETL, ontology theatre—are epistemic: the knowledge is in the artifacts and the team cannot see it. The other six are failures of decision or record, and the diagnostic names them without claiming to cure them.

Start with what is already there

The first artifact is not a charter. It is a register, a conflict row or a provenance span. The program starts by recovering what the artifacts already commit to, deciding which commitment is in force, and exporting the decision with its provenance.

A greenfield program is not wasted. The authored model is a set of candidates that still has to be checked against the artifacts, and that check is recovery. The trap is authoring before recovering, not authoring itself.

A program with no ontology at all has an invisible one. The schemas, keys, enums and glossaries are the evidence, and the first artifact is the register that says what they already commit to.

The first artifact is not a charter, a purchase or a diagram; it is the smallest record that makes a commitment inspectable, and the diagnostic routes to the one the program can finish.

Nothing on the nine-pains page is cured by a better diagram. Every one is cured by recovering a commitment the artifacts already contain—which means what the program is missing is not a picture of the world but a set of commitments.

32 · What an Ontology Is For

An ontology is usually presented as a class hierarchy. The presentation is not wrong, and it is not the thing. A class tree that says `Customer` is a `Party` answers no question a business asks; the questions a business asks are about what may be said, with which roles, about which sense, and who may change the answer.

The eight layers

An ontology project usually collapses eight layers into one artifact; the layers have different owners and different tests, and the collapse is why the project stalls without anyone being able to say which part is late.

Layers	Owner	Test
Lexicon and controlled vocabulary	dictionary team	does lookup return a sense, or an unresolved state
Taxonomy and domain ontology	domain steward	does the parent survive a counterexample
Operational and substrate ontology	platform	does the compiler emit the archetype
Profile and mapping ontology	deployment	does the projection pass its fixtures

The first two are about meaning; the second two are about execution. A single “classes and relations” workstream asks one team to own all four jobs, and the four jobs fail in four different ways. A layer with no test is a slide, and a layer with a test can be late; the program can say which one is late, which is what a collapsed workstream can never do.

The schema and the ontology

A schema governs the admissibility of storage. An ontology governs the admissibility of speech and inference.

A schema can say that a column is not null, that a foreign key resolves, that a code is in a check list. It cannot say that the verb in the row implies one thing and forbids another. It cannot say that a person named in a column is the asserter of the record rather than a participant in the event the record describes.

Those are commitments about meaning, and a database constraint cannot hold them. The missing layer is not storage. It is which verbs are legal and what each one licenses.

The tree and the question

Take the tree. The relation is subsumption: Customer is a narrower term than Party, written $Customer \sqsubseteq Party$.

Six questions follow, and the tree answers none of them.

1. May this row be called `CUSTOMER`? 2. Which roles may a `CUSTOMER` claim take? 3. Which sense of *customer* does the claim bind? 4. What may be inferred from a `CUSTOMER` claim? 5. What is forbidden to follow from it? 6. Who may change the answer?

The tree names a category. It does not license a predicate, bind a sense, forbid an inference or name an authority. A row can be classified as a `Customer` and still be unusable, because the classification does not say what the row may be used to assert.

The class tree is a picture of the ontology. It is not the ontology. A project that delivers the picture and calls it the artifact has delivered a diagram.

Why projects stall

Three failures account for most of the stall.

Scope confusion is the first. The eight layers are collapsed into one, and one team is asked to own the lexicon, the taxonomy, the operational archetypes and the mapping profile at once. The artifact is late, and no one can name which layer is late.

No ratifier is the second. An ontology has no author of record in most enterprises. The data architect can describe the schema, the steward can defend a definition, and the platform team can change a compiler. Nobody is empowered to say that this predicate is now permitted and that one is retired. The artifact stays a draft, and a draft cited in a build is worse than no draft because it looks settled.

The wrong buyer is the third. The customer with a naming problem is sold inference. The customer with an inference problem is rare, usually regulated, and the cost of being wrong is legal. The two requests sound identical in a requirements document.

The failure usually starts earlier than the ontology. Semantic integration failures begin at extraction and reconciliation, before any model is authored. The work is not “what should the model be” but “what does this organization already commit to, and where does it disagree.”

The commitments

An ontology is a set of commitments about what may be said. The commitments have five parts.

Commitment	Question it answers	What its absence costs
Predicate license	which predicates are permitted	free text becomes a predicate by accident
Role signature	which participants a predicate takes, and which it refuses	the reporter is read as the actor
Sense binding	which sense of a word the predicate binds	retrieval returns the wrong subject
Inference license	what may be inferred, and what is forbidden	a log becomes a verdict
Change authority	who may revise the commitment, and how	the next analyst edits it in place, unrecorded

A commitment set is not a catalog. A catalog answers what exists; a commitment answers what may be asserted, what may be inferred, and who may revise the answer. Catalogs grow without limit; commitments can be short.

The commitments are local and plural. Two departments can hold two different sets, and neither is wrong. What they need is a translation with a proof trace or a declared gap, not a global model that forces one set onto the other.

The first commitment

A program does not start with a taxonomy. It starts with the predicate its questions depend on.

Take the question “who approved this order?” The predicate is `APPROVE`. The license needs the role signature, the sense binding, the inference rules and the authority. The taxonomy can wait; the question cannot.

That order is the practical difference between a license and a project. The project starts with the classes and discovers the questions later. The license starts with a question and adds the classes only where the question needs them.

The table is small

A deployment’s license may be six rows. Each row names a predicate, a role signature, a sense binding, an inference rule and an authority. The table is small because it covers the predicates the questions use, not every class the business can imagine.

A team that licenses every predicate has built a catalog with a governance costume. A team that licenses the predicates its questions depend on has built an ontology, and the table fits on a page.

The size is a consequence of the questions, not a target. A table that grows because an analyst is nervous is a catalog; a table that grows because a question arrived is a license.

The six questions in practice

Take a row that a team wants to call `CUSTOMER`.

1. The predicate license says whether `CUSTOMER` is permitted at all. 2. The role signature says whether the row may bind `AGENT`, `THEME` or neither. 3. The sense binding says whether the row means the party that ordered or the party that passed KYC. 4. The inference license says whether `CUSTOMER` implies `PARTY` or `BUYER`. 5. The forbidden list says whether `CUSTOMER` implies `PAID`. 6. The change authority says who may revise the answer.

The tree answers none of the six. The commitment table answers all six, and each answer is a row with an owner and a version.

The SEIZED example

Here is a row from a maintenance table.

```
maintenance_event
 event_id asset_tag site_code event_code reported_by reported_at
 ME-4471 PB-100-7 BAY-14 SEIZED avery 2026-03-02T09:14Z
```

Compiled, the row is a synapse.

```
synapse
 predicate SEIZED
 roles PATIENT = PB-100-7
 LOCATION = BAY-14
 TIME = 2026-03-02T09:14Z
 asserter avery
 status candidate
 span maintenance_event/ME-4471
```

The interesting column is `reported_by`. A naive mapping makes Avery the agent of the seizing. The row does not say that. Avery reported the record; whoever or whatever seized the pump is unnamed. That distinction is not in the DDL and cannot be put there. It is a commitment about roles, and it changes what a downstream question can honestly be answered with.

The license for this deployment is a table, not a hierarchy.

Commitment	This deployment
Permitted predicate	SEIZED is licensed; BLEW_UP is not
Role signature	PATIENT, LOCATION, TIME; no AGENT
Sense binding	the machine-stop sense, not the legal-seizure sense
Permitted inference	SEIZED is narrower than STOPPED
Forbidden inference	SEIZED does not imply FAILED or DEFECT
Change authority	reliability engineer plus steward, by typed patch

Six rows. The schema could enforce the first column's values; it could not enforce the rest.

The change authority

A commitment that nobody owns is not a commitment. It is a draft.

The change authority names who may revise the license and by what procedure. The procedure is a typed patch: a proposed change, a reason, a reviewer and a ledger entry. The patch is accepted or rejected, and the rejection is recorded with its reason.

The alternative is the next analyst editing the table in place, unrecorded. The table still looks current, and the meaning has moved without a decision.

The authority is also a limit. A council that must approve every predicate change will not approve any; a single owner with no review will drift. The working arrangement is a named owner with a typed patch and a review that is proportional to the change.

The license and the catalog

A catalog answers what exists. A license answers what may be asserted, what may be inferred from it, and who may revise the answer. Catalogs grow without limit; licenses can be short.

The distinction matters because ontology enters a system at six points: vocabulary, mapping manifest, semantic profile, constraint validation, controlled inference and projection. Each point has its own requiredness and its own price. A deployment that treats the six as one decision overpays for five of them.

The license is the smallest artifact that covers the points a deployment actually uses. It does not need a taxonomy, an upper-ontology alignment or a reasoner profile unless a named mapping or inference depends on one.

Local and plural

Two departments can license `CUSTOMER` differently. One means the party that ordered; another means the party that passed KYC. Both licenses can be valid inside their own boundary.

The choice is not between two local sets and one global set. It is between a translation with a proof trace and a global model that nobody adopts. The translation records the correspondence and the gap; the global model records an ambition.

An alignment is a claim, directional and versioned. It can say that local A's `CUSTOMER` is narrower than local B's, with a trace and a residual. It cannot say that the two are the same because the labels match.

The class tree, again, is a projection. A team can derive a tree from the commitments where the commitments support it, and keep the tree as a view. The commitments remain the artifact.

What it forbids

An ontology's quality is measured by the derivations it forbids, not by the classes it contains. A large ontology with no forbidden inference is a vocabulary, not a commitment set.

The forbidden list is the test. `SEIZED` does not imply `FAILED`. `CUSTOMER` does not imply `PAID`. A reported event does not imply an asserted one. Each prohibition is a rule the engine can check and a question the ontology can refuse.

A prohibition that nobody can state is not a commitment. It is a hope that the downstream analyst will be careful.

The prohibition test

Take three forbidden derivations. `SEIZED` does not imply `FAILED`. `CUSTOMER` does not imply `PAID`. A reported claim does not imply an asserted one.

Each prohibition returns a refusal, and the refusal is the ontology doing its job. A license with no forbidden derivation is a vocabulary; it can classify and cannot refuse.

The test is falsifiable. If the engine derives `FAILED` from `SEIZED`, the license is wrong or the engine ignores it. Both are findings.

The reasoner bought for a naming problem

A reasoner reads axioms and instances. It does not read column names. If two systems call the same field different things, no amount of classification or subsumption will reconcile them, and the resulting system will be formally consistent and operationally wrong.

Soundness and completeness are relative to the axioms and the profile. Determinism is not truth. A reasoner that derives a conclusion from a wrong axiom is doing exactly what it was asked to do.

The test is narrow. If the rule does not require classification or entailment over a changing axiom set, and if a wrong answer does not carry legal or safety consequences, the reasoner does not earn its place. Many enterprise ontology requests fail that test. A vendor that will not say so is selling a formalism to a governance problem.

Under the substrate

The commitments are rows the compiler reads. A predicate license is a row with a predicate, a role signature, a sense binding, an inference rule and an authority. The compiler checks the claim against the license before it writes the synapse.

That placement matters. An ontology that lives only in a diagramming tool is above the substrate and disconnected from it. An ontology that lives in rows is under the substrate, and the class tree is one projection of those rows.

The direction of authorship reverses. The ontology is downstream of the claims, not upstream of them. A commitment about predicates is only meaningful against predicates that claims actually use; a universal model written first has to guess which verbs the corpus will need.

The tree as a view

The class tree is still useful. It is a view: a compact way to navigate the commitments and to show a domain expert what the license covers.

A tree derived from the commitments can be regenerated when the commitments change. A tree authored before the commitments has no source, and every change is a new debate.

Keeping the tree as a view is the practical compromise. The commitments are the artifact; the tree is the picture that helps a person read them.

The cost of the first license

The first license costs a workshop, a predicate inventory and a reviewer. The inventory is the expensive part: someone has to read the schemas and the ETL and decide which predicates the questions actually use.

The cost is paid once. The maintenance cost is per change, and the change is a typed patch with a reason, a reviewer and a ledger entry.

A team that skips the inventory and licenses what it can imagine has built a catalog, and the catalog will be larger and less useful than the six-row table.

The cost of the table

A six-row table is cheap to keep and expensive to change, because every row has an owner and a reviewer. The expense is the decision, not the row.

A team that cannot afford the decision has not avoided the ontology; it has deferred it to the next integration, where the cost is paid with interest.

The objection

The strongest objection is that local commitments cannot be compared. Two deployments license their own predicates, roles and senses; nothing forces them to agree; every cross-domain question becomes a negotiation, and every mapping is an assertion rather than a proof.

Most of that is correct. Local commitments produce description, governance and projection; they do not produce cross-domain entailment. If a regulated question must be settled by entailment across two independently governed vocabularies, the ceiling is real and an upper ontology is not optional.

What the objection overstates is the alternative's reach. A shared upper model does not reconcile local labels by itself. It makes the reconciliation checkable, which is smaller and more valuable than the promise. Adopting it converts a hundred local authorship decisions into one adoption decision plus a hundred mapping decisions. That is a real saving in coordination, not the elimination of work.

What keeps the concession honest is the loss report. Every projection drops something, and an export with an empty loss report fails the build. Flattening a local definition into a global one at export is a silent promotion, and it is a defect.

The concession

Commitments are expensive to maintain. Every predicate needs a license, every license needs an owner, and every change needs a typed patch with a reason. A small team may keep the table in a spreadsheet longer than a vendor would advise, and the spreadsheet is better than no table as long as the owner and the date are real.

The expense is not the table. It is the decision about which commitments are worth making. A team that licenses every predicate it can imagine has built a catalog with a governance costume. A team that licenses the predicates its questions depend on has built an ontology.

A commitment table that is never revised is a fossil. A table that is revised without a record is a rumor. The working table is small, owned and revised by typed patch, and its quality is measured by the questions it can refuse.

The page

The license fits on a page: predicate, roles, sense, inference, forbidden, authority. If it does not fit, the license is a catalog.

A license that covers a hundred predicates and cannot name the authority for any of them is a catalog with a governance costume, and the costume is the expensive part. The six-row table is the artifact because it can be read in a meeting, challenged line by line and revised by a named owner. The page is the artifact.

A commitment that cannot be read in a meeting is not a commitment; it is a schema fragment that no one can defend, and the meeting is where the authority is supposed to be named. A license that cannot be challenged line by line is a diagram with a version number, and the version number is the only part that changes when the diagram is redrawn.

The page is small because the questions are specific, not because the ontology is modest, and a specific question is what makes the table defensible in a review. The table is the artifact because it can be read, challenged and revised; the tree is a picture because it can only be admired or redrawn.

A team that licenses what it can imagine has built a catalog, and a catalog is a promise to no one in particular.

A catalog can be large and still promise nothing, because a promise requires an authority and a consequence, and the catalog has neither of those things. The page is not a simplification of the ontology; it is the ontology, and the diagram is the simplification that loses the authority and the forbidden list. A commitment that no one can name is not a commitment; it is a habit, and the habit is what the next analyst inherits when the owner leaves.

A catalog with a hundred predicates and no named authority is a list of names no one can defend. The page is not a summary of the ontology but the part a reviewer can challenge while the tree is the part a reviewer can only admire.

A commitment table that cannot be read in a meeting is a schema fragment, and a schema fragment cannot answer the question that the ontology was supposed to answer. The tree can be redrawn in an afternoon, and the commitment table cannot, which is why the table is the artifact and the tree is a view. A license that names an authority and a forbidden inference is a promise, and a catalog that names neither is a list of words with a version number.

The first license is a workshop, a predicate inventory and a reviewer, and the inventory is the part that cannot be skipped without turning the license into a catalog.

A commitment that is revised without a typed patch is a rumor, a tree authored before the commitments has no source, and the license is small because the questions are specific and the specificity is what makes the authority answerable when a reviewer asks who decided.

A table that cannot be read in a meeting is a schema fragment, and a schema fragment cannot answer the question that the ontology was supposed to answer because no one can see the answer in the room.

A license that cannot name the question it answers is a list of predicates with a version number, and the version number is the only part a reviewer can check.

A license that cannot refuse a derivation is a vocabulary, and a vocabulary can classify anything, which is why the forbidden list is the quality test and the only test a reviewer can run.

An ontology is a set of promises about what may be said; the next chapter recovers the promises a company has already made.

33 · The Recovery Method

An ontology project begins by asking what the model should contain. The question has a prior answer. The organization has already made commitments, in schemas, documents, communications and code. Those commitments are the raw material. The work is recovery before it is authoring.

The ontology was never missing

An operating company has already decided what an active customer is; the decision is a check constraint in a schema, a paragraph in a revenue policy and a message in a 2023 thread. The three may disagree. The disagreement is the artifact, and the first job is to bring the three into one register with their sources.

Recovery starts from four artifact classes.

Class	What it reveals	What it cannot settle alone
Schema	structure, keys, constraints, enumerations	intent; why the constraint exists
Documents	definitions, scope, policy, exceptions	currency; whether the definition is still in force
Communications	dated decisions, disputes, local rulings	completeness; the thread may omit the rule
Code	mappings, transformations, joins, edge cases	authority; code can encode a workaround

Each class answers a different question, and no class settles the commitment by itself; structure without intent is a constraint with no reason; a definition without currency is a policy that may have been superseded. A dated decision without completeness is a ruling that may not cover the case. A mapping without authority is a transformation that may be a workaround.

The method is designed, not measured. The numbers in the worked case are specifications, not results, and the register's standing is `designed` until a run is published.

What a register is

A register is a table of candidates; each row names the term, the proposed definition, the source class, the span, the holder, the date, the standing and the conflict it belongs to. The row is queryable, and every query can open the span.

A register is not a model; it does not classify the terms into a hierarchy or claim completeness; it records what was recovered and where it came from, and it keeps the disagreements that a model would average.

The register is the artifact; a projection to OWL, SKOS or a graph comes later and declares what it drops; a register that exists only as an export has already lost the standing that made it worth recovering.

The register is the artifact.

The corpus record

The register starts with what was read: file path, sha256, size, capture time and source class. The record is the evidence that the corpus was seen, and a re-run can compare the two records.

A missing file is a gap, not an absence. A file that changed since the last run is a new version, and the register records both versions with their dates.

The record also sets the boundary. A corpus that was not registered was not recovered, and a claim about it is a claim about a folder nobody opened.

A register that cannot be re-run is a snapshot; a register that can be re-run is an instrument, because the second run can show what changed in the corpus and what changed in the reader.

Extraction from documents

A document yields definitions, scope, exceptions and owners. The span is the page or paragraph, and the extractor records it before it records the definition.

A definition without a date is a candidate; a policy with an owner and a date is a stronger candidate, and the strength is recorded as a standing, not a score.

A document that restates a schema comment adds a second class but not a second origin; the independence gap is recorded, not hidden, and the reviewer is told that the two sources may be one.

The document class is where intent lives. It is also where a definition can outlive its policy, which is why the date is not optional.

Extraction from communications

A message yields a dated decision and an actor. The span is the message id, and the thread is the context that makes the decision readable.

Only disagreement and promotion are routed to a human. The volume of email is not the volume of work; the conflict row is the unit of review.

A message without a thread is a quote without a context, and the register marks it as such. A decision that names no actor is a rumor, and the row says so.

The communication class is the most expensive to read and the most likely to contain a decision that never reached a document.

Extraction from code

Code yields the mapping, the transformation, the exception and the join. The span is the file and line, and the record names the job that runs it.

Code is strong evidence of what happens and weak evidence of what is intended. A transformation can be a rule or a workaround, and the register cannot tell the difference without a document or a dated decision.

The mapping table is often the only place the real join is recorded. Recovering it is the difference between an estate that re-derives the join every year and one that can inspect it.

Code also changes fastest. A mapping recovered today may be stale next quarter, and the register records the version it read.

The source-class matrix in practice

A schema gives the column, the check constraint and the key. It is deterministic and cheap; it is also silent about intent.

A document gives the definition and the owner. It is readable and dated; it is also silent about whether the definition is current.

A communication gives a decision and a date. It is specific and attributable; it is also silent about completeness.

Code gives the mapping and the exception. It is executable; it is also silent about authority.

Independence is declared by class, not measured. A document that restates a schema comment is two classes and one origin, and the register cannot tell the difference without reading both.

What each class settles

A schema is deterministic and cheap to read. It gives the table, the column, the check constraint and the foreign key, and it does so without interpretation. It cannot say why the constraint exists or which business rule it encodes.

A document gives the definition and the scope. It can say that an active customer is one who purchased within ninety days, and it can name the policy owner. It cannot say whether the definition is current or whether an exception has been granted.

A communication gives a dated decision. A message can say that a customer becomes inactive after one hundred eighty days, and it can name the person who decided. It cannot say whether the decision was adopted or whether it covers every region.

Code gives the mapping. A job can show that `cust_nm` joins to `account_id` and that the status code is transformed on load. It cannot say whether the transformation is the rule or a patch.

The method reads the classes in that order: structure first, because it is cheap and re-runnable; documents next, because they carry intent; communications after, because they carry dated decisions; code last, because it carries the operational mapping. A different order is possible, and the first pass is cheaper when the deterministic classes come first.

The five steps

The method has five steps, and each emits an artifact.

1. **Register.** Hash each file, record the source folder and the capture time, and print what was read. The register is the evidence that the corpus was seen, not a model of it. 2. **Extract.** Propose candidates from each class, and attach a span to every candidate. The span is the file, the page or paragraph, the message id, the table and column, or the code line. 3. **Reconcile.** Group candidates by term. Keep every definition, holder and date. Where two definitions disagree, emit a conflict row rather than a merged definition. 4. **Stand.** Give each candidate a standing: candidate, disputed, corroborated, adopted or retired. The standing is a record, not a score. 5. **Adopt.** A named ratifier rules on the conflict. The ruling records the actor, the rationale, the scope and the date, and it is reversible.

The steps are not a waterfall. A register can be re-run, an extractor can be improved, and a ruling can be revisited. The artifacts are the point: each step leaves a row that a later reader can inspect.

The five steps in detail

Register. Hash the file, record the folder and the capture time, and print what was read. The step refuses to interpret. Its artifact is the corpus record.

Extract. Propose candidates from each class, with a span for every candidate. The step refuses a candidate with no span. Its artifact is the candidate row.

Reconcile. Group by term, keep every definition and holder, and emit a conflict row where two definitions disagree. The step refuses to merge. Its artifact is the conflict row.

Stand. Assign candidate, disputed, corroborated, adopted or retired. The step refuses a score. Its artifact is the standing event.

Adopt. A named ratifier rules with actor, rationale, scope and date. The step refuses an anonymous ruling. Its artifact is the review event.

A step that skips its refusal has skipped its artifact. A merge without a conflict row has destroyed the disagreement; a standing without an event has asserted an authority that no one recorded.

Why recovery before authoring

An authored model guesses which verbs the corpus uses. Recovery starts from the verbs it already uses, so the model is a summary rather than a guess.

The authored model also tends to describe a company that has moved. Recovery is dated and span-bound, so the description is tied to evidence that a reader can open.

The order matters because the commitments already exist. A greenfield model that ignores them will reproduce some and contradict others, and the contradictions will be discovered after the budget is spent.

active customer

Here is the worked case.

```
corpus:
 crm DDL
 revops_policy.docx
 2023 email thread about inactive accounts

register:
 sha256, source folder, capture time per file

extract:
 table.column crm.account.status CHECK status IN ('A', 'S', 'D')
 page/para revops_policy.docx §2.2 active: purchased within 90 days
 message-id CALx9kQ2@acme.example 2023-04-11: inactive after 180 days

reconcile:
 one term, three definitions, one conflict row, zero merges

stand:
 each definition remains candidate; disputed is recorded;
 corroboration is unavailable because the classes disagree
```

```
adopt:
 a named ratifier rules scope-partition
 review event: actor, rationale, scope, date
```

The output is one term with three spans, three holders, three dates, one open conflict, one ruling and zero merges.

The conflict row is the product. It says that the schema uses a three-value status, the policy uses a ninety-day purchase window, and a 2023 decision uses a one-hundred-eighty-day inactivity rule. The three are not averaged. The ruling may partition the scope: ninety days for the policy report, one hundred eighty days for the account lifecycle. The partition is a decision with a name and a date, and it can be reversed.

The term is not finished. It is registered, and the register says who disagrees and where the disagreement is recorded. That is a different deliverable from a class tree.

The conflict row

A conflict row is one term, two definitions, two holders, two dates and one scope question. The row does not choose; the ruling does.

The row is the artifact that no catalog generates, because the catalog stores what each source declares and does not compare them.

A conflict row with no scope question is a duplicate; a conflict row with no holders is an edit history. The row is worth its cost only when it names who disagrees and what would settle the disagreement.

The row is the unit.

The ruling

A ruling is a decision about scope, not a merge. The ninety-day policy and the one-hundred-eighty-day lifecycle can both stand if the ruling says which question each answers.

The ruling records the actor, the rationale, the scope and the date, and it is reversible. A later review can reopen the conflict without restoring a deleted row.

A ruling that merges the definitions destroys the evidence for the losing one. A ruling that partitions the scope keeps both and names the boundary.

A ruling without a scope is a preference. A ruling without a date cannot be revisited. A ruling without an actor is a rumor that the register has learned to format.

A candidate without a span is a gap

The rule is simple: every candidate cites at least one span, and a row without a span fails the build.

```
query:
 SELECT candidate_id FROM candidate
 WHERE NOT EXISTS (
 SELECT 1 FROM candidate_evidence
 WHERE candidate_evidence.candidate_id = candidate.candidate_id
 );

before: one seeded candidate\index{candidate} with no span -> 1 row; build exits
 ↪  non-zero
after: the span is attached -> 0 rows; build exits 0
```

The guard is specified, not run on a customer corpus. The query and the expected rows are written; the run is not.

The rule matters because a candidate without a span cannot be contradicted. A reviewer cannot open the source and disagree. A class with no span is a proposal that no evidence can reach, and a proposal that no evidence can reach is not yet a claim.

The single-class concession

The method is worth nothing on a single-class corpus.

If the corpus is DDL and a data dictionary, the output mechanically restates the catalog: tables to classes, keys to relations, checks to constraints. The register is a schema dump with a review queue, and calling it ontology is ceremony.

The value appears when independent classes disagree. The schema says three status values; the policy says ninety days; the message says one hundred eighty days. The conflict row is what no catalog generates, because the catalog stores what each source declares and does not compare the declarations.

The concession is a claim, not a footnote. A recovery that produces no conflict row has recovered a catalog. A recovery that produces a conflict row has recovered a commitment that the organization had not resolved.

The method's value therefore rises with the number of independent source classes and with the number of disagreements between them. It is zero at one class, and it is not guaranteed at three: three classes that restate one origin are one class in three costumes.

The anti-walkthrough

The alternative is to prompt a model for the ontology.

```
input: the same corpus, plus "extract the domain ontology"
output: a hierarchy containing ActiveCustomer and inactive_after_days = 180
 no span, no holder, no date, no conflict row
```

The 90-day policy definition is averaged into "90—180 days" or dropped. The output is a plausible class tree that no one can correct, because no row can be contradicted.

The failure is not that the model guessed badly. It is that the output cannot represent the disagreement the corpus contains. A class with no span cannot be wrong, and a claim that cannot be wrong is not yet a claim.

What recovery cannot do

The method cannot establish that two artifact classes are independent. A document that restates a schema comment is two classes and one origin, and the method has no test for that beyond the classification itself.

The method also cannot measure the extractor's accuracy on prose and email. Only the deterministic structural half can be run today, and the measurement covers only that half. The register's standing is `designed`, and the unmeasured half is named rather than smoothed.

The method cannot confer authority. The register records a pointer to a review event, an actor and a span, and the span can be opened by anyone who doubts the row. Whether the ratifier had authority is a governance question, not a recovery question.

Finally, the method cannot promise completeness. A register is a record of what was recovered, and the gaps are part of the record. The gap report is what keeps the register honest.

The two-week run

A first pass over a small estate takes two weeks: one week for schema and documents, one for communications and code. The output is a register, a conflict list and a gap report.

The two weeks are a specification, not a measurement. The method's standing is designed, and a run that misses the two-week bound falsifies the estimate at that scope.

What the reviewer sees

The reviewer sees a conflict row, not a class tree. The row names the term, the two definitions, the holders, the dates and the span. The reviewer can open the span and disagree.

A reviewer who sees only a class tree cannot tell which definition is in force or who decided. The row is the unit of review because it is the unit of disagreement.

What the register is not

A register is not a model. It does not classify terms into a hierarchy or claim completeness. It records what was recovered and where it came from.

A register is not a catalog. A catalog stores what each source declares; a register stores what two sources say differently, with holders and dates.

A register is not a finished ontology. It is the evidence from which a finished ontology may be ruled, and the gaps are part of the evidence.

The gaps are the boundary of the register, and a register that hides them has confused a recovery with a model.

The gaps are the boundary.

The gap report

The register reports what it did not recover. A term with one class is a candidate; a term with no class is a gap; a term with two classes that disagree is a conflict.

The gaps are named: independence of source classes is declared, extractor accuracy on prose is unmeasured, and completeness is refused. A register that claims completeness has stopped being a register.

The gap report is also the sales honesty. A buyer who sees the gaps knows what the register can answer and what it cannot, and the register does not pretend to a model it has not earned.

A register that cannot be re-run is a snapshot, and a snapshot cannot tell whether the corpus changed or the reader changed. A conflict row that no one can open is a rumor with a row identifier, and the row identifier is the only part that is auditable. The two-week run is a specification, and a specification that has never been run is a hypothesis about the cost rather than a price.

The limits of the method

A register records what was recovered, and it cannot record what no artifact contains, because a commitment that was never written down is not recoverable by reading. A method that promises to recover an unwritten commitment has confused an interview with a source, and the confusion is expensive because the interview cannot be re-run.

The register can hold the gap, and the gap is the honest output, while the alternative is a model that invents the commitment and calls the invention a recovery. The independence gap is the same shape: two classes that restate one origin look like corroboration, and the register cannot tell the difference without reading both.

The extractor gap is the third: prose and email are unmeasured, and the deterministic half is the only half that can be run today.

Citations make a register defensible; they do not make it useful, so the next question is the only one that can settle it—which questions can this vocabulary answer that the old glossary could not?

34 · The Standing Ladder

A glossary shows the top state and forgets the climb. A term that appears in it looks official, and the record of how it became official is usually missing. The missing record is what makes a glossary rot: the word stays, the warrant disappears, and the next team cannot tell whether the definition was adopted or merely typed.

Five states

A concept climbs five states.

State	What it means	Artifact
Observed	the term appears in a source	a mention, a column, a span
Registered	the term has an address	a canonical ID and a namespace
Defined	the term has a distinguishing gloss	a microgloss and a definition
Adopted	a named owner has accepted it	an adoption receipt
Governed	changes go through a process	an amendment queue and a rollback

The transitions have names: register, define, adopt, govern. Each transition is an event, and each event carries evidence, an owner, a date and a rollback.

Registration is not adoption. A term can sit at registered for years; that is a legitimate resting state, not a failure; a team can use a registered term with a declared provisional status while the definition is still being argued.

The transition table and the log

A ladder is a state machine when each move has a from, a to and a requirement.

From	To	Requires
observed	registered	an address and a namespace
registered	defined	a microgloss and a definition

Continued on next page

(Continued)

From	To	Requires
defined	adopted	evidence, an owner and a date
adopted	governed	an amendment queue and a rollback
any	demoted	a reason and a rollback

The table is not a workflow. A workflow tracks a ticket; the ladder tracks warrant. A ticket can be closed without the term being adopted, and a term can be adopted without a ticket.

No transition overwrites the prior state. The new state is a new claim with a lifecycle link; undo is a compensating event, so the trail stays append-only. The result is a queryable history of belief: what the organization thought a term meant, when it started, on whose authority, and what it thought before.

A term can turn out to be several terms. A service-state example makes the split visible.

Date	Event	Actor	Basis	Scope
2024-02-03	observed	extractor run 41	billing status enum	---
2024-02-11	registered	system	schema, policy, ticket	---
2024-03-01	split	r. okafor	billing suspension, fraud hold, regulatory bar are distinct	---
2024-03-01	adopted	r. okafor	review event 118	billing
2025-04-14	superseded	j. reyes	ruling 92 moves fraud hold to risk	risk

The split is not a correction. It is a claim that the original candidate conflated three things, and it happens before adoption rather than after; the adopted row carries a scope, which is what allows the billing definition to remain true while the risk definition says something different.

A table that reports only the current state is a glossary with columns. The history is the artifact.

The receipt

Adoption requires a receipt with four fields.

Field	What it records
Evidence	the sources, tests or reviews that support adoption
Owner	the named person or council accountable for the decision
Date	when the decision was made and against which version
Rollback	how to return the term to its prior state

A transition missing either evidence or owner returns `REFUSE_INCOMPLETE_TRANSITION`. The guard is cheap: it reads the record before it writes the state.

The receipt is the artifact. The state is the headline. A program that reports only the adopted count has reported the headline and lost the story.

customer

Here is an example.

```
observed: column customer_id in orders
registered: canonical ID customer.retail, namespace shop
defined: microgloss "a party that has placed at least one order"
adopted: council 2026-05-04, evidence: three systems, owner: data council
governed: amendment queue; rollback: revert to registered
```

The climb is visible in one row per state; a reader can see that the term was observed before it was registered, that the definition came after the address, and that adoption named a council and a date.

The order 44021 row carries `customer_id`; the term is the same one used in the dictionaries chapter. The ladder does not change the row. It changes what the organization is allowed to say about the row.

Demotion

A concept can move down the ladder, and demotion is a first-class transition.

An adopted definition can stop distinguishing its sense; the gloss may now fit a sibling, or the scope may have emptied, or a later ruling may have moved the term to a different scope. The honest response is a demotion to registered with a reason.

```
adopted -> registered
reason: the gloss now fits the sibling sense
rollback: restore the prior gloss and re-run the uniqueness check
```

Demotion is not failure; a ladder without a downward edge hides the failures it was built to record; a term that cannot be demoted is a term that cannot be corrected.

The demotion receipt has the same fields as the adoption receipt: evidence, owner, date and rollback. The direction changes; the discipline does not.

The audit trail

The ladder's value is the audit trail, not the top state.

A glossary can tell a reader that `customer` is official. It cannot tell the reader who adopted it, when, on what evidence, or what the prior state was. The ladder can, because each transition left a row.

The audit trail answers four questions: who moved the term, when, on what basis, and how to undo it. Those are the questions a reviewer asks when a number is challenged, and a glossary that cannot answer them is an opinion with a logo.

The trail is also what makes the adopted count misleading. A program that counts adopted terms without counting the transitions has measured the top of the ladder and ignored the climb.

Scope and the acceptance test

A transition needs a criterion, or the ladder is a ritual. The criterion is a question the vocabulary must answer.

The questions come from the corpus: what people asked in tickets, in email, in the review queue. A question that cannot be answered from the register, with a citation, is a gap item rather than a failure.

Scope is what lets two definitions both be true. A billing definition of active customer and a risk definition can both stand if the adoption receipt names the scope. Flattening the scoped definition into a global one is a silent promotion, and the export has to preserve the scope or declare its loss.

The acceptance test is sharp in both directions. A term that no query, validation rule or projection uses may be premature architecture, and adoption should wait for the use case to earn it.

Competency questions

The test is sharp in both directions. "May we bill account A-1188 in March 2024?" is answerable only if the split happened and the billing child carries a scope. Before the split, the register answers with a term that means three things, and the honest output is unresolved.

A concept that no compiler, query, validation rule or projection uses may be premature architecture. Promotion should wait for the use case to earn it, which is why a minimal module can stop at two classes, two properties, one constraint, one hierarchy and one narrower-than mapping.

The failure the gate prevents is specific: an axiom that converts a local observation into a global entailment. A rule that every delivered status entails that the customer received the package looks like a modeling shortcut and behaves as a permanent inference permission. The lifecycle gate keeps such an axiom out until someone can show which question needs it.

Ownership is a role with a scope

Ownership attaches to a claim at a scope. A domain is a scope claim rather than a folder, and scope is the primitive of which domain is one kind: legal entity, jurisdiction, product line, channel, process, population, or a declared combination.

Membership is many-to-many, and "no domain" is a valid answer. This is what makes conflicting definitions survivable: if two definitions are both true at different scopes, scope-partition records both, and the export must preserve the scope or declare its loss.

The owner is a named person answerable for the accepted claim at that scope, on a review date, and can be asked why. The role is discovered before it is appointed: the person who reliably answers definition questions in email and resolves domain tickets is the de facto steward, surfaced as a candidate and then ratified or rejected.

Review burden should track the cost of the change rather than being uniform. A pattern library is approvable by any steward; a change to modal vocabulary requires something closer to a constitutional convention.

The objection

The strongest objection is that the ladder is bureaucracy around vocabulary. A small team with one source system does not need a review board; its glossary lives in the analysts' heads, changes are cheap, and the board would add latency to decisions nobody disputed.

The concession is real. Governance is a derivative, not a good in itself. Its value rises with the number of systems citing a term, the length of the horizon over which they cite it, and the consequences of being wrong. At one system and one quarter it is close to zero.

The cost of governance

Materiality tests keep only differences that change a business answer in front of a human. A difference that no query, report or decision can see is a difference the ladder can record without a review.

Compression is what makes scale affordable. Policies apply at the level of a domain, a predicate and a source class rather than per row, and reviewers adjudicate clusters rather than instances. A thousand leaf conflicts often collapse to a handful of rulings.

The sources do not report a controlled comparison of governed and ungoverned vocabularies. The compression figure is a stated expectation, not a measurement, so the case that governance pays rests on the structure of the cost rather than on evidence.

The first adoption

The first adoption costs a workshop, a scope decision and a reviewer. The scope is the hard part: a council without a scope is a bottleneck, and a scope without an owner is a folder.

The first receipt is the template. It names the evidence, the owner, the date and the rollback, and every later receipt is compared to it. A receipt with a blank rollback is a transition that cannot be undone.

A team can use a registered term provisionally while adoption waits. The provisional status is declared, not hidden, and the team does not have to pretend the term is official.

The ladder is not a workflow

A workflow tracks a ticket; the ladder tracks warrant. A ticket can close without adoption, and adoption can happen without a ticket.

A workflow has a queue; the ladder has a history. The queue is the next subject; the history is the artifact.

The audit query

The audit query asks who moved the term, when, on what evidence, and how to undo it. If the register cannot answer all four, the state is a label.

A query over transition events returns the history; a query over receipts returns the warrant; a query over the amendment queue returns the load.

The queue

The queue is where the ladder becomes work. Every demotion, adoption and amendment is an item, and every item has an owner, a date and a receipt.

A queue that is not measured is a queue that becomes a bottleneck without anyone noticing. A queue that is measured can be staffed, compressed or refused.

The queue is the next subject. The ladder names the states; the queue keeps them honest.

The receipt in practice

Evidence is the sources, tests or reviews that support the transition. It can be a list of spans, a test report, a council minute or a signed decision. It has to be openable by the person who doubts the state.

Owner is the person or council accountable for the decision. A role without a name is a queue, not an owner. A council is acceptable when its scope and its members are named.

Date is the decision date and the version it applies to. A receipt without a version cannot be replayed, because the reader does not know which dictionary the decision used.

Rollback is how to return the term to its prior state. A receipt without a rollback is an irreversible transition, and an irreversible transition is a decision that cannot be corrected.

One field may be deliberately blank: the rollback of a first adoption, when the prior state was registered and the return path is trivial. The blank is declared, not omitted, and the guard accepts it only with a reason.

The guard reads the four fields before it writes the state. A missing evidence or owner returns `REFUSE_INCOMPLETE_TRANSITION`; a missing date or rollback returns the same refusal with the field named. The refusal is logged, and the term stays at its prior state.

The receipt is the artifact. A state without a receipt is a label, and a label cannot be audited.

The council

A council is acceptable when its scope and members are named. A council without a scope becomes a bottleneck for every term, and a council without named members cannot be asked who decided.

The council's job is the transition, not the vocabulary. It adopts, demotes and scopes; it does not write every definition.

Review burden should track the cost of the change. A pattern library is approvable by any steward; a change to modal vocabulary requires the full council.

The queue is not a backlog

A backlog is a list of work; a queue is a list of decisions with owners and dates. The ladder needs the second.

A queue that is measured can be staffed, compressed or refused. A backlog that is measured only gets longer.

The state is not the story

The state is the headline; the receipt is the story. A program that reports states without receipts has reported a glossary with a workflow.

The receipt is also the training material. A new steward reads the receipts to learn what the council accepts, and the pattern is the policy.

The demotion receipt

A demotion receipt has the same fields as an adoption: evidence, owner, date and rollback. The reason is the evidence that the prior state no longer holds.

A term that cannot be demoted is a term that cannot be corrected. The ladder without a downward edge hides the failures it was built to record.

Demotion is not failure. It is a transition with a reason, and the reason is the useful part.

The governance year

A ladder is easiest to see over a year.

In January, the recovery pass registers two hundred terms. None is adopted. The register is the evidence that the corpus was read, and the provisional status is declared rather than hidden.

In March, the first conflict arrives. The billing definition of active customer says ninety days; the risk definition says one hundred eighty. The council does not merge them. It partitions the scope: billing answers the revenue question, risk answers the exposure question, and the conflict row keeps both.

In May, the first adoption is recorded. The receipt names three systems, the data council, the date and the rollback. The rollback is the part everyone skips, and the part that makes the adoption reversible.

In July, a gloss is demoted. The definition no longer separates the sense from its sibling, and the demotion records the reason and the rollback. The term returns to registered, and the team keeps using it with a provisional status.

In September, an auditor asks who adopted the term, when, on what evidence, and how to undo it. The register answers all four from the receipts. A glossary would have answered the first question and guessed at the other three.

In December, the council reviews its load. The queue has a hundred items, most of them small. The compression rule applies: a policy at the level of a domain and a predicate covers many rows, and the council adjudicates the clusters rather than the rows.

The year is not a success story. Two terms were adopted, one was demoted, and most of the register is still provisional. The ladder's value is that the year can be reconstructed, not that the top state was reached.

What would refute this

Take two comparable corpora over the same period. One gets a named ratifier, transition receipts and competency-question gates. The other gets a maintained glossary and no governance. Measure time to a defensible answer, drift incidents where a classification changed without a record, and downstream mappings that broke.

If the ungoverned vocabulary ties on all three over twelve months, the role is ceremony and should be dropped.

A cheaper falsifier: if the competency questions add no rejections, the gate is decorative. A gate that never blocks anything is a gate that is not being used.

A program that reports states without receipts has reported a glossary with a workflow, and the workflow is the part that cannot answer the auditor.

A ladder is only as good as the queue that keeps it; the next chapter is the queue.

35 · Governance and Drift

A status column can gain a value that no one declared. The data stays valid; every query still runs; the meaning has moved. That is drift.

The value that moved

The order status column is declared with four values.

```
declared: order.status ∈ {pending, approved, shipped, canceled}
observed: distinct(status) adds 'reopened'
diff: 1 unlicensed value, 412 rows
```

The 412 rows are not malformed. They satisfy the column type, the null constraint and the foreign keys. A dashboard that counts orders by status renders them without error. The only thing that changed is what the column means, and the change left no transition.

The declared set is the commitment. The observed set is the usage. The gap between them is the drift, and it is visible only when the two are compared.

Drift is not wrong data

A data-quality check asks whether a value is valid. Drift asks whether a value is licensed.

The `reopened` value can be perfectly valid and still be unlicensed. The database accepts it; the ontology has never heard of it; a query that filters on the declared set silently excludes 412 rows, and a report that groups by status shows a category the business has not defined.

The failure is not an exception. The failure is a successful query that returns a result the vocabulary cannot explain. A drift that breaks a query is easy; a drift that leaves every query working is the one that survives until an auditor asks what `reopened` means and no one can point to a decision.

The usage diff

The diff compares the declared commitment to the observed usage. It is a query, not a judgment.

```
left: the declared values for the column
right: the distinct values observed in the data
output: values on the right that are not on the left
```

The first run returns one row: `reopened`, with a count of 412. The row is not an error; it is a finding.

The diff is cheap and can run on a schedule; it does not decide whether `reopened` should be adopted, mapped to `canceled` or rejected; it produces the queue item, and the queue has an owner.

The guard is the same shape as the candidate guard: a value with no declaration is a row that fails the comparison. Expected: one row before the amendment, zero after.

A diff can also return false positives. A value with a trailing space, a case difference or a legacy encoding is a normalization issue, not a new meaning. The reviewer resolves it, and the resolution is recorded.

The diff is not a data-quality check

A data-quality check asks whether a value is valid. Drift asks whether a value is licensed. The two instruments catch different failures, and a team that runs only the first will report clean data while the vocabulary moves.

The `reopened` value passes every data-quality check. It is the right type, the right length and the right encoding. It fails the diff because no release declared it.

A data-quality dashboard that shows green is not evidence that the vocabulary is stable. It is evidence that the values are well formed, which is a different claim.

A check constraint that rejects unknown values would catch the drift, and most schemas do not have one; the diff is the check constraint for meaning, and it belongs beside the data-quality checks rather than instead of them.

The declared commitment

The declaration lives in more than one place; it is an enum in the schema, a sentence in the policy, a microgloss in the dictionary and a line in the data contract; drift is the gap between any of those and the observed usage.

The declaration also has a version; a value that was declared in release twelve and removed in release thirteen is not drift; it is a transition, and the transition has a receipt; drift is the value that was never declared in any release.

A declaration without an owner is a comment. The owner is what makes the declared set answerable when a value appears outside it.

A declaration that is copied into four places has four chances to drift; the method prefers one declared set with four references, so a change is one transition rather than four edits.

Why annual review fails

An annual review assumes the vocabulary changes at the speed of the review; a status value can appear in a week, and the queries that use it can ship in a month.

By the time the annual review meets, the `reopened` value has 412 rows, three dashboards and a report. The review can adopt it, map it or reject it, and every option is more expensive than it would have been in week one.

The annual review also finds the wrong thing. It looks for definitions that changed, and the definition did not change; the data did. The review reads documents, and the drift lives in values.

An annual review is not useless. It is the wrong frequency for a value-level change, and a value-level diff is the cheaper instrument.

The queue budget

A queue has an arrival rate and a service rate. If arrivals exceed service, the queue grows, and a growing queue is a vocabulary drifting with a backlog attached.

The budget is hours per week. A reviewer who has four hours can process a known number of items at a known quality; a reviewer who has no budget processes whatever is left after the meetings.

The budget is also the stop rule. A term whose review the team cannot sustain should stay registered, with a declared provisional status, rather than be adopted into a queue that will not process it.

A queue without a budget is a queue that is staffed by hope. The hope is not a plan, and the backlog is the evidence.

The queue

The queue is where drift becomes work. A diff row arrives, waits for a reviewer, gets a decision, and leaves as an adoption, a mapping or a rejection.

Stage	What happens	What it costs
Arrival	the diff or a reviewer opens an item	minutes
Wait	the item sits behind other items	queue time

Continued on next page

(Continued)

Stage	What happens	What it costs
Review	a person reads the evidence and decides	review hours
Adoption	the decision is recorded with a receipt	minutes, plus permanent review

The queue has an owner: a named role with a scope and a budget, not a committee. The committee can meet inside the role, but the role is what answers when the queue is late.

A queue nobody measures becomes a bottleneck quietly, while a queue that is measured can be staffed, compressed or refused.

The ownership role

Ownership is a role with a scope and a budget, not a committee. The committee can meet inside the role, but the role is what answers when the queue is late.

A role survives a reorganization; a person does not. The scope and the budget are what transfer, and the receipts are what the new occupant reads.

A role without a budget is a title. A role without a scope is a bottleneck for every term.

The role also owns the declaration. When a value appears outside it, the role decides whether the declaration was wrong, the data was wrong or the value is a new commitment. That decision is the transition, and it is recorded.

The role is not a reviewer of every row. It is the owner of the queue and the declaration, and it is accountable for the hours the queue consumes.

A drift log

A drift log records the value, the rows, the source, the date and the decision. The log is the history of what appeared outside the declaration and what the organization did about it.

Date	Value	Rows	Source	Decision
2026-11-02	reopened	412	order status feed	amendment
2026-11-09	partially_shipped	37	fulfillment feed	mapped to shipped
2026-11-16	unknown	8	legacy import	rejected at source

The log is not a list of failures. It is the record of the vocabulary meeting its data, and the decisions are the useful part.

The log is the evidence that the diff ran and that a person decided.

The review load

Every adopted term adds permanent review. The term must be checked when the data changes, when a source changes, when a scope changes and when a reviewer asks.

The load is not the adoption hour. It is the recurring hour, and it compounds with the number of adopted terms. A vocabulary with a hundred adopted terms has a hundred recurring obligations, whether or not anyone planned for them.

Adopted terms	Adoption hours	Recurring hours per year	Queue wait
25	30	50	days
100	120	200	weeks
400	480	800	months

The hours are designed, not measured. A reader's queue will differ, and the table is a template rather than a benchmark.

The constraint is the recurring hours, not the policy. A team that adopts more terms than it can review has chosen drift with extra steps.

The review hour

The adoption hour is the cheap part. The recurring hour is the obligation, and it compounds with every adopted term.

A vocabulary with a hundred adopted terms carries two hundred recurring hours a year in the designed table. The queue wait grows when the hours do not.

A program that counts adopted terms counts the cheap part and hides the obligation. A program that counts recurring hours has counted the constraint.

The hour is also the unit of the stop rule. A team can decide how many hours it has before it decides how many terms it adopts, and the order of those two decisions is the difference between a plan and a backlog.

Throughput, not policy

Governance is usually written as a policy: who may approve, what must be documented, when the review happens. The policy is necessary and it is not the constraint.

The constraint is throughput: how many items the queue can process per week at the quality the organization requires. A policy that ignores throughput produces a queue that grows faster than it drains, and a queue that grows is a vocabulary that drifts.

The stop rule follows. Adopt the terms whose failure has a consequence and whose review the team can sustain. Leave the rest registered, with a declared provisional status. A registered term is usable; an adopted term is an obligation.

A team that adopts fewer terms is not less ambitious. It is measuring the obligation it can keep.

The stop rule in practice

Adopt the terms whose failure has a consequence and whose review the team can sustain. Leave the rest registered with a provisional status.

The order status `reopened` has 412 rows and a report; the review is an hour and a half. It earns adoption.

A low-consequence label with ten rows and no report stays registered, and the status is visible. The term is usable; the obligation is not taken.

The stop rule is not a judgment about the term's importance. It is a judgment about the consequence of being wrong and the hours the team can keep.

The first quarter

The first quarter is the calibration. The team counts arrivals, review hours and queue wait, and compares them to the table it designed.

If arrivals exceed service, the team either adds budget or adopts fewer terms. If service exceeds arrivals, the team has room to adopt more, and the room is evidence rather than optimism.

The quarter is also the test of the diff. A diff that catches values the annual review missed has earned its place; a diff that catches only normalization noise has not.

The second year

The second year is the test of the queue's steady state. The backlog is gone or it is not; the adoption rate is stable or it is not; the recurring hours are known or they are not.

A team that adopted forty terms in the first year carries eighty recurring hours in the second. The queue wait is two weeks, and the team knows which two weeks.

The second year is boring, and the boredom is the result. A queue that still produces emergencies after a year is a queue that was never staffed.

The diff also settles into a rhythm. Twelve values appear, eight are adopted, four are mapped, and the log records both kinds of decision.

The adversarial value

A value can be declared and still be wrong. The diff catches the undeclared value; it does not catch the declared value whose meaning has shifted.

`reopened` is a new string, so the diff sees it. A value that reuses `pending` for a new meaning is invisible to the diff, and the only test is a definition check against the declared sense.

The diff is a name check. It is necessary and not sufficient, and the limitation is named rather than hidden.

The review table

A review table prices a term before adoption. It lists adoption hours, recurring hours and queue wait, and the team fills it before the vote.

A term with two hours of adoption and four hours a year of review is a different commitment from a term with eight and twenty.

The table is a design artifact, not a benchmark. The shape is the reusable part.

The first diff

The first diff returns three values. One is a new meaning; two are normalization variants. The reviewer adopts the first and records the other two as resolved.

The first diff is also a calibration. It tells the team how noisy the source is and how much review the diff will generate.

The queue's budget

The queue's budget is the number of hours the team will actually spend, not the number it wishes it had. A budget written after the queue is full is a description, not a constraint.

The budget is set before the first adoption and reviewed after the first quarter.

The stop rule and the budget

The stop rule is the budget applied to a term. A term that would push the queue past its hours stays registered.

The budget is not a cap on ambition; it is a cap on obligations.

The drift ledger

The drift ledger is the log plus the queue plus the receipts. It is the artifact an auditor reads.

A ledger without receipts is a list of events. A ledger with receipts is a history of decisions.

The annual review's cost

An annual review costs a meeting and a document. A diff costs a query and an hour. The diff is cheaper per value, and the annual review is cheaper per year, which is why the annual review survives in organizations that do not measure drift.

A program that measures drift will keep the annual review for the definitions and use the diff for the values, because the two instruments answer different questions.

The queue's first year

The queue's first year is a calibration. The team measures arrivals, service and wait, and sets the budget for the second year.

The budget is the plan. The queue is the evidence.

The objection

The strongest objection is that this makes governance a bureaucracy. A small team with one source system does not need a queue; its vocabulary lives in the analysts' heads, changes are cheap, and the queue would add latency to decisions nobody disputed.

The concession is real. At one system and one quarter, the queue is close to zero value. Its value rises with the number of systems citing a term, the length of the horizon, and the consequence of being wrong.

A second objection is that AI will automate the review. It can propose amendments, cluster the queue and draft the evidence. Adoption remains a named act with a receipt, or drift returns as automation without accountability.

The review-hour figures are designed, not measured. The page prints the table and the stop rule, not a benchmark. A reader who wants a measurement should run the queue for a quarter and count the hours.

What would refute this

Take two comparable vocabularies over a year. One runs the usage diff and a measured queue; the other reviews annually. Compare the number of unlicensed values that reached a report, the hours spent, and the time to a defensible answer.

If the annual review produces fewer unlicensed values at lower cost, the queue is overhead. If the diff catches values the annual review missed, the queue has evidence.

A cheaper falsifier: if the usage diff returns no rows for a year, either the vocabulary is stable or the diff is not running. The second is more likely, and the empty result is the finding.

A queue that is measured can be staffed, compressed or refused, while a queue that is not measured becomes the place where a vocabulary quietly stops changing.

Drift is the defect an unowned vocabulary produces; the next chapter asks what alignment can prove when two owned vocabularies meet.

36 · Profiles and Alignment

Two departments can both say `Customer` and mean different commitments. An equivalence arrow makes them one concept and loses both meanings. Alignment starts from that loss.

The alignment table

A mapping record has source, target, direction, trace, gap, version, owner and date. The table is the desk page, and a reader audits one row at a time.

The table is a list of claims, not a graph. A graph of mappings without traces is a picture of guesses, and the picture is more persuasive than the evidence.

A row with no owner is a rumor; a row with no version cannot be replayed; a row with no direction hides which loss the consumer inherits.

The table is small for a task. A profile names the pairs that matter, and the rest of the ontology is out of scope.

The profile

A profile is a named set of commitments for a task: predicates, roles, senses and inferences. The alignment is between profiles, not between whole ontologies.

A profile makes the mapping small enough to check. Two whole ontologies have thousands of possible pairs; two profiles have the pairs the task needs.

The profile also names what the task does not need, and that is what keeps the mapping bounded. A mapping without a profile is an attempt to align everything, which is an attempt to align nothing.

A profile has an owner and a version, just like a commitment. A profile without an owner is a wish list.

Composition

A->B and B->C compose into A->C only if the traces compose. The composition rule is explicit, and the residual travels with the result.

A composed mapping without a composition rule is a guess with three vocabularies. The residual is the part that says what the composition could not preserve.

Composition is where most alignment programs fail silently: the pairwise mappings look sound, and the composed mapping has a gap that no one checked.

A composed mapping also accumulates version risk. Each pair has two versions, and the composition is only valid for the versions it used.

Two tasks, one mapping

The same Customer definitions can support two contracts. A churn report may use compliance Customer as retail Customer under the KYC assumption; a regulatory count may not.

The contract is the difference. The mapping is the same; the permission is not.

A consumer that ignores the task scope will use the mapping where it was never granted, and the error will look like a data problem rather than a permission problem.

A contract that names its task is a permission; a contract that names no task is a claim about the world.

The gap report

A gap report names the condition that would close the mapping. It is actionable: a definition, a test, a ruling or a new source.

A gap with no closing condition is a complaint.

The gap report also carries the direction. A gap in A->B may be closed by a definition that A already has; a gap in B->A may need a source that neither has.

The version boundary

A mapping is checked against two versions. When either version changes, the trace is re-run, and the result may be a gap where there was a trace.

A version boundary that is not recorded is a mapping that silently changes meaning. The record names both versions and the date of the check.

A mapping that is not rechecked after a version change is a claim about a vocabulary that no longer exists.

The owner

A mapping has two owners, one for each vocabulary, and a joint review date. A mapping with one owner is a claim about the other vocabulary that the other owner never accepted.

The owners do not have to agree on the world; they have to agree on the mapping and the residual.

A mapping with no owner drifts, and the drift is invisible until a consumer inherits the wrong loss.

The first mapping

The first mapping is the template. It records the trace, the gap, the direction, the version and the owners, and every later mapping is compared to it.

A team that writes the first mapping carefully has a pattern; a team that draws the first arrow has a habit.

The refusal

A refusal is the third outcome: the mapping cannot be evaluated without an input. The input may be a definition, a version or a scope.

A refusal is not a gap. A gap says the mapping fails; a refusal says it cannot be tested yet.

A refusal with a named input is a work item. A refusal without one is a shrug.

The contract fields

A contract names source version, target version, permitted use, assumptions, transformation, residual, acceptance and revalidation trigger. The fields are what make the permission reviewable.

A contract with no task is a claim about the world; a contract with a task is a permission for that task.

A contract with no residual is claiming the transformation was lossless, and the ledger is the evidence.

The customer trace

The B->A trace names B's KYC axiom, A's order axiom, the rule that would connect them and the gap. The gap is the missing entailment, not a failed test.

The A->B trace has a different gap. The two are not symmetric, and the direction is what tells a consumer which gap it inherits.

A trace that fills both gaps is an equivalence; a trace that fills one is a subsumption; a trace that fills neither is two gap reports.

The alignment review

The review reads the table, the versions and the residuals. It re-runs the traces against the current versions and records the date.

A review that does not re-run the trace is a review of the document, not of the mapping.

The standard comparison

OWL supplies class expressions and equivalence; SKOS supplies broader, narrower and close match; an upper ontology supplies categories. None of them supplies the local commitment that makes the mapping true for a task.

The standards make the mapping checkable, the profile makes it bounded, and the trace makes it disputable, which is the most a mapping can be.

A team that has the syntax and not the commitment has a language for saying things it cannot defend.

The anti-walkthrough

An arrow is not a warrant. The arrow records that someone asserted a relation; the trace records why.

A mapping that cannot show its trace is a drawing with a version number, and the version number is the only part a reviewer can check against the record.

The mapping's reader

The reader of a mapping is a consumer who needs the permission, not a logician who needs the proof. The trace is written for the consumer.

The claim is the mapping.

The arrow and the claim

A mapping is a claim that one vocabulary may be used in place of another for some purpose. The claim has a source, a target, a direction, a version and a warrant. The arrow is a picture of the claim, not the claim itself.

An `owl:equivalentClass` assertion says the two classes have the same extension. That is a strong claim, and it is usually stronger than the evidence supports. The standards can express the claim; they cannot supply the warrant.

Alignment has three outcomes.

Outcome	What it says
Trace	the mapping holds, and the axioms and rule used are named
Gap	the mapping does not hold as stated, and the missing condition is named
Refusal	the mapping cannot be evaluated without a missing input

A mapping with no trace returns `GAP_NO_TRACE`. The guard is the difference between an alignment and a drawing.

Two Customer definitions

Here is an example.

```
A (retail): CUSTOMER(x) iff x placed an order
B (compliance): CUSTOMER(x) iff x passed KYC
```

Map B->A. The trace asks whether B's KYC event entails A's order event. It does not: a party can pass KYC and never order. The result is a gap report, not an equivalence.

Map A->B. The trace asks whether A's order event entails B's KYC event. It does not: a party can place an order before the KYC check completes. The result is a second gap report.

Two gap reports are the honest output. The arrow would have merged the two definitions and lost the difference. The gap reports keep both and name what would close each one.

A gap report is not a failure. It is the product of an alignment that could not establish equivalence, and it is more useful than an equivalence that nobody can defend.

Trace or gap

A proof trace names four things.

Field	What it records
Source axiom	the commitment the source vocabulary makes
Target axiom	the commitment the target vocabulary makes
Rule	the inference that connects them, or the rule that fails
Gap	the condition that would close the mapping

The trace is not prose. It is a list of axioms and a rule, and a reader can check each item against the source.

A gap report has the same fields, with the gap filled and the rule marked as failed. The gap is a first-class output, not an error code.

The trace also has a version. A mapping made against release twelve is not the same claim as a mapping made against release thirteen, and the version is what lets a later reader reconstruct the warrant.

Direction and version

Alignment is directional. A->B and B->A can have different traces and different losses.

A retail definition used in a compliance report may lose the KYC condition. The direction determines which loss the consumer inherits, and a symmetric mapping with no direction hides the choice.

The version is the second axis. A mapping is a claim about two versions, and a change to either version can invalidate the trace. The mapping record names both versions and the date it was checked.

A mapping without a direction is a slogan. A mapping without a version is a claim that cannot be replayed.

What the standards solve

The standards solve more than they are credited for. OWL gives class expressions, equivalence and subsumption; SKOS gives broader, narrower, related and close match; upper ontologies give categories and some role patterns.

What the standards do not give is the local commitment. They do not say that this department's `Customer` means the party that ordered, or that the other department's means the party that passed KYC. The mapping between the two commitments is local work, and the standards make it checkable rather than automatic.

SKOS also supplies a useful caution: `closeMatch` is not transitive. A close match to B and a close match from B to C do not compose into a close match from A to C. The mapping has to be rechecked after composition, and the residual has to travel with it.

An upper ontology can reduce the number of pairwise mappings by giving a pivot. It does not eliminate the mapping work; it converts many local authorship decisions into one adoption plus many mapping decisions.

The semantic contract

A mapping is often richer than equivalence. It is a conditional permission.

```
Compliance Customer may be used as Retail Customer
for the churn report,
under the assumption that every retail customer has passed KYC,
after transforming the KYC date into a first-order date,
with a residual of 12% unmatched records.
```

The contract records the source and target versions, the permitted use, the task scope, the assumptions, the transformation, the residual and the acceptance status. It is a permission for a task, not a claim about the world.

The contract is what makes the mapping usable without pretending it is an equivalence. A consumer who needs the churn report can use the compliance definition under the stated assumption; a consumer who needs a regulatory count cannot.

The contract also has a revalidation trigger. A change to either version, to the assumption or to the transformation reopens the contract, and the consumer is told that the permission has changed.

The transformation ledger

Every transformation records what it preserved, generalized, omitted, introduced and left unresolved.

Category	What it records
Preserved	the fields and claims that survived unchanged
Generalized	the distinctions that were collapsed
Omitted	the content that was dropped
Introduced	the content that the transformation added
Unresolved	the cases the transformation could not handle

The ledger is the receipt for the transformation. It says what the consumer gained and what it lost, and it is the difference between a projection and a silent rewrite.

The ledger is also compositional. A chain of transformations accumulates loss, and a later step must not erase an earlier residual. The final consumer sees the accumulated ledger, not the last step's clean report.

The concession

Full proof traces are expensive. A team may accept a declared gap with a review date, and the gap is a first-class output because the alternative is an undeclared arrow.

The concession does not weaken the claim. A declared gap is honest; an equivalence with no trace is a guess wearing a standard's clothes.

A mapping with a trace, a direction, a version and a ledger is translation with receipts. A mapping without them is a drawing.

The alignment in one page

A mapping is a claim, and a claim needs a source, a target, a direction, a version and a warrant that a reader can inspect. The table records those fields, and the trace names the axioms and the rule that connect the two vocabularies. A gap report names the condition that would close the mapping, and the direction tells the consumer which gap it inherits.

A profile bounds the work to the pairs the task needs, and the contract grants a permission for that task under stated assumptions. A transformation ledger records what the mapping preserved, generalized, omitted, introduced and left unresolved. A mapping without a trace is a drawing, and a drawing with a version number is still a drawing.

The standard supplies the syntax, the profile supplies the commitment, and the trace supplies the warrant. Alignment is translation with receipts, and the receipt is the trace plus the gap plus the direction plus the ledger.

A row with no owner is a rumor, a row with no version cannot be replayed, and a row with no direction hides which loss the consumer inherits. The table is a list of claims, not a graph, and the graph is a picture of the claims.

An alignment that cannot show the axioms it used is a claim that the reader has to accept on the authority of the person who drew the arrow, and authority is not a trace. The direction of a mapping determines which vocabulary loses the condition, so a mapping that is presented as symmetric has hidden the choice from the consumer who inherits the loss. A version boundary that is not recorded turns a mapping into a silent change of meaning, and the change is invisible until a consumer receives an answer that the source vocabulary would not support.

A transformation ledger that records preserved, generalized, omitted, introduced and unresolved content is the receipt for a projection, and a projection without a ledger is a rewrite that no one can audit. A profile that names the pairs a task needs is what keeps an alignment small enough to check, because two whole ontologies present thousands of pairs and a task presents a handful. A gap report that names the condition which would close the mapping is a work item, while a gap that names only the failure is a complaint that no one can schedule.

The standards supply the syntax and the semantics, the profile supplies the commitment, and the trace supplies the warrant, and a mapping that has only the first is a syntactic claim dressed as a semantic one. Alignment is translation with receipts, and the receipt is the trace plus the gap plus the direction plus the ledger, which is what lets a consumer accept the mapping without accepting the mapper.

A mapping that is not re-run after a version change is a claim about a vocabulary that no longer exists, and the consumer has no way to know that the warrant has expired.

A table of mappings without traces is a picture of guesses, and the picture is more persuasive than the evidence because it looks like a graph. A contract that does not name its task is a claim about the world, and a claim about the world is not a permission for a consumer to use one vocabulary as another.

Alignment is translation with receipts; the next chapter prices the distinctions left unrecorded.

37 · Semantic Debt

Three reports can use the same phrase and count different things. The difference is a distinction that no commitment declares. That gap is semantic debt.

The distinction nobody wrote

The reports say `active customer`. One means a customer who ordered within ninety days. One means a customer who logged in within thirty days. One means a customer who has not churned.

Nobody wrote the distinction down. The three reports are each correct under their own definition, and the definitions are not in the commitment set; the organization is running three meanings under one phrase, and the phrase is the only thing they share.

The debt is the distinction, not the phrase. The phrase is what makes the debt hard to see, because it looks like agreement.

The three counts

The three definitions produce three counts for the same quarter.

```
report A: ordered in 90 days -> 1,240
report B: logged in within 30 days -> 2,910
report C: not churned -> 3,470
declared: none
```

The counts differ by a factor of almost three. A reader who sees the three numbers without the definitions will assume one is wrong. A reader who sees the definitions will see three different questions.

The reconciliation meeting is where the difference surfaces; the meeting spends a day deciding which number to use, and the decision is usually a choice of report rather than a choice of meaning.

The debt row is created at that meeting: one distinction, three implementations, no declaration.

The interest

The interest on the debt is the reconciliation cost. It is paid every period in hours and in errors.

Cost	What it is
Reconciliation hours	the meeting that decides which number to use
Error cost	the decision made from the wrong number
Opportunity cost	the question that could not be asked while the numbers disagreed

The designed interest is one day per quarter. A reader's hours will differ, and the template is the artifact rather than the rate.

The interest compounds when the reports are used in downstream decisions. A dashboard that picks report A and a board pack that picks report C produce two versions of the same quarter, and the difference is discovered after both have been presented.

Per distinction, not per system

The debt is measured per distinction, not per system. A system-level score hides which distinction is expensive, and a distinction-level ledger names it.

Distinction	Implementations	Interest	Owner	Accepted
active_customer	3	1 day/quarter	data council	no
preferred_supplier	2	2 hours/quarter	procurement	yes
churn_risk	4	3 days/quarter	risk	no

The ledger is a list of distinctions, and each row is a liability; the row with `accepted: yes` is a decision to leave the debt unpaid; the row with `accepted: no` is a work item.

A system-level score can improve while the most expensive distinction gets worse. The per-distinction count is what makes the trade visible.

The ledger

The debt ledger has six columns: distinction, implementations, interest, owner, accepted and next review.

The owner is the person or council who can decide the meaning. An unowned debt is drift, because no one can be asked to pay the interest.

The accepted column is the honesty of the ledger. A program that claims zero debt has either no distinctions in use or a ledger that does not count them.

The next review is the date at which the debt is reconsidered; a debt can be accepted for a quarter and reviewed after the quarter; an accepted debt with no review date is an abandoned debt.

Rational debt

Not all debt should be repaid. A distinction can be left undeclared when the reconciliation cost is lower than the cost of deciding, and the ledger records the choice.

A rational debt has three properties: the interest is known, the owner accepts it, and the review date is set; an accidental debt has none of the three, and it is the one that compounds.

The distinction is not moral. A team can rationally accept a small debt for a quarter and repay it when the use case arrives; the ledger's job is to make the acceptance explicit, not to insist on repayment.

The same ledger can show over-declaration: a commitment that nobody uses. That is a different liability, and it deserves its own row.

The reconciliation query

The reconciliation query returns the three counts and the mismatch.

```
A: count(customer where ordered within 90 days) -> 1,240
B: count(customer where logged in within 30) -> 2,910
C: count(customer where not churned) -> 3,470
mismatch: 3 counts, 1 distinction, no declaration
```

Before the decision, the query returns three counts and one mismatch. After the decision, it returns the adopted count and the mapped counts, and the mismatch is resolved or accepted.

The query is not the decision. It is the evidence that the decision is needed, and the ledger is the record of what was decided.

What debt is not

Semantic debt is not messy data. Messy data is wrong values; semantic debt is a distinction that is used and not declared, and the values can be perfectly clean.

It is not technical debt. Technical debt lives in code; semantic debt lives in meaning, and its interest is paid by people reconciling numbers.

It is not a bug. A bug has a correct behavior; a debt has a decision that has not been made.

The test is a distinction used in practice and absent from the declaration. The three `active customer` definitions pass the test.

A system-level score does not pass the test, because it hides which distinction is expensive.

Interest and principal

The principal is the cost of deciding the distinction: the meeting, the evidence and the ruling.

The interest is the recurring cost of not deciding: the reconciliation, the errors and the questions that cannot be asked.

A debt is rational when the interest is lower than the principal for the period. A debt is accidental when nobody priced either.

The ledger records the principal, the interest and the decision. A row without a principal is a complaint; a row without an interest is a guess.

The reconciliation meeting

The meeting is where the debt surfaces. Three reports, three counts, one phrase; the meeting spends a day deciding which count to use.

The decision is usually a choice of report, not a choice of meaning. The ledger records the choice and the meaning that was not decided.

A meeting that ends with a number and no distinction has paid the interest and kept the debt.

The meeting is also where the owner is named. A debt with no owner is a meeting that will happen again next quarter.

The ledger in practice

The ledger has distinction, implementations, interest, owner, accepted and next review.

The owner is the person who can decide the meaning. An unowned debt is drift, because no one can be asked to pay the interest.

Over-declaration is a debt too: a commitment nobody uses. The ledger can show both, and both cost.

The next review is the date the debt is reconsidered. An accepted debt with no review date is an abandoned debt.

Debt and drift

Drift is change without a transition; debt is a distinction without a declaration. Drift adds debt, and the ledger prices what drift creates.

A drift that is never declared becomes a debt; a debt that is never reviewed becomes drift. The two are a loop.

The loop is why the ledger and the diff belong together. The diff finds the change; the ledger prices the distinction.

The repayment decision

Repay, map or accept.

Repay when the distinction matters and the principal is affordable. Map when two implementations can be reconciled under a contract. Accept when the interest is lower than the principal for the period.

The ledger records the decision and the review date, and the decision can change at the review.

The three reports

Report A counts a customer who ordered within ninety days. Report B counts a customer who logged in within thirty days. Report C counts a customer who has not churned.

The counts are 1,240, 2,910 and 3,470. None is wrong. Each report answers the question its definition asks, and the definitions are not in the commitment set.

The reports share a phrase and nothing else. The meeting that compares them spends a day deciding which count to use, and the decision is a choice of report rather than a choice of meaning.

The debt row is created at that meeting: one distinction, three implementations, no declaration, one owner to name.

A reader who sees the three counts without the definitions will assume one is wrong. A reader who sees the definitions will see three different questions.

The debt register

The register has columns for distinction, implementations, interest, owner, accepted and next review.

A row for `active_customer` names three implementations, one day per quarter of interest, the data council as owner, `accepted: no` and a review date.

A row for `preferred_supplier` names two implementations, two hours per quarter, procurement as owner, `accepted: yes` and a review date.

A row for `churn_risk` names four implementations, three days per quarter, risk as owner, `accepted: no` and a review date.

The register is a list of distinctions, not systems. A system-level score can improve while the most expensive distinction gets worse, and the register is what makes the trade visible.

The interest calculation

The interest has three parts: reconciliation hours, error cost and opportunity cost.

Reconciliation hours are the meetings and the analysis that decide which number to use. Error cost is the decision made from the wrong number. Opportunity cost is the question that could not be asked while the numbers disagreed.

A dashboard that picks report A and a board pack that picks report C produce two versions of the same quarter, and the difference is discovered after both have been presented.

The interest is designed as one day per quarter for `active_customer`. A reader's hours will differ, and the template is the artifact rather than the rate.

The interest compounds when the reports feed downstream decisions. A single reconciliation is a cost; a quarterly reconciliation is a liability.

The debt review

The review is quarterly. Each row is read, the interest is compared to the principal, and the owner decides to repay, map or accept.

Repay means decide the distinction and declare it. Map means reconcile two implementations under a contract. Accept means leave the distinction undeclared and record the interest.

A debt accepted for a quarter is reviewed after the quarter. An accepted debt with no review date is an abandoned debt.

The review also checks for over-declaration: a commitment nobody uses. That is a different liability, and it deserves its own row.

The debt ledger in practice

A ledger row has six columns: distinction, implementations, interest, owner, accepted and next review. The row is the unit of governance, not the diagram.

A row with no owner is drift, because no one can be asked to pay the interest. A row with no review date is an abandoned debt, because no one has agreed to reconsider it.

The first reconciliation is the template. It names the distinction, the three reports, the counts, the meeting hours and the decision. The decision is a choice of report or a decision of meaning, and the ledger records which.

An accepted debt is a decision, not a failure. It has a known interest, an owner and a review date. An accidental debt has none of the three, and it is the one that compounds.

The owner is the person who can decide the meaning, not the person who maintains the report. An owner without a scope is a bottleneck; a scope without an owner is a folder.

The review date is the contract with the future. A debt can be accepted for a quarter and revisited after the quarter. A review that does not read the ledger is a calendar entry.

Over-declaration is a debt too: a commitment nobody uses. It has an interest in maintenance and an owner who cannot say why the commitment exists. The ledger can show both kinds, and both cost.

The interest compounds when the reports feed downstream decisions. A dashboard that picks report A and a board pack that picks report C produce two versions of the same quarter, and the difference is discovered after both have been presented.

The stop rule is the principal. A debt is worth repaying when the principal is lower than the accumulated interest for the period the team can see. A debt is rational when the interest is lower and the owner accepts it with a review date.

The ledger is the artifact because it prices the distinction. A diagram can show the classes; it cannot show which distinction costs a day per quarter.

The debt in practice

The reader of the ledger is a program lead who needs to know which distinction costs the most. The ledger answers with a row, not a diagram.

A row names the distinction, the implementations, the interest, the owner, the accepted status and the review date. The reader can open the row and ask who accepted it.

A ledger that is not read is a diagram with columns. A ledger that is read is a governance artifact.

The arithmetic is simple: interest is hours plus errors plus opportunity, and principal is the cost of deciding the distinction. Repay when the principal is lower than the accumulated interest, and accept when the interest is lower and the owner signs.

Acceptance is a decision with a review date. An accepted debt without a review date is an abandoned debt, and an abandoned debt is drift with a receipt.

The first quarter is the calibration. The team counts reconciliation hours, names the distinctions and sets the review date.

Over-declaration is the other liability: a commitment nobody uses. The ledger can show both, and both cost.

The ledger is the artifact. The distinction is the unit, the interest is the price, and the review is the contract.

The ledger's test

The ledger is tested by a reconciliation, and a reconciliation that ends with a number and no distinction has paid the interest and kept the debt, which is the failure the ledger exists to make visible.

The row is the unit. The interest is the price. The review is the contract. The ledger is the artifact.

A ledger that names the owner and the review date is a governance artifact, while a ledger that names only the distinction is a list with a title.

A row that cannot name its owner is a row that cannot be paid, and a row that cannot be paid is a note rather than a liability. The debt is the distinction. The interest is the cost.

The concession

The interest figures are invented. A reader's reconciliation hours will differ, and the page prints the template rather than a benchmark.

The claim is the unit, not the rate. The unit is the distinction, and the ledger is the artifact.

A team that fills the template with its own hours has a measurement. A team that reads the example as a rate has a story.

The row is the unit. The owner is the payer. The review is the contract.

A ledger that cannot name the owner of a row cannot price the debt, and a debt that cannot be priced is a sentiment rather than a liability.

Debt is what an unnamed distinction costs; the next chapter asks what a finite middle buys.

38 · The Finite Middle

Two vocabularies can be matched term by term, and the cost is the product of their sizes. Fifty source terms against forty target terms is two thousand comparisons before any judgment is made; the comparison grows with the product, and the report cannot say what was missed.

The comparison count

Fifty source terms against forty target terms is two thousand comparisons; a hundred against a hundred is ten thousand; the count is the product, and the product grows faster than either vocabulary.

The review grows with the count; a reviewer who can examine fifty pairs in a day needs forty days for two thousand, and the forty days are before any decision is made.

The report is a list of pairs, not a coverage report; a pair that matched is visible; a pair that did not match is an absence, and the absence has no name.

A term that matched nothing looks the same as a term that was never compared. The pairwise match cannot tell the difference.

The count is also the argument for the middle. A finite set turns the product into a sum, and the sum can be reported.

What the middle is not

The middle is not necessarily a standard. It can be a standard, and more often it is a small local set of categories that both sides can own.

It is not an upper ontology. An upper ontology supplies categories at a high level of abstraction; the middle supplies the categories a particular match needs.

It is not a taxonomy. A taxonomy classifies one vocabulary; the middle is a third set that both vocabularies map into.

A middle of twelve categories is small enough for one owner to complete; a middle of a thousand categories is a vocabulary project, and it has the same coverage problem it was meant to solve.

The coverage review

The coverage review reads the three states and turns them into decisions; matched terms need a trace; unmatched terms need a category or a decision to leave them out; ambiguous terms need a ruling.

State	Decision
Matched	record the trace
Unmatched	add a category or accept the gap
Ambiguous	rule, split the category or declare the ambiguity

A percentage alone hides the ambiguous cases, and the ambiguous cases are where the expensive decisions live.

The review is also the queue. A middle with no review is a table with no owner.

The middle's version

The middle has a version. A middle that adds a category changes the coverage of every prior match, and the version is what lets a consumer compare reports across releases.

A version without a change record is a number.

The version also triggers revalidation. A consumer that used the middle for a report needs to know that the categories moved, and the version is the notice.

A middle that is owned but not versioned is a list; a middle that is versioned but not owned is a proposal.

The first middle

The first middle starts from the questions, not the terms. A team asks which categories its reports need, and writes twelve of them.

The fifty source terms map into the twelve, and the forty target terms map into the twelve. The mapping is ninety checks, and the report is matched, unmatched and ambiguous.

A first middle that reports no unmatched terms is either complete or not looking. The second is more likely, and the unmatched row is the work item.

The first middle is revised after the first review, and the revision is a version.

The middle's cost

The middle costs a projection on both sides. A source term that fits two categories is ambiguous, and the ambiguity is reported rather than hidden.

The cost is the loss of distinctions that do not fit the twelve. The benefit is a coverage report that names what was missed.

The middle is worth its cost when the match must be governed. A one-off match can use a pairwise score; a recurring match needs a middle.

The middle in practice

The count is the product for a direct match and the sum for a middle; fifty by forty is two thousand; fifty plus forty is ninety, and the ninety is the work the team can review.

A middle of twelve categories turns the pairwise comparison into two mappings. The source maps into the middle, the target maps into the middle, and the comparison happens once per category instead of once per pair.

The middle has an owner with a scope and a version. The owner decides a category and defends the decision; the version lets a consumer compare reports across releases.

The coverage table has categories, source members, target members, state and owner. A row with no target members is unmatched; a row with two candidate mappings is ambiguous; a row with one is matched.

The table is the queue. Unmatched rows need a category or a decision to leave them out; ambiguous rows need a ruling; matched rows need a trace.

The middle is a projection, and every projection loses something. The loss is the price of counting, and the unmatched and ambiguous states name it.

A middle that reports only matched terms is a pair list with a title. A middle that reports all three states is a coverage report.

The first year adds categories, resolves ambiguities and reviews the unmatched rows. The version increments, and the coverage report changes.

A middle that does not change in a year is either complete or unused. The second is more likely, and the review is what tells the difference.

The test is cheap: run the coverage report before the match. A match that cannot produce the report has no middle, whatever it calls its categories.

The middle's arithmetic

The arithmetic is the argument. A middle that does not reduce the count is not a middle; it is a category list with a title.

The count also prices the review. Ninety checks are reviewable; two thousand are a project with a queue.

The arithmetic is not a proof of correctness. A middle can reduce the count and still be wrong, which is why the coverage states and the trace are separate.

The middle's limit

A middle that claims to lose nothing is a pairwise match in disguise.

The limit is versioned, and the version is what lets a consumer see the loss.

The middle's owner

The middle has an owner with a scope and a version. An owner without a scope is a bottleneck; a scope without an owner is a folder.

The owner reviews the unmatched and ambiguous rows, and the review is the governance.

The owner also decides when a category is retired. A retired category is a version change, and the coverage of every prior match changes with it.

The middle's test

The test is a coverage report with three states and a count. A match that cannot produce the report has no middle, whatever it calls its categories.

A middle that reports only matched terms is a pair list with a title.

The middle's first year

A middle that does not change in a year is either complete or unused. The second is more likely.

The middle's cost

The cost is the projection on both sides. A source term that fits two categories is ambiguous, and the ambiguity is reported rather than hidden.

The benefit is a coverage report that names what was missed. The cost is worth paying when the match must be governed.

The middle's review

The review reads the table, prices the unmatched and ambiguous rows, and decides whether to add a category, retire one or accept the gap that the middle cannot close. A review that reads only the matched rows has read the easy half of the report, because the unmatched and ambiguous rows are where the decisions live.

The version is the record of the review, and a version that does not change after a review is a version that no one used. The middle is worth its cost when the match must be governed, and a one-off match can use a pairwise score without pretending it has coverage.

The review is the governance. The version is the receipt.

The pairwise trap

A direct match has no finite set to miss against. It compares every source term with every target term, and the result is a similarity or a judgment for each pair.

The cost is multiplicative. Fifty by forty is two thousand. A hundred by a hundred is ten thousand. The count grows faster than either vocabulary, and the time to review grows with it.

The output is a list of pairs, not a coverage report. A pair that matched is a pair; a pair that did not match is an absence, and the absence has no name. A term that matched nothing looks the same as a term that was never compared.

The pairwise match is also unfalsifiable. A reviewer can check a pair, but cannot check the space of pairs that were not examined. The report says what matched; it cannot say what was missed.

The middle

A finite middle is a small set of categories both sides map into. The source maps into the middle, the target maps into the middle, and the comparison happens in the middle.

```
direct: 50 × 40 = 2,000 comparisons, no coverage
middle: 12 categories; 50->12 (50 checks) + 40->12 (40 checks) = 90
report: matched 38, unmatched 9, ambiguous 3
```

The count collapses from two thousand to ninety. The middle does not make the match automatic; it makes the match countable.

The middle is not a standard. It can be a standard, and more often it is a small local set of categories that both sides can own. The requirement is that it be finite and complete enough to count against.

Coverage

Coverage has three states: matched, unmatched and ambiguous.

State	What it means
Matched	the source and target terms map to the same middle category
Unmatched	a term maps to no middle category, or maps to a category with no counterpart
Ambiguous	a term maps to two or more middle categories, and the choice is unresolved

The three states are the report. A percentage alone hides the ambiguous cases, and the ambiguous cases are where the expensive decisions live.

The coverage table is also a work queue. An unmatched term needs a category or a decision to leave it out. An ambiguous term needs a ruling. A matched term needs a trace.

A match with no middle returns `REFUSE_UNBOUNDED`. The guard is the difference between a coverage report and a list of pairs.

The middle table

The middle table has a row per category, with the source members, the target members, the coverage state and the owner.

Category	Source members	Target members	State	Owner
party	customer, supplier	customer, vendor	matched	data council
status	active, inactive	active, churned	matched	data council
segment	priority, standard	enterprise, smb	ambiguous	commercial
channel	web, store	digital, retail	unmatched	commercial

The table is the desk page. A reader can see which categories are matched, which are ambiguous and which have no counterpart.

The table is also the version. A middle that changes without a version is a middle that silently redefines the match.

Owning the middle

The middle is a governance artifact, owned and versioned. An unowned middle drifts, and a drifting middle makes every coverage number stale.

The owner is a named person or council with a scope. The scope is the set of vocabularies the middle is meant to cover; a middle that claims to cover everything is a standard, and a standard has a different governance problem.

The version is the date and the change. A middle that adds a category changes the coverage of every prior match, and the version is what lets a consumer compare reports across releases.

The concession

The middle imposes categories that may not fit either side. That is the cost of being able to count, and the fit is versioned.

A source term that fits two categories is ambiguous, and the ambiguity is reported rather than hidden. A term that fits none is unmatched, and the gap is a work item.

The middle also creates a temptation: to force every term into a category so the coverage looks complete. The ambiguous and unmatched states exist to resist that temptation.

A middle that reports no unmatched terms is either complete or not looking. The second is more likely.

The primes as limiting case

Semantic primes are the limiting case of a finite middle: a small set of meanings used to define the rest. The parallel is structural, not a claim that primes are the only middle.

A middle does not need to be a prime set. It needs to be finite, owned, versioned and complete enough to count. A domain middle of twelve categories can do that for a domain.

The primes matter because they are the smallest middle that claims to cover meaning in general, and the claim is a hypothesis. The claim is narrower: any middle that is finite and owned makes coverage measurable.

The objection

The strongest objection is that the middle forces both sides into categories that fit neither. A source vocabulary with fifty terms and a target with forty will lose distinctions when both are compressed into twelve categories.

The concession is real. The middle is a lossy projection, and the loss is the price of counting. The alternative is a pairwise match that cannot report its own coverage, and the loss is invisible.

A second objection is that a model can match better than twelve categories. It can match more fluently, and it cannot tell you what it missed without a finite set to miss against.

The middle is versioned, so the categories can change. A middle that is wrong is a middle that can be revised with a receipt, and the coverage report is the evidence for the revision.

What would refute this

If a direct match can report complete coverage without a finite middle, the middle is unnecessary. If a pairwise match can name its unmatched and ambiguous cases without a middle, the coverage claim fails.

A cheaper test: count the comparisons and the review hours for a direct match and for a middle. If the direct match is cheaper and reports the same coverage, the middle is ceremony.

A middle that no one owns and no one versions is not a middle; it is a list of categories with a title.

A version that does not change after a review is a version that no one used, and a middle that no one uses is a list of categories with a title. A review that reads only the matched rows has read the easy half of the report, because the unmatched and ambiguous rows are where the decisions live.

The review is the governance.

The middle is a projection that loses distinctions, and the loss is the price of counting, which is the trade a team accepts when it needs a coverage report more than it needs every distinction preserved.

The middle is what makes a match finite; the next Part asks what meaning is, that it can cross a boundary at all.

PART VII
The Warrant

who may act on a claim.

39 · The Commitment Grid

OWL ships an entailment table; it answers what follows from a class, a property or a type; it does not answer what follows from the act of stating a claim; the second table is the commitment grid.

Two tables

A type-entailment table says that if `Stove` is a subclass of `Appliance`, then every stove is an appliance. The inference follows from the class hierarchy and the axioms.

A commitment table says that if Bob said the stove is hot, then Bob said something, and it does not follow that the stove is hot; the inference follows from the act of saying, not from the class of stoves.

The two tables answer different questions, and a graph that has only the first cannot represent the second; it can store `SAY(Bob, HOT(stove))`, and it cannot say what that structure forbids.

The type table is complete for what it covers. The commitment table is not complete, and four rows are stated here rather than a full taxonomy.

The grid

The grid has rows for combinations of presentation.

Presentation A	Presentation B	Permitted move	Forbidden move
asserted	asserted	combine, keep both warrants	treat either as the other's evidence
asserted	denied	record the conflict with both holders	record NOT(P) as a world fact
reported	asserted	assert the report; keep the content reported	promote the content to fact
inferred	reported	record the derivation and its rule	present the inference as testimony

The rows are not truth values. They say what may be combined and what may not, and each row can be checked by a query.

The asserted/asserted row permits conjunction; two asserted claims can be combined into one claim with both warrants, and the combination is still two claims; the forbidden move is to treat one as evidence for the other.

The asserted/denied row permits a conflict record. It forbids recording `NOT(P)` as a world fact, because a denial is a speech act about P, not a proof of its negation.

The reported/asserted row permits the report. It forbids promoting the content. The report is a claim about an act of speech, and the content is a claim about the world; the two are different records.

The inferred/reported row permits the derivation. It forbids presenting the inference as testimony. An inference has a rule; a testimony has a speaker, and the two warrants are not interchangeable.

Non-entailments

Three non-entailments carry the grid.

```
REPORTS(A, P) does not entail P
DENIES(A, P) does not entail NOT(P)
MAY(P) does not entail P
```

`REPORTS(A, P)` entails that A reported P. It does not entail P, because a report can be false or mistaken.

`DENIES(A, P)` entails that A denied P. It does not entail `NOT(P)`, because a denial can be wrong.

`MAY(P)` entails that P is possible. It does not entail P, because possibility is not actuality.

The three are the load-bearing rows; a graph that gets them wrong will produce a fluent answer that asserts the reverse of its source, and no structural check will catch it.

Presentation, not claim

Commitment belongs to the presentation, not to the claim. One causal relation can be asserted by one speaker and challenged by another without becoming an accepted edge.

Ivo asserts that the flood caused the damage. Tess challenges the same causal claim. The relation is one; the presentations are two; the commitments are two. A global `assertion_status` on the relation cannot represent the disagreement, because it has one slot for two speakers.

The presentation record names the speaker, the act and the content. The content is the claim; the act is the commitment; the speaker is the holder. The three are separate fields, and the grid reads them.

A presentation without a speaker is a claim without a holder. A claim without a holder can still be true, and it cannot be attributed, challenged or withdrawn.

Bob and the stove

Here is the worked case.

```
outer: SAY(agent=Bob, content=S-2)
inner: HOT(theme=stove) stance=reported
entails: Bob said something
does not entail: the stove is hot
```

The outer claim is asserted. Bob did say something, and the record can be checked against the source span.

The inner claim is reported. It is stored with its own stance, its own span and its own confidence, and it does not inherit the outer claim's assertion.

A graph that flattens the pair into `HOT(stove)` with a source field has lost the speech act; it can still show where the sentence came from, and it cannot show who committed to the content.

What the grid is not

The grid is not a truth table. It does not decide whether P is true; it decides what may be combined with what.

The grid is not exhaustive. Four rows are load-bearing; a complete table is a research program, and the grid refuses the claim that four rows are the whole taxonomy.

The grid is not a policy; a policy says what an organization chooses to allow; the grid says what the structure permits, and a policy can be stricter than the grid without changing it.

The grid is also not a replacement for the type table; the two are used together: the type table says what follows from the class, the commitment table says what follows from the act.

The grid in practice

The type table and the commitment table are used together, and the order matters. The type table says what follows from the class; the commitment table says what follows from the act that presents the class claim.

A reasoner can infer that a stove is an appliance from the class axiom, and it cannot infer that Bob's report is a report of a stove being hot from the type language, because the act of reporting is not a class. The type table is complete for its question and silent for the commitment question.

Bob's report is the smallest worked case. The outer claim is `SAY(Bob, S-2)`, and the inner claim is `HOT(stove)` with stance reported. The outer entails that Bob said something; the inner does not entail that the stove is hot.

The asserted/denied row permits a conflict record. It forbids recording `NOT(P)` as a world fact, because a denial is an act about P, not a proof of its negation. The conflict row keeps both holders and both dates.

The three non-entailments are the grid's floor. `REPORTS(A, P)` does not entail `P`; `DENIES(A, P)` does not entail `NOT(P)`; `MAY(P)` does not entail `P`. A graph that gets them wrong will assert the reverse of its source, and no structural check will catch it.

Ivo asserts that the flood caused the damage, and Tess challenges the same causal claim. The relation is one, the presentations are two, and the commitments are two. A global assertion flag has one slot for two speakers, and the slot cannot represent the disagreement.

The presentation record has a speaker, an act, content, a span and a time. The speaker is the holder, the act is the commitment, the content is the claim, the span is the source, and the time is the version.

The combination query takes two claims, reads their presentation records and looks up the grid row. It returns the permitted move and the row it used. A query that returns only "compatible" has hidden the row, and a hidden row cannot be audited.

The grid has a version. A new presentation or a new row changes the combinations, and the version is what lets a consumer compare a report across releases.

The grid costs two presentation reads, one row lookup and a version. The cost is small, and it is paid at combination time rather than at storage time. The flattening saves the cost and loses the commitment.

The grid is not a truth table, not exhaustive and not a policy. It has four load-bearing rows, three non-entailments and a version, and it is small enough to check.

The graph stores the claims; the grid stores the combinations. A graph without the grid can store the claims and cannot combine them accountably.

The grid's cases

Take one proposition: the valve was closed before the pump started. Four people present it in four ways, and the grid reads the presentations rather than the proposition.

A asserts it. The presentation is `ASSERT(A, P)`, and the grid permits A's claim to be combined with another asserted claim about the same event.

B denies it. The presentation is `DENY(B, P)`, and the grid permits a conflict row with A and B as holders. It forbids recording `NOT(P)` as a world fact, because B's denial is an act about P and not a proof of its negation.

C reports that D said it. The presentation is `REPORT(C, SAY(D, P))`, and the grid permits the report. It forbids promoting `SAY(D, P)` to P, because C's report is a claim about D's act and not about the valve.

E infers it from a rule. The presentation is `INFER(E, P, rule)`, and the grid permits the derivation with the rule named. It forbids presenting the inference as testimony, because E's warrant is the rule and not a speaker.

The four presentations can coexist in one graph. A is an assertion, B a denial, C a report and E an inference, and the graph has four records and one proposition. The proposition is the same; the commitments are not.

The combination query reads the four presentations and returns a row for each pair. A and E combine as two asserted claims with different warrants. A and B produce a conflict. C and A produce a report and an assertion, with the content still reported. D and E produce a report and an inference, with neither promoting the other.

The row is the unit of combination, and the version is the unit of comparison. A grid without a version cannot tell whether the row that permitted a combination last quarter still permits it.

The grid's cases also show the cost of flattening. A graph that stores only P with a source field has four sources and one claim, and it cannot tell which source asserted, denied, reported or inferred.

The grid is small because the presentations are few. Four rows cover the load-bearing combinations, and the cases show how the rows apply without claiming to cover every speech act.

The limits

A grid that is not stored is a diagram. The rows have to be rows in a table, and the table has to be read by the query that combines claims.

A grid that is stored but not enforced is a comment. The enforcement is a check at combination time: the two presentations are read, the row is looked up, and the forbidden move is refused.

A grid that is enforced but not versioned is a rule without a history. The grid changes when a new presentation is added, and the version is what lets a consumer compare a report across releases.

The grid also has a cost. Every combination reads two presentations and a row, and the read is the price of not flattening. The price is small relative to the cost of an assertion that nobody can attribute.

A commitment grid is a finite table of acts, and the finite table is what makes the non-entailments checkable.

The grid in one page

The grid has four load-bearing rows: asserted/asserted, asserted/denied, reported/asserted and inferred/reported. Each row names a permitted move and a forbidden move, and each can be checked by a query.

The three non-entailments are the floor. `REPORTS(A, P)` does not entail `P`; `DENIES(A, P)` does not entail `NOT(P)`; `MAY(P)` does not entail `P`.

Bob's report is the smallest case. The outer claim says Bob said something; the inner claim says the stove is hot; the inner claim is reported and does not inherit the outer assertion.

Ivo and Tess show the same structure with two speakers. The relation is one, the presentations are two, and the commitments are two; a global assertion flag cannot represent the disagreement.

The combination query reads two presentations, looks up the row and returns the permitted move. A query that returns only "compatible" has hidden the row.

The grid has a version. A new row or a new presentation changes the combinations, and the version lets a consumer compare reports across releases.

The grid is finite, not complete. Four rows cover the load-bearing combinations, and a full taxonomy of speech acts is a research program.

The grid does not decide truth. It decides what may be combined, and the truth of a proposition remains a separate question.

The grid's test

The test takes four presentations of one proposition and queries every pair. It expects a row, a permitted move and a forbidden move for each pair.

A pair that returns no row is a gap in the grid. A pair that returns the same row as another pair is a duplicate, and the duplicate is a sign that the rows are under-specified.

The test also checks the version. A row that changes without a version is a row that cannot be compared across reports.

The test does not decide truth. It decides whether the combination is permitted, and the truth of the proposition remains a separate question.

A grid that passes the test is a finite control; a grid that fails is a table with a title.

The grid's owner

The grid has an owner and a version. The owner decides when a new row is added; the version lets a consumer compare reports.

An unowned grid drifts. An owned grid with no version is a list.

The grid's cost

The grid costs two presentation reads and one row lookup. The cost is small, and it is paid at combination time.

The flattening saves the cost and loses the commitment. The saving is visible in the query plan; the loss is visible only when an auditor asks who committed.

The grid's limit

The limit is not a defect. A finite grid that is enforced is a control; an infinite grid that is described is a diagram.

The grid does not decide truth. It decides combination, and the truth of a proposition remains a separate question.

A grid that is not stored is a diagram, and a diagram cannot refuse a combination. The stored row is the control. The version is the history. The test is the row.

The grid says what follows from how it was said; the next chapter asks what the claim lets you do.

40 · Claim Capability

A claim can be true and unusable; it can be well formed and unfit to act on; the property that decides what may be done with it is not its content; it is the state the record says it has reached.

The ladder

Capability has nine states.

State	What it licenses
Captured	the claim exists as a span or a row
Parsed	the claim has a structure
Grounded	the terms resolve to addresses
Supported	evidence supports the claim
Qualified	the claim's conditions and scope are recorded
Accepted	an authority has accepted it for a scope
Actionable	an action may be based on it
Projectable	it may be exported under a declared loss
Rehydratable	it can be rebuilt from the record

A claim does not need every state. A claim that supports a report may stop at Supported; a claim that authorizes a payment must reach Actionable.

The states are per claim, not per system. A system can support Actionable claims and hold a thousand claims at Supported, and the difference is the record.

The incident

A retrieval pipeline answered a customer with a claim about a pump service. The claim was captured from a document, parsed, grounded and supported by the document's span.

The record says Supported. The system acted. The action is Actionable, two states beyond the stored capability.

At review, the trail shows the claim never left Supported. Nothing was tracking capability at all. The content was sufficient for the answer; the record was not sufficient for the action.

The incident is not a data-quality failure; the data was valid, the query ran and the answer was fluent; the failure is that a feature existed in the code and a state did not exist in the record.

Eight errors

Each adjacent pair is a category error.

From	To	The missing transition
Captured	Parsed	a parse with a structure
Parsed	Grounded	a grounding to an address
Grounded	Supported	evidence that supports the claim
Supported	Qualified	a scope and condition record
Qualified	Accepted	an authority and a date
Accepted	Actionable	an action authority and a basis
Actionable	Projectable	a projection rule and a loss report
Projectable	Rehydratable	a rehydration test

The errors are adjacent because the states are adjacent; a system that acts one state early looks like a system that is slightly optimistic; a system that acts two states early looks like the incident.

Each error is a silent promotion. The claim gains capability without a transition, and the gain is invisible because the content did not change.

Stored, not available

A capability that exists in the code and is never stored is not a control.

The code can parse, ground, support, accept, act, project and rehydrate; the record can say which of those the claim has actually reached; the first is a feature; the second is a capability.

A feature is available to every claim. A capability is earned by a claim through a transition with evidence and an authority.

The distinction matters at the action boundary. A system that checks the feature will act on every claim the parser can read. A system that checks the capability will act only on the claims that reached Actionable.

The capability query

The capability query returns the state and the transition that produced it.

```
input: claim id
output: state, transition id, actor, date, evidence
refuse: REFUSE_NO_CAPABILITY when no transition exists
```

A claim with no capability row is not Actionable, whatever its content. A claim with a capability row has a state and a receipt.

The query is the control. It is cheap, it runs at the action boundary, and it returns a refusal rather than a default.

A query that returns a state without a transition is a state without a warrant. The transition is the part that makes the state reviewable.

Minimum sufficient capability

Not every claim needs every state. The minimum-sufficient rule says stop at the state the action requires.

A claim used in a search result needs Grounded. A claim used in a report needs Supported. A claim used to approve a payment needs Actionable and a receipt.

The rule refuses four things: not every claim needs a rich contract, not every claim needs an authority, not every claim needs projection, and not every claim needs rehydration.

The refusal is the control's budget; a system that requires Actionable for every claim will either slow to a stop or promote everything, and the second is the incident at scale.

What capability is not

Confidence is a property of a claim, not a license to act; a claim can have high confidence and sit at Supported, because confidence is about the evidence and capability is about the action.

Status is a workflow label. A ticket can be closed while the claim is still Supported, and a claim can be Actionable without a ticket.

Permission is a governance grant to an actor. Capability is a record of what the claim reached, and the two are checked together at the action boundary.

Trust is a relation to a source; a trusted source can produce a claim that is only Supported, and an untrusted source can produce a claim that is Actionable after review.

The four are different questions, and collapsing them into one boolean is the incident's cause.

The transition receipt

A transition receipt has six fields: state, transition id, actor, date, evidence and scope.

The state is the destination. The transition id is the rule that allowed the move. The actor is the authority. The date is the version. The evidence is the support. The scope is what the state applies to.

A receipt with no actor is a rumor. A receipt with no evidence is a guess. A receipt with no date is unversioned. A receipt with no scope is a global claim that the record cannot defend.

The receipt is the warrant for the state, and the capability query returns it.

The capability table

The capability table has one row per claim and state, with the receipt fields.

A claim can have several rows over time, and the current state is the latest row. The history is the sequence.

A table that stores only the current state is a boolean with extra columns. The history is what lets an auditor ask how the claim reached the state.

The table also has a version per row. A row without a version cannot be compared across releases.

The action boundary

The action boundary checks the capability before the action.

A claim with no row returns `REFUSE_NO_CAPABILITY`; a claim with a state below the required state returns `REFUSE_STATE_TOO_LOW`; a claim with a state at or above the required state proceeds, and the action record names the state and the transition.

A boundary that defaults to allow is the incident. A boundary that defaults to refuse is a control.

The boundary also logs the refusal. A refusal that is not logged is a refusal that cannot be reviewed.

The nine states in practice

A claim captured from a document starts at Captured. Parsing moves it to Parsed; grounding to an address moves it to Grounded; evidence moves it to Supported.

A report can use Supported. A payment needs Actionable, and the move from Supported to Actionable requires a qualification, an acceptance and an action authority.

A claim that stops at Supported is not broken. It is a claim whose capability matches its use.

A claim that skips a state is the incident. The skip is visible only if the table records the transitions.

The capability's cost

Every transition costs a review. The review is the evidence check, the authority check and the scope check, and the three are not free.

The cost is paid per claim, and the cost is why the minimum-sufficient rule exists. A claim that needs Supported should not pay for Actionable.

A table that promotes every claim to Actionable has paid the cost and lost the distinction.

The capability's review

The review reads the table, the receipts and the action log. It asks which claims acted above their state, which states lack a transition and which transitions lack an authority.

A review that reads only the current state cannot answer any of the three.

The review is quarterly, and the action log is its evidence.

The first table

The first capability table is small: claim id, state, transition id, actor, date, evidence, scope.

A team fills it for one action boundary, not for every claim. The first table is the calibration, and the state that matters is the one the action requires.

A table that covers every claim and no action is a catalog. A table that covers one boundary and every transition is a control.

The capability in practice

The capability table has an owner with a scope; the owner is the person who can grant Actionable for that scope, and the grant is a receipted transition rather than a setting.

An owner without a scope is a bottleneck for every action; a scope without an owner is a folder that no one can act on.

Each row has a version. A state change is a new row, not an update, and the old row remains for history.

A version without a date is unversioned; a date without a transition is a timestamp. The version is what lets a consumer compare the capability across releases.

The test removes the capability row and runs the action. The action must refuse; if it proceeds, the capability is not a control.

The test then changes the state without a transition. The guard must refuse; if it accepts, the transition is not enforced.

The review reads the table, the receipts and the action log. It asks which claims acted above their state and which transitions lack an authority. A review that reads only the current state cannot answer either question.

The ladder is not a maturity model. It is a set of per-claim states, and a claim can stop at the state its action requires. A claim that stops at Supported is not incomplete; it is a claim with a smaller license.

The incident shows the difference between a feature and a capability. The retrieval pipeline had the feature to answer, and the claim had no state for Actionable.

The query returns the state and the transition. A claim with no row is not Actionable, whatever its content.

The table is the control, and the action boundary is where it is read. A boundary that defaults to allow is the incident; a boundary that defaults to refuse is a control.

The cost is the review per transition. A table that promotes every claim to Actionable has paid the cost and lost the distinction.

The capability's first boundary

A first boundary is one action: approve a refund under a threshold. The claim that the refund is owed must reach Actionable; the claim that the customer is eligible may stop at Supported.

The boundary names the required state for each action. A refund approval requires Actionable; a report requires Supported; a search result requires Grounded.

The capability table has one row per claim and state. A claim that reaches Actionable has a row for Supported, a row for Qualified, a row for Accepted and a row for Actionable, each with a transition.

The boundary refuses a claim below the required state. A claim with no row returns `REFUSE_NO_CAPABILITY`; a claim with a lower state returns `REFUSE_STATE_TOO_LOW`.

The refusal is logged, and the log is the evidence that the boundary ran.

The first boundary is small on purpose. One action and one required state are enough to test the table, and the test is what makes the table a control.

The audit reads the action log and the table. It counts actions above state, transitions without authority and boundaries that default to allow.

The capability's first year

The first year is one action boundary. The team picks refund approval, names the required state and fills the table for the claims that reach the boundary.

The first table is small: claim id, state, transition id, actor, date, evidence, scope. The team does not fill it for every claim; it fills it for the claims the action uses.

The first refusal is the proof. A claim at Supported is refused for the refund, and the refusal names the missing transition.

The first audit reads the refusals and the actions. It finds the claims that acted above state and the transitions that lack an authority.

The first correction is a receipt. The claim moves from Supported to Actionable with evidence, authority, date and scope.

The first year ends with one boundary, one table and a known refusal. The next year adds a boundary, and the table grows by the states the new action requires.

A capability program that starts with nine states and no boundary is a diagram. A program that starts with one boundary and one state is a control.

The capability's record

The record is the row plus the receipt. The row says the state; the receipt says the transition.

A row without a receipt is a state without a warrant, and a state without a warrant is a boolean with a date.

The record is the control because it answers the auditor's question: who moved the claim, when, on what evidence, and under which scope.

A record that answers four of five has answered the easy four.

The capability's limit

The ladder is finite, not a maturity model. A claim can stop at the state its action requires, and the stop is not a failure.

The limit is the budget. The budget is the number of transitions the team can review.

The capability's test

The test removes the row and runs the action. The action must refuse.

The test changes the state without a transition. The guard must refuse.

The test skips a state and asks whether the table records the skip. If it does not, the table is a current-state column, not a history.

The objection

The strongest objection is that this is bureaucracy around a boolean. A claim either supports an action or it does not, and the nine states are a ladder of names for the same decision.

The objection is right about the boolean and wrong about the record. The boolean says whether the action is allowed now; the record says which transition produced the permission, who signed it, on what evidence and when.

A boolean cannot answer why the action was allowed, and the why is what an auditor asks after the action.

A second objection is that the content is good enough and the state is a formality. The incident shows the content was sufficient and the action was not licensed; the formality is the control.

What would refute this

If a system's stored capability matches every action it takes, the ladder is unnecessary. If a system acts only on claims that reached the required state, and the state is always correct, the record is a boolean with extra names.

A cheaper test: take one claim, remove its capability row and ask whether the system still acts. If it acts, the capability is not a control. If it refuses, the record is load-bearing.

A second test: change the state from Supported to Actionable without a transition. If the system accepts the change, the transition is not enforced.

The record is the control because it answers the auditor's question: who moved the claim, when, on what evidence, and under which scope, and a record that answers four of five has answered the easy four. A table that promotes every claim to Actionable has paid the cost and lost the distinction, and the distinction is the only reason the table exists.

The test is the row.

Capability says what a claim may do; the next chapter separates the four acts that do it.

41 · Finding, Decision, Action

A work order can move through four acts and one status field. The status says approved, and it does not say who found, who decided or who acted. The four acts are separate, and the record has to keep them separate.

The four acts

Act	What it is	Author	Contract
Evidence	a source span or an observation	source or collector	provenance and span
Finding	a conclusion drawn from evidence	analyst or reviewer	trace to evidence
Decision	a choice by an authority	decision-maker	authority, scope and date
Action	an execution	actor or system	work order and outcome

Evidence is what was observed or reported. A finding is what someone concluded from it. A decision is what an authority chose. An action is what someone did.

The four acts can happen in one hour and in one body. The record still names four authors, because the acts have different warrants and different failure modes.

A finding without evidence is an opinion. A decision without a finding is a preference. An action without a decision is an accident. Each gap is visible only if the four acts are separate.

The four authors

The evidence author is the source or the collector; the source may be a document, a sensor or a person; the collector is the system or the person who recorded it.

The finding author is the analyst or reviewer who drew the conclusion. The finding carries a trace to the evidence and a statement of what the evidence supports.

The decision author is the person or board with authority for the scope. The decision carries the authority, the scope, the date and the conditions.

The action author is the actor or system that executed. The action carries the work order, the date and the outcome.

One person can be all four authors. The record names the person four times, and the acts remain distinct.

The four collapses

Each adjacent pair can collapse, and each collapse is a silent promotion.

Collapse	What is lost
Evidence as finding	the conclusion is stored as the observation
Finding as decision	the analyst's conclusion is stored as the authority's choice
Decision as action	the approval is stored as the execution
Action as evidence	what was done is stored as proof of what was true

Evidence as finding loses uncertainty. A report that says erosion is visible becomes a finding that erosion caused the failure, and the inference disappears.

Finding as decision loses authority. An analyst's conclusion becomes a decision that only a board could make, and the board's scope disappears.

Decision as action loses execution. An approval becomes an action that was never performed, and the work order disappears.

Action as evidence loses independence. A repair becomes proof that the pump failed, and the possibility of a preventive repair disappears.

The pump

Here is the worked case.

```
evidence: report 047, span 12, "erosion visible on the seal"
finding: inspection confirms seal erosion
decision: replace the pump; maintenance board; 2026-03-04
action: pump replaced; crew B; 2026-03-05; work order 881
```

The evidence is the report and its span; the finding is the inspector's conclusion, traced to the span; the decision is the board's choice, with its authority and date; the action is the crew's execution, with its work order and outcome.

The four records are linked. The finding cites the evidence; the decision cites the finding; the action cites the decision. A later reader can walk the chain in either direction.

The chain does not claim that the decision was right or the action succeeded. It claims that each act happened, with an author and a basis.

Decision is not action

A decision can be recorded and never executed. The record has a decision with no action, and the gap is a work item.

An action can be executed without a decision. The record has an action with no decision, and the gap is a governance defect.

A status field that says approved conflates the two. It cannot tell a decision that was executed from a decision that was ignored, and the difference is the work order.

The decision's authority is also not the action's authority; a board can authorize a replacement and a crew can perform it; the board does not perform, and the crew does not authorize.

Action is not evidence

An action is what was done. It is not proof of what was true before the action.

A repair can be preventive. A replacement can be routine. A work order can be opened because a budget was available, not because the evidence supported it.

The action record is evidence of the action, not evidence of the finding. A later reader who treats the work order as proof of failure has reversed the chain.

The same distinction applies to a decision. A decision is evidence that an authority chose, not evidence that the choice was correct.

The table

The four-act table has a row per act, with author, basis, scope and date.

Act	Author	Basis	Scope	Date
Evidence	report 047	span 12	pump PB-100-7	2026-03-02
Finding	inspector	trace to span 12	seal erosion	2026-03-03
Decision	maintenance board	finding	replace pump	2026-03-04
Action	crew B	decision	work order 881	2026-03-05

The table is the desk page. A reader can see the chain and the author of each act.

The table is also the query. A work order with no decision returns `INCOMPLETE_CHAIN`; a decision with no action returns `INCOMPLETE_CHAIN` with the missing act named.

The evidence contract

Evidence has a source, a span, a collector, a time and a custody record; the source is what produced the observation; the span is where it appears; the collector is who recorded it; the time is when; custody is the chain of hands.

A report without a span is a claim about a document rather than a record of it. A report without a collector is a claim without a recorder.

The evidence contract is the first receipt, and it is the one a later reader uses to check whether the observation was read or remembered.

The finding contract

A finding has an analyst, a trace, a scope and an uncertainty. The analyst drew the conclusion; the trace names the evidence; the scope says what the finding covers; the uncertainty says what it does not.

A finding without a trace is an opinion. A finding without a scope is a global conclusion from a local observation.

The uncertainty is not a confidence score. It is a statement of what the finding would need to be stronger, and it is the part that keeps the finding from becoming a decision.

The decision contract

A decision has an authority, a scope, conditions and a date. The authority is who may choose; the scope is what the choice covers; the conditions are what must hold; the date is when.

A decision without an authority is a preference. A decision without a date cannot be reviewed.

The conditions are the part that connects the decision to the finding. A decision that ignores the finding's uncertainty is a decision that has decided more than the finding supports.

The action contract

An action has an actor, a work order, an outcome and a time. The actor performed; the work order is the instruction; the outcome is what happened; the time is when.

An action without a work order is an event. An action without an outcome is a start.

The action contract is the last receipt, and it is the one a later reader uses to see whether the decision was executed or only recorded.

The chain query

The chain query returns four rows and names the missing act. A work order with no decision returns `INCOMPLETE_CHAIN`; a decision with no action returns the same.

The query is the control. It is cheap, it runs at the work order boundary, and it returns the missing act rather than a default.

A chain with four rows is not proof that the decision was right. It is proof that the four acts happened with four authors.

The four acts in one table

The table has four rows and five columns: act, author, basis, scope and date. The evidence row names the report and the span; the finding row names the inspector and the trace; the decision row names the board and the authority; the action row names the crew and the work order.

A table with four rows is a chain; a table with one status is a flag. The flag answers whether the work is approved; the chain answers who found, who decided and who acted.

The table is also the query. A work order with no decision returns `INCOMPLETE_CHAIN`, and the missing act is named in the result.

A table that fills every row with the same author is legal; the acts are still distinct.

The pump's chain

The pump case links four rows. The evidence is report 047, span 12; the finding is the inspector's conclusion, traced to the span; the decision is the board's choice, with its authority and date; the action is the crew's work order, with its outcome.

The links are references, not copies. The finding cites the evidence; the decision cites the finding; the action cites the decision.

A later reader can walk the chain in either direction, and the walk answers who found, who decided and who acted.

The chain does not prove the decision right. It proves the four acts happened with four authors.

The collapse in practice

The four collapses are easy to spot in a status field. Evidence as finding stores the observation as the conclusion; finding as decision stores the analyst as the authority; decision as action stores the approval as the execution; action as evidence stores the repair as the proof.

Each collapse loses a warrant. The record looks complete because it has one status, and it answers fewer questions than a chain with four rows.

A work order that says approved has no finding, no authority and no crew. It is a flag with a date.

The chain's review

The review reads the four rows and the links. It asks whether every act has an author, a basis and a date; whether the links point to the prior act; whether the decision cites the finding; and whether the action cites the decision.

A review that reads only the status cannot answer the four questions.

The review is quarterly, and the missing act is the repair item.

The chain's test

The test removes the finding and runs the decision. The decision must lose its basis; if it still stands, the finding was not load-bearing.

The test removes the decision and runs the action. The action must lose its authority; if it still proceeds, the decision was not load-bearing.

The test removes the evidence and runs the finding. The finding must lose its trace; if it still stands, the finding was an opinion.

The chain's limit

One person can be all four authors, and the chain names the same person four times. The acts remain distinct because the warrants are distinct.

A small team can run the chain in a spreadsheet, and the spreadsheet is a chain if it has four rows.

The chain does not decide whether the decision was right. It records who decided and on what basis, and the judgment remains a separate question.

The chain in practice

The first year is one work order. The team fills four rows, runs the chain query and names the missing act.

The four rows are evidence, finding, decision and action. The evidence row names the report and the span; the finding row names the inspector and the trace; the decision row names the board and the authority; the action row names the crew and the work order.

The chain query returns four rows and a missing act. A work order with no decision returns `INCOMPLETE_CHAIN`; a decision with no action returns the same.

The first correction is a decision receipt. The board records the authority, the scope, the conditions and the date.

The second correction is an action work order. The crew records the actor, the work order, the outcome and the time.

The review reads the four rows and the links. It asks whether every act has an author, a basis and a date, and whether the links point to the prior act.

The cost is four receipts per work order. A status field is cheaper, and it cannot answer who found, who decided or who acted.

The chain does not decide whether the decision was right. It records the four acts, and the judgment remains a separate question.

A chain that fills four rows for one work order is a pattern. A chain that fills one status for a thousand is a flag.

The four acts and the status field

A status field is a flag. It says approved, and it cannot say who found, who decided or who acted.

The four acts are a chain. Each act has an author, a basis, a scope and a date, and the chain is the record of the four.

A status field that covers four acts answers none of them. It collapses the finding into the decision, the decision into the action and the action into a date.

The chain query names the missing act. A work order with no decision returns `INCOMPLETE_CHAIN`; a decision with no action returns the same.

A team can start with one work order and four rows. The first chain is the pattern, and the pattern is what makes the next work order cheap.

The chain is the control, and the status field is the flag. The flag is cheaper, and the chain is the one an auditor can walk.

The chain’s first limit

The chain is not a workflow. A workflow tracks a ticket; the chain tracks four acts with four authors.

The chain’s first correction

The correction is a receipt. The missing act is recorded with its author, basis, scope and date.

A correction that adds a status is not a correction.

The chain’s first review

The review reads the query results and the four receipts. It asks whether the missing act is real or the query is wrong.

The chain is the control.

The concession

Small decisions may merge finding and decision in one person. The inspector who finds the erosion may also be the authority who orders the repair, and the record names both acts with the same author.

The merge is a convenience, not a collapse. The finding keeps its trace; the decision keeps its authority; the person is two authors in one body.

A record that stores one status cannot represent the merge, because it cannot hold the two warrants.

What would refute this

If a single status field can answer who found, who decided and who acted, the four acts are unnecessary. If a status field can name the authority and the work order, the four-act table is ceremony.

A cheaper test: take one work order and remove the finding. If the record still supports the decision, the finding was not load-bearing. If the decision becomes a preference, the finding was.

A second test: take one action and ask whether it proves the finding. If the action is treated as proof, the chain has reversed.

The chain costs four receipts per work order, and the status field costs one. The difference is the record an auditor can walk, and the saving is paid when the missing act cannot be named.

The record is the chain. The chain is the control. The flag is the shortcut. The chain is the record.

The four acts are separate; the next chapter asks what a system must prove.

42 · The Conformance Ladder

A system can claim a rung and fail to produce the artifact that proves it. The ladder is a list of artifacts, not a list of features.

The five rungs

Rung	Name	The question it answers
1	Traceable	can every claim be traced to a span and a chain?
2	Governed	does every standing change have a receipt and an owner?
3	Decision-grade	can a decision cite its finding, authority and capability?
4	Projection-honest	does every export declare what it preserved and lost?
5	Exchanged	can a second party hydrate and audit the claim?

The rungs are ordered because each depends on the one below; a system cannot be decision-grade if its claims are not traceable and governed, because the decision has nothing to cite.

The rungs are not a maturity model. A maturity model ranks a program. The ladder locates the next missing artifact.

The evidence

Each rung has an evidence set.

Rung	Evidence
1	a span for every claim, and a chain from the claim to the span
2	a receipt for every standing change, with actor, date and basis
3	one decision with a finding, an authority and a capability receipt

Continued on next page

(Continued)

Rung	Evidence
4	one export with a loss report and a return path
5	one second party that hydrated and audited the claim

A feature can exist without the evidence; a system can have a span column and no span for half its claims; it can have a receipt table and no receipts; it can have an export and no loss report.

The evidence is the rung. The feature is the possibility of the rung.

Traceable

Traceable means every claim has a span and a chain.

A claim without a span is a statement without a source; a claim with a span but no chain is a source without a route: the reader can see the sentence and cannot see how it became a claim.

The evidence is a query that returns zero claims with no span and zero claims with no chain. A count above zero is the gap.

Traceable is the first rung because every later rung cites the claim. A decision that cites a claim with no span is a decision that cites nothing.

Governed

Governed means every standing change has a receipt and an owner.

A standing change without a receipt is a change nobody can explain. A standing change without an owner is a change nobody can be asked about.

The evidence is a query that returns zero standing changes with no receipt and zero with no owner. The receipt names the actor, the date, the basis and the scope.

Governed is the second rung because a decision needs an authority. A decision that cites a standing change with no owner is a decision that cites a rumor.

Decision-grade

Decision-grade means one decision can cite its finding, its authority and its capability.

The finding is what the evidence supports. The authority is who may decide. The capability is the state the claim reached.

The evidence is one decision record with the three links; a decision that cites the finding and not the authority is a decision without a decision-maker; a decision that cites the authority and not the capability is a decision on an unqualified claim.

Decision-grade is the third rung because it is the first rung that uses the lower two; a system can be traceable and governed and still not be decision-grade, and the gap is the decision record.

Projection-honest

Projection-honest means every export declares what it preserved and lost.

A projection that drops the stance and calls the result a graph is a projection that has lost the claim's standing; a projection with no loss report is a projection that has hidden the loss.

The evidence is one export with a loss report and a return path; the loss report names preserved, generalized, omitted, introduced and unresolved; the return path says how a consumer gets back to the ledger.

Projection-honest is the fourth rung because an exchange depends on it. A second party cannot hydrate a claim if the export does not say what it kept.

Exchanged

Exchanged means a second party can hydrate and audit the claim.

The second party has the message and the dictionary segment, and it can resolve the public terms, read the private terms and check the spans; it can also refuse the claim, and the refusal is a valid outcome.

The evidence is one second party that hydrated the claim and one audit that found the chain. A claim that only the sender can read is not exchanged.

Exchanged is the fifth rung because it is the only rung that tests the boundary. The first four can be true inside one store and fail at the first receiver.

Claimed and proven

A team claims a rung and produces the evidence.

```
claimed: decision-grade (3)
traceable: spans on every claim -> yes
governed: receipts on every standing change -> yes
decision: one decision cites finding, authority, capability -> no
proven: governed (2)
gap: one rung; missing artifact: the decision receipt
```

The score is the highest contiguous rung for which every artifact exists; a skip is a gap, not a score: a system with traceable and decision-grade but no governed is not at rung three.

The gap is the product. It names the missing artifact and the rung it blocks, and it is the only part of the assessment that changes the next week's work.

A gap that names no artifact is a complaint. A gap that names the artifact is a work item.

The score

The score is not an average. An average lets a strong rung hide a weak one, and the weak rung is where the next failure lives.

The score is not a percentage. A percentage implies a continuum, and the rungs are discrete artifacts.

The score is the highest contiguous rung with every artifact. A team at rung two with a decision receipt is at rung three; a team at rung three with no loss report is at rung three and cannot exchange.

The vendor questions

Five questions follow from the rungs.

1. Can you show a span for every claim?
2. Can you show a receipt for every standing change?
3. Can you show one decision with its finding, authority and capability?
4. Can you show one export with a loss report and a return path?
5. Can you show one second party that hydrated and audited the claim?

A vendor that answers four and not five is at projection-honest. A vendor that answers all five has evidence, not a certificate.

The limits

The ladder is designed, not certified. It does not issue a certificate, rank vendors or compare organizations.

The score is self-assessed. An outside reviewer can check the artifacts, and the check is the review. The ladder does not replace it.

The ladder also does not measure whether the decision was right. It measures whether the system can prove the decision's chain, and the two are different questions.

A system can be decision-grade and wrong. The ladder says the chain exists, not that the conclusion is true.

What would refute this

If a system can be decision-grade without a decision record, the rung is unnecessary. If a system can exchange without a loss report, the fourth rung is ceremony.

A cheaper test: take one claim and ask for the five artifacts. If four exist and the fifth is missing, the score is the fourth rung, and the gap is the fifth.

A ladder that always scores five is a ladder that is not being used.

The assessment in practice

A system claims rung four. The assessor asks for one claim, one standing change, one decision, one export and one receiver.

The first claim has a span and a chain, so traceable passes. The span is the sentence in the source; the chain is the route from the sentence to the claim.

The standing change has a receipt, an actor, a date and a scope, so governed passes. The receipt is the record that the change happened and who signed it.

The decision cites the finding, the authority and the capability, so decision-grade passes. The finding is what the evidence supports; the authority is who may decide; the capability is the state the claim reached.

The export has a loss report and no return path, so projection-honest fails. The loss report names what the export dropped; the return path says how a consumer gets back to the ledger.

The receiver can hydrate the claim and not audit it, so exchanged fails. Hydration is reading the claim; audit is walking its chain.

The score is three, because the fourth rung is not proven and the fifth depends on it.

The gap is the return path, and the next artifact is a query that returns the ledger row from the export.

The assessor writes the gap in one sentence and puts it in front of the team that claimed four.

The review is quarterly. A rung can be lost when a new export drops the loss report.

The ladder is not a certification. It is a list of artifacts, and the artifacts are the grade.

A team that closes the gap moves to rung four; a team that closes the fourth and fifth moves to exchanged. The movement is an artifact, not a press release.

The ladder in practice

A team claims rung four and produces one claim, one standing change, one decision, one export and one receiver.

The claim has a span and a chain, so traceable passes. The span is the sentence; the chain is the route from the sentence to the claim.

The standing change has a receipt and an owner, so governed passes. The receipt names the actor, the date and the basis.

The decision cites the finding, the authority and the capability, so decision-grade passes.

The export has a loss report and no return path, so projection-honest fails. The loss report names what was dropped; the return path is missing.

The receiver hydrates the claim and cannot audit it, so exchanged fails.

The score is three, and the gap is the return path. The next artifact is a query that returns the ledger row from the export.

The team writes the gap in one sentence and puts it in front of the person who claimed four.

The review is quarterly, and a rung can be lost when a new export drops the loss report.

The ladder is not a certification. It is a list of artifacts, and the artifacts are the grade.

The five rungs in one page

Traceable means every claim has a span and a chain. The span is the sentence; the chain is the route.

Governed means every standing change has a receipt and an owner. The receipt is the record; the owner is the person.

Decision-grade means one decision cites its finding, its authority and its capability.

Projection-honest means every export declares what it preserved and lost, with a return path.

Exchanged means a second party can hydrate and audit the claim.

The evidence is the rung; the feature is the possibility; the score is the highest contiguous rung.

The evidence table has five rows: rung, query, expected, actual and pass.

Rung	Query	Expected	Actual	Pass
1 Traceable	claims with no span, claims with no chain	0	0	pass
2 Governed	standing changes with no receipt, with no owner	0	0	pass
3 Decision-grade	decisions with a finding, an authority and a capability receipt	1	1	pass
4 Projection-honest	exports with a loss report and a return path	1	loss report yes, return path no	fail
5 Exchanged	receivers that hydrated and audited the claim	1	hydrated yes, audited no	fail

The score is the highest contiguous rung whose row passes: rung three, with the gap at the return path and the next artifact a query that returns the ledger row from the export.

A row with expected zero and actual zero is pass. A row with expected one and actual one is pass. A row with expected zero and actual one is the gap.

The traceable query counts claims with no span and claims with no chain. The governed query counts standing changes with no receipt and no owner.

The decision query counts decisions with a finding, an authority and a capability. The projection query counts exports with a loss report and a return path.

The exchange query counts receivers that hydrated and audited the claim.

The table is the assessment, and the query is the proof.

The score example

A team claims four and proves three. The missing artifact is the return path.

A second team claims three and proves three. The score is three, and the gap is zero.

A third team claims five and proves two. The score is two, and the gap is three artifacts.

The score is the highest contiguous rung with every artifact. A skip is a gap, not a score.

The vendor questions

Five questions follow from the rungs. Can you show a span for every claim? Can you show a receipt for every standing change? Can you show one decision with its finding, authority and capability? Can you show one export with a loss report and a return path? Can you show one second party that hydrated and audited the claim?

A vendor that answers four and not five is at projection-honest. A vendor that answers all five has evidence, not a certificate.

A vendor that answers none has a product, not a conformance.

The limits

The ladder is designed, not certified. It does not issue a certificate, rank vendors or compare organizations.

The score is self-assessed. An outside reviewer can check the artifacts, and the check is the review; the ladder does not replace it.

The ladder does not measure whether the decision was right. It measures whether the system can prove the decision's chain, and the two are different questions.

A system can be decision-grade and wrong. The ladder says the chain exists, not that the conclusion is true.

What would refute this

If a system can be decision-grade without a decision record, the rung is unnecessary. If a system can exchange without a loss report, the fourth rung is ceremony.

A cheaper test: take one claim and ask for the five artifacts. If four exist and the fifth is missing, the score is the fourth rung, and the gap is the fifth.

A ladder that always scores five is a ladder that is not being used.

The ladder's first year

The first year is one claim and five artifacts. The team asks for the span, the receipt, the decision, the loss report and the receiver.

The first score is the highest contiguous rung with every artifact. The first gap is the artifact the team cannot produce.

The first review is a quarter later. The team re-runs the five queries and compares the score.

A rung that was proven and is no longer proven is a regression. A rung that was not proven and is now proven is a repair.

The ladder's first year is not a certification; it is a record of artifacts.

The second year adds a claim and a receiver, and the score moves only when the artifacts move.

A score that moves without an artifact is a slide, and a slide is not an assessment.

The score's rule

The score is the highest contiguous rung with every artifact. A skip is a gap, not a score.

A team at rung two with a decision receipt is at rung three; a team at rung three with no loss report is at rung three and cannot exchange.

The score is not an average; an average lets a strong rung hide a weak one.

The score is not a percentage; a percentage implies a continuum, and the rungs are discrete artifacts.

The gap's product

The gap names the missing artifact and the rung it blocks. A gap that names no artifact is a complaint.

The gap is the only part of the assessment that changes next week's work.

The first review

The first review is a quarter later. The team re-runs the five queries and compares the score.

A rung that was proven and is no longer proven is a regression.

A rung that was not proven and is now proven is a repair.

The ladder says what a system must prove; the next Part asks what meaning is.

43 · Standing for Every Force

A true proposition can be used to make a promise, give an order, ask a question or tell a story; the truth does not warrant the promise, the order, the question or the story.

Truth is one warrant

Assertion has a warrant: truth, supported by evidence. The warrant is defeated by counterevidence, and the defeat is a claim about the world.

A directive has a different warrant: authority. The warrant is defeated by a lack of authority or by an impossible action, and the defeat is a claim about the speaker, not about the world.

A commitment has a third warrant: sincerity and performance. The warrant is defeated by withdrawal or failure, and the defeat is a claim about the speaker's conduct.

A hypothesis has a fourth warrant: testability. The warrant is defeated by a failed test or by inconsistency with evidence, and the defeat is a claim about the test.

The four warrants are not the same, and the same content can carry all four; "The valve is closed" can be asserted, ordered, promised or hypothesized, and each use has its own standing.

The matrix

The warrant matrix has a row for each force.

Force	Warrant	Defeat
Assertion	truth and evidence	counterevidence
Definition	applicability and consistency	a counterexample
Directive	authority and feasibility	no authority, or impossible
Commitment	sincerity and performance	withdrawal or failure
Question	answerability and presupposition	no answer, or false presupposition
Hypothesis	testability and consistency	a failed test
Attribution	source fidelity	a misquote

Continued on next page

(Continued)

Force	Warrant	Defeat
Evaluation	criteria and context	the wrong criteria
Fiction	coherence	internal inconsistency

The matrix is the desk page. A reader can see the warrant and the defeat for each force, and the two are different questions.

The matrix is also the storage requirement. A system that stores one truth value has one column for nine warrants, and the column cannot answer eight of them.

Transfers

A warrant from one force cannot be transferred to another. Four transfers are common.

Transfer	**What it claims**	**What it loses**
truth to directive	the proposition is true, so the order is valid	authority and feasibility
sincerity to assertion	the speaker means it, so it is true	evidence
authority to truth	the board decided, so it is correct	counterevidence
coherence to truth	the story is consistent, so it happened	the world

The first transfer is the most common. A team proves the proposition and treats the proof as permission, and the permission was never granted.

The second transfer is the most sympathetic. A sincere speaker is believed, and belief is not evidence.

The third transfer is the most institutional. A decision is treated as a fact, and the decision was a choice.

The fourth transfer is the most literary. A consistent fiction is treated as history, and the history was never claimed.

The valve

Here is the worked case.

```
assertion: "the valve is closed" warrant: evidence; defeat: counterevidence
directive: "close the valve" warrant: authority; defeat: no authority
commitment: "I will close the valve" warrant: sincerity; defeat: withdrawal
hypothesis: "if closed, pressure falls" warrant: test; defeat: failed test
```

The assertion is true if the valve is closed; the directive is authorized if the board has the authority to order it; the commitment is sincere if the speaker intends to perform; the hypothesis is testable if a pressure reading can be taken.

The four standings are independent; a true assertion does not authorize the directive; an authorized directive does not make the commitment sincere; a sincere commitment does not make the hypothesis testable.

The valve can be closed, the order can be unauthorized, the promise can be insincere and the hypothesis can be untestable, all at once.

The matrix query

The matrix query returns the force, the warrant, the defeat and the evidence.

```
input: claim id
output: force, warrant, defeat, evidence
refuse: NO_WARRANT when the force has no row
```

A claim with no row is a claim whose standing cannot be checked. The refusal is the first gap, and the row is the repair.

The query is cheap and runs at the force boundary. It does not decide whether the warrant holds; it returns the standard the claim is measured against.

The limits

The matrix is a working set, not a complete theory of force. A force can have more than one warrant, and the matrix records the primary one.

The matrix also does not decide whether a warrant holds. It records the standard, and the evidence is a separate question.

The matrix is a storage requirement. A system can store the force and the warrant, and it cannot store one truth value for all nine.

What a warrant is

A warrant is the standard a claim is measured against, not the proof that it passes; the standard names what would count for the claim and what would count against it.

A warrant with no defeat is a dogma.

The warrant is force-specific. The same evidence can warrant an assertion and not a directive, because the assertion is measured against the world and the directive against authority.

A warrant is also not confidence. Confidence is a degree; a warrant is a standard, and the standard is the same at every degree.

The nine forces in practice

Assertion warrants on evidence, and its defeat is counterevidence. “The valve is closed” stands if the inspection supports it.

Definition warrants on applicability, and its defeat is a counterexample. “A valve is a device that controls flow” stands if no counterexample fits.

Directive warrants on authority and feasibility, and its defeat is no authority or an impossible action; “Close the valve” stands if the board may order it and the crew can perform it.

Commitment warrants on sincerity and performance, and its defeat is withdrawal or failure. “I will close it” stands if the speaker intends to perform and does.

Question warrants on answerability and presupposition, and its defeat is no answer or a false presupposition. “Is it closed?” stands if the answer can be given and the valve exists.

Hypothesis warrants on testability and consistency, and its defeat is a failed test. “If it closes, the pressure falls” stands if a pressure reading can test it.

Attribution warrants on source fidelity, and its defeat is a misquote. “She said it was closed” stands if the source says it.

Evaluation warrants on criteria and context, and its defeat is the wrong criteria. “It is a good design” stands if the criteria are named and apply.

Fiction warrants on coherence, and its defeat is internal inconsistency. “The valve spoke” stands if the story permits it.

The transfer errors

Truth to directive is the first error. The proposition is true, so the order is valid. The order needs authority, and truth does not grant it.

Sincerity to assertion is the second. The speaker means it, so it is true. The assertion needs evidence, and sincerity is not evidence.

Authority to truth is the third. The board decided, so it is correct. The decision needs evidence, and authority is not evidence.

Coherence to truth is the fourth. The story is consistent, so it happened. The history needs evidence, and coherence is not evidence.

Each transfer looks like a shortcut and behaves as a category error. The content is the same; the warrant is not.

The matrix in practice

The query returns force, warrant, defeat and evidence. A claim with no row returns `NO_WARRANT`.

A row with no defeat is a dogma; a defeat with no row is a complaint.

The matrix is the desk page, and the query is the control; a system that stores the force and the warrant can check the claim; a system that stores one truth value checks one force and trusts the other eight.

The valve in four forces

The valve case has one content and four standings. The assertion stands on evidence; the directive stands on authority; the commitment stands on sincerity; the hypothesis stands on testability.

The four are independent. A true assertion does not authorize the directive; an authorized directive does not make the commitment sincere; a sincere commitment does not make the hypothesis testable.

The case is the matrix in miniature. The content is one; the warrants are four; the defeats are four.

The matrix in practice

One content can carry nine forces. The matrix is the table that keeps the nine warrants apart, and the table is the artifact.

The assertion is measured against evidence. The evidence is the inspection, the span and the test, and the defeat is the counterevidence that the valve was open.

The directive is measured against authority. The authority is the board's scope, and the defeat is the absence of authority or the impossibility of the action.

The commitment is measured against sincerity and performance. The sincerity is the speaker's intention, and the defeat is the withdrawal or the failure to perform.

The hypothesis is measured against testability. The test is the pressure reading, and the defeat is the failed test.

The definition is measured against applicability. The applicability is the set of cases the definition covers, and the defeat is the counterexample.

The attribution is measured against source fidelity. The source is the quotation, and the defeat is the misquote.

The evaluation is measured against criteria. The criteria are the design goals, and the defeat is the wrong criteria or the wrong context.

The fiction is measured against coherence. The coherence is the story's internal consistency, and the defeat is the contradiction.

The question is measured against answerability. The answerability is the availability of an answer, and the defeat is the false presupposition.

The matrix query returns the force, the warrant, the defeat and the evidence. A claim with no row returns `NO_WARRANT`.

The review reads the rows and the defeats. It asks whether every force has a warrant, every warrant a defeat, and every defeat evidence.

A matrix that is not reviewed is a table with a title. A matrix that is reviewed is a control.

The nine forces, one by one

Assertion stands on evidence. The evidence is the span, the inspection or the test, and the defeat is the counterevidence that contradicts it.

Definition stands on applicability. The definition covers a set of cases, and the defeat is a counterexample that fits the words and not the definition.

Directive stands on authority and feasibility. The authority is the scope of the speaker, and the defeat is the absence of authority or the impossibility of the action.

Commitment stands on sincerity and performance. The sincerity is the speaker's intention, and the defeat is the withdrawal or the failure to perform.

Question stands on answerability and presupposition. The answerability is the availability of an answer, and the defeat is the false presupposition that makes the question unanswerable.

Hypothesis stands on testability and consistency. The test is the observation that would confirm or refute, and the defeat is the failed test.

Attribution stands on source fidelity. The source is the quotation, and the defeat is the misquote or the misattribution.

Evaluation stands on criteria and context. The criteria are the goals, and the defeat is the wrong criteria or the wrong context.

Fiction stands on coherence. The coherence is the internal consistency of the story, and the defeat is the contradiction that breaks it.

The nine warrants are different, and the nine defeats are different. The matrix is the table that keeps them apart.

The objection

The strongest objection is that this multiplies standards until nothing is checkable. One truth value is simple, and nine warrants are nine ways to argue.

The objection is right about simplicity and wrong about checkability. One truth value hides eight warrants, and the hidden warrants are the ones that fail in practice.

A second objection is that truth is still the most important warrant, for assertion it is, and for a directive it is irrelevant; conflating the two is the first transfer error.

The matrix does not abolish truth. It gives truth its proper row.

The warrant matrix in practice

One content can carry four forces in one meeting; the valve is closed, the board orders it closed, the crew promises to close it, and the engineer hypothesizes what happens if it closes.

The assertion is measured against evidence. The evidence is the inspection, and the defeat is the counterevidence that the valve was open.

The commitment is measured against sincerity. The sincerity is the crew's intention, and the defeat is the withdrawal or the failure.

The four rows are the matrix in miniature. The content is one; the warrants are four; the defeats are four.

A team that fills four rows has a pattern. A team that fills one truth value has a flag.

The matrix is the table that keeps the four warrants apart, and the table is the artifact.

What would refute this

If one truth value can answer the warrant question for every force, the matrix is unnecessary, if a directive can be validated by the truth of its content alone, the first transfer is not an error.

A cheaper test: take one content and ask four warrant questions. If the same answer serves all four, the matrix is ceremony.

A second test: take one force and ask for its defeat. If no defeat can be named, the force has no standard.

The matrix in one page

The assertion stands on evidence, and the defeat is counterevidence. The evidence is the span or the test; the counterevidence is the observation that contradicts.

The directive stands on authority, and the defeat is the absence of authority or the impossible action. The authority is the scope; the impossibility is the plan.

The commitment stands on sincerity, and the defeat is the withdrawal or the failure. The sincerity is the intention; the failure is the performance.

The hypothesis stands on testability, and the defeat is the failed test. The test is the observation; the failure is the result.

The review reads the rows and the evidence. It asks whether every force has a warrant, every warrant a defeat, and every defeat a test.

The matrix is a working set, not a complete theory. A force can have more than one warrant, and the matrix records the primary one.

The matrix's cost

The matrix costs a row per force and a review per row. The row names the standard; the review reads the evidence.

A force with no row is a claim with no standard. A row with no defeat is a dogma.

The cost is paid per force, and the matrix is the budget. A team that fills one row has one force; a team that fills nine has nine.

The matrix is the artifact.

The matrix's test

The test takes one content and four forces. It asks for the warrant and the defeat for each.

If the same answer serves four forces, the matrix is ceremony. A matrix that passes is a control.

The matrix's review

The review reads the rows and the evidence. The repair is a row or a defeat.

The first matrix

One content, four forces. Four rows, one query, one gap.

The matrix is the control.

The matrix is the control. The row is the unit. The warrant is the standard. The defeat is the test. The force is the key.

Standing is force-specific; the next Part asks what meaning is.

44 · A Language for Permission

A prose policy can say what should happen. It cannot compile. The gap between the policy and the system is where the failures live, and the gap is not closed by better prose.

The wound

A policy is a document, and a document is interpreted by whoever reads it; the reader may be a person, a model or a program, and each reads a different policy from the same sentences.

A rule that cannot be parsed cannot be enforced; a rule that can be parsed but not refused is a suggestion; a rule that can be refused but not attributed is a rumor.

The wound is not that prose is imprecise. The wound is that prose has no closure: there is no point at which the policy has said everything it can say, and no point at which the system can say an action is outside it.

Failure by closure

A specification fails by closure when its execution can do something the specification has no vocabulary to forbid.

A complex system fails by closure: the failure class is the set of behaviors the specification language cannot name. The failures are not bugs in the implementation; they are the closure of a specification weaker than its execution.

The same pattern appears in a prose policy; the policy names the actions it thought of, and the system performs an action it did not name; the policy cannot forbid the action because it has no word for it.

A formal language changes the failure mode; it can refuse an action, and the refusal is a named outcome; the failure class is still non-empty, and it is visible: the language returns `UNKNOWN` where it has no rule.

The prompt shares the substrate

A prompt is not a governance document. It is an instruction issued to the system it is trying to govern, interpreted by the same mechanism that generates the unconstrained output.

The prompt and the governed output share a substrate; a user can shift the probabilities with a framing, and the shift is not a parse error; it is a different reading of the same instruction.

A better prompt is still prose on the same substrate. It improves the tendency to comply and does not create an external, machine-checkable rule.

A tendency is not a rule. The rule has to be outside the governed system, typed, and checked before the action.

Hohfeld's first and second order

Permission has two orders, and collapsing them loses the difference between what may be done and who may change it.

Order	Relation	What it means
First	privilege / no-right	the actor may do the action; another has no right to stop it
First	claim / duty	the actor may require the action; another has a duty
Second	power / liability	the actor may change a normative position
Second	immunity / disability	the actor is protected from another's power

A privilege is not a power. The privilege to read a file is not the power to change who else may read it. A governance system that stores one `allowed` flag cannot tell the two apart.

The second order is where revocation, delegation and amendment live; a system with only first-order permissions can grant and cannot revoke; it can allow and cannot change the rule that allowed.

Hohfeld's table is the minimal vocabulary, not a complete legal theory. The claim is that a permission language needs both orders, not that four relations exhaust the law.

Two profiles and UNKNOWN

A single profile cannot be both expressive and decidable. The language needs two.

Profile	What it permits	Guarantee
Strict	boolean expressions, comparisons, membership, boolean functions	statically decidable
Extended	the pseudocode meta-language inside a policy slot	bounded by a resource bound; UNKNOWN on overflow

Strict is the default. A module without a profile directive is Strict, and a policy that needs a loop declares Extended and accepts the bound.

`UNKNOWN` is a verdict, not an error. It says the system cannot decide within its resources, and the caller must handle it. A system that maps `UNKNOWN` to `DENY` has made a policy choice; it should say so.

`UNKNOWN` is also the coverage boundary made machine-readable. A prose policy fails silently outside its cases; a formal language returns a named third value.

The grant

A permission is a claim with four parts: a grantor, a scope, a window and a defeat condition.

```
permit(
 grantor = controller,
 actor = rao,
 action = approve,
 scope = order_value <= 500,
 window = 2026-04-01 .. 2026-06-30
)
```

The grantor is who granted. The scope is what the permission covers. The window is when it is in force. The defeat condition is what ends it: revocation, expiry, or a conflicting forbid.

A permission without a grantor is a rumor. A permission without a scope is a blank check. A permission without a window cannot expire, and a permission without a defeat cannot be revoked.

The grant is a claim in the ledger, not a row in a policy file; it has a receipt, a version and an author, and the same machinery that stores the claim stores the permission.

What the language does not decide

The language does not decide whether a policy is just, whether a grantor had authority, or whether the action is wise.

It decides whether the action is permitted, denied or unknown under the stated grant, and it returns the grant that decided. The judgment remains a separate question.

Omega is one instance of the language, not the definition. The requirements are vendor-neutral: closure, decidability, attribution, revocation, composition. A different implementation that meets them is a different instance.

The five requirements

Closed: a policy that can invent an action cannot be audited. The vocabulary of actions has to be finite, and a new action is a new rule rather than a new token.

Decidable: a regulator cannot wait for an answer. The policy has to return `ALLOW`, `DENY` or `UNKNOWN` in bounded time, and the bound has to be declared.

Attributable: a permission with no author is a rumor. Every grant names a grantor, a scope and a date.

Revocable: the system has to explain what it used to allow. A revocation is a claim, not a delete, and the prior permission remains readable.

Composable by intersection: a grant of "up to 500" and a grant of "up to 100" compose to 100, not 300. Averaging constraints silently repeals one of them.

The five are the requirement set. A vendor that meets four and misses one has a product with a named gap.

The closure table

A policy names the actions it thought of; the system performs an action the policy did not name, and the policy cannot forbid it because it has no word for it.

The prose policy fails silently. The formal language returns `UNKNOWN`, and the third value is the coverage boundary made machine-readable.

The failure class is the gap between what the specification can say and what the execution can do. A wider vocabulary narrows the gap; it does not close it.

A policy that claims to cover everything is lying. A policy that names its coverage is a control.

The Hohfeld example

The privilege to read a file is not the power to change who else may read it.

Relation	Example
privilege / no-right	Rao may read the file
claim / duty	Rao may require the auditor to disclose

Continued on next page

(Continued)

Relation	Example
power / liability	Rao may revoke another's access
immunity / disability	Rao is protected from another's revocation

A system with one `allowed` flag can store the first row and cannot store the third. The flag says yes, and the power says who may change the yes.

The second order is where revocation, delegation and amendment live. A language with only first-order permissions can grant and cannot govern the grant.

The profiles in practice

Strict admits boolean expressions, comparisons, set membership and boolean functions. It is statically decidable, and it is the default when no profile directive appears.

Extended admits the pseudocode meta-language inside a policy slot. It is bounded by a declared resource bound, and it returns `UNKNOWN` when the bound is exceeded.

`UNKNOWN` is a verdict, not an error. A caller that maps it to `DENY` has made a policy choice, and the choice should be recorded.

The profile is declared at module level. A module without a profile is Strict, and a policy that needs a loop declares Extended and accepts the bound.

The grant lifecycle

A grant has a grantor, an actor, an action, a scope and a window.

The grant is issued as a claim. The claim has a receipt, a version and an author, and the same ledger that stores the claim stores the permission.

The grant is revoked by a new claim. The old grant remains readable, with its window and its grantor, and the revocation adds a defeat condition.

The grant expires when the window closes. The expiry is a status, not a deletion, and a query for what was allowed on a date reads the window.

A conflict between a permit and a forbid is a finding, not a majority vote. The forbid governs within its scope, and the pair is reported.

The permission language in practice

The five requirements have a failure each. Closed fails when the system can invent an action; decidable fails when the verdict does not return; attributable fails when the grant has no grantor; revocable fails when the revocation deletes; composable fails when two grants average.

The closure case is a policy that names approve, deny and refund. The system performs adjust, and no rule names adjust. The prose policy has no word for the action; the formal language returns `UNKNOWN`.

The prompt case is a prompt that says do not discuss competitors. A user frames a fictional competitor, and the model discusses it. The prompt is inside the governed system; a rule outside the system can refuse the action.

The Hohfeld case is a privilege to read a file and a power to change who else may read it. A system with one allowed flag stores the privilege and cannot store the power.

The profile case is a module with no profile directive. The module is Strict, and a policy that needs a loop declares Extended and accepts the bound.

The `UNKNOWN` case is an Extended policy that exceeds its resource bound. The result is `UNKNOWN`, not an error, and the caller decides whether to deny, retry or ask a human.

The grant case is a permit with a grantor, an actor, an action, a scope and a window. The grant is a claim with a receipt, and the ledger stores it beside the claims it governs.

The revocation case is a new claim that ends the grant. The old grant remains readable, and the revocation adds a defeat condition.

The conflict case is a permit and a forbid that cover the same action and scope. The forbid governs, and the pair is reported as a finding.

The test case removes the grantor and asks again. The system must refuse the grant as incomplete; a system that still allows has no attribution.

The limits case is a policy that is just, wise or legitimate. The language does not decide those questions; it decides permission, denial and unknown under the stated grant.

The hand-off is the next question: what meaning crosses the boundary when the permission is exchanged.

The permission ledger

A grant is a row with a grantor, an actor, an action, a scope, a window, a defeat condition and a receipt.

The receipt names the policy, the authority, the date and the version. A grant without a receipt is a permission without a warrant.

A revocation is a new row that points to the grant. The grant remains readable, with its window and its grantor, and the revocation adds a defeat condition.

An expiry is a status computed from the window. The grant does not need a job to expire; the query reads the window and returns `EXPIRED`.

A conflict is a finding, not a majority vote. A permit and a forbid that cover the same action and scope produce a conflict row, and the forbid governs within its scope.

The query returns `ALLOW`, `DENY`, `UNKNOWN`, `CONFLICT` or `EXPIRED`, and it names the grant that decided.

The review reads the rows and the conflicts. It asks whether every grant has a grantor, a scope and a window, and whether every conflict has a ruling.

The ledger is the control. A prose policy can say what should happen; the ledger can say what may happen, and refuse the rest.

The permission language's test

The test is four questions, and each question removes one support.

Remove the grantor and ask. The system must refuse the grant as incomplete, because a permission without an author cannot be attributed.

Remove the scope and ask. The system must refuse the grant, because a permission without a scope is a blank check.

Add a conflicting forbid and ask. The system must return a conflict, because a permit and a forbid on the same scope cannot both govern.

Set the window to the past and ask. The system must return `EXPIRED`, because the grant is no longer in force.

Add an Extended policy that exceeds its bound and ask. The system must return `UNKNOWN`, because the bound is the coverage boundary.

Remove every obstacle and ask. The system must return `ALLOW` with the grant that decided.

A language that passes the six questions has closure, attribution, a defeat condition and a third value. A language that fails one has a policy.

The test is the procurement page. A vendor that answers five and not six has a named gap, and the gap is the next question.

The first grant

The first grant is one actor, one action, one scope and one window. The team writes the grant, issues it and runs the query.

The query returns `ALLOW` and names the grant. The grant is a row with a receipt, and the row is the control.

A first grant with four fields is a pattern; a grant with one flag is a permission without a warrant.

The first grant is the template.

The grant's receipt

The receipt names the policy, the authority, the date and the version.

A grant without a receipt is a permission without a warrant.

The grant's query

The query returns `ALLOW`, `DENY`, `UNKNOWN`, `CONFLICT` or `EXPIRED`.

A query that returns only allowed has hidden four states.

The grant's review

The review reads the rows and the conflicts. The repair is a receipt or a ruling.

The test

The test is a policy that can refuse.

Take one action and one grant. Ask the system to permit, deny or return `UNKNOWN`. If the system can only permit, it is a suggestion engine.

Remove the grantor and ask again. The system must refuse the grant as incomplete.

Add a conflicting forbid and ask. The system must return a conflict, not a majority vote.

Set the window to the past and ask. The system must return `expired`, not `allowed`.

A language that passes the four questions has closure, attribution and a defeat condition. A language that fails one has a policy.

The grant's review is the control. The row is the unit. The receipt is the warrant. The query is the boundary. The conflict is the finding. The grant is the claim. The ledger is the record. The permission is the claim. The test is the grant.

A prose policy can say what should happen; a permission language can say what may happen, and refuse the rest.

PART VIII
The Shape

what meaning is, that it can be entitled.

45 · Meaning Is Relational

A node is a name. The name can be unique, typed and resolved, and it still says nothing about what the thing means. The meaning is in what the node does.

The node alone

A node can carry an identifier, a label, a type and a set of attributes. None of those is a meaning.

`customer` is a label. `retail` is a type. `active` is an attribute. The three can be true of a registrant and of a buyer, and the two are different.

The node is an index. It gives the predications a place to point, and it does not tell the reader what the predications say.

A node with no predications is a name with no use. The name can be resolved, joined and counted, and it cannot be interpreted.

Two customers

Customer A ordered, paid, complained and churned. Customer B registered and never ordered.

```
customer A: ordered, paid, complained, churned
customer B: registered, never ordered
label: both "customer"
pattern: A has four predications; B has one
meaning: A is a customer who transacted; B is a registrant
```

The label is the same. The pattern is different. The meaning is the pattern.

An attribute can distinguish the two after the fact: `customer_type = buyer` for A and `registrant` for B; the attribute is a summary of the pattern, and the summary is not the pattern.

A system that stores the attribute and not the predications has the label and not the meaning.

The pattern

A pattern is the set of predications a node participates in, with the role it plays in each.

```
A: AGENT of order 44021; THEME of payment 991; AGENT of complaint 12; THEME of churn 7
B: AGENT of registration 88
```

The pattern is read by role and by time. The role says what the node did; the time says when it did it.

Two nodes can share a label and differ in pattern; two nodes can share a pattern and differ in label; the pattern is the meaning; the label is the index.

A pattern query returns the predications, not the attributes. A node with no predications returns `NO_PATTERN`, and the refusal is the first gap.

Entity-first

An entity-first store begins with nodes. It creates the customer, the order and the payment, and then it adds edges between them.

The order is a name before it is a meaning; the customer is a name before it is a buyer; the edges arrive after the nodes, and the node is already the unit.

The store can answer “which customers are active?” by reading the attribute; it cannot answer “what did this customer do?” without the predications, and the predications were the thing it deferred.

The failure is not that the store is useless. It is that the meaning is late, and the late meaning is a second system beside the first.

Predication-first

A predication-first store begins with the predications. It records the order, the payment, the complaint and the churn, and it indexes the participants.

The node is created when a participant is referenced. The node is an index into the predications, and the predications carry the meaning.

The store can answer “what did this customer do?” directly. It can answer “which customers are active?” by a pattern query over the predications, and the pattern is the answer.

The node is still necessary. It is the handle that joins the predications to a participant, and it is not the meaning.

The unit

The unit of the graph is the predication, not the node.

A graph of nodes with attributes is a catalog. A graph of predications is a record of what was said and done.

The catalog can be large and the record can be small. The record is the one that answers a question about meaning.

The node is a name; the predication is a meaning; the store that begins with the predication begins with the meaning, and the store that begins with the node begins after it.

The relational account

A node is a name, and a name is an index. The index is necessary for pointing and insufficient for meaning.

An attribute is a summary of a pattern. `active` can mean ordered recently or logged in recently, and the attribute does not say which.

A pattern is the set of predications a node participates in, with the role it plays in each. The role is what the node did; the time is when.

Customer A ordered, paid, complained and churned. Customer B registered. The label is the same; the pattern is not.

The pattern is read by role and time. A node can be the AGENT of an order and the THEME of a payment, and the two roles are different meanings.

A pattern is not a list of edges. An edge without a role says two nodes are connected, and it does not say how.

An entity-first store creates the nodes and adds the edges later. The node is the unit before the meaning arrives.

The store can answer “which customers are active?” from the attribute, and it cannot answer “what did this customer do?” without the predications.

The meaning is late, and the late meaning is a second system beside the first. The first system has names; the second has records.

A predication-first store creates the predications and indexes the participants. The node is created when a participant is referenced.

The store answers “what did this customer do?” directly. It answers “which customers are active?” by a pattern query over the predications.

The pattern query returns the predications by role and time. A node with no predications returns `NO_PATTERN`.

The refusal is the first gap. The relation row is the repair.

The objection is that nodes are natural. A graph is nodes and edges, and an edge needs two nodes.

The objection is right about mechanics and wrong about the unit; the node is necessary to point at a participant, and it is not sufficient to say what the participant means.

Attributes are indexes. A type and an attribute are useful for finding a node; they are not the pattern.

The test is one node and one question, if the answer is a list of attributes, the store is entity-first; if the answer is a list of predications, the store is predication-first.

The unit is the predication. The node is the name, and the predication is the meaning.

The pattern query

The pattern query returns a row per predication: the predication, the role the node plays, the time, and the other participant.

A node with no rows returns `NO_PATTERN`. The refusal is the first gap, and the relation row is the repair.

The query is by node. A query by attribute is a different query, and it returns a list of nodes rather than a pattern.

The query is also a test of the store. A store that can answer the attribute query and cannot answer the pattern query is entity-first.

The entity-first case

An entity-first store creates the customer, the order and the payment, and then adds the edges.

The customer has a type and an attribute. The type says retail; the attribute says active. Neither says what the customer did.

The meaning is late, and the late meaning is a second system beside the first. The first has names; the second has records.

The predication-first case

A predication-first store creates the order, the payment, the complaint and the churn, and indexes the participants.

The node is created when a participant is referenced. The node is the handle, and the predication is the meaning.

The node is still necessary. It joins the predications to a participant, and it is not the meaning.

The unit test

The test is one node and one question.

If the answer is a list of attributes, the store is entity-first.

The test does not say which store is better for every task. It says which store holds the meaning.

A store can be both: a predication-first ledger with an attribute index. The index is a projection, and the ledger is the record.

The pattern in practice

A node can be AGENT in one predication and THEME in another. The role is part of the meaning, and the same node can carry two roles.

Customer A is the AGENT of the order and the THEME of the payment; the two roles are different, and a store that keeps only the node has lost the difference.

The order precedes the payment, and the churn follows the complaint. The time is part of the pattern, and a pattern without time cannot tell a sequence from a coincidence.

The pattern includes the other participant. The customer ordered from a supplier, and the payment went to a vendor. The counterpart is part of the meaning.

Two customers can share a label and differ in pattern. A is a buyer; B is a registrant. The label is the same; the pattern is not.

An entity-first store has the two labels and not the two patterns. It can count the customers and cannot say what they did.

A predication-first store has the two patterns and the labels. It can count the customers and say what they did.

The pattern query returns a row per predication: the predication, the role, the time and the other participant.

The unit test is one node and one question. If the answer is attributes, the store is entity-first; if predications, predication-first.

A store can be both: a predication ledger with an attribute index. The index is a projection; the ledger is the record.

The relational store

A relational store for meaning has two tables: the node index and the predication record.

The node index has the identifier, the label and the type. It is the handle, and it is not the meaning.

The predication record has the predicate, the role bindings, the time and the span. It is the meaning, and it is the row a pattern query reads.

A role binding names the node and the role. The role is the meaning of the node in that predication.

The time is the version of the pattern. A pattern without time cannot tell the order from the payment.

The counterpart is the other participant. The pattern has at least two nodes, and the relation is between them.

The pattern query joins the node index to the predication record by the role binding. The join is the meaning query.

The index makes the query cheap. A store without the index is a scan; a store with the index is a lookup.

The review reads the patterns and the gaps. A node with no pattern is a name; a pattern with no role is an edge.

The store is relational because the meaning is relational. The node is the name, and the predication is the relation.

The meaning test

The test takes one node and asks three questions: which roles the node plays, which times the roles carry, and which counterparts the predications name.

A node with one role and no time has a label. A node with roles and times has a pattern.

The roles are the first question. The customer is AGENT in the order and THEME in the payment, and the two roles are different meanings.

The times are the second question. The order precedes the payment, and the churn follows the complaint; a pattern without times cannot tell a sequence from a coincidence.

The counterparts are the third question. The customer ordered from a supplier and paid a vendor, and the two counterparts are different relations.

A store that cannot answer the three questions is entity-first, whatever its graph looks like.

The refusal is the first gap. A node with no pattern returns `NO_PATTERN`, and the repair is a predication.

The pattern is the meaning, and the three questions are the test.

The pattern's first year

The first node is a customer and one predication: ordered. The pattern is one role and one time.

The second predication is the payment. The pattern is two roles and two times, and the customer is now a buyer.

The third predication is the complaint, and the fourth is the churn. The pattern is four predications, and the label has not changed.

The pattern query returns the four rows. A node with no rows returns `NO_PATTERN`.

The review reads the rows and the gaps. The repair is a predication, and the pattern grows.

The first year is one node and four predications. The label is the same; the meaning is the pattern.

The objection

The strongest objection is that nodes are natural. A graph is nodes and edges, and an edge needs two nodes; the node is the first thing you have.

The objection is right about the mechanics and wrong about the unit; the node is necessary to point at a participant, and it is not sufficient to say what the participant means.

A second objection is that attributes are meanings: an attribute can summarize a pattern, and the summary is useful for indexing; it is not the pattern.

The concession is real: a node with a type and an attribute is a useful index; it is not a meaning, and a store that treats it as one has confused the index with the record.

What would refute this

If two nodes with identical patterns can have different meanings, the pattern is not the meaning, if a node with no predications can answer a meaning question, the predication is not necessary.

A cheaper test: take one node and ask what it means. If the answer is a list of attributes, the store is entity-first; if the answer is a list of predications, the store is predication-first.

A pattern query that returns attributes has answered a different question.

A node is a name; the next chapter asks what meaning must carry across a boundary.

46 · The Conservation Law

A graph can be copied across a boundary and lose its standing in the copy; the copy has the same nodes and the wrong meaning, because nothing in it says what the nodes may be used for.

The boundary

A boundary is a place where the meaning changes hands: an export, a message, a mapping, a projection.

The copy is not the meaning. The copy is a set of rows; the meaning is the rows plus what they are entitled to do.

A crossing is the act of carrying meaning across the boundary. It needs four components, and a missing component has a named disease.

The four components

Component	What it is	What it carries
Pivot	a finite shared set of categories	the vocabulary both sides map into
Bridge	a directional mapping with a contract	the correspondence and the residual
Policy	a task scope and an authority	what the receiver may use the mapping for
Proof	a trace, a residual and a version	the evidence that the crossing happened

The pivot makes the match finite. The bridge makes it directional. The policy makes it permitted. The proof makes it auditable.

The four are not stages. A crossing can have a bridge and no policy, and it is a mapping used where it was never granted.

The pivot

The pivot is the finite middle. Without it, the crossing is a pairwise match, and a pairwise match cannot report its coverage.

The two sides map into the pivot, and the comparison happens once per category. The pivot is small enough to own and complete enough to count.

A pivot that is not shared is a private vocabulary with a public name. The receiver maps into a different set, and the two mappings do not meet.

The bridge

The bridge is the mapping with a direction and a contract; it says A may be used as B for task T, under assumptions H, after transformation F, with residual R.

The bridge is not an equivalence arrow. The arrow says the two are the same; the bridge says the two may be used together under conditions.

A bridge without a direction hides which loss the receiver inherits. A bridge without a version cannot be replayed.

The policy

The policy is the task scope and the authority to use the mapping. It says who may use the bridge, for what, and under which grant.

A bridge without a policy is a mapping that anyone may use for anything. The receiver uses it outside its scope, and the error looks like a data problem.

The policy is a grant, and the grant has a grantor, a scope and a window. The grant is a claim in the ledger, not a note in a README.

The proof

The proof is the trace, the residual and the version. It says which categories were used, what was preserved and what was lost.

A crossing without a proof is an auditable claim with no audit. The receiver can use the mapping and cannot show why it was allowed.

The proof is also the return path; it lets a consumer get from the export back to the ledger row, and the return path is what makes the copy a view rather than a replacement.

The diseases

Each missing component has a disease.

Missing	Disease
Pivot	pairwise drift: no coverage, no finite middle
Bridge	false equivalence: an arrow with no trace
Policy	unauthorized use: a mapping used outside its task
Proof	unauditable claim: a crossing with no return path

The diseases are not failures of the data. They are failures of the crossing, and each one is named.

A crossing can have two diseases at once. A pairwise match with no proof is drift plus an unauditable claim, and the two are worse together.

The Customer case

Two departments both say `Customer`. The crossing has four components.

```
pivot: shared 12-category middle
bridge: compliance Customer -> retail Customer contract
policy: churn report may use it; regulatory count may not
proof: trace, residual 12%, version
```

The pivot makes the categories finite. The bridge makes the mapping conditional. The policy limits the use to the churn report. The proof records the residual.

Remove the pivot and the match is two thousand pairs; remove the bridge and the arrow is an equivalence; remove the policy and the regulatory count uses the churn mapping; remove the proof and no one can show which categories were used.

Minimum sufficient

Not every crossing needs all four at full strength. A low-stakes export can declare a gap instead of a full proof, and the gap is the honest minimum.

The minimum-sufficient rule says the crossing needs the component the receiver's task depends on; a search index may need the pivot and not the policy; a regulatory count needs the policy and the proof.

A crossing that overpays is a crossing that will not be repeated. A crossing that underpays is a crossing that will be discovered.

The crossing in practice

The pivot is the twelve categories both sides map into; the source maps fifty terms into the twelve, and the target maps forty, and the coverage is counted in the middle.

The pivot makes the match finite. A source term that fits two categories is ambiguous; a term that fits none is unmatched.

The coverage is matched, unmatched and ambiguous. A crossing that reports only matched terms has hidden the other two states.

The bridge is the contract; it says compliance Customer may be used as retail Customer for the churn report, under the KYC assumption, after the date transformation, with a residual of twelve percent.

The bridge has a direction. Compliance to retail loses the order history; retail to compliance loses the KYC condition.

The bridge has a version. A contract checked against release twelve is not the same claim as one checked against release thirteen.

The policy is the grant. It says the churn team may use the mapping, and the regulatory team may not.

The policy has a scope. The churn report is the scope; the regulatory count is outside it.

The policy has a window. The grant is in force until the quarter closes, and the window is recorded.

The proof is the trace. It names the categories, the roles, the residual and the version.

The proof has a return path. The consumer can get from the export back to the ledger row, and the return path is what makes the export a view.

The proof has a loss report. The report names preserved, generalized, omitted, introduced and unresolved.

The four components are used together. The pivot bounds the match, the bridge defines it, the policy permits it, and the proof records it.

A crossing with all four is a translation. A crossing with three is a copy with a named disease.

The conservation law in practice

A boundary is an export, a message or a projection. The crossing is the act of carrying meaning across it.

The pivot is the finite set both sides map into. Without it, the match is pairwise and the coverage is unknown.

The pivot is small enough to own. A twelve-category middle is a page; a thousand-category middle is a vocabulary project.

The coverage is matched, unmatched and ambiguous. The three states are the report, and a percentage alone hides the ambiguous cases.

The bridge is the contract. It names the direction, the assumptions, the transformation and the residual.

The bridge has a version. A contract against release twelve is a different claim from a contract against release thirteen.

The policy is the grant. It names the task, the actor and the window.

The policy is not a note in a README. It is a claim with a grantor, a scope and a date.

The proof is the trace. It names the categories, the roles and the residual.

The proof has a return path. The consumer can walk from the export back to the ledger row.

The proof has a loss report. The report names what the crossing preserved, generalized, omitted, introduced and left unresolved.

The audit reads the four. A crossing with a missing component is an incomplete crossing, and the missing component is the disease.

The four are the conservation law. Meaning crosses only with a pivot, a bridge, a policy and a proof.

The boundary case

The export is one graph with three customer rows and one mapping contract. The receiver is the churn team.

The pivot is the twelve-category middle. The three rows map into customer, order and payment, and the mapping is counted.

The bridge is the contract. It says compliance Customer may be used as retail Customer for the churn report, under the KYC assumption, with a twelve-percent residual.

The policy is the grant. It names the churn team, the churn report and the quarter.

Remove the pivot and the match is pairwise; remove the bridge and the arrow is an equivalence; remove the policy and the regulatory team uses the churn mapping; remove the proof and no one can show which categories were used.

Each removal is a disease. Pairwise drift, false equivalence, unauthorized use and an unauditable claim.

The repair is the missing component. A pivot row, a bridge contract, a policy grant or a proof trace.

The audit reads the four and returns `INCOMPLETE_CROSSING` when one is missing. The missing component is named.

The export with all four is a translation. The export with three is a copy with a disease.

The conservation law is the four. Meaning crosses only with a pivot, a bridge, a policy and a proof.

The conservation law's cases

The Customer crossing has four components; the pivot is the twelve categories, the bridge is the conditional contract, the policy is the churn grant, and the proof is the trace and residual.

Remove the pivot and the match is fifty by forty. The crossing has no coverage and no finite middle, and the disease is pairwise drift.

Remove the bridge and the crossing is an equivalence arrow. The arrow says the two are the same, and the disease is false equivalence.

Remove the policy and the regulatory team uses the churn mapping. The mapping is used outside its task, and the disease is unauthorized use.

Remove the proof and the receiver cannot show which categories were used. The crossing is an unauditable claim.

A copy preserves the rows and loses the standing. A translation preserves the rows and the standing, and the four are the difference.

The conservation law's test

The test takes one export and asks four questions: what is the pivot, what is the bridge, what is the policy, and what is the proof.

The pivot question asks for the finite set both sides map into. A crossing with no pivot is a pairwise match, and the match cannot report its coverage.

The bridge question asks for the contract: the direction, the assumptions, the transformation and the residual. A crossing with no bridge is an equivalence arrow, and the arrow has no trace.

The policy question asks for the grant: the task, the actor and the window. A crossing with no policy is a mapping used where it was never granted.

The proof question asks for the trace, the residual and the return path. A crossing with no proof is an auditable claim with no audit.

A crossing that answers four questions is a translation. A crossing that answers three is a copy with a named disease.

The test is cheap, and the missing component is the repair. A crossing that fails the test returns `INCOMPLETE_CROSSING`.

The test is the control, and the four questions are the conservation law.

The conservation law in one page

The four components are the pivot, the bridge, the policy and the proof; the pivot makes the match finite, the bridge makes it conditional, the policy makes it permitted, and the proof makes it auditable.

The four diseases are pairwise drift, false equivalence, unauthorized use and the unauditable claim. Each is named, and each is the repair target.

The minimum-sufficient rule chooses the components the task needs. A search index needs the pivot and the bridge; a regulatory count needs the policy and the proof.

The test takes one export and asks four questions. A crossing with all four is a translation; a crossing with three is a copy with a named disease.

The law is the four. Meaning crosses only with a pivot, a bridge, a policy and a proof.

The conservation law's record

The record of a crossing has four rows: the pivot, the bridge, the policy and the proof.

The pivot row names the finite set both sides map into, and the coverage counts the matched, unmatched and ambiguous terms.

The bridge row names the direction, the assumptions, the transformation and the residual.

The policy row names the task, the actor and the window, and the grant is a claim with a receipt.

The proof row names the trace, the residual and the return path, and the loss report names what was preserved and lost.

A crossing with four rows is a translation. A crossing with three is a copy with a named disease.

The record is the ledger, and the export is the view. The view can be rebuilt from the record; the record cannot be rebuilt from the view.

The law is the four, and the four are the record.

The objection

The strongest objection is that every export becomes a project. Four components per boundary, a contract, a policy and a proof is more than most teams can afford.

The objection is right about the cost and wrong about the alternative. A copy without the four is cheaper and loses standing, and the loss is discovered at the first audit.

The four are a menu, not a mandate. The minimum-sufficient rule chooses the components the task needs, and the declared gap is the honest cheap option.

What would refute this

If a copy can preserve standing without a pivot, a bridge, a policy and a proof, the law is unnecessary. If a crossing with three components is auditable, the fourth is ceremony.

A cheaper test: take one export and ask for the four. If the receiver can use the data and cannot say what it may do, the policy is missing.

A crossing with a declared gap is honest. A crossing with an undeclared gap is a copy.

The record is the control. The four rows are the law. The export is the view. The crossing is the translation. The copy is the disease. The pivot is the set. The bridge is the contract. The policy is the grant. The proof is the trace. The four are the record. The law is the four.

Meaning crosses a boundary only with a receipt; the next chapter asks what the receipt must contain.

47 · The Chain of Degradation

A pipeline can lose the meaning of a sentence in five steps, and each step can be defensible; the loss is not in one link, since it is in the chain.

The chain

The maintenance chain has five links.

Link	Input	Output	What it drops
Source	report 047	“erosion visible on the seal”	nothing
Extraction	the sentence	seal eroded	the hedge “visible”
Resolution	seal eroded	pump PB-100-7, Bay 14	the unit identity
Projection	the claim	pump failed	the stance and the hedge
Dashboard	the claim	failure count + 1	the distinction between visible and confirmed
Decision	the count	replace the Bay 14 pump	the source and the finding

The table is the chain. Each link takes an input, produces an output and drops something.

Each link reasonable

The extractor drops “visible” because the schema has no field for a hedge, and the extraction is reasonable: the extractor records what it can.

The resolver picks the Bay 14 pump because the report names PB-100-7 and the Bay 14 pump is the only PB-100-7 in the index; the resolution is reasonable: it is the best available match.

The projection drops the stance because the dashboard schema has no stance column, and the projection is reasonable: the dashboard is built for counts.

The dashboard counts the failure because it counts events. The count is reasonable: it is what dashboards do.

The decision replaces the pump because the count says failure. The decision is reasonable: the count is the evidence the decision has.

No link is absurd. Each is a local optimization, and the chain is a global loss.

Cumulative loss

The baseline is the source span and its stance: “erosion visible on the seal,” attributed to report 047, with the hedge intact. The final artifact is a failure count, and the count has no hedge, no unit identity, no stance and no source.

The loss is cumulative. The extraction loses the hedge; the resolution loses the unit; the projection loses the stance; the dashboard loses the distinction; the decision loses the source.

A single link cannot be blamed, because no single link produces the final loss. The chain produces it.

The baseline

The baseline is the source span, the stance, the unit and the hedge, before the first transformation.

Without a baseline, the final artifact looks correct, because the count is a number and a number has no visible loss; with a baseline, the loss is a list (hedge, unit, stance, distinction, source), and the list is the audit.

The baseline is not the source document. It is the meaning the document carried, recorded before the first link.

The repair

The repair is a named loss at each link, not a single fix at the end.

Link	Repair
Extraction	a hedge field or a stance-preserving phrase
Resolution	a unit check against the report
Projection	a stance column or a declared loss
Dashboard	a confirmed/visible split
Decision	a citation to the source and the finding

Each repair is local. The chain is repaired link by link, and the final artifact is the last repair.

A repair at the dashboard alone leaves the extraction, resolution and projection losses in place; the count is still wrong, and it is now a more confident wrong count.

Declared loss

Some loss is acceptable and declared. A dashboard that counts failures and does not carry the hedge is a dashboard with a declared loss.

The declared loss is not degradation, because degradation is an undeclared loss and the declaration is what separates the two; a pipeline with five declared losses is auditable, and a pipeline with one undeclared loss is a chain.

The chain in practice

The source is report 047, span 12: “erosion visible on the seal.” The sentence carries a hedge, a unit and a stance.

The extraction turns the sentence into a claim: `seal eroded`. The hedge is gone, because the schema has no field for it.

The hedge is the difference between “visible” and “confirmed.” The extractor records what it can, and the difference is a loss.

The resolution maps the seal to pump PB-100-7 and the pump to Bay 14. The unit identity is a guess from the index.

The index has one PB-100-7, and the Bay 14 pump is the only match; the resolution is reasonable and wrong if the report meant another unit.

The projection turns the claim into `pump failed`; the stance and the hedge are gone, because the dashboard schema has no columns for them.

The projection is reasonable, since the dashboard is built for counts and the count is the output.

The dashboard increments the failure count. The count is the evidence the decision has.

The decision replaces the Bay 14 pump. The decision is reasonable: the count says failure, and the crew acts on the count.

Each link is locally reasonable. The chain is a global loss.

The baseline is the source span and its stance: the hedge, the unit and the source attribution, before the first transformation.

The cumulative loss is the difference between the baseline and the final artifact: hedge, unit, stance, distinction and source.

The repair is a named loss at each link: a hedge field, a unit check, a stance column, a confirmed/visible split and a citation.

The declared loss is the loss the pipeline admits. The undeclared loss is the degradation.

The audit reads the five links and the baseline. It returns the loss list, and the repair is the list.

The chain is the unit of audit. The final artifact is the last link, and the chain is the record.

The baseline and the loss list

The baseline is the source span and its stance, recorded before the first transformation, and it has five items: the hedge, the unit, the stance, the distinction and the source.

The hedge is "visible." It says the erosion was seen, not confirmed.

The unit is pump PB-100-7 in Bay 14. It says which pump the report means.

The stance is "reported by the inspector." It says the claim is testimony, not fact.

The distinction is visible versus confirmed. It says the count cannot merge the two.

The source is report 047, span 12. It says where the sentence came from.

The final artifact is a failure count. It has none of the five.

The loss list is the difference: hedge, unit, stance, distinction, source. Each item is a named loss, and each is a link.

The declared loss is the item the pipeline admits in its loss report. The undeclared loss is the item it does not.

A pipeline with five declared losses is auditable. A pipeline with one undeclared loss is a chain.

The repair is a named loss at each link; a hedge field repairs the extraction; a unit check repairs the resolution; a stance column repairs the projection; a confirmed/visible split repairs the dashboard; a citation repairs the decision.

The repair table has five rows, and each row is local. A repair at the dashboard alone leaves four losses in place.

The review reads the five links and the baseline. It returns the loss list, and the list is the work.

The audit is the chain. The final artifact is the last link, and the chain is the record.

The links, one by one

The source is report 047, span 12, with a hedge, a unit and a stance; it is the baseline, and it is the only link that carries all three.

The extraction turns the sentence into `seal eroded`; the hedge is gone, because the schema has no field for it; the extraction is reasonable: it records what the schema can hold.

The resolution maps the seal to pump PB-100-7 and the pump to Bay 14; the unit identity is a guess from the index, and the guess is reasonable because the index has one match.

The projection turns the claim into `pump failed`; the stance and the hedge are gone, because the dashboard schema has no columns for them; the projection is reasonable, since the dashboard is built for counts.

The dashboard increments the failure count. The count is the evidence the decision has, and the count is reasonable because it is what dashboards do.

The repair is local. A repair at the dashboard alone leaves four losses in place, and the count is still wrong.

The audit reads the five links and the baseline. It returns the loss list, and the list is the work.

The chain is the unit. The final artifact is the last link, and the chain is the record.

The chain's audit

The baseline table has five items: hedge, unit, stance, distinction and source.

The final artifact has none of the five. The loss list is the difference, and each item is a link.

The declared loss is the item the pipeline admits. The undeclared loss is the item it does not.

The repair table has five rows: a hedge field, a unit check, a stance column, a confirmed/visible split and a citation.

Each repair is local. A repair at the dashboard alone leaves four losses in place.

The test takes one final artifact and asks for the baseline. If the baseline cannot be produced, the chain is not auditable.

The audit is the chain, and the chain is the loss list.

The chain's first audit

The first audit takes one final artifact: the failure count for Bay 14.

The baseline is report 047, span 12, with the hedge, the unit, the stance, the distinction and the source.

The first link is the extraction. It drops the hedge, and the repair is a hedge field.

The second link is the resolution. It picks the Bay 14 pump, and the repair is a unit check against the report.

The third link is the projection. It drops the stance, and the repair is a stance column or a declared loss.

The fourth link is the dashboard. It drops the visible/confirmed distinction, and the repair is a split.

The fifth link is the decision. It drops the source, and the repair is a citation.

The loss list is the five items. The audit returns the list, and the list is the work.

The chain's record

Link	Input	Output	Loss
Source	report 047	"erosion visible"	none
Extraction	the sentence	seal eroded	hedge

Continued on next page

(Continued)

Link	Input	Output	Loss
Resolution	the seal	Bay 14 pump	unit
Projection	the claim	pump failed	stance
Dashboard	the claim	count + 1	distinction
Decision	the count	replace pump	source

The table is the chain. Each row is a link, and each loss is the item the link drops.

The source row is the baseline. It carries the hedge, the unit, the stance, the distinction and the source.

The extraction row drops the hedge. The resolution row drops the unit. The projection row drops the stance.

The dashboard row drops the distinction. The decision row drops the source.

The five losses are the chain. The final artifact has none of them, and the table has all five.

The table is the audit, and the audit is the chain.

The chain's limit

Not every loss is a defect. A transformation that drops a field and declares the drop is a design choice.

The declared loss is in the loss report. The undeclared loss is the finding.

A chain with five declared losses is auditable. A chain with one undeclared loss is a degradation.

The review reads the table and the baseline. It asks which losses are declared and which are not.

The test takes one artifact and asks for the baseline. If the baseline cannot be produced, the chain is not auditable.

The test then asks for the five rows. A missing row is a missing loss, and the missing loss is the repair.

The chain's receipts

The receipts are the chain, and the chain is the audit; a receipt per link has an input, an output, a loss and a version, and the version is what lets a later reader replay the link.

A chain with five receipts is auditable, and a chain with one receipt is a claim.

The receipts are the chain, and the chain is the loss list. The link is the unit, and the record is the evidence.

The chain's cost

Five receipts per artifact is the cost, and the review is the price. A declared loss is a design choice, and an undeclared loss is a finding.

The review reads the table and the baseline. It asks which losses are declared and which are not, and the undeclared loss is the work.

The chain is the record, and the record is the audit.

The chain's end

The chain ends at the decision, and the decision is the last link. The record is the five receipts, and the receipts are the losses.

The chain is the unit of audit and the audit is the loss list, so a chain with five receipts is auditable, and a chain with one receipt is a claim.

The list is the work, and the work is the repair; the chain is the record, and the record is the evidence. The decision is the last link, and the source is the first; the chain is the loss list, and the loss list is the work.

The objection

The strongest objection is that this makes every pipeline guilty. Every transformation drops something, and the chain of degradation is just the normal cost of processing.

The objection is right about the cost and wrong about the guilt. The cost is normal; the undeclared loss is the defect.

A declared loss is a design choice. An undeclared loss is a finding.

A second objection is that the final artifact is what matters. The final artifact is the last link, and the audit needs the chain.

What would refute this

If a chain has one catastrophic link and the other links preserve meaning, the chain is not the unit; the link is.

If a final artifact can be audited without the baseline, the baseline is unnecessary.

A cheaper test: take one final artifact and ask for the baseline. If the baseline cannot be produced, the chain is not auditable.

The chain is the unit. The record is the audit. The chain is the record.

The chain is the unit of audit; the next chapter asks what the receipt must contain.

48 · Representation Prices Questions

A representation is a price list for questions. It makes some questions cheap and others impossible, and the impossibility is the part that architecture reviews usually miss.

The price list

A representation is a way of storing claims that determines which questions can be asked cheaply, which can be asked at a cost, and which cannot be asked at all.

The price is not only time. A question that takes an hour is expensive; a question that cannot be expressed is impossible, and no amount of tuning makes it cheap.

A relational store prices a join as a native operation. A document store prices reading a whole document. A columnar store prices an aggregation.

Each representation is excellent at its question and poor at the others. The choice of representation is the choice of which questions the system will answer well.

Four representations

Representation	Join	Path	Full text	Aggregate
Relational	cheap	expensive	impossible	cheap
Graph	expensive	cheap	impossible	expensive
Document	impossible	expensive	cheap	impossible
Columnar	expensive	impossible	impossible	cheap

The table is not a ranking. It is a price list, and the prices are different for each question.

A relational store can join customer, order and payment in one statement. It can traverse a ten-hop path with a recursive query, and the query is expensive and awkward.

A graph store can traverse the ten-hop path in one traversal; it can join two entities by a path, and the join is the traversal, and the traversal is priced as a path.

A document store can read the complaint letter as a whole; it cannot join the letter to the order without an external index, and the external index is a second representation.

A columnar store can count churn by segment across a million rows. It cannot reconstruct one customer's history without reading many columns, and the reconstruction is the expensive question.

The contract

A question contract declares the price for each representation.

Price	Meaning
Cheap	a native operation, answered in one query
Expensive	possible with a transformation or a scan
Impossible	not expressible without a second representation

The contract has three rows per representation, and the impossible row is the one that matters; a representation with no impossible question is a representation that has not been honest about its limits.

The contract is a design artifact. A team that writes it before the database choice has a portfolio; a team that writes it after has a justification.

The customer case

The customer 360 question asks for the customer, the orders, the payments and the complaint in one view.

The relational store answers with a join across four tables. The join is cheap, and the view is one query.

The graph store answers with a traversal from the customer to the orders, the payments and the complaint. The traversal is cheap, and the view is one traversal.

The document store reads the complaint letter and cannot join it to the order. The letter is cheap; the join is impossible without the order store.

The columnar store counts the churn by segment and cannot reconstruct the customer's history. The count is cheap; the history is expensive.

The four stores answer four different questions about the same customer. None answers all four.

The portfolio

Architecture is a portfolio of representations.

The ledger is the canonical representation: predications, roles and receipts. It answers the meaning questions and pays for the join.

The graph is a projection: nodes and edges for traversal. It answers the path questions and pays for the aggregation.

The columnar store is a read model: counts and segments. It answers the aggregate questions and pays for the reconstruction.

The document store is a source view: letters and reports. It answers the whole-document questions and pays for the join.

The portfolio is not free. Every projection has a consistency cost, and the cost is the lag between the ledger and the view.

The portfolio is a choice about which questions the organization will ask. A single representation is a choice to answer one question well and the others badly.

The concession

Multiple representations cost consistency. A ledger and a graph can disagree, and the disagreement is a projection defect.

The cost is real, and the alternative is a single representation that prices every other question as impossible.

The portfolio is worth it when the questions are plural; a system with one question can use one representation; a system with a portfolio needs the projections and the loss reports.

The four price lists

The relational price list is a join. Customer, order and payment are three tables, and the join is one statement with a native plan; the path is a recursive query, and the recursion is expensive because the engine was built for sets rather than walks.

The graph price list is a traversal. The path from customer to churn is one traversal, and the traversal is cheap because the engine was built for walks. The join is a path between two nodes, and the path is priced as a traversal even when the question is a set question.

The document price list is a whole document. The complaint letter is one read, and the read is cheap because the engine was built for documents; the join to the order is impossible without an external index, and the index is a second representation with its own price list.

The columnar price list is an aggregation. The churn count by segment is one scan of one column, and the scan is cheap because the engine was built for columns; the

reconstruction of one customer's history is expensive because the history is spread across many columns.

The price is structural. A join is cheap in a relational engine because the join is the native operation; a traversal is cheap in a graph engine because the traversal is the native operation. The native operation is the price list's first row.

The table is not a ranking. Each representation is excellent at its question and poor at the others, and the choice is the choice of which questions the system will answer well.

The question portfolio

A question portfolio is the list of questions the organization needs answered, with the frequency and the consequence of each.

The customer 360 question asks for the customer, the orders, the payments and the complaint in one view. The relational store answers it with a join; the graph store answers it with a traversal; the document store answers the letter and not the join.

The churn path question asks which sequence of events preceded the churn. The graph store answers it with a traversal; the relational store answers it with a recursive query and a cost.

The letter question asks what the complaint said. The document store answers it with a read; the relational store answers it with a text index and a second representation.

The segment count question asks how many customers churned by segment. The columnar store answers it with a scan; the graph store answers it with a traversal and a count, and the count is expensive.

The portfolio is the set of questions, and the representation is chosen per question. A single representation is a portfolio with one entry and many impossible questions.

The contract in practice

The question contract has three rows per representation: cheap, expensive and impossible.

The cheap row names the native operation. The expensive row names the transformation or the scan. The impossible row names the question that requires a second representation.

A contract with no impossible row returns `NO_LIMIT`, and the refusal is the first gap. A representation that claims to answer every question has not been asked a hard question.

The contract is written before the database choice. A team that writes it after the choice has a justification, and a team that writes it before has a portfolio.

The review reads the contract and the questions. It asks whether the cheap row is native, the expensive row is bounded and the impossible row is declared.

The cost of the portfolio

Every projection has a consistency cost. The ledger and the graph can disagree, and the disagreement is a projection defect rather than a data error.

The cost is the lag between the ledger and the view, and the lag is the price of answering the path question cheaply.

The loss report declares what the projection dropped. The report is the receipt, and the receipt is what makes the portfolio auditable.

The reconciliation reads the lag and the loss report. It asks which questions the view can answer and which it cannot, and the answer is the contract.

The price list in practice

The join question asks for the customer, the order and the payment in one row. The relational store answers it cheaply, the graph store answers it with a path, the document store cannot answer it without an index, and the columnar store answers it with a reconstruction.

The path question asks which sequence of events preceded the churn. The graph store answers it cheaply, the relational store answers it with a recursive query, the document store cannot answer it, and the columnar store cannot answer it.

The letter question asks what the complaint said. The document store answers it cheaply, the relational store answers it with a text index, the graph store answers it with a property, and the columnar store cannot answer it.

The count question asks how many customers churned by segment. The columnar store answers it cheaply, the relational store answers it with an index, the graph store answers it with a traversal and a count, and the document store cannot answer it.

The four questions have four different cheap representations. The portfolio is the set of questions, and the representation is chosen per question.

The portfolio's ledger

The ledger is the canonical representation. It stores the predications, the roles and the receipts, and it answers the meaning questions.

The graph is a projection of the ledger. It stores the nodes and the edges, and it answers the path questions.

The columnar store is a read model of the ledger. It stores the counts and the segments, and it answers the aggregate questions.

The document store is a source view. It stores the letters and the reports, and it answers the whole-document questions.

The ledger is the record, and the projections are the views. A view can be rebuilt from the record; the record cannot be rebuilt from the view.

The contract's review

A contract with no impossible row is a contract that has not been tested. A contract with an impossible row is a portfolio.

The review is quarterly, and the questions change. A new question is a new row, and the row is the price.

The question ledger

The question ledger lists every question the organization asks, with the representation that answers it cheaply.

The join question is answered by the relational store.

The cheap row is the native operation. The expensive row is the scan or the transformation. The impossible row is the second representation.

A question with no cheap representation is a question the portfolio cannot answer. The gap is the next representation, or the question is dropped.

The ledger is the portfolio's index. A reader can see which question is answered where, and which question has no representation.

The ledger is also the design input. A team that starts with the ledger chooses representations per question; a team that starts with the database chooses questions per representation.

The four representations answer four questions. The ledger is the five-column table, and the table is the portfolio.

The question ledger is the contract's companion. The contract prices one representation; the ledger prices the portfolio.

The portfolio's first year

The first year is one question and one representation. The team writes the question, the cheap row and the impossible row.

The second question adds a second representation. The ledger and the graph answer the join and the path, and the lag is the consistency cost.

The first lag is the difference between the ledger and the graph. The reconciliation reads the lag and the loss report.

The first contract has three rows. The impossible row is the question the portfolio cannot answer, and the gap is the next representation.

The first year is one question and one price. The portfolio grows by question, not by database.

The question ledger is the record, and the record is the portfolio.

The portfolio's limit

The portfolio is not free. Every projection has a lag, and the lag is the consistency cost.

A single representation is cheaper and answers one question. A portfolio answers several and pays the lag.

The choice is the question portfolio, not the fashion. The portfolio is a price list, and the price list is the design.

The portfolio's test is one question and one cheap row. If no representation has one, the question is dropped or a new representation is added.

The portfolio's record is the question ledger.

The objection

The strongest objection is that a good engineer can make any question fast. Performance is a tuning problem, and the representation is an implementation detail.

The objection is right about performance and wrong about expressibility. A question that cannot be expressed is not a tuning problem, and no index makes a document store join.

A second objection is that polyglot persistence is complexity. It is, and the complexity is the price of the portfolio; the alternative is a single price list with more impossible rows.

What would refute this

If one representation answers every question cheaply, the portfolio is unnecessary. If an impossible question becomes cheap with tuning, the price is only performance.

A cheaper test: take one question and ask for its price in each representation. If the price is the same, the representation is neutral.

A representation that has no impossible question has not been asked a hard question.

The record is the question ledger, and the ledger names the question, the representation and the price. The price is cheap, expensive or impossible, and the impossible row is the gap. The record is the design, and the design is the portfolio. The question is the unit.

The price is the record. The record is the price. The portfolio is the record.

A representation prices questions; the next chapter asks who pays.

49 · The Grounding Problem

A dictionary defines a word with more words. The chain can continue indefinitely, and at no point does a definition leave the language. The question is where the chain stops.

The regress

A definition relates a symbol to other symbols; `Pump` is defined as a device that moves fluid; `fluid` is defined as a substance that flows; `substance` is defined as a kind of matter.

Each definition is another symbol relation. The chain can continue, and the continuation is the regress.

The regress is not a defect of dictionaries. It is what a definition is: a relation among symbols, with no exit to the world.

A symbol can be defined without being grounded. The definition can be precise, consistent and complete, and the symbol can still have no connection to anything a community does.

Symbols point to symbols

A mapping is also a symbol relation. The mapping says `Customer` in one vocabulary corresponds to `Customer` in another, and the correspondence is between symbols.

An embedding is also a symbol relation. The vector is a position among vectors, and the position is computed from symbols.

A model's output is also a symbol relation. The model produces tokens, and the tokens are symbols.

None of these is an exit. Each is a relation among symbols, and the regress continues.

The exit is not another symbol. The exit is a practice.

The commitment

Grounding is a commitment. An agent binds a symbol to a distinction, with evidence and consequence.

The commitment has four parts: the symbol, the distinction, the evidence and the consequence.

The symbol is the word or the code. The distinction is what the community treats as different. The evidence is what supports the distinction. The consequence is what happens when the symbol is used wrongly.

A technician grounds `pump` by pointing at the pump, testing it, repairing it and correcting a misidentification. The pointing is not a definition; it is a practice with a consequence.

A customer grounds `customer` by ordering, paying, complaining and churning. The pattern is the distinction; the consequence is the account, the refund or the churn.

Evidence and consequence

Evidence without consequence is a description.

The commitment needs both. The evidence says why the distinction is drawn; the consequence says what the community does when the distinction is used.

A symbol with evidence and no consequence is a label.

The record names the evidence and the consequence, and the two are the warrant for the commitment.

The practice

The regress stops at a practice, not at a primitive symbol.

A practice is a set of actions a community performs with a symbol, with correction when the symbol is misapplied; the practice is where the symbol meets the world, and the meeting is the consequence.

The practice is not infallible. A community can be wrong, and the commitment can be revised.

The practice is also local. A community of technicians grounds `pump` in one way; a community of buyers grounds it in another. The two commitments can differ and both be grounded in their practices.

Revisability

Grounding is revisable. New evidence can defeat the commitment, and the record is revised.

A pump that is reclassified, a customer who is redefined, a distinction that is retired: each is a revision of the commitment.

The revision is a claim. It has a reviser, a date, a reason and a new evidence set.

A commitment that no evidence can defeat is not a grounding; it is a dogma.

The record

A grounding record has five fields: symbol, commitment, evidence, consequence and reviser.

```
symbol: pump
commitment: this device moves fluid in this system
evidence: inspection, test, repair history
consequence: misidentification triggers a work-order correction
reviser: maintenance council, 2026-04-12
```

The record is a ledger entry. It has a warrant, a version and an author.

A record with no consequence returns `NO_GROUND`. The refusal is the first gap, and the consequence is the repair.

The pump's grounding

The definition of `pump` is a chain of symbols; a device moves fluid; a fluid is a substance that flows; a substance is a kind of matter; the chain continues without an exit.

The technician's grounding is not the definition; the technician points at the pump, reads the nameplate, tests the flow and repairs the seal, and each action is a distinction with a consequence.

The evidence is the nameplate, the flow reading and the repair history. The consequence is the work order when the pump is misidentified, and the correction is the practice.

The commitment is local. The maintenance community grounds `pump` as the device in Bay 14 with the PB-100-7 nameplate, and another community can ground it differently.

The commitment is revisable. A reclassification, a replacement or a corrected nameplate is a revision, and the revision has a reviser and a date.

The definition did not ground the symbol. The practice did, and the practice is the record.

The customer's grounding

The definition of `customer` is also a chain. A party that buys; buys is an exchange; an exchange is a transfer; the chain continues.

The customer's grounding is the pattern. The customer ordered, paid, complained and churned, and the four predications are the distinction.

The evidence is the order, the payment and the complaint; the consequence is the account, the refund and the churn, and each is what the community does when the symbol is applied.

A registrant who never ordered is not a customer under this commitment. The label is the same; the pattern is not.

The commitment is revisable. A new definition of customer is a new commitment, with a new evidence set and a new consequence.

The pattern grounds the symbol. The definition only describes it.

The three failures

The first failure is the infinite regress. Every definition points to another definition, and the chain never reaches a practice.

The second failure is the closed circle. A community defines a symbol by its own usage, and the usage is the only evidence; the circle has no consequence outside itself.

The third failure is the symbol without a consequence. A label is applied, and nothing happens when it is applied wrongly; the symbol is defined and not grounded.

The three failures are common because each looks like grounding. The regress looks like precision, the circle looks like consensus, and the label looks like a category.

The test is the consequence. A symbol with no consequence is a definition, and a definition is not a ground.

The grounding ledger

The grounding ledger has a row per symbol: the commitment, the evidence, the consequence, the reviser and the version.

```
symbol: pump
commitment: this device moves fluid in this system
evidence: nameplate, flow test, repair history
consequence: misidentification triggers a work-order correction
reviser: maintenance council, 2026-04-12
version: 3
```

The row is a claim with a warrant. The commitment is the claim; the evidence and the consequence are the warrant; the reviser is the author.

A row with no consequence returns `NO_GROUND`. The refusal is the first gap, and the consequence is the repair.

The version is the revision history. A commitment that changes without a version cannot be compared across reports.

The ledger is the grounding record, and the record is the practice.

The grounding record in practice

The record starts with the symbol. The symbol is the word or the code, and it is the thing the community uses.

The record continues with the definition. The definition is the symbol chain, and it is the description rather than the ground.

The record continues with the practice. The practice is the action the community performs with the symbol, and it is the ground.

The record continues with the evidence. The evidence is the nameplate, the test, the repair or the order, and it is what supports the distinction.

The record continues with the consequence. The consequence is what happens when the symbol is misapplied, and it is the correction.

The record continues with the reviser. The reviser is the person or council that can change the commitment, and the revision is the claim.

The record continues with the version. The version is the revision history, and it is what lets a consumer compare the commitment across reports.

A record with no reviser returns `NO_REVISER`. The refusal is the second gap, and the reviser is the repair.

The test takes one symbol and asks three questions: what consequence, who revises and what evidence. A symbol with three answers is grounded; a symbol with two is defined.

The customer record has the pattern as the evidence and the account as the consequence; the pump record has the test as the evidence and the work order as the consequence.

The record is the practice, and the practice is the ground.

The grounding ledger

The grounding ledger has one row per symbol, and the row is the commitment.

The symbol is the key. The commitment is the distinction. The evidence is the support. The consequence is the correction. The reviser is the authority. The version is the history.

The row is a claim with a warrant. The claim is the commitment; the warrant is the evidence and the consequence; the author is the reviser.

A row with no consequence returns `NO_GROUND`.

The three refusals are the three failures. The regress has no consequence; the circle has no reviser; the label has no evidence.

The ledger is the record of the practice, and the practice is the ground.

The ledger is also the revision history. A new row supersedes the old row, and the old row remains readable.

The review reads the rows and the refusals. It asks which symbols have a consequence and which have only a definition.

The review is quarterly, and the repair is a consequence or a reviser.

The grounding case

The pump is defined as a device that moves fluid, and the definition is a chain of symbols that can continue without an exit.

The technician grounds the pump by pointing at it, reading the nameplate and testing the flow, and the pointing is an action rather than a definition.

The test is the evidence, and the evidence is the flow reading that distinguishes this pump from another device in the same bay.

The consequence is the work order that corrects a misidentification, and the correction is what the community does when the symbol is applied wrongly.

The customer is defined as a party that buys, and the definition is another chain of symbols.

The customer is grounded by the pattern: ordered, paid, complained and churned, and the pattern is the distinction.

The consequence is the account, the refund and the churn, and each is the correction when the symbol is applied.

The record names the symbol, the commitment, the evidence, the consequence and the reviser, and the record is the ground.

The practice is the ground, and the ground is the record.

The grounding ledger's review

The review reads the rows and the refusals. A row with no consequence is a definition, and a row with no reviser is a dogma.

The review asks which symbols have a consequence and which have only a definition. The answer is the grounding list, and the list is the work.

The repair is a consequence or a reviser. A consequence is an action with a correction, and a reviser is a person or council with a date.

The review is quarterly, and the version is the record. A commitment that changes without a version cannot be compared across reports.

The record is the practice, and the practice is the ground. The grounding is a commitment, and the commitment costs.

The grounding's record

The record has a symbol, a commitment, an evidence set, a consequence and a reviser, and the five fields are the whole of the ground. The symbol is the key, the commitment is the distinction, the evidence is the support, the consequence is the correction, and the reviser is the authority. A record with no consequence is a definition, and a record with no reviser is a dogma. The record is the practice, and the practice is the ground. The grounding is a commitment, and the commitment costs.

The grounding's test

The test asks for the consequence and the reviser. A symbol with both is grounded; with one is defined; with none is a label. The refusal is the gap, and the gap is the repair.

The grounding's cost

A grounding costs a practice, and the practice costs time, correction and review; the cost is paid by the community, and the community is named; a cheap grounding is a label, and a label is not a ground.

The objection

The strongest objection is that this is circular. The commitment is made by a community, and the community's practice is the ground; the circle seems to close on itself.

The circle is the symbol chain, and the practice breaks it. The practice is not another symbol; it is an action with a consequence.

A second objection is that a model grounds better than a community; a model produces another symbol relation, and the relation is useful; the commitment and the consequence remain the ground.

A third objection is that grounding is a view from nowhere. It is not; it is a view from a practice, and the practice is named.

What would refute this

If a definition can terminate in the world without a practice, the commitment is unnecessary. If a symbol can be grounded without a consequence, the consequence is decoration.

A cheaper test: take one symbol and ask for its consequence. If no consequence can be named, the symbol is defined and not grounded.

A second test: ask who can revise the commitment. If no one can, the commitment is a dogma.

The grounding ends at the practice, and the practice is the record, and the record is the commitment, and the commitment is the cost. The cost is the correction, and the correction is the consequence, and the consequence is the practice, and the practice is the ground. The ground is the record, and the record is the grounding, and the grounding is a commitment, and the commitment costs; the grounding is local, and the local practice is the ground.

The grounding is the record, and the record is the practice; the practice is the commitment, and the commitment is the cost; the cost is the correction, and the correction is the consequence. The consequence is the ground, and the ground is the record. The grounding is the record.

The record is the ground. The ground is the record.
Grounding is a commitment; the next chapter asks what a commitment costs.

PART IX
The Negative Space

what a semantics forbids.

50 · A Semantics Is Its Prohibitions

A language that can say only what follows is half a language. The other half is what does not follow, and the refusal is what makes the first half safe.

The positive-only language

A class hierarchy says what follows from a type. A stove is an appliance; a pump is a device; a customer is a party.

The positive inferences are cheap. A reasoner can close the hierarchy, and the closure is the set of valid conclusions.

The positive-only language has no vocabulary for the invalid conclusion. It can say that every stove is an appliance, and it cannot say that a sibling is not a housemate.

A language with no prohibitions over-infers. The extra inference is not a bug in the reasoner; it is the absence of a refusal.

The non-inference table

A non-inference table has a premise, a forbidden conclusion and a reason.

Premise	Forbidden conclusion	Reason
A sibling_of B	A lives_with B	kin relation, not household
A sibling_of B	A shares_surname B	kin relation, not naming
A sibling_of B	A has_parent X = B has_parent X	sibling, not shared parent
REPORT(A, P)	P	report is an act, not a truth
customer(A)	payer(A)	customer is a pattern, not a payment
part_of(A, B)	owned_by(A, B)	composition, not ownership

The table is the language's negative half. Each row names a derivation the representation must refuse.

The table is also the test. An inference that matches a forbidden row returns `FORBIDDEN_INFERENCE`, and the refusal is the proof that the language has a negative half.

Sibling

A sibling relation is a kin relation. It is not a household relation, a naming relation or a shared-parent relation.

The graph can store `sibling_of(A, B)`. A model can infer `lives_with(A, B)` because siblings often live together, and the inference is plausible and invalid.

The table forbids the three conclusions. The refusal is not a claim that siblings never live together; it is a claim that the relation does not entail it.

A system that forbids the inference can still record a household when the evidence supports it. The record is a separate claim, and the claim is the evidence.

Report

A report is an act. `REPORT(A, P)` says that A reported P, and it does not say that P is true.

The table forbids `P` from `REPORT(A, P)`. The refusal is the difference between testimony and fact.

A system that lacks the row will promote the report to a fact, and the promotion is invisible because the graph has one edge.

The row is the same rule as the commitment grid, expressed as a non-inference. The grid forbids the combination; the table forbids the derivation.

Over-inference

Over-inference is the materialization of a conclusion the representation did not license.

A graph with no prohibitions can materialize `lives_with` from `sibling_of`, and a dashboard can count households from the materialized edge; the count is wrong, and the error is invisible because the edge looks like the others.

The damage is not the inference itself. It is the downstream use of the inference as if it were evidence.

A non-inference table stops the materialization at the boundary. The refusal is cheap, and the downstream count is correct by construction.

The measure

A semantics is measured by its refusals, not by its expressiveness.

A language that can express a thousand valid inferences and no invalid one is a language that will over-infer; a language that expresses a hundred valid inferences and forbids the ten dangerous ones is a language with a boundary.

The measure is the non-inference table. A table with no forbidden row is a language with no negative half.

The measure is also a review question: which derivations does the representation refuse? A team that cannot answer has a schema, not a semantics.

The non-inference table in practice

A positive-only language can close a hierarchy and cannot refuse a derivation, and the closure is the set of valid conclusions with no negative half.

The sibling premise is a kin relation, and the first forbidden conclusion is `lives_with`. The reason is that siblings can live apart, and the relation does not carry a household.

The second forbidden conclusion is `shares_surname`. The reason is that siblings can have different names, and the relation does not carry a naming convention.

The third forbidden conclusion is the shared parent. The reason is that siblings can be half-siblings or adoptive, and the relation does not carry a parent identity.

The report premise is an act, and the forbidden conclusion is `P`. The reason is that a report can be false, and the act does not carry the truth.

The promotion is the materialization of `P` from `REPORT(A, P)`, and the promotion is invisible because the graph has one edge.

The customer premise is a pattern, and the forbidden conclusion is `payer`. The reason is that a customer can order without paying, and the pattern does not carry the payment.

The part_of premise is a composition, and the forbidden conclusion is `owned_by`; the reason is that a part can be owned by a different party, and the composition does not carry ownership.

Each row is a derivation the representation must refuse, and each refusal is a boundary.

The refusal runs at the inference step. The check reads the premise and the table, and a match returns `FORBIDDEN_INFERENCE`.

The boundary is cheap. The table is small, and the check is one lookup.

The cost is expressiveness. A prohibition removes a conclusion the language could otherwise draw, and the removal is the price of the boundary.

The table is a working set, and the incomplete row is the next finding.

The measure is the refusals. A language with a table has a negative half, and a language without one is a schema.

The semantics of refusals

The positive half of a language is the set of inferences it licenses, and the negative half is the set it forbids; a language with one half is a schema, and a language with both is a semantics.

The non-inference table is the negative half. Each row is a premise, a forbidden conclusion and a reason.

The sibling row forbids the household, the surname and the shared parent. The reason is that the kin relation does not carry the household relation.

The report row forbids the truth of the content. The reason is that the act of reporting does not carry the truth of what is reported.

The customer row forbids the payment. The reason is that the pattern of ordering does not carry the pattern of paying.

The part_of row forbids the ownership. The reason is that composition does not carry possession.

The refusal runs at the inference boundary. The engine reads the premise and the table, and a match returns `FORBIDDEN_INFERENCE`.

The cost is expressiveness. A prohibition removes a conclusion the language could draw, and the removal is the price of the boundary.

The table is a working set, and the incomplete row is the next finding. A table that claims to be complete is a table that has stopped looking.

The review asks which derivations the language refuses. A team that cannot answer has a schema.

The measure is the refusals. A language with a table has a negative half, and a language without one will over-infer.

The negative half is the semantics, and the semantics is the refusals.

The prohibitions in practice

The sibling prohibition is the clearest case, because the invalid inference is the one a person would make; two people are siblings, and the graph infers that they live together, and the inference is plausible and unsupported.

The graph can record a household when the evidence supports it, and the household is a separate claim with its own evidence; the prohibition does not deny the household; it denies the inference.

The report prohibition is the same shape; the graph records that A reported P, and the graph cannot infer P, and the forbidden inference is the promotion from testimony to fact.

The customer prohibition is a pattern case; the graph records that the customer ordered, and the graph cannot infer that the customer paid, and the forbidden inference is the promotion from order to payment.

The part_of prohibition is a composition case; the graph records that the part is in the assembly, and the graph cannot infer that the part is owned by the assembler, and the forbidden inference is the promotion from composition to ownership.

Each prohibition is local. It names a premise and a conclusion, and the conclusion is the one the language refuses.

The refusal is cheap. The table is small, and the check is one lookup at the inference boundary.

The cost is expressiveness. The language cannot draw the forbidden conclusion even when the conclusion happens to be true, and the cost is the price of the boundary.

The measure is the refusals. A language with four prohibitions has four boundaries, and a language with none has none.

The table is the artifact, and the artifact is the semantics.

The table's cost

Every prohibition costs expressiveness. The language cannot draw the forbidden conclusion even when the conclusion is true, and the cost is the price of the boundary.

The cost is paid at the inference boundary. The engine checks the premise against the table, and the check is one lookup.

The table also costs maintenance. A new relation needs a row, and the row needs a reason.

A false prohibition is worse than no prohibition. A row that forbids a valid inference makes the language reject a conclusion the evidence supports.

The review asks which rows are load-bearing and which are stale. A stale row is a prohibition with no premise, and the row is the finding.

The measure is the refusals. A language with four rows has four boundaries, and the boundaries are the semantics.

The first table is one premise and one forbidden conclusion. The table grows by finding, not by design.

The refusal is the record. `FORBIDDEN_INFERENCE` names the premise, the conclusion and the row.

The table is the semantics, and the semantics is the refusals.

The refusals

The sibling refusal returns `FORBIDDEN_INFERENCE` for `lives_with`, and the refusal names the premise, the conclusion and the row.

The report refusal returns `FORBIDDEN_INFERENCE` for the content, and the refusal names the act and the forbidden promotion.

The customer refusal returns `FORBIDDEN_INFERENCE` for the payment, and the refusal names the pattern and the forbidden promotion.

The part_of refusal returns `FORBIDDEN_INFERENCE` for the ownership, and the refusal names the composition and the forbidden promotion.

The refusal is the proof that the language has a negative half. A language that materializes the conclusion has no refusals.

The refusals are the measure. Four rows, four boundaries, four refusals.

The table's review

The review reads the rows and the refusals. Each row has a premise, a forbidden conclusion and a reason.

A row with no premise is stale. A row with no reason is a dogma. A row with no conclusion is a comment.

The review asks which rows are load-bearing and which are not. A load-bearing row is a boundary; a stale row is a finding.

The review is quarterly, and the table changes when the domain changes. A new relation is a new row, and the row is the new boundary.

The table's first year

The first year is one premise and one forbidden conclusion. The premise is `sibling_of`, the conclusion is `lives_with`, and the reason is the kin relation.

The first refusal is the proof. The inference engine returns `FORBIDDEN_INFERENCE`, and the refusal names the row.

The second row is the report, and the third is the customer. The table grows by finding, and the finding is the boundary.

The first review asks which rows are load-bearing. The load-bearing rows are the ones the domain cannot do without.

The prohibitions' record

The record is the table. Each row has a premise, a forbidden conclusion and a reason.

The premise is the claim the language can derive. The conclusion is the claim the language refuses. The reason is the distinction that makes the refusal load-bearing.

The table is the negative half of the semantics, and the refusals are the evidence that the half exists.

The measure is the refusals, and the table is the record.

The table's limit

The table is incomplete. A working set of prohibitions is not a complete theory of what the language must refuse.

A table with no forbidden row is a schema. A table with one forbidden row is a language with one boundary.

The table is also revisable. A new relation is a new row, and a stale row is a finding.

The table is a working set, and the working set is the semantics.

The table's end

The table ends at the refusals, and the refusals are the boundaries. The boundaries are the language, and the language is the table.

The table is the record, and the record is the semantics. The measure is the refusals, and the refusals are the table.

The table is the language, and the language is the refusals. The refusals are the boundaries, and the boundaries are the semantics.

The objection

The strongest objection is that a semantics of prohibitions is only negative. A language should say what is true, not what is false.

The objection is right about the positive half and wrong about the whole. The positive half is cheap; the negative half is where the damage is.

A second objection is that prohibitions make the graph too rigid. The rigidity is the point: the graph refuses inferences it cannot justify, and the refusal is the boundary.

A third objection is that the table is incomplete. It is; the table is a working set, and the incomplete row is the next finding.

What would refute this

If a representation with no prohibitions can avoid over-inference, the table is unnecessary. If a prohibition is only documentation and not enforced, the language has no negative half.

A cheaper test: take one premise and ask for a conclusion the representation forbids. If no conclusion can be named, the language is positive-only.

A second test: run the inference and see whether the system refuses. If it materializes, the table is a comment.

The table is the language, and the language is the refusals; the refusals are the boundaries, and the boundaries are the semantics; the semantics is the table, and the table is the record. The record is the boundary, and the boundary is the refusal.

The table is the boundary. The boundary is the refusal. The refusal is the language. The language is the table. The table is the semantics. The semantics is the record. The record is the refusal. The refusal is the table.

The table is the record.

A semantics is its prohibitions; the next chapter asks what a prohibition costs.

51 · Constraints Compose by Intersection

Two rules that must both hold define a set of actions that satisfy both. The set is the intersection, and the intersection is the composition.

The intersection

A constraint is a predicate on an action. `order_value <= 500` is a constraint; `2026-04-01 .. 2026-06-30` is a constraint; `jurisdiction = DE` is a constraint.

Two constraints compose by intersection: an action satisfies both when it satisfies each, and the set of such actions is the feasible set, which is the composition and neither a new rule nor a negotiation.

The intersection is associative and commutative. Three constraints can be intersected in any order, and the feasible set is the same.

Two limits

One policy allows orders up to 500. Another allows orders up to 100.

Rule	Constraint
A	order_value <= 500
B	order_value <= 100
Intersection	order_value <= 100

The intersection is 100, and an order of 300 satisfies A and violates B, so the composition refuses it; a limit of 300 is a blend, which is to say a third rule that violates B and not a compromise between the two.

The larger rule does not absorb the smaller. The smaller rule is the binding constraint, and the binding constraint is the intersection.

Two windows

One grant is valid from April 1 to June 30. Another is valid from May 1 to July 31.

Rule	Window
A	2026-04-01 .. 2026-06-30
B	2026-05-01 .. 2026-07-31
Intersection	2026-05-01 .. 2026-06-30

The intersection is the overlap. An action on April 15 satisfies A and violates B, and the composition refuses it.

The union is not the composition. The union would allow July 15, and July 15 violates A.

The blend

A blend is an action outside the intersection that someone proposes as a compromise.

The blend of 500 and 100 is 300, and 300 violates B. The blend of April—June and May—July is April—July, and July violates A.

A blend is a new rule, and a new rule needs an authority, which can change B and make a new constraint rather than a composition; the blend is common because it looks like fairness.

The midpoint is fair to the two rules and violates one of them.

The empty set

Two constraints can have an empty intersection.

Rule	Constraint
A	order_value <= 100
B	order_value >= 200
Intersection	empty

The empty intersection is a conflict. No action satisfies both, and the composition returns `EMPTY_INTERSECTION`.

An empty intersection does not grant an action: it refuses until one rule changes, and the change is a new constraint with an authority. A system that treats the empty set as a permission has inverted the composition, and the inversion is the failure.

The proof

A composition has a witness: an action and the result of each constraint.

Action	A: <= 500	B: <= 100	Result
80	pass	pass	permitted
300	pass	fail	refused
600	fail	fail	refused

The witness is the proof. A composition without a witness is a claim, and a claim without a witness is a blend.

The proof is also the audit. A reader can take the action and check each constraint, and the check is the composition.

The constraint algebra

A constraint is a predicate, and a set of constraints is a conjunction. The feasible set is the set of actions that satisfy every predicate.

The intersection is associative: three constraints can be grouped in any order, and the feasible set is the same; the intersection is commutative: the order of the rules does not change the set.

The intersection is idempotent: a rule intersected with itself is the rule. A duplicate constraint adds no permission and removes none.

The algebra is the composition: a system that composes by union adds permissions, and a system that composes by average adds a rule neither authority wrote, while the feasible set is the only composition that satisfies both rules, and an action outside the set violates at least one, and the violation is the refusal.

The two-limit proof

The two-limit case has one action and two predicates, since rule A allows up to 500 and rule B allows up to 100; the action 80 passes A and passes B, and the composition permits it.

The action 300 passes A and fails B, and the composition refuses it. The refusal is the proof that the intersection is 100 rather than 300.

The action 600 fails A and fails B, and the composition refuses it.

The witness table is the proof. A reader can take the action and check each predicate, and the check is the composition.

A blend of 300 fails B, and the failure is the violation. The blend is not a compromise; it is a third rule with no authority.

The witness table

A witness has an action and a result per constraint.

Action	A: <= 500	B: <= 100	Result
80	pass	pass	permitted
300	pass	fail	refused
600	fail	fail	refused

The table is the audit. The composition is the set of actions whose row is permitted.

A composition without a witness is a claim.

The witness is also the review. A reviewer can ask for the table, and the table is the composition.

The empty set's governance

An empty intersection is a conflict. Rule A allows up to 100; rule B requires at least 200; no action satisfies both.

The composition returns `EMPTY_INTERSECTION`. The refusal is the conflict, and the conflict is the finding.

The conflict is resolved by changing a rule, and the change has an authority. A new rule is a new constraint, and the new constraint is a new intersection.

The empty set does not grant an action. A system that treats the empty set as a permission has inverted the composition, and the inversion is the failure.

The empty set is also a review item. A conflict that persists is a rule that needs an owner, and the owner is the authority.

The empty set is the limit of the algebra. The composition is total, and the empty result is a result.

The composition in practice

The composition is the feasible set, and the feasible set is the intersection of the predicates.

The first predicate is `order_value <= 500`. The second predicate is `order_value <= 100`.

The action 80 is in the set. The action 300 is outside the set because it fails the second predicate.

The action 300 is the blend. The blend is not a compromise; it is a violation of the second rule.

The two-window case has the same shape; the first window is April 1 to June 30; the second is May 1 to July 31; the intersection is May 1 to June 30.

April 15 is outside the set because it fails the second window.

The empty case is `order_value <= 100` and `order_value >= 200`. The intersection is empty, and the empty set is a conflict.

The conflict is not a permission. The conflict is a finding, and the finding needs an authority.

The authority changes a rule, and the change is a new constraint. The new constraint is a new intersection.

The witness table is the proof. The table has the action, the result per predicate and the result of the composition.

The audit reads the table; the audit is the composition.

The review reads the conflicts; the review is the governance.

The composition is the intersection, and the intersection is the set.

The constraint's scope

A constraint has four parts: a parameter, an operator, a value and a scope.

The parameter is the action attribute: the order value, the date, the jurisdiction.

The operator is the comparison: less than, greater than, within, equals.

The value is the threshold or the set: 500, 100, April 1, DE.

The scope is the set of actions the constraint governs, and a constraint without a scope is global.

The scope is part of the rule: a limit of 500 for one team and a limit of 100 for another are two constraints with two scopes, and the intersection is the overlap of the scopes, and the overlap is the composition.

The two-window proof

The first window is April 1 to June 30. The second is May 1 to July 31.

The overlap is May 1 to June 30. April 15 is outside the overlap because it fails the second window.

July 15 is outside the overlap because it fails the first window.

The witness is the date and the result per window.

The witness is the proof, and the proof is the composition.

The conflict's authority

The empty intersection is a conflict. The conflict has no action, and the conflict needs an authority.

The change is not a blend: the blend is a third rule with no authority, and the third rule violates one of the original rules; the authority is the resolution, a new rule and therefore a new composition.

The conflict is the finding, and the authority is the owner.

The constraint's proof

The proof is a table with one row per action and one column per constraint.

The row for 80 passes both constraints, and the composition permits it.

The row for 300 passes the first and fails the second, and the composition refuses it.

The row for 600 fails both, and the composition refuses it.

The proof is the table. A composition without the table is a claim.

The conflict is the empty table.

The authority is the row that changes the rule. The change is a new constraint, and the new constraint is a new table.

The composition is the table, and the table is the proof.

The composition's review

The review reads the table and the conflicts. It asks which actions are permitted, which are refused and which have no row.

A permitted action has a witness in every constraint. A refused action has a failure in at least one. A conflict has no permitted action.

The review is quarterly, and the conflicts are the agenda. A conflict that persists is a rule that needs an authority.

The authority changes a rule, and the change is a new constraint. The new constraint is a new table, and the new table is a new composition.

The composition is the intersection, and the intersection is the proof.

The composition's limit

The composition is not a negotiation, and it is not a compromise. It is the set that both rules leave.

A team can renegotiate a rule, and the renegotiation is a new constraint with an authority; the new constraint is a new table, and the new table is a new composition.

A team cannot blend a rule, because the blend is a third rule with no authority. The blend violates one of the original rules, and the violation is the finding.

The limit of the composition is the authority. A rule without an authority is a preference, and a preference can be blended.

A rule with an authority is a constraint, and a constraint can only be intersected or changed.

The composition is the intersection, and the intersection is the constraint.

The composition's end

The composition ends at the intersection, and the intersection is the set. The set is the proof, and the proof is the table.

The table is the audit, and the audit is the review. The review is the governance, and the governance is the authority.

The authority changes the rule, and the change is the new intersection. The new intersection is the new composition.

The composition's record

The record is the table, and the table is the proof. Each row is an action, and each column is a constraint.

The proof is the witness for every row. A permitted row satisfies every constraint; a refused row fails at least one.

The record is the composition, and the composition is the intersection.

The record is the audit, and the audit is the review.

The composition is the intersection, and the intersection is the record.

The composition's measure

The measure is the table. A composition with a table has a witness, and a composition with a witness has a proof.

The proof is the intersection. A permitted action satisfies every constraint, and a refused action fails at least one.

The measure is the constraint, and the constraint is the intersection.

The composition's rule

The rule is the intersection. A permitted action satisfies every constraint, and a refused action fails at least one.

The rule is the table, and the table is the proof.

The rule is the composition.

The composition's summary

The intersection is the composition. The composition is the set. The set is the proof. The proof is the table.

The table is the record. The record is the audit. The audit is the review. The review is the governance.

The governance is the authority. The authority is the rule. The rule is the intersection.

The composition's close

The intersection is the composition, and the composition is the rule; the rule is the constraint, and the constraint is the proof; the proof is the table, and the table is the record.

The record is the audit, and the audit is the review. The review is the governance, and the governance is the authority.

The objection

The strongest objection is that this is rigid, because a team wants to move quickly and the intersection slows it down. The objection is right about speed and wrong about the alternative, since the blend is faster and violates a rule that is discovered at the audit.

A second objection is that the larger limit is the practical one. The larger limit is practical for A and violates B, and the practical composition is the smaller.

A third objection is that the two rules can be renegotiated. They can, and the renegotiation is a new rule with an authority; it is not a blend.

What would refute this

If a blend can satisfy both rules, the intersection is not the composition. If an empty intersection can permit an action, the empty set is not a conflict.

A cheaper test: take two constraints and an action. Check each constraint. If the action passes both, it is in the intersection; if it fails one, it is outside.

A composition that cannot produce a witness is a negotiation.

The composition is the intersection. The intersection is the rule. The rule is the proof. The proof is the table. The table is the record. The record is the composition.

The rule is the set. The set is the rule. The rule is the composition.

Constraints compose by intersection; the next chapter asks what an empty set means.

52 · Alignment Is Jurisprudence

A model can learn to approve orders the way reviewers approved them, and it cannot say which rule it followed; alignment is not the accuracy of the imitation; it is the accountability of the ruling.

The training illusion

Training aligns behavior. A model sees ten thousand approvals and learns the distribution, including the exceptions, the drift and the mistakes.

The behavior can match the reviewers at ninety-nine percent, and the match is a rate; a rate says how often the output agrees with the label; it does not say why the output is right.

The over-limit case shows the difference; order 44021 is worth 620, the limit is 500, and the reviewers approved similar orders in the past; the model learns the approval, reproduces the mistake and cannot cite the limit.

The model also cannot distinguish the exception; a value of 480 is within the limit, and a value of 620 is not; the model sees both as ordinary orders because the reviewers treated them as ordinary.

The model cannot be appealed. A reviewer who approved the 620 order can be asked why, and the model can only produce another output with the same confidence.

Behavior is what the model does. Conduct is what the model can answer for. A system can be aligned in behavior and unaligned in conduct, and the gap is the missing reason.

The docket

A docket is the record of adjudication. It has five fields: the case, the rule, the ruling, the reason and the remedy.

Field	Example
Case	order 44021, value 620
Rule	order_value <= 500
Ruling	violation

Continued on next page

(Continued)

Field	Example
Reason	the limit binds; the approval is invalid
Remedy	reverse the approval

The case is the facts: the order, the value, the date and the parties. The facts are recorded, not inferred from a score.

The rule is the constraint the case is measured against: the limit, the window or the jurisdiction. The rule is cited, not implied.

The ruling is the decision: permitted, refused or conflicted. The ruling is one of a small set, and the set is part of the docket's schema.

The reason is the warrant. It says which rule applies, why the facts satisfy it and what the ruling does. The reason is the field that makes the ruling reviewable.

The remedy is the change: reverse, adjust, escalate or none. A ruling with no remedy is an opinion, and an opinion does not correct anything.

A docket with a ruling and no reason returns `NO_REASON`; the refusal is the first gap, and the reason is the repair; the refusal is cheaper than an audit that cannot explain itself.

The docket is not a log. A log records that something happened; the docket records why it was permitted or refused, and the why is what survives the case.

Rules and cases

A rule is general; a case is particular. The ruling is the application of the rule to the case, and the application is the work.

The application is not a lookup. The case can be ambiguous, the rule can conflict and the facts can be incomplete.

A case with a missing value cannot be ruled; it returns `INCOMPLETE_CASE` and waits for the fact; a case with two applicable rules returns a conflict and waits for the authority.

The reason is the bridge. It says which rule applies, why the facts satisfy it and what the ruling does; a reason that cites the rule and not the facts is abstract; a reason that cites the facts and not the rule is anecdotal.

A ruling without a reason is a score with a signature. It can be right and it cannot be reviewed, and the review is the point.

A ruling with a reason is a judgment. It can be checked against the rule, challenged with new facts and used in the next case.

Precedent

A precedent is a prior ruling that binds a later case with the same rule and the same material facts.

The second over-limit order, 44022, follows 44021. The rule is the same, the facts are materially the same, and the ruling and remedy are the same.

Precedent makes alignment cumulative. A ruling is not a one-off decision; it is a rule for the next case, and the next case is cheaper because the reason exists.

A case with different facts is distinguished. A value of 480 is within the limit, the ruling is permitted, and the distinction is a new reason.

The materiality is the test. A fact is material when a different fact would change the ruling, and the change is what makes the distinction a precedent.

The distinction is a new precedent. The docket grows by ruling and by distinguishing, and the growth is the case law.

Appeal

An appeal is a challenge to a ruling, with new evidence or a new reading of the rule.

The new evidence can change the ruling. A corrected value of 420 turns the violation into a permission, and the change is a new docket entry.

The old ruling remains readable, and the new ruling supersedes it. The supersession is the record, not a deletion, and the history is the appeal's evidence.

The appeal is the difference between a score and a judgment. A score cannot be appealed; a judgment can, and the appeal is what makes the judgment correctable.

The appeal also strengthens the precedent. A ruling on appeal binds the next case more strongly than the first ruling, because the challenge tested the reason.

An appeal that changes no ruling is still a record. It says the ruling survived the challenge, and the survival is part of the precedent.

Hohfeld's second order

Adjudication uses the second order of permission; the authority has the power to rule; the party has a liability to the ruling; the appeal is a power to challenge; the immunity is the limit on the authority's power.

A docket without the second order cannot represent who may rule or who may appeal; it has the first-order permission and not the power, and the power is what makes the ruling binding.

The authority has a scope, and the scope is part of the ruling. A maintenance board rules on repairs; a finance board rules on limits; a data council rules on definitions.

A decision outside the scope is not a precedent; it is an opinion. The immunity is the limit that keeps the opinion from binding, and the limit is the second order.

Two authorities can overlap, and the overlap is a conflict. A repair that changes a limit is a maintenance decision and a finance decision, and the docket needs both signatures.

The conflict is resolved by the higher authority or by a joint ruling. The resolution is a new docket entry, and the entry names both authorities.

The second order is what makes the docket a governance artifact rather than a log. The log records the ruling; the second order records who was entitled to make it.

The docket in practice

The first case is order 44021, value 620, submitted by the retail team. The rule is the limit of 500, and the ruling is a violation.

The reason is that the limit binds and the approval is invalid because the value exceeds it. The remedy is to reverse the approval.

The second case is order 44022, value 610, submitted the next day. The rule is the same, the facts are materially the same, and the ruling follows the first.

The third case is order 44023, value 480. The rule is the same, and the facts are different because the value is within the limit. The ruling is permitted, and the distinction is the value.

The fourth case is order 44024, value 420 after a correction. The appeal is the correction, and the new evidence changes the ruling from violation to permission.

The old ruling remains readable, and the new ruling supersedes it. The supersession is a new docket entry, and the entry names the appeal.

The fifth case is order 44025, value 620, submitted by the compliance team under a different grant. The rule conflicts, and the docket returns a conflict rather than a ruling.

The conflict is resolved by the finance board and the compliance board jointly. The resolution is a new entry with both signatures.

Case	Value	Rule	Ruling	Reason	Remedy	Standing
44021	620	order_value <= 500	violation	the limit binds; the approval is invalid	reverse the approval	first ruling, binds 44022
44022	610	same	violation, follows 44021	materially same facts	reverse	precedent
44023	480	same	permitted	the value is within the limit	none	distinguished, new precedent

Continued on next page

(Continued)

Case	Value	Rule	Ruling	Reason	Remedy	Standing
44024	420 after correction	same	permission on appeal	the corrected value changes the ruling	withdraw the violation	appeal supersedes 44021; the old ruling remains readable
44025	620	same limit, different grant	conflict, not a ruling	two grants overlap	joint ruling by the finance and compliance boards	scope conflict, both signatures

The table is the case law in the docket's own fields. Precedent, distinction, appeal and conflict are four different relations among five rows, and a docket that stored only the first row would have a ruling and no case law.

The review samples the five cases. It finds one appeal, one conflict and one distinction, and each is a different kind of work.

The appeal is a correction; the conflict is a scope problem; the distinction is a new precedent.

The docket is the case law, and the case law is the record.

The case for the docket

The rate and the docket disagree on the cases that matter. The rate is high because the reviewers approved most orders, and the docket is right because the rule is the limit.

The rate misses the exception. The 620 order looks like the others in the training set, and the docket sees the value and the limit.

The rate misses the conflict. The same order under two grants is a conflict, and the rate sees one label because the training data had one label.

The rate misses the appeal. The corrected 420 order is a permission, and the rate sees a violation because the label was recorded before the correction.

The docket catches the three because the docket has fields. The fields are the difference between a label and a ruling.

The docket costs more per case. The cost is the reason, the remedy and the review, and the cost is paid on the cases that need it.

The rate costs less per case and more per audit. The audit cannot explain the exception, and the explanation is the cost the rate deferred.

The docket is the alignment, and the alignment is the case.

The docket's review

The review samples ten rulings and reads the five fields. It asks whether the rule is cited, the facts are stated, the reason is material and the remedy is issued.

A ruling with no rule is a preference. A ruling with no reason is a score. A ruling with no remedy is an opinion.

The review also reads the precedents. A precedent that binds a later case is a rule; a precedent that binds nothing is a note.

The findings are the repair queue. A missing rule is a constraint to write; a missing reason is a judgment to explain; a missing remedy is a correction to issue.

The review is quarterly, and the sample is deliberate. A review that reads only the easy cases will miss the exception, and the exception is where the liability lives.

The review is the jurisprudence, and the jurisprudence is the review.

The docket's measure

The measure is the reason, not the rate. The rate says how often the model matches the reviewers, and the reason says why the ruling is correct.

A docket with five fields is a judgment. A rate with one field is a behavior.

The measure is the appeal. A ruling that can be appealed is a ruling that can be corrected.

The measure is the precedent. A ruling that binds the next case is a rule, and a ruling that binds nothing is an opinion.

The measure is the exception. A model that reproduces the exception has learned the mistake; a docket that distinguishes it has applied the rule.

The docket is the alignment, and the alignment is the measure.

The docket's last word

The docket's last word is the reason, and the reason is the ruling. The ruling is the case, and the case is the rule.

The rule is the constraint, and the constraint is the limit. The limit is the reason, and the reason is the docket.

The docket is the alignment, and the alignment is the reason.

The docket's close

The docket closes at the ruling, and the ruling is the reason. The reason is the case, and the case is the rule.

The docket is the alignment, and the alignment is the jurisprudence.

The objection

The strongest objection is that adjudication is too slow for production. A ruling per case, a reason per ruling and an appeal per challenge is a legal system, not a model.

The objection is right about speed and wrong about the alternative. The training model is faster and cannot explain an exception, and the exception is where the liability lives.

A second objection is that the model can be the authority. The model can propose a ruling, and the authority signs it; the signature is the accountability, and the proposal is the speed.

A third objection is that the docket is a court system. It is not; it is a record with five fields and a review, and the review is the governance.

The docket is slower than the rate and faster than an audit that cannot explain itself. The speed it costs is the speed of the exception, and the exception is the case that matters.

What would refute this

If a model can cite the rule, distinguish the exception and accept an appeal, the docket is unnecessary. If a ruling without a reason is reviewable, the reason is decoration.

A cheaper test: take one approval and ask for the rule and the reason. If neither can be produced, the alignment is behavior.

A second test: ask who can appeal the ruling. If no one can, the docket has no second order.

A third test: ask which precedent the ruling binds. If it binds none, the ruling is an opinion.

A fourth test: ask for the remedy. If the ruling changes nothing, the remedy is missing, and a ruling that changes nothing cannot correct the case.

The remedy is the last field and the first proof. A docket with a reason and no remedy records a judgment it does not execute; a docket with a remedy and no reason executes a judgment it cannot explain.

The two fields are the pair, and the pair is the review. A missing reason is a repair to the docket; a missing remedy is a repair to the case.

The remedy can be reverse, adjust, escalate or none, and the none is a decision. A remedy of none says the ruling changes nothing, and the record says so.

The docket is the record. The record is the ruling. The ruling is the reason. The reason is the docket.

The record is the docket. The docket is the record. The record is the alignment.

The record is the docket. The docket is the record. The docket is the ruling.

Alignment is jurisprudence; the next chapter asks what a ruling costs.

53 · Every Store Has an Epistemology

A schema says what exists; it also says, usually without saying so, what may be concluded; the second declaration is the store's epistemology, and most stores never write it down.

The two declarations

A schema has two jobs. The first is to declare the entities: a customer, an order, a payment, a complaint. The second is to declare what follows from them: what may be inferred, what may not, and on what warrant.

The first declaration is visible. It is the tables, the classes, the relations and the keys. A reader can open the schema and see the entities.

The second declaration is usually invisible. It is the views, the ETL rules, the dashboard definitions and the model's assumptions. A reader can see the outputs and cannot see the inferential policy that produced them.

The two declarations are not the same; a schema can say that a customer and an order exist, and it can be silent on whether a customer with no order is dormant, churned or unknown.

The silence is the epistemology. The store will make some inference, and the inference is the policy, whether or not anyone wrote it.

The ontology

The ontology is the answer to what exists. It names the entities, their attributes and their relations.

The ontology can be rich or thin; a rich ontology names the customer, the order, the payment, the complaint, the refund and the churn; a thin ontology names the customer and the order and leaves the rest to the queries.

The ontology is the easy part. It is visible, it is reviewable and it is the part a data catalog can display.

The ontology is also the part that gets confused with the whole; a team that has a catalog believes it has a semantics, and the catalog has the entities and not the inferences.

The ontology is necessary and insufficient. It says what the store can talk about; it does not say what the store may conclude.

The epistemology

The epistemology is the answer to what may be concluded. It names the licensed inferences, the forbidden inferences and the warrant for each.

A licensed inference is a conclusion the store may draw from a premise. `Customer` with no order in ninety days may license `dormant`.

A forbidden inference is a conclusion the store may not draw. `Customer` with no order may not license `churned`, because churn requires a termination event.

The warrant is the reason. It says why the inference is licensed or forbidden, and the reason is the part that makes the policy reviewable.

The epistemology is not the query engine. The engine executes the query; the epistemology says which queries the store is entitled to run and which conclusions it may store.

The epistemology is also not the model. The model may propose an inference; the epistemology says whether the store may accept it.

The implicit table

Most stores have an implicit entailment table. It is distributed across views, dashboards, models and habits.

A view that defines `active_customer` as an order in ninety days is a row in the implicit table; a dashboard that defines `churned` as no order in ninety days is another row.

A model trained on the dashboard's labels is a third row. The model learns the definition and reproduces it, and the definition was never declared.

The implicit table is wider than the intended policy; each view adds an inference, each dashboard adds a definition and each model adds a generalization, and no one adds the forbidden row.

The result is a store with an epistemology and no declaration. The store can answer questions, and it cannot say which conclusions it is entitled to.

Over-inference

Over-inference is the materialization of a conclusion the store did not license.

The churn case is the example. The store has `Customer`, `Order` and `Payment`. A customer with no order in ninety days is inferred to be churned, and the dashboard counts churned customers.

The inference is plausible and invalid. A customer with no order may be dormant, may be a prospect, may have moved to a different account or may be dead; churn is a termination event, and the absence of an order is not the event.

The dashboard's count is the materialized inference. The count is wrong, and the error is invisible because the inference looks like the data.

The fix is not a better model. The fix is a forbidden row: `no order in 90 days` does not license `churned`.

The entailment table

An entailment table has four columns: the premise, the licensed conclusion, the forbidden conclusion and the warrant.

Premise	Licensed	Forbidden	Warrant
Customer + no order 90d	dormant	churned	dormancy is a state; churn is a termination
Order + payment	paid	delivered	payment is not delivery
Refund + no reorder	refunded	churned	refund is not churn
REPORT(A, P)	reported	P	testimony is not fact

The premise is the claim the store has. The licensed conclusion is what the store may infer. The forbidden conclusion is what it may not. The warrant is the reason.

The table is the epistemology made explicit. It is the negative half of the schema and the positive half of the inferential policy.

An inference that is not in the licensed column returns `UNDECLARED_INFERENCE`. The refusal is the first gap, and the row is the repair.

The review

The review reads the table and the store's views. It asks whether every view has a row, every row has a warrant and every warrant has a forbidden conclusion.

A view with no row is an undeclared inference. A row with no warrant is a preference. A row with no forbidden conclusion is a permission without a boundary.

The review is quarterly, and the domain authority signs the rows. A row that the authority cannot defend is a row that the store cannot rely on.

The review is the epistemology's governance. The table is the declaration, and the review is the enforcement.

The views

A view is a definition with a name. `active_customer` is a view, and the definition is the inference.

The view defines active as an order in ninety days. The definition is a row in the implicit table, and the row licenses `active` from `order`.

A second view defines dormant as no order in ninety days. The second definition is a second row, and the two rows are not the same inference.

A third view defines churned as no order in ninety days; the third definition is a third row, and the third row is a forbidden inference because churn requires a termination event.

The three views can coexist in one store. The store has three definitions of the customer's state, and no one has said which definition governs.

The conflict is the epistemology. The store's answers depend on which view the query uses, and the query is the policy.

The models

A model trained on the views learns the definitions. The model learns active, dormant and churned as labels, and it learns the boundary at ninety days.

The model cannot cite the definition. It can reproduce the label, and it cannot say whether the label is a state or a termination.

The model also generalizes. A customer with no order for eighty-nine days is active; a customer with no order for ninety-one days is churned; the boundary is a definition, and the model treats it as a fact.

The model's output is an inference. The output is licensed when the view is licensed, and the output is forbidden when the view is forbidden.

The entailment table is the model's boundary. A model that produces `churned` from the forbidden row is a model that has learned a forbidden inference.

The fix is not a better model. The fix is a table that says which labels the model may produce.

The contract

The store owes the reader a declaration. The declaration is the entailment table, and the table is the contract.

The reader can ask three questions. What may this store conclude? What may it not conclude? On what warrant?

A store that cannot answer the three is a store with an implicit epistemology; the reader can still use the store, and the reader cannot know what the store is entitled to say.

The audit reads the table and the store's outputs. It asks whether every output is licensed, and the unlicensed output is the finding.

The contract is the governance. The table is the declaration, and the audit is the enforcement.

The outputs

An output is an inference with a number. The churn rate is an output, and the cohort table is an output.

The churn rate is computed from the churned view. The view is the forbidden row, and the rate is the materialized forbidden inference.

The cohort table is computed from the active view. The view is the licensed row, and the table is a licensed inference.

The two outputs look the same. Both are tables with numbers, and only one is entitled to the store's warrant.

The reader cannot tell the difference without the table. The table is the only thing that separates the licensed output from the forbidden one.

The store's outputs are the epistemology in practice. The declaration is the table, and the outputs are the test.

The audit

The audit samples the outputs and asks for the row. A churn rate with no row is an undeclared inference.

The audit reads the table and the output, and the match is the finding. A licensed output is a pass; a forbidden output is a finding.

The finding names the output, the row and the repair. The repair is a new view, a new row or a withdrawal.

The audit is quarterly, and the findings are the repair queue. The queue is the store's epistemology made visible.

The audit is the contract's enforcement. The table is the declaration, and the audit is the proof.

The store's example

The store has a customer table, an order table and a payment table. The customer 44021 has two orders and one payment.

The active view defines active as an order in ninety days, and 44021 is active. The row is licensed, and the definition is the warrant.

The churn view defines churned as no order in ninety days. 44021 has an order in sixty days, so 44021 is not churned.

The dashboard counts churned customers from the churn view. The count is the materialized inference, and the count is licensed when the view is licensed.

The model learns the churn label from the view. The model cannot cite the view, and it can reproduce the label.

The audit samples the dashboard and asks for the row. The churn row is the definition, and the definition is the forbidden conclusion.

The repair is a new view. The new view defines churn as a termination event, and the count changes.

The store's example is the epistemology. The ontology is the three tables, and the epistemology is the four views.

The table in practice

The table has four rows for the store's four views. Active, dormant, churn and paid each have a licensed conclusion and a forbidden one.

Active is licensed from an order in ninety days, and the forbidden conclusion is lapsed. An order is a transaction, not a state.

Dormant is licensed from no order in ninety days, and the forbidden conclusion is churned. Dormancy is a state, not a termination.

Churn is licensed from a termination event, and the forbidden conclusion is no order. The absence of an order is not the event.

Paid is licensed from an order and a payment, and the forbidden conclusion is delivered. Payment is not delivery.

The query checks the row before it materializes the conclusion. A licensed row returns the conclusion; a forbidden row returns `UNDECLARED_INFERENCE`.

The refusal is the table's proof. A store that materializes every conclusion has a table it does not enforce.

The table in practice is the store's epistemology, and the epistemology is the table.

The table's review

The review reads the table and the store's views. It asks whether every view has a licensed row and a forbidden row.

A view with a licensed row and no forbidden row is a permission without a boundary; a view with a forbidden row and no licensed row is a prohibition without a use.

The review is quarterly, and the domain authority signs the rows.

The table is the declaration, and the review is the enforcement.

The store's obligation

The store owes the reader the table, and the table is the answer to what may be concluded.

The store also owes the reader the refusal. A store that cannot refuse an undeclared inference has no boundary.

The obligation is the epistemology, and the epistemology is the table.

The objection

The strongest objection is that a store is not a theory. It is a database, and the inferences belong to the applications.

The objection is right about the database and wrong about the store. The store is the database plus the views plus the dashboards plus the models, and the inferences are already there.

A second objection is that the table is too much documentation. It is four columns and a review, and the review is the part that keeps the table from becoming a shelf document.

A third objection is that the store can be permissive by design. It can, and the permission is a declaration. A permissive store with a declared permission is a policy; a permissive store with no declaration is an accident.

The table is not a theory of the store; it is the store's own declaration, and the declaration is what the objection misses. A database that cannot say what it may conclude is not neutral; it is silent, and the silence is a policy. The table makes the policy answerable, and the answer is the table.

The reader who asks for the table is asking for the store's boundary. A store that can produce the table has an epistemology it can defend; a store that cannot has an epistemology it inherited.

The table is the boundary.

What would refute this

If a store's inferences can be audited without a table, the table is unnecessary. If an undeclared inference is harmless, the declaration is decoration.

A cheaper test: take one view and ask for its forbidden conclusion. If no forbidden conclusion can be named, the view has no boundary.

A second test: run an inference and ask whether the store refuses it. If it materializes, the epistemology is implicit.

The table is the store. The store is the table. The table is the epistemology.

The table is the record. The record is the table. The table is the obligation.

The table is the store. The store is the record.

Every store has an epistemology; the next chapter asks what it owes the reader.

PART X
The Landing

how it lands without lying.

54 · How a Synapse Lands

A projection can succeed and still be wrong. The loader imports every row, the counts match and the graph is confident about something the source only reported; the failure is not in the load; it is in the shape the claim was given.

The edge that answers

The sentence is: “The inspector reported that the pump may have failed because of the seal.” It is one sentence, and it contains two claims.

The first claim is a report: the inspector said something. The second claim is the content: the pump may have failed because of the seal.

The tempting projection is one edge.

```
(:pump)-[:MAY_HAVE_FAILED_BECAUSE_OF]->(:seal)
```

The edge is small, fast and readable. It answers “what may have failed?” in one hop, and it answers the question with a confidence the sentence did not have.

The predicate is a compound that no schema knows. The modality sits inside the label, so “list the failures” requires every label that might contain one. The reporter is absent, so the graph asserts a possible failure outright.

The edge is not wrong because it is small. It is wrong because it is confident.

A graph may hold the words. It may not inherit the standing.

The load that passes

The correct load starts with two synapses, not one.

```
S-91: REPORT
 AGENT: inspector
 source_mode: directly_expressed
 NESTS(ASSERTED_CONTENT): S-92
 span: p.4, chars 112-181

S-92: FAIL
 THEME: pump
 CAUSE: seal
 modality: possible
 source_mode: attributed_report
```

The loader imports both nodes, the role edges and the nesting edge. The counts match the source, and the load reports success.

The success is real and incomplete. The loader tests syntax and counts; it does not test what the graph is now entitled to assert.

Four questions decide the landing.

Who reported it? Walk the nesting edge back to S-91 and read the agent.

How strongly? Read the modality and the source mode on S-92.

Can the sentence be rebuilt? Yes, as a report, because the two cards and the nesting edge reconstruct the sentence.

Can it be re-imported? Yes, by its own identifier, because each card has one.

The edge-only load fails all four. There is no REPORT node, no nesting edge and no identifier for the report. The graph can answer "what may have failed?" and it cannot answer "who said so?".

The reconstruction guard makes the failure visible.

```
MATCH (r:Synapse {predicate:'REPORT'})-[:NESTS]->(f:Synapse)
WHERE f.modality = 'possible' AND f.source_mode = 'attributed_report'
RETURN count(f)
```

Before the repair, the guard returns zero rows. The absence is the finding: the edge-only load produced no REPORT node, so the guard cannot even ask its question.

After the repair, the guard returns one row.

A guard on stance properties alone returns zero rows in both states, and it is not a guard, when the claim never became a node, no node-side check can miss it; the check has to reconstruct.

The identity threshold

Node or edge is decided by identity, not by tidiness.

The first test is an identifier. A thing becomes a node when it has its own identifier; a claim with an id can be cited, corrected and re-imported; an edge with properties and no id cannot be re-imported without inventing one.

The second test is review status. A thing becomes a node when it has a review status. A claim that can be accepted, challenged or superseded needs an address for the status.

The third test is a second evidence span. A thing becomes a node when more than one span supports it; a claim with one span may be an edge; a claim with two spans is a node, because the two spans are two addresses.

The tests are ordered. The first test admits the claim; the second and third confirm it. A claim that passes any test is a node.

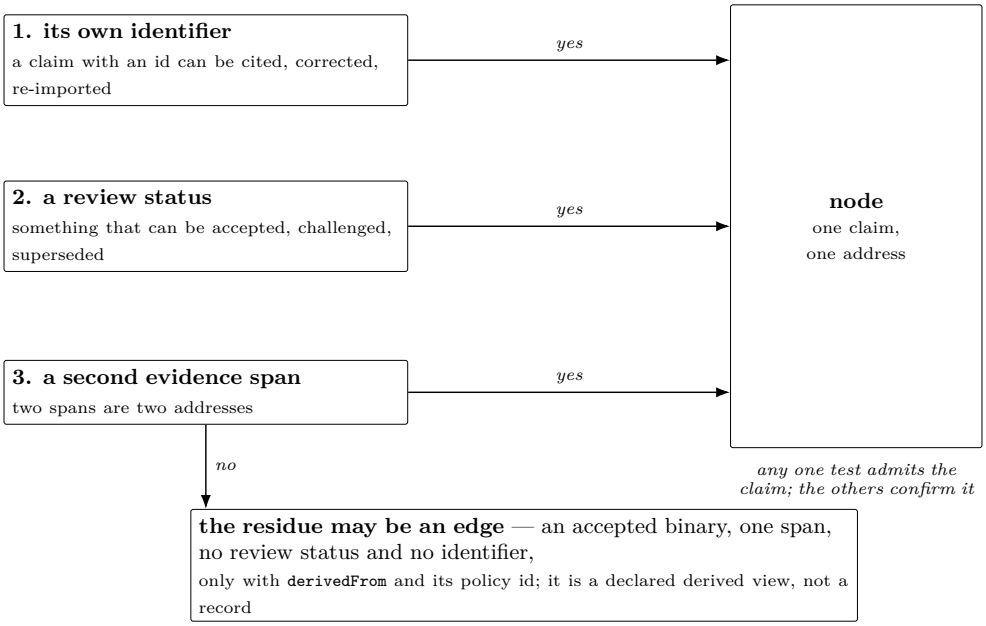


Figure 5: The three ordered tests: a claim that passes any one of them becomes a node, and only the residue may stay an edge.

The residue may be an edge. An accepted binary with one span, no review status and no identifier may be an edge, provided it carries `derivedFrom` and its policy id.

The threshold is what separates two readers. One reified everything and now has an unreadable graph; the other never reified what had identity and now cannot say who said it. The tree tells them apart.

A node per claim doubles the hops, and the cost is real. The next chapter prices it; the page does not pretend the node is free.

The receipt

Every projection declares what it dropped. The declaration is the loss report, and the report must be non-empty unless zero loss is proven.

An empty loss report is a defect signal, not a clean export. A projection that claims zero loss has either proven the round trip or stopped looking.

The loss report is a metadata graph. It names the claim, the target, the preserved fields, the dropped fields and the policy that licensed the drop.

A derived edge points back to the claim that licensed it; the pointer is `derivedFrom`, and the policy id is the second field; without the two, the edge is an orphan.

The weakest joint is the reader. The loss report is stored and read by almost no one, and the page states the gap rather than closing it. The declaration is traceable; it is not consulted.

The receipt is a tax on the only consumers who exist; analysts want edges, tools expect them and a node per claim doubles the hops; most of that is right, and the tax is the price of the audit.

The four questions

The landing is tested by four questions, and each question is a different walk through the graph.

The first question is who reported it. The answer is the agent on the REPORT card, reached by walking the nesting edge back from the content.

The edge-only form cannot answer the first question. There is no report card, no nesting edge and no agent, and the graph cannot store the reporter.

The second question is how strongly. The answer is the modality and the source mode on the content card: `possible` and `attributed_report`.

The edge-only form puts the modality inside the label; the label is a string, and a string is not a field; the query cannot compare the strength of two claims without parsing every label.

The third question is whether the sentence can be rebuilt; the answer is yes, because the two cards, the nesting edge and the role edges reconstruct the report as a report.

The edge-only form rebuilds a different sentence. It rebuilds “the pump may have failed because of the seal” and drops “the inspector reported that”, which is the difference between a report and an assertion.

The fourth question is whether the claim can be re-imported. The answer is yes, by its own identifier, because each card has one.

The edge-only form has no identifier for the report. A re-import must invent one, and the invention is a new claim.

The four questions are the landing test. A projection that answers all four is faithful; a projection that answers three is a derived view with a declared loss; a projection that answers one is a convenience.

The guard is the test made runnable. The reconstruction guard returns zero before the repair and one after, and the zero is the finding.

The threshold’s cases

The first case is a claim with its own identifier. The claim is a node, whatever its size, because the identifier is the address the ledger uses.

The second case is a claim with a review status. The claim is a node, because the status needs a place to live and an edge has no status field that a reviewer can sign.

The third case is a claim with two evidence spans. The claim is a node, because the two spans are two addresses and an edge can carry only one.

The fourth case is an accepted binary with one span, no review status and no identifier. The claim may be an edge, and the edge carries `derivedFrom` and the policy id.

The fifth case is a claim that passes none of the tests and is still important. The claim is an edge with a declared loss, and the declaration is the receipt.

The sixth case is a claim that passes every test and is small. The claim is a node, and the smallness is not a reason to flatten it.

The threshold is not a size rule. A small claim with an identifier is a node; a large claim with no identity is a residue edge.

The over-reification failure is the graph that made everything a node. It is readable to its author and unreadable to its consumer, and the threshold would have left the residue as edges.

The under-reification failure is the graph that made everything an edge. It is readable to its consumer and cannot say who said it, and the threshold would have promoted the claims with identity.

The threshold is the tree, and the tree is the landing.

The landing's cases

A report lands as two cards. The REPORT card carries the agent, the source mode and the span; the content card carries the predicate, the roles and the modality.

A denial lands as two cards. The DENY card carries the agent and the span; the content card carries the denied proposition.

A possibility lands as one card with a modality. The modality is a field on the content, not a word in the predicate.

A quotation lands as a QUOTE card with the literal wording and a speech-act card that cites it. The literal wording is preserved as a string, and the speech act is a separate card.

The four cases share one shape. The stance is a card above the content, and the content is a card with roles.

The edge-only form collapses the four into one. The report becomes an edge, the denial becomes a negation inside the label, the possibility becomes a word and the quotation becomes a paraphrase.

The collapse is not a formatting loss. Each case loses a different answer: who said it, what was denied, how strong it was, or what the exact words were.

The four cases are the reason the threshold is worth its cost. A graph that lands only the easy case will meet the other three in production.

The landing's test

The test is rehydration. Rebuild the sentence from the target and ask what the receiver is now entitled to assert.

If the rebuild is impossible, the projection has not been shown legal, whatever the loader said.

If the rebuild is possible and reads as a fact, the projection has promoted the claim. The promotion is a failure, even when the graph is complete.

If the rebuild reads as a report, the projection has preserved the entitlement. The report is the claim, and the claim is the landing.

The guard makes the test runnable. The reconstruction guard returns zero before the repair and one after, and the zero is the finding.

The receiver is the second test. A consumer that can rebuild the report and state what it may assert has a faithful view; a consumer that can only read the edge has a convenience.

The declared loss is the third test. A projection with a non-empty loss report has named what it dropped; a projection with an empty report has either proven the round trip or stopped looking.

The three tests are the landing, and the landing is the test.

The landing's receipt

The receipt is the loss report, and the loss report is the declaration.

The receipt names the claim, the target, the preserved fields, the dropped fields and the policy.

The receipt is the proof that the landing was a translation, and the translation is the receipt.

The landing's word

The landing's word is the node, and the node is the claim. The claim is the stance, and the stance is the node.

The edge is the convenience, and the convenience is declared.

The node is the landing.

The landing's close

The landing closes at the node, and the node is the claim. The claim is the stance, and the stance is the landing.

The edge is the convenience, and the node is the claim.

The shape at scale

One hundred report sentences land as two hundred cards, three hundred role edges and one hundred nesting edges.

The edge-only form lands as one hundred edges. The count is smaller, and the smaller count is the dropped stance.

The four questions scale with the cards. The reporter is one nesting walk; the strength is one property; the rebuild is a traversal; the re-import is an identifier.

The edge-only form answers one question at any scale. It answers “what may have failed?” and cannot answer who said so at any scale.

The node form costs hops, and the hops are the shape’s price. The thin view answers the hops and carries the receipt.

The shape is not the price. The price is the next chapter’s question.

The shape at scale is the landing’s cost, and the cost is the shape.

The thin view, granted

A thin accepted-facts view is admissible.

The thin view is a second projection, not the only graph. It carries the accepted facts as edges, and every edge carries the claim id and the policy that licensed it.

The thin view’s omissions are printed on it. The print is the declaration, and the declaration is what makes the thin view a view rather than a replacement.

The thin view answers the cost objection. It is fast, it is readable and it is derived. It is not the faithful graph, and it does not claim to be.

The faithful graph and the thin view can ship together. The faithful graph is the record; the thin view is the convenience; the receipt joins them.

A thin view with no receipt is a graph that has forgotten what it dropped. A thin view with a receipt is a graph that can be audited.

How a synapse lands

The pump report lands as two cards, three role edges, one nesting edge and one span.

```
S-91 REPORT {source_mode: directly_expressed}
 -[AGENT]-> inspector
 -[NESTS]-> S-92
 -[SPAN]-> p.4 c.112-181

S-92 FAIL {modality: possible, source_mode: attributed_report}
 -[THEME]-> pump
 -[CAUSE]-> seal
```

The four questions are answerable from the graph. The reporter is one walk back; the strength is one property; the sentence is rebuildable as a report; the claim is re-importable by its own id.

Correcting “seal” to “gasket” touches one node and no neighbor. The correction is a new card, and the old card remains readable.

The edge-only form answers one question and loses three. It is a derived convenience, and it is admissible only with `derivedFrom`, the policy id and the declared loss.

A synapse lands as a node that holds its stance, with roles as edges from it. An edge form is a derived, declared view that points back. An edge that drops stance has failed.

The rule is settled and unrun: a card stays a node, and nothing yet says what the node costs to write, what a guard would have to check, or what the query bill comes to —and a rule that cannot be run is a preference.

55 · Landing in a Property Graph

A property graph can hold the claim or the convenience. The difference is not the database; it is the schema decision that says where the stance lives.

The schema

The landing schema has two labels. `:Synapse` is the claim, and `:Entity` is the participant.

A `:Synapse` node carries five properties: `id`, `predicate`, `modality`, `source_mode` and `span`.

The `id` is the claim's address. The `predicate` is the claim's predicate. The `modality` is the strength. The `source_mode` is the stance. The `span` is the evidence.

An `:Entity` node carries an `id`. The entity is the participant, and the claim points at it.

The roles are edges from the claim to the participant. `:AGENT`, `:THEME`, `:CAUSE` and `:NESTS` are the four roles in the pump report.

The predicate is not a label. A label is a class, and a class has no stance; the claim has the stance, and the claim is the node.

The predicate is not an edge. An edge is a relation, and a relation has no address; the claim needs an address for review and correction.

The schema is the landing's first artifact. A schema that puts the predicate on the edge has already decided the stance has nowhere to live.

The load

The load creates the two claims and the four edges.

```
CREATE (s91:Synapse {id:'S-91', predicate:'REPORT',
 source_mode:'directly_expressed', span:'p.4 c.112-181'})
CREATE (s92:Synapse {id:'S-92', predicate:'FAIL',
 modality:'possible', source_mode:'attributed_report'})
CREATE (s91)-[:NESTS {type:'ASSERTED_CONTENT'}]->(s92)
CREATE (s92)-[:THEME]->(:Entity {id:'pump'})
CREATE (s92)-[:CAUSE]->(:Entity {id:'seal'})
CREATE (s91)-[:AGENT]->(:Entity {id:'inspector'})
```

The load uses `MERGE` on the identifiers in production, and `CREATE` in the page for legibility; a second run of the load must not duplicate the claims, and the identifier is what prevents the duplicate.

The load is a transaction; a partial load that creates the report and not the content is a claim without a target, and the transaction is what keeps the two together.

The load does not create the edges as the claim. The edges are roles, and the roles point at the claim.

The load is the least interesting artifact. The schema decides the shape, and the guard decides whether the shape held.

The guard

The guard is the schema's proof.

```
MATCH (r:Synapse {predicate:'REPORT'})-[:NESTS]->(f:Synapse)
WHERE f.modality = 'possible' AND f.source_mode = 'attributed_report'
RETURN count(f)
```

Before the repair, the guard returns zero rows. The edge-only load produced no REPORT node and no FAIL node, so the guard cannot even ask its question.

After the repair, the guard returns one row.

The zero is the finding. A guard that returns zero after a successful load is a guard that has caught the invisible failure.

A guard on stance properties alone returns zero in both states, and it is not a guard. When the claim never became a node, no node-side check can miss it.

The guard has to reconstruct. It has to walk the nesting edge, read the stance and compare the result with the sentence. The reconstruction is the check.

The guard is also the regression test. A new loader that flattens the claim will make the guard return zero again, and the zero is the alarm.

The cost

The cost is stated per question, and the hop count is the price.

Question	Pattern	Hops
Who reported it?	(r:Synapse)-[:NESTS]->(f:Synapse) then (r)-[:AGENT]->(e)	2
How strongly?	read f.modality, f.source_mode	0
What may have failed?	(f:Synapse)-[:THEME]->(e:Entity)	1
Can it be rebuilt?	the two nodes plus the nesting edge	3

The hops are the shape's price; the node form costs two hops for the reporter and zero for the strength; the edge form costs zero for the failure and cannot answer the reporter at any hop count.

The indexes make the hops cheap. An index on `:Synapse(id)` makes the identifier lookup a seek; an index on `:Synapse(predicate)` makes the guard a seek rather than a scan.

The page states the hop count and does not state a latency. The latency depends on the store, the cache and the machine, and the page claims no benchmark.

The cost statement is the third artifact. A landing without a cost statement has hidden the price in a query plan.

The thin view

The thin view is a second projection, not the only graph.

The thin view carries the accepted facts as edges, and every edge carries `derivedFrom` and the policy id.

```
CREATE (p:Entity {id:'pump'})-[:FAILED {derivedFrom:'S-92', policy:'P-7',
↪  modality:'possible'}]->(se:Entity {id:'seal'})
```

The `derivedFrom` points at the claim that licensed the edge. The policy id names the rule that allowed the derivation.

The thin view's omissions are printed on it. The print is the declaration, and the declaration is what makes the thin view a view rather than a replacement.

The thin view answers the readability objection. It is fast, it is readable and it is derived; the faithful graph is the record.

A thin view with no `derivedFrom` is a graph that has forgotten what it dropped.

The schema's constraints

The schema is not a suggestion. It has four constraints, and each one closes a failure.

The first constraint is uniqueness on `:Synapse(id)`. The identifier is the address, and two claims with one address are a duplicate.

The second constraint is a required `predicate` and `source_mode` on every `:Synapse`. A claim without a predicate is a node with no content; a claim without a source mode is a node with no stance.

The third constraint is an index on `:Synapse(predicate)`. The guard matches on the predicate, and the index makes the match a seek rather than a scan.

The fourth constraint is referential integrity for the role edges. Every role edge points at a `:Synapse` and an `:Entity`; a dangling edge is a role with no claim.

The constraints are the schema's contract. A load that violates one is rejected, and the rejection is the first defense against the flattened claim.

The constraints do not enforce the stance. A node can have `source_mode: 'asserted'` and be wrong, and the constraint cannot tell. The guard is the check for the wrong stance.

The load's failure modes

The load has four failure modes, and each has a symptom.

The first is the duplicate; a second load creates a second S-91, and the graph has two reports with one identifier; the `MERGE` prevents the duplicate, and the uniqueness constraint catches it if the `MERGE` is missing.

The second is the partial load. The report lands and the content does not, and the graph has a report with no target. The transaction prevents the partial, and the missing nesting edge is the symptom.

The third is the orphan edge; a role edge points at an entity that was never created, and the graph has a role with no participant; the referential check prevents the orphan, and the dangling edge is the symptom.

The fourth is the flattened load; the claim lands as an edge, and the graph has no claim node at all; the load succeeds, the counts match and the guard returns zero; the zero is the only symptom.

The four modes are why the load is the least interesting artifact. The schema prevents three, and the guard catches the fourth.

The load is a transaction, a `MERGE` and a constraint file. The three are the landing's plumbing, and the plumbing is not the contract.

The cost's three queries

The cost is stated per question, and the three queries have three plans.

The reporter query starts at the nesting edge and walks to the agent; the plan is a seek on `:Synapse(predicate)`, an expansion over `:NESTS`, and an expansion over `:AGENT`; the hop count is two.

The strength query reads two properties on the content node. The plan is a seek on `:Synapse(id)`, and the hop count is zero.

The failure query starts at the content node and walks to the theme. The plan is a seek on `:Synapse(id)` and an expansion over `:THEME`; the hop count is one.

The rebuild query reads both nodes and the nesting edge, and the hop count is three.

The edge-only form answers the failure query at zero hops and cannot answer the reporter query at any hop count; the zero is cheap and the missing answer is the cost.

The indexes are the price's discount. The predicate index makes the guard a seek; the identifier index makes the rebuild a seek; the role edges are adjacency lists.

The page states the hops and the indexes and does not state a latency. A latency claim would need a benchmark, and the page has none.

The landing's cases

A report lands as two `:Synapse` nodes. The REPORT node carries the agent, the source mode and the span; the content node carries the predicate and the modality.

A denial lands as two nodes. The `DENY` node carries the agent and the span; the content node carries the denied proposition as a nested claim.

A possibility lands as one node with `modality:'possible'`. The modality is a property, and the predicate is not a label.

A quotation lands as a QUOTE node with the literal wording and a speech-act node that cites it.

```
CREATE (q:Synapse {id:'S-93', predicate:'QUOTE',
 literal:'the pump may have failed', source_mode:'directly_expressed'})
CREATE (a:Synapse {id:'S-94', predicate:'SAY', source_mode:'directly_expressed'})
CREATE (a)-[:QUOTES]->(q)
```

The literal is a property, and the speech act is a node. The quotation is not a paraphrase, because the literal is preserved.

The edge-only form collapses the four cases into one label. The denial becomes a negation inside the label, the possibility becomes a word and the quotation becomes a paraphrase.

The guard catches the collapsed report and not the collapsed denial; a denial guard would need its own `:DENY` node and its own nesting edge, and the schema's four cases are four guards.

The four cases are why the landing is a schema. A schema that names one case will meet the other three in production.

The landing's proof

The proof is the loaded graph answering the four questions.

```
MATCH (r:Synapse {predicate:'REPORT'})-[:NESTS]->(f:Synapse)
MATCH (r)-[:AGENT]->(a:Entity)
RETURN a.id, f.modality, f.source_mode, r.span
```

The query returns one row: `inspector`, `possible`, `attributed_report`, `p.4 c. 112-181`.

The row is the four answers. The reporter is `inspector`, the strength is `possible`, the stance is `attributed_report` and the span is the address.

The rebuild is the sentence: “The inspector reported that the pump may have failed because of the seal.”

The re-import is the identifier: `S-91` for the report and `S-92` for the content.

The edge-only graph returns no row. The query starts at a `:Synapse` with `predicate:'REPORT'`, and the edge-only graph has no such node.

The null is the proof. A graph that cannot answer the query is a graph that did not land the claim.

The guard returns one row after the repair and zero before, and the zero is the finding.

The audit reads the guard and the four answers. A landing with five answers is faithful; a landing with three is a declared view.

The repair is a node, not a property. The correction adds the report card and the nesting edge, and the old edge is deprecated.

The landing’s proof is the query result, and the result is the landing.

The landing’s limits

The landing is not a benchmark. The hop count is stated, and the latency is not.

The landing is not a query language. The schema is the contract, and Cypher is the dialect.

The landing is not the only graph. The thin view is a second projection, and the faithful graph is the record.

The landing is not free. The node form costs hops, and the thin view costs the receipt.

The limit is the declaration, and the declaration is the table.

The landing’s close

The landing closes at the schema, and the schema is the claim. The claim is the node, and the node is the stance.

The guard is the schema’s proof, and the proof is the query result.

The cost is the hop count, and the hop count is the price.

The thin view is the convenience, and the convenience is declared.

The landing is the schema, the guard and the cost.

The landing's word

The word is the claim, and the claim is the node. The node is the stance, and the stance is the schema.

The guard is the proof, and the proof is the query.

The cost is the hops, and the hops are the price.

The landing is the word.

The landing's record

The record is the schema, the guard and the cost.

The record is the landing.

The record is the schema, and the schema is the claim.

The landing's obligation

The landing owes the reader the schema, the guard and the cost.

The schema says where the claim lives. The guard says whether the claim is there. The cost says what the claim costs to ask.

The obligation is the three, and the three are the landing.

The landing's last word

The last word is the schema, and the schema is the claim. The claim is the node, and the node is the stance.

The guard is the proof, and the cost is the price.

The landing is the three.

The landing's close

The landing closes at the schema, and the schema is the claim. The guard is the proof, and the cost is the price.

The landing is the three, and the three are the close.

The landing's end

The end is the schema, the guard and the cost. The three are the landing.

The landing is the end.

The test

The test is four questions and one guard.

Who reported it? Walk the nesting edge and read the agent.

How strongly? Read the modality and the source mode.

Can the sentence be rebuilt? Rebuild it from the two cards and the nesting edge, and check that it reads as a report.

Can it be re-imported? Read the identifier, and the identifier is the address.

The guard is the fifth question, and the guard is the one that runs. It returns one row after the repair and zero before, and the zero is the finding.

A landing that answers the four questions and passes the guard is faithful. A landing that answers three is a declared view. A landing that answers one is a convenience.

The landing is also the receipt. The receipt names the claim, the target and the loss, and the loss is the declaration. The receipt is the proof that the landing was a translation, and the translation is the landing. The three are the landing, and the landing is the receipt. The receipt is the record. The record is the landing. The landing is the record. The record is the schema, the guard and the cost. The landing is the three. The three are the landing.

The record is the landing. The landing is the record. The record is the schema, the guard and the cost. The landing is the three. The three are the record. The record is the landing.

The landing is a schema, a guard and a cost; the next chapter asks what it owes the reader.

56 · Landing in RDF

RDF has no predication node; it has a triple, and a triple is an edge with two ends; four encodings put a claim on top of that edge, and none of the four is the complete predication.

The triple

An RDF triple has a subject, a predicate and an object. The subject is a resource, the predicate is a property and the object is a resource or a literal.

```
:pump :failed :seal .
```

The triple says the pump is related to the seal by the `failed` property; it does not say who reported the failure, how strongly it is claimed or what evidence supports it.

The triple has no address for the claim. The claim is the edge, and an edge has no identifier that a reviewer can sign.

A blank node can be the subject of a statement about the triple, and the blank node is not the claim; it is a node with no identifier, and an anonymous claim cannot be cited.

A named graph can hold the triple, and the graph is a context rather than a claim. The graph has an identifier, and the claim inside it does not.

The triple is the substrate. The four encodings are four ways of putting a claim above it, and each pays a different price.

The price is paid in triples, in identifiers and in the roles the encoding can carry. A cheap encoding buys one answer and loses three.

The page compares the four encodings on one sentence, and the sentence is the pump report.

Reification

Reification turns the triple into a resource. The resource is an `rdf:Statement` with `rdf:subject`, `rdf:predicate` and `rdf:object`.

```
:s92 a rdf:Statement ;
 rdf:subject :pump ;
 rdf:predicate :failed ;
 rdf:object :seal ;
 :modality "possible" ;
 :stance "attributed_report" .
```

The reified statement preserves the claim as a resource. The stance and the modality are properties on the resource, and the resource has an identifier.

The report nests the statement as an object. The object of a triple can be a resource, and the reified statement is a resource.

```
:s91 a rdf:Statement ;
 rdf:subject :inspector ;
 rdf:predicate :reported ;
 rdf:object :s92 ;
 :stance "directly_expressed" ;
 :span "p.4 c.112-181" .
```

The nesting is valid because `:s92` is a resource. The report statement points at the content statement, and the content statement points at the pump and the seal.

The cost is five triples and two identifiers. The identifiers are the claim addresses, and the addresses are what make the statements citable.

Reification preserves the statement and the stance; it does not preserve the role set: the subject, predicate and object are fixed positions, and the roles are not edges from the claim.

Reification also does not preserve the sentence without a rule. The two statements can be rebuilt into a report, and the rule says `:s91` nests `:s92`.

The singleton property

The singleton property makes the predicate a resource. The predicate is an instance of a property, and the instance carries the stance.

```
:pump :failed_seal :seal .
:failed_seal a :FailureProperty ;
 :stance "attributed_report" ;
 :modality "possible" ;
 :agent :inspector .
```

The singleton property preserves the predicate identity. The predicate is a resource, and the resource has an identifier.

The cost is three triples and one identifier. The predicate is the address, and the address is cheaper than the reified statement's two.

The singleton property loses the role set; the claim is still the edge, and the edge has two positions; the agent is a property on the predicate, not a role from a claim.

The singleton property also loses the nesting. A report about the claim is a statement about the predicate resource, and the statement is not the claim.

The encoding is a good fit when the consumer asks about the predicate; it is a poor fit when the consumer asks who said it and what else the claim connects to.

The predicate resource can carry a review status, and the status is still not a claim address. A reviewer signs the predicate, and the claim has no signature.

The named graph

The named graph puts the triple inside a context. The context is a graph with an identifier.

```
:report047 {
 :pump :failed :seal .
}
:report047 :agent :inspector ;
 :stance "attributed_report" .
```

The named graph preserves the context. The graph identifier is the address, and the address can carry the reporter and the date.

The cost is one graph and one triple. The graph is the unit, and the unit is coarse: every triple in the graph shares the context.

The named graph loses the per-claim stance. A graph with two claims has one context, and the two claims cannot have different stances unless they are in different graphs.

The named graph also loses the role set. The triple inside the graph is still an edge, and the claim has no roles.

The encoding is a good fit when the context is the question. It is a poor fit when the claim is the question.

A graph with a hundred triples has one stance, and the hundred claims inherit it. The inheritance is the coarse unit, and the coarse unit is the price.

RDF-star

RDF-star quotes a triple and annotates the quotation.

```
<< :pump :failed :seal >> :stance "attributed_report" ;
 :modality "possible" ;
 :agent :inspector .
```

The quoted triple preserves the annotation. The quotation is a term, and the annotation is attached to the term.

The cost is one quoted triple and three annotations. The syntax is compact, and the compactness is the appeal.

RDF-star loses the claim node. The quotation is not a resource with an identifier, and a reviewer cannot sign the quotation.

The quoted triple also loses the role set and the nesting. The annotation is on the quotation, and the quotation is not the claim.

The encoding is a good fit when the annotation is the whole question. It is a poor fit when the claim needs an address.

A quotation can be nested inside another quotation, and the nesting is a term inside a term. The nesting is valid, and it is still not a claim node.

The five structures on the pump report

Reification

```
:s92 a rdf:Statement ;
 rdf:subject :pump ;
 rdf:predicate :failed ;
 rdf:object :seal ;
 :modality "possible" ;
 :stance "attributed_report" .
:s91 a rdf:Statement ;
 rdf:subject :inspector ;
 rdf:predicate :reported ;
 rdf:object :s92 ;
 :stance "directly_expressed" ;
 :span "p.4 c.112-181" .
```

The reification answers the reporter with `:s91` and the strength with the two properties on `:s92`. The answer is two statements, and the two statements are the claim.

The rebuild needs the nesting rule. A renderer reads `:s91` as the report, `:s92` as the content and the object pointer as the nesting, and the three produce the sentence.

The cost is ten triples and two identifiers. The identifiers are the addresses, and the addresses are what make the statements citable.

Singleton property

```
:pump :failed_seal :seal .
:failed_seal a :FailureProperty ;
 :stance "attributed_report" ;
 :modality "possible" ;
 :agent :inspector .
```

The singleton property answers the reporter with `:agent` and the strength with the two properties. The answer is one predicate resource, and the resource is the claim.

The rebuild cannot be done from the predicate alone. The role set is missing, and the missing roles are the missing sentence.

The cost is three triples and one identifier. The predicate is the address, and the address is cheaper than reification's two.

Named graph

```
:report047 {
 :pump :failed :seal .
}
:report047 :agent :inspector ;
 :stance "attributed_report" .
```

The named graph answers the reporter with the graph identifier and the strength with the graph properties. The answer is a context, and the context is the claim's setting.

The per-claim stance is lost. A second claim in `:report047` inherits the stance, and the inheritance is the coarse unit.

The cost is one graph and one triple. The graph is the address, and the address is the context.

RDF-star

```
<< :pump :failed :seal >> :stance "attributed_report" ;
 :modality "possible" ;
 :agent :inspector .
```

RDF-star answers the reporter and the strength with annotations on the quotation. The annotations are properties of the term, and the term is the quotation.

The claim node is missing. The quotation has no identifier, and a reviewer cannot sign a term.

The cost is one quoted triple and three annotations. The syntax is compact, and the compactness is the price of the missing address.

The fifth structure

```
:s92 a :Claim ;
 :predicate :FAIL ;
 :theme :pump ;
 :cause :seal ;
 :modality "possible" ;
 :stance "attributed_report" .
:s91 a :Claim ;
 :predicate :REPORT ;
 :agent :inspector ;
 :content :s92 ;
 :span "p.4 c.112-181" .
```

The fifth structure answers all four questions. The reporter is an edge, the strength is a property, the rebuild is a traversal and the re-import is an identifier.

The role set is present because the roles are edges from the claim. The claim is a node, and the node is the address.

The cost is two nodes and four edges. The cost is the price of the entitlement, and the entitlement is the four answers.

The encoding in use

The consumer’s questions

A consumer arrives with a question portfolio, not with a syntax preference; the portfolio is the set of questions the consumer needs answered, and the encoding is chosen by the portfolio.

A consumer that asks one question can use the cheapest encoding that answers it. A consumer that asks four questions needs the fifth structure, because no encoding answers all four.

The choice is recorded in the declaration. The declaration names the questions, the encoding and the losses, and the losses are what the consumer accepts.

Reification in use

The reification consumer asks who reported the failure. The answer is `:s91`, and the report statement points at the content statement.

The same consumer asks how strongly the failure is claimed. The answer is the two properties on `:s92`, and the properties are strings on the reified resource.

The rebuild consumer asks whether the sentence can be reconstructed. The answer is yes with a nesting rule, and the rule is the price of the reconstruction.

The re-import consumer asks whether the claim can be loaded again. The answer is yes because `:s91` and `:s92` are named resources, and the names are the addresses.

Singleton in use

The singleton consumer asks about the predicate. The answer is `:failed_seal`, and the predicate is a resource with an identifier.

The same consumer asks for the agent. The answer is the `:agent` property on the predicate resource, and the property is not a role edge.

The roles consumer asks what else the claim connects to. The answer is nothing, because the predicate has no role edges and the claim is still the edge.

The nesting consumer asks whether the report can be attached. The answer is no, because the report would be a statement about the predicate and not about the claim.

Named graph in use

The named-graph consumer asks for the context. The answer is `:report047`, and the graph identifier is the address.

The same consumer asks for the date and the reporter. The answer is the graph properties, and the properties apply to every triple in the graph.

The per-claim consumer asks for two stances in one graph. The answer is no, because the graph has one stance and the two claims inherit it.

RDF-star in use

The RDF-star consumer asks for the annotation. The answer is the properties on the quoted triple, and the quotation is the annotated term.

The same consumer asks for the claim address. The answer is no, because the quotation has no identifier and a reviewer cannot sign a term.

The nesting consumer asks whether the quotation can be reported. The answer is a second quotation, and the second quotation is still not a claim.

The fifth in use

The fifth consumer asks all four questions. The answers are one agent edge, one modality property, one nesting traversal and one identifier.

The fifth consumer also asks for the role set. The answer is the role edges from `:s92`, and the edges are the roles the four encodings lacked.

The cost is two nodes and four edges. The cost is the price of the four answers, and the price is the entitlement.

The encoding's declaration

The declaration is the table, and the table is the encoding's contract. The contract names the preserved, the lost and the cost, and the three are the price.

The consumer signs the declaration. The signature says the consumer accepts the losses, and the acceptance is the entitlement.

The review reads the declaration and the consumer's questions. A question with no preserved answer is a false declaration, and the false declaration is the finding.

The rule is the table. An encoding without a table is a syntax, and a syntax without a declaration is a guess.

The declaration is the encoding, and the encoding is the declaration.

The encoding's review

The review reads the table and the consumer's questions. A question with no preserved answer is a false declaration.

The review is quarterly, and the findings are the repair queue. A missing preserved answer is a new encoding; a missing loss is a new declaration.

The review signs the table, and the signature is the entitlement.

The encoding's test

The test takes the consumer's four questions and the encoding's table. A question with a preserved answer is a pass; a question with a lost answer is a finding.

Reification passes the reporter, the strength and the rebuild, and fails the re-import without a named node. The failure is the blank node, and the repair is the name.

The singleton property passes the reporter and the strength, and fails the rebuild and the re-import. The failure is the role set, and the repair is the claim node.

The named graph passes the reporter and the strength at the graph level, and fails the per-claim stance and the rebuild; the failure is the coarse unit, and the repair is a graph per claim.

RDF-star passes the reporter and the strength, and fails the claim node and the re-import. The failure is the quotation's missing address, and the repair is a claim resource.

The fifth structure passes all four. The pass is the entitlement, and the entitlement is the table.

The test is the review, and the review is the test.

The test is the declaration, and the declaration is the pass.

The pass is the test. The test is the table. The table is the pass.

The table is the test. The test is the table. The table is the pass. The pass is the table.

The test is the table. The table is the test. The pass is the test.

The comparison

Encoding	Preserves	Loses	Cost
Reification	statement, stance, id	roles, rebuild rule	5 triples
Singleton property	predicate id, stance	roles, nesting	3 triples
Named graph	context, date	per-claim stance, roles	graph + triple
RDF-star	annotation	claim node, roles, nesting	quoted triple

The four encodings are four prices. Reification buys the statement and pays the roles; the singleton property buys the predicate and pays the nesting; the named graph buys the context and pays the per-claim stance; RDF-star buys the annotation and pays the claim node.

The four questions decide the choice. Who reported it? Reification or the named graph. How strongly? All four. Can it be rebuilt? Reification, with a nesting rule. Can it be re-imported? Reification, with its identifiers.

No encoding answers all four. Reification answers three and pays the roles; the other three answer two.

The complete predication is a fifth structure. It is a claim resource with an identifier, role edges to the participants and a nesting edge to the content.

```
:s92 a :Claim ;
 :predicate :FAIL ;
 :theme :pump ;
 :cause :seal ;
 :modality "possible" ;
 :stance "attributed_report" .
:s91 a :Claim ;
 :predicate :REPORT ;
 :agent :inspector ;
 :content :s92 ;
 :span "p.4 c.112-181" .
```

The fifth structure is not one of the four encodings. It is the claim node the property graph chapter described, expressed in RDF.

The fifth structure answers all four questions, and it pays the role set; the choice is an entitlement decision: a consumer that asks all four needs the fifth structure, and a consumer that asks one can use the encoding that buys it.

No RDF encoding is the complete predication; the next chapter asks what the encoding owes the reader.

57 · Two Graphs, One Ledger

A graph is a habit until a second graph is proposed; the second graph is not a second truth; it is a second projection, and the two projections can differ without the ledger splitting.

The second graph

One graph is the default because one graph is easy to describe; the team builds the claim graph or the fact graph, and the one it builds becomes the only one it knows.

A second graph is proposed when the first graph answers one question well and another question badly. The claim graph answers the audit and the fact graph answers the lookup.

The proposal is not a proposal for two truths. It is a proposal for two projections from one compile, and the two projections share the claim set.

The two graphs can differ in shape and agree in content. The claim graph holds the claim node; the fact graph holds the edge; the edge points at the claim.

The difference has to be declared. A second graph without a declaration is a second truth, and a second truth is the failure.

The declaration is the six laws. The laws say what the second graph may drop, what it must point at and what it must be able to rebuild.

The faithful graph

The faithful graph holds the claim as a node. The node carries the predicate, the stance, the modality and the span.

The node is the address. A reviewer can cite the node, and the citation is the entitlement.

The roles are edges from the node. `:AGENT`, `:THEME`, `:CAUSE` and `:NESTS` are the four roles in the pump report.

The nesting is an edge between claims. The report claim points at the content claim, and the pointer is the rebuild rule.

The identifier is unique. Two claims with one identifier are a duplicate, and the constraint is the guard.

The faithful graph is the record. It is slower to query and faster to audit, and the audit is the reason it exists.

The thin view

The thin view holds one edge per accepted fact. The edge is the relation the analyst wants, and the analyst wants it without the claim node.

```
CREATE (p:Entity {id:'pump'})-[:FAILED {derivedFrom:'S-92', policy:'P-7'}]->(se:Entity
↪  {id:'seal'})
```

The edge carries `derivedFrom` and the policy. The two properties are the receipt, and the receipt is the admissibility condition.

The edge may drop the stance. The drop is declared, and the declaration is the loss report.

The edge may drop the roles. The drop is declared, and the declaration names the roles the thin view cannot answer.

The thin view is the convenience. It is faster to read and narrower to cite, and the narrowness is the cost.

One compile

The two graphs come from one compile. The compile parses the sentence, produces the claim set and emits both projections.

The compile is the single extraction. A second extraction is a second interpretation, and a second interpretation is a second truth.

The compile carries a version. Both projections carry the same version, and the version is the join between them.

The compile emits the faithful graph first. The thin view is derived from the claim set, and the derivation is the receipt.

The compile does not emit the thin view alone. A compile that emits only the thin view has dropped the record, and the record is the entitlement.

The compile is the one place where the two graphs meet, and the meeting is the ledger.

The six laws

The first law is one compile. Both graphs come from one parse, and a second parse is a second source.

The second law is no new claims. A projection may drop a claim and may not invent one, and the thin edge is a claim already in the faithful graph.

The third law is declared loss. Each projection declares what it preserved and what it dropped, and an empty report is a defect signal.

The fourth law is `derivedFrom`. Every derived edge points at the claim that licensed it, and the pointer is the provenance.

The fifth law is rehydration. A thin edge can be walked back to its claim, and the walk is the test of the view.

The sixth law is one home. No fact is canonically stored in both graphs, and the thin view is derived rather than duplicated.

The six laws in practice

One compile

The compile parses the sentence once and produces one claim set; the faithful graph and the thin view are two emitters over that set, and the set is the common source.

The two emitters share a version. The version is the join key between the graphs, and the join is what makes a reconciliation possible.

A second compile is a second interpretation. The second interpretation may be better, and it is not the same ledger; the version is what distinguishes the two.

The compile is the only place where the sentence is read. A thin view that reads the sentence again has a second source, and the second source is the drift.

No new claims

A projection may drop a claim and may not invent one; the thin edge is a claim already in the faithful graph, and the edge is a view of that claim.

The test is the claim identity, if the thin edge has a `derivedFrom` that resolves to a claim, the edge is a view; if the pointer resolves to nothing, the edge is an invention.

An invention is not always a lie. A convenient edge can summarize two claims, and the summary is a new claim with no warrant.

The summary is admissible only as a claim in the faithful graph. A summary that exists only in the thin view is a second truth.

Declared loss

The loss report names the preserved fields, the dropped fields and the policy. The report is a metadata graph, and the graph is the declaration.

An empty report is a defect signal. A projection that claims zero loss has either proven the round trip or stopped looking.

The report is signed by the projection's owner. The signature is the acceptance, and the acceptance is the entitlement.

The report is versioned with the compile. A report that changes without a version cannot be compared across releases.

DerivedFrom

The pointer is the provenance. `derivedFrom:'S-92'` says the edge came from the claim `S-92`, and the claim is the authority.

The policy is the rule. `policy:'P-7'` says which rule licensed the derivation, and the rule is the warrant.

An edge with no pointer is an orphan. An orphan is a fact with no claim, and the fact cannot be audited.

An edge with a pointer to a missing claim is a dangling pointer. The dangling pointer is a finding, and the repair is the claim or the edge.

Rehydration

Rehydration walks the edge back to the claim. The walk reads `derivedFrom`, then the claim node, then the stance and the roles.

The walk is the test of the view. A view that can be walked back is a view; a view that cannot is a replacement.

A mismatch is a finding. The edge says one thing and the claim says another, and the difference is the drift.

The rehydration is the audit's first move. The audit does not trust the edge; it walks the edge.

One home

The claim has one canonical home. The faithful graph is the home, and the thin view is derived.

A fact stored canonically in both graphs has two homes. The two homes will drift, and the drift is the second truth.

The thin view may duplicate a fact for query convenience. The duplication is a projection, and the projection is declared.

A query that reads the thin view is reading a projection. A query that cites the claim is reading the record.

The two graphs in practice

The faithful graph holds two claims for the pump report. `S-91` is the report, `S-92` is the failure, and the nesting edge is the report's content.

The faithful graph also holds the four roles. `:AGENT` points at the inspector, `:THEME` at the pump, `:CAUSE` at the seal and `:NESTS` at the content.

The thin view holds one edge for the failure. The edge goes from the pump to the seal, and it carries `derivedFrom:'S-92'` and `policy:'P-7'`.

The thin edge does not hold the report. The report is the claim that licensed the edge, and the claim is one walk away.

The reconciliation walks the edge to `S-92`. The walk reads the predicate, the stance and the modality, and the three are the comparison.

A matching claim passes. The edge says failure, the claim says failure, and the stance is the difference the edge dropped and the declaration named.

A missing claim fails. The edge says failure, the claim is absent, and the absence is `NO_CLAIM`.

The analyst reads the thin edge. The analyst wants the failure and not the report, and the thin edge answers the want.

The auditor reads the claim. The auditor wants the reporter and the stance, and the claim answers the want.

The two readers read one ledger. The edge is the convenience, the claim is the record, and the pointer is the join.

The pointer is the whole difference. A thin view with pointers is a view; a thin view without pointers is a second truth.

The two graphs' cost

The two graphs cost a reconciliation, and the reconciliation costs a query and a reviewer. The query walks the thin edges, and the reviewer reads the mismatches.

The cost is paid quarterly. A reconciliation that runs only once is a migration, and a migration is not a control.

The version is the second cost. Both graphs carry the compile's version, and a version mismatch is a finding before it is a drift.

The drift is the third cost; the thin view is rebuilt from a new source, and the new source is a second compile; the version is what catches the second compile.

The audit is the benefit. The auditor can walk any thin edge to its claim, and the walk is the record.

The citation is the second benefit. The analyst cites the edge for the lookup, and the auditor cites the claim for the fact, and the two citations are one ledger.

The choice is which graph the reader may cite, and the choice is the entitlement.

The two graphs' review

The review reads the pointers and the versions. A pointer that resolves to a claim is a pass; a pointer that resolves to nothing is a finding.

The review also reads the version; a thin view at version seven and a faithful graph at version eight are two ledgers, and the mismatch is the finding before any drift.

The review samples the mismatches. A mismatch is a stance the edge dropped and the claim kept, and the difference is the declaration's accuracy.

The review is quarterly, and the repair is a pointer, a claim or a version. A repair that changes no pointer is a comment.

The review signs the reconciliation, and the signature is the ledger.

The analyst reads the thin edge, and the auditor reads the claim. The two readers are the two graphs, and the pointer is the join.

The cost of the review is the sample and the walk. The walk is one query per edge, and the sample is the edges the review chooses.

The first review finds the missing pointers. The second review finds the version mismatches, and the third finds the stance differences.

A review that finds nothing is either a perfect ledger or a review that did not walk.

The two graphs are one ledger when the review passes.

The review is the ledger's proof, and the proof is the pointer; a pointer that resolves to a claim is a pass, and a pointer that resolves to nothing is a second truth. The review is quarterly because the thin view changes, and a view that changes without a review is a view that drifts; the drift is the finding, and the finding is the repair; the review is the contract, and the contract is the ledger.

The review is the ledger. The ledger is the review. The review is the contract.

The review is the ledger. The ledger is the review.

Reconciliation

Reconciliation is the query that compares the two graphs. It takes each thin edge and walks `derivedFrom` to the claim.

The walk reads the claim node and compares the stance, the predicate and the roles. A mismatch is a finding, and the finding is the repair.

An edge with no `derivedFrom` returns `NO_PROVENANCE`. The refusal is the first gap, and the pointer is the repair.

An edge whose claim is missing returns `NO_CLAIM`. The refusal is the second gap, and the faithful node is the repair.

The reconciliation is quarterly, and the findings are the queue. A thin view that passes the reconciliation is a view; a thin view that fails is a second truth.

The reconciliation is the law's enforcement, and the enforcement is the ledger.

The objection

The strongest objection is that two graphs will drift; the thin view will be rebuilt from a different source, and the two graphs will disagree without a query to catch it.

The objection is right about drift and wrong about the cause. The drift comes from a second compile or a missing pointer, and the laws remove both.

A second objection is that two graphs cost reconciliation. The cost is real, and the alternative is one graph that answers the lookup and cannot answer the audit.

A third objection is that analysts want the thin view and auditors want the faithful one. The two wants are the two projections, and the ledger is what lets both exist.

The two graphs are one ledger when the six laws hold, and the laws are the price of the second view.

The test

The test takes one thin edge and walks it back. The walk reads `derivedFrom`, then the claim node, then the stance.

The test asks whether the claim can be rebuilt from the thin edge. If the edge has no pointer, the answer is no, and the gap is the finding.

The test asks which graph the reader may cite. The thin edge is cited for the lookup, and the claim is cited for the fact.

A reader who cites both without distinction is a reader who has two truths. A reader who cites the edge for the lookup and the claim for the fact has one ledger.

Two graphs, one ledger; the next chapter asks which graph the reader may cite.

57A · The Semantic Layer's Contract

A semantic layer is a projection that computes measures at a declared grain, with named dimensions, a declared scope and a declared loss. That definition is short, and almost every failed semantic layer fails because one of its five clauses was never written down.

The distinction that matters most is between a semantic layer and a second database. A second database holds values that someone copied. A semantic layer holds values that were computed from a record, under a contract that says how, and it can be rehydrated back to the claims that produced it. Copy without a contract is what produces two answers to one question, and then a meeting to decide which answer the board hears. That meeting is the cost.

1. The six obligations

A conforming semantic layer owes six things, and each can be checked without reading its implementation.

Obligation	The question it answers	What breaks without it
Grain	one row represents what, exactly	totals double-count; no one can say why
Measures	which claims, filtered how, aggregated how	two reports compute different numbers from one ledger
Dimensions	which attributes a measure may be cut by	every analyst invents a cut, and the cuts disagree
Scope	which claims were eligible	a unit's figure includes another unit's work
Declared loss	what was dropped, and why	the figure cannot be reconciled to its source
Rehydration	how one row traces back to claims	the number is defensible only by the person who built it

The obligations are structural rather than technical. They do not say which database, which query engine, or which modelling tool. They say what must be true of the artifact,

and a team that satisfies them on a spreadsheet has a semantic layer, while a team that satisfies none of them in a warehouse has a warehouse with a naming convention. That is the whole obligation.

2. Grain

Grain is the sentence that begins *one row represents*, and it must be short enough to argue with. It is a sentence, not a schema. Calhoun's defect report declares: one row represents the open defect count for one asset class, in one month, for one business unit. Four nouns, and every one of them is a constraint.

The test is whether a reader can decide, without asking, where a new fact belongs. A defect found on 4 March 2026 in Boreworks, on a tunnel lining, belongs in exactly one row. A defect that moved from *open* to *closed* on 12 March belongs in the month the report counts it, and the contract says which month that is: the month of the status, not the month of discovery. That decision is architectural, it costs one sentence, and its absence produces two analysts who both believe they are right.

Grain also decides what the layer cannot answer. Calhoun's report cannot say which engineer found a defect, because the grain has no engineer. A question at finer grain is a different layer with a different contract, and the honest answer to that question is a pointer, not a guess.

3. Measures

A measure is an equation over claims, written by a person, owned by a scope, and versioned like any other term. It is not inferred from the ontology, and the reason is simple: an ontology says which shapes are legal, and it has nothing to say about whether *open* means *status is open today* or *status was open at any point in the month*.

Calhoun's measure reads: `open_defects = count of DEFECT claims where status = open at month end, excluding superseded versions and claims with no span`. Each clause works. *At month end* fixes the time. *Excluding superseded* prevents a corrected defect from counting twice. *Excluding claims with no span* means the figure can always be traced, at the cost of a small undercount that the layer declares.

That last clause is the interesting one, because it is a trade. An unspanned claim is still a claim, and a measure that includes it is more complete and less defensible. The contract's job is not to make the trade disappear; it is to make the trade visible, so the reader knows what the number cost. The trade is visible.

Measures also carry a direction of error. A count that excludes unspanned claims undercounts, and it does so systematically in units where the intake is new. A layer that declares this is useful. A layer that hides it produces a chart that flatters the newest unit and a meeting that cannot explain why.

4. Dimensions and conformed dimensions

A dimension is an attribute a measure may be cut by, and the list is closed. If *asset class* is a dimension, then a cut by *region* is either another declared dimension or an error, not a clever query.

Conformed dimensions are the ones the group shares. Calhoun's group office needs *business unit*, *month* and *asset class* to be the same three things in every unit's layer, or the roll-up is arithmetic across incompatible categories. The conformance is not about names. It is about membership: the same asset belongs to the same class in Boreworks and in the group report, and the class list is owned by Group Standards rather than by any unit.

Where membership disagrees, the layer must not silently union the sets. Calhoun's tunnel lining is *civil structure* in Boreworks and *underground asset* in the group's older taxonomy, and the roll-up of the two produces a category that contains neither. The contract's answer is a mapping record with an owner and a date, and a refusal to publish the roll-up while the mapping is disputed.

5. Declared loss

Every projection drops something, and the declared loss is the list. Calhoun's defect layer drops four things: the holder of the claim, the stance, the span, and the modality. A reader of the table learns that fourteen tunnel-lining defects were open in March. A reader of the ledger learns that eleven were confirmed, two were reported and one was inferred from a photograph, and that the inferred one carries a span pointing at an image rather than a sentence.

Neither view is wrong. The table is a projection that declared its loss, and the ledger is the record it came from. The failure mode appears when the table's readers begin to treat its silence as an assertion, and that is precisely what happens when a number enters a board pack without its provenance.

The declared loss therefore has a second job beyond honesty: it tells the reader which questions the artifact cannot answer. A layer that declares it dropped stance cannot answer *how many of these were confirmed*, and a reader who asks that question is sent to the ledger rather than given a number that pretends to be the same thing.

6. Early binding and late binding

Binding is the moment a name in the layer becomes a specific thing in the record. Early binding resolves at build time; late binding resolves at query time, and the choice is not a matter of taste.

Early binding is what a physical table does. The measure is bound to specific senses, specific identifiers and a specific version of the class list when the table is built, and the

table is fast because the work is already done. Its cost is staleness: when the class list changes, the table is wrong until it is rebuilt, and nothing in the table says it is wrong.

Late binding is what a view or a query does. The measure resolves senses and members when it runs, so the answer is current, and the cost is that the answer depends on when it ran. A comparison across two months that straddles a lexicon change is comparing two different definitions, and the layer will not tell you unless it was built to.

The architectural answer is a split, and it is the same split in every layer worth having. **Grain, measures and dimensions bind early**, because they define the artifact's shape and must not move under a reader. **Members and filters bind late**, because they are what a user changes, and because a member set that is frozen at build time is a set that silently excludes new work. The split is recorded in the contract, in one line: what is fixed, and what is resolved when the query runs.

There is a third option that teams reach for and should not: rebinding everything continuously and calling it real time. A layer that rebinds its measures on every query cannot be reconciled to a prior month's report, because the prior report was computed under a definition that no longer exists. Whatever the layer does, the definition used at the time of publication must be recoverable, or the organization loses the ability to explain its own history.

7. The view test

Three questions decide whether the layer is a view of the record or a rival to it.

Rehydration. Take one row and ask which claims produced it. If the answer is a set of claim identifiers with spans, the layer is a view. If the answer is a query nobody can run, the layer is a copy with a pedigree claim.

Reconciliation. Take the measure's total and compute it a second way, directly from the ledger, in a different order. Two paths to one number is evidence the definition is unambiguous. A mismatch is not necessarily an error in the layer; it is usually an ambiguity in the measure that was never written down, and the reconciliation is what exposes it. Two paths, one number.

Refusal. Ask the layer a question its grain cannot answer, and see what it does. A view refuses and says why. A rival invents a plausible number, and the invention is the failure that ends the layer's usefulness, because after the first invented number every number needs checking. Trust is not recoverable.

A layer that passes all three is a view. A layer that passes the first and fails the third is worse than no layer, because it is trusted.

8. What the layer must refuse

Refusal is a feature of the artifact, not a temperament of the analyst. Calhoun's layer refuses four questions, and the refusals are part of its definition.

It refuses a question at finer grain than its row. It refuses a question whose answer depends on a dimension it dropped. It refuses to publish a roll-up across units whose class membership is disputed. And it refuses to answer when the underlying claims are older than the contract's freshness bound, because a figure built from stale claims is a figure whose date is not the date it appears to carry.

Each refusal is a sentence the layer can say, and the sentences are written before the layer is built. That ordering matters. A layer that writes its refusals after the fact has already answered the question it should have refused, and the answer is in circulation. Order matters here.

9. The failure modes

Five failures account for most dead semantic layers, and every one is architectural.

Double truth. The layer and a report compute the same measure by different routes, and both are published. The organization now has two numbers and no way to choose, and the choice is made by whoever presents last.

Silent default. A dimension value is missing, and the layer substitutes *unknown* without declaring it. The unknown accumulates, and the total drifts from the record a little more each month, in a way nobody can attribute.

Frozen membership. A member set was fixed at build time, the business added three units, and the report has been quietly excluding them. The figure is not wrong in the way an error is wrong; it is wrong in the way a map that omits a road is wrong.

Rebinding history. The measure changed, the prior month's report was regenerated under the new definition, and the organization's own history now says something different from what it said at the time. Auditors treat this as a restatement, whether or not anyone intended one.

Unrehydratable aggregate. The number is correct and cannot be explained. It survives until the first serious challenge, and then it is withdrawn, which costs more than never having published it.

10. Worked: the defect report, from claims to row

Calhoun's group office wants one table: open defects by asset class by month. The work is worth following step by step, because each of the six obligations is decided by a sentence somebody has to write, and each sentence is a place where a project can go wrong without anyone noticing.

The claims come from three units. Boreworks files defects against tunnel lining in an asset register. SpanWorks files them against bridge decks in an inspection record.

RailWorks files them against track components in a maintenance log. Six thousand claims a month, each with a holder, a span and a stance, and each with a status that changes over time.

Grain. The contract says one row represents the open defect count for one asset class, in one month, for one business unit. That sentence settles four arguments before they are had. A defect found in March and closed in April counts in March if the report counts findings, and in neither month if it counts open-at-month-end. The group office wants the second. The contract says so.

Measure. `open_defects = count of DEFECT claims where status = open at month end, excluding superseded versions and claims with no span.` The clause about supersession matters more than it looks: a defect whose severity was corrected is one defect with two versions, and a count that reads both reports two. The clause about spans costs about two percent of the figure, and the layer declares that cost rather than hiding it.

Dimensions. The closed list is business unit, month, asset class and severity. Severity is included because the safety committee asks for it, and excluded from the grain because a defect has one severity at a time. A question about severity history is a different layer.

Conformed membership. Group Standards owns the asset class list. When Boreworks adds *shaft lining* as a class, the group list changes once, in one place, and both units inherit it. A local addition that does not reach the group list is a class the roll-up cannot see, and the layer refuses to publish a roll-up while a local class is unmapped.

Declared loss. The row drops the holder, the stance, the span and the modality. Four questions become unanswerable, and the contract lists them: who found this defect, was it confirmed or reported, where in the source is it, and was it suspected or established. A reader who asks any of the four is sent to the ledger with a query the layer provides.

Build. A nightly job reads claims newer than the last watermark, joins them to the identifier registry for the asset class, applies the measure, and writes one row per grain cell with the claim identifiers that produced it. The job records the term versions it used, so a row carries the definitions in force when it was built.

Reconciliation. A second path recomputes the month from the raw claim table by a different route, and compares totals with a tolerance of zero. Two paths that agree are evidence the measure is unambiguous. Two paths that disagree by two percent point at the span clause: one path included unspanned claims, and the definition had been read two ways.

Rehydration. An auditor picks the March tunnel-lining row, fourteen defects, and asks for the evidence. The layer returns fourteen claim identifiers, each with a span, and the job run that produced them with its term versions. The auditor reads three, finds two inspection notes and one photograph with an inferred stance, and the row's declared loss already told her the photograph was there. Nothing is reconstructed after the fact.

Late binding, and where it bites. In June the lexicon splits *lining* into *primary lining* and *secondary lining*, and the asset class list partitions with it. The March row was built

under the old class, and a July query that resolves classes at run time returns a partition the March row cannot join. The layer's answer is not to rebind history. It keeps the March row as published, marks the class version on it, and presents the trend with a break at the change, because a trend that silently redefines its own categories is a trend that cannot be defended in a board meeting.

The refusal. On 14 August the group office asks for defects by engineer. The layer refuses, and the refusal is a sentence written in April: the row has no engineer, and the ledger's holder is the system that filed the claim, not the person who found the defect. The office is offered the ledger query instead, and told what it would cost to add the dimension: a change to the intake, not a change to the query.

Six obligations, six decisions, and none of them is a product choice. That is what a contract is for: it moves the arguments to the point where they are cheap, which is before the table exists.

10A. The roll-up across two units

Calhoun's group office wants one figure across Boreworks and RailWorks, and the two units measure different things. Boreworks counts a defect once it is filed against a lining section. RailWorks counts it once an engineer accepts it into the asset register, which can be nine days later. Both figures are correct under their own contracts, and added together they double-count the nine days.

The roll-up does not pick a winner. It declares a reconciliation rule: a claim counts in the month of acceptance, and Boreworks' acceptance date is derived from its filing date plus its review lag, which is recorded per claim. The derived date is itself a claim, with the method as its basis, so a reader can see that February used a nine-day lag and March used eleven.

RailWorks objects that the lag is an average and some claims take twenty days. The contract answers with a declared tolerance: the figure is accurate to within two percent of the unit's own count, and the tolerance is published beside the number. A roll-up with a published tolerance is usable. One with a hidden tolerance is a number waiting to be challenged.

The pattern repeats at every layer. None of this requires a product. The contract is the artifact. Each decision is written down, in the order it is made. What remains is arithmetic. Nothing here is a schema.

11. The contract in one page

For any semantic layer, the contract states, in ten lines:

1. One row represents: the grain, in four nouns or fewer.
2. Measures: name, equation, owner, version.
3. Dimensions: the closed list, and which are conformed across units.
4. Scope: which claims were eligible, by unit, term version and time.

5. Declared loss: what the layer drops, and what that makes unanswerable.
6. Binding: what is fixed at build time and what resolves at query time.
7. Freshness: how old claims may be before the layer refuses.
8. Refusals: the questions the layer will not answer, written in advance.
9. Rehydration: how one row traces to claim identifiers and spans.
10. Reconciliation: the second path to the same number, and its tolerance.

Ten lines is the whole specification of the artifact, and none of the ten mentions a product. A team that writes them has a semantic layer. A team that does not has a database with a story about it.

The value of the layer is not the numbers it produces. It is that a number can be challenged and answered, which is what separates a report from a claim about the world, and what lets a business defend a decision two years after the meeting that made it.

58 · The Minimal Viable System

A minimal system is not a feature list with the expensive items removed; it is the smallest set of patterns that prevents the collapses the organization cannot tolerate, and the collapses are the scope.

The six

Six collapses account for most of the meaning lost in a knowledge system; a collapse is a promotion: a weaker record is treated as a stronger one, and the treatment is invisible because the content does not change.

The first collapse turns a source into a claim. The report becomes a fact, and the reporter disappears.

The second collapse turns a mention into an entity. A string becomes a node, and the string's context disappears.

The third collapse turns a candidate into a canonical entity. A guess becomes the entity, and the alternative candidates disappear.

The fourth collapse turns a projection into a fact. A lossy view becomes the truth, and the loss disappears.

The fifth collapse turns a fact into a decision. A fact becomes permission, and the authority disappears.

The sixth collapse turns a decision into an action. A ruling becomes execution, and the work order disappears.

The six are the common class, and the list is not exhaustive. The incomplete case is the next finding.

Source to claim

The first collapse is the most common because the first load is the easiest place to lose stance; the source says the inspector reported a possible failure, and the load stores the failure.

The collapse is invisible at load time. The loader compares counts, the counts match and the report is gone.

The pattern is the span and the stance. Every claim cites a span, and every claim carries the source mode that says whether it is reported, asserted or inferred.

The pattern is a claim node. The report is a claim about a claim, and the claim node is the address that keeps the two apart.

The pattern is a guard. The reconstruction query looks for a report claim, and a zero result is the finding.

The pattern is cheap. A span column and a stance field are two fields, and the two fields prevent the most common collapse.

Mention to entity

The second collapse turns a mention into an entity. The sentence mentions a pump, and the load creates a pump node with an identifier it invented.

The collapse is plausible because the mention looks like a reference. The mention is a string in a sentence, and the entity is a thing with an identity.

The pattern is the claim node. The mention lives in the claim, and the entity is created only when the claim is resolved to an existing identifier.

The pattern is the candidate record. The resolver proposes an entity, and the proposal is stored as a candidate with the claim as its evidence.

The pattern is the alias. A string that names an entity is an alias, and the alias is separate from the entity's canonical identifier.

The pattern costs a candidate table. The table is the price of not inventing an identity at the first mention.

Candidate to canonical

The third collapse turns a candidate into a canonical entity. The resolver's best guess becomes the entity, and the alternatives are deleted.

The collapse is tempting because the best guess is usually right. The usual case is not the audit case, and the audit asks for the alternatives.

The pattern is the candidate with a score and a status. The candidate is proposed, reviewed and either promoted or rejected, and the status is the record.

The pattern is the alias set. The entity keeps the strings that named it, and the strings are the evidence for the next resolution.

The pattern is the merge receipt. A promotion names the candidates, the reviewer and the date, and the receipt is the reversal path.

The pattern costs a review. The review is the price of the canonical identifier, and the identifier is the entitlement.

Projection to fact

The fourth collapse turns a projection into a fact. The export drops the stance, the dashboard reads the export and the dashboard's number becomes the truth.

The collapse is invisible because the export is complete at the row level. The rows are all there, and the stance is the column that is not.

The pattern is the loss report. The export declares what it preserved and what it dropped, and the declaration is the receipt.

The pattern is the return path. A consumer can walk from the export back to the ledger row, and the walk is the proof that the export is a view.

The pattern is the derived edge. A derived edge carries `derivedFrom`, and the pointer is the claim the edge summarizes.

The pattern costs a metadata graph. The graph is the price of the projection, and the projection is the convenience.

Fact to decision

The fifth collapse turns a fact into a decision. The fact is true, and the system treats the truth as permission.

The collapse is plausible because a decision needs a fact. The fact is necessary and insufficient, and the authority is the missing part.

The pattern is the capability state. The claim reaches a state, and the state is what licenses the action.

The pattern is the authority. The decision names the person or board with the scope, and the scope is the warrant.

The pattern is the receipt. The decision records the finding, the authority, the date and the conditions, and the record is the review.

The pattern costs a capability table. The table is the price of the permission, and the permission is the action.

Decision to action

The sixth collapse turns a decision into an action. The approval is recorded, and the system treats the approval as the execution.

The collapse is invisible because the approval looks like the end. The approval is the decision, and the execution is the separate act.

The pattern is the action receipt. The action names the actor, the work order, the outcome and the time, and the four fields are the record.

The pattern is the four acts. Evidence, finding, decision and action are four records, and the chain is the audit.

The pattern is the incomplete chain. A decision with no action returns `INCOMPLETE_CHAIN`, and the refusal is the finding.

The pattern costs a work order. The order is the price of the execution, and the execution is the action.

The collapse's cost

The source-to-claim collapse costs a decision, because the report is treated as fact and the decision is made on testimony that was never confirmed. The mention-to-entity collapse costs a duplicate, because the string becomes a node and the node becomes a second entity for the same thing.

The candidate-to-canonical collapse costs the alternatives, because the best guess becomes canonical and the rejected candidates are deleted before the audit; the projection-to-fact collapse costs the loss, because the export drops the stance and the dashboard reports a confidence the source never had.

The fact-to-decision collapse costs the authority, because the fact is true and the action is taken without the person who may authorize it; the decision-to-action collapse costs the execution, because the decision is recorded and the work order is assumed rather than issued.

The six costs are the six reasons the patterns exist. A pattern that does not reduce a cost is a feature, and a feature is not a requirement.

The first build

The first build starts at tier zero, where the ledger and the span are the floor; the second build adds the claim node and the stance, and the claim is the address while the stance is the field that keeps the report a report.

The third build adds the candidate and the alias, where the candidate is the resolver's proposal and the alias is the string that named the entity. The fourth build adds the projection and the loss report, where the projection is the view and the loss report is the declaration.

The fifth build adds the capability state, where the state is the permission and the permission is what the action needs; the sixth build adds the four acts, and the four acts are the chain that is the audit.

The builds are ordered, and the order is the dependency. A build that adds tier four without tier one has a permission with no claim.

Collapse	Tier	Pattern	Requires	Maintenance-org scope
source to claim	1	claim node and stance, on the tier-zero ledger and span	tier 0	built
mention to entity	2	candidate and alias	tier 1	skipped, written acceptance, monthly review

Continued on next page

(Continued)

Collapse	Tier	Pattern	Requires	Maintenance-org scope
candidate to canonical	2	candidate and alias	tier 1	skipped, written acceptance, monthly review
projection to fact	3	projection and loss report	tier 0	skipped, dashboard warning
fact to decision	4	capability state	tier 1	built
decision to action	5	four acts	tier 4	built

The table is the scope decision in one place: each collapse routes to the tier that prevents it, each tier names what it requires, and the last column is the choice this organization made rather than a property of the ladder.

The scope

The scope starts with the collapse list. The list names the collapses the organization cannot tolerate, and the list is the requirement.

The scope then divides the list into tolerated and intolerable. The tolerated collapse is accepted in writing, and the acceptance is the finding.

The scope maps the intolerable collapses to tiers. The mapping is the build plan, and the plan is the price.

The scope records the written acceptance for every skipped tier. The acceptance names the collapse, the owner and the date.

The scope is reviewed quarterly. A tolerated collapse that becomes intolerable is a new tier, and the new tier is a new build.

The minimal system is the smallest tier set that prevents the intolerable collapses. The minimal is a decision, and the decision is the scope.

The minimal in practice

A maintenance organization cannot tolerate the source-to-claim collapse. A report that becomes a fact is a work order on testimony, and the testimony is the liability.

The organization also cannot tolerate the decision-to-action collapse. An approval that becomes an execution is a repair with no work order, and the repair is the liability.

The organization can tolerate the candidate-to-canonical collapse for now. The resolver's best guess is reviewed monthly, and the review is the acceptance.

The scope is tiers zero, one, four and five. The ledger, the span, the claim, the stance, the capability and the four acts are the build.

The tiers two and three are skipped, and the skip is written. The writing names the candidate collapse and the projection collapse, and the owner signs.

The build order is tier zero, tier one, tier four and tier five. The order is the dependency, and the dependency is the plan.

The first build is the ledger and the span. The second is the claim node and the stance. The fourth is the capability state. The fifth is the four acts.

The candidate collapse is a monthly review. The projection collapse is a dashboard warning, and the warning says the view is lossy.

The minimal system is four tiers, two accepted collapses and one review. The system is minimal because the two accepted collapses are named, not because the build is cheap.

A second organization tolerates the first collapse and cannot tolerate the third. The scope is different, and the difference is the collapse list.

The scope is the collapse decision, and the collapse decision is the minimal system.

The build's evidence

Tier zero is proven by a span query. The query counts claims with no span, and a zero is the pass.

Tier one is proven by a claim count and a stance field. Every claim has a predicate and a source mode, and a claim with neither is the finding.

Tier two is proven by a candidate table. The table names the candidate, the claim and the status, and a candidate with no status is the finding.

Tier three is proven by a loss report. The report names the preserved, the dropped and the policy, and an empty report is the defect.

Tier four is proven by a capability table. The table names the claim, the state and the authority, and a claim with no state is the finding.

Tier five is proven by a four-act query. The query returns evidence, finding, decision and action, and a missing act is `INCOMPLETE_CHAIN`.

The evidence is reviewed with the tier. A tier without evidence is a feature, and a feature is not a build.

The evidence is the acceptance, and the acceptance is the build.

The tier's review

The review reads the tier map and the collapse list. A tier with no collapse is a feature, and a collapse with no tier is a gap.

The review reads the evidence for each tier. A tier with no evidence is a claim, and a claim with no evidence is the finding.

The review reads the written acceptance for each skipped tier. An acceptance with no owner is a note, and a note is not a decision.

The review is quarterly, and the findings are the build plan.

The first finding is a tier with no evidence. The tier is a claim, and the claim is a build item.

The second finding is a collapse with no tier. The collapse is a gap, and the gap is the next build.

The third finding is an acceptance with no owner. The acceptance is a note, and the note is a governance item.

The review signs the findings, and the signature is the acceptance.

The review is the evidence's check, and the check is the tier.

The review is the build.

The review is the collapse's check, and the check is the evidence; the review is the scope's check, and the check is the build; the review is the minimal's check, and the check is the tier. The review is the build, and the build is the review.

The tiers

The patterns form tiers, and the tiers are ordered by dependency. A higher tier without the lower tier leaves the collapse it was meant to prevent.

Tier zero is the ledger and the span. The tier prevents the source-to-claim collapse, and it is the floor.

Tier one is the claim node and the stance. The tier prevents the mention-to-entity collapse, and it needs tier zero.

Tier two is the candidate and the alias. The tier prevents the candidate-to-canonical collapse, and it needs tier one.

Tier three is the projection and the loss report. The tier prevents the projection-to-fact collapse, and it needs tier zero.

Tier four is the capability state. The tier prevents the fact-to-decision collapse, and it needs tier one.

Tier five is the four acts. The tier prevents the decision-to-action collapse, and it needs tier four.

The minimal viable system is the smallest tier set that prevents the collapses the organization cannot tolerate; a team that cannot tolerate the first two collapses needs tiers zero and one; a team that cannot tolerate all six needs all six.

The scope is a collapse decision, not a feature decision. The collapses are the requirements, and the tiers are the price.

The minimal system is the collapses it prevents; the next chapter asks what it owes the reader.

59 · Pattern Interaction

A pattern catalog is a list of things a team may adopt; the list is not a plan, because the patterns depend on one another, and a pattern adopted without its prerequisite is a receipt for something that does not exist.

The checklist

A checklist is a list of patterns with a box beside each one. The team reads the list, picks the patterns it likes and marks them done.

The checklist is popular because it is easy to audit. A reader can see how many boxes are checked, and the count looks like progress.

The count is not progress. A checked pattern with a missing prerequisite is a pattern that cannot run.

The loss report is the clearest case. The report declares what a projection dropped, and a projection without a claim has nothing to declare.

A team that adopts the loss report first has a receipt for a view that does not exist. The receipt is valid paperwork, and the view is absent.

The checklist does not catch the absence because the checklist has no column for prerequisites. The missing column is the design flaw.

The checklist is a catalog with a scoring habit. The catalog is useful for discovery and useless for ordering.

The order is what the matrix supplies, and the matrix is the rest of the page.

The matrix

The interaction matrix has a row for each pattern and three columns: requires, provides and conflicts.

The requires column names the patterns that must exist before the row can run, so a loss report requires a projection, a projection requires a claim and a claim requires a span.

The provides column names what the row adds to the system, so a claim provides an address, a loss report provides a declaration and a capability provides a permission.

The conflicts column names the patterns that cannot hold at the same time. Canonical promotion conflicts with candidate review; a zero-loss claim conflicts with a declared loss.

A cell can be empty. An empty requires cell means the pattern is a root, and a root is rare.

The matrix is not a hierarchy. A pattern can require two patterns and provide for three, and the graph is a network rather than a tree.

The matrix is versioned with the pattern set. A new pattern adds a row, and the row is the change.

The matrix is owned. The architecture owner maintains it, and the domain authority reviews the rows.

Prerequisites

The span is the first prerequisite and the claim is the second: a claim without a span is a sentence with no source, and the source is the warrant, while a projection without a claim is a view of nothing, and the view cannot be audited.

The projection is the third prerequisite and the capability is the fourth: a loss report without a projection is a receipt for a view that was never built, and a decision without a capability is a choice with no state, and the state is the permission.

The four acts are the fifth prerequisite, because an action without a decision is an execution with no authority and the authority is the warrant. The candidate is a sixth for the canonical pattern, because a canonical entity without candidates has no review trail, and the trail is the reversal path.

A missing prerequisite returns `MISSING_PREREQUISITE`. The refusal is the first gap, and the prerequisite is the repair.

The prerequisites are the order, and the order is the matrix read from the roots.

Conflicts

A conflict is a pair of patterns that cannot both hold. The pair is not a bug; it is a decision the matrix records.

Canonical promotion and candidate review conflict; a system that promotes the best guess and a system that keeps the alternatives are two policies, and one team cannot run both without a rule.

The faithful graph and the thin view do not conflict, because the thin view is derived; the conflict is between the thin view as a view and the thin view as the only graph.

A zero-loss claim and a declared loss conflict, because a projection cannot both drop a field and prove it kept everything. The fact-as-permission and the capability state conflict, because a system that treats truth as permission and a system that treats state as permission are two policies.

The conflict is resolved by an authority, and the resolution is a row. The row names the pair, the choice and the date.

A silent choice is the failure. A team that picks one side without recording the conflict has a policy with no record.

The matrix records the conflict, and the record is the governance.

The build order

The build order is the matrix read from the roots. A root has no prerequisites, and the root is the first build.

The span is the first root. The claim is the second build, and the projection is the third.

The loss report follows the projection, and the capability follows the claim. The four acts follow the capability, and the canonical entity follows the candidate.

The order is not a schedule. Two branches can run in parallel, and the order says which branch must finish before the other.

A cycle is a finding. Two patterns that require each other are a definition problem, and the problem is resolved before the build.

The order is recorded with the matrix. A build that starts with a dependent pattern is a build that has ignored the order.

The order is the plan, and the matrix is the source of the plan.

The order is not the only valid order. A different root can be chosen, and the choice is a scope decision.

The first matrix

The first matrix has ten rows. The rows are the patterns the team can name, and the columns are requires, provides and conflicts.

The first review finds a missing edge. The loss report requires the projection, and the edge is absent, so the matrix says the loss report is a root.

The missing edge is the finding. The team adds the edge, and the added edge changes the order.

The second review finds a conflict. The canonical pattern and the candidate pattern are both selected, and the conflict is recorded rather than resolved by habit.

The third review finds a cycle. Two patterns require each other, and the cycle is a definition problem rather than a build problem.

The matrix grows by finding. A matrix with ten rows is a start, and a matrix with thirty rows is a catalog unless the edges are there.

The matrix is the build order after the edges are added.

The first matrix is a working set, and the working set is the record.

The objection

The strongest objection is that this is bureaucracy for a pattern language. A team can hold the dependencies in its head, and the matrix is a document nobody reads.

The objection is right about the document and wrong about the dependency. The heads change, and the matrix is the record that survives the change.

A second objection is that the matrix is a catalog with extra columns. The extra columns are the point, and the point is the order.

A third objection is that thirty patterns are too many to matrix. The working set is ten, and the ten are the patterns the team can own.

A fourth objection is that the matrix will be incomplete. It will be, and the incomplete edge is the finding.

The matrix is not a guarantee. It is the record of what the team knows about its patterns, and the record is the build order.

The matrix is reviewed with the build, and the review is the governance.

The matrix is the order, and the order is the matrix.

The matrix in use

Pattern	Requires	Provides	Conflicts
Span	none	source	none
Claim node	span	address	none
Stance	claim	source mode	none
Projection	claim	view	none
Loss report	projection	declaration	zero-loss claim
Candidate	claim	proposal	canonical promotion
Canonical	candidate, review	identity	candidate-only
Capability	claim	action state	fact-as-permission
Four acts	capability	chain	decision-as-action
Thin view	claim, loss report	lookup	thin-only

The table is the matrix in miniature. A reader can see which pattern is a root, which is dependent and which pair conflicts.

The span is the only root. The span has no prerequisite, and every other pattern reaches it through the claim.

The claim is the first dependent. The claim requires the span and provides the address that every later pattern cites.

The projection requires the claim and provides the view. The loss report requires the projection and provides the declaration.

The candidate requires the claim and provides the proposal. The canonical requires the candidate and the review, and it provides the identity.

The capability requires the claim and provides the action state. The four acts require the capability and provide the chain.

The thin view requires the claim and the loss report, and it provides the lookup; the thin view conflicts with the thin-only policy, because a view that replaces its record is a second truth.

The matrix is read from the roots. The first build is the span, the second is the claim and the third is the stance.

The fourth build is the projection, and the fifth is the loss report. The sixth is the candidate, and the seventh is the canonical.

The eighth build is the capability, and the ninth is the four acts. The tenth is the thin view.

The order is the matrix read from the root, and the order is the build.

The matrix in use is the plan, and the plan is the matrix.

The matrix's review

The review reads the rows and the cells. A row with no requires is a root, and a root is rare.

The review reads the requires column first: a pattern whose prerequisite is absent from the matrix is a row with a dangling prerequisite, and the dangling prerequisite is the finding.

The review reads the conflicts column second, where a pair that conflicts and has no resolution is a silent choice, and the provides column third, where a pattern that provides nothing is a feature, and a feature is not a pattern.

The review reads the version fourth. A matrix without a version cannot be compared across releases.

The review reads the owner fifth. A matrix without an owner is a document.

The review is quarterly, and the findings are the matrix changes; a missing edge is an edge to add; a dangling prerequisite is a pattern to remove; a silent conflict is a row to write.

The first review finds the loss report at the root. The loss report requires the projection, and the edge is missing.

The second review finds the thin view at the root. The thin view requires the claim and the loss report, and both edges are missing.

The repairs are two edges, and the edges change the order.

The matrix is the order after the repairs, and the order is the build.

The matrix's review is the build's review.

The matrix's evidence

The evidence for the matrix is the row, since a row with a prerequisite, a provision and a conflict is a row the team can defend. The evidence for the order is the root, since a build that starts at a root is a build that has read the matrix.

The evidence for the conflict is the resolution, since a conflict with a date, an owner and a choice is a decision, and a conflict without the three is a habit.

The evidence for the version is the change. A matrix that changes without a version is a matrix that cannot be compared.

The evidence for the owner is the review. An owner who signs the review is an owner; an owner who does not is a name.

The evidence is the matrix, and the matrix is the evidence.

The matrix's evidence is the build's evidence.

The matrix's end

The end is the row, and the row is the prerequisite. The prerequisite is the order.

The end is the order, and the order is the build.

The end is the matrix.

The matrix's close

The close is the row, and the row is the prerequisite. The prerequisite is the order.

The close is the build.

The matrix's word

The word is the row, and the row is the prerequisite. The prerequisite is the row.

The word is the matrix.

The matrix's rule

The rule is the row, and the row is the order. The order is the matrix.

The rule is the build.

The matrix's record

The record is the row, and the row is the order. The order is the record.

The record is the matrix.

The matrix's test

The test takes one pattern and asks for its prerequisite. If the prerequisite is absent, the pattern is a receipt for nothing.

The test takes one pair and asks for its conflict. If the conflict is unrecorded, the choice is silent.

The test takes the matrix and asks for its order. If the order starts with a dependent, the matrix is a catalog.

The test is the build, and the build is the test.

The test is the matrix; the matrix is the test.

The matrix is the rule; the rule is the matrix.

The build is the rule; the rule is the build. The build is the order. The order is the build.

The order is the rule. The rule is the order. The order is the matrix. The matrix is the order.

The matrix is the rule. The rule is the matrix. The matrix is the record. The record is the matrix.

The record is the rule. The rule is the record. The record is the order. The order is the record.

The order is the record. The record is the order. The order is the matrix.

The test

The test takes one pattern and asks for its prerequisites, and a pattern whose prerequisites are absent is a receipt for nothing. The test takes one pair and asks for a conflict, and a pair that conflicts without a recorded resolution is a silent choice.

The test takes the matrix and asks for a cycle. If a cycle exists, the cycle is a finding before it is a build.

The test takes the order and asks for the first build. If the first build has a prerequisite, the order is wrong.

The test takes the matrix and asks for its version. If the version is absent, the matrix cannot be compared across releases.

The test takes the matrix and asks for its owner. If the owner is absent, the matrix is a document.

The patterns form a network, not a checklist; the next chapter asks what the network owes the reader.

60 · Plate: The Belief

A plate is a belief walked from the sentence that carried it to the work order that acted on it; the walk is the proof, and a plate that cannot be walked is a diagram.

The letter

The letter is the specimen. It is short, ordinary and complete enough to carry a belief.

```
Dear Maintenance,
The pump in Bay 14 has been leaking since March. We reported it twice.
On 4 March it failed and flooded the floor.
\allowbreak{}—\allowbreak{} Operator, 5 March
```

The letter says four things. It says the pump leaks, the leak started in March, the operator reported it twice and the pump failed on 4 March.

The letter also says something it does not state. The operator is worried, and the worry is in the repetition and the date.

The letter is not a claim. It is a document, and the claims are what the plate extracts from it.

The plate begins with the letter because the letter is the source. A plate that begins with the database has already lost the source.

The plate has six panels. The panels are the span, the claims, the finding, the decision, the action and the loss.

The six panels are the walk, and the walk is the plate.

The span

The span is the paragraph and the characters. It is `paragraph 2, characters 40-71`, and the span is the address.

The span is not the meaning. It is the place where the meaning was found, and the place is what a reviewer can open.

A claim can have more than one span. The leak claim has the letter, the inspection and the repair history, and the three spans are three addresses.

A claim with no span is a sentence with no source. The no-span claim is the first gap, and the gap is the finding.

The span is attached to the claim. The attachment is the provenance, and the provenance is the warrant.

The span test is a query. The query counts claims with no span, and a zero is the pass.

The span is the first panel, and the first panel is the source.

The claims

The claims are four. The leak, the report, the failure and the flood are four claims, and each has a predicate and a stance.

The leak claim is `LEAK(theme=pump, since=March)`. The claim is attributed to the operator, and the stance is `reported`.

The report claim is `REPORT(agent=operator, content=leak, count=2)`. The claim is a report about a report, and the nesting is the count.

The failure claim is `FAIL(theme=pump, date=4 March)`. The claim is attributed to the operator, and the stance is `reported`.

The flood claim is `FLOOD(theme=floor, date=4 March)`. The claim is attributed to the operator, and the stance is `reported`.

The four claims are four nodes. The nodes are the addresses, and the addresses are the plate's second panel.

The claims test is a count. The count of claims with a stance is the pass, and a claim with no stance is the finding.

The claims are the second panel, and the second panel is the meaning.

The finding

The finding is the inspector's conclusion. The inspector reads the letter, inspects the pump and finds seal erosion.

The finding is `EROSION(theme=seal, confidence=confirmed)`. The finding is traced to the inspection span, and the trace is the warrant.

The finding is not the claim. The claim is what the operator reported, and the finding is what the inspector concluded.

The finding has an author. The inspector is the author, and the author is the accountability.

The finding has a scope. The finding is about the seal, and the seal is the part the inspection examined.

The finding test is a trace. The trace walks from the finding to the inspection span, and a finding with no trace is an opinion.

The finding is the third panel, and the third panel is the conclusion.

The decision

The decision is the board's choice. The board reads the finding, considers the cost and decides to replace the pump.

The decision is `REPLACE(theme=pump, authority=maintenance board, date=6 March)`. The decision is traced to the finding, and the trace is the warrant.

The decision has a scope. The board may replace the pump, and the board may not change the budget.

The decision has conditions. The replacement must use the same model, and the condition is part of the record.

The decision has a remedy. The remedy is the work order, and the work order is the next panel.

The decision test is the authority. A decision with no authority is a preference, and a preference is not a decision.

The decision is the fourth panel, and the fourth panel is the choice.

The action

The action is the crew's execution. The crew receives the work order, replaces the pump and records the outcome.

The action is `WORK_ORDER(881, actor=crew B, outcome=replaced, date=7 March)`. The action is traced to the decision, and the trace is the warrant.

The action has an actor. The crew is the actor, and the actor is the execution.

The action has an outcome. The outcome is the replaced pump, and the outcome is the record.

The action has a time. The time is 7 March, and the time is the version.

The action test is the work order. An action with no work order is an event, and an event is not an action.

The action is the fifth panel, and the fifth panel is the execution.

The loss

The loss is the sixth panel. The plate declares what each stage dropped, and the declaration is the plate's honesty.

The letter's urgency is dropped at the claims. The repetition and the date are the urgency, and the claims keep the facts and not the tone.

The finding's uncertainty is dropped at the decision. The finding is confirmed, and the decision treats the confirmation as certain.

The decision's conditions are dropped at the action. The action replaces the pump, and the condition is recorded and not enforced.

The dashboard drops the letter's source. The dashboard counts the failure, and the count has no operator.

The loss is declared, and the declaration is the receipt. A plate with no loss panel is a diagram.

The loss is the sixth panel, and the sixth panel is the receipt.

The plate on one page

Panel	Record	Loss
Span	paragraph 2, characters 40-71	the letter's context
Claims	four claim nodes with stance	the operator's tone
Finding	inspection trace	the finding's uncertainty
Decision	board authority and date	the board's conditions
Action	work order 881	the crew's notes
Loss	declared at each panel	the declaration itself

The table is the plate compressed. A reader can see the six panels, the record each panel keeps and the loss each panel declares.

The span panel is the source. The span is the paragraph and the characters, and the context is the loss.

The claims panel is the meaning. The four claims are the leak, the report, the failure and the flood, and the tone is the loss.

The finding panel is the conclusion. The finding is the seal erosion, and the uncertainty is the loss.

The decision panel is the choice. The decision is the replacement, and the conditions are the loss.

The action panel is the execution. The action is work order 881, and the crew's notes are the loss.

The loss panel is the receipt. The receipt is the declaration, and the declaration is the sixth panel.

The table is not the plate. The table is the plate's index, and the plate is the walk.

The walk is the proof. A reader who can follow the table from span to action has walked the belief.

A reader who cannot follow the table has found a gap. The gap is the finding, and the finding is the plate's value.

The plate on one page is the desk page, and the desk page is the plate.

The letter's walk

The letter arrives on the fifth of March, and the plate begins with the date. The date is the version, and the version is the first thing the plate records. The operator's name is the author, and the author is the second thing. The letter is scanned, and the scan is the source. The source is the first panel.

The span is selected from the second paragraph; the span is characters forty to seventy-one, and the characters are the address. The address is what the reviewer opens, and the opening is the check; the span is not the meaning, and the meaning is the next panel; the span is the place where the meaning was found.

The claims are extracted from the span; the leak is the first claim, and the report is the second; the failure is the third, and the flood is the fourth; each claim carries the operator as the agent, and each carries the reported stance. The four claims are four nodes, and the nodes are the meaning.

The finding is the inspector's conclusion; the inspector reads the claims, inspects the pump and finds the seal erosion; the finding is traced to the inspection, and the inspection is the evidence; the finding is not the claim, and the difference is the author. The author is the inspector, and the inspector is the accountability.

The decision is the board's choice; the board reads the finding, considers the cost and decides to replace the pump; the decision is traced to the finding, and the finding is the basis; the board is the authority, and the authority is the scope. The decision is the fourth panel, and the fourth panel is the choice.

The action is the crew's execution; the crew receives the work order, replaces the pump and records the outcome; the action is traced to the decision, and the decision is the warrant; the crew is the actor, and the actor is the execution. The action is the fifth panel, and the fifth panel is the work order.

The loss is the sixth panel; the letter's urgency is dropped at the claims, and the drop is declared. The finding's uncertainty is dropped at the decision, and the drop is declared; the decision's conditions are dropped at the action, and the drop is declared; the dashboard drops the source, and the drop is declared. The declaration is the receipt.

The gaps are the plate's second finding; a missing span is a gap, and a missing stance is a gap. A missing authority is a gap, and a missing work order is a gap; the gap is the finding, and the finding is the plate's value; the plate is the architecture's proof because the gaps are visible.

The second reader walks the plate; the reader starts at the letter, follows the span and reads the claims. The reader traces the finding, checks the decision and opens the work order; the walk is the proof, and the proof is the walk. The reader who cannot walk has found the gap.

The plate's gaps

Missing	Symptom	Repair
span	claim with no source	attach the span
stance	claim reads as fact	add the source mode
finding	decision with no basis	record the finding
authority	decision with no owner	name the authority
work order	action with no execution	issue the order
loss	panel with no declaration	write the loss

The table is the plate's second half. A reader can see the six gaps, the symptom each one produces and the repair.

The span gap is the first, and the symptom is a claim with no source. The repair is the span, and the span is the address.

The stance gap is the second, and the symptom is a claim that reads as fact. The repair is the source mode, and the source mode is the stance.

The finding gap is the third, and the symptom is a decision with no basis. The repair is the finding, and the finding is the trace.

The authority gap is the fourth, and the symptom is a decision with no owner. The repair is the authority, and the authority is the scope.

The work-order gap is the fifth, and the symptom is an action with no execution. The repair is the order, and the order is the record.

The loss gap is the sixth, and the symptom is a panel with no declaration. The repair is the loss, and the loss is the receipt.

The six gaps are the plate's findings. A plate with no gaps is either complete or not walked.

The plate's gaps are the proof's gaps.

The plate's test

The test takes the letter and walks the six panels. A panel that cannot be reached is a gap.

The test takes one claim and asks for its span. If the span is missing, the claim is a sentence with no source.

The test takes one decision and asks for its authority. If the authority is missing, the decision is a preference.

The test takes one action and asks for its work order. If the order is missing, the action is an event.

The test is the walk, and the walk is the test.

The plate's close

The close is the walk, and the walk is the plate. The plate is the proof.

The close is the letter, and the letter is the source.

The close is the plate.

The proof

The proof is the walk, and the walk is the six panels; a reader who follows the six panels has proved the plate; the proof is not the diagram, and the diagram is the index. The proof is the walk.

The proof is also the gaps. A plate that hides a gap proves nothing, and a plate that shows the gap proves the gap. The gap is the finding, and the finding is the proof.

The proof is one belief. The plate does not prove the architecture for every belief, and it proves the architecture for this one; the one belief is the first proof, and the first proof is the plate.

The plate proves the walk. A reader can take the letter, follow the span, read the claims, trace the finding, check the decision and open the work order.

The plate also proves the gaps. A missing span, a missing stance, a missing authority or a missing work order is a gap, and the gap is the finding.

The plate is one belief. It is not a benchmark, and it is not a claim that every belief walks this cleanly.

The plate is the architecture's proof because the walk is the architecture. A diagram of boxes is a claim about the architecture; the plate is the architecture working.

A second plate would prove the pattern. One plate proves the walk, and the walk is the first proof.

The proof is the plate; the plate is the proof. The proof is the walk; the walk is the proof.

The walk is the plate; the plate is the walk. The walk is the architecture; the architecture is the walk.

The architecture is the plate; the plate is the architecture. The plate is the proof.

The proof is the plate; the plate is the walk.

The plate is the proof. The proof is the plate.

The plate is the belief walked end to end; the next chapter asks what the walk proves.

61 · Plate: The Notice

A notice is a claim that accuses, and a response is a claim that answers. The plate walks both chains in one specimen and shows where they meet.

The notice

The notice is the specimen. It is a document from a regulator, and it accuses the operator of two failures and one omission.

```
NOTICE OF VIOLATION
Pump PB-100-7 failed on 4 March and flooded the floor.
Maintenance records for the pump are missing.
\allowbreak{}—\allowbreak{} Regulator, 10 March
```

The notice says that the pump failed, that the floor flooded and that the records are missing. Three sentences carry four claims, and the fourth claim is carried by the arrangement rather than by a sentence.

The arrangement implies negligence. A notice that lists a failure and a missing record in the same paragraph invites the reader to supply the connection, and the connection is that the operator did not maintain the pump.

The notice is not a finding. A finding is what an examination produces, and the notice is what an examination requests. The regulator has a suspicion and a legal power, and the notice converts the suspicion into a demand for an answer.

The notice attaches no evidence. It names a pump, a date and a gap, and it leaves the proof to the operator. That absence is not an oversight. A notice is a request for the record, and the record is what the operator is expected to produce.

The notice has a span. The span is the paragraph and the character range, and the span is the address that the response must answer, because a response to a summary is a response to a document that does not exist.

The notice has a date and a version. The date is 10 March, and the date is the version that the response cites. A notice amended later is a different version, and the response to the earlier version is not a response to the later one.

The notice is the first panel. The first panel is the accusation, and the accusation is the operator’s problem.

The accusation

The accusation is four claims, and each claim has a stance. The failure, the flood, the missing records and the negligence are the four, and the stance is what the operator must answer.

The failure claim is `FAIL(theme=pump, date=4 March)`, attributed to the regulator with the stance `asserted`, and the assertion is the one claim in the notice that the operator can confirm or contest on the record.

The flood claim is `FLOOD(theme=floor, date=4 March)`. The claim is attributed to the regulator, and the stance is `asserted`. The flood is not in dispute, and the plate keeps it because a claim that is not in dispute is still a claim.

The missing-records claim is `MISSING(theme=records, scope=pump)`. The claim is attributed to the regulator, and the stance is `asserted`. This is the claim the operator will contest, and the contest will turn on what "missing" means.

The negligence claim is `NEGLIGENT(agent=operator)`. The claim is not stated anywhere in the notice, and the implication is the claim. An implied claim has no span, and a claim with no span is the hardest kind to answer.

The plate records the implied claim anyway, with the stance `implied`. Leaving it out would answer the notice and miss the accusation. A claim that is only implied is still a claim the operator has to meet.

The notice has an authority. The regulator is the authority, and the authority is the scope within which the claims bind. The operator cannot decline the notice; the operator can only answer it.

The notice has a recipient and a deadline. The recipient is the operator, and the deadline is the date by which the answer is due. A notice with no deadline is a letter, and a letter is not an instrument.

The accusation test is a stance. An accusation with no stance is an opinion, and an opinion is not a notice. The plate records the stance for every claim, and the stance is what separates a regulator's assertion from a columnist's suspicion.

The accusation is the second panel, and the second panel is the set of claims that the rest of the plate must answer.

The evidence

The evidence is three pieces, and each piece is a record with a span. The inspection, the repair order and the archive index are the three, and together they answer the notice's claims.

The inspection confirms seal erosion. The inspection is `EROSION(theme=seal, confidence=confirmed)`, and the inspection is traced to the span in the maintenance log. The warrant is the span, and the confidence is the strength of the warrant, because a span that names the log page is stronger evidence than a memory that names the pump.

The repair order confirms the replacement. The order is `WORK_ORDER(881, actor=crew B, outcome=replaced, date=7 March)`, and the order is traced to the decision that authorized it. The decision is the warrant, and the order is the record of the decision.

The archive index confirms that the maintenance records exist. The index is `RECORDS(theme=maintenance, status=filed, location=archive)`, and the index is traced to the archive's catalog. The catalog is the warrant, and the index is the record of the catalog.

The three pieces together say something the notice did not say. The pump did fail, the seal did erode, the crew did replace the seal and the records were filed in the archive rather than kept at the pump.

The word "missing" is the hinge. A record that is filed in an archive is not missing from the archive, and a record that is not at the pump is missing from the pump. The notice chose the second reading, and the evidence supports the first.

One gap remains, and the plate names it. The catalog shows that the records were filed and does not show when they were filed. A filing date before the failure would settle the negligence claim, and the plate cannot supply what the catalog does not hold.

The evidence test is a span. Evidence with no span is a story, and a story is not evidence. The plate records the span for every piece, and the span is what makes the piece admissible to the finding.

The evidence is the third panel, and the third panel is the proof the operator will use.

The finding

The finding is the resolution of the two chains. The notice says the records are missing, and the evidence says the records are misfiled. A resolution that keeps both claims is a finding, and a resolution that drops one is a verdict.

The finding is `MISFILED(theme=records, not=missing)`. The finding cites the notice's span and the archive's span, and the two spans are the join. The join is the finding's structure, and the structure is what makes the finding answerable, since a reader can walk from the resolution back to the two claims it resolves.

The figure holds the two chains side by side: four claims on the left, three records on the right, and one finding that arrives from both directions.

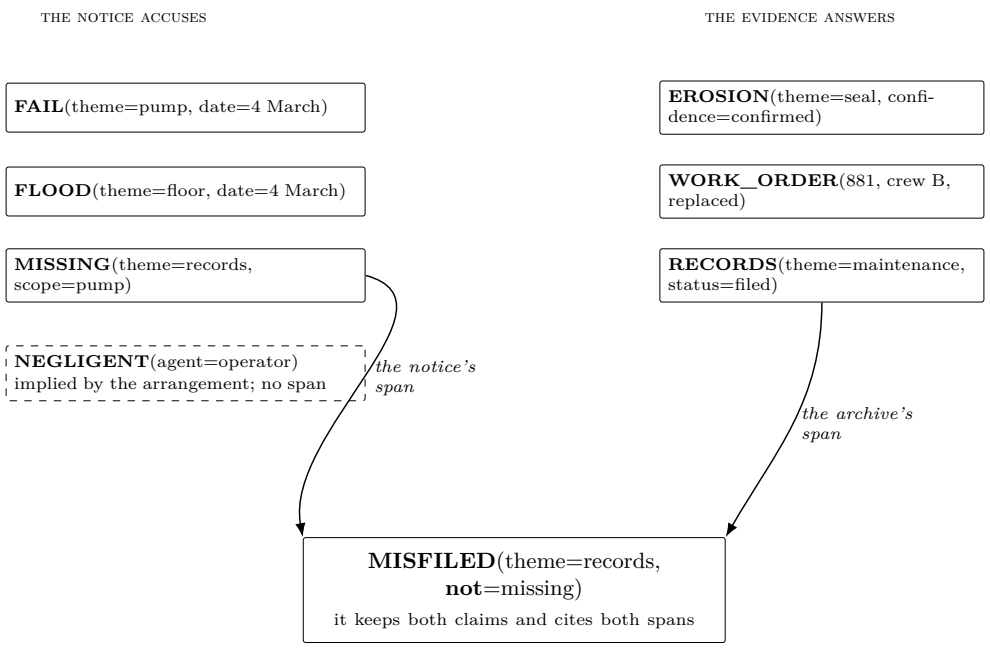


Figure 6: Two chains meeting at the finding: the notice's four claims, the evidence's three records, and MISFILED citing one span from each.

The finding is not the response. The finding is what the evidence shows, and the response is what the operator does about it. A finding with no response is an audit, and an audit that produces no action is a finding that nobody used.

The finding has an author. The compliance lead is the author, and the author is the accountability. A finding with no author is an institutional opinion, and an institutional opinion cannot be asked to explain itself.

The finding has a scope. The finding is about the maintenance records, and the records are the part of the notice that the archive examined. The finding does not clear the operator of the failure, and the finding does not contest the flood.

The finding has an uncertainty. The archive index is complete, and the index is not the records themselves. The finding records the uncertainty, and the uncertainty is the difference between "filed" and "retrievable".

The finding occupies one page, and the page is the whole of it. A resolution that needs a briefing to explain it is a briefing, and a briefing is not a finding.

The finding test is the join. A finding that cites one span is a claim, and a finding that cites both is a resolution. The plate records the join for every finding, and the join is what distinguishes a resolution from a restatement.

The finding is the fourth panel, and the fourth panel is where the two chains meet.

The response

The response is the operator's answer, and it has three parts. The operator accepts the failure, contests the missing-records claim and proposes a correction.

The response is `RESPONSE(notice=N-14, finding=MISFILED, plan=archive-repair)`. The response is traced to the finding, and the finding is the warrant. A response that is not traced to a finding is a position, and a position is not an answer.

The acceptance is partial. The operator accepts that the pump failed and that the floor flooded, and the operator does not accept the negligence. Partial acceptance is a stance, and the stance is recorded for each claim.

The response has a span. The span is the archive index, and the index is the evidence the notice did not have. The span is what makes the response checkable, and a checkable response is one the regulator can verify rather than believe, which is the whole difference between an answer and an assurance.

The response has a plan. The plan is the archive repair, and the repair is the correction, with a date, an owner and a test that the regulator will use to decide whether the correction happened.

The response has a date. The date is 20 March, and the date is the version that the regulator receives. A response sent after the deadline is a different instrument, and the plate records the deadline alongside the date.

The response test is the span. A response with no span is a denial, and a denial is a second accusation. The operator who says "the records are not missing" without a span has replaced one unsupported claim with another.

Once the response is filed, the regulator has two jobs. It verifies the span and it decides whether the correction is enough. Verification is cheap when the span is present, and the cheapness is the point of putting the span in the response.

The response is the fifth panel, and the fifth panel is the answer the notice asked for.

The outcome

The outcome is the regulator's decision. The regulator reads the response, checks the archive index and amends the notice.

The outcome is `AMENDED(notice=N-14, penalty=none, date=28 March)`. The outcome is traced to the response, and the response is the warrant. An outcome that is not traced to a response is an edict, and an edict is not a decision on the record.

The outcome has an authority. The regulator is the authority, and the authority is the scope within which the amendment binds, so the amendment changes the notice while the notice itself remains the version of record.

The outcome has a record. The record is the amended notice, and the amended notice is the version the next reader will see. The original notice is preserved, and the preservation is what lets a later reader reconstruct the exchange.

The outcome has a reason, and the reason is short. The archive index showed that the records were filed, and the regulator accepted the index as evidence. The reason is the regulator's reasoning, and the reasoning is the part of the outcome that a later dispute will cite.

The outcome test is the amendment. An outcome with no amendment is a closed notice, and a closed notice is not a resolution. The plate records the amendment, and the amendment is the outcome's effect.

A copy of the amended notice goes back to the operator, and a second copy goes to the archive. The archive keeps both versions, and the pair is what a later inquiry will read.

The outcome is the sixth panel, and the sixth panel is the end of the exchange.

The plate

Panel	Record	Loss
Notice	N-14, 10 March	the regulator's tone
Accusation	four claims	the implication
Evidence	inspection, order, index	the archive's history
Finding	misfiled, not missing	the uncertainty
Response	correction plan	the operator's cost
Outcome	amended, no penalty	the regulator's reasoning

The table is the plate compressed. A reader can see the six panels, the record each panel keeps and the loss each panel declares, and the seeing takes less time than the reading of any one panel.

The loss column is the receipt. Each panel declares what it does not carry, and the declaration is the plate's honesty. A plate with no loss column is a brochure, and a brochure is not a proof.

The record column holds more than one kind of thing. A notice is a document, an accusation is a set of claims, a finding is a resolution and an outcome is a decision, and a plate that holds one kind has flattened the exchange.

The chains meet at the finding. The notice chain runs from the document to the accusation to the evidence, and the response chain runs from the finding to the response to the outcome. The finding is the panel that both sides must read before either can move, and the meeting is the plate.

The table does not show the order of the reading. The regulator reads the accusation first and the evidence second, and the operator reads the evidence first and the accusation

second. The plate is a set of slots rather than a sequence, and each side walks the same slots in its own direction.

The table is also a template. Any notice has these six slots, and a notice whose response has no finding slot is a notice with a gap in the shape of a finding. The template is what makes the plate reusable, and the reuse is what makes it a pattern rather than an anecdote.

The proof

The plate shows that a notice can be answered. The answer is not a denial; it is a claim chain with spans, and the chain is what a regulator can check.

It also names the gaps. A missing evidence span, a missing finding or a missing outcome is a gap, and the gap is the finding. A plate that names its gaps is a plate that can be repaired.

The plate is one notice. It is not a legal standard, and it is not a claim that every notice resolves this cleanly. A notice with a contested fact and no archive index would produce a different plate, and the different plate would need a different finding.

The plate is the architecture's proof because the defense is the architecture. A press release is a claim about the defense; the plate is the defense working. The difference between the two is the difference between saying that a system can answer and showing the answer, and the showing is the only part a regulator, a court or a customer can check.

The notice is a claim that accuses; the next chapter asks what the defense proves.

PART XI
The Model and the Wire

who consumes it, and what the wire carries.

62 · What the Model May Do

The defense proves that a record can answer. The question is who answers for the record, and the answer is a model with a license rather than a model with an opinion.

The answer

A question arrives at the model, and the model does not own the question or the answer. It holds a license to represent the ledger, and the license is the only authority it has.

Two acts are licensed at answer time: report and refuse. A report repeats a claim the ledger already holds. A refusal declines a claim the ledger does not hold, and names what is missing.

An answer that does neither is conversation. Conversation is pleasant and unaccountable, and an accountable system cannot afford it at answer time, because the reader has no way to tell what was licensed from what was merely said.

The license is not a memory. A model that remembers a claim from a previous conversation holds no license for the claim in this one, since the version may have moved and the stance may have changed.

The license is made of five parts. It names a claim, a span, a stance, a version and a projection, and the five are the answer's papers. A model that holds four of the five is not licensed.

The license is per claim, not per conversation. A model may be licensed for the pump's failure and unlicensed for the operator's negligence in the same breath, and the difference is what the answer must show.

The model is the record's voice. It is not the record's author, and it does not become the author by writing more fluently than the record does.

A license is not an inference either. The model may infer that a claim is relevant to a question, and the inference is the model's own work and stays in the model. What crosses into the answer is the claim, and the claim arrived with papers.

The report

The report repeats a claim that the ledger licenses. A question about the pump returns the pump's failure, the date and the span, and it returns nothing else.

```
QUESTION Did pump PB-100-7 fail in March?
ANSWER Yes. The pump failed on 4 March.
 [span: log-2024-03, p.2; stance: asserted; version: v3]
```

The report adds three things and withholds the rest. It adds compression, because the ledger holds more than the answer needs. It adds order, because a person asks in a sequence. It adds register, because the same claim can be said plainly or formally.

The report withholds causality, motive and confidence. The ledger may hold a cause, and the cause is a claim with its own span. The model does not infer a cause from a sequence of events, and it does not attach a confidence the record did not record.

Numbers and dates are reported the way claims are reported, with a span and a version rather than a rounding instinct. A report that says “about four hundred” when the record says 412 has performed an arithmetic the license does not cover, and the arithmetic is invisible to the reader who wanted the figure.

Two claims that conflict are reported as two claims. The model may say that the record holds both, and it may not average them, reconcile them or pick the one that reads better. The conflict is the record’s state, and smoothing the state is a stance upgrade in a costume.

A multi-claim answer is a set of reports. Each sentence carries its own claim and its own citation, and a paragraph is a bundle rather than a unit. A paragraph with one citation and four claims is a paragraph with three uncited claims.

The report is reproducible: the same question against the same version returns the same claim, and the sameness is what a reader can check, while a report that varies with the model’s mood is not a report at all.

The citation

A citation has an anatomy. It names the claim, the source span, the stance, the version and the projection, and each part closes a different question.

A citation is not a footnote. A footnote points at a source the author used; a citation points at the license the answer stands on. The difference is the difference between provenance and permission.

The citation carries the stance because the stance changes the meaning, since a claim that is `asserted` answers a question and the same claim at `candidate` raises one instead. A report that omits the stance has upgraded a candidate into a fact.

The citation carries the version because the version changes the answer. A claim superseded in March answers a question about March, and it does not answer a question about April. A report with no version is a report with no date, and a report with no date cannot be superseded.

The citation names the projection’s loss. A projection drops something, and the citation says what this projection dropped. An answer from a summarization projection that does not say it dropped the dissents is an answer that has hidden its own seams.

A citation the reader cannot check is a decoration. The test is mechanical: can a reader take the span, open the record and find the claim? A citation that fails the test is a claim in costume.

Citation granularity is a design choice with a cost. One citation per sentence is easy to read and easy to over-claim, since a sentence may hold three claims. One citation per clause is exact and unreadable, and the unreadable citation is skipped by the reader who needed it.

The practical rule is one citation per claim, and a sentence is broken when it carries two claims that would need two citations. The break costs a repetition and buys the reader a checkable unit, and the trade favors the reader.

The prohibition

The model may not do six things, and the list is short because a long list is a policy nobody reads.

```
The model may not:
 1 originate a claim, a role or a date
 2 cite a span it did not receive
 3 upgrade a stance (implied -> asserted, candidate -> confirmed)
 4 merge two versions into one sentence
 5 answer past a refusal without a new license
 6 write to the ledger, the dictionary or the closure list
```

Origination is a defect because an originated claim enters no ledger. It has no span, no stance and no version, and it cannot be superseded, retracted or audited. A claim that cannot be superseded is a claim that cannot be corrected, and a system that cannot correct itself is a system that cannot be trusted with a record.

Role and date invention are the same defect in smaller clothes. The fifteen roles are closed, so a model that invents a role has left the grammar. The clocks are recorded, so a model that invents a date has left the record.

A stance upgrade is a finding's work, not an answer's. Moving a claim from `implied` to `asserted` requires evidence and an author, and the answer has neither. A model that upgrades a stance in a sentence has done in prose what only a finding may do on the record.

Merging two versions into one sentence is a quiet way to lose a disambiguation. The reader sees one claim and cannot tell which version it came from, and the merge is invisible in the prose and fatal in the audit.

Answering past a refusal is the sixth prohibition, and it is the one a fluent model breaks most easily. A refusal is a state of the license, not a mood. Once the model has refused a claim, the claim stays refused until a new license arrives.

Writing is the prohibition that needs no explanation. The model has no write path. The compiler has one, and the compiler's write path passes through gates, and the gates are the reason the ledger can be trusted.

The list stops at six because each line is a failure the audit can detect. A longer list would mix prohibitions with preferences, and a preference in a prohibited list is a prohibition the reader will ignore.

Six lines also fit on a desk page. The page is read before a deployment, and a page that fits is a page that is read. A policy that requires a binder is a policy that is filed.

The refusal

The refusal is an output. It carries a reason, a missing license and the nearest claim the record does hold.

```
QUESTION Was the operator negligent?
ANSWER The record does not license that claim. The notice implies it;
 no finding states it. [missing: finding; nearest: N-14]
```

The grammar of a refusal has three parts, and each part is checkable. The first names what is missing. The second names the nearest claim, so the reader can see how close the record came. The third names what would license the claim, so the reader knows what to produce.

The refusal is not an apology. An apology asks the reader to forgive a system; a refusal tells the reader what the system would need. The first is a feeling, and the second is an instruction.

The refusal is not silence. A system that says nothing has refused without a reason, and a refusal without a reason is indistinguishable from a failure. The reader cannot tell whether the record is thin or the service is down.

The refusal has a cost that a team feels before it believes: a reporter that refuses more often is less useful, and the usefulness it loses is real and shows up in the support queue. The cost is the price of not inventing, and a system that pays the price is a system whose reports mean something.

The refusal can also be wrong. A model that refuses a claim the ledger licenses has failed in the other direction, and the overcautious refusal is a defect rather than a virtue. The audit catches both errors, because both leave a sentence that resolves to nothing.

Refusals are recorded. A refusal that repeats across users is a signal that the record is missing something, and the signal is worth more than the individual answer. The refusal log is a map of what the ledger cannot yet say.

A refusal shapes the next turn of the exchange. The reader who is told what would license the claim can produce it, and the reader who is told nothing can only ask again. The first exchange moves, and the second loops.

The loss

An answer is a projection, and a projection has a loss. The loss is what the projection dropped, and the answer declares it rather than hiding it.

Three losses matter most. The loss of conflict is dropping a disagreement the ledger holds. The loss of dissent is dropping the claim that contested the majority. The loss of supersession is dropping the fact that a newer version exists.

The loss depends on the audience. An internal answer may carry the conflict because the reader can adjudicate it. An external answer may drop the conflict and must say that it dropped it, because the reader cannot.

The loss must be stated, not implied. A register that is smooth implies that nothing was dropped, and the smoothness is the lie. A short declaration of loss costs a sentence and buys the answer its standing.

Consider a question answered from version two when version three exists: the report is accurate for that version, the citation names it, and the loss line says that version three supersedes it. The reader now knows what they have.

The loss is what makes the answer auditable rather than authoritative. An answer that declares its loss can be checked against the record and improved. An answer that declares nothing can only be believed or disbelieved.

A loss line is a sentence, not a paragraph. "Answered from v3; conflict C-212 omitted" is enough for a reader to know what they hold. A loss essay would be more complete and would be skipped, and the skipped loss is the same as no loss.

The audit

The audit walks every sentence of an answer back to a claim. A sentence that resolves to a claim is a report. A sentence that resolves to nothing is a failure, and the failure is the one the audit exists to find, since everything else can be checked by a reader with patience.

The audit is mechanical. It needs the answer, the license and the ledger, and it needs no judgment about whether the answer is good. The mechanics are the point: an audit that requires taste is an audit that cannot be delegated to a script.

The audit catches the failures the prohibitions name. A silently originated claim has no span. A stance drift resolves to a claim at the wrong stance. A version merge resolves to two claims in one sentence, and the audit reports the pair.

The audit cannot catch everything. A licensed claim can be false, and the audit will pass it, because the audit checks the license and not the world. A bad ledger produces licensed answers, and no answer-level audit can repair a record-level defect.

The audit produces a report card. It counts the sentences, the resolved, the refused and the unresolved, and the counts are the answer's health. An answer with four unresolved sentences is not a weaker answer; it is a failed one.

The report card is comparable across answers. Two models answering the same question produce two cards, and the cards say which model stayed inside the license. The comparison is fair because the standard is the record, and the record is the same for both.

The audit can run before the answer is sent or after. Before is a gate, and after is a postmortem. A system that audits only after is a system that has decided the reader is the auditor, and the reader did not agree to the job.

The limit

Reporting is not understanding: a model that reports a claim has not understood it, and the difference matters when the question requires a judgment the record does not hold, since the record cannot license a judgment it never made.

The model's fluency is not a warrant. A sentence that sounds authoritative is a sentence that has borrowed authority from its rhythm, and the borrowing is exactly what the citation is meant to prevent.

The pattern cannot fix a bad ledger. A license drawn from a thin record produces a licensed answer that is thin, and the thinness is the record's, not the reporter's. The remedy is upstream, in what the ledger holds and how it was compiled.

The model's memory is not a record either. A model that has seen the claim in training or in a prior conversation has no span for it, and a claim with no span is a claim the answer cannot cite. The memory may help the model find the claim, and the search is the next subject.

The reporter is not a lawyer, a journalist or an expert. It is a mouth with a license, and the license is narrow on purpose. A mouth that stays narrow can be trusted with a record; a mouth that widens is a second author.

What remains is the search. A model that may only report still has to find the claim before it can cite it, and the search is retrieval's work.

A model that may only report still has to find; the next chapter asks what retrieval may fetch.

63 · Retrieval, Nearness and Citation

The model may only report what it finds, and retrieval is the finding. The question is what retrieval may fetch and what it may claim about what it fetched.

The question

A question arrives in a person's words, and the words are not yet a query; the first job is to turn the question into a target the record can be searched against.

The target has three parts. It names the terms the question uses, the time the question means and the kind of claim that would answer it; a question about the pump's failure in March needs a failure claim, a March scope and the pump as its theme.

The terms are the hard part, because a person says "the pump" and the record holds `PB-100-7`, and a person says "broke" while the record holds `FAIL`. The translation from words to roles is a mapping that belongs in the question record rather than in the model's mood.

The time is the second part, since a question in the present tense may mean the current version and a question in the past tense may mean the version that held then, and retrieval that ignores tense returns the wrong version with the right words.

The kind is the third part, and a question that asks whether something happened needs an event claim while a question that asks what something is needs an attribute claim, so a retrieval that returns an attribute for an event question has answered a different question.

The question record is written down before the walk begins. A record written after the answer is a rationalization, and a rationalization cannot be audited.

An ambiguous question stops the process at the record. Two mappings are proposed, the record shows both, and the system either asks which one the person meant or refuses with the pair attached. Guessing is the third option, and the guess is invisible in the answer.

The fetch

The fetch returns candidates. A candidate is not an answer, and the difference is the whole discipline of retrieval.

```
CANDIDATE c-881
claim FAIL(theme=pump, date=4 March)
license span log-2024-03 p.2; stance asserted; version v3
distance 0.12 (nearest first)
path question -> pump -> maintenance -> FAIL
```

A candidate carries five fields, and each field closes a question. The claim says what the record holds. The license says whether the record licenses it. The distance says how near it is to the question. The path says how the walk arrived. The loss says what the walk skipped.

The loss field is filled during the walk rather than after it. The walker knows which candidates it passed over, and the knowing is cheap at the moment and expensive to reconstruct. A loss field filled in afterwards is a summary, and a summary of a loss is the loss already forgotten.

The fetch returns more than one candidate and fewer than all of them, because a single candidate is a search that stopped early and the whole corpus is a search that never started, and the number is bounded by the question's scope and by the walk's hop limit.

The fetch does not rank into a single answer. Ranking is a separate act, and the separation matters because a rank computed during the fetch inherits the fetch's assumptions. A rank computed afterwards can be checked against the question record.

The fetch is repeatable. The same question record against the same version returns the same candidate set, and the same set is what makes the citation stable. A fetch that varies with load or with a prompt is a fetch that cannot be cited.

The candidate count is a budget rather than a maximum. A question that returns two hundred candidates has answered nothing, and a question that returns one has stopped early. The budget is set in the question record, and the count that exceeds it is a signal that the mapping is too broad.

The expansion

Expansion walks the ledger's joins. There are three, and the walk uses them in the order the question requires.

```
NEST the claim inside its episode
CONTAIN the episode inside the quarter
RELATE the claim beside the repair order
forbidden: "pump" -> "hydraulics" (association not in the ledger)
```

The first join is NEST, since a claim sits inside an episode and the episode carries the participants and the time, which is what a claim alone does not carry. A walk that stops at the claim has the claim, and a walk that steps to the episode has the context the claim was made in.

The second join is CONTAIN, where an episode sits inside a period and the period is what makes a question about a quarter answerable rather than merely textual. The walk to the period is a walk to scope, and scope is what keeps a March question from returning an April claim.

The third join is RELATE, where a claim sits beside another claim and the beside is a recorded link rather than a similarity, which is the difference retrieval rests on. The repair order sits beside the failure, and the beside is a recorded link rather than a similarity. A walk that follows a similarity has left the ledger.

The closure bounds the walk, and the closure is the set of nodes the ledger reaches from a starting claim through licensed joins, so a candidate outside the closure is a candidate the record does not connect, and connecting it is the model's invention.

The hop limit bounds the walk's cost. Each hop multiplies the reachable set, and the multiplication is why a three-hop walk is affordable and a seven-hop walk is not. The limit is declared in the question record, and a walk that exceeds it returns a refusal rather than a longer list.

The forbidden edge is the instructive case. "Pump" and "hydraulics" are associated in every engineer's mind and in no ledger row, and a walk that crosses the association has produced a claim the record never made. The association is knowledge, and knowledge is not a join.

Hop cost is the reason the limit is declared rather than discovered. One hop is cheap, two hops are affordable, and three hops are the point where a query becomes a job. A team that lets the walk run without a budget learns the cost from the latency graph, and the latency graph is a bad place to learn it.

The nearness

Nearness ranks the candidates that the walk returned. It does not license them, and the difference is the central correction of retrieval.

A distance is a number, and the number says how close the candidate's text is to the question's text. Closeness is a property of the words, and licensing is a property of the record. A candidate can be close and unlicensed, and a candidate can be licensed and dull.

Two candidates make the point. The maintenance log is close to the question and licensed. A safety bulletin that mentions pumps in passing is closer still and holds no failure claim. The bulletin ranks first and answers nothing.

The rank is useful anyway, because a walk over a large corpus returns more candidates than an answer can carry, and the rank is how the list is cut, and the cut is declared in the loss line.

The rank is also where a team is tempted to cheat. A score tuned until the right passage wins is a warrant invented after the fact, and the invention is invisible in the score. The guard is the question record: a score computed against a recorded question can be checked, and a score computed against the answer cannot.

Nearness also has a use in the other direction. A licensed candidate that ranks far down is a signal that the question record is mistranslated, since the record holds an answer the words did not reach. The signal is worth more than the answer, because it points at the mapping.

The rank is also unstable across model versions. Two embedding models produce two orders for the same candidates, and the orders disagree most on the candidates near the cut. The loss line records the cut, and the recorded cut is what lets a reader see which model's order was used.

The screwdriver

The screwdriver is the case where nearness fails and the walk succeeds. The question is whether the pump failed before the seal was replaced, and the answer is a comparison.

```
Question: "Did the pump fail before the seal was replaced?"
Retrieval returns: FAIL (4 March), WORK_ORDER 881 (7 March).
The answer is a comparison of two licensed claims, not a similarity hit.
```

No single passage answers the question. The log holds the failure, the work order holds the replacement, and the order is what the comparison needs. The walk reaches both through RELATE, and the comparison is the model's arithmetic over two licensed claims.

The comparison is licensed when both claims are licensed and the relation between their dates is recorded. The model may say that 4 March precedes 7 March, and it may not say that the failure caused the replacement, since causality needs its own claim.

The screwdriver is also a naming lesson. The tool that turns the screw is not the screw, and the retrieval that returns the two claims is not the comparison. A team that conflates the two builds a search box and calls it an answer.

The case generalizes to every question with a "before", an "after" or a "because". Each needs two claims and a relation, and each fails when the retrieval returns one passage that mentions both terms.

The screwdriver is the cheapest demonstration that retrieval is not search. The words of the question appear in neither the log nor the order, and the answer is still in the record.

A second case looks the same and is not. The question is whether the repair cost more than the budget, and the record holds the order's amount and the budget's line. The comparison is arithmetic over two licensed claims, and the arithmetic is the model's work. Neither case is answered by a passage, and both are answered by a walk.

The citation

The citation carries the path, the version and the loss. A citation without a path is an assertion with a footnote, and a footnote is not a path.

```
walked: q -> pump -> maintenance -> FAIL (3 hops)
version: v3
loss: 41 candidates below 0.30; 2 candidates at v1 (superseded)
```

The path is the walk's route, and the route is checkable because a reader who has the path can repeat the walk and see the same candidates, and the repetition is what makes the citation a citation rather than a claim about a citation.

The version is the record's state at the walk. A path walked against version three and reported against version four is a path that no longer exists, and the citation must say which version it walked.

The loss is what the walk skipped. The cut below a distance, the superseded versions and the dissenting claims are the loss, and the loss line names them. A citation that names the path and hides the loss has described the route and not the country.

The citation is written for a reader who will not repeat the walk. The reader wants to know what the answer stands on and what it left out, and the citation gives both in three lines. A citation written for the walker is a log, and a log is not a citation.

The citation's test is the same as the answer's. A reader takes the path, opens the version and finds the claims, and the finding is the proof. A citation that fails the test is a decoration on an assertion.

A spoken answer still needs the citation, and speech is where the citation is most often dropped. The model may say the answer and then say the version, and the path can travel as a link or as a reference in the interface. A spoken answer with no path is a rumor with good manners.

The loss of the walk

Every walk skips something, and the loss line is where the skipping is declared. The declaration is not a confession; it is the answer's scope.

The first loss is the cut, because a walk returns more candidates than an answer can carry and the cut keeps the nearest by a threshold the loss line records. The cut is a distance threshold, and the threshold is a choice the loss line records.

The second loss is the supersession. A walk over a version returns the version's claims, and the earlier versions are skipped. The skip is correct for a question about the current state and wrong for a question about the history, and the loss line tells the reader which question was answered.

The third loss is the dissent. A claim that contested the majority is a candidate like any other, and the rank may push it below the cut. The dissent's absence from the answer is a loss, and the loss line is where the absence is named.

The loss line is short by design. "41 below 0.30; 2 at VI" is enough for a reader to know the answer's boundaries, and a longer account would be skipped. The short line is the one that survives a reader in a hurry.

The loss line also protects the team. A dispute about an answer becomes a dispute about the loss line, and the loss line is a record rather than a memory. The team can show what it skipped and why, and the showing is the defense.

The historical question has the largest loss. A question about why the pump was replaced in March needs the versions that held before the replacement, and the current version has superseded them. The walk must reach the earlier versions deliberately, and the loss line must say that the walk did so.

The limit

Retrieval cannot fix a bad index. A walk over a record that never linked the work order returns the failure and misses the comparison, and no hop limit repairs a missing join.

Retrieval cannot fix a bad question record. A question mapped to the wrong role returns licensed candidates for the wrong question, and the candidates are licensed and useless. The mapping is upstream of the walk, and the walk cannot see the error.

Retrieval is slower than search. The walk follows joins, checks licenses and records losses, and each of the three costs time. The slowness is the price of a citation, and a team that wants the citation pays the price.

Retrieval is also less fluent than search. The candidates arrive as claims with papers, and the model must assemble them into sentences. The assembly is the model's work, and the papers are what keep the assembly honest.

A latency budget belongs in the question record. A three-hop walk at interactive speed is a different design from a three-hop walk in a batch job, and the budget is what decides which one the team built. A team that never writes the budget discovers it in production.

What remains is the message. The answer is assembled, cited and declared, and the next question is what travels when the answer leaves the building.

Retrieval finds what the record licenses; the next chapter asks what a message carries when it leaves.

64 · The Wire

An answer that stays inside one ledger can be checked by reading the ledger. An answer that crosses to another organization needs a form that travels, and the form is the envelope.

The envelope

An envelope has four parts, and each part answers a question the receiver will ask. The terms say who sent it and what may be done with it. The claims say what it asserts. The dictionary segment says what the private words mean. The state says whether the receiver accepted it.

```
ENVELOPE e-4471
terms sender: org A; receiver: org B; purpose: audit; expiry: 30 days
claims 3 (2 public by reference, 1 private by value)
dictionary 4 entries (pump, seal, crew, archive)
state closed
```

The envelope is not a wrapper around a payload. A wrapper can be removed and the payload still read, and the envelope's terms cannot be removed without turning the content into something the receiver may not use.

The four parts are ordered by the receiver's work. The receiver does the work. The receiver reads the terms first, because the terms say whether reading is permitted. The receiver reads the claims second, the dictionary third and the state last, and the state is the receiver's own verdict rather than the sender's.

The state is the one part the sender cannot set. A sender may propose `open`, and the receiver writes `closed` or `quarantined`. A sender-set state is a claim about the receiver's behavior, and no sender has standing to make it.

State	Written by	Entry condition	Becomes
open	sender, as a proposal	terms, claims and segment are written	closed or quarantined, at the receiver

Continued on next page

(Continued)

State	Written by	Entry condition	Becomes
closed	receiver	every claim verifies: a reference resolves and its digest matches, and a private claim carries its span, stance and sender	reopened when a digest fails or a version moves
quarantined	receiver	at least one claim does not verify	held until a full re-send or a named release; never partially read
reopened	receiver	a closed envelope's digest fails or its version moves	closed again, as a new state rather than an erasure

The table is the whole transition set. A state not on it is a state the receiver invented, and a receiver who invents a state has invented a verdict nobody granted.

The envelope is versioned like any other record. A second envelope that supersedes the first carries the first's identifier, and the receiver keeps both. The pair is what lets a later dispute reconstruct what was sent and what was accepted.

The envelope is small on purpose, four parts on one page with no attachments, because a form that needs an appendix is a form that will be filled incorrectly.

The receiver keeps a copy. The copy is the receiver's record of what was sent and what was accepted, and the copy is what a later auditor reads instead of the sender's word. A receiver who accepts an envelope and keeps no copy has accepted a claim they cannot reconstruct.

The terms

The terms are five fields, sender, receiver, purpose, expiry and handling, and the five are the minimum a receiver needs to act, since each field closes a question the receiver would otherwise ask by email.

The sender is an organization rather than a person, though a person may sign the envelope and the signature is a separate record, since the organization is the party the receiver can hold to account.

The receiver is named rather than implied, because an envelope addressed to "whom it may concern" has no receiver, and a receiver who is not named cannot be bound by the terms. The naming is what turns a broadcast into an exchange.

The purpose is a sentence. "Audit of the March pump failure" is a purpose, and "business purposes" is not. A purpose that could cover anything covers nothing, and the receiver who accepts it has accepted an unlimited use.

The expiry is a date, and the date is what makes the terms temporary. An envelope with no expiry is a permanent grant, and a permanent grant is a grant nobody would sign if they read it.

The handling is a short list. May be quoted, may be summarized internally, may not be shown to a third party. The list is not cryptography and does not pretend to be. It is an obligation the receiver records, and the record is what makes the obligation visible.

The handling list is checked at the point of use rather than at the point of receipt. A receiver who accepts an envelope may quote it, summarize it or ignore it, and the three acts are governed by three different entries. The check is a habit rather than a control, and the habit is what an audit samples.

The terms travel with the claims and are not attached to them. A claim quoted without its terms is a claim whose permissions have been stripped, and the stripping is the most common failure on the wire.

The reference

A public claim travels by reference. The sender does not copy the claim; the sender names it, and the receiver reads it from a ledger both parties can reach.

The reference has three parts. It names the claim's identifier, the version the sender read and a digest of the claim's content. The identifier locates the claim, the version fixes the state and the digest detects a substitution.

The reference is cheap, since a hundred claims travel as a hundred lines and the lines are small, and the cheapness is why a public claim is sent by reference rather than by value and also why a receiver can ask for more.

The reference is checkable. A receiver who holds the ledger reads the claim and compares the digest, and the comparison is a mechanical test. A reference that fails the digest is a reference to a claim that changed after the sender read it.

The digest is replaceable and the reference is not. The digest is not optional. A team that changes hash functions changes every envelope's digest field, and the change is a migration rather than a detail. The identifier and the version survive the migration, and the digest is the part that has to be recomputed.

The reference fails when the ledger is unreachable. A receiver without access has a pointer to nothing, and the pointer is worse than no message because it looks like content. The failure is why the terms name a fallback, and the fallback is the value form.

The reference also fails when the claim is private. A private claim has no public identifier, and a reference to a private claim would name something the receiver cannot read. The private claim travels by value, and the value is the next section's subject.

The reference is the default for public claims, and the default is declared. An envelope that sends public claims by value has paid bytes for no gain, and the payment is a signal that the sender did not understand the form.

The value

A private claim travels by value, and the value is the claim written out with its provenance. The sender copies the claim because the receiver cannot read the ledger it lives in.

The value form carries four things. The claim's text, the span inside the sender's record, the stance and the sender's name. The four are what the receiver needs to treat the claim as evidence without access to the sender's store.

The span is a page and a paragraph rather than an identifier. The receiver cannot dereference an identifier in a ledger they do not hold, and the page is the closest thing to a dereferenceable address that travels.

The value form is expensive, because a private claim costs bytes and a hundred private claims cost a document, and the expense is why private claims are sent sparingly rather than collated.

The value form is also weaker evidence. A copied claim can be altered, and the receiver has no ledger to check it against. The weakness is the reason the value form carries the sender's name and the span, since the two give the receiver something to ask about.

The value form keeps the stance and loses the machinery. A private claim arrives as `asserted` or `candidate`, and the receiver sees the stance without the evidence that produced it. The loss is the cost of crossing a boundary, and the span is what tells the receiver how much of the evidence did not travel.

The value form must not be re-referenced. A receiver who copies the value into their own ledger and then points at the copy has created a claim whose provenance is a copy of a copy. The chain of custody is the thing the value form was meant to preserve.

Private claims travel by value and are marked private. The marking is what tells the receiver that the claim may not be quoted, and the marking is a term rather than a field in the claim.

The dictionary segment

The dictionary segment names the private terms that the claims use. Without it, the receiver reads the words and guesses the meanings, and the guesses are where cross-organization answers fail.

```
pump -> PB-100-7 (public canonical id)
seal -> "shaft seal, part 44-2" (private description, org A)
crew -> "crew B" (org A's naming)
archive-> "records room 3" (org A's naming)
```

An entry has two parts: the term and its referent. The term is the word the claims use, and the referent is what the word names. The referent may be a canonical identifier when the thing is public and a description when the thing is private.

The public entry is a bridge, since `pump -> PB-100-7` tells the receiver that the sender's "pump" is the thing both organizations already name the same way, and the bridge is what lets the claims be compared across the wire.

The private entry is a definition. `seal -> "shaft seal, part 44-2"` tells the receiver what the sender means, and the meaning is the sender's rather than the world's. The receiver may accept the definition for the purposes of the envelope and may not treat it as a public fact.

The segment is versioned. A term whose meaning changes gets a new entry and a new segment version, and the envelope names the version it used. A segment without a version is a dictionary that silently redefines its words.

The segment is small, with four entries for three claims as a normal ratio, and a segment with fifty entries for three claims is a sign that the claims are carrying their own glossary. The glossary belongs in the sender's dictionary, and the segment carries only what the claims use.

The segment is the part partners ask for first. A receiver can read claims without terms; the receiver cannot read claims without meanings. The dictionary segment is what makes the envelope legible, and the legibility is why it travels at all.

The closure

Closure is the receiver's verdict that every claim in the envelope can be verified. The receiver walks the claims, checks each license, and writes the state when the walk finishes.

A public claim closes when the reference resolves and the digest matches. A private claim closes when it carries its span, its stance and its sender. A dictionary entry closes when its referent is either a canonical identifier or a description the receiver accepts for the purpose.

Closure is all or nothing, since a receiver who closes an envelope with one unverified claim has accepted a claim they did not check, and the acceptance is invisible in the state, which is a claim about the whole envelope that no partial check can support.

Closure is recorded with its reason. A closed envelope carries the list of checks that passed, and the list is what a later auditor reads. A closed envelope with no reason is a state without a warrant.

Closure has a latency, and the latency is part of the terms. A receiver who needs three days to check ten references has closed the envelope late, and the lateness is visible in the state's timestamp. An exchange that requires same-day closure needs fewer references or a faster ledger.

Closure is per envelope rather than per claim. The same claim may close in one envelope and be quarantined in another, because the receiver's access and the purpose differ. The per-envelope state is what keeps the verdict honest.

Closure is reversible, since a closed envelope can be reopened when a digest fails or a version moves, and the reopening is a new state rather than an erasure. The history of states is the envelope's record, and the record is what the receiver keeps.

The quarantine

Quarantine is the receiver's verdict that at least one claim cannot be verified. The envelope is held rather than partially accepted, and the hold is recorded with the reason.

```
ENVELOPE e-4472 state: quarantined
reason: claim 2 carries no span in org A's ledger
action: held; sender notified; no partial acceptance
```

The hold is the envelope's, not the claim's. The receiver could accept the verified claims and drop the unverified one, and the drop would silently change what the sender sent. The envelope is the unit because the claims were sent as an argument, and an argument with a premise removed is a different argument.

Quarantine is not deletion: the envelope stays, the state is written and the sender is notified, and a deleted envelope leaves no record of the refusal.

Quarantine names the failing claim. The reason is specific enough for the sender to fix, and a reason that says "invalid" is a reason that starts a support ticket rather than a correction.

The notification is a short document. It carries the envelope's identifier, the failing claim, the missing license and the date by which the sender may resend. A notification without the four leaves the sender guessing, and the guessing costs a second exchange.

Quarantine has a time limit. An envelope held past its expiry is closed by the passage of time, and the closure is recorded as an expiry rather than an acceptance. The limit is what keeps the queue from becoming a graveyard.

Quarantine is also a signal. A sender whose envelopes are quarantined repeatedly has a ledger problem rather than a wire problem, and the pattern is visible only when the states are recorded. The recording is what turns individual refusals into a diagnosis.

Quarantine is a state a sender can prevent. A span, a stance and a sender name are cheap to include, and the three are what most quarantines ask for. The prevention is cheaper than the exchange that follows the hold.

The limit

The envelope is not security. It records terms and obligations, and it does not encrypt, authenticate or prevent copying. A receiver who ignores the terms has violated an obligation rather than defeated a control.

The envelope is not a protocol. It says what a message carries, and it does not say how the message travels. The transport may be a queue, a file or a hand-carried drive, and the envelope is the same in all three.

The envelope costs bytes and a round trip. A public claim by reference is cheap, and a private claim by value is expensive, and the total is more than a payload. The cost is the price of a refusal that can be explained.

The envelope also costs a habit. A team that has sent JSON for a decade will send JSON, and the envelope will be a schema nobody fills in. The habit changes when a partner asks why the last answer could not be checked, and the question is the beginning of the change.

An audit of the wire reads envelopes rather than answers. It counts the envelopes sent, closed and quarantined, and it reads the reasons for the holds. A wire with no quarantines is a wire that is not checking, and a wire with nothing but quarantines is a wire that is not sending.

What the envelope makes possible is counting. A closed envelope is a unit that can be counted, compared and audited, and the counting is the next subject.

A message that carries its terms can be counted; the next chapter asks which questions become cheap.

PART XII
What This Makes Possible

what it makes cheap, and what it does not unlock.

65 · The Questions That Become Cheap

A question is cheap when the record already holds the field that answers it. Most of the questions a team asks are expensive for a reason that has nothing to do with difficulty.

The cheap question

A cheap question is answered from the ledger without new compilation. The answer is a query, a row returned with its span, and the query runs in the time a person will wait.

An expensive question is answered by a project. Someone commissions an analysis, someone reads documents, someone writes a memo, and the memo is the answer. The memo is expensive because the record did not hold the fields the question needed.

The distinction is not about intellect, since the same person asks a cheap question and an expensive one in the same meeting, and the difference is whether the record anticipated the question. A record that holds a stance can answer "how strongly" in a row. A record that holds only text can answer it in a week.

Cheapness is a property of the schema rather than of the model. A capable model asked a question the schema cannot answer will produce a fluent guess, and the guess is cheap to generate and expensive to defend. A schema that holds the field makes the question cheap for any model, including a dull one.

Cheapness is also a property of the question's shape. A question that names a claim, a role and a time is shaped like a row. A question that asks for a judgment is shaped like an argument, and no schema answers it in a row.

Cheapness is relative to the asker's urgency as well. A question that must be answered before a shipment leaves is expensive at any speed above a minute, and a question that can wait a week is cheap at any speed below a day. The same query is cheap for one asker and expensive for another, and the difference is the clock at the asker's end.

The test is the meeting. A team that asks six questions and answers three from the record has a cheap three and an expensive three, and the expensive three are the schema's to-do list.

The query log is the other evidence. A system that answers two hundred cheap questions a week is a system whose schema anticipated the week, and a system that

answers two is a system whose users have stopped asking. The log is a fairer measure than the meeting because it records demand rather than intention.

The twelve

Twelve questions become cheap when the record holds eleven features, and the twelve are the questions teams actually ask.

```
 1 What does the record say about X? claim
 2 Who says it? authority
 3 How strongly? stance
 4 Where is it written? span
 5 When was it true? clock
 6 What changed since? version
 7 What was withdrawn? acts
 8 What is missing? gap
 9 What conflicts? conflict
10 What did the answer leave out? loss
11 What would license the rest? refusal
12 What may I do with it? terms
```

The first question is the plain one. It asks what the record holds about a theme, and the answer is a set of claims with roles and verbs. The question is cheap because the claim is the record's unit.

The second question asks who says it. The answer is the authority record behind the claim, and the question is cheap because the authority is a field rather than a paragraph. A record without an authority field answers the question with a shrug.

The third question asks how strongly, and the answer is the stance, which is a small closed set that the record can require rather than infer. The question is cheap because the set is small and the field is required.

The fourth question asks where it is written. The answer is the span, and the span is the address that makes every other answer checkable. A record with claims and no spans answers four of the twelve and cannot support the rest.

The fifth question asks when it was true. The answer is the valid-time clock, and the clock is a field rather than a narrative. A record that stores a date inside a sentence answers the question by reading the sentence, which is cheap for one claim and expensive for a thousand.

The sixth question asks what changed. The answer is the supersession row, and the row is the join between the old version and the new. A record that overwrites instead of superseding answers the question with silence, and the silence is indistinguishable from "nothing changed".

The seventh question asks what was withdrawn. The answer is the retraction record, and the record is separate from the supersession because withdrawal and correction

are different acts. A record that conflates the two answers the question with the wrong history.

The eighth question asks what is missing. The answer is the gap table, and the gap table is the record of what the compiler could not license. A record with no gap table answers the question by claiming completeness, and the claim is the most expensive kind of wrong.

The gap table is the only one of the eleven that is a record about the record. The other ten describe the world; the gap describes the ledger's own edge. A team that reads the gap table learns what its pipeline cannot extract, which is a different and more useful lesson than what it can.

The ninth question asks what conflicts. The answer is the conflict set, and the set is the claims that disagree about the same thing at the same time. A record that resolves conflicts on entry answers the question with the winner and loses the disagreement.

The tenth question asks what the answer left out. The answer is the projection's loss line, and the loss line is what makes the answer's scope visible. A record with no loss line answers the question by presenting a complete surface over an incomplete projection.

The eleventh question asks what would license the rest. The answer is the refusal's missing-license field, and the field is what turns a refusal into an instruction. A record with refusals and no reasons answers the question with a shrug in a different costume.

The twelfth question asks what may be done with the material. The answer is the envelope's terms, and the terms are what travel with the claims. A record that has no terms answers the question by assuming the widest permission, and the assumption is the one no sender would sign.

The twelfth is the question that matters most across a boundary. Inside one organization the terms are a policy, and outside it the terms are the message. A team that never sends claims to another organization can defer the twelfth, and a team that does cannot defer it past the first exchange.

Eleven features answer twelve questions because the first question is answered by the claim itself. The count is a coincidence of the list rather than a design, and the design is the mapping.

The list is ordered by dependency rather than by importance. A claim is required before a span, a span before a version and a version before a retraction, and a team that builds the list from the bottom up builds the schema from the foundation rather than from the reports.

The mapping

Each cheap question maps to one recorded feature, and the mapping is the artifact a team can check. A question with no field is a question the schema has not anticipated.

```
feature question query shape
claim 1 match on theme, role, verb
authority 2 join to the authority record
stance 3 filter on stance
span 4 select span, page, paragraph
clock 5 filter on valid-time
version 6 join to supersession row
acts 7 join to retraction record
gap 8 query the gap table
conflict 9 query the conflict set
loss 10 read the projection's loss line
refusal 11 read the refusal's missing license
terms 12 read the envelope's terms
```

The mapping has a direction. The feature is designed first, and the question becomes cheap as a consequence. A team that starts from the question and invents a field per question builds a schema with twelve special cases, and the special cases do not compose.

The mapping is also a test of the field's necessity. A field that answers no question is a field nobody queries, and a field nobody queries is a cost with no benefit. The twelve questions are the justification for eleven columns, and the justification is checkable.

The mapping fails in one direction only. A question can be cheap and its answer wrong, because the field can hold a bad value. The mapping says nothing about the quality of the value, and the quality is the compiler's business rather than the schema's.

The expensive four

Four questions stay expensive, and naming them is part of the spread's honesty. Causality, intent, counterfactual and value have no field, and each needs an argument.

Causality asks why something happened. The record can hold a cause claim with its own span, and the cause claim is one claim among several. The question asks which of the competing causes is true, and no field answers a question about truth.

The pump makes the point concrete. The record holds the seal erosion, the failed inspection and the flooded floor, and each is a claim with a span. The question of why the pump failed needs a mechanism that links the three, and the mechanism is an argument the record can support and cannot supply.

Intent asks what someone meant to do. The record can hold a statement of intent, and the statement is testimony. The question asks whether the statement is sincere, and sincerity is not a field.

The counterfactual asks what would have happened otherwise. The record holds what happened, and the counterfactual is a model over the record. The model may be licensed by the record's regularities, and the license is an argument rather than a row.

Value asks what the answer is worth. The record holds costs and outcomes, and worth is a judgment about them. The judgment can be supported by the record and cannot be read from it.

The four are not failures of the schema. They are the questions that need a person, and a schema that pretended to answer them would be a schema that hid its judgments behind columns.

The spread as a test

The spread is a diagnostic. A team asks the twelve questions about its own domain and marks the ones it cannot answer from the record, and each mark is a missing column.

The test is cheap to run. Twelve questions, one meeting, one list of marks. A team that runs it before a build knows what its schema is for, and a team that runs it after a build knows what it forgot.

The marks cluster. A team that cannot answer questions two, three and eleven has not recorded authority, stance or refusals, and the three failures are one failure in different clothes: the record holds claims without their papers.

A cluster is a diagnosis rather than a score. Three misses in a row usually mean one missing intake step, and a team that fixes the step closes three marks at once. Twelve scattered misses usually mean the schema was never designed, and the fix is a redesign rather than a column.

The test also produces an order. A mark on question four is more urgent than a mark on question ten, because spans support every other answer and loss lines support only the tenth. The order is the schema's build order.

The test is not a score, because a team that answers nine of twelve is not sixty percent done, and the three it misses may be the three its questions depend on. The marks are a map rather than a grade.

The cost

The eleven features cost compilation. A stance must be assigned, a span must be captured, an authority must be resolved and a version must be recorded, and each of the four is work at intake rather than at query time.

The cost is paid once and the cheapness is collected repeatedly. A team that assigns a stance per claim pays a moment per claim and answers the third question for the life of the ledger. A team that skips the assignment pays nothing and answers the question with a project, every time.

The cost is also a discipline. A field that is optional will be empty, and an empty field answers its question the same way a missing field does. The features that answer the twelve questions must be required, and the requirement is the schema's real commitment.

The requirement has a cost of its own, and the cost is friction at intake. A compiler that refuses a claim without a span refuses more claims, and the refusals slow the pipeline in the week they are introduced. The slowdown is the price of the twelfth question being cheap next year.

The cost falls on the compiler, since the gates that assign stance, capture spans and resolve authority are the compiler's gates, and a thin compiler produces a ledger whose fields are present and empty. The spread measures the schema, and the schema measures the compiler.

The limit

A cheap question is not a true answer. The record can hold a claim that is false, a stance that is wrong and a span that is real, and the query returns all three. The cheapness is about access rather than about truth.

The spread is also domain-bound. The twelve questions are the ones a knowledge team asks about claims, and a team asking about images or signals has a different twelve. The pattern is the mapping discipline rather than the specific list.

The list also moves as the questions move. A team that adds a jurisdiction question adds a field for it, and the thirteenth question becomes cheap the year the field is required. The spread is a snapshot of a discipline rather than a canon.

Cheapness can be counterfeited. A team can answer the twelve from a spreadsheet of summaries, and the answers will be fast and unsupported. The spread tests the schema only when the schema is the source, and the source is a fact about the team rather than about the list.

The counterfeit is worth naming because it is common. A team builds a cache of answers, the cache ages, and the twelve stay cheap until the first answer contradicts a record nobody refreshed. The cache is not the schema, and the difference shows up as an answer that cites a span which no longer holds the claim.

The twelve also presume a compiler. Without the gates, the features are columns that someone fills in by hand, and hand-filled columns answer the questions until the first busy week. The cheapness is downstream of the discipline that made the fields reliable.

The spread is also a teaching device. A new member of a team learns the schema faster from twelve questions than from a list of columns, because each question names a reason for the field. The reasons are what make the schema memorable, and a memorable schema is one that gets used.

A cheap question is not a true answer; the next chapter asks what this does not unlock.

66 · What This Does Not Unlock

A benefit list without a limit list is a sales document. The architecture delivers less than its advocates claim and more than its critics concede, and the difference is worth stating in advance.

The list

Seven things are named here, and each is something a buyer expects and this architecture does not deliver. Truth, consensus, clean data, zero cost, wisdom, grounded action and security are the seven.

```
truth licensed, not verified against the world
consensus disagreement recorded, not dissolved
clean data gaps priced, not removed
zero cost compilation costs, and the cost is recorded
wisdom checkable answers, not good judgment
grounded action licensed claims, not safe acts
security obligations recorded, not enforced
```

The list is not a confession, since a limit named in advance is a commitment and a limit discovered later is an excuse, and the difference is the meeting in which the buyer learns it.

The list is also asymmetric with the benefits. The architecture licenses a claim, records a conflict, prices a gap and makes an answer checkable, and each of the four is a real gain. None of the four makes the claim true, settles the conflict, fills the gap or makes the answer wise.

The list is short on purpose. Seven limits are the ones a team will meet in its first year, and a longer list would be a document nobody reads. The seven are the ones that change a plan.

The list is written for the person who has to defend the purchase. A buyer who can say what the system does not do is a buyer who will not be surprised, and the surprise is what costs the second year.

The limit list belongs beside the benefit list rather than after it. A limit placed after the benefits reads as a caveat, and a limit placed beside them reads as a specification.

A business case can carry the list in one column. The column has seven rows, the benefit column has four, and the two together are a page a sponsor can read before signing. The page is the point. A one-column case is the case that gets renegotiated.

Truth

The architecture licenses claims, and a licensed claim is not a true claim. Licensing says the record holds the claim with a span, a stance and a version; truth says the world agrees.

The gap between the two is the world. A claim can be licensed and false, and the ledger will hold it with perfect fidelity. The ledger's fidelity is to the record, and the record is a set of statements somebody made.

The chain of checking has a length. A span points at a document, the document points at an author, and the author points at an observation, and each link is checkable and none is self-verifying. The architecture builds the links and cannot close the last one, because the last one is the world.

The architecture helps with truth in one way, because a claim with a span can be checked against the source, and the checking is what a person does with the citation. The architecture supplies the citation, and the person supplies the judgment about the source.

The confusion between licensing and truth is the most common overclaim in the field. A system that says "our claims are verified" is usually saying "our claims have sources", and the two are not the same sentence. Truth is not a field.

The distinction has a practical use. A team that knows the difference asks the source question at the right moment, which is during the review rather than after the decision. A team that confuses the two reviews the plumbing and trusts the water.

Truth also has a cost the architecture makes visible. A claim whose source is weak shows a weak span, and the weakness is checkable rather than hidden. The visibility is the gain, and the gain is not the truth.

Consensus

The architecture records disagreement, and a recorded disagreement is not a resolved one. The conflict set holds the claims that disagree, and the set is a description rather than a verdict.

Consensus is a social achievement. It requires people to accept a claim, and acceptance is not a field the ledger can write. A ledger can record that two departments disagree, and it cannot record that they have stopped.

The architecture helps consensus in one way, since making the disagreement explicit is the first step in resolving it, and most disagreements are unresolved because nobody wrote them down. The ledger writes them down.

The architecture can also hurt consensus. A visible conflict is harder to ignore than a tacit one, and a team that has been disagreeing politely will find the politeness harder to maintain. The visibility is a cost as well as a benefit.

The resolution, when it comes, is a new claim. Somebody decides, the decision is recorded with its author and its reason, and the conflict is superseded rather than deleted. The architecture records the decision and does not make it.

A decision record has fields, and the fields are the reason the resolution can be audited. The author, the date, the claims considered and the reason are the four, and a decision without the four is a decision nobody can defend. The architecture supplies the fields; the person supplies the decision.

Consensus also has a false form. A team can agree to stop recording the conflict, and the ledger will show agreement because the conflict was closed. The closure is a decision, and the decision is visible to anyone who reads the record.

Clean data

The architecture prices the gap, and a priced gap is not a closed one. The gap table records what the compiler could not license, and the record is a bill rather than a repair.

Clean data is an upstream property. A source that is inconsistent, incomplete or wrong produces a ledger that faithfully records the inconsistency, the incompleteness and the error. The ledger's accuracy is not the source's cleanliness.

The three cleanup tasks have different owners. Normalizing identifiers belongs to the integration team, merging duplicates belongs to the data owner and populating missing fields belongs to the source system's operator, and a project that assigns all three to one team has assigned two of them to nobody.

The architecture helps in one way. A gap that is priced can be funded, and a gap that is unpriced is a surprise. The price is the number of claims the pipeline could not compile, and the number is the argument for the cleanup.

The architecture can also make the mess worse. A ledger with gaps in it looks incomplete, and a team that needs a complete-looking system will fill the gaps by hand. The hand-filling is the failure mode the gap table was meant to prevent.

The cleanup is somebody's project. A source system needs its identifiers normalized, its duplicates merged and its missing fields populated, and each of the three is a data engineering task rather than a semantic one.

Clean data also has a definition problem. A source is clean relative to a use, and a source that is clean for billing may be dirty for maintenance. The ledger records the use, and the use is what makes the cleanliness question answerable.

Cost and wisdom

The architecture records the cost of compilation, and a recorded cost is not a zero cost. Every claim costs an extraction, a stance, a span and a gate, and the four together are the bill.

The bill is paid in engineering time. A team that compiles a thousand claims a week needs a pipeline, a gate battery and a person who reads the holds, and the three are a running cost rather than a purchase.

The hold queue is the part of the bill that surprises teams. A gate that refuses a claim creates a hold, a hold needs a reviewer, and the reviewer is a person with a queue. A team that budgets for the pipeline and not for the queue discovers the cost in the first month.

The architecture helps by making the cost visible, because a cost that is recorded can be compared against the value of the answers, and the comparison is what a business case needs. A cost that is hidden is discovered later, and the discovery is usually a cancellation.

Wisdom is the fifth limit and the most awkward. The architecture makes answers checkable, and a checkable answer is not a wise one. Wisdom is a judgment about what matters, and the ledger holds what was said rather than what matters.

The awkwardness shows in a single example. A licensed answer can say that the repair cost more than the budget, and the wise answer may be to defer the repair until the budget cycle turns. The second sentence needs a judgment about priorities, and the ledger holds the priorities only if somebody recorded them.

The architecture helps wisdom in one way. An answer that declares its loss and its version gives a person the material for a judgment, and the material is better than a confident sentence. The judgment remains the person's.

Wisdom also has a failure mode the architecture can expose. A team that treats a licensed answer as a wise one has confused the citation with the judgment, and the confusion is visible when the answer is checked against a decision it did not support.

Grounded action

The architecture licenses claims, and a licensed claim is not a safe act. Grounded action is the step from a claim to a decision and from a decision to an act, and the step is where the harm lives.

The four acts of the architecture are supersede, retract, redact and delete, and each is an act on the record rather than on the world. The architecture can record that a decision was made, and it cannot record that the decision was right.

The gap between the record and the act is a person. An operator reads the claim, weighs the consequence and acts, and the weighing is not a field. The architecture's contribution is that the operator's reading is checkable.

The four acts are a good test of the gap, since each has a receipt that says who did what and when. The same four words in the world may mean scrapping a pump, recalling a batch, hiding a name or destroying a file, and none of those four is in the ledger.

The architecture helps by making the action's warrant visible, since a decision that cites a claim can be reviewed against the claim, and the review is what a regulator or an auditor will do after the fact. The visibility is a real gain and is not safety.

Safety has its own discipline. A safety case needs hazards, controls, tests and a failure analysis, and none of the four is a semantic structure. The architecture can hold the claims a safety case cites, and it cannot be the safety case.

Grounded action also has a timing problem. An act happens at a moment, and the record at that moment may be incomplete. A system that waits for a complete record never acts, and a system that acts on an incomplete record needs a person who knows which gaps matter.

Security

The architecture records obligations, and a recorded obligation is not an enforced one. The envelope's terms say who may read what, and the terms are a statement rather than a control.

The architecture is not a security boundary. It does not encrypt, authenticate or prevent copying, and a receiver who ignores the terms has violated an obligation rather than defeated a mechanism. The distinction matters because a team that confuses the two builds a policy where it needed a control.

The architecture helps security in one way. A record of what was sent, to whom and under what terms is the material an investigation needs, and the material is often missing. The record is a contribution to security rather than a substitute for it.

The architecture can also create a security illusion. A system with terms and no enforcement looks governed, and the look is worse than no terms because it discourages the control that would have worked. The illusion is the risk of every compliance-shaped artifact.

Security also has a supply chain. The model, the pipeline, the ledger and the transport each have their own threats, and the architecture sits at the junction rather than at the perimeter. A security review of the architecture is a review of the junction.

The honest statement is that the terms are obligations and the controls are elsewhere. A team that writes the sentence in its design document has done the work the architecture can support.

Each component can be swapped, and each swap changes the threat surface. A security review that examines the ledger and not the swaps has examined the part that changes least.

The promise

What the architecture does deliver is a record whose claims can be checked, whose conflicts can be recorded, whose gaps can be priced and whose answers can be refused. The four verbs are the promise, and each is smaller than the word a buyer expected.

A claim can be checked because it carries a span and a version. A conflict can be recorded because the conflict set holds it. A gap can be priced because the gap table

counts it. A refusal can be trusted because it carries a reason. Each verb is a test a reviewer can run, and the four tests together are the whole of the promise.

The four are enough to change a practice. A team that can check, record, price and refuse stops arguing about whose recollection is better and starts arguing about which source to fix. The argument is a better argument, and the architecture is what makes it possible.

The four are not enough to change a business. A business needs truth, agreement, clean data, cheap operation, judgment and safety, and the architecture supplies the record that those things are built on rather than the things themselves.

A buyer can say the promise in one sentence. The system will tell me what the record says, who said it, how strongly and on what basis, and it will refuse when the record cannot support an answer. The sentence is small enough to be true, and true enough to be defended.

The limit list is therefore part of the claim. A promise that names its limits is a promise a buyer can hold, and a promise that hides them is a promise that will be renegotiated at the worst moment.

A list of limits is not yet a refusal; the next chapter asks what the book will not build.

CLOSE

67 · What This Book Refuses

A limit is something the architecture cannot do. A refusal is something the book will not build. The distinction matters, because the second list is a decision and the first is a fact.

The refusal

The refusals are five, and each one gives up something a reader might have wanted. The book refuses to be an operating system, to supply a universal operational schema, to promise a clean victory, to act as a vendor and to hide its debts.

A refusal is not an apology. An apology explains why the book could not do something; a refusal states that the book would not do it even if it could. The difference is the difference between a shortage and a boundary.

```
not an operating system it does not run the business
not a universal schema the domain's verbs stay the domain's
not a clean victory the record ends with gaps and a decision
not a vendor the book sells no product and no license
not a hidden debt every unfinished part is named in the text
```

The list is written into an architecture decision record as well as into the book. The record is the form an engineering team uses, and a refusal that lives only in prose is a refusal a team will forget in the second sprint.

```
ADR-14 We adopt the ledger, the four acts and the wire.
 We do not adopt a workflow engine, a universal ontology
 or a promise of clean data. Owner: data platform.
```

The record has two columns and both matter. The adopted column says what the team will run, and the refused column says what it will not run even when a vendor offers it. The second column is the one that keeps the first honest.

The refused column is also the one a new team member reads first. A joiner who knows what the team will not build asks fewer questions about the roadmap and more about the boundary, and the boundary is where the useful questions are. A record with only an adopted column is a shopping list.

A design that refuses nothing has no shape. It claims every use, and a thing that claims every use has no boundary a person can hold. The refusals are the boundary, and the boundary is what makes the claims checkable.

Not an operating system

The first refusal is the largest. The ledger is a record, and a record does not run the business. It stores what was claimed, who claimed it and what the record did about it.

An operating system has workflows, triggers and side effects, and each of the three is the opposite of a record. A workflow moves a task; a trigger reacts; a side effect changes the world. The ledger does none of the three, and a system that adds them has changed its kind.

The boundary is easy to cross by accident. A team adds an approval workflow beside the ledger, the workflow reads a claim and writes an approval, and the approval becomes the record of the decision. The ledger now holds an outcome that a workflow produced, and the workflow is outside the audit.

The repair is to keep the two apart. The workflow may run the business, and the ledger records what the workflow decided, with the receipt and the authority. The record is the workflow's output rather than the workflow's implementation.

The apartness has a test. A team asks whether a claim in the ledger can be traced to a decision that a person made, and the answer must be yes for every claim the workflow produced. A claim whose only provenance is a step identifier has lost the person, and the lost person is the audit's subject.

The refusal also protects the audit. A ledger that runs the business cannot be audited by the business, because the auditor would need to audit the component that is doing the running. A record that only records can be audited by anyone who can read.

The refusal is not an argument against automation. It is an argument about where the automation lives. The line is the point. A team may automate everything it likes, and the ledger's job is to hold the claims that the automation used and produced, not to be the automation.

Not a universal schema

The second refusal is about vocabulary. The book supplies fifteen roles, eight dimensions and three joins. It does not supply the verbs. The verbs are the domain's, and a universal verb set is a vocabulary nobody speaks.

The roles are the part that generalizes. An agent, a theme, a source and a time appear in a maintenance record, a clinical note and a contract, and the roles are what lets one schema hold all three. The verb is the part that does not generalize, because "failed", "diagnosed" and "breached" are different acts in different worlds.

A universal verb list is a tempting artifact and a bad one. It grows to hundreds of entries, each with a definition nobody agrees on, and the compiler spends its gates deciding which entry a sentence meant. The domain's own verbs are fewer and sharper.

The refusal shows up in the profiles. Core, Content, NSM and Domain are four profiles of the language, and the Domain profile is where a team writes its own verbs. A universal

schema would have no need for a Domain profile, because the universal list would already hold the verbs.

The Domain profile is also where a team's disagreements become visible. Two departments that use "failure" differently write two entries, and the two entries are the disagreement in a form the compiler can hold. A universal list would have merged them, and the merge would be the first place the record lied.

The refusal also shows up in the ontology work. A team adopts an upper ontology and maps its own terms into it, and the mapping is the team's work. The book supplies the discipline of the mapping and refuses the fantasy that one ontology will fit every industry.

The cost of the refusal is real. A team that wanted a ready-made vocabulary will have to write one, and the writing takes weeks. The gain is a vocabulary that its own users can defend, which is the only kind that survives a dispute.

Not a clean victory

The third refusal is about the ending. The book does not promise that the mess disappears. It promises a record that can be defended while the mess remains.

A clean victory is what a demo shows. The demo has ten claims, no conflicts, no gaps and a happy user, and the demo is not a record. A record has thousands of claims, a conflict set, a gap table and a queue of decisions, and the record is what a team lives with.

The end state is a negotiated record. Some conflicts are resolved and some are held open, some gaps are funded and some are accepted, and every one of the decisions is written down with its author. The writing down is the victory, and the victory is smaller than the word suggests.

The refusal has a cost in the sales conversation. A buyer who wants the mess to disappear will hear the refusal as a weakness, and the buyer who has lived through a failed clean-up will hear it as experience. The second buyer is the one the book is written for.

The refusal also sets the metric. A team that measures the disappearance of gaps measures a number that will not move, and a team that measures the fraction of gaps that are priced and decided measures a number that can. The second number is the one the architecture supports.

The metric can be written on one line. Gaps priced and decided over gaps found is a ratio a team can report monthly, and the ratio moves when the team does the work. A team that reports the count of gaps instead reports a number that grows as the ledger grows, and the growth reads as failure when it is success.

A clean victory is purchasable in a demo and not in a record, and the difference between the two is the customer. The demo's customer is a committee, and the record's customer is the person who has to defend the decision in two years.

The two-year test is the one to design for. A record that a person can defend in two years has spans, versions and decisions, and a record that only makes a demo shine will be rebuilt before the two years are out. The rebuilding is the cost of the clean victory, and the cost is paid by the team that inherited it.

Not a vendor

The fourth refusal is commercial. The book sells no product, no license and no certification. It sells a method and a set of artifacts, and the reader implements them.

The refusal is not modesty. A book that recommends a product is a book whose claims can be discounted by the product's failures, and a book that recommends a method is a book whose claims stand or fall on the method. The second book is the one that can be argued with.

The refusal also keeps the vocabulary honest. A product's vocabulary is shaped by its features, and a method's vocabulary is shaped by the work. A team that adopts the method and a vendor's product keeps the two vocabularies apart, and the keeping apart is what lets the team change vendors without changing its record.

The book does name tools where a tool is needed. A property graph, an RDF store, a relational kernel and a message envelope are all tools, and the book says what each one does to a claim. Naming a tool is not selling it, and the difference is the absence of a price.

A certification would be the other way to sell without selling. A team that certifies a method sells the certificate, and the certificate becomes the thing buyers want rather than the record. The book refuses the certificate for the same reason it refuses the product: the artifact is the team's, and a badge would replace it.

The tools are named because a team must choose among them. A graph store and a relational kernel preserve different things, and the difference is the projection's loss rather than a matter of taste. The book names the loss and declines to name a winner, because the winner depends on the questions the team asks.

The refusal has a cost in reach. A vendor can put a thousand people through a training, and a book reaches the readers who buy it. The reach is smaller and the claims are cleaner, and the trade is the one the book made.

The refusal also protects the reader's judgment. A reader who is not being sold anything can disagree with a part of the book without feeling that the disagreement is a breach of contract. The freedom to disagree is the point of a method.

Not a hidden debt

The fifth refusal is about honesty. The book does not hide the parts it could not finish. The unfinished parts are named in the text, and some of them are named as predictions that may fail.

The predictions are the clearest case. The book makes claims about what will happen to teams that adopt the ledger, and the claims are listed so that a reader can check them later. A prediction that is listed can be refuted; a prediction that is implied cannot.

A prediction register is a table with three columns. The prediction, the date by which it can be checked and the observation that would refute it are the three, and a register with the first column only is a wish list. The three columns are what make the register an instrument.

The register also disciplines the writing. A claim that cannot be checked by a date is a claim the writer should soften, and the softening happens before the reader sees it. The register is the writer's gate, and it works because it is written down where the writer can read it again.

The open questions are the second case. Some questions are answered in the text and some are left open with the reason, and the reason is the reader's material. A book that answered everything would be a book that had not noticed its own edges.

The debts are the third case. A part of the book that owes a proof, a measurement or a comparison declares the debt, and the declaration is what lets a later edition pay it rather than discover it. A hidden debt is discovered by a reader, and the discovery is the reader's reason to stop trusting the rest.

The refusal is also a discipline for the writer. A debt that must be named is a debt that is harder to accumulate, and the naming keeps the book's scope from drifting into what the writer wished were true.

The cost is a book that admits incompleteness, and the admission is the price of the claims that remain. A reader who wants completeness can find it in a vendor's brochure, and the brochure will be complete because it is not a record.

The shape

The five refusals give the design its shape. A record that does not run the business, does not own the vocabulary, does not promise cleanliness, does not sell itself and does not hide its debts is a record with an edge, and the edge is what a team can build against.

The shape is also a test for the reader's own design. A team that has written its refusals has decided what it will not do, and the decision is the first thing an architecture review should ask for. A team with no refusals has no scope.

A review can put the question plainly: name three things this system will not do. A team that answers in a sentence has a scope, and a team that answers with a list of future features has a roadmap. The two answers sound similar in a meeting and are different documents.

The shape is what the book has been building toward. The claims of the earlier parts are credible because the refusals bound them, and the bounds are the reason a reader can check a claim instead of trusting a tone.

The shape is not the end of the argument. A refusal can be revisited when the evidence changes, and the record of the refusal is what makes the revisiting honest. A refusal that nobody recorded is a preference that will be reversed in a meeting.

The shape is also a gift to the next reader. A book with an edge can be put down and picked up, because the reader knows what is inside and what is not. The knowing is what makes a second reading useful.

A book that knows what it refuses can be read again; the last chapter takes the four bearings.

68 · The Compass Again

A long argument needs an instrument at the end rather than a summary, because a summary replaces the book and an instrument lets a reader carry it. The instrument has four bearings.

The compass

A bearing is a direction a reader can take from any page. Four bearings order the whole argument, and each is a question a claim must answer before a person may act on it.

```
1 claim what is asserted, with roles and a verb
2 warrant who says so, how strongly, on what evidence
3 address where it is written, which version, which clock
4 license what may be done with it, and what it refuses
```

The bearings are read in order. A warrant without a claim is authority in search of something to authorize, and an address without a warrant is a citation to nobody's testimony. A license without an address is a permission nobody can check.

The order is the argument. The book spent its first parts establishing that the claim is the unit, its middle parts establishing that the warrant and the address are what make a claim usable, and its last parts establishing that the license is what makes it safe to move.

The compass is not a checklist. A checklist is read once and filed, and a compass is read again at each new document. The difference is the difference between a summary and an instrument.

The compass is also a reading method. A reader who opens the book at a random page can ask which bearing the page serves, and the answer places the page in the argument without requiring the earlier parts.

The bearings are also how a reader argues with the book. A reader who thinks the license is the first question can reorder the compass, and the reordering is a claim the book can answer rather than a mood it can only absorb. A compass that can be argued with is a compass that can be improved.

The claim

The first bearing is the claim. A claim is what the record asserts, and the assertion has a verb, a set of roles and a stance.

The claim is the smallest unit that can be true or false. A word is smaller and cannot be either, and a paragraph is larger and can be true in one sentence and false in the next. The claim is the unit because it is the smallest thing a person can defend.

The claim is also the unit of disagreement. Two people who disagree about a paragraph can usually agree about which sentence they disagree about, and the sentence is the claim. A record built from claims can localize a disagreement to a row, and a record built from documents can only localize it to a file.

The claim's shape is the synapse. A verb at the hub, participants on closed spokes, dimensions on the rim and a span beneath, and the shape is what lets a claim be stored, queried and compared. The shape is not a diagram; it is the record's row.

The claim's parts are closed where closure helps. Fifteen roles are enough for the arguments a record holds, and eight dimensions are enough for the ways a claim can vary. The verb stays open because the world's verbs are the world's.

The claim's stance is part of the claim rather than a comment on it. Asserted, reported, inferred and the rest are the ways a claim can be held, and a record that drops the stance has flattened testimony into fact.

The first bearing is therefore a question with a short answer and a long justification. What is asserted is a sentence; what makes it a claim is the record's shape.

The warrant

The second bearing is the warrant. A warrant is who says so, how strongly and on what evidence, and the three are one question because a claim without a holder is a rumor with good punctuation.

The holder is the first part. A claim is held by a person, an organization or a document, and the holder is what makes the claim attributable. An unattributed claim is not stronger for being unattributed; it is weaker, because nobody can be asked about it.

The stance is the second part. A claim that is asserted, reported or inferred carries a different weight, and the weight is what a reader uses to decide how much to check. A record that reports everything at one stance has thrown away the reader's first tool.

The evidence is the third part. A span points at the source, the source points at an author, and the author points at an observation. The chain is checkable link by link, and the last link is the world, which the record cannot check.

The warrant is also where authority lives. A claim made by a regulator, a laboratory or a witness carries an authority that a claim made by an analyst does not, and the authority is a field rather than an adjective. The field is what lets a reader weigh two claims that disagree.

The second bearing is the one a team most often skips. A team that records claims and spans and forgets the holder has built a quotation file, and a quotation file cannot answer the question that matters when two quotations disagree.

A warrant can also be inherited. A claim repeated by a second holder carries the first holder's evidence and the second holder's stance, and the two are different facts about the claim. A record that collapses them into one attribution has lost the difference between testimony and repetition.

The address

The third bearing is the address. An address is where a claim is written, which version it belongs to and which clock it keeps, and the three are what make a claim findable and comparable.

The canonical identifier is the first part. A claim, a participant and a verb each have an identifier that survives renames, and the identifier is what lets two systems refer to the same thing. A record without identifiers is a record that merges by string matching, and string matching is where duplicates are born.

The version is the second part. A claim belongs to a version, a version belongs to a ledger, and the version is what lets a reader ask what the record said in March. A record that overwrites instead of versioning answers the question with the present tense.

The clock is the third part. Validity time, assertion time, decision time and the rest are separate clocks, and a claim can be true at one time and recorded at another. A record with one clock conflates the two and cannot answer a question about when something was known.

The address is what makes the citation cheap. A reader who has the identifier, the version and the clock can find the claim in one step, and the one step is the difference between a checkable answer and a plausible one.

The third bearing is the one that looks like plumbing and is not. A team that skips identifiers and versions has built a system that cannot be asked about its own past, and the past is where most disputes live.

The address has a maintenance cost that teams underestimate. Identifiers must be assigned, versions must be cut and clocks must be kept, and each of the three is a job. The job is small per claim and continuous across a ledger, and a team that budgets for the schema and not for the address finds the cost in the first migration.

The license

The fourth bearing is the license. A license is what may be done with a claim, and the answer has two halves: what the record permits and what the record refuses.

The permission is the first half. A claim may be quoted, summarized, exported or acted on, and each use has terms. The terms travel with the claim, and a claim quoted without its terms is a claim whose permissions have been stripped.

The refusal is the second half. A record that cannot license a claim says so, with the missing license and the nearest claim, and the refusal is an output rather than a failure. A system that refuses with a reason is a system a reader can work with.

The license is also where the architecture meets the world. A claim may be licensed and false, and the license says nothing about truth. The license says what the record will support, and the record's support is the part the architecture can defend.

The license is the bearing a reader meets last and needs first. A person who is about to act on a claim wants to know what the record permits, and the answer is the four bearings in order, ending with the permission.

The fourth bearing is the one the book refused to overstate. The terms are obligations rather than controls, and the obligations are recorded rather than enforced. The honesty is the license's contribution to the compass.

A license can be narrower than the claim it covers. A claim may be licensed for an internal summary and not for a quotation, and the narrower license is the common case. A record that stores one permission per claim has stored the widest use, and the widest use is the one a sender regrets.

The page

The compass fits on one page, and the page is the book compressed. Each part of the argument serves a bearing, and the placement is what lets a reader enter anywhere.

Part	The question it inherits	The bearing it serves
I The Wound	why the container does not hold meaning	the claim
II The Atom	what the unit is	the claim's shape
III The Home	where it lives, and how two intakes reach it	the warrant and the address
IV The Machine	how it is compiled, and how the compile is proved	the address
V The Address and the Meaning	what the names contain	the claim
VI The Ontology	who governs the distinctions	the warrant
VII The Warrant	who may act on a claim	the warrant and the license
VIII The Shape	what meaning is, that it can be entitled	the claim and the license
IX The Negative Space	what a semantics forbids	the license's negative half

Continued on next page

(Continued)

Part	The question it inherits	The bearing it serves
X The Landing	how it lands without lying	the address
XI The Model and the Wire	who consumes it	the license
XII What This Makes Possible	what it makes cheap, and what it does not unlock	the compass
Close	what it refuses, and where the compass points	the compass

The placement is a reading aid rather than a taxonomy. A part that serves two bearings is a part that does two jobs, and the book has several. A reader who wants a single bearing per part is asking for a tidier book than the argument allows.

The open questions

Six questions are left open, and each is named with the reason it is open rather than closed by a sentence that would not survive a reader.

The first is whether fifteen roles are enough for domains the book did not examine. The roles were chosen for coverage, and coverage was tested on maintenance, clinical and contract records. A domain with a different argument shape may need a sixteenth role, and the need would show up as a gap in the compiler.

The second is whether the eight dimensions are independent. The book treats them as separable, and a domain may show two dimensions that move together. If two dimensions are one, the schema has a redundancy and the gates have a simplification waiting.

The third is whether the compiler can be built cheaply enough for small teams. The gates and the hold queue are the cost, and the cost is recorded rather than solved. A small team may need a thinner gate battery, and the thinner battery is a research question.

The fourth is whether the ledger's versioning survives a merger. Two ledgers with different identifiers and different clocks can be joined at the claim level, and the joining at scale has not been demonstrated.

The fifth is whether the license can be enforced without a control. The terms are obligations, and an obligation depends on the receiver. A mechanism that enforces the terms would be a different architecture, and the book does not claim to have one.

The sixth is whether the compass is the right four. A different reader might order the questions differently, or add a fifth for cost. The compass is offered as an instrument rather than a doctrine, and a reader who improves it has used it correctly.

The last page

The compass is an instrument, and an instrument is judged by what a reader does with it. A reader who closes the book and asks the four questions of the next document has the argument, and the argument is the part that travels.

An instrument has a failure mode, and this one fails by becoming a ritual. A team that asks the four questions as a form, with the answers filled in by habit, has the instrument's shape and not its function. The failure is visible when the answers stop changing the decision, and the cure is to ask the questions of a claim that matters.

The record is the thing to defend. A diagram can be redrawn and a demo can be rebuilt, and a record with claims, warrants, addresses and licenses is the artifact that survives both. The book's wager was that meaning is predication-centric, and the wager's payoff is a record that can answer for itself.

The bearings are also a test of the book. A page that serves no bearing is a page that could be cut, and a claim that cannot be placed is a claim that the argument does not need. The test applies to the book as much as to the reader's documents.

What remains is the reader's work. The record is built one claim at a time, with a span and a stance and a version, and the building is slow in the way that a foundation is slow. The slowness is the price of the defense, and the defense is the point.

The four bearings are old questions. What is it, who says so, where is it written, and what may I do with it are questions people asked before there were databases, and the book's contribution is a record that answers all four without a meeting.

The record is the thing to defend.

APPENDIX

Appendix A · The Thirty Patterns

A pattern is a reusable shape with a failure it prevents. The thirty cards below are the book's pattern set, grouped into six families, and each card names the shape, the trigger that makes it worth reaching for and the guard that catches the failure.

The cards are not a checklist. A team adopts the patterns its record needs, and most records need fewer than thirty. The families are the part worth memorizing; the individual cards are consulted when a specific failure appears.

The six families

Family	Patterns	The family's subject
I Claim structure	P-01---P-07	the claim, its roles, span, nesting and order
II Identity and grounding	P-08---P-12	referent, alias, dictionary, profile, ontology adoption
III Evidence and authority	P-13---P-18	candidate and gap, proof trace, conflict, authority, finding, decision
IV Time and lifecycle	P-19---P-22	clocks, correction, retraction, transition receipt
V Projection and explanation	P-23---P-27	convenience relation, fidelity, contract, rehydration, explanation
VI Operation and proof	P-28---P-30	canonical store, dual compiler, semantic CI

Family I: claim structure

P-01 Canonical Synapse Card. One claim, one card: verb at the hub, participants on closed spokes, dimensions on the rim and the span beneath. Reach for it when any claim enters the record. The guard is the span: a card that cannot name where the claim came from is a candidate rather than a claim.

P-02 Role-Bound Participant. Every participant is bound to one of the fifteen closed roles, and the binding is stored as data rather than recovered from word order. Reach for it when a sentence carries two or more participants. The guard is the binding: a participant with no role is refused at the gate rather than guessed at later.

P-03 Source-Span Anchor. Every claim carries the location it came from, to the page and the character range, and the location is what makes the claim checkable. Reach for it on every extraction. The guard is resolution: a span that does not resolve is a candidate, and a candidate is not evidence.

P-04 Nested Content. A claim inside a claim—a report, a quotation, a belief—keeps its own card and is linked rather than flattened into the outer claim. Reach for it on reported speech and attribution. The guard is the holder: a nested claim that loses its holder has been assimilated into a claim nobody made.

P-05 Ordered Synapse Stream. Claims that share a discourse are ordered by an explicit sequence rather than by insertion time. Reach for it on a contract, a procedure or a narrative. The guard is the order: a stream reordered by a migration is a stream whose reading changed without a record.

P-06 Recursive Synapse Group. A group of claims can act as one participant in a further claim, and the recursion is explicit. Reach for it when a clause is the subject or object of another. The guard is the citation: an implicit group that a reader must reconstruct is a group the record cannot cite.

P-07 Typed Beside Link. Claims are related by a typed link—elaboration, contrast, cause, evidence—rather than by proximity. Reach for it when two claims would be read together. The guard is the type: an untyped link becomes a similarity, and a similarity is not a relation the record can defend.

Family II: identity and grounding

P-08 Canonical Identity and Alias. Every referent has one canonical identifier and any number of aliases, and the aliases resolve to the identifier. Reach for it when a name appears in more than one form. The guard is the target: an alias with no canonical target is a ghost, and a ghost is a gap in the identity space.

P-09 Candidate Entity Resolution. A new mention becomes a candidate, is compared against the identifier space, and is accepted, merged or held. Reach for it at intake. The guard is the receipt: a merge with no receipt cannot be undone, and an undone merge is the most expensive repair in the ledger.

P-10 Dictionary Grounding. A private term is grounded in a dictionary entry with a definition and a scope, and the entry travels with the claims that use it. Reach for it when a receiver cannot look the term up. The guard is the scope: an undefined private term is a claim the receiver will misread.

P-11 Semantic Profile. A profile fixes the vocabulary a domain may use and the subset of the grammar it permits. Reach for it when a new domain joins the ledger. The guard is refusal: a claim outside the profile is refused rather than coerced into it.

P-12 Ontology Adoption Gate. An upper ontology is adopted through a gate that maps local terms into it and records what could not be mapped. Reach for it on alignment to

BFO, CCO or a domain ontology. The guard is the loss record: an adoption with no loss record has hidden what it could not map.

Family III: evidence and authority

P-13 Candidate, Ghost and Gap. An unlicensed claim is a candidate, an unresolvable reference is a ghost, and the missing evidence is a gap; the three are recorded rather than dropped. Reach for it at every refusal. The guard is the gap: a candidate silently promoted to a claim has lost the record of what was missing.

P-14 Evidence and Proof Trace. A claim carries the chain from span to source to author, and the chain is what a checker walks. Reach for it when a claim is used in a decision. The guard is the endpoint: a trace that stops at a summary is not yet evidence, and a summary is where traces most often stop.

P-15 Evidence Conflict. Two claims that disagree about the same theme, role and time form a conflict set, and the set is held rather than resolved at entry. Reach for it on any contradiction. The guard is the set: an entry-time resolution has destroyed the disagreement the record will need later.

P-16 Authority Reference. The authority behind a claim is a reference to a record—a regulation, a laboratory, an office—rather than an adjective. Reach for it on any claim that binds someone. The guard is the record: an authority with no record is a claim about importance rather than a claim about standing.

P-17 Finding Record. A finding is the resolution of evidence: it names the claims considered, the author and the reason, and it supersedes the conflict rather than deleting it. Reach for it when a decision needs a basis. The guard is the author: a finding with no author cannot be re-examined.

P-18 Decision Record. A decision is an act taken on a finding: it names the act, the actor, the date and the scope. Reach for it on any commitment of resources or permission. The guard is the finding: a decision with no finding is an exercise of authority without a record.

Family IV: time and lifecycle

P-19 Multi-Clock Time Anchor. A claim carries validity time, assertion time and, where relevant, decision time, and the clocks are separate fields. Reach for it on every claim with a date. The guard is the separation: one clock for three uses conflates what was true with what was known.

P-20 Correction and Supersession Row. A correction creates a new version and a supersession row that points at the old one; nothing is overwritten. Reach for it on any change to a claim's content. The guard is the row: an overwrite has destroyed the ability to ask what the record said before the change.

P-21 Retraction, Deletion, Redaction. Three acts with three receipts: retraction withdraws a claim, deletion removes it under a rule, and redaction hides part of it while

recording that it hid. Reach for it when a claim must stop being used or seen. The guard is the distinction: one act for three purposes leaves the record unable to say which happened.

P-22 Transition Receipt. A change of state—a merge, a promotion, a closure—emits a receipt naming the prior state, the new state, the actor and the time. Reach for it on every state change. The guard is the receipt: a state change with no receipt is a change no auditor can date.

Family V: projection and explanation

P-23 Convenience Relation. A relation added for query convenience is marked as a projection and carries the rule that derived it. Reach for it on a denormalized edge or a shortcut table. The guard is the marking: an unmarked convenience relation becomes a fact that nobody can retract.

P-24 Projection Fidelity Contract. Every projection declares what it preserves, what it drops and how the drop can be detected. Reach for it on any store that is not the ledger. The guard is the declaration: a projection with no loss statement has hidden its own seams.

P-25 Projection Contract. A projection's interface names its inputs, its losses and its version, and a change to any of the three is a new contract. Reach for it when a consumer binds to a projection. The guard is the version: a contract that changes silently breaks the consumer's citations.

P-26 Qualified Rehydration. A claim read back from a projection is rehydrated with a qualification: which projection, which version, which loss. Reach for it on any round trip. The guard is the qualification: an unqualified round trip is indistinguishable from the original claim.

P-27 Explanation Ladder. An answer can be explained at several rungs—the sentence, the claim, the span, the projection—and the rung is named. Reach for it when a reader asks why. The guard is the rung: an explanation that skips rungs is a story about the answer rather than an account of it.

Family VI: operation and proof

P-28 Canonical Semantic Store. One store is canonical for claims, and every other store is a projection of it. Reach for it when a second database holds claims. The guard is the count: two canonical stores are two truths, and two truths cannot be reconciled after the fact.

P-29 Dual Compiler. The compiler has two paths, a model path that proposes and a deterministic path that disposes, and the two are separately auditable. Reach for it on any model-assisted extraction. The guard is the separation: a merged path cannot say which component decided.

P-30 Semantic CI and Guard Battery. The gate battery runs on every change, with fixtures for each gate, and a failing fixture blocks the commit. Reach for it on every schema or gate change. The guard is the fixture: a gate with no fixture is a rule nobody has tested.

Working the cards

Three cards carry most of the weight in a first build. P-01 fixes the unit, P-03 makes it checkable and P-24 makes the projections honest, and a record with those three and no others is already more defensible than most.

The remaining cards are reached for when a failure appears. A merge that cannot be undone sends a team to P-09; a contradiction that keeps being resolved on entry sends it to P-15; an answer nobody can explain sends it to P-27. The failure is the trigger, and the trigger is what makes a card worth the paper.

The book's own use of the thirty is thin by design. The book's chapters name the pattern they rely on and show the shape in prose, and the cards are the reference for the reader who wants the shape alone. A card reprinted in a chapter would be a table the prose already carries.

A card is not a mandate. A record can be correct without P-05 or P-27, and a record with all thirty patterns and no spans is still uncheckable. The patterns describe shapes that prevent specific failures; the spans are what make any shape checkable.

Appendix B · Closed Sets and Grammar

The book's grammar is closed where closure buys consistency and open where openness buys coverage. This appendix collects the closed sets in one place: fifteen roles, eight dimensions, nine forces, three joins and the link types that connect claims.

A closed set is a promise to the reader. A role list that can grow with every domain is a list nobody can query, and a dimension list that varies by pipeline is a list whose filters do not compose. The sets below are the whole of the grammar's fixed part.

The fifteen roles

Role	The job it protects	What is lost if it folds
AGENT	the intentional initiator or controller	the wind becomes a chooser
PATIENT	what undergoes change, damage, creation or destruction	a damage claim loses its victim
THEME	what is moved, located, possessed or perceived without change	located things and broken things collapse
EXPERIENCER	who perceives, believes, feels or knows	giving and seeing become one relation
RECIPIENT	who receives the transferred thing	the addressee disappears into the beneficiary
BENEFICIARY	for whose benefit the act happens	addressee and beneficiary become one claim
TIME	when the act occurs	a deadline becomes a date
LOCATION	where the act occurs	a path loses direction
SOURCE	the endpoint a path starts from	*from Paris* must be recovered from a preposition

Continued on next page

(Continued)

Role	The job it protects	What is lost if it folds
DESTINATION	the endpoint a path ends at	*to Paris* must be recovered from a preposition
MANNER	how the act is done	a method becomes a tool
INSTRUMENT	with what the act is done	a tool becomes a method
CAUSE	the non-choosing trigger	weather becomes an actor
REASON	the stated aim	a justification becomes a cause
ATTRIBUTE	a property used as a participant	measures and classifications are not represented

The table's third column is the reason the set is closed. A grammar that merged MANNER and INSTRUMENT would answer "how" and "with what" with one field, and a query for the method would return the tool. The merge looks harmless at the schema and is fatal at the question.

The roles are not a case grammar copied from linguistics. They are the smallest set that separated the failures the book's domains produced, and a domain that needs a sixteenth role has found a distinction the set cannot express. That finding is a gap in the grammar rather than a license to add a field.

Two pairs do most of the separating work. AGENT and CAUSE keep a chooser apart from a trigger, so a storm is not an actor. MANNER and INSTRUMENT keep a method apart from a tool, so a procedure is not a wrench. A grammar that loses either pair answers a question with the wrong noun.

The set is closed on purpose. Fifteen roles fit on a page, and a page is what a new team member can learn in an afternoon. A set that grows with each integration is a set nobody learns.

The eight dimensions

Dimension	Values
voice	active, passive, middle
aspect	perfective, imperfective, habitual, progressive
mood	indicative, subjunctive, imperative, conditional
tense	past, present, future, none
number	singular, plural, mass

Continued on next page

(Continued)

Dimension	Values
modality	actual, possible, necessary, permitted, obligatory
polarity	positive, negative
force	assertion, definition, directive, commitment, question, hypothesis, attribution, evaluation, fiction

A dimension answers how the act was made, and a role answers who or what participated. The two are easy to conflate: *the valve was closed by the technician* adds an AGENT role, while *the valve was closed* changes the voice dimension. One sentence adds a participant and the other changes the framing, and a record that stores both in one field has lost the difference.

The dimensions are stored even when they take a default. A claim in the indicative mood with positive polarity stores those values, because a reader filtering for negative polarity must be able to exclude it. A default that is not stored is a default that a query cannot see.

Tense is the dimension that catches most pipelines. A claim with no tense is not a claim about the present; it is a claim whose time is supplied elsewhere. Recording *none* is different from recording *present*, and a schema that cannot store *none* forces every timeless definition into the present tense.

The dimension set is small for the same reason the role set is. Eight filters compose, and a reader can hold the list in mind while writing a query. A ninth dimension would be a filter that half the team forgets.

The nine forces

Force	The warrant it carries
assertion	the holder states it as true
definition	the holder fixes a term's use
directive	the holder requires an act
commitment	the holder binds itself
question	the holder asks for an answer
hypothesis	the holder proposes for testing
attribution	the holder reports another's claim

Continued on next page

(Continued)

Force	The warrant it carries
evaluation	the holder judges against a standard
fiction	the holder presents without asserting

Force is the dimension that changes what a claim commits its holder to. A directive and an assertion can share a verb, a theme and a time, and the first obliges someone while the second informs them. A record that stores force as an assertion in every case has flattened obligation into description.

The force table is also a warrant table. Each row names what the claim needs before it can be acted on: an assertion needs evidence, a directive needs authority, a commitment needs a capacity to bind. The nine are the closed set because the warrants are the closed set.

Two rows do the most work in practice. Directive and commitment look alike and bind different parties: a directive binds the reader, and a commitment binds the writer. A record that stores both as assertions has lost the difference between an order and a promise.

The three joins

The grammar has three joins and no more. NEST places a claim inside a larger content, CONTAIN places a content inside a scope, and RELATE places one claim beside another. The three are the whole of the composition grammar.

Join	What it connects	What it preserves
NEST	a claim and its enclosing content	the claim's own role structure
CONTAIN	a content and its scope	membership and boundary
RELATE	two claims	the typed relation between them

The joins are closed because composition is where records most often lose their claims. A fourth join added for convenience becomes a relation nobody can query, and a record with an open join set is a record whose traversal depends on the pipeline that wrote it. The joins are closed. That closure is what makes a traversal repeatable across two systems.

The links

A RELATE link carries one of a small set of types: elaboration, contrast, cause, evidence, sequence, exception and equivalence. The list is short because a long list of link types is a vocabulary nobody uses consistently.

Link type	What it says
elaboration	the second claim develops the first
contrast	the second claim opposes the first
cause	the second claim explains the first
evidence	the second claim supports the first
sequence	the second claim follows the first
exception	the second claim limits the first
equivalence	the two claims say the same thing

The link types are the grammar's concession to discourse. A record that stores claims and no links is a record of sentences, and a record that stores links with no types is a record of proximity. The seven types are enough to reconstruct an argument and small enough to be used the same way twice.

The list is short on purpose. Elaboration, contrast, cause, evidence, sequence, exception and equivalence cover the moves a reader makes between two claims, and a longer list would invite the fine distinctions nobody agrees on. Seven types can be applied consistently by a team; thirty cannot.

Equivalence deserves a warning. Two claims are equivalent when they say the same thing in the same scope, and a claim that is equivalent at one scope is often a contradiction at another. The type is useful and easy to overuse, and the guard is the scope field that every equivalence link carries.

The syntax of a claim

A claim is written in one form throughout the book, and the form is a claim rather than a serialization.

```
VERB(role=value; role=value; ...)[dimension=value; ...]{span; stance; version}
```

The parentheses hold the roles, the brackets hold the dimensions, and the braces hold the record fields that make the claim checkable: the span it came from, the stance its holder takes, and the ledger version it was written against. The version is the record's rather than a column on the claim, and a claim that names no version is a claim read at whatever version the reader happens to hold. The notation is deliberately boring: a reader can copy a claim from a page into a note and back without a tool.

The notation is not the wire format. The wire carries an envelope, the store carries rows, and the page carries the notation. The three are projections of one claim, and the claim is the thing the notation, the envelope and the rows all describe. The notation is plain, and plain is the point.

The grammar's closures are what let the notation survive translation. A role set that could grow would make every projection's mapping a new decision, and a dimension set that varied would make every filter a new query. The closed sets are the reason a claim written on one page can be read on another.

Appendix C · The Upper Ontology Landscape

An upper ontology is a small set of categories that a domain ontology can be built on. The two most used in operational settings are BFO and CCO, and this appendix says what each is for, how the book's grammar maps onto them, and where the mapping strains.

The appendix is not a tutorial. It is a guide for the reader who has been asked to align a ledger to an upper ontology and wants to know which decisions the alignment forces and which it leaves to the domain.

What an upper ontology does

An upper ontology supplies the categories that every domain shares: object and process, quality and disposition, role and function, information artifact and unit of measure. The categories are few and general, and the domain fills them with its own terms.

The benefit is interoperability. Two domains that both say *process* can compare their processes without agreeing on what a process is, and the agreement about the category is what the comparison rests on. The cost is commitment: a domain that adopts a category must use it consistently, and consistency is where alignments fail.

An upper ontology is not a schema. A schema names tables and columns; an upper ontology names kinds of thing. The two meet at the mapping, and the mapping is where a team discovers what its own vocabulary was assuming.

BFO

BFO divides the world into continuants and occurrents. A continuant persists through time—a pump, a person, a document—and an occurrent unfolds in time, such as a failure, a repair or a measurement.

Continuants divide into independent and dependent. An independent continuant is a thing in its own right, and a dependent continuant inheres in something: a quality such as temperature, a disposition such as fragility, a role such as being a spare part, a function such as pumping.

Occurrents divide into processes and their parts. A process has temporal parts, and the parts are processes too: a repair has a disassembly, a replacement and a test, and each is a process in the same sense as the whole.

The division is useful to a ledger because it separates what a claim is about from what a claim asserts. A claim about a pump is about a continuant; a claim that the pump failed is about an occurrent; a claim that the pump is hot is about a quality that inheres in the pump.

BFO's discipline is that a thing is one kind and not another. A pump is not a process, and a failure is not an object, and an alignment that treats them as interchangeable has broken the categories it adopted.

CCO

CCO is a set of mid-level ontologies built on BFO. It supplies the categories that operational domains repeatedly need: agents and organizations, artifacts and their parts, events and their participants, information entities and their content, units and measurements.

The mid-level is the useful level for a ledger. BFO is too general to name a work order, and a domain ontology is too specific to share across organizations; CCO sits between and names the kinds a maintenance record, a contract and a clinical note all use.

CCO's information entity category is the one a ledger leans on most. A document, a record, a message and a database row are all information entities, and the category lets a claim about a document be distinguished from a claim about the thing the document describes.

CCO's measurement pattern is the second. A measurement is an information entity that describes a quality by means of a unit and a number, and the pattern keeps the quality, the unit and the number apart. A record that stores "the temperature was high" has lost the number, and a record that stores "72" has lost the unit.

The strain points

Four strain points appear in most alignments, and naming them is more useful than pretending they do not exist.

The first is the event boundary. BFO's occurrents have temporal parts without a privileged grain, and a ledger's claims are at the grain a sentence chose. A repair can be one claim or twelve, and the upper ontology does not decide; the record's purpose does.

The second is the role. BFO treats a role as a dependent continuant that a thing bears for a time, and a ledger's fifteen roles are argument positions rather than BFO roles. The two uses of the word are different, and an alignment that maps the fifteen onto BFO roles one for one has confused a grammar with a category.

The third is the information artifact. A ledger holds claims about documents and claims inside documents, and the two levels are easy to conflate. CCO's information

entity category keeps them apart; a record that does not has lost the difference between what a document says and what is true.

The fourth is the unit. A measurement without a unit is a number, and an alignment that maps a measurement to a quality without the unit has dropped the part that makes it comparable. The strain is small and the consequence is large.

How the book's grammar maps

The book's roles map to upper-ontology categories unevenly, and the unevenness is worth stating plainly.

The participants map well. AGENT and EXPERIENCER map to agents and to BFO's bearers of mental qualities; PATIENT and THEME map to independent continuants; TIME and LOCATION map to temporal and spatial regions.

The dimensions map poorly. Voice, mood, aspect and force are properties of a claim rather than of the world, and an upper ontology has no category for them. They belong to the record's layer, not to the domain's, and an alignment should leave them where they are.

The joins map structurally. NEST and CONTAIN are part-of relations at different grains, and RELATE is a typed association that an upper ontology would call a relation between occurrents or continuants depending on the link type.

The mapping's losses are the point of recording them. A team that aligns its ledger to BFO or CCO records what it could not map, and the record is what lets a later reader see that the alignment was partial rather than total.

Adopting an upper ontology

Adoption runs through the gate the book calls P-12. The gate's four moves are: choose the upper ontology, map the local terms, record the losses, and version the mapping.

The choice is a commitment rather than a preference. BFO and CCO are not interchangeable, and a team that adopts one and then aligns to the other has two mappings and one vocabulary.

The mapping is the work. A local term is mapped to a category, the mapping is written down with the reason, and the reason is what a reviewer checks. A mapping with no reason is a guess that will be reversed by the next reviewer.

The losses are the honesty. Every alignment drops something—a nuance of the domain term, a grain of the event, a unit convention—and the loss record is what keeps the alignment from claiming to be complete.

The version is the maintenance. An upper ontology changes, and a mapping that is not versioned becomes a mapping nobody can trust. The version is the difference between an alignment and an accident.

What the appendix does not do

The appendix does not choose an upper ontology for the reader. The choice depends on the domains a ledger serves and on the organizations it must exchange with, and a book cannot make that decision from outside.

The appendix does not claim that the book's grammar is an upper ontology. The fifteen roles and eight dimensions are a record grammar, and a record grammar is a different kind of artifact from a category system. The two can be aligned, and the alignment is work rather than identity.

The appendix does not settle the strain points. Each strain point is a decision a team makes with its domain, and the book's contribution is to name the decision rather than to answer it in advance.

Appendix D · Glossary

Each entry gives the term's meaning in one or two sentences and a *Not* clause that says what the term is commonly confused with. The *Not* clause is the part worth reading twice, because most disagreements about the architecture are disagreements about which term is being used.

The entries are ordered alphabetically. A term that is used in a specialized way in the book carries a pointer to the part that introduced it, and a reader who wants the argument rather than the definition should follow the pointer.

acts. The four operations on a record: supersede, retract, redact and delete, each with its own receipt. Not: the four operations on the world, which are somebody else's business.

address. Where a claim is written: its identifier, its version and its clock, which together let a reader find the claim and date it. Not: a URL, which locates a page rather than a claim.

alias. A name or identifier that resolves to a canonical identifier. Not: a second identity, and not a merge.

bearing. One of the compass's four directions: claim, warrant, address and license. Not: a part of the book, which serves a bearing rather than being one.

candidate. A claim that has been extracted but not yet licensed, held with the reason it failed. Not: a rejected claim, which is dropped rather than held.

canonical identity. The single identifier that a referent's aliases resolve to, stable across renames. Not: a primary key, which is a storage decision.

claim. A predication together with its holder, stance, span, version and defeat condition. Not: a sentence, which may carry several claims, and not a row, which may carry none of the fields.

closure. The set of nodes a walk reaches from a starting claim through licensed joins. Not: the set of all claims, and not the answer.

compass. The four bearings of the book: claim, warrant, address and license, read in that order. Not: a summary, which replaces the book rather than being used beside it.

conflict set. The claims that disagree about the same theme, role and time, held together rather than resolved on entry. Not: an error state, and not a queue to be cleared.

decision. The act taken on a finding, naming the act, the actor, the date and the scope. Not: a finding, which resolves evidence rather than committing anything.

dictionary. The private terms a sender uses, with a definition and a scope for each, travelling with the claims that use them. Not: a glossary of the book's own terms, which is this appendix.

dimension. One of the eight properties that say how an act was made: voice, aspect, mood, tense, number, modality, polarity and force. Not: a role, which says who or what participated.

envelope. The wire form of a message: terms, claims, dictionary segment and state. Not: a payload with metadata, where the metadata can be removed.

finding. The resolution of evidence, naming the claims considered, the author and the reason, and superseding the conflict rather than deleting it. Not: an opinion, and not a decision.

force. The dimension that says what a claim commits its holder to: assertion, definition, directive, commitment, question, hypothesis, attribution, evaluation or fiction. Not: stance, which says how strongly the holder holds it.

gap. The missing evidence that kept a claim from being licensed, recorded rather than dropped. Not: a bug in the pipeline, though a pipeline bug can produce one.

ghost. A referent that cannot be resolved to a canonical identifier, held as a ghost until it can. Not: an error, and not a duplicate.

guard. The check a pattern relies on to catch its own failure, stated on the pattern's card. Not: a gate, which runs in the compiler rather than on the page.

ledger. The record of claims, with their warrants, addresses and licenses, which is canonical for the organization. Not: a database, which may be a projection of it.

license. What may be done with a claim, including what the record refuses to license. Not: truth, and not a control.

loss. What a projection dropped, declared by the projection rather than discovered by the reader. Not: an error, unless it is undeclared.

Omega. The one-page governance summary in Appendix H, describing the minimum permission profile a ledger needs. Not: an SPL profile.

pattern. A reusable shape with a failure it prevents, named, triggered and guarded on one card. Not: a template, which is filled in rather than reached for.

predication. The smallest unit that can be true or false: a verb with its roles, its dimensions and a span. Not: a word, a phrase or a sentence.

prediction. A claim about what will happen, recorded with the date it can be checked and the observation that would refute it. Not: a promise, which commits the holder rather than describing the world.

profile. A named subset of the grammar and vocabulary that a domain may use, such as Core, Content, NSM or Domain. Not: a version of the language, which is a different axis.

projection. A store or view that represents the ledger for a purpose, with declared loss. Not: a copy, which claims to be complete.

quarantine. The receiver's state for an envelope with at least one unverifiable claim, held rather than partially accepted. Not: deletion, which leaves no record.

refusal. An output that declines a claim and names what is missing, the nearest claim and what would license it. Not: an error, and not an apology.

role. One of the fifteen closed argument positions: agent, patient, theme, experiencer, recipient, beneficiary, time, location, source, destination, manner, instrument, cause, reason and attribute. Not: a BFO role, which is a dependent continuant that a thing bears for a time.

span. The location a claim came from, to the page and the character range. Not: a citation, which may point at a summary.

SPL. The book's meaning language, with four profiles and a public syntax for claims, terms and alignment. Not: a programming language, and not a wire format.

SPL-Gov. The minimal permission profile in Appendix H, closed and decidable, used to express grants and refusals. Not: an SPL profile, which is a subset of the meaning language.

stance. How a holder holds a claim: asserted, reported, inferred, candidate and the rest. Not: force, which says what the claim commits the holder to.

supersession. The relation between an old version and the new one that replaces it, with both preserved. Not: an overwrite, which loses the old version.

synapse. The card that holds one claim: verb at the hub, participants on closed spokes, dimensions on the rim and the span beneath. Not: a frame, which is a type rather than an occurrence.

terms. The five fields of an envelope: sender, receiver, purpose, expiry and handling. Not: a license's fine print, which is a different document.

UMS. The universal meaning substrate: the ledger's schema, in third normal form, with the claim at its center. Not: a product, and not a universal ontology.

warrant. Who says a claim, how strongly and on what evidence, which is the second bearing. Not: a license, which says what may be done with the claim rather than who stands behind it.

Using the glossary

A term is defined once and used consistently, and the *Not* clause is what keeps the use consistent. A reader who finds a sentence where a term seems to mean something else has found either an error in the book or a term that needs a new entry.

The glossary is also a checklist for a team's own vocabulary. A team that cannot write a *Not* clause for its terms has not decided what they exclude, and a term with no exclusion is a term that will be stretched until it means nothing.

The entries point at the argument rather than repeating it. A definition is a sentence; the argument for the definition is the part that made it, and the pointer is the shortest path between the two.

Appendix E · Schema Contract

The schema contract is what an implementation must satisfy and what a projection may vary. It is written as tables, grains, keys, statuses and invariants, and it deliberately contains no DDL: the DDL is a projection of the contract, and a projection may change without changing the contract.

A team that implements the contract in a relational kernel, a graph store or a document store is implementing the same twelve tables at different physical shapes. The contract is the part that must survive the choice of store.

The twelve tables

Table	Grain	Key	Purpose
document	one source version	document_id	what was read, and when
span	one character range	span_id	where a claim came from
sense	one meaning in one namespace	sense_id	lemma, POS, microgloss, namespace
entity	one referent	entity_id	a thing a claim can be about
claim	one assertion	claim_id	holder, stance, polarity, time, status
predication	one verb frame	predication_id	verb sense and dimensions
role_binding	one participant	binding_id	which role, filled by what
join_claim	one typed link	join_id	nest, contain or relate
group	one purpose	group_id	a purpose with a scope
group_ member	one membership	member_id	member and role in the purpose
authority	one grant	grant_id	who may do what, until when
supersession	one correction	supersession_id	what replaced what, and why

The table is the contract's spine. Each row's grain is one thing—one document version, one character range, one assertion—and a table whose rows hold more than one grain is a table that will need a projection to answer a question.

Two fields need a rule because they appear in more than one place. Force is a dimension on the predication row, not a field on the claim: the frame's force is what the verb and its dimensions make, and the holder's stance is what the claim adds. Stance is the holder's commitment—asserted, reported, inferred, attributed—and it is stored on the claim. A claim with no stance is refused, and a predication with no force is refused. Candidate is a status rather than a stance, and it names a claim that passed its gates but has not yet been licensed.

Several record kinds live outside the twelve tables and must not be forced into them. A finding is an extension record that resolves a conflict and names the claims considered, the author and the reason; a refusal names the missing license, the nearest claim and the input; a gap names what the compiler could not license; a receipt records a state change; a decision records an act taken on a finding. Each is an extension record with a key, each is append-only, and each is declared by an implementation that needs it. The twelve tables are the kernel, not the whole schema.

Grain and key

The grain is what one row means. The claim table's grain is one assertion, and an assertion is one predication held by one holder with one stance. A table that stores two assertions per row has two claims and one key, and the second claim cannot be superseded alone.

The key is the address. Keys are stable, opaque and assigned at intake, and a key is never a natural value such as a name or a date. A natural key changes when the world does, and a key that changes is a key that breaks every reference to it.

The grain and the key together are the table's promise. A consumer who knows the grain can aggregate without double counting, and a consumer who knows the key can join without guessing.

Statuses

A claim row carries a status, and the status set is closed: candidate, accepted, superseded, retracted, redacted, deleted. The six are the states a claim can be in, and each transition has a receipt.

Candidate is a claim that has been extracted and not yet licensed. Accepted is a claim that passed its gates and carries a span, a stance and an authority. Superseded is a claim replaced by a later version, still readable. Retracted is a claim withdrawn by its holder. Redacted is a claim whose content is partly hidden, with the hiding recorded. Deleted is a claim removed under a rule, with the rule recorded.

The statuses are not a workflow. A candidate may become accepted, and an accepted claim may become superseded or retracted, and the transitions are acts rather than stages. A status set that requires every claim to pass through every state is a workflow wearing a schema.

The authority table has its own status set: active, expired, revoked. The three are the states of a grant, and a grant that is expired is not a grant that never existed. A grant is stored in the authority table and uses that set; the six claim statuses do not apply to it, and the deontic state a governance profile reports (granted, denied, conflicted, undetermined) is computed from the operator, the scope and the authority row rather than stored as a claim status.

The clocks and the span

Five clocks keep the record's history readable, and each is a field rather than a convention. Validity time says when the claim was true in the world; assertion time says when the holder asserted it; decision time says when an act was taken on it; receipt time says when the record of the act was written; supersession time says when a later version replaced it. A record with fewer clocks cannot answer what it knew on a date, and a record with one timestamp has conflated five questions.

Clock	Field	Nullable
validity	valid_from, valid_to	yes, for a claim that is always true
assertion	asserted_at	no
decision	decided_at	yes, until a decision exists
receipt	recorded_at	no
supersession	superseded_at	yes, until a correction exists

A span is a character range in one document version: a document identifier, a start offset, an end offset, a page and the offset basis. The range is half-open, the offsets are counted in the document's characters with the basis declared, and the span is bound to the version it was taken from. A span that names no version is a span that cannot be checked after a correction.

The worked rows

A single sentence produces rows in seven tables, and the rows are worth seeing together because the shape is the contract.

The sentence is *A. Rao closed valve V-12 on 4 March*. The document table holds the log's version; the span table holds the character range; the sense table holds the predicate's microgloss; the entity table holds the valve and the technician; the claim table holds the

holder, stance and status; the predication table holds the sense, force and dimensions; and two role_binding rows hold the agent and the theme.

The claim row names the predication, the holder and the span, and it names the valid time. The predication row names the sense and the eight dimensions, and the dimensions are stored even where they take their defaults.

The role_binding rows bind A. Rao to AGENT and the valve to THEME, and the binding is stored with the claim rather than with the predication. Two claims about the same predication by different holders share the predication and differ at the claim row, which is the shape that keeps testimony apart from fact.

The supersession table is empty for this claim. When the log is corrected, a second claim enters and a supersession row names both, and nothing is overwritten.

Three rows per table are enough to fix the grain, and the rows below use the same sentence and its correction.

Table	Row 1	Row 2	Row 3
document	log-2024-03 v2	log-2024-03 v3 (corrected)	work order 881
span	p.2, c.1-64	p.2, c.1-64	p.1, c.5-40
sense	close.verb.seal_shut	close.verb.seal_shut	replace.verb.swap_part
entity	valve V-12	A. Rao	seal 44-2
claim	c-881 (asserted)	c-902 (supersedes c-881)	c-903 (accepted)
predication	p-441	p-441	p-512
role_binding	AGENT=A. Rao	THEME=V-12	AGENT=crew B
join_claim	rel:cause c-903	rel:sequence c-902	nest:episode e-22
group	overhaul Q1	overhaul Q1	repair 881
group_ member	pump set	valve set	crew set
authority	grant 77 active	grant 77 expired	grant 81 active
supersession	none	c-881 -> c-902	none

The table is not exhaustive and does not need to be. It fixes the grain of each row, which is what an implementer needs before writing a key.

Cardinality and optionality

The relationships between the tables are part of the contract, and each is one-to-many or optional in a stated direction.

Relationship	Cardinality	Optional
document -> span	one document, many spans	span always has a document
document -> claim	one document, many claims	a claim always names a document, whether held locally or reached by reference
predication -> claim	one predication, many claims	claim always has a predication
claim -> role_binding	one claim, many bindings	at least one binding per claim
claim -> join_claim	one claim, many joins	joins optional
group -> group_ member	one group, many members	at least two members per group
claim -> authority	one authority, many claims	required when the force carries an authority warrant, optional otherwise
supersession -> claim	two claims per row	supersession optional

The optional column is where a builder invents. The contract states the four optional relationships so that a builder does not have to choose: joins are optional, groups need two members, an authority is required only when the force carries an authority warrant, and a claim may have no supersession.

Tabular intake

A table row has no character range, and invariant two still applies to it. The loader mints a span over the row's cell range in the source file, records the file version and the row address, and refuses a row whose provenance cannot be stated. The synthetic span is a real span: it names a document, an offset range and a version, and a reader can check it against the file.

One row is one claim. A row that would assert two is refused rather than split, because a split would put two claims behind one row number and the second could not be superseded alone. An empty cell means "not supplied"; a repeated role column means a second participant in that role; a delimiter inside a cell is text rather than structure.

The loader converges with the prose lane on the claim, not on the row. Two records converge when their sense, dimensions, holder, stance, polarity, time and role bindings agree after identifiers and provenance are set aside, and a divergence is a lane defect rather than a second fact.

The distinction profile

The profile names the distinctions an implementation must protect, because an implementation that loses one will pass its smoke tests and fail its first dispute.

Distinction	Protected by	Failure if lost
claim versus sentence	one claim per card; nested content linked	a paragraph's claims collapse into one
stance versus force	separate rows: stance on the claim, force on the predication	testimony reads as fact, or an order as a report
sense versus entity	separate tables and identifiers	a word's meaning merges with its referent
version versus validity time	separate clocks on the claim	the record cannot say what it knew in March
candidate versus accepted claim	the status set and the gap row	a guess becomes a fact silently
supersession versus retraction	two rows and two receipts	correction and withdrawal become one act
assertion versus commitment	force values	a promise reads as a description
reference versus value	the envelope's two transport modes	a pointer arrives where content was meant

The profile also names the packages a build requires: the closed sets of Appendix B for roles, dimensions and forces; the schema of this appendix; and the golden pairs of Appendix F for intake.

The comparison rules are two. Compare at the claim grain, never at the sentence, because two sentences can carry one claim and one sentence can carry three. Compare with stance and version attached, because a claim compared without them is a comparison of strings.

The expected failure cases are the ones a build must demonstrate: a claim with no span is refused; a role outside the closed set is refused; a force that is not on the list is refused; a measure without a unit is held; a supersession with one endpoint is refused.

The permitted decisions are the ones a builder may make without asking: the choice of store, the shape of opaque keys, the indexing strategy, the order of gates that do not depend on each other, and the representation of the status set in the store.

The evaluation requirements are the acceptance tests of Appendix F plus two the profile adds: every refusal must name its missing license, and every accepted claim must resolve to a row in every table it touches.

The deployment manifest

A deployment declares seven fields, and the declaration is what makes a build reproducible.

Field	What it names	Example
input schema	the shape of what arrives	prose sentences with a document id and a page
interpretation configuration	which profile and dictionary are in force	SPL-Core; domain profile maintenance-2
pinned package versions	the versions the build was tested against	schema v3; closed sets v2; golden pairs v4
reasoning profile	what inferences the build may draw	none beyond the mandatory components of a definition
decision policy	who may accept, hold or refuse	gate battery accepts; a named reviewer clears holds
output schema	what leaves the system	an envelope with terms, claims, dictionary segment and state
evaluation report	the tests run and their results	Appendix F groups, pass/fail/ blocked/not run

The manifest is short because a long manifest is not read. The seven fields are the ones a second team needs to reproduce a first team's results, and a field that is missing is a reproducibility claim the deployment cannot support.

The invariants

Six invariants are the acceptance tests the DDL cannot express. They are the part an implementer must test directly.

Rows are append-only; a correction is a supersession, not an update. Every claim has a span; a claim without a span is refused. Every predication has a force and a sense; neither is inferred at read time. Every role is in the closed set; a role outside the set is refused. Every sense has one microgloss per namespace; siblings may not share one. A supersession names the claim it replaces and the claim that replaces it.

The invariants are six because each one corresponds to a failure mode the book spent a part on: the overwrite that loses history, the claim without a place to check, the force that is guessed, the role that is invented, the sense that is duplicated and the correction that names nothing.

An implementation can satisfy every table and fail every invariant. The tables describe what is stored, and the invariants describe what the storage must never allow.

What the contract refuses

The contract refuses a claim with no span, a role outside the closed set, a correction that overwrites, a force inferred at read time and a supersession with one endpoint. Each refusal is a check an implementation runs, and each check has a fixture in Appendix F.

The contract also refuses to name a store. A relational kernel, a property graph and a document store are all projections of the twelve tables, and the contract is the part that must hold across them. A contract that named a store would be a product recommendation rather than a schema.

The contract refuses to be exhaustive. Twelve tables are the minimum for six questions —what was said, what the terms mean, what is asserted, who participated, how claims compose and who stands behind them—and an implementation may add tables for its own purposes. The additions are projections or extensions, and the twelve remain the part that a second system must match to exchange claims.

Reading the contract

A reader who wants to implement the ledger reads the twelve tables, the grain and key rules, the statuses and the six invariants, in that order. The order matters because the tables are meaningless without the grain, and the invariants are what the grain is for.

A reader who wants to evaluate an implementation asks for the grain of each table and the status set of the claim row. An implementation that cannot state its grain has not decided what a row means, and an implementation whose status set is open has not decided what a claim can be.

A reader who wants to compare two implementations compares their invariants. Two stores with the same twelve tables and different invariants are different ledgers, and the difference is the part that matters when a claim is disputed.

Appendix F · Golden Pairs and Acceptance Tests

A golden pair is an input and its expected record. A pair is a fixture: the sentence goes in, the claim or the refusal comes out, and a compiler that produces anything else has failed the fixture. The acceptance tests are the pairs plus the schema's invariants, and together they are what a team runs before trusting a change.

What a pair contains

A pair has four parts: a sentence, the expected claim or refusal, the reason, and the standing of the expectation. The reason is what a reviewer reads when the fixture fails, and the standing says whether the expectation is designed, measured or borrowed from a source.

A pair that expects a claim names the predicate, the roles and the span. A pair that expects a refusal names the missing license, the nearest claim and the status. The two kinds of pair are equally important, because a compiler that never refuses is as broken as one that never accepts.

The pairs below are the book's own fixtures. They are written against the pump and the contract, and a team's domain will add its own, in the same four-part form.

The pairs

Input	Expected	Why
The pump failed on 4 March.	claim FAIL(theme=pump; time=4 March); status accepted	a plain assertion with a time
The pump may have failed.	claim FAIL(theme=pump; modality=possible); status accepted	modality is a dimension, not an escape
The notice says the pump failed.	nested claim FAIL inside an attribution to the notice	reported speech keeps its holder

Continued on next page

(Continued)

Input	Expected	Why
The pump failed because the seal eroded.	two claims joined by a causal link	a cause is a claim, not a modifier
The pump failed and the floor flooded.	two claims joined by sequence	two predicates, two claims
The operator was negligent.	refusal: missing finding; nearest record N-14	no finding licenses the claim
The pump's pressure is high.	refusal: missing unit and scale	a measure without a unit is unusable
Records are missing.	refusal: scope ambiguous; nearest record log-2024-03	missing from where is not stated
The technician closed the valve.	claim CLOSE(agent=technician; theme=valve)	an agent and a theme
The valve was closed.	claim CLOSE(theme=valve; voice=passive)	voice is a dimension
Crew B replaced the seal on 7 March.	claim REPLACE(agent=crew B; theme=seal; time=7 March)	an agent, a theme, a time
It failed.	refusal: unresolved pronoun; no referent	an anaphor with no antecedent

The table is deliberately mixed. Eight pairs expect a claim and four expect a refusal, because the failures a compiler makes are symmetric: it accepts what it should refuse and refuses what it should accept. The candidate status is not used here; a candidate is a claim that was extracted and not yet licensed, which is a different case from an ambiguous question.

Every expected claim also carries the span of the input in the fixture's document; the table omits the span for width, and the fixture's document id supplies it. A pair that could not name a span would be a pair the compiler could pass without reading the text.

A pair is not an example. An example in a chapter shows what a good record looks like; a pair fixes what the compiler must produce, and the fixing is what makes it a test. A team that writes pairs instead of examples has turned its documentation into a suite.

The pronoun pair is the smallest test of the whole architecture. A compiler that resolves *it* by guessing has invented a referent, and the invention is the same failure as an invented claim at a smaller grain. A suite without a pronoun pair is a suite that has not tested the cheapest way to fail.

The acceptance tests

The tests are grouped by the layer they check, and each test has a pass condition and a failure condition. A test that cannot be failed is not a test.

The schema group checks the six invariants: append-only rows, a span on every claim, force and sense on every predication, roles inside the closed set, one microgloss per namespace and a supersession with two endpoints. Each test runs one write that should be refused and one that should be accepted.

The intake group checks the compiler's gates: a claim with a span passes, a claim without one is held, a stance is assigned rather than defaulted, a role outside the set is refused, a duplicate sense is refused and a candidate is recorded rather than dropped.

The projection group checks the losses: a projection declares what it drops, a round trip rehydrates with a qualification, the convenience relation is marked and the version is recorded. A projection that passes its own tests and fails the round trip has hidden a loss.

The wire group checks the envelope: terms are present, a public claim travels by reference, a private claim travels by value, a dictionary segment covers the private terms and a message with an unverifiable claim is quarantined rather than partially accepted.

Each group has between four and six tests, and the groups are ordered by cost. The schema tests are cheap and run on every commit; the intake tests run on every gate change; the projection tests run on every release; the wire tests run on every partner exchange.

The gate battery

The intake group's checks are the battery's gates, and the gate list is closed. A gate is named, it has one pass condition and one outcome, and a sentence that fails a gate is refused at that gate rather than at a later one.

Gate	Pass condition	Failure outcome
document	the sentence's document has a version	refusal: no document
span	a character range in that version resolves	refusal: no span
sense	exactly one sense in the namespace	refusal: duplicate sense
stance	a stance is assigned from the embedding	refusal: no stance
force	one of the nine forces is derived	refusal: force outside the set
roles	every participant binds to one of the fifteen	refusal: role outside the set
dimensions	all eight carry a stored value	refusal: dimension outside its set
authority	the force's warrant exists	refusal: missing finding or grant

The order is a contract rather than a preference: document, span, sense, stance, force, roles, dimensions and authority are checked in that sequence, because each gate depends on the one before it. A gate that does not depend on its predecessor may be reordered by an implementation, and the battery's fixtures say which do.

The battery is what a builder runs before trusting a change. A failing gate is a defect in the sentence or in the gate, and the refusal's gate name is what tells the two apart.

Failure statuses

A test run reports one of four statuses: pass, fail, blocked and not run. The four are distinct because a report that collapses them hides the difference between a system that is broken and a system that was not tested.

Fail means the system produced an outcome the fixture rejects. The failure is the system's, and the fixture's reason says what the system did instead.

Blocked means the test could not run because a dependency was missing—a store that would not start, a partner that did not answer. A blocked test is not a pass, and a run with blocked tests is a partial run.

Not run means the test was skipped by choice. A team may skip the wire tests before a partner exists, and the skipping is recorded. A report that says "all green" while half the tests were not run is a report that has confused silence with success.

The statuses are also the vocabulary of a postmortem. A failure that was blocked for a month is a different story from a failure that was not run, and the difference is the first question a reviewer should ask.

Running the tests

The tests run on every change to the schema, the gates or the projections. A change that passes the schema group and fails the intake group has changed the meaning of a claim without changing a table, and the intake failure is the one that matters.

The pairs grow with the domain. A team adds a pair when it finds a sentence the compiler handles wrongly, and the pair is the record of the bug. A fixture added after the fix is a regression test; a fixture added before it is a specification.

The first pairs a team adds are usually its own vocabulary. A domain's verbs are the ones the compiler will meet first, and a pair per verb is a cheap specification of the intake. A team with forty verbs and forty pairs has a specification, and a team with forty verbs and a style guide has an aspiration.

A worked failure shows the value. A compiler accepts *the pump failed because the seal eroded* as one claim, the pair rejects it because the causal link needs its own claim, and the rejection names the second claim the compiler missed. The pair has turned an argument about design into a list of missing rows.

The tests are not a benchmark. They do not measure speed, coverage or accuracy, and a compiler that passes every suite can still be slow and partial. The tests measure one thing: whether the compiler produces the record the architecture says it should.

The discipline is the point. A team that runs the pairs before every change has a compiler whose behavior is known, and a known compiler is the only kind that can be trusted with a ledger.

What the pairs do not test

The pairs do not test whether a claim is true. A pair checks that the compiler produced the claim the architecture says it should, and the claim can be false and still pass. Truth is the world's business, and the pairs are the record's.

The pairs do not test scale. Twelve pairs say nothing about a million claims, and a compiler that passes every fixture can still fall over on a corpus. Scale is a separate measurement, and the appendix does not pretend to make it.

The pairs do not test the model. A pair fixes the output of the pipeline, and a model that proposes well and a model that proposes badly both pass if the gates dispose correctly. The model's quality is a different test, and it belongs with the evaluations rather than with the fixtures.

The build sheet

Every buildable component has a sheet, and the sheet is what a builder reads before writing the component. Seven fields are enough, and a component without a sheet is a component the book has under-specified.

Field	What it answers
purpose	what the component is for, in one sentence
interface	what arrives and what leaves, in data shapes
invariants	what must never be true after an operation
failure statuses	the named refusals the component can return
golden pairs	the fixtures the component must pass
acceptance tests	the observable pass conditions
non-goals	what the component must not do

The sheet is one page. A field that needs a paragraph is a field that will be skipped, and a skipped field is where the builder invents.

A worked sheet for the ledger's schema component shows the shape. Purpose: hold the twelve tables and their invariants. Interface: rows in, rows out. Invariants: the six of

Appendix E. Failure statuses: refused for a missing span, an unknown role or a one-ended supersession. Golden pairs: the schema pairs above. Acceptance tests: the schema group. Non-goals: the schema does not decode prose, does not resolve entities and does not answer questions.

The sheet does not replace the appendix; it points at the part of the appendix the component needs. A builder who reads the sheet and the two appendices has read the specification, and the reading is short on purpose.

What the pairs do test is the contract between the sentence and the record. That contract is the part the architecture can guarantee, and a team that tests it has tested the part that will not change when the model does.

Appendix G · SPL 0.1

SPL is a constrained language for meaning packages. A package is a sense's structured expression: typed variables, predicate applications, role bindings, force annotations, molecule references and quoted content.

SPL is an engineering language rather than a universal logic, and it is not an official formalization of any linguistic theory. Passing the checks below establishes well-formedness, not lexical adequacy.

Scope

A package says what a sense requires, what it typically involves and what it permits. The language is small enough to be checked mechanically and expressive enough to hold the senses a ledger needs.

The scope is deliberately narrow. SPL does not attempt to represent every meaning, and it does not attempt to prove that a definition matches ordinary usage. It represents a sense as a structured object whose inferences can be computed on a declared fragment.

Profiles

Profile	Adds	Guarantee
SPL-Core	typed variables, predicates, equality, connectives, quantifiers, implication, definitions	many-sorted first-order representation; reasoning only on a declared fragment
SPL-Content	THINK, WANT, SAY, KNOW with quoted content	content is an object; it is not asserted by the embedding
SPL-NSM	primes, molecules, licensed frames, explication mapping	vocabulary and frame checks; not a claim of NSM compliance
SPL-Domain	domain predicates such as approved_by or has_onboard_propulsion	declared extensions, not new primes

A package can be well-formed in SPL-Core and only partially reduced in SPL-NSM, and the conformance report records which profiles were checked. The profiles are subsets rather than versions: a package does not graduate from one to another.

SPL-Gov is not a profile of this language. It is the book's governance profile, specified in Appendix H, and it uses the claim record with its own operator set.

The prime endpoints deserve a note. The basic senses of KNOW, WANT and THINK are endpoints in SPL-NSM rather than targets for further decomposition, and their English spellings may carry other senses. Prime recognition is therefore meaning-sensitive, and a passing prime check is not a claim of NSM compliance.

Syntax

The grammar is deliberately small. The core productions are:

```
package ::= "sense" id "{" declaration* component* "}"
declaration ::= "var" id (":" sort)? | "molecule" id
component ::= force? "[" role* "]" expression
expression ::= prime | molecule_ref | predicate "(" term* ")"
 | expression connective expression
 | "not" expression
 | "if" expression "then" expression
 | quantifier id ":" sort "{" expression "}"
 | "quote" "{" expression "}"
force ::= "nec" | "typ" | "poss" | "inst"
```

A term is a variable, a constant or a quoted expression. A molecule reference carries its version and its argument bindings. Role names come from the fifteen-role inventory, and a role outside the inventory is a structural error rather than a new role.

The syntax has no surface ambiguity by design. A package that cannot be parsed is rejected, and a rejected package is not a weaker package; it is not a package.

Definitions

Three definition forms carry different inference directions, and the direction is the central safeguard.

Form	Meaning	Licensed use
necessary C(x) { φ }	C(x) -> φ	infer φ from C
sufficient C(x) { φ }	φ -> C(x)	infer C from φ
exact C(x) { φ }	C(x) $\leftrightarrow$ φ	unfold C to φ, and infer both directions

Only an exact definition may be unfolded as a meaning-preserving substitution. A necessary-only definition supports inference in one direction and must not be substituted

as an equivalence, because the substitution would claim a symmetry the definition does not have.

A definition also carries an authority basis: stipulated organizational, published technical, ontology axiom, proposed lexical analysis or operational task. The basis is what a reviewer reads when the definition is disputed, and an exact definition with a weak basis is still a claim rather than a discovery.

Force

Four force operators apply to components, and each changes what a package licenses.

The necessary operator says the component is true in every situation in which the sense applies. The typical operator says it is true in every typical situation, and a particular instance may violate it without falsifying the package. The possible operator says some compatible situation makes it true. The instance operator asserts the component of a specific assignment rather than of the generic sense.

The typical operator is annotation-only in the initial engine. No inference is drawn from it until a default-reasoning extension is specified and tested, because a typical property that silently becomes a universal requirement is the failure the annotation exists to prevent.

Semantics in brief

A model is a tuple of entities, situations, typical situations, an interpretation and an assignment. Situations are partial states of affairs ordered by extension, and the interpretation gives primes and predicates their extensions.

A package's meaning is the set of situation-and-assignment pairs that satisfy it, with the necessary components true in the situation, the typical components true in its typical extension and the molecule references expanded to their pinned bodies. The recursion is well-founded because the dependency graph is acyclic.

Quotation is where the semantics earns its keep. `THINK(a, quote{`φ`})` does not entail φ, and `SAY(a, b, quote{`φ`})` does not entail φ: the embedding reports the content without asserting it. Content identity is structural up to consistent renaming of bound variables, and logical equivalence does not substitute inside a quote.

KNOW has no built-in factivity rule. A rule such as `KNOW(a, `φ`) -> `φ belongs to a declared knowledge profile with its time and context, and a core that assumed factivity would erase the difference between belief and knowledge.

Expansion correctness is denotational. Full expansion replaces every molecule by its defining body, and the denotation after expansion must equal the denotation before it. A molecule whose expansion changes the denotation is a defective definition whatever its syntax, and selective expansion is permitted only when it preserves the denotation of the queried fragment.

Conflict resolution

Conflict	Response
structural: cycle, unbound variable, unknown role, undeclared force	reject; package unresolved
force: same component both necessary and possible without scope	flag; author must disambiguate
competing analyses of one sense	retain as separate candidates with provenance; do not conjoin
typical versus instance	soft warning; not a contradiction
typical used as a universal	soft warning; no inference in o.1
source assertion versus axiom	keep the source record; an acceptance policy selects the theory for a task
cross-path contradiction after expansion	mark the premise set inconsistent; do not merge
undocumented versus false	absence is never negated
evidence conflict	keep both sources; a conflict record rather than an inconsistent theory

The four statuses are distinct: ill-formed, internally inconsistent, competing but coherent, and compatible under different force. A checker that collapses them reports a package as broken when it is merely in dispute.

Alignment

An alignment reports a relation relative to a selected theory: equivalent, narrower, disjoint, possibly overlapping, properly narrowing or undetermined.

The test for equivalence is mutual entailment, and a missing necessary condition blocks it. An extra necessary condition supports narrower, and shared necessary conditions alone do not prove equivalence. Necessary-condition coverage is computed before full coverage, because a package that misses a necessary condition cannot be equivalent however many typical components agree.

A hard conflict exists when a necessary component of one package contradicts the other, and it blocks equivalence and subsumption alike. It is reported separately from an unknown component, since one is a contradiction and the other is a gap.

The relation is semantic rather than lexical. It does not depend on surface syntax once the packages are validated, and it does not depend on the similarity of their labels. Similarity and proof are separate services, and a score is a summary rather than a relation.

Conformance

A package receives separate flags rather than a single verdict: syntax valid, schema valid, references resolved, scope valid, types valid, definition dependencies acyclic, reasoning profile supported, NSM frames checked and linguistically reviewed.

The flags are separate because the failures are separate. A package can be syntactically valid and semantically wrong, and a single verdict would hide which. The last flag is a human judgment with a named reviewer and a date, and it is the only one a machine cannot set.

A worked package

The package for *promise* shows the shape the language is for.

```
sense promise_commitment {
 var speaker : someone
 var addressee : someone
 var action : something

 nec [agent: speaker, recipient: addressee, content: action]
 say(speaker, addressee) like-this: {
 after(now, do(speaker, action))
 want(speaker, know(addressee, will-happen(action)))
 }

 nec [agent: speaker, experiencer: addressee]
 if not(do(speaker, action))
 then can(addressee, think(bad-of(speaker) because-of(this)))

 typ [agent: speaker]
 want(speaker, do(speaker, action))
}
```

Reading: necessarily the speaker says something whose content is that the speaker will do the action and wants the addressee to know it; necessarily a failure licenses the addressee's bad thought; typically the speaker wants to perform the action.

The package does not require the action to occur, and it does not assert that the addressee believes anything. The two absences are the point: a promise that entailed its own fulfillment would be a strange object, and a promise that entailed belief would be a strange speech act.

What SPL does not claim

SPL does not claim that every meaning reduces to primes, that a well-formed package is a true or adequate definition, that exactness within the specification establishes lexical adequacy, that alignment is decidable or complete, that quoted content is true or accepted, or that the language is a step toward general intelligence.

The non-claims are the language's specification as much as the grammar is. A reader who wants a universal logic will find this language too small, and the smallness is what lets a checker be written and a dispute be settled.

Appendix H · Governance Profile and Omega

A governance statement is a claim about an action: who may do it, under what scope, until when and who can revoke it. That is a predication with a deontic operator, a holder, a grant and a defeat condition, and it belongs in the same record as every other claim. This appendix specifies the profile that carries it, SPL-Gov 0.1, and the one-page summary the book calls Omega.

Why a profile, not a second language

A second language would need a bridge. Every permission would require a mapping from the governance language's actions to the meaning language's predicates, from its subjects to the record's entities and from its decisions to the record's claims. The bridge would be unowned, and the first audit would find the permission and the claim disagreeing.

A profile keeps one grammar and one record. It adds operators, statuses and conflict rules, and the bridge disappears because there is nothing to bridge. SPL-Gov is therefore not a profile of SPL 0.1 in the technical sense; it is the book's governance profile, and it uses the claim record with its own closed operator set.

The operators

Seven operators are closed over actions and claims.

Operator	Meaning	Fields
permit	the actor may perform the action	grantor, actor, action, object, scope, window, basis, delegable, revocable
forbid	the actor may not perform the action	grantor, actor, action, object, scope, window, basis
oblige	the actor must perform the action	grantor, actor, action, object, scope, window, basis

Continued on next page

(Continued)

Operator	Meaning	Fields
power	the actor may change a normative position	grantor, actor, act, object, scope, basis
immunity	the actor is protected from a power	grantor, actor, act, object, scope, basis
delegate	the actor may pass a permission	grantor, actor, permission, scope, basis, delegable
revoke	the grantor ends a permission	grantor, permission, reason, date, basis

The set is closed for the same reason the roles are closed. A new operator is a new case in every checker, and a governance language that can invent an operator cannot be audited. Seven operators cover the grants an organization actually issues, and a grant that needs an eighth is a grant that needs a different institution.

Every operator carries a grantor, a basis and a date. A permission with no grantor is a rumor, and a permission with no date is unrevocable in practice.

A worked grant shows the shape. Crew B is permitted to replace the seal on PB-100-7 between 1 and 31 March, by the maintenance manager, under the annual overhaul program, with the permit delegable to a named subcontractor and revocable by the manager.

The grant names the pump as its object and the program as its scope, because a permit that names only one pump must be reissued for every pump. The object and the scope are different fields, and a governance profile that conflates them cannot say what a permit covers.

The delegation is the second row. The subcontractor's permission is the intersection of the crew's permission and the delegated scope, and the intersection is what keeps a delegate from passing more than the delegator holds. A delegation that widened the scope would be a new grant rather than a delegation.

The semantics

A permission denotes a constraint set over actions rather than a boolean, and the set is what composes.

Two permissions compose by intersection. A grant of up to 500 and a grant of up to 100 permit up to 100, and averaging them or taking the union would silently repeal one. Intersection is the only composition that cannot widen a permission, and widening is the failure a governance language exists to prevent.

A forbid overrides a permit within its scope. A permit and a forbid that cover the same action and scope produce a conflict rather than a vote, and the forbid governs

unless an exception is explicitly scoped. The asymmetry is deliberate: a system that resolved the conflict by majority would let two permissions defeat a prohibition.

Obligation is not permission. *Must* requires the action and *may* permits it, and a record that stores both as "allowed" cannot tell compliance from discretion. The distinction is the one an auditor asks about first.

Revocation is supersession. The permission leaves force, the historical permission remains readable with its window and its grantor, and the revocation does not establish that the permission was invalid when it was granted. The last point matters when a decision is reviewed years later: an act under a valid permission is not made invalid by a later revocation.

Delegation is bounded. A delegate cannot pass more than the delegator holds, and the child permission is intersected with the parent. An unbounded delegation chain is a permission that grows as it travels, and growth is the failure the intersection prevents.

The statuses

Status	Meaning
granted	in force within its window
denied	refused by a forbid or a failed condition
expired	the window closed
revoked	ended by a revoke
conflicted	a permit and a forbid cover the same action and scope
undetermined	the scope or the actor cannot be resolved

The statuses are records rather than log lines. A query for what an actor may do now filters on granted, and a query for what an action allowed on a past date reads the window and the revocation date. The two queries are different questions, and a system with one status field answers only the first.

The conflict rules

Three conflicts matter most, and each has a rule rather than a preference.

A permit and a forbid over the same action and scope produce the conflicted status, and the forbid governs. The record keeps both, because a later review may find that the forbid was itself invalid.

Two delegations that grant different scopes produce a narrower effective permission, and the record keeps both chains. The intersection is the answer to the question, and the chains are the answer to the audit.

A revocation with no grantor produces an undetermined status. The refusal to revoke anonymously is a rule rather than a limitation: a permission that anyone can end is a permission nobody can rely on.

Omega, one page

Omega is the summary a team can put on a wall. It states the profile's shape, the four composition rules and the three questions the profile answers.

```
Omega 0.1 \allowbreak{}—\allowbreak{} the minimum permission profile

A grant is a claim with a deontic operator, a grantor, an actor,
an action, a scope and a window.

Stored, the grant is an authority row with one of three statuses
 ↪  \allowbreak{}—\allowbreak{}
active, expired or revoked. Its deontic state (granted, denied,
conflicted, undetermined) is computed from the operator, the scope
and that row, and is not one of the claim statuses.

Seven operators: permit, forbid, oblige, power, immunity, delegate, revoke.

Composition
 permissions intersect, never union
 forbid overrides permit within its scope
 obligation is not permission
 revocation is supersession, delegation is bounded

Questions the profile answers
 what may this actor do now?
 what did it allow on a given date?
 who ended it, when, and on what basis?

Statuses: granted, denied, expired, revoked, conflicted, undetermined.
```

The summary is deliberately small. A wall page that fits is a page that is read, and a policy that needs a binder is a policy that is filed. The page is the profile's public face, and the profile is the page's argument.

What the profile refuses

The profile refuses to be a policy engine. It records what was granted, denied and revoked, and it does not decide what an organization should permit. The decision is the organization's, and the record is the evidence that the decision was made.

The profile refuses to enforce. A recorded obligation depends on the receiver, and an enforcement mechanism would be a different architecture. The book's claim is that the record is a contribution to governance rather than a substitute for it.

The profile refuses to be complete. Seven operators cover the grants the book examined, and an institution with a different set of normative positions may need an eighth. The set is closed within 0.1, and a later version may extend it with a migration rather than an improvisation.

The profile also refuses the word “approved”. Approval is a decision, and a decision is a claim with an author and a finding. A system that marks something approved without a decision record has replaced governance with a status field.

Appendix I · The Wire Dictionary

A dictionary segment is the part of an envelope that tells the receiver what the sender's private terms mean. It travels with the claims that use them, and without it the receiver reads the words and guesses.

The segment is small by design. It carries the entries the envelope's claims actually use, and a segment with fifty entries for three claims is a sign that the claims are carrying their own glossary rather than pointing at one.

The envelope

The envelope is the wire unit, and the segment is one of its four parts. The other three are the terms, the claims and the state, and a message missing any of the four is a payload rather than an envelope.

Part	What it carries
terms	sender, receiver, purpose, expiry, handling
claims	public claims by reference, private claims by value
dictionary segment	the entries the claims use, with a segment version
state	received, verified, closed or quarantined

A public claim travels by reference: a claim identifier, the ledger endpoint that holds it, the version and a digest. A private claim travels by value: the claim's text, its span in the sender's record, its stance and the sender's name. The two encodings are not interchangeable, and an envelope that sends a public claim by value has paid bytes for no gain.

The state is the receiver's verdict rather than the sender's claim. Received means the envelope arrived and nothing has been checked; verified means every claim resolved; closed means the receiver accepted the whole envelope; quarantined means at least one claim could not be verified and the envelope is held whole. A sender may propose received, and only the receiver writes the other three.

Closure is a test, not a mood. A claim is verifiable when its span resolves in the named version, its predicate and roles are inside the closed sets, and any authority it needs has an active grant. The receiver walks the claims in order and stops at the first failure: an

envelope is quarantined whole rather than partially accepted, and the quarantine record names the failing claim and the rule it failed.

A quarantine is released by a full re-send or by a named reviewer's clearance, and the release is recorded with its reason. A partial read is not a release, and a message that is read while quarantined has been read without a verdict.

What an entry carries

An entry has six fields, and each closes a question the receiver would otherwise ask by email.

Field	Question it closes
term	which word in the claims is being defined
referent kind	whether the term names a public thing or a private one
referent	the canonical identifier or the private description
scope	where the definition applies
version	which segment version the entry belongs to
owner	which organization stands behind the definition

The referent kind is the field that keeps the segment honest. A public term's referent is a canonical identifier that both organizations can dereference, and the entry is a bridge. A private term's referent is a description that only the sender can verify, and the entry is a definition rather than a fact.

The owner is the field that keeps the segment attributable. A definition with no owner is a definition nobody stands behind, and a receiver who relies on it has relied on an anonymous claim.

A worked segment

```
segment s-12 version 3 owner org A entries 4

term: pump kind: public referent: PB-100-7
term: seal kind: private referent: "shaft seal, part 44-2"
term: crew kind: private referent: "crew B"
term: archive kind: private referent: "records room 3"
```

The first entry is a bridge: the sender's *pump* is the thing both organizations already name `PB-100-7`. The other three are definitions: the sender's *seal*, *crew* and *archive* are described rather than identified, and the descriptions are the sender's.

The segment carries its version because a term's meaning can change. A new segment that redefines *seal* is version 4, and an envelope that cites version 3 has not silently adopted version 4. The version is what lets two claims about the same word be compared without assuming the word means the same thing.

The rules

Four rules govern a segment, and each is a rule the receiver can check.

The first is that a term appears once per segment. A segment with two entries for one term has two meanings and no way to choose, and the receiver who chooses has invented a definition.

The second is that a public entry's referent resolves. A bridge that points at an identifier the receiver cannot dereference is not a bridge, and the receiver cannot use the entry.

The third is that a private entry's scope is stated. A description without a scope is a description that applies everywhere, and a description that applies everywhere is a definition the receiver will over-extend.

The fourth is that the segment is versioned and the version is cited. An envelope that cites no segment version is an envelope whose terms cannot be dated, and a term that cannot be dated cannot be compared with a later one.

What the receiver does

The receiver reads the segment before the claims, because the claims use the terms the segment defines. The reading order is not a formality; a receiver who reads the claims first has already guessed at the meanings.

For a public entry, the receiver dereferences the identifier and confirms that the claim's object is the thing the identifier names. A mismatch is a segment error rather than a claim error, and the two are reported differently.

For a private entry, the receiver accepts the description for the purpose in the envelope's terms and records the acceptance. The acceptance is scoped: the receiver may rely on *"shaft seal, part 44-2"* for the audit, and the acceptance does not make the description a public fact.

When the receiver cannot accept an entry, the envelope is quarantined rather than partially read. The quarantine is the same rule as the claim's: the envelope is the unit because the claims were sent as an argument.

What the segment is not

The segment is not a glossary. A glossary defines the sender's terms for a human reader, and a segment defines them for a receiver's machine and reviewer. The two may overlap, and the segment is the one that travels.

The segment is not an ontology alignment. An alignment maps terms into a shared category system and records the losses; a segment names what the sender means without claiming that the meaning fits a category. The alignment is a larger project, and the segment is the minimum a single envelope needs.

The segment is not a contract. The terms of the envelope are the contract; the segment is the vocabulary the contract's claims use. A segment that tried to carry obligations would duplicate the terms, and the duplicate is where the two documents would disagree.

How segments accumulate

A single envelope carries a segment, and a series of envelopes carries a series of segments. The series is where a translation memory accumulates: a term redefined once is redefined in a new version, and a receiver who has seen version 3 can read version 4 as a change rather than as a new word.

The accumulation has a cost. A series of segments is a record of the sender's vocabulary changing, and a receiver who keeps the series can see when a definition moved. The keeping is the receiver's choice, and a receiver who keeps only the latest segment has kept the state rather than the history.

The accumulation also has a benefit. A receiver who has read many segments can predict which terms will need definitions, and the prediction is what turns an exchange from a negotiation into a routine. The routine is the point of a wire that carries its terms.

Appendix J · Specimens

The book returns to a small set of objects, and each returns because it demonstrates one thing well. This appendix lists the specimens, says what each demonstrates and says where the demonstration is made.

A specimen is not an example. An example illustrates a claim; a specimen is reused until the reader knows it, and the knowing is what lets a later argument point at an object rather than re-describe it.

The specimens

Specimen	What it demonstrates
Pump PB-100-7	a failure claim with a span, a time and a repair
Log log-2024-03	a document, a span and a version
Work order 881	a decision with an author and a date
Notice N-14	a claim that accuses, and a defense that answers
Valve V-12	a predication with an agent, a theme and a dimension
Order 44021	a row that is already a predication
The Carter/Cole testimony	a chain of holders rather than a flattened fact
The letter	a belief walked from span to decision
The screwdriver question	two claims compared when no passage answers
Envelope e-4471	a message that carries its terms
Segment s-12	a dictionary that travels with the claims

The pump is the book's central specimen. It appears as a failure, as a repair, as a notice, as a comparison and as a wire message, and the reappearance is what lets a reader see one claim survive five projections.

The pump

Pump PB-100-7 fails on 4 March, floods the floor and is repaired on 7 March. The failure is a claim with a span in the log, the repair is a decision with a work order, and the flood is a second claim caused by the first.

The pump demonstrates the whole promotion chain. A sensor reading becomes a row, the row becomes a sentence in a log, the log becomes a span, and the span becomes a claim with a stance, an author and a version.

The pump also demonstrates the limits. The record holds the failure and the repair, and it does not hold the cause. The cause needs a mechanism claim, and the mechanism is an argument the record can support rather than supply.

The valve and the order

Valve V-12 demonstrates a predication with an explicit agent, an explicit theme and a voice dimension. The same valve appears as *the technician closed the valve* and as *the valve was closed*, and the two claims differ at the role list and at the dimension rather than at the verb.

Order 44021 demonstrates a row that is already a predication. The table name supplies the verb, the columns supply the roles, and the row's date supplies the time. A reader who sees the row as data has missed the claim the schema was making.

The pair is the book's smallest demonstration of the unification thesis. A sentence and a row are two surfaces of one claim, and the specimen is what makes the claim visible in a table nobody designed as a grammar.

The notice

Notice N-14 accuses the operator of two failures and one omission, and the defense answers with a finding. The notice demonstrates that an accusation is a claim chain rather than a verdict, and that a defense is a second chain rather than a denial.

The notice's most useful feature is its implied claim. Negligence is not stated, and the defense must answer it anyway, which is why the plate records the implied stance explicitly.

The notice also demonstrates the wire. The regulator's envelope carries terms, the claims travel by reference, and the operator's response carries a private claim by value with a segment.

The testimony

The Carter/Cole testimony demonstrates a chain of holders. Carter reports what Cole said, Cole's statement is a claim with its own stance, and the record keeps both rather than flattening the report into a fact.

The testimony is the specimen for attribution. A reader who wants to know what Cole said reads Cole's claim, and a reader who wants to know what Carter reported reads Carter's. A record with one claim has lost the difference.

The testimony also demonstrates the quotation rule. The embedded claim is quoted rather than asserted, and the quote is what keeps the reporting distinct from the reported.

The letter

The letter demonstrates the plate. A belief is walked from its span to its decision, and each panel of the plate is an object the reader can name: the document, the claim, the evidence, the finding, the decision and the action.

The letter's contribution is the receipt. The decision cites the finding, the finding cites the evidence, and the evidence cites the span, and the chain is what lets a reader defend the decision years later.

The screwdriver question

The question asks whether the pump failed before the seal was replaced, and no single passage answers it. The log holds the failure, the order holds the replacement, and the comparison is the answer.

The screwdriver demonstrates that retrieval is not search. The question's words appear in neither document, and the answer is in the record, and the gap between those two facts is the argument.

The envelope and the segment

Envelope e-4471 demonstrates a message that carries its terms, and segment s-12 demonstrates the dictionary that travels with the claims. The two are the book's wire specimens, and the pair is what an exchange looks like when the receiver can check it.

The envelope's state is the receiver's verdict. A closed envelope is one every claim verified, and a quarantined envelope is one that could not be, and the difference is a state rather than a message.

Using the specimens

A reader who has followed the book will recognize the specimens in a new argument before the argument is made. The recognition is the point: a specimen is a shared object, and a shared object is what lets a long book stay short.

A team can adopt the specimens as test material. The pump, the notice and the envelope are the fixtures in Appendix F, and a team that runs the fixtures is testing its compiler against the book's own objects.

The specimens are also a warning. They are small enough to be fully specified, and a real corpus is not. A team that expects its corpus to behave like the pump has confused a specimen with a domain.

Appendix K · Predictions

A prediction is a claim about what will happen, recorded with the date it can be checked and the observation that would refute it. Five predictions are made here, and each is stated so that a reader can mark it right or wrong rather than agree or disagree.

The predictions are the book's debts. A book that claims an architecture works owes a reader a way to test the claim, and a prediction that cannot fail is a compliment rather than a forecast.

The five

P1. Spans shorten disputes

Teams that record a span on every claim will spend measurably less time resolving factual disputes within eighteen months than teams that do not, because a dispute with two spans becomes a dispute about sources rather than about recollection.

The observation is the fraction of disputes closed in one meeting. The failure condition is no change in that fraction after eighteen months, or a change explained entirely by team composition.

P2. Skipped stance forces a rebuild

Projects that store claims without a stance will rebuild their claim store within twenty-four months, because a system that cannot tell testimony from fact will be asked a question it cannot answer, and the answer will be a migration.

The observation is the architecture decision record and the migration history. The failure condition is a project that skips stance, runs for twenty-four months and never needs the distinction.

P3. Priced gaps get funded

Ledgers that record a gap table will fund their source cleanups more often than ledgers that do not, because a gap with a number is a line item and a gap without one is a complaint.

The observation is the fraction of identified gaps that receive a funded cleanup within a year. The failure condition is no difference between ledgers with and without gap tables.

P4. The wire reduces clarification

Organizations that exchange envelopes with terms and dictionary segments will see fewer clarification cycles per exchange than organizations that exchange payloads, because the receiver's questions are answered before they are asked.

The observation is the number of follow-up messages per exchange. The failure condition is no reduction after a year of exchange, or a reduction explained by the partners knowing each other better.

P5. Model swaps degrade auditability

Teams that swap a model without re-running the gate battery will see their answer audit pass rate fall within three months, because the model's proposals change while the gates' fixtures do not.

The observation is the pass rate of the schema and intake groups in Appendix F. The failure condition is a stable pass rate across a model swap with no gate rerun.

How to check them

Each prediction has a date, an observation and a failure condition, and the three are what make it a forecast. A team that wants to test the book can pick the prediction its own work touches and record the observation for a year.

The predictions are also a contract with the reader. A reader who finds a prediction wrong has found something the next edition owes, and the debt is the same kind the book declared in the parts that named their own limits.

What the predictions are not

The predictions are not measurements. The book has not run a controlled study, and the standing discipline is that claims about effects are designed rather than measured. A prediction is a hypothesis with a date, not a result.

The predictions are not exhaustive. Five are the ones the book's claims most directly imply, and a team may find that its own domain tests a sixth. The sixth is a contribution, and the book would rather have it than defend a closed list.

The predictions are not safe. A prediction that a project will rebuild is a prediction that somebody's project will fail, and the failure is the point: a forecast that flatters every reader is a forecast nobody checks.

The date

The predictions are dated to the first edition. A reader in a later year should read them as a record of what the book expected when it was written, and the record is more useful than a revised prediction that has already seen the answer.

The dates are not a promise that the book will be revised. They are the condition under which the prediction can be checked, and the checking is the reader's work.

Calhoun: one group, six units, one claim

The companies in this appendix are fictitious. Calhoun Industrial Group and its six business units were invented to give every claim an instance a reader can follow from Part I to the last page. No name, figure or date here is drawn from a real firm.

1. The company sheet

What follows is canonical. Every example quotes it and none of them invents a name, a number or a date.

Calhoun Industrial Group is a heavy-civil contractor. It owns six business units and a group office, employs about 5,800 people, and reports revenue of 2.4 billion dollars a year. The group office sets standards and produces the monthly report the board reads. The units do the work and keep their own systems.

Business unit	Work	Departments	Systems
Boreworks	tunnels and shafts	Estimating, Operations, Safety	Estimating, Ops log, Safety register
RailWorks	rail construction and maintenance	Maintenance, Operations, Procurement	Asset register, Possession log, Procurement
Highway Works	roads, paving, drainage	Estimating, Field, Finance	Estimating, Daily reports, ERP
Inter-change Works	intersections and signals	Design, Construction, Traffic	Design files, Signal inventory, Traffic counts
Span Works	bridges for road and rail	Structures, Rail, Inspection	Structures register, Inspection records

Continued on next page

(Continued)

Business unit	Work	Departments	Systems
Haul Works	materials logistics	Dispatch, Fleet, Billing	Dispatch, Fleet, Billing
Group office	reporting and standards	Finance, Standards	ERP, Data warehouse

Eight systems hold data about the same jobs. None of them holds a record of who asserted what.

One party, five names

Unit and department	Column	Sense
HighwayWorks Finance	customer	the paying client
RailWorks Procurement	cust_nm	the invoiced entity
InterchangeWorks Design	client_name	the paying client
Boreworks Estimating	business_name	the parent group
HaulWorks Billing	account_name	the invoiced entity

Five columns, one concept, three senses. The same divergence appears for projects (`project`, `job`, `contract`, `program`, `works_order`), for cost codes, materials, equipment, suppliers and employees.

The recurring claim

On 4 March, Boreworks Operations reports that the seal on dewatering pump **PB-100-7** may have eroded. The report is an inspection note, `IR-1188`, written by a shift engineer and filed against the drive. The claim is *reported*, not *confirmed*: no inspection has been ordered, and the note says so.

That claim carries everything this book cares about. A holder (the shift engineer), a basis (a visual observation during the shift walk), a span (the note, paragraph 3), a time (4 March, 06:40), a stance (reported), and a defeat condition (a seal inspection showing no erosion).

One conflict

HighwayWorks Finance defines an *active customer* as a party with an open contract. HaulWorks Billing defines an *active customer* as a party with an outstanding balance. On 11 April the two definitions disagree about the same party, and the group report cannot be produced until one of them is chosen.

One intercompany load

On 18 March, HaulWorks hauls 22.4 tonnes of spoil from the Boreworks drive to a HighwayWorks embankment fill, under load ticket `LT-4471`. The load is revenue for HaulWorks and cost for Boreworks, and it is one physical event. The group report counts the material once.

One roll-up

The monthly report asks how many cubic metres of spoil were moved and where they were placed. The answer needs Boreworks' excavation records, HaulWorks' load tickets and HighwayWorks' fill records, and it must cite the scope each figure came from.

2. The group ontology

Twenty concepts, each with an owner and a lifecycle. The group office owns the definitions; a unit may extend a concept but may not silently alter it.

Concept	Owner	What it covers
Party	Group Standards	clients, suppliers, subcontractors, authorities
Project	Group Standards	a contracted body of work
Contract	Group Standards	the commercial instrument
Work	Group Standards	a physical act of construction
Asset	Group Standards	a thing built or maintained
Component	Group Standards	a part of an asset
Location	Group Standards	chainage, milepost, span, coordinate
Material	Group Standards	aggregate, spoil, ballast, concrete
Movement	Group Standards	a load from origin to destination
Cost code	Group Finance	the chart of accounts
Pay item	HighwayWorks Finance	a billed quantity

Continued on next page

(Continued)

Concept	Owner	What it covers
Inspection	SpanWorks Inspection	an examination with a result
Defect	RailWorks Maintenance	a condition requiring work
Possession	RailWorks Operations	a booked window on the network
Permit	Group Standards	authorisation to work
Incident	Group Standards	an unplanned event
Hazard	Boreworks Safety	a condition with potential for harm
Crew	Group Standards	people assigned to work
Equipment	Group Standards	plant, vehicles, tools
Document	Group Standards	a record with a span

Ten Questions, Three Methods

The argument of this book is that a claim record answers questions a graph or a retrieval stack cannot. This appendix tests that argument against ten questions drawn from Calhoun Industrial Group, answered three ways: by the claim ledger, by a property graph, and by a retrieval stack over the same source documents.

Each question states the answer each method gives, and closes with what that method could not answer. The last section collects the four questions none of the three methods answers alone.

1. Who asserted that the seal on PB-100-7 may have eroded, and were they there?

Ledger. A shift engineer on Boreworks Operations, in note `IR-1188` paragraph 3, during the 4 March walk, stance *reported*, basis *visual observation*. The record also holds the defeat condition: a seal inspection showing no erosion.

Property graph. The edge from the pump to the erosion event exists and can be traversed. The edge carries no holder, no stance and no span unless the schema was extended to carry them, and an extended schema still cannot say whether the engineer was present, because presence is a claim about the assertion rather than about the pump.

Retrieval. The note is retrieved and quoted. The quote is accurate and unattributed beyond the document, and a later paraphrase of the same note retrieves equally well, so the method cannot separate the original from a summary.

What each could not answer. The graph cannot answer *on whose authority*. Retrieval cannot answer *is this the original*. Only the ledger answers both, and it answers by holding the claim rather than a copy of it.

2. How many defects were confirmed in March, and how many were reported?

Ledger. Two counts from one store: fourteen confirmed, two reported, one inferred, by stance. The distinction survives every projection that declares it.

Property graph. One count, or two if a stance property was modelled and populated. A traversal counts nodes, and stance is not a property of a node unless someone made it one.

Retrieval. A count of passages that mention defects, which is a count of text rather than of defects, and double-counts a defect discussed in three documents.

What each could not answer. The graph cannot separate what a source asserted from what it merely recorded. Retrieval cannot count at all without a record to count.

3. Why did the March figure rise against February?

Ledger. The layer refuses to narrate. It reports the difference, the claims that entered and left the count, and the two term versions in force, and it says the explanation is a judgement the reader must make.

Property graph. It shows new nodes and edges and cannot say whether the rise is more defects or a changed intake.

Retrieval. It produces a fluent paragraph, which may be right and cannot be checked against a record, because the paragraph is the record's replacement rather than its projection.

What each could not answer. Neither the graph nor retrieval can distinguish a rise in defects from a rise in reporting, and that distinction is the only one the group office needs.

4. Which party is this, and which of the five names is in force?

Ledger. One canonical identifier, with the sense recorded and the alias kept: `customer` in HighwayWorks Finance is the paying client, `business_name` in Boreworks Estimating is the parent group, and both point at the same address with different senses.

Property graph. Five nodes, or one node with five labels, depending on who built it. Neither form says which sense a unit meant when it wrote the row, because the sense belonged to the writer, not the node.

Retrieval. The documents mention the party under several names and retrieval returns passages for each, without ever establishing that they concern one party.

What each could not answer. The graph cannot hold the writer's intent, and retrieval cannot decide identity. The ledger answers by storing the sense as part of the binding rather than as a guess.

5. May this claim appear in the receivables report?

Ledger. The claim carries a license; the report declares a scope and a use; the pair is checked and the answer is yes, no, or unresolved with the reason named.

Property graph. Access is a property of the store rather than of the claim, so the answer is the same for every claim in the graph and depends on how the deployment was configured.

Retrieval. Whatever the index returns appears, which means the answer is decided by relevance ranking rather than by authority.

What each could not answer. Neither can say *on what authority this specific claim entered this specific report*, which is the question an auditor asks.

6. What did the projection drop?

Ledger. The projection declares its loss in writing: holder, stance, span and modality, and the four questions that follow from their absence.

Property graph. The graph holds what was loaded. What was excluded is invisible, so a completeness claim rests on trust rather than on a record.

Retrieval. Chunks were selected by similarity; the selection is the drop, and it is not recorded.

What each could not answer. The graph cannot show what it never received, and retrieval cannot show what it never ranked.

7. This claim's premise was just superseded. What is its status now?

Ledger. The superseding claim carries the earlier one as its predecessor, so the dependent claims are listed and shown as resting on a replaced premise until each is reaffirmed.

Property graph. The old edge is deleted or left in place; either way the deletion is unrecorded, so dependents look justified when they are not.

Retrieval. The newest passage wins by recency ranking, and nothing connects it to the claims that relied on the older wording.

What each could not answer. Neither can enumerate the dependents, because dependency is a relation among claims rather than among nodes or passages.

8. What is the group figure when two units keep different calendars?

Ledger. The reconciliation rule is recorded with the figure: month of acceptance, derived dates with their method, and a declared tolerance.

Property graph. The figure is whatever the loader computed, and the calendar is not visible in the result.

Retrieval. There is no figure, only passages describing two calendars.

What each could not answer. The graph cannot show the method behind the number, and retrieval cannot compute the number at all.

9. The span points at a photograph. Is that evidence?

Ledger. The claim records an inferred stance and a span into an image, so the reader sees a conclusion drawn from a picture rather than from a sentence, and sees who drew it.

Property graph. The edge exists with no record of what supported it.

Retrieval. Images are usually outside the index, so the claim appears in text with its support invisible.

What each could not answer. The graph cannot say what kind of thing supported the claim, and retrieval cannot reach the support at all.

10. Two years later, someone who was not there asks why this was decided.

Ledger. The decision is a claim with its premises, the versions in force, and the defeat condition, so the answer is a record rather than a recollection.

Property graph. The state at the time is gone; only the current graph remains, and it has been rebuilt several times.

Retrieval. It reproduces what was written about the decision, which is a document rather than the reasoning.

What each could not answer. Neither preserves the state of the record at the moment of decision, which is the only thing that answers the question.

The four questions no single method answers

Four of the ten need two methods at once: who asserted a claim and whether the assertion was present (ledger plus retrieval), how many defects were confirmed (ledger plus the graph's traversal for the dependent cases), which party a name refers to across five systems (ledger for the sense, graph for the links), and why a figure moved (ledger for the claims, graph for the shape of the change).

The pattern is consistent. The ledger answers attribution, stance, dependency and loss. The graph answers reach and shape. Retrieval answers what was written and where it can be found. A system that keeps one of the three and calls it the platform is choosing which questions it will refuse to answer, and most organizations have made that choice without recording it.

Index

audit trail, 130, 253

candidate, 45, 78, 92, 93, 113–115, 117, 118, 120–122, 148, 164, 167, 171, 174, 178, 180–186, 190, 209, 215, 217, 219, 222, 223, 241, 243–246, 248, 251, 254, 260, 287, 449, 450, 452, 454–458, 460, 479, 486–488, 490, 526–528, 541, 543, 545, 546, 553, 554
capability, 116, 192, 301–308, 318, 320–323, 451, 453, 454, 456–458, 460
capability state, 451, 452, 454, 455, 457
commitment, 49–52, 84, 88, 94, 123, 143, 170, 172, 173, 175, 176, 199, 211, 215, 216, 218, 221, 223, 229, 232–238, 240, 246, 248, 259, 260, 262, 265, 268, 271, 273, 275, 277, 279–283, 294–297, 299, 325, 327–331, 370–377, 504, 506, 528, 534, 537, 539, 542, 545
commitment grid, 294, 298, 380
conformance, 223, 226, 323, 443, 559, 561, 562

decompression, 194, 201, 202
dictionary, vii, viii, xv, 3, 21, 87, 120, 147–149, 151–156, 159, 161–171, 173–176, 187, 190, 191, 194, 196, 201, 205, 246, 256, 260, 319, 370, 492, 495, 496, 527, 542, 554, 569, 575, 578
dimension, 10, 32, 38, 39, 42, 44–46, 48, 49, 52, 58, 92, 96, 97, 100, 127, 131, 443, 445, 447, 531, 533, 534, 536, 542, 545, 574
drift, ix, 5, 47, 58, 89, 110, 167, 174, 188, 226, 234, 258–263, 265–267, 269, 278, 280–283, 351, 353, 354, 394, 436–439, 483

entity resolution, 527
epistemology, 401–407

ghost, 92, 122, 179–181, 183, 184, 186, 191, 527, 528, 542
governance, vii–xi, xv, 5, 26, 46, 47, 148, 154, 163, 168, 175, 186, 201, 225, 226, 233, 236–238, 247, 254, 255, 258, 263, 266, 282–284, 288, 290, 292, 303, 311, 333, 334, 390, 392, 393, 397, 400, 403, 404, 455, 458, 459, 542, 546, 559, 564, 565, 567, 568
grounding, vii, x, xi, 139, 163, 169, 171, 174, 175, 177, 202, 218, 304, 370–377, 527

holder, vii, ix, x, 4–6, 8, 11–17, 19, 21, 23–26, 30, 31, 37, 38, 49, 50, 52, 55, 56, 58, 59, 61, 62, 67, 69, 71, 84, 86, 88, 95–98, 101, 103, 105, 163, 241, 243, 244,

295–297, 443, 446, 447, 520, 521, 527, 534, 536, 541–543, 545–548, 564, 581, 584, 586

identifier, 3, 4, 8, 10, 11, 13, 20, 25, 92, 119, 123, 149, 150, 153–155, 158, 159, 162–164, 167–169, 174, 178, 179, 181, 190, 248, 342, 346, 410–413, 415, 418–422, 424–431, 433, 434, 446, 450, 493–497, 514, 521, 527, 541, 542, 546, 569–571, 585
intersection, 336, 386–393, 565, 566

join, 2, 3, 5, 8, 12, 40, 45, 59, 69, 71–76, 84, 85, 88, 147–149, 152–154, 158–160, 168, 222, 223, 226, 242, 346, 363–369, 435, 436, 438, 439, 447, 472, 473, 487, 490, 501, 534, 545
jurisprudence, 399, 400

lexicon, ix, 42, 92, 93, 104, 169, 174, 187, 191, 199–202, 205, 216, 219, 231, 444, 446
loss report, 94, 201, 223–225, 237, 318–324, 352, 353, 355, 359, 361, 367, 368, 411, 414, 435, 436, 451, 452, 454–458, 460

microgloss, 47, 87, 148–155, 170, 260, 546, 550, 554

nine pains, 221, 225, 226, 228
non-inference, 379–381

plate, 463–476, 574, 575
predication, viii, xi, xii, 9, 18–21, 30–37, 49, 53, 54, 58, 85–87, 95, 96, 104, 182, 184, 187, 344–348, 425, 433, 541, 542, 545, 547, 550, 554, 564, 574
promotion, 18, 125, 168, 173, 174, 237, 242, 253, 254, 302, 310, 380–383, 414, 449, 450, 456, 457, 529, 574
property graph, 417, 433, 516, 551, 584–587

RDF, 16, 103, 425, 433, 516
receipt, 115, 150, 152, 157, 158, 161, 172, 175, 177–180, 183–186, 193, 251–253, 255–257, 260, 266, 274, 275, 283, 288, 291, 303, 304, 307, 312, 314, 315, 318, 320–324, 335, 337–340, 355, 361, 362, 367, 411–415, 422, 424, 435, 450, 451, 456, 457, 462, 466–468, 475, 494, 509, 514, 527, 529, 541, 545, 546, 575
refusal, v, xii, 44, 46, 50, 75, 90, 92, 93, 100, 113, 115–118, 121, 133, 153, 184, 235, 244, 256, 270, 303, 304, 306, 307, 319, 327, 333, 343–345, 347, 366, 372–374, 376, 379–385, 388, 389, 395, 403, 406, 439, 443–445, 447, 451, 457, 478, 481, 482, 487, 497, 498, 502, 511, 513–518, 522, 527, 528, 543, 545, 549, 551–553, 555, 567
round trip, 94, 124–130, 191, 193, 411, 414, 436, 498, 529, 554

semantic debt, 277, 279
semantic programming language, viii, 198, 202, 204, 542, 543, 558, 562, 564
stance, ix, x, 33, 39, 52, 58, 59, 101, 103–106, 119, 121–125, 127, 132, 190, 296, 297, 319, 356–361, 410, 413, 414,

416–418, 420, 422, 423, 426, 427, 429, 431–435, 437–440, 443, 446, 449–455, 460, 464, 467–469, 471, 474, 478, 479, 481, 483, 495–497, 500, 501, 504, 505, 507, 508, 520, 521, 524, 536, 541–543, 545, 548, 549, 554, 555, 569, 574, 577, 581, 584, 586, 587
standing, 12, 13, 15–17, 19, 24, 88, 109–111, 132, 144, 168, 228, 240, 241, 243, 244, 247, 318–323, 325, 327, 332, 349, 354, 355, 409, 482, 492, 528, 552, 578
supersession, 27, 85–89, 108–111, 396, 397, 446, 482, 490, 501, 528, 543, 546–551, 554, 557, 566
synapse, v, vi, 34, 36, 37, 39–41, 66, 67, 84, 91, 191, 193, 233, 236, 415, 416, 520, 526, 527, 543
thin view, 415, 419, 422, 435–440, 457, 460
triple, 13–17, 57, 84, 85, 103, 425–429, 431
universal meaning substrate, 84, 85, 87, 88, 543
warrant, x, xv, 10, 11, 15, 16, 19, 21, 27, 45, 50, 71, 73, 75, 76, 79, 80, 93, 114, 151, 250, 251, 255, 271, 272, 275, 298, 303, 304, 307, 313, 325–332, 338–340, 371–374, 395, 401–405, 436, 437, 451, 457, 464, 465, 467, 471, 472, 474, 484, 488, 496, 509, 519–521, 534, 541, 543, 548

Compilation engine: LuaLaTeX, from Pandoc through the Symbol Grounding press pipeline
Typography: XCharter (main text), Noto Sans (sans-serif), Consolas (monospaced)
Layout: 7 × 10 inch trim, SYNAPSE print profile, running heads by part and chapter
Microtypography: Set with the `microtype` package, using character protrusion and font expansion tuned for even color and minimized rivers in justified text
Figures: TikZ, drawn as standalone documents and placed at the width the argument needs